

Catapult HLS User View Scheduling Rules

Stuart Swan, Platform Architect, Siemens EDA

15 August 2026

Abstract

This document defines a user-visible scheduling contract and modeling methodology for high-level synthesis (HLS). Its practical objective is to support a proof-backed flow in which much of the system-level functional verification and debug is performed on the pre-HLS SystemC model, for designs and implementations that satisfy the stated applicability and conformance obligations. The contract treats message-passing operations, signal I/O, and explicit synchronization through a small set of uniform rules while allowing important implementation transformations such as loop pipelining, added FSM states, permitted memory-access reordering, direct-input late sampling, and bounded or elaborated channel buffering.

The formal safety result is an ordered observable trace-compatibility result. Its principal deductive direction is Post $\rightarrow$ Ref: covered post-HLS behaviors are matched by behaviors of the prescribed source reference model. Design-specific process, channel, atomic-interaction, environment, and reset evidence is composed into that system-level result by the generic proof architecture. Use in the opposite direction is deliberately restricted to annotated RTL-consistent source executions or executions selected by the Appendix L witness-selection mechanism. Deadlock preservation is a separate result for internal message-passing deadlock. Its primary source-side route reduces the scheduler-robust liveness premise to one selected, complete deterministic serial-issue pre-HLS execution with total finite-bound response coverage. Under the route's stated applicability and flow-qualification conditions, a clean execution discharges the source-side premise; extending the result from certified RTL cutpoints to every relevant reachable cutpoint additionally requires flow-supplied certificate coverage and evidence that a persistent relevant blocking condition cannot remain indefinitely in transient internal activity without reaching a state where the deadlock analysis applies. The result remains conditional on the stated capacity, fairness, progress, drain, environment, and structural obligations.

The proofs do not by themselves certify a particular HLS invocation or RTL instance. The applicable source well-formedness, scheduling, channel-realization, capacity-faithfulness, reset, environment, and progress obligations must be established by the tool or verification flow through documentation, generated certificates, static checks, targeted RTL/interface checks, or equivalent evidence. Subject to those conditions, the document provides a Catapult-compatible, tool-neutral basis for industrial HLS verification and for possible scheduling-contract standardization.

Introduction

HLS loses much of its practical advantage when design teams must return to machine-generated RTL as the primary level for functional verification and debug. RTL verification can be one of the most costly and time-consuming parts of a hardware development flow, and generated RTL may differ substantially in structure and timing from the source model. A useful HLS methodology should therefore let designers and verification engineers reason primarily at the source level while retaining a precise way to qualify the resulting implementation.

This document presents a user-visible scheduling contract intended to provide "no surprises" at process and interface boundaries. It reduces interface scheduling to uniform rules for message-passing operations, signal I/O, and explicit synchronization. A common verification intent can then be reused across the prescribed pre-HLS source reference model and the post-HLS implementation, without requiring the implementation to preserve every source-level cycle or internal state.

Subject to the conditions stated in the main text and formal appendices, the methodology accommodates sequential and combinational logic, pipelined designs, shared and external memories through the stated mapping conditions, permitted RAM-access reordering, direct-input late sampling, finite channel buffering, and localized latency-sensitive protocols. These capabilities reflect requirements encountered across a broad range of industrial HLS designs and are important for achieving competitive hardware quality of results (QOR).

The methodology builds on established verification practices such as SystemVerilog UVM, constrained-random stimulus, and functional and code coverage. The system under test is modeled and verified in SystemC, while implementation-level verification focuses on establishing that the synthesized or otherwise implemented RTL falls within the scope of the applicable theorem instance. When the selected abstract channel boundaries and back-annotated capacities faithfully represent the RTL storage or credit at those boundaries, the pre-HLS model can also be used to assess capacity and throughput effects rather than deferring those questions entirely to RTL simulation.

The formal architecture separates safety from liveness. Safety is largely compositional: design-specific evidence is supplied for individual processes, explicit channel boundaries, named atomic interactions, the environment, and reset behavior, and a generic theorem composes that evidence into a system-level observable trace-compatibility result. Liveness and deadlock preservation have a more global dependency structure and therefore require additional cutpoint-coverage, progress, fairness, drain, environment, and source-side premises.

The most important default deadlock-avoidance rule can be stated plainly. HLS may overlap or shift message-passing operations in time, but it may not reverse the dynamic source order of message-passing calls under the default scheduling contract. It therefore cannot introduce a new deadlock merely by reversing the order of source calls. This rule is not a complete deadlock theorem: overlapped loop iterations, eager issue, finite capacities, and environment behavior can still affect deadlock, and those effects are addressed by the separate liveness machinery, whose primary practical route checks one selected, complete serial-issue execution of the pre-HLS model with total bounded-response coverage (Appendix K.4b and K.4c).

The guarantees remain conditional. The mathematical proofs do not establish that a particular HLS tool invocation or generated RTL instance satisfies the source-side and implementation-side obligations. Those obligations must be discharged by the tool or flow through documentation, generated certificates, lint or static analysis, targeted RTL/interface checks, formal verification, or another

appropriate validation method. The same framework can also be applied to hand-written, modified, or otherwise generated RTL when equivalent evidence is supplied.

Because the rules are expressed as a user-visible, tool-neutral scheduling contract rather than as the internal behavior of a particular HLS implementation, they can also serve as a starting point for standardization, for example within the Accellera Synthesis Working Group. The next section summarizes the guarantees without exposing the internal proof API. A separate Guide to the Formal Appendices appears immediately before Appendix G, Appendix F provides user-oriented modeling guidelines, and Appendix M provides a more detailed Formal Results Overview.

Formal Guarantees at a Glance

The statements below provide a user-level orientation. The formal definitions, assumptions, scope conditions, and evidence requirements appear in the appendices.

Implementation safety. Within the stated scheduling, channel, environment, reset, and qualification scope, the generated RTL does not introduce new observable functional behavior that is absent from the prescribed pre-HLS reference model. This allows most functional verification and debug to remain at the source level when those conditions are established.

Use in the reverse direction. A particular pre-HLS simulation is not automatically guaranteed to be reproduced cycle for cycle, or even as the directly corresponding execution, by the RTL. Direct comparison of a selected source execution with RTL requires the alignment and witness-selection conditions described in Appendices G and L.

Internal message-passing deadlock. The primary practical method uses one complete deterministic execution of the pre-HLS model in serial-issue Policy-S mode, together with finite response checks on blocking operations. For a terminating design, a clean complete execution can provide the required source-side deadlock evidence without exploring alternative scheduler interleavings, provided the tool or verification flow separately establishes the method's applicability and coverage conditions. The method does not cover barrier or reset stalls, environment-only stalls, starvation without the modeled internal wait structure, timed or reactive multi-frontier cases, or designs whose correctness depends on favorable same-cycle issue. Dynamic reset during the comparison is supported only when either no reset affects the blocked component during the required comparison or a corresponding non-waived serial response monitor has already triggered and is then cancelled by the reset; otherwise another deadlock-proof route is required for that cutpoint.

RTL coverage and qualification. The clean pre-HLS execution is only one part of the overall verification argument. A claim covering every relevant reachable RTL state also requires complete implementation-side correspondence and coverage evidence, including evidence that a persistent relevant blocking condition cannot remain indefinitely in transient internal activity without reaching a state where the deadlock analysis applies. These conditions must be established for the selected source model, synthesis run, channel and capacity realization, environment/reset contract, and generated RTL instance; ordinary synthesis and one clean source simulation do not establish them. The serial-reduction proof and certificate machinery are not yet fully mechanized and qualified. Appendices G, I, and J give the safety and correspondence results; Appendix K gives the deadlock results; and Appendix M.7 summarizes the division of verification responsibilities and current framework status.

Document Goals

This document presents HLS tool scheduling rules and a modeling methodology. The goals are:

1. To provide easy-to-understand rules to HLS users.

2. To ensure precise and consistent rules in both SystemC and C++.
3. To offer rules that are effectively compatible with how Catapult currently operates.
4. To enable the best possible *quality of results* (QOR) in Catapult synthesis.
5. To cover all known user requirements and scenarios.
6. To serve as a suitable starting point for a standardization proposal (e.g., in Accellera SWG).

To illustrate the goals of this document more specifically, consider what an engineer writing a testbench for an HLS model needs to understand about how HLS tools operate. This engineer may be using SystemVerilog UVM or may be writing a testbench in C++/SystemC.

They likely are not an expert in any specific HLS tool (and may not want to be), but their testbench should be usable across both the pre-HLS model and the post-HLS model, subject to the latency-insensitivity, observation-boundary, and witness-selection conditions stated in this document. Thus, it is crucial for the DV engineer to have a precise understanding of how the HLS tool will transform the design while still enabling the resulting RTL behavior to be checked against the prescribed source reference model. This document describes what transformations the HLS tool is allowed to perform so that the pre-HLS and post-HLS models can be effectively verified with a common verification intent, and in the deterministic or annotated RTL-consistent cases with the same directly corresponding testbench execution. The overarching philosophy of the scheduling rules is to present “no surprises” to such a DV engineer, while still giving the HLS tool ample freedom to optimize the design.

The main body of this document is intended to be sufficiently precise and complete to support the Accellera standardization effort while still being understandable to HW architects. The formal appendices are intended to have a very high degree of precision and completeness sufficient to support AI-driven Lean mechanization of the formal theorems.

Background

HLS tools generate RTL from C++ models. Broadly speaking, this conversion takes a sequential C++ model and turns it into concurrent hardware that maintains the same behavior. HLS tools identify concurrent processes within the C++/SystemC model and then independently synthesize each process. Briefly, some of the techniques that HLS tools use to achieve good HW QOR when synthesizing each process include:

- Optimized scheduling based on the selected silicon target technology.
- Automatic HW pipeline construction according to the user’s specifications.
- Automatic HW resource sharing.
- Automatic scheduling of memory accesses.

The internal behavior of each process is specified by the control and dataflow behavior of the C++ code within the process. However, the external communication that each process has with other processes and HW blocks is specified via IO operations that are coded within the model. To enable a reliable, scalable, and verifiable HLS flow that generates high quality hardware, the scheduling behavior of these IO operations needs to be precisely handled at all steps of the flow. This document specifies the rules that govern the scheduling behavior for these IO operations within HLS models.

These rules are specified with respect to an individual process, but the intent of the rules is to enable reliable and verifiable behavior of large sets of interacting processes operating as a system in real-world

designs. (Appendix G provides the formal trace-compatibility guarantees between the prescribed source reference model and the post-HLS system, including the unrestricted Post → Ref direction and the restricted annotated RTL-consistent / witness-selected reverse use.)

By default, HLS tools can insert additional clock cycles (or latency) anywhere within a process -- for example, when pipelining a loop or to enable HW resource sharing. The overall approach used in this document is to make the entire design and testbench system *latency insensitive* to the maximum extent possible, while still fully enabling key HW optimizations. In addition, the overall approach enables the pre-HLS system simulation to be capacity-accurate and throughput-accurate with respect to the post-HLS RTL system.

In some cases, designs or testbenches may use protocols which are latency-sensitive. These situations can be handled by isolating the latency-sensitive portions to small, self-contained parts of the design or testbench, and then keeping the rest of the design and testbench latency insensitive. See Appendix D for more information.

In many cases the overall design will need to satisfy end-to-end latency requirements. For these designs it is still highly advantageous to use a latency-insensitive modeling approach and verify in the post-HLS model that the overall design latency requirements have been satisfied, since this is typically easy to do.

This document focuses on sequential HW processes. Combinational HW processes are supported but mostly not discussed since their synthesis is straightforward. All the examples and discussion in this document are for SystemC processes that are sensitive only to a single rising clock edge. (Appendix O discusses support for multiple clock domains.)

This document distinguishes between the following:

1. The *conceptual model* for the scheduling rules.
2. The simulation behavior of a model using the rules in C++ or SystemC.
3. The synthesis of a C++/SystemC model using the rules in an HLS tool such as Catapult.

The goal is to align each of these three cases as closely as possible, so that the user has easy-to-understand rules, while simulation and synthesis work without surprises. However, as we will see, there are practical considerations which may in certain cases cause small deviations from the conceptual model in either simulation or HLS.

A simple real-world example that illustrates the motivation for the scheduling rules is provided in Appendix A of this document.

The examples referred to in this document are available here:

<https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master>

The most up-to-date version of this document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

A summary of the scheduling rules is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/brief_scheduling_rules.txt

A formal architecture guide is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/formal_architecture_guide.pdf

An Analogy from RTL Synthesis

To better understand the specific purpose of this document, let's consider how RTL synthesis works. Say you have a sequential block that you are modeling in Verilog RTL, and it has an output port coded like:

```
Out1 <= #some_delay new_val;
```

In Verilog simulation, if `some_delay` is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if `some_delay` is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if `some_delay` is greater than the clock period. Some RTL synthesis tools might give a warning or error for code like the above such as "Simulation and synthesis results are likely to mismatch because the delay in model is greater than clock period." Some RTL synthesis tools might outright reject a model containing such delays.

One might argue that RTL synthesis tools should always match the Verilog simulation behavior of the input model. But the overall approach works well because RTL is a good and simple *conceptual model* that users and tool vendors can align around. The slight differences between the *simulation model* and the *conceptual model* used by RTL synthesis tools can be easily managed.

We'll return to this example later in this document.

Next, let's clarify some terminology related to signal IO. Say you have a Verilog model like:

```
always begin
  @(posedge clk); // wait for 1st rising clock edge
  output1 <= input1 + 10;
  @(posedge clk); // wait for 2nd rising clock edge
end
```

In this example, `input1` is sampled when the process wakes on the first `@(posedge clk)` and evaluates the RHS. For the purposes of the scheduling rules, we anchor that `Read(input1,.)` to the most recent synchronization point. The resulting `Write(output1,.)` is anchored to the next synchronization point (the cycle in which the written value is treated as stable for others to observe).

Cycle-level anchoring convention (used throughout this document). For cycle-level reasoning, each signal IO is assigned an anchor cycle relative to surrounding synchronization calls (e.g., `wait()` / `SyncChannel` / `@(posedge clk)`):

- Read(sig, val) is anchored to the closest preceding synchronization call, i.e. pred_S(Read).
- Write(sig, val) is anchored to the closest succeeding synchronization call, i.e. succ_S(Write).

This convention is only for cycle-level reasoning and does not depend on simulator update regions or delta-cycle ordering.

Catapult HLS Status Concerning Rules in this Document

A separate document provides a list of clarifications related to support within Catapult HLS for the rules described in this document. The items in the list are named *Cat#*, so that each item has a unique number. This document annotates certain rules with *Cat#* to refer to the items in the separate document. These *Cat#* annotations are not part of the formal proof itself.

Terms Used in this Document

Latency-Insensitive: In digital hardware design, *latency-insensitive* refers to a system that operates correctly despite variable communication delays between components. This is achieved by using mechanisms to decouple computation from communication timing. Such designs improve scalability and reliability in complex systems with unpredictable or variable latencies. ARM's AXI4 and APB are examples of latency-insensitive protocols. ARM's AMBA 5 CHI credit-based NOC protocol is also an example of a latency-insensitive protocol.

Process: In Verilog, a process is an *always block* and its equivalent constructs. In SystemC, a process is an instance of an SC_THREAD or SC_METHOD.

Message-passing Interface: A *message-passing interface* reliably delivers messages (or transactions) from one process to another. This document uses this term to denote the type of communication found in Kahn Process Networks. See https://en.wikipedia.org/wiki/Kahn_process_networks

Message-passing read interfaces are always separate from message-passing write interfaces – there are no bidirectional message-passing interfaces. In this document, message-passing channels are assumed to be point-to-point: for each channel *c*, exactly one producer performs all Push_*c* operations and exactly one consumer performs all Pop_*c* operations.

Synchronization Interface: A *synchronization interface* synchronizes one process with another and/or with a global clock. For an example of a synchronization interface, see [https://en.wikipedia.org/wiki/Barrier_\(computer_science\)](https://en.wikipedia.org/wiki/Barrier_(computer_science))

Signal IO: In digital HW design, signals are the fundamental communication mechanism. Signals enable communication between two HW blocks/processes, but communication with signals in real HW always incurs at least some delay because communication cannot be faster than the speed of light. In HDLs and in SystemC, signal delays are modeled with the *delayed update* semantics.

blocking / non-blocking: In this document, a blocking message-passing operation is modeled as issuing a persistent request that remains asserted (and with stable payload, if applicable) until the operation commits (the message is sent or received). A non-blocking message-passing operation returns immediately with a status indicating whether the operation committed in that cycle.

One-way / two-way handshake protocols: A one-way handshake protocol only has one signal between a sender and receiver to synchronize communication. A two-way handshake protocol has two signals (one in each direction) between a sender and receiver to synchronize communication.

shall: This term indicates that a compliant tool or flow is required to follow the indicated rule.

may: This term indicates that a compliant tool or flow is allowed to follow the indicated rule but is not required to do so.

Classes of Operations Involved in Scheduling Rules

There are three classes of operations involved in the scheduling rules:

1. Calls to message-passing interfaces (which are all `ac_channel` methods, all SystemC `Push/PushNB/Pop/PopNB` calls except calls to `SyncChannel`)
2. Calls to synchronization interfaces (which are calls to `ac_sync` and Matchlib `SyncChannel`, also `ac_wait` and SystemC `wait`)
3. Signal IO (which are SystemC signal reads and writes, also C++ model *direct inputs*)

All the operations above are referred to as *IO operations*.

Basic Conceptual Model

The basic conceptual model encompasses processes that have no loop pipelining but may have preserved loops. If a process has a preserved loop, then the user may place a wait statement in the loop. Wait statements explicitly placed in the model are called explicit wait statements in this document and are classified as calls to a synchronization interface. However, for a loop to be treated as an HLS-pipelined loop under the formal guarantees in this document, the loop body shall not contain any `wait/ac_wait` statement or other explicit synchronization call used as an in-loop wait boundary. Such synchronization boundaries must be placed outside the pipelined loop, or the loop must be treated as non-pipelined/preserved or outside the theorem instance.

The basic conceptual model rules are:

1. Synchronization interface calls within a process always remain in the source code order.
2. Signal read operations occur at the closest preceding call to a synchronization interface. (Cat1)
3. Signal write operations occur at the closest succeeding call to a synchronization interface.
4. Message-passing operations may be shifted in time, issued in the same cycle, or overlapped by simulation or synthesis, subject to the following constraints:
 - All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (Here “the synchronization interface commits” refers to the commit of that synchronization call in the calling process’s trace. For blocking message-passing operations (`Push/Pop`), the process cannot complete the synchronization call until any earlier issued blocking message requests in that interval have committed.)

- All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
- Message-passing operations on the same message-passing interface shall be issued in source order; they shall not be reversed. This same-interface order requirement applies both within a single source interval and across overlapped iterations of a pipelined loop.
- Two message-passing operations on separate interfaces which appear in sequence in the model are source-ordered. They may be issued in the same source sequence, and the later operation may in some permitted modes issue before the earlier operation commits, including in the same clock cycle where allowed by the issue policy. They shall not be issued in the reverse sequence. (Cat2)

Here “parallel” issue means that requests may overlap or be asserted in the same cycle while the calls remain ordered by their dynamic source order. Preserving that order prevents HLS from introducing a deadlock solely by reversing source-ordered message calls. It does not by itself guarantee deadlock preservation, because loop overlap, request timing, and finite channel capacity can still affect deadlock behavior. The formal appendices address those additional cases.

Pipelined Loops

When a loop is pipelined in HLS, the body of the loop is split into pipeline stages. HLS may start the next iteration of the loop before the current iteration has completed, thus increasing throughput as compared to a non-pipelined implementation.

A loop selected for HLS pipelining shall not contain `wait/ac_wait` statements, `SyncChannel/ac_sync` calls, or any other explicit synchronization interface call inside the loop body. The pipelined-loop rules in this document therefore apply to wait-free pipelined loop bodies, with ordinary synchronization boundaries only outside the pipelined loop or around the pipelined region. If a design requires a synchronization call inside the loop body, the flow shall either reject that loop for pipelining under this contract, require the source to be rewritten so that the synchronization boundary is outside the pipelined loop, or exclude that pipelined-loop instance from the Appendices G-K formal guarantees unless a separate specialized theorem/certificate is supplied.

To guarantee equivalent behavior between the pre-HLS and post-HLS systems, pipelined loops must be implemented with automatic flush to prevent deadlocks. See Appendix B for a formal presentation of pipelined loops.

When HLS pipelines a loop, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, for all IO operations, HLS shall ensure that an access to a specific message-passing interface, signal, or synchronization interface shall not be reordered relative to same-interface accesses from other iterations.

During pipelining, signal reads and writes are physically embedded in specific pipeline stages, but the loop body itself contains no explicit `wait/ac_wait` or other in-body synchronization call under the formal contract above. The logical anchor of an ordinary signal Read/Write is therefore still determined by the surrounding real synchronization calls selected into S under R2/E2, not by an in-body wait. Considering the entire set of signals read or written by the process, if HLS pipelining would cause the order of signal IO to differ between the pre-HLS and post-HLS models, or if any two signal IO operations that occur on the same clock edge in the pre-HLS model would not occur on the same clock edge in the post-HLS model, then the HLS tool shall detect such a situation and require the user to explicitly indicate in the model source (e.g., via a pragma) that HLS pipelining can still be used. (Cat7) In other words, if a process

communicates with other processes using only latency-insensitive protocols, then HLS pipelining by default shall not break any of the protocols.

Synchronization boundary for pipelined loops

When a pipelined region ends at an explicit synchronization call, work already accepted before the boundary may continue to drain. However, the implementation and the back-annotated source model must not expose capacity for work belonging to the next synchronization interval until the synchronization call completes. This prevents upstream transfers from entering the next interval prematurely. See Appendix B for the formal rule and modeling treatment.

Direct Inputs

Ordinary SystemC signal reads occur at the closest preceding synchronization interface call (e.g., wait statement) in both the pre-HLS and post-HLS models. (Cat1). If the HLS tool adds states to the process, or if it pipelines the process, then this implies that the HLS tool must add registers for each such read operation such that the read occurs where specified in the pre-HLS model, and the value is stored until the point where it is consumed in the post-HLS model. The area cost of such registers may be high if there are a lot of signals, and in some cases, it may be unneeded area since the value of the signals may not actually need to be stored internally to the process.

The simplest case is if such external signals are held stable after the process comes out of reset. In this case, HLS may assume that it is free to physically sample the signal values as late as possible, with no need for register storage, while the logical Σ -visible Read event remains anchored as specified by E2. This case is handled with the following pragma on SystemC signals and ports (Cat6):

```
#pragma hls_direct_input
```

To ensure that there are no pre-HLS versus post-HLS simulation mismatches, the environment that drives the signal shall hold it stable within each relevant reset epoch: after all receiving processes that use this signal with this pragma have come out of a reset boundary, the signal value shall remain stable until the next reset boundary that affects those receiving processes, unless the signal is instead governed by an explicit `hls_direct_input_sync` update contract as described below. A *dynamic reset* starts a new stability epoch for the affected receiving logic and its associated direct-input signals. Thus it remains allowable to have *dynamic resets*, i.e. activation of block resets and associated resetting of direct input signals as part of the normal operation of the HW. Use of dynamic reset with the Appendix K Policy-S deadlock reduction is additionally subject to K.ResetEpoch; see Appendix M.6.

A related but somewhat more complex case is where input signals to the HW block may only be changed at “agreed upon” times, typically while the portion of the HW block that relies on them is temporarily idle. For example, a block may process 2D images. At the start of each new image, it may be desirable for the TB or external environment to update the control signals for how the block will process the next image. Typically HLS designs are pipelined, and the HW pipeline for the current iteration must be fully *ramped down* before the input signals can be updated to affect the next iteration. In the pipelined-loop case, this relies on the general synchronization-boundary rule for pipelined loops: while the process is pending at the designated sync, any back-annotated message-passing capacity whose accepted data would belong to the next post-sync interval is temporarily unavailable to upstream producers until the sync commits. In this case we can use the SystemC *SyncChannel* or C++ *ac_sync* primitives to precisely synchronize the DUT with the TB/environment to enable the input signals to be updated at the correct

time. The `#pragma hls_direct_input_sync` directive shown below associates the sync operation with the direct inputs that it controls. The precise synchronization scheme shown here ensures that there are no pre-HLS versus post-HLS simulation mismatches even though we are using direct inputs and also changing their values while the design is executing.

When a signal or port covered by `#pragma hls_direct_input` is also included in the scope of a later `#pragma hls_direct_input_sync` directive, the sync directive selects the effective update contract for that controlled signal set. Such signals remain direct inputs for purposes of late physical sampling, but they are not required to remain stable for the entire reset epoch; instead, they must satisfy the sync-controlled discipline described here: updates may occur only at the designated synchronization handshake, and the values must remain stable between permitted sync-controlled updates and reset boundaries.

```
// This is example 61 in Catapult Matchlib examples
sc_in<bool> SC_NAMED(clk);
sc_in<bool> SC_NAMED(rst_bar);

Connections::Out<uint32_t> SC_NAMED(out1);
Connections::In<uint32_t> sample_in[num_samples];
Connections::SyncIn SC_NAMED(sync_in);
#pragma hls_direct_input
sc_in<uint32_t> direct_inputs[num_direct_inputs];

void main() {
    out1.Reset();
    sync_in.Reset();

#pragma hls_unroll yes
    for (int i=0; i < num_samples; i++) {
        sample_in[i].Reset();
    }

    wait(); // reset state

    while (1) {
#pragma hls_direct_input_sync all
        sync_in.sync_in();

#pragma hls_pipeline_init_interval 1
#pragma hls_stall_mode flush
        for (uint32_t x=0; x < direct_inputs[0]; x++) {
            for (uint32_t y=0; y < direct_inputs[1]; y++) {
                uint32_t sum = 0;
#pragma hls_unroll yes
                for (uint32_t s=0; s < num_samples; s++) {
                    sum += sample_in[s].Pop() * direct_inputs[2 + s];
                }
                ac_int<32, false> ac_sum = sum;
                ac_int<32, false> sqrt = 0;
                ac_math::ac_sqrt(ac_sum, sqrt); // internal loop unrolled in cat .tcl
file
                if (sqrt > direct_inputs[7])
                    out1.Push(sqrt);
            }
        }
    }
}
```

```
};
```

From the perspective of the testbench or the environment, the updating of the signals controlled by the `hls_direct_input_sync` directive shall only occur at a precise point. The TB shall first wait for the `rdy` signal for the sync to be asserted by the DUT, and then the TB shall update all the input signals it wishes to change while simultaneously driving the `sync_vld` signal high for one cycle.

It is important to note that the only safe operation to use to synchronize the updating of direct inputs is the sync operation as shown above. Other operations such as Push/Pop or `ac_channel` operations should not be used for this. In terms of the Appendix G trace model, the Σ -visible Read(sig,val) events remain anchored to their synchronization points; any additional internal sampling made by the implementation between anchors is treated as an ϵ -step and must not change the recorded value label.

In the example above the DUT block that is being synthesized determines when to call sync, and thus it determines when the direct inputs will be updated. In some cases, it may be necessary for the environment around the DUT to determine when the direct inputs should be updated. In this case the same approach as shown above should be used; however a separate input from the environment to the DUT (either using a signal or a message-passing interface) should request that the DUT call sync as soon as feasible. This will ensure that the DUT has properly ramped down its pipeline and is ready to receive the newly updated direct inputs as per the overall synchronization scheme described above.

Additional Options for Scheduling Message-passing Interfaces

The following option may be added during HLS (Cat2):

```
STRICT_IO_SCHEDULING=relaxed
```

When this is specified, the HLS tool is allowed to reorder message-passing interface calls freely on distinct interfaces (still not moving calls across synchronization interface calls). This relaxed mode may violate the default no-reverse discipline and therefore may introduce deadlocks that are not present under the default rules. It is recommended that this relaxed option only be used when the user wants to see what order the HLS tool prefers to schedule message-passing interface calls (e.g., to achieve best QOR).

Once the user knows the preferred order, it is recommended that the user modify the pre-HLS source code to reflect the preferred order, and then return to the use of the default scheduling modes. This methodology ensures that HLS cannot introduce new deadlocks *via reversal of source order on distinct message interfaces* as described earlier. Deadlock behavior may still depend on bounded-capacity effects and on overlapped execution (e.g., loop pipelining); those cases are addressed separately in this document via the pipelined-loop discussion (automatic flush / WFG-inertness) and the later deadlock reasoning. The formal equivalence and deadlock-preservation results in Appendices G–K assume `STRICT_IO_SCHEDULING` is not set to relaxed.

Scheduling of Array Accesses

Arrays may appear in HLS models, and they may be preserved through synthesis and mapped to RAMs. Pointers may also appear in HLS models, and pointer dereferences are resolved to array accesses during HLS.

There are two cases to consider for arrays for the purposes of the scheduling rules:

1. Array instantiation in the HW is internal to the process

- The array accesses are not visible outside the process, and thus their scheduling is also not visible externally.
- All the scheduling rules described elsewhere in this document remain unaffected in this case.

2. Array instantiation in the HW is external to the process

- In this case the user model shall indicate how array accesses are mapped onto IO operations that are external to the process.
- We call this the *array access mapping layer*. The *array access mapping layer* maps array accesses onto IO operations described above (signal IO, message-passing interface calls, and synchronization calls).
- The user model may indicate that it is allowable for HLS to transform array accesses, for example, to cache, merge, split, or reorder array accesses (e.g., to improve QOR). These transformed operations, if allowed, are an outcome of using the *array access mapping layer*.
- In all cases the scheduling rules described elsewhere in this document for the core IO operations (signal IO, message-passing calls, synchronization calls) remain unaffected.
- In all cases array accesses shall remain constrained by any explicit synchronization interface calls present in the process in the source model. Precisely speaking, all array reads or writes before a synchronization call shall commit before or in the same cycle at which the synchronization call commits, and all array reads or writes after a synchronization call shall commit in a cycle after the synchronization call commits. This rule is stated in terms of commit cycles (the cycle the mapped memory storage is accessed); the scheduler enforces it by choosing issue cycles consistent with the fixed access latency defined by the array access mapping layer.
- Note that if transformed operations occur and array accesses are visible externally in both the pre-HLS and post-HLS model, then comparison of pre-HLS and post-HLS behaviors may need to account for the transformed operations.

When a loop is pipelined, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, an access to a memory interface (or array) may be moved over or in parallel with an access to the same memory if the HLS tool can prove the reordering is conflict-free. If the array/memory is external to the process, the *array access mapping layer* shall indicate that such reordering is allowable if such reordering is to occur during loop pipelining.

Definitions: Issue vs. Commit

Informally, we can define **commit** and **issue** as follows:

Definition: Commit (message channel) — as observed by an external clocked observer

Commit is the cycle in which an external clocked observer sees a transfer *complete*:

- A message commits in the cycle when the observer sees **both valid and ready high at the same time**.
- The committed payload is the data value seen in that same cycle.

Definition: Issue (message channel) — message request assertion

Issue is the cycle in which the calling process begins requesting the transfer at the selected message-

channel boundary. For a Push, its boundary manifestation is assertion of vld; for a Pop, it is assertion of rdy. Appendix C defines the precise attribution of an observed request to a dynamic source operation.

For RAM reads/writes as presented above, **commit** denotes the precise cycle the memory storage is accessed; since the access latency is fixed and known to the HLS scheduler, it can schedule the operation's issue cycle so that the commit cycle satisfies the required ordering relative to Sync. The "Scheduling of Array Accesses" constraints above are therefore written in commit-time.

Sequential processes use registered cycle semantics: a result returned by a message-passing call at a cycle boundary cannot be used to generate a dependent request in that same cycle. Such a dependent request may first issue in a later cycle. Appendix C and Definition Core.RegSeq give the precise rule, including its treatment of failed non-blocking calls and independently active pipeline iterations.

Additional States Added by HLS Synthesis

By default, HLS synthesis tools may add additional states to processes (e.g. add latency to enable resource sharing), which may introduce latency differences in the interface behavior between the pre-HLS and post-HLS models. These additional states are never included in the set of *synchronization interface calls* as described above.

When the directive `IMPLICIT_FSM=true` is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, the internal state machines of the pre-HLS and post-HLS models will be the same.

When the directive `IO_MODE=FIXED` is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, it is still possible that the state machine internal to the process is different between the pre-HLS and post-HLS models (e.g. the post-HLS model may choose to use a pipelined multiplier where the pre-HLS model did not.)

Avoiding Pre-HLS and Post-HLS Simulation Mismatches

The scheduling rules described in this document are designed to be easy to understand, while providing good QOR via HLS and generally avoiding any mismatches between the pre-HLS and post-HLS simulation behaviors.

Non-blocking message-passing operations (PushNB/PopNB in SystemC, C++ `ac_channel nb_read/nb_write`) are a potential source of mismatches between pre-HLS and post-HLS simulations since their behavior is inherently dependent on the latency within the model, which often changes during HLS. Because of this, non-blocking message-passing interfaces should only be used when no alternative approach is possible. For example, non-blocking message-passing interfaces are required to model time-based arbitration of multiple message streams which access a shared resource. A full discussion of recommended guidelines on the use and verification of non-blocking message-passing interfaces is provided in Appendix L. Note that the scheduling rules described previously in this document fully specify how HLS tools are required to schedule such operations.

Unidirectional message-passing between two processes should not be relied on to achieve synchronization between the two processes, since in general the message latency and storage capacity

between the processes may be variable. Also, depending on buffering/pipelining in the realization, the time between a producer's committed Push and the consumer's committed Pop (and the transient in-flight occupancy) may vary. Two unidirectional message-passing channels in opposite directions can be relied upon for synchronization between two processes. (An example of this is the AXI4 `ar` and `r`, and `aw` and `b`, channels). Note that such synchronization is weaker than explicit synchronization like `SyncChannel` or signal IO that uses a two-way handshake.

SystemC signal IO operations are a potential source of pre-HLS versus post-HLS simulation mismatches since timing behaviors may change between the two models. The following section provides guidance and rules to help avoid potential mismatches due to signal IO.

When signal IO operations are synchronized with a wait statement, there generally should be a proper two-way handshake associated with the wait statement so that the signal IO is latency insensitive. (Note that this statement does not apply to the signal synchronization approaches described in the Direct Inputs section.)

For example, here's a simple two-way handshake protocol when writing the signal `out_dat`:

```
out_dat = value;
out_vld = 1;
do {
    wait();
} while (out_rdy != 1);
out_vld = 0;
```

And here's a two-way handshake example when reading signal `in_dat`:

```
in_rdy = 1;
do {
    wait();
} while (in_vld != 1);
value = in_dat;
in_rdy = 0;
```

Some signal-level protocols have different two-way handshaking approaches (e.g. ARM APB), but they are still latency insensitive.

If signal IO operations are associated with a wait statement and that wait statement does not have a proper two-way handshake, then the signal IO is likely to be latency-sensitive and may result in pre-HLS versus post-HLS simulation mismatches. In some systems a one-way signal handshake is sufficient for reliable system operation. See Appendix E for further discussion.

The scheduling rules state that signal IO operations occur at either SystemC wait statements or `SyncChannel` calls (`sync_in` and `sync_out`). In the remainder of this section, we will use *wait* statements to refer to both.

RULE 1: It is always best coding style to group signal write operations just before their corresponding wait statement, and signal read operations just after their corresponding wait statement. (Cat3). An example is below:


```

sc_in<int> i1;
sc_in<bool> go;
sc_out<int> o1;
void my_thread() {
    int new_val=0;
    while (1) {
        o1.write(new_val);
        do {
            wait();
        } while (!go.read());
        new_val = i1.read();
        new_val = some_function(new_val); // function has no internal IO
    }
}

```

By placing the signal IO operations as close as possible to their corresponding wait statement, the HW intent is very clear. And there is no benefit either in terms of simulation performance or HLS QOR if they are placed further away from their corresponding wait statement.

Let's look at another similar example, which now also uses a Matchlib Connections blocking Pop operation:

```

sc_in<int> i1;
sc_in<bool> go;
sc_out<int> o1;
Connections::In<int> pop1;
void my_thread() {
    int new_val=0;
    while (1) {
        o1.write(new_val);
        do {
            wait();
        } while (!go.read());
        int pop_val = pop1.Pop();
        new_val = i1.read();
        new_val = some_function(new_val + pop_val); // function has no internal
IO
    }
}

```

Under the anchoring rules, `i1.read()` remains associated with the same synchronization point regardless of the intervening Pop. However, in raw pre-HLS simulation, a blocking Pop may stall for one or more cycles, so `i1` can change before the `i1.read()` executes—creating behavior that differs from the anchored interpretation (and potentially from post-HLS scheduling). The fix is simply to place `i1.read()` immediately after its intended `wait()` anchor; this removes the mismatch risk without affecting QOR or simulation performance.

For a proof-backed flow, RULE 1 conformance is a source well-formedness obligation, not merely a recommended diagnostic. If a blocking message-passing operation separates a signal read or write operation from its corresponding synchronization interface call, then the flow shall either reject the model for purposes of applying the formal guarantees, require the source to be rewritten so the signal IO is adjacent to its intended anchor, or discharge an explicit source-level well-formedness argument showing that the signal IO has a unique unconditional real synchronization anchor on every dynamic path and cannot observe a value inconsistent with that anchor. HLS tools may implement this

requirement using errors, warnings, lint checks, or other diagnostics, but an unresolved warning is not sufficient to claim the RULE 1 / RULE 2-based guarantees.

Another scenario in which RULE 1 applies is shown below:

```
sc_in<bool> go;
sc_out<int> o1;
void my_thread() {
    while (1) {
        wait();           // WAIT 1
        o1.write(some_value);
        if (go.read()) {
            some_value = some_function();
        }
        else {
            wait();        // WAIT 2
        }
        some_other_function();
        wait();            // WAIT 3
    }
}
```

The signal read of `go` is clearly and uniquely associated with WAIT 1. However, the signal write of `o1` associates with WAIT 2 if `go` is false and WAIT 3 if it is not. This is a violation of RULE 1 and should be flagged as an error during HLS. The fix, as before, is to move the signal IO operation as close as possible to its intended wait statement so that the association is unconditional.

Next, let's consider *rolled* (or *preserved*) loops that perform signal IO within the loop body. Consider the following example:

```
sc_in<int> i1;
sc_out<int> o1;
void my_thread() {
    wait(); // reset state
    while (1) {
        wait(); // start of while loop
        #pragma hls_unroll no
        for (int i=0; i < 10; i++) {
            o1.write(i1.read() * i);
        }
    }
}
```

Note that the `i1.read()` operation is located inside the `for` loop, so presumably the user's intent is that it should be read as the loop iterates. *If that is not the user's intent, they simply should move the `i1.read()` operation before the loop start.*

In the post-HLS simulation, each iteration of the loop will consume at least one clock cycle, and a new value for `i1` will be read (and a new value for `o1` written) on each iteration. Again, this is the user's intent as per the code. In the pre-HLS simulation, the `for` loop body will execute in zero time, and only the last write to `o1` will have any effect. The solution to avoid this mismatch is to manually place a `wait()` statement within the `for` loop body so that the signal IO synchronization is explicit in the pre-HLS simulation.

RULE 2: If you have signal IO operations within *rolled* (or *preserved*) loops, manually place a wait statement within the body of the loop to avoid pre-HLS versus post-HLS simulation mismatches, and while doing so also follow **RULE 1**.

Scope of RULE 2 for pipelining. RULE 2 applies only to rolled or preserved loops that are not selected for pipelining under this contract. A loop body selected for pipelining shall not contain wait/ac_wait statements, SyncChannel/ac_sync calls, or any other explicit synchronization interface call. Therefore, if ordinary per-iteration signal IO inside a loop requires a per-iteration wait anchor, that loop is outside the pipelined-loop contract unless the source is rewritten: move the signal IO outside the pipelined loop, model the interaction through message-passing, use hls_direct_input / hls_direct_input_sync under their stated contracts, or forgo pipelining for that loop. Without an in-body synchronization anchor, repeated per-iteration signal reads or writes inside one surrounding source interval do not become distinct per-iteration Σ -visible Read/Write events merely because HLS later pipelines the loop; their logical anchors remain the surrounding synchronization calls selected into S under R2/E2.

For a proof-backed flow, a rolled loop with signal IO operations but no wait statement in the loop body shall be treated as a source well-formedness failure unless an explicit source-level argument shows that each observable signal Read/Write still has a unique real synchronization anchor on every dynamic path. HLS tools may implement this requirement using errors, warnings, lint checks, or other diagnostics. However, if a diagnostic is emitted as a warning rather than an error, the warning must be resolved, waived with an explicit well-formedness justification, or treated as excluding that model from the formal guarantees. (Cat4)

If a pre-HLS model has established RULE 1 and RULE 2 conformance, then all signal IO in the post-HLS model shall occur only at clock cycles that correspond to explicit wait statements or explicit synchronization statements. For direct-input signals/ports (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync), this requirement refers to the logical (Σ -visible) anchor cycle of the Read/Write: the event is still considered to occur at its synchronization point even if HLS physically samples a stable signal later; such late-sampling micro-steps are ϵ -steps and are not Σ -visible. User designs that do not adhere to both RULE 1 and RULE 2 are ill-formed and are outside the formal guarantees unless the nonconformance has been eliminated or explicitly discharged by a valid source-level well-formedness argument.

Returning to the Analogy from RTL Synthesis

At the beginning of this document, we presented the example of a Verilog sequential block with an output coded like:

```
Out1 <= #some_delay new_val;
```

Recall that in Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning for the code above like “Simulation and synthesis results are likely to mismatch because delay in model is greater than clock period.”

HLS tools or associated lint/qualification flows that adhere closely to the conceptual model presented in this document should provide a concrete mechanism for detecting or discharging violations of RULE 1 and RULE 2 as described in the section above. This is analogous to the error or warning that the RTL synthesis tool would provide in the example directly above.

However, the formal guarantees in this document do not depend on any particular diagnostic mechanism. If an HLS synthesis tool does not report a RULE 1 / RULE 2 issue, the design is not thereby made well-formed. The proof-backed flow must still establish that the source model satisfies the RULE 1 / RULE 2 well-formedness obligation before applying the trace-compatibility guarantees.

Summary

At the beginning of this document, we said that the intent was to present “no surprises” to a DV engineer who wants a common verification methodology for the pre-HLS and post-HLS models. The key aspects of the document which support this are:

- Three groups of IO operations are defined (message-passing, signal IO, and synchronization calls) and each is treated uniformly. These IO operations are easy for verification engineers to understand because they are already using them in their testbenches.
- The document specifically avoids complex constructs such as *protocol regions* used in some HLS tools.
- The document preserves the ability of the pre-HLS SystemC model to be *throughput accurate* by using a library such as Matchlib.
- Synchronization calls can affect the scheduling (anchoring) of signal IO operations, and synchronization calls can affect the scheduling of message-passing calls. In the conceptual anchoring semantics of this document, message-passing calls do not affect the anchor cycle of signal IO operations (and signal IO does not affect the legality/order of committed message transfers) except through explicit synchronization. Practically, in raw pre-HLS SystemC simulation a blocking message operation may introduce additional cycles of waiting, so signal IO statements that are lexically separated from their intended anchor can observe different values; RULE 1 addresses this by recommending placement of signal IO adjacent to its anchor.
- HLS cannot by default reverse the order of message-passing calls, so it cannot introduce new deadlocks *via call-order reversal*. For pipelined loops (overlapped iterations) and other transformations that change the timing of request/commit under bounded capacities, deadlock preservation depends on the additional pipelined-loop discipline described in this document (automatic flush / WFG-inertness), not on the no-reverse rule alone.
- HLS pipelining is largely a *don't care* from the perspective of the verification engineer. If the design and the testbench are insensitive to changes in latency, and if external array accesses are not reordered or rearranged during loop pipelining, then the possible use of HLS pipelining will not affect verification, provided the pipelines flush automatically. Even if the design or testbench are sensitive to changes in latency, or if they are sensitive to reordering or rearranging of external memory accesses due to the use of HLS loop pipelining, then the

behavior of the DUT will only change in expected (rather than unexpected) ways, provided the applicable observation-boundary and witness-selection conditions are satisfied. (Appendix G provides formal trace-compatibility guarantees rather than an unrestricted bidirectional equivalence theorem.)

- For sequential processes, signal IO operations in the post-HLS model always occur exactly at synchronization points (e.g., wait statements) that are explicit in the pre-HLS source. For direct-input signals/ports, “occur at synchronization points” is meant in the logical/anchoring sense: the Σ -visible Read/Write is timestamped at the synchronization point even if the implementation samples the stable signal later; those internal late samples are ϵ -steps. For combinational processes, observable signal Read/Write events are instead interpreted at the cycle’s ϵ -quiescent combinational fixed point, as specified by R2C.

The principal formal safety and deadlock-preservation results, their scope limitations, and the evidence required to apply them are summarized in Appendix M. Appendix F provides user-oriented modeling guidelines.

Appendix A – Factory Analogy

The scheduling rules described in this document apply to pre-HLS and post-HLS HW models. Although the rules may seem abstract and perhaps even arbitrary, they are shaped by the need to model systems in the real world that are required to have predictable behavior.

To understand the motivation behind the rules, it might be helpful to draw a simple real-world analogy and its correspondence to the rules described earlier.

Consider a factory that produces various types of wooden furniture:

The factory consists of people (processes) stationed at workbenches with various tools.

Each person is given written instructions about the specific tasks they are to perform (C++ code within a process).

People are instructed to send or receive objects (messages) to or from other people in the factory.

Sending or receiving objects may be blocking or non-blocking from the perspective of a person.

There is a clock with a second hand on the factory wall that everyone can see. (HW clock).

People have colored flags they can raise or lower to communicate with other people (signals).

If someone raises or lowers a flag, this is only seen by others the next time the clock second hand is at the top of the clock. (propagation delay of signals between sequential processes)

A person can choose to pipeline the tasks that they were assigned by hiring subordinates (pipeline stages) and having each one do a subtask. In general, this will improve the throughput for the tasks that the person was assigned.

It is possible for two people to explicitly synchronize their work by communicating via a synchronization protocol such as a barrier (synchronization).

It is possible to restart the work of some or all the people by raising a reset flag (HW resets).

Assume:

1. That the time that people take to complete their various tasks is in general variable, and similarly that the time that objects (messages) take to pass between people is in general variable.
2. That each person in general wants to complete their tasks as quickly and efficiently as possible.
3. That the factory needs to be able to make multiple types of furniture at the same time. For example, it might make chairs that need to be different colors or have different styles.

To reliably produce output, each person in the factory will need to adhere to rules like those described in this document.

Here's a sketch of a specific way the above example relates to the scheduling rules:

Person 1 sends chair seats and chair backs to Person 2. This is done with blocking operations, and there is no storage capacity between the people when the objects are sent. (This means that a Push operation cannot complete until the corresponding Pop operation is performed.) Assume that the fastest an object can be sent between people is 1 minute.

The written instructions that person 1 is given are:

```
while (1) {  
    // internal processing code for seats and backs ...  
    seats.Push(seat_object);  
    backs.Push(back_object);  
}
```

The written instructions that person 2 is given are:

```
while (1) {  
    seat_object = seats.Pop();  
    back_object = backs.Pop();  
    // internal processing code for seats and backs ...  
}
```

If both person 1 and person 2 choose to perform their tasks sequentially as written, then objects will be passed over time as:

	Person 1	Person 2
Minute 1:	seats.Push(seat1);	
Minute 2:	backs.Push(back1);	seat1 = seats.Pop();
Minute 3:	seats.Push(seat2);	back1 = backs.Pop();
Minute 4:	backs.Push(back2);	seat2 = seats.Pop();
Minute 5:		back2 = backs.Pop();

If both person 1 and person 2 choose to perform their IO operations in parallel, then objects will be passed over time as:

	Person 1	Person 2
Minute 1:	seats.Push(seat1); backs.Push(back1);	
Minute 2:	seats.Push(seat2); backs.Push(back2);	seat1 = seats.Pop(); back1 = backs.Pop();
Minute 3:		seat2 = seats.Pop(); back2 = backs.Pop();

If person 1 chooses to perform their IO operations in parallel, and person 2 chooses to perform their IO operations sequentially as written, then objects will be passed over time as:

	Person 1	Person 2
Minute 1:	seats.Push(seat1); backs.Push(back1);	
Minute 2:		seat1 = seats.Pop();
Minute 3:	seats.Push(seat2); backs.Push(back2);	back1 = backs.Pop();
Minute 4:		seat2 = seats.Pop();
Minute 5:		back2 = backs.Pop();

If person 1 chooses to perform their IO operations sequentially as written, but person 2 chooses to perform their IO operations sequentially in the reverse order as it was written, then objects will be passed over time as:

	Person 1	Person 2
Minute 1:	seats.Push(seat1);	
Minute 2:	backs.Push(back1);	back1 = backs.Pop();
Minute 3:		seat1 = seats.Pop();

In this case the person 1 seats.Push(seat1) operation at minute 1 will not complete at the start of minute 2. This means that the person 1 backs.Push(back1) operation will never start, and thus the Person 2 back1 backs.Pop() operation will never complete. So, the system will be in deadlock.

This example directly corresponds to the ordering rules related to message-passing operations in the *basic conceptual model* within this document, and in particular the specific rule that disallows reversing the order of message-passing operations.

Appendix B – Formal Presentation of Pipelined Loops

Modeling convention for pipelined-loop overlap.

Pipelined-loop body no-wait condition. The formal pipelined-loop model in this appendix applies only to HLS-pipelined loop bodies that contain no wait/ac_wait statement, SyncChannel/ac_sync call, or other explicit synchronization interface call inside the loop body. All ordinary synchronization calls selected into S must be outside the pipelined loop body or at explicit boundaries around the pipelined region. This is a well-formedness condition for applying the Appendices G-K theorem stack to a pipelined-loop instance; an in-loop synchronization call requires source rewriting, non-pipelined/preserved-loop treatment, or a separate proof/certificate outside this appendix.

The post-HLS pipelined-loop implementation may overlap iterations and may internally accept/stage up to the finite bound $B(c_in)$ of input values for younger overlapped iterations before older iterations' outputs are produced. This internal accept/stage behavior is part of the back-annotated channel realization counted in $B(c_in)$ between the Σ -visible endpoints of c_in (producer $Push_c_in$ and consumer Pop_c_in). Here $B(c_in)$ is an effective abstract-boundary capacity/credit bound, not a claim that the RTL contains a single physical FIFO named c_in ; the proof obligation is that the chosen Sys_ref boundary exposes E4-consistent ChannelEnabled/ChannelDisabled behavior, with hidden physical storage represented as ϵ -state unless it is explicitly elaborated into Sys_B^{elab} channels. These internal

accept/stage movements are modeled as ϵ -steps. **Σ records only the committed boundary transfers at the Σ -visible endpoints.** In particular, the consumer-side boundary $\text{Pop}_{\{c_in\}}$ is the loop-body $\text{Pop}_{\{c_in\}}$ operation from the pre-HLS source when that Pop is included in the chosen Σ / Σ_DUT projection. The externally relevant effect of pipelining is therefore captured entirely by $B(c_in)$ (via boundary backpressure/acceptance behavior and overlap timing at the Σ -visible boundary), without introducing any new auxiliary or duplicate Σ -visible boundary Pops: the Σ -visible consumer-side $\text{Pop}_{\{c_in\}}$ commits are still the same loop-body $\text{Pop}_{\{c_in\}}$ operations from the pre-HLS source, though they may commit at different times (and thus at different prefixes) than in an unpipelined execution. Once the pipeline reaches a quiescent blocked state under the automatic flush discipline below, it does not initiate new blocking request assertions for younger overlapped iterations.

B-faithfulness obligation

The use of $B(c_in)$ in this modeling convention assumes that the selected abstract boundary and the reported capacity faithfully summarize the corresponding RTL realization: all storage/credit that can accept, hold, or backpressure messages between the chosen producer and consumer endpoints is either counted in $B(c_in)$ or represented as an explicit $\text{Sys_B}^{\text{elab}}$ boundary, and movements inside that realization are ϵ -steps that do not create, delete, duplicate, reorder, or relabel Σ -visible boundary transfers. This is a tool/RTL-flow compliance obligation, not a fact derived by Appendix B.

Multi-input Pop points and coupler policies.

When a pipelined loop consumes from multiple input channels (e.g., $c1, c2$), back-annotation still assigns a separate finite bound $B(c_i)$ and cycle-boundary occupancy $\text{occ}_{c_i}(t)$ to each outer Σ -visible boundary channel c_i . However, in the multi-input / coupler cases of Appendix Q the formal source-side proof target is no longer merely the outer-boundary Sys_B ; it is the elaborated reference model $\text{Sys_B}^{\text{elab}}$ of Definition Core.1 and Appendix J, Remark 2. In $\text{Sys_B}^{\text{elab}}$, $B(\cdot)$, $\text{occ}_{\cdot}(t)$, E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG machinery are interpreted per explicit abstract channel, including any introduced staged/bundle/join channels. Any conjunctive/join dependency (e.g., “join not met”) shall therefore be represented only via such explicit staged/bundle/join boundary channels in $\text{Sys_B}^{\text{elab}}$ (not as cross-channel disablement at the outer Σ -visible endpoints), so that for Appendix K the blocking point is that explicit boundary and the relevant $\text{Front_P}(\sigma)$ members there are jointly channel-disabled (“ALL disabled $\Rightarrow$ stall”). For the detailed elaboration patterns and examples, see Appendix Q.

Practical note (outer Σ_DUT projection and proof-internal Σ)

The loop-body $\text{Pop}_{\{c_in\}}$ in the pre-HLS source (i.e., the consumer-side $\text{Pop}_{\{c_in\}}$ operation, conceptually after the in-flight capacity $B(c_in)$) may be exposed in the outer DUT/testbench-visible alphabet Σ_DUT if desired, but is typically omitted since it is latency-insensitive, and it is often practically difficult to expose/collect in the post-HLS RTL. The liveness/deadlock reasoning still accounts for this blocking point without requiring it to be Σ_DUT -visible: Appendix K defines WFG edges using the ChannelEnabled/ChannelDisabled status of the corresponding pending boundary operation instance q with $\text{kind}(q)=\text{Pop}_{\{c_in\}}$ (via E4/ $\text{occ}_c/B(c)$). In ordinary non-elaborated Sys_B cases, this does not require introducing any additional proof-internal abstract channel. In elaborated cases $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$, however, any introduced staged/bundle/join channel needed to represent a blocking point is an explicit proof-internal Σ boundary for Appendix K, even if its committed Push/Pop events are projected away from the outer Σ_DUT trace by Π_elab .

Drain-only blocked-frontier discipline and automatic flush.

Potential deadlocks are avoided by having the pipeline automatically flush.

In this document, “automatic flush” does not mean a global pipeline clear or a mandatory drain-to-empty phase. It is the pipelined or otherwise overlapped implementation of the more general drain-only blocked-frontier discipline used by Appendix K. Once the minimal pending blocking frontier is fully channel-disabled after ε -settling, the process is blocked at that frontier; no new later source work may issue past that frontier; already asserted blocking requests remain asserted until commit; and already-issued downstream work may continue to drain only insofar as it does not create new WFG-relevant blocking causes. For ordinary non-overlapped control, the same Appendix K discipline may instead be discharged structurally, as described below.

Terminology (pending blocking frontier / $\text{Front_P}(\sigma)$)

At a settled boundary-evaluation cutpoint, consider the set of pending source-level blocking message-operation frontier items $x \in \Omega_{\text{msg},P}$ in the current interval between adjacent synchronization calls, where each such item has an owning dynamic message-call instance $q = \text{MsgOf_P}(x)$ whose $\text{Issue}(q)$ has occurred and whose $\text{Commit}(q)$ has not yet occurred. By the Issue/Commit linkage (Appendix C) together with BoundaryReq ’s definition at the Σ -visible boundary (as given in “Normative Formal Core for Appendices G–K”), each such pending blocking frontier item x has its endpoint-owned boundary request asserted through q (Push: vld, Pop: rdy) until q commits; purely combinational channel-side structure (including couplers) may affect only the complementary mirror signal (Push: rdy, Pop: vld) and thus whether x commits in that cycle after ε -settling. Define $\text{Front_P}(\sigma)$ to be the set of such pending blocking message-operation frontier items that are minimal with respect to the source-induced order $\leq_P$ within that interval. By Lemma $\text{Core.OrdFrontSingleton}$, an ordinary non-elaborated source interval has at most one minimal pending blocking message-operation frontier item. If $\text{Front_P}(\sigma)$ has multiple message-operation members, that multiplicity must come from explicit $\text{Sys_B}^{\text{elab}}$ proof-internal structure such as staged/bundle/join/epoch-gate abstract channels; it is not inferred merely from the fact that ordinary source calls use distinct interfaces. We refer to $\text{Front_P}(\sigma)$ as the pending blocking frontier (or “minimal pending blocking frontier”). This $\text{Front_P}(\sigma)$ notion is distinct from NextFront_P (Definition Core.6 ; reconstructed from KObs by Lemma J.KObs-Proc and Corollary J.1), which denotes the next source-level frontier-item set over Ω_P and may include synchronization-call items, signal-anchored action items, and message-operation items. A message-operation item in NextFront_P may be pending before it contributes any committed Σ -event.

A pipelined loop supplies the drain-only blocked-frontier discipline by automatically flushing if, whenever the pending blocking frontier $\text{Front_P}(\sigma)$ is non-empty and every frontier item $x \in \text{Front_P}(\sigma)$ cannot make progress because it is not channel-enabled (evaluated after combinational ready/valid/backpressure has ε -settled for the cycle), then:

1. **Block point.** The process is blocked on message passing at that pending blocking frontier $\text{Front_P}(\sigma)$, as in the unpipelined source.
2. **No new later work.** While blocked at that frontier (i.e., while $\text{Front_P}(\sigma)$ remains non-empty and all of its members are channel-disabled), the implementation does not issue any new blocking Pop/Push message-operation frontier item $y \in \Omega_{\text{msg},P}$ that is later in source order than some $x \in \text{Front_P}(\sigma)$, including any younger overlapped iteration, and therefore does not begin asserting $\text{BoundaryReq}(y, \cdot)$ for such later work. Here issue is read through the owning message instance $q_y = \text{MsgOf_P}(y)$.

3. **No withdrawal.** While blocked, the implementation does not withdraw any already-asserted blocking Pop/Push request (no “try-and-withdraw”); outstanding blocking requests remain asserted until their owning message instances commit.
4. **Clarification (Frontier-minimality vs. other asserted requests).** The Wait-For Graph in Appendix K is intentionally defined from the minimal pending blocking frontier $\text{Front_P}(\sigma)$, i.e., the current blocking causes. Accordingly, an already-asserted blocking request whose frontier item is not in $\text{Front_P}(\sigma)$ (for example, a request from a younger overlapped iteration that was issued before the blocked condition, or a downstream request draining work already past the frontier) may persist as protocol state under the “no withdrawal” rule, but it is not treated as a wait-for source while an earlier pending blocking frontier item remains frontier-minimal. Such a request contributes WFG blocking only if/when its frontier item later becomes a member of $\text{Front_P}(\sigma)$.
5. **Drain-only downstream progress.** Work that has already passed the blocked frontier point(s) may continue to drain, and any already-asserted resulting output-Push requests may commit when channel-enabled, provided no work advances past the blocked frontier point(s). (Formally: “no work advances past” is meant in the $\leq_P$ / issue sense—while $\text{Front_P}(\sigma)$ is fully disabled, P does not issue any new blocking message-operation frontier item y , with owning instance $q_y = \text{MsgOf_P}(y)$, such that, for some $x \in \text{Front_P}(\sigma)$, $x \leq_P y$ and $x \neq y$; downstream drain may only complete/commit operations whose BoundaryReq was already asserted prior to reaching this blocked condition.) (Equivalently: this is the “ALL disabled $\Rightarrow$ stall” policy— P stalls only when none of the frontier items in $\text{Front_P}(\sigma)$ are channel-enabled; downstream work may drain while $\text{Front_P}(\sigma)$ is fully disabled.)
6. **Finite explicit boundary storage/credit.** Every selected abstract boundary that can hold or credit already accepted work has a finite declared capacity $B(c)$, or is represented by an explicit finite $\text{Sys_B}^{\text{elab}}$ boundary chain whose component capacities are finite and B -faithful. Together with “no new later work,” this makes the set of drainable work finite and non-replenishing.
7. **Synchronization-boundary withdrawal of future-epoch acceptance.**
Let s be an explicit synchronization interface call that separates the current source interval from later work. While the process is pending at s , the implementation shall not present producer-facing availability for any back-annotated capacity intended only for $\text{suff_S}(s)$; equivalently, any asserted upstream producer request targeting such capacity shall remain channel-disabled until s commits. This does not prevent drain of work already accepted before reaching s . **Formal elaboration requirement.** For purposes of Appendices J–K, this withdrawal shall be represented in Sys_ref by the elaborated source model $\text{Sys_B}^{\text{elab}}$, not by adding a hidden extra enablement predicate to an otherwise E4-enabled outer channel. $\text{Sys_B}^{\text{elab}}$ shall introduce explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that capacity usable only by $\text{suff_S}(s)$ is separated from capacity available to the current interval. While s is pending, the gate/boundary for future-epoch acceptance is channel-disabled in the ordinary E4 sense at that explicit abstract boundary. After s commits, the corresponding post-sync capacity may again become producer-facing available according to the ordinary ChannelEnabled rules for the elaborated channels.

General drain-only blocked-frontier obligation used by Appendix K

Appendix K applies the drain-only blocked-frontier discipline to every source interval in which Policy L can leave source-later same-interval blocking work issued while an earlier minimal blocking frontier remains uncommitted. This includes pipelined-loop overlap and other explicitly modeled eager/overlapped issue admitted by the selected Sys_ref transition system; it is not a license to treat ordinary source-sequenced operations as incomparable, and it is not limited to physically pipelined control.

For pipelined or otherwise overlapped control, Appendix B automatic flush is the implementation-side discharge. For ordinary non-overlapped control, the obligation may be discharged by proving that, after the process reaches a channel-disabled blocked point, its control cannot launch any new source-later dynamic operation until the blocker commits. In both cases, Appendix C supplies the non-withdrawal discipline, and E4/B-faithfulness plus explicit $\text{Sys_B}^{\text{elab}}$ accounting supply finite boundary storage or credit. The resulting drain set may shrink through already-issued commits but cannot be replenished across the blocked frontier.

Example (II=1, latency=4, automatic flush): a loop that performs one blocking $\text{Pop}_{\{c_in\}}$ and one blocking $\text{Push}_{\{c_out\}}$ per iteration in the pre-HLS source may, after pipelining in the post-HLS RTL, internally accept/stage up to four input values into pipeline staging (pipeline overlap) before the first output $\text{Push}_{\{c_out\}}$ commits, subject to the back-annotated bound $B(c_in)$ (E4). Values may then reside for several cycles in internal pipeline storage; such internal staging/hand-off is ϵ -state within the back-annotated realization and does not create any new auxiliary or duplicate Σ -visible $\text{Push}_c/\text{Pop}_c$ events beyond the boundary commits at the Σ -visible endpoints (it may change only the timing of those boundary commits under overlapped execution). The drain-only blocking behavior is as defined above under “automatic flush”; in particular, if the blocked operation is in a late pipeline stage (e.g., the final Push), there may be little or no downstream work to drain.

Appendix C – Formal Definitions of Issue and Commit

Scope / shared definitions

The definitions below are with respect to the Σ -visible endpoints of the chosen source reference model Sys_ref (Definition Core.1 and Appendix J, Remark 2)—i.e., the ready/valid interfaces at which Σ records message-transfer events in the raw rendezvous case Sys , the ordinary bounded-FIFO case Sys_B , or the elaborated back-annotated case $\text{Sys_B}^{\text{elab}}$. All shared notions about (i) Σ vs. ϵ steps, (ii) BoundaryReq / ChannelEnabled / ChannelDisabled , (iii) non-blocking polls as ϵ on failure, and (iv) the $B(c)$ / $\text{occ}_c(t)$ (cycle-boundary, non-fall-through) interpretation of E4 are as defined in “Normative Formal Core for Appendices G–K” and are to be interpreted over the explicit abstract channels of Sys_ref . Appendix C defines Commit, Issue, ReturnedAtCycle, and the Issue/Commit linkage conditions that make $\text{Issue}(q)$ well-defined.

Definition: Commit (message channel) — as observed by an external clocked observer
Commit is the cycle in which an external clocked observer sees a transfer complete at a Σ -visible boundary endpoint: A message commits in the cycle when the observer sees both valid and ready high at the same time. The committed payload is the data value seen in that same cycle.

Definition: Issue (message channel) — as an auxiliary (scheduler/witness) timestamp
Issue(q) is an auxiliary timestamp used to state scheduling constraints (R4/E3/E5). Intuitively, it is the first cycle in which the calling process begins requesting the transfer corresponding to the source-level message operation instance q at the chosen Σ -visible boundary endpoint.

Definition: ReturnedAtCycle (message result availability). $\text{ReturnedAtCycle}(q,t)$ means that the process-visible result of message-call instance q becomes available at the cycle- t boundary. For a successful blocking or non-blocking call, this is the transfer-commit cycle. For a failed $\text{PushNB}/\text{PopNB}$ call, no message commits, but the success/failure status is nevertheless returned at the cycle- t boundary.

and is therefore a cycle- t result. ReturnedAtCycle is an operational timestamp used for sequential-result availability; it is not a KObs field.

Same-cycle rendezvous clarification. For a rendezvous channel $B(c)=0$, if a matched Push_c operation instance and Pop_c operation instance first assert their endpoint-owned requests in the same cycle t , and the ready/valid rendezvous is selected/observed in that cycle, then both operation instances have Issue = t and both committed transfer events occur in cycle t . There is no implicit extra cycle between same-cycle matching endpoint requests and the corresponding rendezvous commit. If one endpoint request was already asserted from an earlier blocking operation, then its Issue may be earlier than t , but the matched Push_c/Pop_c commit is still a same-cycle commit in cycle t .

Issue/Commit linkage (well-formedness conditions)

Fix a Σ -visible endpoint direction, and let req(t) denote the endpoint-owned boundary request signal in cycle t (valid for Push, ready for Pop), i.e., the process-driven request level captured by BoundaryReq at that boundary (per “Normative Formal Core for Appendices G–K”). For each source-level message operation instance q on that endpoint direction:

- Boundary-request soundness (blocking requests are non-withdrawable). req(Issue(q)) = 1. If q is a blocking Push/Pop, then within its reset epoch, once issued it remains the unique outstanding request on that endpoint direction until it commits: req(t)=1 in every cycle t from Issue(q) through the commit cycle of q (inclusive), with stable payload while asserted for Push. If an affecting RESET occurs first, RW2 ends that dynamic instance rather than withdrawing it within the epoch. (This is the “no deassert-before-commit” discipline used throughout the document.)
- Stable attribution for a single outstanding source-level request. If req(t)=1 and no transfer commits on that endpoint direction in cycle t , then any continued assertion req($t+1$)=1 is attributed to the same outstanding source-level message-operation instance q only so long as the requesting source-level operation remains that same outstanding instance. For blocking Push/Pop, this is the mandatory non-withdrawable case above, so attribution cannot change before commit. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal, by itself, does not identify later executed non-blocking call instances as the same q ; a new executed source-level non-blocking call creates a new dynamic instance $q' \in Q_{\text{exec}}, P$ for the owning process P , even if req does not deassert between cycles.
- Back-to-back issuance under continuously asserted req. If q_0 commits in cycle t and the next source-level operation q_1 on the same endpoint direction is issued back-to-back, then Issue(q_1)= $t+1$ (even if the request signal does not deassert between commits), provided q_0 and q_1 lie within the same source interval between adjacent ordinary synchronization calls selected into S .
- Sequential-result availability (no same-cycle dependent continuation). Let ReturnedAtCycle(q_0, t) hold in a sequential process P . If a dynamically succeeding operation q_1 is reachable only after q_0 returns, or if q_1 ’s control choice, endpoint, address, payload, or other required operand depends on the value or success/failure status returned by q_0 , then Issue(q_1) > t . This rule applies to successful commits and failed non-blocking returns, across distinct channels and endpoint directions as well as on the same endpoint. Same-cycle issue or commit remains permitted only when q_1 was independently reached and all required inputs, payload values, and control predecessors were available before the cycle- t boundary under Core.RegSeq; overlap between independently active pipeline iterations is not prohibited, but an unresolved NB result in an earlier iteration places any dependent later-iteration action in a strictly later Horizon.
- Sync-boundary discipline. If $q_0 \in \text{pref}_S(s)$ and $q_1 \in \text{suff}_S(s)$ for an ordinary synchronization call s selected into S , then Issue(q_1) is strictly after s ’s commit in the same trace (i.e., Issue(q_1) >

$\text{clk_pre}(s)$ in τ_{pre} and $\text{Issue}(q_1) > \text{clk_post}(s)$ in τ_{post}). If q_0 is a blocking Push/Pop, q_0 must process-facing commit no later than s ; no issued-but-uncommitted blocking operation in $\text{pref_S}(s)$ may survive the commit of s into the succeeding source interval. Same-cycle completion is permitted: q_0 and s may both commit in cycle t when q_0 's process-facing commit is part of the selected cycle- t boundary actions and is accounted for before the interval advance caused by s . Any boundary request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in $\text{pref_S}(s)$ and does not constitute issuance of any operation in $\text{suff_S}(s)$. Under the pipelined-loop body no-wait condition, there are no in-body wait events that create additional sync-boundary cuts inside an HLS-pipelined loop.

For non-blocking message passing (PushNB/PopNB), a failed poll produces no Σ -event and is modeled as an ϵ -step, while a successful poll contributes the corresponding committed Push_c(v) / Pop_c(v) event; see “Non-blocking message-passing” in “Normative Formal Core for Appendices G–K” and Appendix I.2. Each executed non-blocking call is a distinct dynamic source-level message-call instance $q \in Q_{\text{exec},P}$ for the owning process P . Thus, if repeated PopNB/PushNB polls occur in later loop iterations or at later source-level call instances, they are distinct q 's even if the endpoint request remains continuously asserted and no intervening commit occurs. Stable attribution therefore applies to a single outstanding source-level request instance; it does not collapse distinct non-blocking dynamic call instances across iterations into one q . Successful non-blocking calls that commit induce message-operation frontier items $x \in \Omega_{\text{msg},P}$ when their committed transfer is relevant to Ω_P , with owning message instances $q = \text{MsgOf}_P(x)$ in $Q_{\text{exec},P}$. Failed non-blocking polls remain only in $Q_{\text{exec},P}$ and contribute no Ω_P or $\Omega_{\text{msg},P}$ item.

Appendix D – Modeling Latency-Sensitive Protocols

In some cases, designs or testbenches may use protocols or local control policies which are latency-sensitive. These situations are handled by isolating the latency-sensitive portions into small, self-contained parts, and then keeping the rest of the design and testbench latency insensitive.

A latency-sensitive portion is any local block of logic whose functional or externally visible behavior depends on cycle-level timing that is not represented by the ordinary latency-insensitive Σ interface. This includes ordinary signal-level protocols with fixed cycle timing, global interrupt/request ordering logic, non-blocking arbitration whose winner depends on request arrival timing, and non-blocking retry/timeout logic whose later behavior depends on the number or cadence of failed PushNB/PopNB polls before success.

For such logic, the intended methodology is not to treat the hidden timing dependence as part of the ordinary latency-insensitive source-core proof. Instead, the latency-sensitive portion should be isolated behind an explicit boundary. On one side of the boundary, the transactor or local protocol block may be modeled cycle-accurately, snooped, or otherwise verified with its detailed timing behavior. On the other side of the boundary, the rest of the system should see a latency-insensitive interface, such as message-passing transactions, synchronization events, or a properly handshaken signal protocol.

Consider a case where a signal-level protocol is latency-sensitive. The protocol will have specific timing behavior that it must meet. To handle this, we create a transactor that has two sides: the first side interacts with the signals and handles the detailed timing requirements, and the second side sends and receives messages with the rest of the system. The message-passing side is latency insensitive. The same pattern applies to cadence-sensitive non-blocking polling logic: if retry counters, timeout counters, or

arbitration decisions depend on the number or cadence of failed non-blocking polls, that polling loop is part of the latency-sensitive local protocol block. Its detailed failed-poll cadence should be contained inside the isolated block or aligned by the Appendix L snooping mechanism; it should not leak as an unmodeled hidden timing dependence into the ordinary Appendix I/J latency-insensitive proof. To properly model the detailed timing behavior, the latency-sensitive transactor or local protocol block is modeled at the cycle-accurate level in SystemC, or is otherwise covered by an explicit snooping/alignment argument. There is an example of such a transactor model in the following document and example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/examples/53_transactor_modeling/transactor_modeling.pdf

For purposes of the formal appendices, once the latency-sensitive portion has been isolated, the ordinary Appendix I/J proof is applied at the chosen observable boundary of that isolated block. The failed-poll cadence internal to the isolated block remains local timing behavior; the rest of the system observes only the latency-insensitive or explicitly modeled boundary behavior.

Appendix E – One-Way Handshake Protocols

In some systems a one-way signal handshake is sufficient for reliable system operation. When using a one-way handshake, the implicit assumption is that the “missing” handshake isn’t required since it is always true.

For example, in time-domain signal processing hardware designs, signal processing HW blocks may input new samples at a fixed rate. The HW blocks are always ready to receive new samples on each clock, but they need to know if the samples are valid. These types of designs can be modeled with the one-way handshake dat/vld protocol demonstrated in this example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master/matchlib_examples/examples/32_dat_vld

Appendix F - Modeling Guidelines and Design Constraints

The following are simple modeling guidelines that align with the modeling methodology described in this document.

- 1) Prefer to use `Connections::In/Out + SyncChannel` over signal IO and `wait()`.
- 2) Prefer to use `Pop()/Push()` over `PopNB()/PushNB()`.
- 3) When pipelining a loop, prefer to use `hls_stall_mode flush`.

When using signal IO:

- 4) If modeling a cycle-accurate process, use `disable_spawn` and follow the example `53_transactor_modeling` style and do not use `Push/Pop` in the process.

- 5) If modeling a *direct input*, use `hls_direct_input` and possibly `hls_direct_input_sync`, and only change the signal at allowed times.
- 6) If combining signal IO with `Connections::In/Out` in the same process, use proper signal IO handshake that does not rely on In/Out ports for process synchronization. Place signal IO operations very close to their `wait()` statements.
- 7) Unless you are modeling a cycle-accurate process, you should expect that latency will change between the pre-HLS and post-HLS models.

Design Constraints for Applying the Formal Guarantees

This section consolidates the design choices that most directly determine whether the formal guarantees in this document can be applied. It is written for hardware architects and HLS users rather than for the tool or proof implementer. A design can synthesize and function correctly while still lying outside the stated theorem scope if one of these behaviors is left implicit.

This section is informative. Each constraint restates a normative requirement located in the referenced sections; the referenced text is authoritative.

The architect is responsible for choosing interfaces, reset behavior, and communication structures that admit the required correspondence. The HLS tool and verification flow are responsible for constructing and checking the detailed process, channel, attribution, coverage, and other certificates. The architect does not need to produce those certificates, but the design must expose enough explicit structure for the flow to do so.

The first list applies to the implementation-safety and observable-correspondence result. The second list contains additional constraints for the internal message-passing deadlock-preservation result. A design outside the simple route is not necessarily prohibited; it generally requires explicit modeling, witness alignment, or a separate proof route.

Design constraints for the safety and correspondence guarantee

These constraints support the Post $\rightarrow$ Ref guarantee that the covered RTL behavior introduces no new observable functional behavior absent from the prescribed source reference model.

1. **Give every signal read and write an unambiguous synchronization anchor.** For a sequential process, place signal reads immediately after the synchronization call that defines their observation point and place signal writes immediately before the synchronization call at which they become visible. Branches must not allow one write to be associated with different later waits on different paths. Signal I/O inside a rolled or preserved loop needs an explicit per-iteration wait when each iteration is intended to create a distinct observation. A loop selected for HLS pipelining cannot use an in-body wait under this contract; move the signal interaction outside the loop, use a supported message-passing or direct-input mechanism, or do not pipeline that loop.

2. **Keep direct inputs stable, or update them through the supported synchronization scheme.** A direct input may be physically sampled later than its logical source-level read only when its value remains stable for the applicable reset epoch. If software or the testbench must change control inputs while the design is running, use the documented `hls_direct_input_sync` handshake and change the values only at the designated synchronization point. An ordinary Push or Pop is not a substitute for that update contract.
3. **Make timing-sensitive environment behavior explicit.** The ordinary safety result assumes that the same external behavior is selected from the same prior observable history, not from hidden differences in absolute cycle count. Testbench branches, watchdogs, stimulus changes, or pass/fail decisions that depend on raw latency must be isolated as latency-sensitive logic, made observable through an explicit protocol, or checked at RTL rather than assumed to transfer automatically from an arbitrary pre-HLS run.
4. **Treat non-blocking polling as an architectural proof-route decision.** The simple route applies when a failed PopNB or PushNB does not change the next observable action, request, payload, endpoint choice, or control path. Typical arbitration code, retry counters, timeouts, backoff, and "work on B if A is unavailable" behavior do change what happens next. Such designs require an approved witness-alignment mechanism, commonly the Appendix L snooping route, or an isolated and explicitly modeled latency-sensitive protocol block. Prefer blocking interfaces when the architecture does not genuinely require poll-dependent control.
5. **Do not encode an unbounded wait as an invisible busy-poll loop in the ordinary proof fragment.** A loop that repeatedly executes failed non-blocking polls continues to take local internal steps and is not a stable waiting state. Use a blocking operation, an explicit stable-wait or handshake construct, or isolate and model the polling loop as timing-sensitive logic. A finite, statically bounded polling sequence can be handled when its outcomes and later effects satisfy the applicable witness and dependency conditions.
6. **Expose cross-process shared-memory value flow through a supported interface.** Synchronization alone can order two processes' memory accesses, but the present safety proof does not carry an independent global shared-memory state that proves both models read the same values. Use the supported memory library or mapping layer so commands, responses, or value-carrying accesses appear as covered message-passing or anchored-signal events. A design using an ordinary shared array between processes instead needs a separate memory-state correspondence proof that is not developed in this document.
7. **Model the channel behavior that the implementation actually provides.** The default buffered-channel abstraction is a bounded FIFO whose availability is evaluated at the beginning of the cycle and which does not permit first-word fall-through. If the RTL permits a value pushed into an empty FIFO to be consumed in the same cycle, or includes a bypass, debug drain, hidden credit path, or side entrance, that behavior must be represented explicitly in the selected source reference model. Back-annotated capacity alone is not sufficient when hidden structure changes enablement or transfer ordering.
8. **Use an elaborated model for coupled multi-input pipelines and other joined structures.** When acceptance on one input depends on another input, a bundle, join, shared stage, coupler, or epoch gate, do not interpret the outer capacity of each input as independent slack. The verification flow must expose the relevant staged, rendezvous, bundle, or join boundaries in the elaborated source reference model. The outer interface remains the user-visible boundary, but proof-relevant occupancy and enablement may be tracked at the introduced internal abstract boundaries.

9. **Give every dynamic reset a channel-consistent disposition.** Reset both endpoints of a live channel together with the channel state, or define an explicit flush, abort, cancellation, or end-of-epoch protocol that determines what happens to outstanding requests and in-flight data. A reset of only one endpoint cannot silently discard, retain, or reattribute a transaction across the reset boundary. The protocol must be represented in both the source model and RTL using ordinary observable actions or another explicitly verified reset-state correspondence.

Additional constraints for the internal message-passing deadlock guarantee

The Appendix K result excludes the defined reachable closed internal wait-cycle cutpoints. It therefore needs the safety constraints above plus the following structural and operational conditions. These are additional requirements for the deadlock claim; they should not be read as extra premises for every use of the Appendix J safety theorem.

1. **Give each analyzed logical channel one producer and one consumer.** A shared bus, crossbar, multi-client queue, or resource with multiple requesters should be represented by an explicit arbiter process and point-to-point channels. This makes the complementary endpoint and wait-for ownership unambiguous. A custom shared-channel abstraction requires its own ownership and arbitration proof rather than being treated as an ordinary point-to-point channel.
2. **Represent multi-lane, burst, and multi-transfer resources explicitly.** The standard scalar-channel model permits at most one Push and one Pop commit per channel direction in one cycle. A resource that transfers several elements, lanes, or beats at once must expose that behavior in its abstract interface and capacity model. It should not be encoded as several hidden transfers on a channel that the proof treats as scalar.
3. **Keep blocking requests asserted and Push payloads stable until completion.** A blocking Push or Pop is a persistent request, not a speculative try that may be withdrawn when back-pressure lasts longer than expected. Request ownership must remain with the same dynamic operation until commit or an explicit reset. A separate nonterminal cancellation/withdrawal action belongs to a different modeled protocol and is not part of the ordinary blocking wait-for interpretation used by the G–K theorem stack.
4. **Ensure that a blocked pipelined process can only drain previously accepted work.** Once a process has reached a disabled blocking frontier, later blocking work from that process must not continue to enter the system behind the blocked operation. Previously accepted pipeline work may finish and release finite downstream state, but the pipeline must not replenish that work indefinitely or expose next-interval capacity before the synchronization boundary completes. This is the practical meaning of the automatic-flush and drain-only discipline used by the deadlock proof.
5. **Do not make deadlock freedom depend on favorable overlap or same-cycle issue.** The primary practical check deliberately uses a conservative serial-issue source execution. If the design avoids deadlock only because two source-ordered blocking operations overlap, issue in the same cycle, or are arbitrated in a particularly favorable order, a clean serial execution cannot justify scheduler-robust deadlock freedom. Rewrite or buffer the protocol, add explicit synchronization, or use a different scheduler-specific proof.

6. **Check deadlock against the actual capacities and explicit boundary structure.** The relevant source model is the back-annotated bounded or elaborated model, not an idealized unbounded network and not necessarily the raw rendezvous model. A capacity-induced deadlock is real for the selected capacity configuration even if it disappears at a larger depth. Conversely, a rendezvous-only run can be a useful early screen but may report a deadlock that the intended buffered or elaborated design does not have.
7. **Account explicitly for reset during the serial comparison.** For a candidate blocked component, the primary serial-reduction route requires either a comparison continuation with no affecting reset or an already-triggered, non-waived response monitor whose exact pending operation is cleared by the matched reset. A reset that occurs before such a monitor exists, or whose scope cannot be matched to the pending operation, is outside that route and needs another deadlock argument.
8. **Keep other kinds of stalls separate from the internal message-passing claim.** Barrier participation errors, a process waiting for a peer that has terminated without an explicit end-of-stream protocol, environment-only stalls, and timed or reactive multi-frontier behavior are not automatically proved safe by the internal wait-cycle theorem. Model EOF, done, abort, and barrier protocols explicitly, and use the supported environment/frontier fragment or a separate proof when external timing can release a candidate wait cycle.

Formal references. The detailed rules are in RULE 1 and RULE 2, Direct Inputs, Scheduling of Array Accesses, Appendices B, D, I, J, K, L, and Q, and the shared channel, memory, reset, and registered-interface definitions in the Normative Formal Core. Appendix M.7a remains the definitive per-instance evidence checklist for the tool and verification flow.

Guide to the Formal Appendices

This guide is informative. It explains how the formal appendices fit together and suggests reading paths; it does not introduce new assumptions, definitions, or proof obligations. The appendix letters and theorem identifiers remain authoritative.

Recommended reading paths

HLS users and hardware architects. Read the main body through Appendix F for the scheduling contract and modeling guidance, then read Appendix M for the formal conclusions and their practical limits. Consult Appendix L when the design uses non-blocking operations or cadence-sensitive behavior. If you need the Policy-S deadlock-preservation route, read Appendix M.6 followed by Appendix K.4b, K.4c, and K.4f. These sections explain the selected serial-mode run, the bounded-response check, and the standard applicability fragment. The design-user activity is one complete instrumented Policy-S execution; the flow separately establishes simulator determinism, applicability, response coverage, and RTL cutpoint coverage. Tool, checker, and formal-proof implementers should additionally read Appendix K.4e and Appendix K-P.

Safety reviewers. Read the Normative Formal Core for Appendices G–K, then Appendix G, Appendix I, and Appendix J. Appendix R provides a compact derived companion, and Appendix T identifies the scheduling-specific obligations that remain beyond generic transition-system and simulation theory.

Liveness and deadlock reviewers. Begin with the Normative Formal Core and Appendix G, then read Appendix J for the source/RTL cutpoint connection, Appendix K for the preservation theorem, Appendix K-P for the serial-reduction proof companion, Appendix L for witness selection and environment-sensitive cases, and Appendix N for fairness and progress assumptions. Appendix Q is required when the selected source reference model uses elaborated staged, bundle, join, coupler, or epoch-gate structure.

Mechanization work. Use the Normative Formal Core as the semantic skeleton and Appendix R as its derived compact map, Appendix S for the Lean proof strategy, and Appendix T for the obligations that a faithful mechanization must preserve. The Normative Formal Core and Appendices G–K are the normative sources for the definitions and theorem interfaces; Appendix R is derived.

Appendix map

Normative Formal Core for Appendices G–K. Defines the shared reference model, operational issue/admissibility semantics, observation and frontier objects, channel/request predicates, drain discipline, KObs record, capacity conventions, reset treatment, and terminology used by the G–K theorem stack. Appendix M.7a supplies the consolidated side-condition discharge record.

Appendix G - Trace Compatibility Rules. Defines the observable trace-compatibility framework, the E1-E5 rules, the selected source-reference/RTL observation boundary, and the asymmetric Post $\rightarrow$ Ref and restricted Ref $\rightarrow$ Post uses.

Appendix H - Possible Criticisms. Collects anticipated objections, limitations, and responses. It is useful for understanding why the formal contract has its present scope and why certain stronger-looking claims are not made.

Appendix I - Per-Process Replay and Preparation Proof. Establishes the process-local replay and finite preparation machinery used by the compositional safety construction, including the treatment of deferred non-blocking decisions and registered phase behavior.

Appendix J - System-Level Trace Compatibility Proof. Composes local process, channel, atomic-interaction, environment, reset, footprint, request-snapshot, dependency, agreement, and coverage evidence into a global source witness and the system-level Post $\rightarrow$ Ref trace-compatibility result.

Appendix K - Preservation of Internal Wait-Cycle Freedom. States the certified-cutpoint deadlock-preservation result, the exact admissibility and drain bridge, the A5-L premise, and the separate certificate-coverage condition needed for a universal reachable-cutpoint conclusion.

Appendix K-P - Proofs Companion for the Serial-Reduction Route. Develops the principal Policy-S/A5-S route for discharging A5-L, including deterministic serial-source dominance and bounded-response evidence, while recording the route's environment/frontier applicability limits.

Appendix L - Snooping Introduction, Implementation and Concerns. Explains witness selection, non-blocking and latency-sensitive cases, and the conditions under which a source execution can be aligned with the RTL for restricted reverse use.

Appendix M - Formal Results Overview. Provides a readable but technically precise summary of the models, principal safety theorem, restricted reverse direction, liveness theorem, coverage requirements, and implementation-qualification obligations.

Appendix N - Scheduler and Environment Fairness Assumptions. Restates the scheduler, channel-progress, and quiescence assumptions in operational ready/valid form and explains that these are execution-environment obligations rather than automatic HLS-tool guarantees.

Appendices O and P. Appendix O gives an informative sketch for multiple clock domains. Appendix P records alternative formal frameworks and their relationship to the chosen architecture.

Appendix Q - Multiple Input Pipelines Back-Annotation. Defines the local elaboration structures and projection/accounting rules used when ordinary bounded-FIFO abstraction is insufficient for staged, bundled, joined, coupled, or sync-gated implementations.

Appendix R - Compact Derived Formal Core and Mechanization View. Provides a downstream review and mechanization map of the Core $\rightarrow I \rightarrow J \rightarrow K$ dependency chain; it introduces no normative definition or theorem interface used by earlier sections.

Appendices S and T. Appendix S records the Lean proof strategy. Appendix T identifies the scheduling-specific proof obligations that cannot be obtained merely by importing generic labeled-transition-system, simulation, or temporal-logic libraries.

Appendix U - Revision Record. Records material changes to the formal interfaces and explanatory text so that reviewers can trace the evolution of the document.

Normative Formal Core for Appendices G–K

Scope of comparison. Appendix G first presents the raw rendezvous source model Sys and the R/E scheduling rules. For the Appendices I–K proof stack, and for the Sys_ref-specific uses in Appendix G, the formal comparison is between RTL_B and Sys_ref (Definition Core.1 and Appendix J, Remark 2), i.e., the same source processes interpreted under the channel semantics of E4 with capacities $B(c)$ matching the RTL. The E4 channel semantics are case-split: $B(c)=0$ is interpreted as a capacity-zero rendezvous channel, and $B(c)>0$ is interpreted as a cycle-boundary, non-fall-through bounded FIFO. The rendezvous case is therefore recovered as the $B(c)=0$ case of the channel model, not by applying the $B(c)>0$ FIFO-availability inequalities with $B(c)=0$.

Authority of the Core. The shared definitions in this Core are normative for the G–K theorem stack. Later appendices may refine these objects and prove properties about them, but they do not redefine them. Material that merely explains the completed J/K architecture or records certificate-discharge choices is placed downstream in Appendix R or Appendix M rather than inside this definitional layer.

Symbol / Rule	Definition — concise but complete
BoundaryReq(x, σ) (shorthand BoundaryReq(q, σ) only in ordinary unique-boundary case)	For any dynamic source-level message-operation frontier item $x \in \Omega_{\text{msg},P}$ on an explicit abstract channel $c = \text{BoundaryOf_P}(x)$ of Sys_ref, with owning source-level message instance $q = \text{MsgOf_P}(x)$ and $\text{kind}(q) \in \{\text{Push_c}, \text{Pop_c}\}$, $\text{BoundaryReq}(x, \sigma)$ means: at that chosen abstract boundary and a $\text{BoundaryEvalPoint } \sigma$, q's endpoint-owned boundary request for item x is asserted. Operationally (ready/valid): · for $\text{kind}(q)=\text{Push_c}$: the producer endpoint at $\text{BoundaryOf_P}(x)$ asserts vld_c, with stable payload as required; and · for $\text{kind}(q)=\text{Pop_c}$: the consumer endpoint at $\text{BoundaryOf_P}(x)$ asserts rdy_c. BoundaryReq is therefore a predicate about the endpoint's boundary-visible request assertion for a specific dynamic operation/frontier item, not about an already-committed Σ-event label. In the ordinary unelaborated case, where q has a unique evaluated boundary and frontier item, $\text{BoundaryReq}(q, \sigma)$ is accepted as shorthand for $\text{BoundaryReq}(x, \sigma)$. In $\text{Sys_B}^{\text{elab}}$, the frontier-item form is authoritative because one source-level call q may own a chain of proof-internal frontier items at multiple explicit boundaries. Purely combinational realization structure on the channel side may affect only the complementary mirror signal at that boundary (Push: rdy; Pop: vld) and thus the commit condition after ϵ-settling, but it does not change the endpoint request assertion captured by $\text{BoundaryReq}(x, \sigma)$. When a cycle-index notation is needed, $\text{BoundaryReq}(x, t)$ is shorthand for $\text{BoundaryReq}(x, \sigma_t)$, where σ_t is the selected BoundaryEvalPoint for cycle t.
ChannelEnabled(x, σ) (shorthand ChannelEnabled(q, σ) only in ordinary unique-boundary case)	$\text{ChannelEnabled}(x, \sigma)$ means: $\text{BoundaryReq}(x, \sigma)$ holds, and at the settled $\text{BoundaryEvalPoint } \sigma$, the channel-side commit-enabling condition for the owning operation $q = \text{MsgOf_P}(x)$ holds at the chosen boundary. Concretely, for bounded-FIFO channels $B(c) > 0$, the channel-side condition is exactly the E4 availability predicate at that boundary: · for $\text{kind}(q)=\text{Push_c}$: $\text{occ_c}(\sigma) < B(c)$, i.e. space is available; and · for $\text{kind}(q)=\text{Pop_c}$: $\text{occ_c}(\sigma) > 0$, i.e. data is available. For rendezvous channels $B(c)=0$, the channel-side condition is the matching complementary endpoint participation in the same settled boundary-evaluation cycle. If the owning operation represented by x commits, it contributes the corresponding committed Σ-event $m(q) \in M$; before it commits, $m(q)$ is absent, but $\text{BoundaryReq}(x, \sigma)$ and $\text{ChannelEnabled}(x, \sigma)$ are still well-defined for the pending frontier item. In the ordinary unique-boundary case, $\text{ChannelEnabled}(q, \sigma)$ is shorthand for $\text{ChannelEnabled}(x, \sigma)$; in $\text{Sys_B}^{\text{elab}}$, the frontier-item/boundary-specific form $\text{ChannelEnabled}(x, \sigma)$ is authoritative.
ChannelDisabled(x, σ) (shorthand ChannelDisabled(q, σ) only in ordinary unique-boundary case)	$\text{ChannelDisabled}(x, \sigma)$ means: $\text{BoundaryReq}(x, \sigma)$ holds but $\text{ChannelEnabled}(x, \sigma)$ does not. Intuitively: the endpoint is requesting the transfer for dynamic operation instance q, but the transfer is blocked by channel-side conditions, such as empty/full status or the rendezvous partner not simultaneously requesting, as evaluated at the settled BoundaryEvalPoint used by the Core enablement definitions. In the ordinary unique-boundary case, $\text{ChannelDisabled}(q, \sigma)$ is shorthand; in

Symbol / Rule	Definition — concise but complete
	Sys_B ^{elab} , the frontier-item/boundary-specific form ChannelDisabled(x,σ) is authoritative.
Σ	Set of observable actions for the equivalence check (committed IO events) at the chosen observable boundary (often the DUT IO boundary for functional equivalence, but optionally expanded—e.g., for Appendix K deadlock reasoning): • channel commit events Push_c(v) and Pop_c(v) on channels designated as observable, labeled with the transferred message value (or abstract message identity) v • synchronization events wait(), Sync, START_P, FINISH_P • single-cycle signal actions Write(sig, val) and Read(sig, val) on signals designated as observable, labeled with the value written/read. Internal-only channels/signals may be excluded from Σ (treated as ϵ-microstate) when they are not part of the chosen observable boundary. If such channels/signals are later brought into the observable boundary (e.g., by snooping for debug or analysis), then they become Σ-events and must match under $\approx$. Deadlock-analysis requirement (Appendix K): for Appendix K, Σ is instantiated to include every message-passing channel whose endpoint transfers at the chosen Sys_ref boundary can contribute a dependency edge in the Appendix K Wait-For Graph, either because the channel is on the declared observable boundary or because it is snooped. The WFG itself is defined over pending blocking message-operation frontier items $x \in \Omega_msg, P$ for the relevant owning process P, with owning dynamic message-call instance $q = \text{MsgOf_P}(x)$, not over already-committed Σ-event labels and not over failed non-blocking polls in Q_exec, P. Thus, when Appendix K refers to a blocked Push or Pop at a boundary, the relevant frontier object is x, and its owning pending source-level operation instance is q with $\text{kind}(q) \in \{\text{Push_c}, \text{Pop_c}\}$; the edge is determined by BoundaryReq(x,σ) together with ChannelEnabled(x,σ)/ChannelDisabled(x,σ) at the selected Core.KCut σ (and hence at a settled BoundaryEvalPoint). Internal micro-interfaces that lie within the back-annotated realization counted in B(c) (e.g., post-HLS RTL pipeline stage implementation) are not treated as separate channels for this requirement: they may be exposed/snooped for debug, but Appendix K's WFG edges are defined using the ChannelEnabled/ChannelDisabled status of the corresponding boundary operation instance q via $E4/occ_c/B(c)$, so such internal points can participate logically in the deadlock argument without being explicitly Σ-observable. Non-blocking message-passing (PushNB/PopNB): A non-blocking call that returns “not completed” produces no Σ-event and is modeled as an internal ϵ-step. When a non-blocking call succeeds (returns “completed”), that success is represented in Σ as the corresponding committed Push_c(v) or Pop_c(v) event on that channel, labeled with the same transferred value/message v (i.e., Σ records transfers and their payloads, not failed polls). A non-blocking call is

Symbol / Rule	Definition — concise but complete
	considered issued in the cycle in which it executes (its Issue cycle). A call that returns “completed” commits in that same cycle and therefore yields a committed transfer $m(q) \in M$; a call that returns “not completed” produces no committed transfer event ($m(q)$ undefined / absent) and may be treated as an ϵ-step with respect to Σ-visible message-transfer events. Each executed non-blocking call is still a distinct dynamic source-level message-call instance $q \in Q_{\text{exec},P}$ for the owning process P. Since failed non-blocking q's have no Σ-label, their cross-trace matching is witness-relative: it is supplied by the chosen μ_Q / WC witness, by NB-CF when failed polls cannot affect the next Σ-visible control flow or frontier, or by Appendix L's snooping / witness-selection mechanism. Continuous assertion of an endpoint request does not by itself identify multiple executed non-blocking calls as one q, and the Σ trace alone is not required to determine how failed non-blocking q's are matched. A successful non-blocking call may also be a member of $Q_{\Omega,P}$ when it contributes a committed $\text{Push}_c(v) / \text{Pop}_c(v)$ event; a failed non-blocking poll remains in $Q_{\text{exec},P} \setminus \Omega_P$.
START_P	First observable action emitted by process P after reset. Marks the moment P begins executing user code. A new START_P is emitted each time P exits reset.
FINISH_P	Last observable action of process P in the current reset epoch. If P executes an unbounded loop within that reset epoch, FINISH_P never occurs for that epoch, and the epoch-local trace is treated as infinite until a later reset boundary, if any. If a later reset affecting P occurs and P exits reset again, a new dynamic START_P begins a fresh reset epoch; any later termination is represented by a distinct dynamic FINISH_P occurrence.
ϵ-step	An <i>internal</i> , unobservable RTL state transition (pipeline advance, FSM micro-state, etc.).
B(c)	Back-annotated effective capacity of channel c at the chosen explicit abstract Sys_ref boundary used by E4 and Appendix K. In the ordinary non-elaborated case this is the chosen Σ-observable Sys_B boundary; in elaborated cases it is the corresponding explicit abstract channel boundary of $\text{Sys_B}^{\text{elab}}$, which may be proof-internal and later projected away from the outer Σ_{DUT} trace. Finite, $B(c) \geq 0$. It reflects the total bounded “composite buffer” storage/credit in the RTL realization represented by that explicit abstract channel boundary, including FIFO storage, skid/elastic buffering, HLS-inserted pipeline staging that can hold in-flight messages, and any explicit staged/bundle/join channel capacity introduced by $\text{Sys_B}^{\text{elab}}$. • $B(c)=0 \Rightarrow$ rendezvous (capacity-zero) channel. • $B(c)>0 \Rightarrow$ bounded FIFO (of that effective capacity). B-faithfulness. For the formal results to apply to a concrete RTL_B instance, each back-annotated $B(c)$ must be faithful to the chosen explicit abstract boundary. This means: • Boundary completeness: every RTL storage element, credit, skid/elastic buffer, pipeline stage, or elaborated staged/bundle/join component that can

Symbol / Rule	Definition — concise but complete
	accept, hold, release, or backpressure messages across the abstract boundary c is either included in the effective capacity $B(c)$ for that boundary or represented as a separate explicit $\text{Sys_B}^{\text{elab}}$ boundary with its own capacity and E4 semantics. • No hidden capacity or hidden enablement: the RTL realization does not permit a producer-facing accept or consumer-facing delivery at boundary c except in cycles allowed by E4 using the corresponding $\text{occ_c}(t)$ and $B(c)$, and any additional enablement restriction that affects boundary behavior is represented by an explicit $\text{Sys_B}^{\text{elab}}$ boundary rather than by an unmodeled predicate. • ϵ-opacity of internal movement: movement of an already accepted message within the RTL realization counted by $B(c)$ is internal ϵ-state. Such movement does not create additional Σ-visible $\text{Push_c}/\text{Pop_c}$ events and does not create, delete, duplicate, reorder, or relabel the Σ-visible boundary transfers. • Evidence obligation: B-faithfulness is a tool/RTL-flow compliance obligation. It may be discharged by a trusted back-annotation algorithm, generated certificate, structural RTL check, interface monitor, formal equivalence/checking flow, or other validation method appropriate to the implementation flow. It is not derived by E4, Proposition J.0, or Theorem J.1. Point-to-point endpoint assumption (single-producer/single-consumer): For every message-passing channel c (including both buffered channels with $B(c)>0$ and rendezvous channels with $B(c)=0$), there is exactly one producer process that may execute Push_c on c, and exactly one consumer process that may execute Pop_c on c. Hence the complementary endpoint relevant to any blocked $\text{Push_c}/\text{Pop_c}$ on c is unique. Modeling note (shared resources / arbitration): If an implementation conceptually has multiple producers and/or multiple consumers for a logical resource, that sharing must be modeled explicitly (e.g., an arbiter process plus per-client point-to-point channels), rather than as a single multi-endpoint “channel.” This keeps WFG wait dependencies well-defined and unique per edge. Single-transfer-per-cycle endpoint constraint (scalar channel interface): For every message-passing channel c, in any clock cycle t, at most one Push_c may commit and at most one Pop_c may commit. Equivalently, the set of commits on a fixed channel in a single cycle contains ≤ 1 Push and ≤ 1 Pop (it may contain one of each in the same cycle). For rendezvous channels ($B(c)=0$), a committed transfer occurs only as a matching same-cycle $\text{Push_c}(v)$ and $\text{Pop_c}(v)$ pair. For buffered channels ($B(c)>0$), a same-cycle Push_c and Pop_c do not imply fall-through; the channel semantics are cycle-boundary, non-fall-through as stated in E4. Reset-empty FIFO convention: after each reset boundary at which the compared execution begins or restarts, every buffered message-passing channel with $B(c)>0$ has logical FIFO occupancy 0 at the chosen Σ boundary. Equivalently, the reset state contains no preloaded in-flight messages in the abstract channel realization used by $\text{Sys_ref} / \text{RTL_B}$. If a design requires

Symbol / Rule	Definition — concise but complete
	preloaded channel contents, those contents must be modeled explicitly as post-reset producer actions or by a separate initialized-state extension outside the present empty-FIFO formal model. Modeling note (multi-lane / burst transfers): If an implementation can transfer $k > 1$ messages per cycle on a logical resource, model it as k parallel point-to-point channels (or as a single widened message transferred by one Push_c/Pop_c per cycle) so that the Σ-level event model continues to satisfy the scalar constraint above.
occ_c(t)	Abstract FIFO occupancy at the chosen explicit abstract Sys_ref channel boundary: number of items committed by Push_c but not yet committed by Pop_c at that boundary, measured from the most recent reset boundary under the reset-empty FIFO convention. In the ordinary non-elaborated case this is the chosen Sys_B channel boundary; in elaborated cases Sys_ref = Sys_B^elab, this is the occupancy of the relevant explicit abstract staged/bundle/join channel boundary. Equivalently (E4), $\text{occ}_c(t) = \text{push}(\pi_t) - \text{pop}(\pi_t)$, where π_t is the Σ-visible boundary-commit prefix for that explicit abstract channel strictly before cycle t (i.e., excluding any Push_c/Pop_c commits that occur in cycle t), so $\text{occ}_c(t)$ is the abstract occupancy at the beginning of cycle t. For pure-FIFO channels with $B(c) > 0$, the FIFO availability predicates (not-full/not-empty) are evaluated against this begin-of-cycle occupancy; hence a Pop_c cannot commit in a cycle that begins empty solely because a Push_c also commits in that same cycle (no fall-through), and likewise a Push_c cannot commit in a cycle that begins full solely because a Pop_c also commits in that same cycle. Clarification (no “second Push/Pop”): movements of an item within the RTL channel realization (e.g., FIFO $\rightarrow$ staging/skid buffering, staging/skid buffering $\rightarrow$ pipeline-stage registers, pipeline-stage $\rightarrow$ pipeline-stage handoff) are internal ε-steps; they do not constitute additional Σ-visible committed Push_c/Pop_c events beyond the boundary events counted by $\text{push}(\pi_t)/\text{pop}(\pi_t)$. Derived cutpoint shorthand (used in the Core and Appendix K): if σ is a BoundaryEvalPoint occurring in cycle t_σ, write $\text{occ}_c(\sigma) \triangleq \text{occ}_c(t_\sigma)$. Likewise write $\text{ChannelEnabled}(q, \sigma) \triangleq \text{ChannelEnabled}(q, t_\sigma)$ and $\text{ChannelDisabled}(q, \sigma) \triangleq \text{ChannelDisabled}(q, t_\sigma)$ when enablement is evaluated at that settled boundary point. Every source SettlePoint and every Core.KCut used by Appendix K is a BoundaryEvalPoint. This is only notational shorthand for cutpoint reasoning; the primary semantic definition of occupancy remains the cycle-boundary quantity $\text{occ}_c(t)$. Note: $\text{occ}_c(t)$ is an abstract analysis quantity derived from Σ-visible boundary commits; in both Sys_B and RTL_B, processes do not directly read $\text{occ}_c(t)$ and instead proceed solely based on the per-channel ready/valid outcome of their own Push/Pop attempts.

Symbol / Rule	Definition — concise but complete
P_push(c)	<i>Finite-progress invariant (producer)</i>: this is a persistent-request premise, not a ChannelEnabled premise. In this paragraph, “the producer’s request remains asserted continuously” means that the producer presents a persistent, non-withdrawable Push_c endpoint request from the starting cycle through the matching discharge event, with no deassert-before-commit and with stable payload while asserted. • Buffered case ($B(c) > 0$): If the producer’s Push_c request remains asserted continuously from some cycle t_0 onward while the channel is full ($occ_c = B(c)$) —with stable payload while asserted—and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_0$ commits a complementary Pop_c (i.e., the full condition is eventually discharged). • Rendezvous case ($B(c) = 0$): If the producer’s Push_c request remains asserted continuously from some cycle t_0 onward (with stable payload while asserted) and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_0$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle.
P_pop(c)	<i>Finite-progress invariant (consumer)</i>: this is a persistent-request premise, not a ChannelEnabled premise. In this paragraph, “the consumer’s request remains asserted continuously” means that the consumer presents a persistent, non-withdrawable Pop_c endpoint request from the starting cycle through the matching discharge event, with no deassert-before-commit. • Buffered case ($B(c) > 0$): If the consumer’s Pop_c request remains asserted continuously from some cycle t_0 onward while the channel is empty ($occ_c = 0$)—and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true)—then some cycle $t' \geq t_0$ commits a complementary Push_c (i.e., the empty condition is eventually discharged). • Rendezvous case ($B(c) = 0$): If the consumer’s Pop_c request remains asserted continuously from some cycle t_0 onward and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true), then some cycle $t' \geq t_0$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle.
Trace-compatibility rules (E1–E5)	E1 Synchronization-order preservation: For each process P, the sequence of synchronization events executed by P (wait(), SyncChannel, START_P, FINISH_P) appears in the same source order in the pre-HLS and post-HLS traces. E2 Signal visibility: For each sequential process P, each signal Read in P occurs at the closest preceding synchronization event in P, and each signal Write in P occurs at the closest succeeding synchronization event in P, in both the pre-HLS and post-HLS traces (per R2/E2).

Symbol / Rule	Definition — concise but complete
	For each combinational process or explicitly modeled combinational network component C_{comb}, observable combinational signal Read/Write labels appear only inside emitted R2C fixed-point signal-observation units for the selected R2C observation component, as defined by R2C. Each emitted R2C unit is evaluated at one ϵ-quiescent combinational fixed point, and all Read/Write labels inside that unit are atomic with respect to that fixed point. Emitted R2C units are matched by component identity and local emitted-occurrence ordinal, not by global observable-cycle bucket number. R2C stutter cycles whose selected observable slice is unchanged do not create separate Σ-visible units unless an explicitly modeled observer/sampler requires a fresh emitted unit. Interpretation (logical timestamping). For sequential processes, the equations in E2 define the logical timestamps of Read/Write Σ-events: the anchor points at which compatibility is checked. An implementation may physically sample a Read later than $\text{pred}_S(r)$, or physically apply a Write earlier than $\text{succ}_S(w)$, provided the value is stable across the anchor interval and the trace emitted for compatibility records the Σ-event at the anchor point. For combinational processes, the logical observation point is the emitted R2C fixed-point signal-observation unit, not a global observable-cycle bucket number; internal delta-cycle propagation before the fixed point and R2C stutter cycles with unchanged selected observable slices are $\epsilon/\text{stutter}$ and are not separately Σ-visible. Direct-input pragmas (e.g., <code>#pragma hls_direct_input</code> and <code>#pragma hls_direct_input_sync</code>) do not change E2 and are not exceptions to E2. For sequential processes, they impose stability/timing contracts that allow later physical sampling, or controlled updates at a synchronization boundary, while the logical Read/Write Σ-events used for $\approx$ checking remain timestamped at the E2 synchronization anchor points and retain the anchored value labels. For combinational processes, ordinary direct-input pragmas do not introduce a separate anchoring rule; any observable combinational signal Read/Write label remains part of an emitted R2C fixed-point signal-observation unit under the R2C unit-generation and matching conventions. Thus the pragma affects only implementation sampling freedom under its stated contract, not the logical Σ observation used for compatibility. E3 (Safe message issue ordering). (Post-HLS preservation of R4). For any two executed message-call instances $q_1, q_2 \in Q_{exec}, P$ of the same process P: (i) Same-interface order is always preserved: if q_1 and q_2 access the same message-passing interface/channel and $q_1 <_{src} q_2$, then $\text{issue_post}(q_1) \leq \text{issue_post}(q_2)$. (ii) Distinct-interface no-reverse: if q_1 and q_2 access distinct message-passing interfaces and appear in sequence in the source ($q_1 <_{src} q_2$), then $\text{issue_post}(q_1) \leq \text{issue_post}(q_2)$ (i.e., they shall not be issued in the reverse sequence). (iii) Prefix-closed issue obligation: at every cycle-aligned cutpoint σ of the post-

Symbol / Rule	Definition — concise but complete
	HLS execution / witness, if $q1 <_{src} q2$ and $q2$ has issued by σ, then $q1$ has also issued by σ and $Issue(q1) \leq Issue(q2)$. This is an implementation-side obligation on issued sets, not merely an inequality over pairs whose timestamps are already defined. (Note: any pipelined-loop internal staging/dequeue discussed in “Pipelined Loops” is ϵ-microstate within the back-annotated channel realization and is not a Σ-visible boundary issue event associated with any $q \in Q_{exec,P}$ nor a committed-transfer event in M; therefore it does not constitute a counterexample to (ii) or (iii).) E4 FIFO legality: for each channel c, the post-HLS committed ($Push_c(v)$, $Pop_c(v)$) history is a legal bounded-capacity execution consistent with Sys_B for that channel (capacity $B(c)$), and reduces to rendezvous consistency when $B(c)=0$. In particular, on each channel, the committed $Pop_c(v)$ events return exactly the FIFO-ordered sequence of values v previously committed by $Push_c(v)$, with no drops, duplications, or reordering. E5 (Messages cannot cross their immediate synchronization boundary). For every synchronization call s (in the source order used by $R3$ / $pref_S$ / $suff_S$): $\forall q \in pref_S(s) : issue_post(q) \leq clk_post(s)$ $\forall q \in suff_S(s) : issue_post(q) > clk_post(s)$ For every blocking $q \in pref_S(s)$: $m(q)$ is defined and $clk_post(m(q)) \leq clk_post(s)$.

Frontier-item predicate convention. $BoundaryReq$, $ChannelEnabled$, and $ChannelDisabled$ are authoritative in their frontier-item form. For a message-operation frontier item $x \in \Omega_{msg,P}$, let $q = MsgOf_P(x)$ and $c = BoundaryOf_P(x)$. $BoundaryReq(x,\sigma)$, $ChannelEnabled(x,\sigma)$, and $ChannelDisabled(x,\sigma)$ are evaluated at that specific explicit abstract boundary and are undefined for non-message frontier items unless the statement explicitly restricts $x \in \Omega_{msg,P}$. In the ordinary unelaborated case, where q has a unique frontier item and a unique evaluated boundary, the q -forms $BoundaryReq(q,\sigma)$, $ChannelEnabled(q,\sigma)$, and $ChannelDisabled(q,\sigma)$ may be used as shorthands for the corresponding x -forms. In Sys_B^{elab} , one source-level call q may own several frontier items at different explicit boundaries, so the x -form is authoritative.

No hidden cross-channel disablement. Any cross-channel dependency introduced by combinational couplers must be reflected only at explicit internal staged/bundle/join boundaries introduced by the elaboration, i.e., via $BoundaryReq/ChannelEnabled/ChannelDisabled$ on message-operation frontier items $x \in \Omega_{msg,P}$, with $MsgOf_P(x)=q$, at those explicit staged/bundle channels. At the chosen Σ -visible channel endpoints, $BoundaryReq(x,\sigma)$ and $ChannelEnabled(x,\sigma)/ChannelDisabled(x,\sigma)$ for a message-operation frontier item $x \in \Omega_{msg,P}$ on channel c may not depend on other channels' state beyond the per-channel E4 predicate for c , and no extra hidden predicate may block an otherwise E4-enabled boundary transfer.

Prescribed reference model

Definition Core.1 (Prescribed source reference model Sys_ref).

For a fixed design, fixed external stimulus, and selected elaboration package, define the prescribed source reference model Sys_ref as follows: • $\text{Sys_ref} = \text{Sys_B}$ in the ordinary abstract back-annotated presentation using the case-split channel semantics of E4: $B(c)=0$ is rendezvous, and $B(c)>0$ is cycle-boundary non-fall-through bounded FIFO. • $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$ in the elaborated/back-annotated case when explicit staged, bundle, join, guard, coupler, or epoch-gate boundaries are required. All source-side semantic notions in Appendices G–K are interpreted with respect to this prescribed source reference model. Appendix J, Remark 2 explains the model ladder and practical back-annotation interpretation but does not redefine Sys_ref .

Definition Core.ElabProj (Elaboration projection/accounting map Π_{elab}).

When $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$, fix the projection/accounting map Π_{elab} supplied by the selected elaboration package E. Π_{elab} maps proof-internal executions, committed events, and explicit-channel occupancy state of $\text{Sys_B}^{\text{elab}}$ to the chosen outer DUT/testbench-visible boundary view. It has four roles:

- (1) Trace projection. Π_{elab} preserves the labels and order of the selected outer Σ_{DUT} -visible boundary events and hides or projects away introduced staged/bundle/join events that are not part of that outer observation boundary.
- (2) Occupancy projection. For every selected outer abstract channel c , Π_{elab} maps the occupancies of the explicit internal channels in the elaboration to the outer occupancy $\text{occ}_c(\sigma)$ used by the outer E4 interpretation.
- (3) Conservation/accounting. Internal ϵ -steps and committed transfers across introduced proof-internal channels preserve the projected outer history and occupancy except when E designates a transfer as the corresponding outer-boundary commit. Each projected outer $\text{Push}_c/\text{Pop}_c$ commit updates projected occupancy exactly as required by E4; bundled or joined items use the accounting declared by E.
- (4) WFG visibility. Projection does not make a proof-internal blocking boundary invisible. Every staged/bundle/join/guard/epoch-gate boundary used for BoundaryReq , $\text{ChannelEnabled}/\text{ChannelDisabled}$, Pending_P , Front_P , or WFG reasoning remains an explicit abstract channel of $\text{Sys_B}^{\text{elab}}$ even when its committed events are projected away from the outer Σ_{DUT} trace. Appendix Q supplies the permitted elaboration constructions and the detailed well-formedness, projection-fiber, join-boundary, and locality obligations that instantiate this Core interface.

Remark Core.1 (Interpretation at explicit abstract channels).

Whenever $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$, the quantities $B(\cdot)$, $\text{occ}_\cdot(t)$, E4, BoundaryReq , ChannelEnabled , ChannelDisabled , and the Appendix K blocking / wait-for-graph machinery are interpreted per explicit abstract channel, including any staged, bundle, or join channels introduced by elaboration. Any conjunctive/join dependency shall therefore be represented only via such explicit abstract channels, not as cross-channel disablement at the outer Σ -visible endpoints.

Remark Core.1a (Operational Sys_ref transition system and issue policies).

Sys_ref is the selected source-reference transition system defined operationally in this Core. Appendix G constrains that system through R1–R5 and E1–E5. Its states contain process-local control/variable state, reached/issued/committed dynamic message-call instances, explicit abstract channel state for every Sys_B or $\text{Sys_B}^{\text{elab}}$ boundary, operation-attributive pending endpoint requests, synchronization/signal-anchor state, and ϵ -microstate. Its steps are pure source-evaluation ϵ -steps, message issue steps, E4 channel-commit steps, synchronization/signal-anchor steps, and admissible ϵ -settling/drain steps. Core.Phase is part of the legality of this transition system: after the selected L1

window closes, L1-only process evaluation and issue steps are phase-disabled until the next phase permits them. This phase restriction may be represented by an explicit phase tag or by a phase-indexed transition relation; it is not inferred merely from which enabled step a scheduler happened to select. The phase-advance markers that close L1 and select L2, advance L2 to L3, advance L3 to L4, and begin the next cycle are meta-level control transitions of the cycle semantics: they are neither ϵ -steps nor Σ -events and do not themselves change the proof-visible state or history; they only select which phase-indexed operational subrelation is active. State changes associated with L3 boundary actions and L4 returned-result/registered updates occur through the operational actions already named by Core.Phase, not through the phase-advance marker itself. Policy L and Policy S are two issue policies over this same transition system. Policy L permits source-later blocking requests to be issued before source-earlier same-interval blockers process-facing commit, subject to source-order prefix closure and all R/E/E4/E5 obligations. Policy S adds the conservative serial-blocking restriction: a later source-order blocking Push/Pop may issue only after every earlier source-order blocking Push/Pop in that interval has process-facing committed in a strictly earlier cycle. In Sys_B^{elab}, process-facing commit is the commit that releases the owning source-level operation at its selected boundary, not an arbitrary internal staged/bundle/join transfer. The authoritative definitions are Definition Core.LLegal for Policy-L step legality and fixed-instance binding, Definition Core.Phase for the L0–L4 cycle phases and pre-settle/settled-frontier boundary, and Definition Core.LAdm for finite-prefix admissibility and complete admissible executions; this remark introduces no alternate legality, phase, or admissibility relation.

Observable alphabet and executions

Definition Core.2 (Observable alphabet Σ).

Let Σ be the observable event alphabet consisting of:

- (1) committed message-transfer events Push_c(v) and Pop_c(v),
- (2) synchronization events selected into S, and
- (3) signal events Read(sig, v) and Write(sig, v), interpreted as anchored signal events for sequential processes under R2 and as ϵ -fixed-point cycle-bucket events for combinational processes under R2C. Under the pipelined-loop body no-wait condition, an HLS-pipelined loop body contributes no additional in-body wait/synchronization events to S or Σ .

Definition Core.3 (ϵ -steps and compact trace notation).

Let ϵ denote internal micro-steps that are not observable at the proof-internal alphabet Σ , including internal staging/hand-off within the concrete realization of a single explicit abstract channel, late direct-input sampling between anchors, and failed non-blocking polls. In elaborated cases Sys_ref = Sys_B^{elab}, transfers across introduced staged/bundle/join channels are not ϵ merely because Π_{elab} hides them from the outer DUT/testbench-visible alphabet Σ_{DUT} ; if such a channel is an explicit abstract boundary used by Appendix K, its committed Push/Pop transfers are Σ -events for the proof-internal comparison.

A finite execution prefix τ is understood in the same sense as in Appendix J: it carries the inherited cycle partition / per-cycle observable buckets of the execution, together with any ϵ -steps between successive Σ -visible cutpoints. For compact notation one may write a linearization over $(\Sigma \cup \{\epsilon\})$, but only modulo permutation of Σ -events that occur in the same cycle and are not additionally ordered by the document's explicit Σ -visible constraints. Accordingly, $\tau \upharpoonright \Sigma$ denotes the observable projection together with that inherited cycle partition; same-cycle observable events are not assigned any extra order merely by this compact notation.

Definition Core.4 (ϵ -quiescent cutpoint).

For any proof model M in this formal stack with a declared Σ/ϵ transition relation (including Sys_ref , Sys_K , RTL_B , and explicit lowered/elaborated variants), write $\text{Quiescent_M}(s)$ when no ϵ -step of M 's full proof transition relation is enabled from state s at its current cycle/phase. For any finite prefix τ of M , $\text{Norm_M}(\tau, v, s)$ means that v is a finite ϵ -only extension of τ ending at s , s satisfies Quiescent_M , and v preserves the inherited Σ -visible history and cycle-bucket structure of τ . A selected maximal finite normalization may still be written $\bar{\tau}$. Core.4 quiescence excludes enabled ϵ -work only; it does not exclude enabled Σ -actions and is therefore weaker than a B5/global fixed point. For $\text{Sys_ref}/\text{Sys_K}$, Core.Phase is part of the full transition legality used by Quiescent_M , and the meta-level phase-advance markers of Remark Core.1a are not ϵ -steps. RTL_B has its own cycle/ ϵ relation and is not assigned the source-side L0–L4 phases. Appendix J/K comparison cutpoints use the model-specific Quiescent predicate through Core.KCut below.

Definition Core.Settle (L2 settled boundary-evaluation point).

For a Policy-L or Policy-S cycle t , after the selected L1 process-owned pre-settle window closes, let $\rightarrow_{\epsilon, L2}$ denote the complete phase-L2 ϵ subrelation: process-owned L1 source-evaluation and message-issue steps are disabled; process-owned request assertions and Push payloads that were fixed at the L1 endpoint remain fixed; and the cycle- t exogenous/environment-driven values treated as inputs to the selected G–K system remain fixed throughout L2. Every ϵ -step that remains phase-legal at L2 is a combinational, explicit-boundary-evaluation, or other cycle-local L2 settling step in $\rightarrow_{\epsilon, L2}$.

L2 frame (normative). Every $\rightarrow_{\epsilon, L2}$ step preserves the cycle index; registered process/FSM/pipeline state; registered explicit-channel storage, occupancy, and credit state; and the selected cycle- t exogenous/environment-driven input values. It emits no Σ -visible commit and performs no L4 or other next-cycle registered update. It may modify only cycle-local ϵ /settling microstate and derived combinational boundary values such as complementary ready/valid, guard, join/coupler, and $\text{ChannelEnabled}/\text{ChannelDisabled}$ evaluation results. A same-cycle reactive component whose value is intentionally combinational dependent on DUT outputs is not a changing exogenous input under this clause: it must be included inside the same qualified settling network or isolated behind an explicitly verified boundary.

$\text{SettlePoint_I}(t, \sigma)$ holds when σ is the first fixed point of $\rightarrow_{\epsilon, L2}$ reached from that cycle's selected L1 endpoint. This definition is conditional on such a fixed point being reached; it does not by itself prove termination or existence. Define $\text{BoundaryEvalPoint_I}(\sigma)$ to mean a state at which the explicit boundary request and $\text{ChannelEnabled}/\text{ChannelDisabled}$ network has reached the applicable combinational fixed point; every SettlePoint is a BoundaryEvalPoint . SettlePoint is the L2 activation/evaluation notion used by Core.16 before L3 boundary actions; Lemma Core.SettleIsKC below records its relationship to Core.4 quiescence for the phase-bearing source models. The standard existence/uniqueness result is supplied by Definition Core.QSync and Theorem Core.QSync-Ref below.

Definition Core.KCut (K-facing normalized cutpoint domain).

Fix a theorem instance I and the explicit abstract boundaries/elaboration applicable to the model being evaluated. For any proof model M used by Appendix K, $\text{KC}_M(I)(s)$ holds when s is reachable, $\text{Quiescent_M}(s)$ holds, and every explicit boundary used for $\text{BoundaryReq}/\text{ChannelEnabled}/\text{ChannelDisabled}$ has reached its selected combinational fixed point. Write $\text{KC}_M(I)$ for the reachable domain. For the prescribed source reference model use the aliases KC_ref, I and $\text{KC}_M(I)$; for RTL_B use KC_post, I and $\text{KC}_M(I)$. In Appendices J–K, “K-relevant cutpoint” means a member of the appropriate $\text{KC}_M(I)$ domain. For a reachable

RTL_B prefix τ_post , $NormPost_I(\tau_post, v_post, p_post)$ abbreviates $Norm_RTL_B(\tau_post, v_post, p_post) \wedge KCut_post, I(p_post)$; the analogous prescribed-source normalization is $NormRef_I$. By Lemma [Core.SettleIsKCut](#), every source-side Sys_ref/Sys_K L2 [SettlePoint](#) is itself a K-facing cutpoint. Existence of the source [SettlePoint](#) is supplied by the qualified synchronous-settling theorem [Core.QSync-Ref](#) in the standard theorem scope. RTL_B has no L0–L4 phases; its [KCut](#) existence remains expressed through [Core.KCutClosure_post](#), which Theorem [Core.QSync-Post](#) derives from the same qualified synchronous-model discipline in the standard scope.

Lemma [Core.SettleIsKCut](#) (source L2 settle implies K-facing cutpoint).

For each phase-bearing source proof model $M \in \{Sys_ref, Sys_K\}$ and its explicit lowered/elaborated variants, $SettlePoint_I(t, \sigma)$ implies $KCut_M, I(\sigma)$. Proof. The [SettlePoint](#) is reachable from the selected L1 endpoint. At L2, all L1-only ϵ /evaluation/issue steps are phase-disabled; by Remark [Core.1a](#) the phase-advance marker is not an ϵ -step; and by Definition [Core.Settle](#), $\rightarrow_{\epsilon, L2}$ contains every ϵ -step that remains phase-legal at L2 and has reached its fixed point. Hence $Quiescent_M(\sigma)$. Definition [Core.Settle](#) also gives [BoundaryEvalPoint_I\(\sigma\)](#), so the remaining [Core.KCut](#) condition holds. $\square$

Hardware interpretation of the qualified synchronous model (informative). At the architectural level, the intended pre-HLS and post-HLS comparison models are ordinary single-clock, cycle-level synchronous hardware systems. At the beginning of cycle t , registered process, pipeline, and explicit-channel state is fixed. Sequential processes determine their cycle- t request, payload, and other process-owned outputs from that registered state and from operands fixed for the cycle; they do not provide a same-cycle combinational through-path from a value newly returned or sampled at an interface in cycle t to a dependent process-owned output in that same cycle. Ordinary combinational derivation of outputs from registered state is permitted. The complete system-level same-cycle network—including R2C combinational processes, ready/valid and enablement logic, and stateless elaboration logic—then settles before the boundary action to one deterministic valuation. Pure combinational feedback is outside the standard theorem scope; feedback that affects later behavior passes through explicit registered or finite-capacity state. Boundary actions are selected from the settled valuation, after which the next registered state is formed.

Relation to the proof vocabulary. Definition [Core.QSync](#) formalizes this hardware discipline rather than introducing an additional hardware mechanism. On the phase-bearing source side, [Core.Phase](#) presents the cycle as L1 reach/issue, L2 settle, L3 boundary commit/decision, and L4 registered update; RTL_B need not expose those proof phases. [SettlePoint](#) and [BoundaryEvalPoint](#) name the ordinary settled source-cycle snapshot, and [Core.KCut](#) names that same kind of settled snapshot when it is used by the J–K proof stack. These are proof views/cutpoints of the cycle-level machine, not additional designer-visible hardware states or extra physical timing phases. HLS-added silent cycles remain real clock cycles and are hidden from the selected observable behavior except where the contract declares timing significant. WC/latency-sensitive choices, Policy-L issue latitude, arbitration, and other explicitly modeled choices remain available, so this interpretation does not assert that Sys_ref has one unique execution.

Definition [Core.QSync](#) (qualified synchronous same-cycle settling model).

Fix a theorem instance I and a proof model M used by the G–K stack. For a phase-bearing source model, the model-specific same-cycle settling relation is $\rightarrow_{\epsilon, L2}$ from Definition [Core.Settle](#). For RTL_B, it is the complete K-facing same-cycle ϵ /combinational settling relation beginning at a reachable K-origin after the current cycle’s registered/FSM/pipeline state and process-owned registered request outputs have been established and ending before any cycle-advancing registered update.

[QualifiedSynchronousModel_M\(I\)](#) holds when all of the following are true.

(QS1) Single-clock scope. There is one selected global clock/cycle relation shared by every process and explicit abstract channel inside the G–K theorem instance. The cycle indices used by K-origins, boundary evaluation, commits, and registered updates are all defined relative to that clock, and no unmodeled cross-clock-domain boundary lies inside the selected instance. The Appendix M.7a ClockScope entry records the concrete evidence for this condition.

(QS2) Registered-phase/frame separation. The same-cycle settling relation satisfies the L2/K-origin frame: the cycle index, registered process/FSM/pipeline state, registered explicit-channel storage/occupancy/credit state, and selected cycle-t exogenous inputs are invariant during settling; no Σ -visible commit and no next-cycle registered update occurs in the settling relation. Sequential process request/payload drives covered by Core.RegSeq remain fixed after their selected pre-settle value is established. R2C combinational-process outputs, and stateless elaboration logic such as couplers, are instead part of the settling network.

(QS3) Settling-network realization and deterministic local evaluation. The complete same-cycle settling relation is realized by the qualified settling network: every state component that may change during settling is represented by a cycle-local settling node or by a derived value of such nodes; each settling step changes only such represented values by evaluating one or more node functions from their predecessor values; every phase-legal same-cycle settling step is accounted for by this representation; and no additional settling transition exists outside it. A state is a fixed point of the settling relation iff all represented node equations are satisfied. Every combinational or other cycle-local settling node has a deterministic value for fixed predecessor values. Any state that can affect a later cycle is represented at an explicit registered/process/channel state boundary rather than being hidden inside the settling relation.

(QS4) Finite acyclic settling dependency. After cutting the network at those explicit registered/storage boundaries, the dependency graph for the complete same-cycle settling network is finite and acyclic. Pure combinational feedback is outside the standard theorem scope; required feedback shall pass through explicit state. An equivalent independently checked structural theorem establishing the same finite acyclic dependency property may be used.

(QS5) Explicit-boundary completeness. Every value used by BoundaryReq, ChannelEnabled/ChannelDisabled, rendezvous pairing, join/coupler/guard/epoch-gate evaluation, or Appendix K WFG construction at a selected boundary is produced by that qualified settling network from the fixed cycle-t state/inputs. No hidden enablement or storage predicate may bypass E4/Core.ElabProj/B-faithfulness. For $\text{Sys_B}^{\text{elab}}(E)$, Appendix Q supplies the elaboration-specific obligations; for RTL_B , the selected boundary mapping and B-faithfulness evidence supply the implementation side.

QualifiedSynchronousModel is a same-cycle structural property. It does not assert one unique Sys_ref execution: latency-sensitive/WC choices, Policy-L issue latitude, arbitration/environment choices across cycles, and other explicitly modeled choices may still admit multiple source executions. Nor does it imply B1; B1 remains the separate observable RTL trace-determinism assumption.

Theorem Core.QSync-Ref (qualified source/elaboration settling gives a unique source KCut).

For $M \in \{\text{Sys_ref}, \text{Sys_K}\}$ and its explicit lowered/elaborated variants, assume QualifiedSynchronousModel_M(I). Then every reachable source K-origin has exactly one L2 fixed-point endpoint σ_{set} in that cycle. That endpoint satisfies $\text{SettlePoint_I}(t, \sigma_{\text{set}})$; by Lemma Core.SettleIsKCUT

it lies in $KCutReach_M(I)$. Hence $KCutClosure_M(I)$ holds, and the K-facing portion of $NormRef$ may select this unique settled endpoint canonically.

Proof sketch. QS2 freezes registered state and cycle-t inputs and excludes $\Sigma/L4$ updates during settling. QS3 realizes the complete settling relation as evaluation of the represented cycle-local nodes, makes each node functional, and identifies relation fixed points with satisfaction of all node equations. QS4 gives a finite acyclic dependency order, so evaluation terminates and produces one order-independent valuation. QS5 makes that valuation complete for every K-facing boundary predicate. Therefore the complete $\rightarrow \epsilon, L2$ relation reaches one fixed point; Lemma $Core.SettleIsKCut$ supplies the $KCut$ conclusion. In the elaborated case, Appendix Q supplies or cites the settling facts for the elaboration-owned template subnetwork. Those template-side facts are not by themselves the whole-source QS2–QS5 discharge: the concrete theorem instance must also establish the applicable design-owned source-settling facts and a checked template/design settling-composition condition, including QS4 for the complete combined same-cycle settling network. Definition $Core.Settle$ alone is not used as an existence axiom.

Theorem $Core.QSync\text{-}Post$ (qualified RTL settling gives a unique post $KCut$ and $KCut$ closure).

Assume $QualifiedSynchronousModel_RTL_B(I)$. Then every reachable RTL K-origin has exactly one same-cycle settled endpoint p_post before the next cycle-advancing registered update. There is a finite ϵ /combinational normalization from that origin to p_post , $p_post \in KCutReach_post(I)$, and therefore $Core.KCutClosure_post(I)$ holds. The theorem also supplies a canonical K-facing $NormPost$ normalization/endpoint for that origin. With $K\epsilon$, Lemma $Core.KCutObserve$ then gives the existing persistent-blocker observability consequence.

This canonicalization is limited to K-facing same-cycle RTL settling. It does not eliminate Appendix I/J $ReplayNormalizationConvention$ or the selected source replay-normalization/congruence machinery, which may include witness-fixed replay-changing ϵ behavior and preparation steps unrelated to combinational settling. In particular, $Core.QSync\text{-}Post$ does not prove that a later clock cycle exists, that a temporarily idle RTL state advances to the next registered state, or that a cross-cycle RTL execution satisfies B2/B3. The finite-prefix Appendix J correspondence and Appendix K RTL cutpoint-closure results do not require a generic RTL fair-extension premise; any later theorem that needs complete RTL continuation must state its own continuation/progress assumptions.

Definition $Core.CycleResult$, $CompleteCycle$, $SilentCycle$, and terminal state.

For a proof model M and theorem instance I , $Core.CycleState_M, I$ is the full cycle-entry theorem state: registered/internal machine state, time-aware environment state, global cycle index, and every cycle-valued auxiliary or monitor component declared by I . $CycleResult_M, I(s_t, r)$ is a typed legal result $r = \langle nextState, committedUnits, resetUnits \rangle$. The $nextState$ field is a $Core.CycleState_M, I$ at cycle $t+1$, so the global clock advances exactly once; its environment component is related to the cycle- t environment by $EnvStep_I$ using the applicable canonical Σ -causal history, and its auxiliary/monitor components advance according to their definitions. In particular, Appendix K's $K.Resp$ age is part of the cycle-valued auxiliary state and advances in $nextState$ on each selected $Policy\text{-}S$ cycle while its monitor remains pending. $committedUnits$ is the finite collection of selected ordinary complete observable units / Σ commits produced in cycle t . $resetUnits$ is the finite collection of dynamic proof-internal $RESET(S_p, S_c)$ occurrences in that cycle; each occurrence carries its reset scope and corresponds exactly to the state-changing reset transition governed by RW1–RW5. For a phase-bearing source model, a legal result is witnessed by a finite L1 preparation/issue segment, finite L2 settling, the selected legal L3 boundary actions/decisions and reset boundary units, and the L4 registered/result update. For RTL_B it is

witnessed by the model-specific real clock transition through its K-facing same-cycle settling/boundary behavior and next registered update; RTL_B is not assigned source L0-L4 phases.

$\text{CompleteCycle_M,l}(s_t, s_{t+1})$ holds exactly when there exists r with $\text{CycleResult_M,l}(s_t, r)$ and $r.\text{nextState} = s_{t+1}$. Thus CompleteCycle remains the next-state projection used by existing theorem statements; CycleResult is the primary typed object carrying the full cycle evidence.

$\text{SilentCycle_M,l}(s_t, s_{t+1})$ holds when such a result exists with no selected Σ event in $r.\text{committedUnits}$. A SilentCycle is a real cycle, not an ϵ -step and not an identity transition: registered/internal machine state, environment state, and cycle-valued monitor state may change. A reset-only cycle may be SilentCycle with respect to the selected outer Σ alphabet when RESET is proof-internal/projected away; that does not make the cycle event-free, because the corresponding occurrence remains in $r.\text{resetUnits}$ and in the proof-internal history. RESET is therefore not an alternative cycle outcome: a partial reset changes only its named scope while the unaffected state still proceeds to the same result's nextState under RW5. When the fixed semantics permits more than one reset occurrence in one cycle, resetUnits retains each dynamic occurrence and only the order required by RW1-RW5 and $\prec_J$ is imposed; independent disjoint resets are not artificially serialized. Lemma J.RS continues to consume the emitted RESET unit itself, not the enclosing cycle. Persistent blocking requests follow Axiom Core.IC: once issued, an outstanding blocking dynamic instance remains asserted and attributed until its process-facing commit or an affecting RESET governed by RW2; a DeclaredTerminal state may instead end the execution while the request remains pending. No other nonterminal transition in the G-K theorem model may deassert, retire, reattribute, cancel, or replace that outstanding blocking instance. Protocols with independent nonterminal cancellation, withdrawal, or retry require a separately defined operational extension and are outside this theorem stack. An activated Core.16 no-new-later-work restriction remains in force while its fully disabled frontier persists.

$\text{DeclaredTerminal_M,l}(s)$ remains a predicate on the full Core.CycleState , not a field or alternative constructor of CycleResult . It means that the selected machine/test contract explicitly declares the execution terminated, aborted, or at end-of-test so no ordinary CycleResult / CompleteCycle successor is required. Only a disposition modeled as part of the selected theorem instance's machine/test semantics counts as DeclaredTerminal . An external verification-harness watchdog that merely stops host simulation or reports a timeout, without being a modeled transition or termination disposition of M , is not DeclaredTerminal and does not truncate Core.CycleResult or $\text{Core.CompleteCycle}$. A modeled cycle-count, watchdog, abort, or end-of-test rule that is part of the selected machine/test semantics does count. Mere absence of a currently enabled Σ -action or ϵ -step is not DeclaredTerminal when the time-aware environment or registered machine can evolve on a later cycle. $\text{TerminalIdle_M,l}(s, H, t, e)$ is the stronger B6 eligibility condition: s is a genuine B5/global fixed point and every future machine/environment evolution permitted by the fixed theorem instance preserves the no-future-enabling condition, so no registered/internal update and no environment step can change non-clock state or re-enable work. Core.EnvTerminal below supplies the environment component of this obligation; the machine-side permanent-idle fact must also be established. A stable DeclaredTerminal state is one sufficient machine-side route, but B6 may also apply to a nonterminating service model proved permanently idle after stimulus exhaustion.

Definition Core.EnvStep and Core.EnvTerminal (time-aware environment evolution).

Let $\text{EnvStep_l}(H, t, e, e')$ be the fixed theorem-instance environment transition from environment state e at cycle t and canonical Σ -causal history H to the next-cycle environment state e' . Every legal CycleResult uses this relation for the environment component of nextState . The requirement that a compatible next-cycle environment evolution exist at every reachable nonterminal source cycle-entry state is folded into Core.CycleClosure below rather than exposed as a separate environment-totality premise. The cycle/environment state is part of the relation: identical Σ history at two different cycles need not imply

identical future availability. $\text{Core.EnvTerminal_I}(H,t,e)$ is the environment component of Core.TerminalIdle : every future environment evolution allowed by the fixed instance preserves the no-future-enabling condition required for an indefinite B6 idle suffix. StandardEnv stimulus exhaustion is one sufficient environment-side discharge route. A timed environment may satisfy Core.CycleClosure 's environment-definedness clause while delivering a new input later and therefore fail EnvTerminal at the earlier idle cycle.

Definition Core.CycleClosure (source full-cycle qualification record).

For a selected source theorem instance I , $\text{Core.CycleClosure_I}$ is the normative full-cycle qualification record used whenever a theorem constructs a complete/fair source continuation, including Lemma K.2b.E. It packages the following obligations. (C1) Full-state cycle typing: every legal CycleResult has a nextState with cycle index advanced exactly once, environment state related by EnvStep_I , every declared cycle-valued auxiliary/monitor component advanced according to its semantics, and committedUnits/resetUnits consistent with the selected source transition semantics. Each resetUnits occurrence corresponds exactly to one scoped state-changing RESET transition and satisfies RW1-RW5; no reset is represented as an alternative to the enclosing cycle result. (C2) Nonterminal source-cycle existence, finite phase execution, and partial-cycle extension: every reachable nonterminal source cycle-entry state admits at least one legal CycleResult / CompleteCycle successor; its L1 part reaches a finite preparation/issue endpoint under $\text{Core.ReachPrep/Core.Phase}$ and its L2 part reaches the required settled endpoint before L3/L4. In addition, every finite phase-legal partial source-cycle segment selected from the operational transition relation—respecting $\text{Core.Phase/Core.16}$, the fixed reset contract, and an EnvStep -compatible environment choice—admits a legal completion to a CycleResult whose phase projection contains that selected segment in the chosen order. This extension clause is what permits a later proof to choose a finite fair-dovetail set of issue/boundary actions and still obtain a typed complete cycle; bare successor existence is insufficient. The standard discharge combines finite source-control/FPD evidence for L1 with Core.QSync-Ref for L2 and an operational cycle-completion theorem, or an allowed equivalent source-cycle theorem. (C3) Environment definedness: at every reachable nonterminal source cycle-entry state, the fixed time-aware environment supplies at least one EnvStep compatible with the fixed stimulus/history and with some legal CycleResult . The former standalone source-cycle-closure and environment-totality obligations are exactly the source-cycle-existence/finiteness and environment-definedness clauses of this record and are no longer independent premises. Core.CycleClosure is an existence/typing qualification, not a fairness premise: Core.IssueFair and B2/B3/B5 remain separate where a complete/fair continuation is required. No generic RTL cross-cycle totality/fair-extension premise is part of Core.CycleClosure : finite-prefix Appendix J uses Reach_post , while the universal Appendix K RTL cutpoint result uses $\text{Core.QSync-Post/Core.KCutClosure_post}$. Any infinite B6 suffix is governed separately by Core.TerminalIdle and B6.

Lemma $\text{Core.CycleChoiceExt}$ (phase-legal source-cycle selection extends to a CycleResult).

Assume $\text{Core.CycleClosure_I}$. Let s be a reachable nonterminal source cycle-entry state and let α be a finite phase-legal partial source-cycle segment from s whose steps are operationally legal in sequence, satisfy $\text{Core.Phase/Core.16}$, respect the selected reset contract, and are compatible with a selected EnvStep_I environment evolution. Then clause C2 supplies a legal CycleResult r from s whose phase projection extends α and completes the remaining L2/L3/L4 work required by the source semantics. Consequently, a later proof may realize a finite fair-dovetail selection inside each cycle and invoke dependent choice only after obtaining these typed per-cycle successors. This lemma does not supply fairness: Core.IssueFair and B2/B3 remain separate obligations. $\square$

Remark Core.Settle-a (δ -settling, silent τ -cycles, and B6 idle ticks).

The following names separate three phenomena that are easy to conflate but are different objects in the formal model.

- δ -settling is the cycle-local same-cycle settling relation. On the phase-bearing source side, a δ_{ref} step is exactly a step of $\rightarrow_{\epsilon, L2}$; on RTL_B in the standard qualified synchronous scope, δ_{post} denotes the same-cycle settling relation of Definition Core.QSync. By the L2/QSync frame, δ -steps preserve the cycle index and registered state. The converse is not intended: other ϵ -steps, including L1 source-evaluation activity, may also preserve the cycle index but are not δ -settling steps. δ is therefore a settling classification, not a renaming of all ϵ .
- A silent τ -cycle is the explanatory name for a Core.SilentCycle: a real CompleteCycle in which no selected Σ event commits. Hidden registered/internal progress may occur, but need not; a completely idle-looking yet nonterminal cycle is still a SilentCycle if the clock/environment advances. Silent cycles carry the CompleteCycle state updates described above, including time-aware environment evolution and any active cycle-valued monitor aging. They are neither ϵ -steps nor identity stutter.
- A B6 idle tick is the special real clock tick used only when Core.TerminalIdle holds: a genuine B5/global fixed point together with machine-side permanent-idle evidence and Core.EnvTerminal. It performs no ϵ -prefix or ϵ -suffix work and leaves all non-clock state unchanged. It is treated as ϵ -stutter only for E1–E5 prefix/trace matching. Core.CycleClosure's environment-definedness clause or Core.EnvTerminal alone does not justify this branch.

The τ name remains explanatory cycle-level terminology and should not be confused with execution-prefix variables such as $\tau_{\text{ref}}/\tau_{\text{post}}$; a mechanization may use the constructor name SilentCycle. No theorem may replace the model-specific ϵ relation by δ or by SilentCycle without the stated typing.

Remark Core.Det (determinism taxonomy and discharge status).

The document uses several distinct determinism notions; they shall not be conflated. (1) Same-cycle settling determinism is the model-class restriction captured by R2C/Core.QSync: fixed registered state, fixed cycle- t inputs, and fixed explicit issue/request drives yield one settled boundary valuation. (2) Conditional synchronous-step determinism is the corresponding functional-cycle view after the currently legal witness/issue/environment choices for that cycle have been fixed; it does not remove Policy-L scheduling latitude or WC-sensitive source nondeterminism. (3) B1 is observable RTL trace determinism under fixed stimulus and initial state; internal ϵ behavior may still differ. (4) I.DR/I.DR' are derived replay theorems, not assumptions to be asserted in a discharge record. (5) K.CV is the Appendix K causal/value-determinism premise. (6) SDet fixes the selected Policy-S simulator/scheduler/arbitration semantics and excludes a second incompatible committed history for that selected instance. (7) Normalization selection/congruence records which proof-relevant normalization is used and why the replay quotient is preserved; Core.QSync canonicalizes only the K-facing same-cycle settling part, not the Appendix I/J replay normalization.

For any item that is a theorem premise, Appendix M.7a's normative evidence axis applies directly: the record states whether the premise is inapplicable to the selected theorem scope, satisfied by a simple syntactic/design restriction, discharged by tool/RTL evidence, or discharged by a source/RTL witness construction. Derived theorems such as I.DR are not separate premises and therefore require no independent M.7a discharge entry.

Definition Core.KCutClosure (K-facing cutpoint existence / non-vacuity).

For a proof model M used by Appendix K, a K-origin is a reachable per-cycle state from which the K-facing boundary network is to be settled without first advancing to the next cycle: for Sys_ref/Sys_K it is the selected L1 endpoint immediately before $\rightarrow \epsilon, L2$ settling; for RTL_B it is the corresponding reachable cycle-evaluation origin after the current cycle's registered/FSM/pipeline state and process-owned request outputs are established and before K-facing same-cycle settling. $\text{KCutClosure_M}(I)$ holds when every reachable K-origin admits a finite ϵ -only/same-cycle settling normalization, within that same cycle and before any later cycle-advancing registered update, to some $\sigma_K \in \text{KCutReach_M}(I)$. For the RTL use $\text{KCutClosure_post}(I) := \text{KCutClosure_RTL_B}(I)$.

In the standard K-facing/cutpoint theorem scope, Core.KCutClosure is a derived interface rather than an independent modeling freedom. Theorem Core.QSync-Ref derives source-side KCutClosure_M from $\text{QualifiedSynchronousModel_M}(I)$, including existence of the source SettlePoint ; Theorem Core.QSync-Post derives $\text{Core.KCutClosure_post}$ from $\text{QualifiedSynchronousModel_RTL_B}(I)$. Downstream Appendix K statements continue to consume $\text{Core.KCutClosure_post}$ so that the liveness proof is factored from the particular structural settling proof. A future non-QSync semantic extension could reuse that downstream interface only by separately proving the same closure property and explicitly extending the theorem scope; it is not part of the standard user-facing G-K model defined here.

Lemma Core.KCutObserve (persistent blocker observability).

Assume $\text{KCutClosure_M}(I)$ and $K\epsilon$ for the selected model. Let s_0 be a K-origin and let σ_K be a KCut supplied by the closure witness. If a K-relevant boundary request/frontier/enablement configuration is present at s_0 after the owning requests are fixed and persists through the selected ϵ -normalization, then the same attributed K-facing configuration is present at σ_K ; WFG-relevant facts cannot be changed solely by the intervening ϵ segment. Therefore a persistent K-relevant blocker cannot be excluded from the Appendix K observation domain merely because unrelated ϵ -work would otherwise continue indefinitely. A configuration that commits, withdraws under an allowed reset disposition, or otherwise ceases to persist before the KCut is transient and is not asserted to satisfy Dead_K . Proof. KCutClosure supplies the finite endpoint; $K\epsilon$ supplies WFG-inertness of the intervening ϵ segment; $\text{BoundaryEvalPoint/Core.KCut}$ supply settled enablement at the endpoint. $\square$

Definition Core.SettleKBridge (L2 trigger to K-facing cutpoint).

Fix a Core.LAdm -admissible source prefix or continuation. $\text{Core.SettleKBridge_I}$ holds when every Core.16 activation at an L2 SettlePoint σ_set is related to every later K-facing cutpoint σ_K used to evaluate the persisting blocked condition as follows: while the same fully disabled frontier persists, the only intervening changes are phase-legal ϵ -normalization and Core.16 -permitted downstream drain commits; no new source-later work is launched; blocking requests are nonwithdrawable; $K\epsilon$ makes each ϵ -only segment WFG-inert; and every intervening Σ -visible drain commit is represented explicitly in $H/E4$ before Front , $\text{ChannelEnabled/Disabled}$, and WFG are re-evaluated at σ_K . By Lemma Core.SettleIsKCut , σ_set is itself a source KCut; the bridge is nevertheless needed because Dead_K additionally uses drain closure and may therefore be evaluated at a later KCut after already-issued work has drained. If the blocked frontier commits or otherwise disappears, no Dead_K conclusion is drawn from the earlier activation. If it persists to a later drain-closed $\sigma_K \in \text{KCutReach_ref}(I)$, Definition K.Dead is evaluated there. K.DrainReach shall discharge this bridge for the exact source witness used by Appendix K.

Source order and source-level frontier items

Definition Core.5 (Per-process execution-call universe, frontier-item universe, observable projection, and source order).

For each process P , let Q_exec, P denote its execution-level set of dynamic source-level message-call instances for the particular execution / witness under the fixed external stimulus being analyzed. Q_exec, P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Let Ω_P denote P 's local dynamic source-level frontier-item universe for that same execution / witness. Thus Ω_P is stimulus-dependent: it contains exactly the local synchronization-call items, signal-action items, and message-operation frontier items that belong to the realized execution being compared, and excludes items on untaken control-flow branches, unentered cases, and unexecuted loop iterations. For sequential processes, signal-action items are positioned by the R2 anchoring rule; for combinational processes, signal-action items are positioned in the observable cycle bucket determined by the R2C ϵ -fixed-point rule. Message-operation frontier items include reached blocking Push/Pop instances, whether pending or committed, and successful non-blocking PushNB/PopNB instances that contribute committed Push_c(v) / Pop_c(v) events. Failed non-blocking polls are ϵ -steps: they are members of Q_exec, P when executed, but they are not members of Ω_P merely because they executed.

Because Q_exec, P is a universe of dynamic message-call instances while Ω_P is a universe of source-level frontier items, their message-operation overlap is not a literal set intersection. Let $MsgOf_P(x)$ be the partial projection from a frontier item $x \in \Omega_P$ to its owning dynamic message-call instance, defined exactly for message-operation frontier items and undefined for pure synchronization or signal-anchor items. Define the message-frontier slice

$$\Omega_msg, P := \{ x \in \Omega_P \mid \exists q \in Q_exec, P : MsgOf_P(x) = q \}.$$

For any message-operation frontier item $x \in \Omega_msg, P$, let $BoundaryOf_P(x)$ denote the explicit abstract channel boundary at which x is evaluated for `BoundaryReq`, `ChannelEnabled`, `ChannelDisabled`, `Front\_P`, and Appendix K WFG reasoning. In the ordinary unelaborated case, each source-level message-call instance q has a unique corresponding frontier item x and a unique evaluated boundary. In $Sys_B^{\wedge}elab$, $MsgOf_P$ need not be injective: one owning source-level message-call instance q may project to a chain or bundle of proof-internal frontier items x at several explicit abstract boundaries. The elaboration package must then provide $BoundaryOf_P(x)$, $MsgOf_P(x)$, and the partial order $\leq_P$ for each introduced item.

Define the corresponding message-instance slice induced by Ω_P as

$$Q_Q, P := \{ q \in Q_exec, P \mid \exists x \in \Omega_msg, P : MsgOf_P(x) = q \}.$$

Let Σ_P denote the corresponding observable event/label projection: synchronization and signal items contribute their Σ labels when observed, and a message-operation frontier item $x \in \Omega_msg, P$ with $MsgOf_P(x)=q$ contributes the committed Σ -event $m(q)$ only if q commits. Pending blocking message-operation frontier items may therefore be in Ω_P and Ω_msg, P , and their owning message-call instances may be in Q_Q, P , without yet contributing any committed Σ -event. Let $\leq_P$ denote the source order on Ω_P , i.e. the order generated by the document's scheduling rules for synchronization calls, signal actions as positioned by R2 or R2C, message-operation frontier items, and any explicit partial-order structure

declared by the selected Sys_ref elaboration. The execution-level order constraints E3/E5 use the corresponding source-induced order on Q_exec, P .

Definition Core.6 (Done_P , Remain_P , and NextFront_P relative to the fixed witness).

Fix the particular execution / witness under analysis, hence the associated Q_exec, P , Ω_P , Ω_msg, P , Q_O, P , Σ_P , and $\leq_P$ from Definition Core.5. For a state σ occurring as a cutpoint of that compared execution, let

$\text{Done_P}(\sigma) \subseteq \Omega_P$

be the set of local frontier items already realized by σ . For synchronization and signal items, “realized” means that the corresponding Σ -event has occurred. For a message-operation frontier item $x \in \Omega_msg, P$ with $\text{MsgOf_P}(x)=q$, “realized” means that q has committed and its committed Σ -event $m(q)$ has occurred. A pending issued-but-uncommitted blocking message-operation frontier item x is therefore not in $\text{Done_P}(\sigma)$, even though $\text{BoundaryReq}(x, \sigma)$, $\text{ChannelEnabled}(x, \sigma)$, and $\text{ChannelDisabled}(x, \sigma)$ may be defined at σ , read operation-attributively through $q = \text{MsgOf_P}(x)$. A failed non-blocking poll $q \in Q_exec, P$ that induces no frontier item in Ω_msg, P is not considered by Done_P , Remain_P , or NextFront_P , because it is an ϵ -step with no frontier item and no committed Σ -event.

Let

$\text{Remain_P}(\sigma) := \Omega_P \setminus \text{Done_P}(\sigma)$.

Then define the next source-level frontier-item set by

$\text{NextFront_P}(\sigma) := \min_{\leq P}(\text{Remain_P}(\sigma))$.

This is the shared next-frontier notion used by Theorem J.1 and reconstructed from KObs by Lemma J.KObs-Proc, interpreted relative to the fixed witness universe Ω_P . It may contain synchronization-call items, signal-anchored action items, and message-operation items. It is a frontier of source-level items, not a set of already-committed Σ -event labels.

Definition Core.7 (Message-operation slice of the next frontier).

For the same fixed compared execution / witness, define

$\text{MsgNextFront_P}(\sigma) := \{ x \in \text{NextFront_P}(\sigma) \mid x \in \Omega_msg, P \text{ and, for } q = \text{MsgOf_P}(x), q \text{ is defined and } \text{kind}(q) \in \{\text{Push_c}, \text{Pop_c}\} \text{ for some channel } c \}$.

This is only the message-operation frontier-item slice of the candidate next frontier; it is not yet the pending blocking frontier of Appendix B / K.

Definition Core.7a (Blocking message-operation slice of the next frontier).

For the same fixed compared execution / witness, define

$\text{BlockingMsgNextFront_P}(\sigma) := \{ x \in \text{NextFront_P}(\sigma) \mid x \in \Omega_msg, P \text{ and, for } q = \text{MsgOf_P}(x), q \text{ is defined and } q \text{ is a blocking Push or Pop instance} \}$.

This is the blocking-message candidate-frontier slice used by the Appendix K Front/NextFront bridge and WFG classification. It may contain a reached blocking operation before that operation has issued; BoundaryReq and Pending determine whether such an item is currently an outstanding request and therefore a member of Front_P . Successful non-blocking transfer items may belong to MsgNextFront_P but never to $\text{BlockingMsgNextFront_P}$.

Clarification Core.5a (ordinary source order versus elaborated partial order).

For ordinary user-written sequential source code, dynamic message-call instances that appear in sequence in one process are ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and unrolled instances whose selected source expansion is ordered. Same-cycle issue, same-cycle commit, or issue before an earlier operation commits is a timing/issue-policy property, not incomparability. A message-operation multi-frontier exists only when an explicit $\text{Sys_B}^{\text{elab}}$ elaboration package introduces incomparable proof-internal frontier items and supplies the

corresponding projection, boundary, order, and attribution facts. Ordinary distinct-interface source calls are not made incomparable merely because they access different channels.

Lemma Core.OrdFrontSingleton (ordinary-source frontier uniqueness).

Fix a process P , a particular execution/witness, and an ordinary non-elaborated source interval. Assume every message-operation frontier item in that interval is induced by a unique ordinary dynamic source-call instance and no $\text{Sys_B}^{\text{elab}}$ partial-order structure is introduced. Then the restriction of $\leq_P$ to the message-operation frontier items in that interval is total. Consequently, for every cutpoint σ in the interval, $|\text{Front_P}(\sigma)| \leq 1$.

Proof. By Clarification Core.5a, all ordinary dynamic source-call instances encountered by P in the fixed execution are ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and ordered unrolled instances. In the ordinary unelaborated case, each such call induces one frontier item and $\leq_P$ is the corresponding reflexive source-induced order. Hence any two pending blocking message-operation frontier items are comparable. Two distinct elements therefore cannot both be $\leq_P$ -minimal. Thus $\text{Front_P}(\sigma)$ contains at most one member. $\square$

Operational interpretation of Sys_ref and source issue policies

The R-rules stated in Appendix G are constraints on a selected source-reference transition system, not merely constraints on a finished list of labels. For a fixed design, fixed stimulus, and selected reference-model choice, Sys_ref has the following operational reading.

A Sys_ref state consists of: (1) one local source state per process, including control location, local variables, call-stack/loop state, the current synchronization interval, and the reached/issued/committed status of dynamic message-call instances; (2) one state for each explicit abstract channel of the selected reference model, including $B(c)$, $\text{occ_c}(t)$, FIFO order when $B(c) > 0$, rendezvous endpoint requests when $B(c) = 0$, and any explicit $\text{Sys_B}^{\text{elab}}$ staged/bundle/join/epoch-gate channel state; (3) signal and synchronization state at the selected observable anchors; and (4) ϵ -microstate for internal staging, direct-input sampling between anchors, and other proof-hidden activity. In $\text{Sys_B}^{\text{elab}}$, introduced abstract channels are explicit proof-internal boundaries: their commits may be Σ -events for Appendices J-K even when projected away from the outer DUT/testbench-visible alphabet Σ_{DUT} by Π_{elab} .

The transition relation contains the following kinds of steps. Pure source-evaluation ϵ -steps advance ordinary C++/SystemC computation and allocate dynamic operation instances in dynamic source order. For a sequential process, ordinary call-return state and the L1 preparation/reach state are distinguished by Definition Core.ReachPrep below: a later operation may become reached before an earlier blocking operation has process-facing committed only through a legal dependency-independent preparation segment, and this does not make the operations unordered. A message-issue step for q may occur only after q has been reached, the endpoint request can be attributed to q , the R3 synchronization-boundary constraints are respected, and the issued set is prefix-closed under $<_{\text{src}}$. Issuing a blocking Push or Pop asserts the endpoint-owned persistent request for q and records $\text{Issue}(q)$; issuing PushNB/PopNB asserts its one-cycle attributed attempt. A channel-commit step for a bounded FIFO boundary $B(c) > 0$ may occur when $\text{BoundaryReq}(q, \sigma)$ and $\text{ChannelEnabled}(q, \sigma)$ hold at the selected abstract boundary; it emits the corresponding Push $_{\text{c}}(v)$ or Pop $_{\text{c}}(v)$ event, updates the E4 channel state, and releases the owning call at the process-facing boundary. For a rendezvous boundary $B(c) = 0$, commit is one atomic two-endpoint transfer step with matching producer and consumer endpoint requests in the same settled cycle and one shared payload value. A failed non-blocking boundary decision emits no Σ event but produces a ReturnedAtCycle status and retires its one-cycle attempt. A synchronization boundary action for call s may be selected only when Axiom Core.SyncFence is satisfied by the already-completed and same-cycle selected process-facing message commits; the interval advance caused by s occurs only after those predecessor commits have been accounted for. Synchronization and signal-anchor steps then emit

wait/Sync and anchored Read/Write events according to R1/R2/R2C. ϵ -settling and drain steps update only proof-hidden or explicitly elaborated internal state and must respect $K\epsilon$ / B-faithfulness when used by Appendices J–K. For a sequential process, Definition Core.RegSeq below governs which process-owned evaluation, request, and payload actions may share one Horizon.

Definition Core.ReachPrep (overlap-aware source reach/preparation semantics).

Fix a sequential process P , its current source interval, and a cycle t . Sys_ref retains the logical source/replay control position together with reached/issued/committed status for dynamic calls. During the L1 pre-settle phase, an operational preparation cursor may advance through a finite source-control segment and mark a later dynamic operation q as reached without claiming that an earlier blocking operation q_0 has committed or returned. Such preparation is legal only when: (1) q remains in the current synchronization interval and all dynamic source-order relations are preserved; (2) the reachability, control choice, endpoint, address, payload, and required operands of every prepared action are independent of any unresolved earlier returned data/status and satisfy Core.RegSeq and the applicable field-sensitive NoDependentUse/footprint restrictions; (3) no synchronization boundary is crossed; (4) no already-activated Core.16 blocked-frontier discipline forbids launching that source-later work; and (5) the selected source/tool witness or certificate validates the preparation segment. Reaching q does not itself assert a request. Issue remains a separate transition and remains prefix-closed under $<_{\text{src}}$.

This permission applies both to overlapped/pipelined iterations and to ordinary straight-line source sequence $q_0; q_1$ when q_1 is dependency-independent of the unresolved q_0 result and the selected Sys_ref theorem instance admits eager preparation. It does not make q_0 and q_1 incomparable, and it does not assert that a raw serial SystemC execution has returned from q_0 early; it is the source-reference scheduling permission used to represent allowed RTL-matched eager issue. Examples K.CE and K.CE-2, together with the Appendix K A5-L blocking-rendezvous example, use this straight-line case. Coverage direction for Post $\rightarrow$ Ref: for every RTL_B prefix included in a Post $\rightarrow$ Ref coverage claim, the selected Sys_ref ReachPrep semantics shall admit every dependency-independent preparation/reach segment required by the accepted J.LocalGWR or J.GWR construction for that prefix. The instance may be more permissive than the observed RTL scheduling, but it shall not exclude preparation needed to realize a covered RTL witness. Once the L2 SettlePoint exposes a nonempty fully disabled pending frontier, Core.16 prohibits every new source-later launch while that frontier persists. Every L1 preparation segment used by a theorem instance must be finite. Lemma I.3/FPD supplies that fact for its certified finite construction segments; Core.CycleClosure's source-cycle-existence/finiteness clause supplies the generic per-cycle finiteness obligation when a complete/fair source continuation is required.

Definition Core.RegSeq (registered sequential-interface semantics).

The Core classifies each process for the Appendices I–K proof stack as either combinational under R2C or sequential under Core.RegSeq. For a sequential process P , every process-driven endpoint request and Push-data output assigned to cycle/Horizon t is a function of registered process or pipeline state and operands fixed before the L2 settle point of cycle t . Combinational derivation from that registered state is permitted, but the request and payload must remain stable for the selected boundary action. Direct inputs count as fixed operands only when their applicable E2, hls_direct_input , or $\text{hls_direct_input_sync}$ contract makes their value stable at the relevant anchor. If $\text{ReturnedAtCycle}(q, t)$ holds, any P -owned evaluation, control selection, endpoint/address selection, payload computation, or message issue that depends on q 's returned data or success/failure status has Horizon strictly greater than t . A same-Horizon source-later action is permitted only when its reachability, control, operands, endpoint, address, and payload are independent of every unresolved cycle- t NB outcome and when its writes do

not change the ReqSnapshot of that unresolved decision except through an explicitly ordered dependency. This is a normative theorem-scope restriction, not a consequence of the word “clocked” alone. Tool-side Cat# or equivalent conformance evidence and source-side simulation/lint/witness evidence shall discharge it. Intentional zero-time outcome-dependent NB cascades or retry loops shall be identified by checked source/tool analysis and then rejected or explicitly excluded from the theorem instance, isolated as latency-sensitive logic, remodeled as one explicit atomic multi-way decision, or aligned by Appendix L; silent scope exclusion is not permitted.

Definition Core.PolicyL (liberal, witness-relative issue).

Policy L is the most permissive source-side issue policy used by the safety witness and by the K.2b dominance/extraction bridge. It may issue a reached source-level message operation whenever the transition-system preconditions above, R3/R4/R5, E4, E5, Core.RegSeq, the selected non-blocking outcome witness, and the source-order prefix-closure requirement are satisfied. A source-later operation may have become reached through Core.ReachPrep even while a source-earlier same-interval blocker remains uncommitted. Policy L may therefore leave the source-later blocking request issued in that state. This is how the reference model represents loop overlap, ordinary dependency-independent eager preparation, RTL-matched eager issue, and other allowed same-cycle/overlapped request timing. Policy L is not a claim that every raw pre-HLS simulator mode can generate every such witness state. Policy L does not override returned-result availability: it cannot issue an operation in cycle t when its Core.ReachPrep reachability, control selection, endpoint or address selection, payload, or any required operand depends on data/status first returned at the cycle- t boundary. Core.LLegal fixes Policy-L step legality, Core.Phase fixes the cycle-settle boundary and the trigger for Core.16, and Core.LAdm below combines those pieces with the activated Core.Drain qualification used by liveness admissibility.

Definition Core.LLegal (Policy-L step legality and fixed-instance binding).

Fix a selected liveness instance I consisting of Sys_ref, the initial/reset state, capacities and elaboration/lowering, the fixed stimulus and reactive environment, witness-selected non-blocking and other ϵ outcomes, arbitration choices, and the selected synchronization, signal-anchor, and ϵ -normalization conventions. A finite or infinite path p is Policy-L-legal for I iff it begins in the selected initial/reset state, every transition is a transition of the operational Sys_ref relation above permitted by Policy L, and every fixed choice in I is respected. R1–R5, E4/E5, dynamic-operation identity, request attribution and persistence, endpoint cardinality, reset-epoch typing, and sequential-result availability are part of this step legality.

Definition Core.Phase (Policy-L cycle phases and settled-frontier trigger).

Each cycle t of a Policy-L-legal path is interpreted using the following normative phase boundary. (L0/L1) Cycle entry and pre-settle issue window. Starting from the cycle-entry state after all prior-cycle boundary updates, sequential processes may take a finite selected sequence of pure source-evaluation ϵ -steps and message-issue steps for operations that were already reached at cycle entry or become reached during this phase through Core.ReachPrep and evaluation independent of every cycle- t boundary result. Their control choice, endpoint, address, payload, status inputs, direct-input values, and other required operands must be available under Core.RegSeq independently of any cycle- t commit or failed/non-failed NB decision outcome. These issue steps remain subject to R3–R5, source-order prefix closure, stable payload, and the deferred-hole restrictions of Lemma I.3. Thus a source-later outcome-independent operation, including an ordinary straight-line successor admitted by Core.ReachPrep, may issue during this window while a source-earlier blocker or unresolved NB attempt remains uncommitted/undecided. A value or status first returned at the cycle- t boundary cannot be used in this

window. Any theorem that constructs a complete/fair source continuation additionally requires the selected L1 preparation/issue segment to terminate finitely as part of Core.CycleClosure.

(L2) settle point. After the selected process-owned pre-settle steps for cycle t , the L1 window closes. Process-owned request assertions and Push payloads are then held fixed while the explicit channel network and every other phase-legal L2 ϵ -step settle under $\rightarrow_{\epsilon, L2}$ to the first $\text{SettlePoint}_I(t, \sigma_t^{\text{set}})$ of Definition Core.Settle. The phrase “the frontier settles” and every Appendix K test of whether a frontier is fully channel-disabled at the activation boundary refer to this L2 SettlePoint. By Lemma Core.SettleIsKCut, the settled source state is also Core.4-quiescent and lies in the applicable source KCut domain; its special role here is the Core.16 activation boundary, not necessarily the later drain-closed Dead_K evaluation point.

(L3) Freeze and boundary action. Front_P and ChannelEnabled/ChannelDisabled are evaluated at σ_t^{set} . If Front_P is non-empty and every frontier item is disabled there, Definition Core.16 activates immediately at σ_t^{set} : no new source-later operation may issue after that snapshot in cycle t or in a later cycle while the same fully disabled frontier persists, although already-issued downstream work may drain as Core.16 permits. If the frontier is not fully disabled, the selected E4/rendezvous, synchronization, signal-anchor, arbitration, and endpoint-cardinality rules determine the boundary actions that commit in cycle t . Any synchronization action s in that selected L3 boundary-action set is additionally gated by Axiom Core.SyncFence: every blocking $q \in \text{pref}_S(s)$ must have process-facing committed earlier or be selected to process-facing commit in the same cycle before s advances P into the next source interval.

(L4) Next-cycle result availability. Boundary commits and failed non-blocking decisions produce their process-visible ReturnedAtCycle results and update registered process/channel state for the next cycle. Process-owned evaluation or issue whose reachability, control, endpoint, address, payload, or required operand depends on a value or success/failure status returned in cycle t may begin only in a strictly later cycle. Every deferred NB hole created for Horizon t is resolved or retired before the exported post-boundary cutpoint for that Horizon.

The L2/L3 freeze is not retroactive. An issue that was legal in the L1 pre-settle window is not invalidated merely because the subsequent settled snapshot exposes an earlier fully disabled frontier. Conversely, once that settled snapshot exists, an operation not already issued before it cannot use the same-cycle pre-settle window to bypass the activated Core.16 block.

Dependency convention. Core.Phase defines the settled-frontier trigger and phase ordering without assuming the continuation discipline. Core.16 separately defines the drain-only continuation obligations activated by that trigger. Core.LAdm, stated after Core.16, combines Policy-L legality, Core.Phase, nonwithdrawal, $K\epsilon$, and the activated Core.Drain qualification; there is therefore no circular definitional dependency between admissibility and the drain discipline.

Definition Core.PolicyS (conservative serial-blocking issue).

Policy S uses the same Sys_ref state space, channels, values, witness choices, and transition rules as Policy L, but adds one process-side restriction: within a source interval between adjacent ordinary synchronization calls selected into S , a later source-order blocking Push/Pop operation may not issue until every earlier source-order blocking Push/Pop operation in that same interval has committed in a strictly earlier clock cycle. In $\text{Sys_B}^{\text{elab}}$, the relevant commit for this restriction is the process-facing commit that completes, accepts, or releases the owning source-level blocking operation at its selected boundary; internal staged/bundle/join/epoch-gate transfers do not by themselves satisfy the serial condition unless the elaboration package identifies that transfer as the process-facing commit for the source operation. Policy S serializes only process-side issue of source-level blocking calls. It does not serialize independent peer processes, E4 channel-side drainage, FIFO movement, or ϵ -steps that are otherwise admissible.

Remark Core.PolicyS-Sim (Relation to practical pre-HLS simulation modes).

A raw serial source simulation mode implements the Policy-S discipline for ordinary unelaborated blocking calls. A Matchlib-style eager or skidbuffered source simulation mode may execute source calls in dynamic program order while issuing requests without consuming a clock cycle until data or space is unavailable; that mode is Policy-L-compatible only to the extent that its request attribution, channel states, and non-blocking outcomes satisfy the transition-system conditions above. Neither mode treats ordinary distinct-interface source calls as unordered. The difference between serial and eager simulation modes is a difference in issue timing and buffering, not a difference in $<_{\text{src}}$.

Commit, issue, and boundary requests

Definition Core.8 (Commit).

For a message transfer at a chosen Σ -visible boundary endpoint, $\text{Commit}(q)$ is the cycle in which an external clocked observer sees both valid and ready high simultaneously at that endpoint. The committed payload is the value seen in that same cycle.

Definition Core.9 (Issue).

For a source-level message operation instance q at a chosen Σ -visible boundary endpoint, $\text{Issue}(q)$ is the first cycle in which the calling process begins requesting the transfer corresponding to q at that endpoint. $\text{Issue}(q)$ is an auxiliary scheduler/witness timestamp used for rules such as R4, E3, and E5.

Definition Core.IssueFair (weak fairness of source issue).

Fix an issue policy $\Pi \in \{\text{Policy L}, \text{Policy S}\}$ and a complete phase-bearing source execution ρ . $\text{IssueEnabled}_{\Pi}(q, t)$ holds at an applicable L1 opportunity when q has been reached, has not yet issued or been retired/reset, and its message-issue transition is legal under the operational Sys_ref relation, $\text{Core.Phase}, \Pi, \text{R3--R5/E4/E5}, \text{Core.RegSeq}$, source-order prefix closure, Core.16 , and the fixed witness/environment/reset/arbitration choices. $\text{IssueFair}_{\Pi}(\rho)$ holds iff, whenever from some cycle t_0 onward within one reset epoch q remains $\text{IssueEnabled}_{\Pi}$ at every applicable L1 opportunity until it issues or the execution reaches an affecting RESET for q or an explicit DeclaredTerminal disposition, q is issued after finitely many such opportunities. This is weak fairness: an operation whose issue legality merely recurs intermittently does not satisfy the antecedent. An operation forbidden by Core.16 is not $\text{IssueEnabled}_{\Pi}$ and imposes no IssueFair obligation. IssueFair governs assertion/issue of a reached source operation; B2/B3 separately govern selection/commit of already-issued boundary actions. A constructive complete-source continuation theorem may use a stronger recurrent-opportunity dovetailing scheduler, but the required property remains Core.IssueFair itself. Appendix K consumes Core.IssueFair as a premise of complete admissible continuations.

Definition Core.10 (Endpoint-owned boundary request).

Fix a Σ -visible endpoint direction at an explicit abstract boundary. For any message-operation frontier item $x \in \Omega_{\text{msg}, P}$ with owning source-level message instance $q = \text{MsgOf}_P(x)$ and evaluated boundary $c = \text{BoundaryOf}_P(x)$, let $\text{req}_x(t)$ denote the endpoint-owned boundary request signal for that endpoint in cycle t , namely valid for a Push endpoint and ready for a Pop endpoint. The predicate $\text{BoundaryReq}(x, \sigma)$ records that this endpoint-owned request is asserted for frontier item x at state σ . $\text{BoundaryReq}(x, \sigma)$ is operation- and boundary-attributive: it is true only when the asserted endpoint-owned request at σ is attributed to the specific frontier item x , its owning source-level operation q , and its evaluated boundary $\text{BoundaryOf}_P(x)$. In particular, if $\text{Issue}(q)$ has not yet occurred at σ , then $\text{BoundaryReq}(x, \sigma)$ is false. In the ordinary unelaborated case, where q has a unique frontier item and a unique evaluated boundary,

BoundaryReq(q, σ) may be used as shorthand for BoundaryReq(x, σ). In Sys_B^{elab}, the x-form is authoritative.

Axiom Core.IC (Issue/Commit linkage).

Fix a Σ -visible endpoint direction. For source-level message operation instances on that endpoint direction, the following all hold:

(1) Boundary-request soundness / non-withdrawal.

If q is blocking, then once issued its endpoint-owned request remains asserted, with stable payload while asserted for Push, until q process-facing commits or an affecting RESET governed by RW2 clears that dynamic instance. Ordinary back-pressure, scheduler delay, a silent cycle, or any other nonterminal transition cannot deassert, retire, reattribute, cancel, or replace it. These are the only nonterminal mechanisms in the G–K theorem model by which an outstanding blocking dynamic instance may cease to be outstanding: process-facing commit or an affecting RW2 RESET. A DeclaredTerminal disposition may instead end the execution with q still pending. Protocols that require independent nonterminal cancellation, withdrawal, retry, or reattribution of a blocking request are outside the G–K theorem model unless an explicit operational extension and its correspondence/progress obligations are separately defined and proved.

(2) Stable attribution while the same source-level request remains outstanding.

If a blocking request remains asserted across a cycle in which no transfer commits on that endpoint direction, the continued assertion remains attributed to the same outstanding q until its process-facing commit or an affecting RW2 RESET. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal does not collapse later executed non-blocking call instances into the same q ; each later executed source-level poll is a distinct $q' \in Q_{\text{exec}}, P$ unless the witness explicitly identifies the same still-outstanding source-level request instance.

(3) Back-to-back issuance.

If q_0 and q_1 are same-direction source-level message operation instances, q_0 commits in cycle t , and q_1 is the next such operation issued back-to-back within the same source interval, then $\text{Issue}(q_1) = t + 1$, even if the request signal does not deassert between commits.

(4) Sync-boundary discipline.

If q_0 and q_1 are same-direction source-level message operation instances and $q_0 \in \text{pref}_S(s)$ and $q_1 \in \text{suff}_S(s)$ for a synchronization call s , then $\text{Issue}(q_1)$ occurs strictly after the commit of s . Any request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in $\text{pref}_S(s)$, not to issuance of any operation in $\text{suff}_S(s)$. Synchronization-completion legality itself is governed by Axiom Core.SyncFence below.

Axiom Core.SyncFence (synchronization-completion fence).

Let s be a dynamic synchronization-call instance of process P selected into S , and let $\text{BPref}_S(s)$ be the blocking Push/Pop instances in $\text{pref}_S(s)$. A boundary action that commits s is legal only if every $q \in \text{BPref}_S(s)$ has process-facing committed no later than that same cycle. Equivalently, after s commits and P advances into the succeeding source interval, no $q \in \text{BPref}_S(s)$ may remain issued and uncommitted. Same-cycle completion is permitted: q and s may both commit in cycle t provided q 's process-facing commit is included in the selected L3 boundary-action set for t and is accounted for before the interval advance caused by s . This “accounted for before” convention is an ordering of the source operational update and of the proof/certificate attribution of the same-cycle boundary batch; on RTL_B it does not require an observable physical sub-cycle ordering between two events that commit on the same active clock edge. In Sys_B^{elab}, “process-facing commit” means the commit/accept/release event designated by the elaboration package as completing the owning source-level operation q ; an internal staged, bundle, join, guard, or epoch-gate transfer that does not release q does not satisfy this

fence. Appendix G rules R3 and E5 state the corresponding source-side and implementation-side trace constraints explicitly.

Remark Core.10a (Non-blocking polls).

For non-blocking message passing, a failed poll produces no Σ event and is modeled as an ϵ -step, while a successful poll contributes the corresponding committed $\text{Push}_c(v)$ or $\text{Pop}_c(v)$ event. Each executed $\text{PushNB}/\text{PopNB}$ call is nevertheless a distinct dynamic source-level instance $q \in Q_{\text{exec}}, P$ for its owning process P ; thus repeated failed polls in later loop iterations are distinct q 's even if the endpoint request remains continuously asserted. The ϵ -modeling concerns observability in Σ , not the dynamic-instance identity of the underlying source-level calls.

Channel enablement

Definition Core.11 (ChannelEnabled / ChannelDisabled).

Let x be a message-operation frontier item on channel $c = \text{BoundaryOf}_P(x)$, and write $q = \text{MsgOf}_P(x)$ with $\text{kind}(q) \in \{\text{Push}_c, \text{Pop}_c\}$ for its endpoint direction. At a $\text{BoundaryEvalPoint}_I(\sigma)$ of Definition Core.Settle, let t_σ denote the cycle of that cutpoint. Define:

$\text{ChannelEnabled}(x, \sigma)$ to mean that $\text{BoundaryReq}(x, \sigma)$ holds and that the channel-side condition required for x 's owning operation q to commit at $\text{BoundaryOf}_P(x)$ is satisfied after combinational settling at that cutpoint, and

$\text{ChannelDisabled}(x, \sigma)$ to mean that $\text{BoundaryReq}(x, \sigma)$ holds but $\text{ChannelEnabled}(x, \sigma)$ does not.

In the ordinary unique-boundary case, $\text{ChannelEnabled}(q, \sigma)$ and $\text{ChannelDisabled}(q, \sigma)$ may be used as shorthand for the corresponding x -form. Every L2 SettlePoint and every Core.KCut cutpoint used by Appendix K is a BoundaryEvalPoint ; the definition therefore has one boundary-evaluation meaning without identifying those two kinds of cutpoint.

Definition Core.12 (Buffered channels).

If $B(c) > 0$ and x is a source-level or elaborated message-operation frontier item on channel $c = \text{BoundaryOf}_P(x)$, with $q = \text{MsgOf}_P(x)$, then the buffered-channel enabling condition is:

- $\text{ChannelEnabled}(x, \sigma) \Leftrightarrow \text{BoundaryReq}(x, \sigma) \wedge \text{occ}_c(\sigma) < B(c)$, when $\text{kind}(q) = \text{Push}_c$
- $\text{ChannelEnabled}(x, \sigma) \Leftrightarrow \text{BoundaryReq}(x, \sigma) \wedge \text{occ}_c(\sigma) > 0$, when $\text{kind}(q) = \text{Pop}_c$

Thus, if x is a buffered Push_c frontier item that is channel-disabled, then $\text{BoundaryReq}(x, \sigma)$ holds and $\text{occ}_c(\sigma) = B(c)$; and if x is a buffered Pop_c frontier item that is channel-disabled, then $\text{BoundaryReq}(x, \sigma)$ holds and $\text{occ}_c(\sigma) = 0$.

Definition Core.13 (Rendezvous channels).

If $B(c) = 0$ and x is a source-level or elaborated message-operation frontier item on channel $c = \text{BoundaryOf}_P(x)$, with $q = \text{MsgOf}_P(x)$, then the rendezvous enabling condition is:

$\text{ChannelEnabled}(x, \sigma) \Leftrightarrow \text{BoundaryReq}(x, \sigma) \wedge$ (the complementary endpoint request on c is asserted at σ).

Equivalently:

- if $\text{kind}(q) = \text{Push}_c$, then

$\text{ChannelEnabled}(x, \sigma) \Leftrightarrow \text{BoundaryReq}(x, \sigma) \wedge$ (the complementary Pop_c endpoint request is asserted at σ)

- if $\text{kind}(q) = \text{Pop}_c$, then

$\text{ChannelEnabled}(x, \sigma) \Leftrightarrow \text{BoundaryReq}(x, \sigma) \wedge$ (the complementary Push_c endpoint request is asserted at σ)

Hence, if a rendezvous frontier item x is channel-disabled, its own `BoundaryReq` holds but the complementary endpoint is not simultaneously requesting the matching partner action at that cutpoint. In the ordinary unique-boundary case, the equivalent q -form may be used as shorthand.

Pending blocking frontier

Definition Core.14 (Pending blocking operations).

Fix a `BoundaryEvalPoint_I( $\sigma$ )` for process P within the current interval $I_P(\sigma)$ between the adjacent synchronization calls that bracket σ in P 's current execution. Define

$\text{Pending_P}(\sigma) := \{ x \mid x \in \Omega_{\text{msg},P} \text{ is a blocking source-level message-operation frontier item in that same interval } I_P(\sigma), \text{ and for } q = \text{MsgOf_P}(x), q \text{ is defined, Issue}(q) \text{ has occurred, and Commit}(q) \text{ has not yet occurred in } \sigma \}$.

By Appendix C's Issue/Commit linkage together with the definition of `BoundaryReq`, each member x of $\text{Pending_P}(\sigma)$ has its endpoint-owned request asserted at the cutpoint, read operation-attributively through its owning dynamic message-call instance $q = \text{MsgOf_P}(x)$. By `Core.SyncFence`, an issued blocking operation from the preceding interval cannot survive the synchronization commit that advances P into $I_P(\sigma)$; therefore the interval restriction cannot discard an outstanding blocker. This definition is valid both at an L2 `SettlePoint` used by `Core.16` and at a `Core.KCut` comparison cutpoint; `WFG` and `Dead_K` use only the latter.

Definition Core.15 (Pending blocking frontier $\text{Front_P}(\sigma)$).

Define

$\text{Front_P}(\sigma) := \min_{\leq P}(\text{Pending_P}(\sigma))$

If several $\leq_P$ -minimal pending blocking message-operation frontier items are simultaneously outstanding, they form a frontier. Lemma `Core.OrdFrontSingleton` proves that this set has at most one member for an ordinary non-elaborated source interval. This is the Appendix B / K notion of the pending blocking frontier (or minimal pending blocking frontier). It is distinct from $\text{NextFront_P}(\sigma)$, which is the broader source-level frontier-item set over Ω_P .

Wait-for graph

Definition Core.WFG (Wait-for graph).

At a K -facing cutpoint $\sigma \in \text{KCUTReach_ref}(I)$, or at the corresponding certified RTL K -facing view, define the wait-for graph $\text{WFG}(\sigma)$ as follows. Its vertices are the modeled processes, and its directed edges are induced from frontier members $\text{Front_P}(\sigma)$ by the corresponding channel-disabled dependencies on internal message-passing channels only. Equivalently, a directed edge $P \rightarrow Q$ is present when some frontier item $x \in \text{Front_P}(\sigma)$, with owning message instance $q = \text{MsgOf_P}(x)$, is channel-disabled at the chosen boundary and the unique complementary endpoint needed to enable q belongs to process Q . Here "internal" means that both endpoints of the channel are modeled processes of `Sys_ref` / `RTL_B`. Channels whose complementary endpoint is the external environment/testbench at the DUT boundary are excluded from `WFG` and from the internal message-passing deadlock notion used in Appendix K. Buffered and rendezvous channels are both included, with `B(c)`, `occ_c( $\sigma$ )`, `BoundaryReq`, `ChannelEnabled`, and `ChannelDisabled` interpreted at the chosen `Core.KCut` boundary-evaluation point on the explicit abstract channels of `Sys_ref` or reconstructed certified RTL view. A single `WFG` may therefore contain a mix of buffered-channel and rendezvous-channel edges. Appendix K consumes this `Core` definition and the clarifications below and adds the deadlock, closure, waiver, and preservation predicates; it does not redefine `WFG`. `Core.16` itself does not require a `WFG` at its L2 `SettlePoint`; it uses

Front_P and the ALL-disabled boundary test there and reaches the K-facing WFG domain only through Core.SettleKBridge/K.DrainReach.

Clarification Core.WFGa (WFG edge versus blocked process).

A directed WFG edge records a channel-disabled dependency of one frontier item; it does not by itself imply that the owning process P is blocked on message passing. If $\text{Front_P}(\sigma)$ contains another explicitly incomparable channel-enabled member, P has an outgoing WFG edge but is not blocked under Appendix K's ALL-disabled test. The internal message-passing-deadlock predicate therefore quantifies only over nonempty sets D in which every $P \in D$ is blocked, so every edge used to close a deadlocked component originates from a process with no channel-enabled frontier item.

Clarification Core.WFGb (Multi-frontier outgoing edges).

When $\text{Front_P}(\sigma)$ has multiple members, P may have multiple outgoing WFG edges, one for each channel-disabled frontier item and its unique complementary endpoint. By Lemma Core.OrdFrontSingleton, such multi-frontier behavior cannot arise from an ordinary non-elaborated source interval. It is admitted only when the explicit Sys_B^{elab} package introduces and types the corresponding incomparable proof-internal frontier items; ordinary source-sequenced message calls do not become incomparable merely because they use distinct channels.

Drain-only blocked-frontier discipline and automatic flush

Definition Core.16 (Drain-only blocked-frontier discipline).

A source interval satisfies the drain-only blocked-frontier discipline if, at the L2 SettlePoint_I(t, σ_t^{set}) of Definitions Core.Phase/Core.Settle, $\text{Front_P}(\sigma_t^{\text{set}})$ is non-empty and every frontier item $x \in \text{Front_P}(\sigma_t^{\text{set}})$ is channel-disabled. The discipline activates at that first settled snapshot. It does not retroactively invalidate source-later operations legally issued during the preceding L1 pre-settle window, but it forbids any new such issue after σ_t^{set} in that cycle and while the same fully disabled frontier persists. Set $\sigma := \sigma_t^{\text{set}}$ at activation. This activation state is a BoundaryEvalPoint but is not assumed to be a Core.KCut cutpoint; any later Dead_K evaluation is connected by Core.SettleKBridge. Once activated, the following all hold:

(1) Block point.

P is blocked on message passing at $\text{Front_P}(\sigma)$.

(2) No new later work.

While $\text{Front_P}(\sigma)$ remains non-empty and fully disabled, no new later blocking message-operation frontier item $y \in \Omega_msg, P$ is issued, where “later” means that, for some $x \in \text{Front_P}(\sigma)$, $x \leq_P y$ and $x \neq y$, with issue understood through the owning message instance $q_y = \text{MsgOf_P}(y)$. More generally, no new source-later dynamic operation is launched past the blocked point. Here “launched” is intentionally broader than Issue(q): it includes a source-evaluation or message-issue transition that advances new source-later dynamic work past the blocked point. The specific no-new-Issue clause and this broader no-new-launch clause are retained as distinct obligations.

(3) No withdrawal.

Already-asserted blocking requests are not withdrawn before the owning message instances commit.

(4) Frontier-minimality.

An already-asserted blocking request whose frontier item is not in $\text{Front_P}(\sigma)$ is not treated as a wait-for source unless and until that frontier item later becomes frontier-minimal.

(5) Drain-only downstream progress.

Work already past the blocked frontier may drain and commit when channel-enabled, but no new work advances past the blocked frontier while that frontier remains fully disabled.

(6) Finite non-replenishing drain.

Every selected explicit abstract boundary that can hold or credit drainable work has finite B-faithful storage or credit, directly or through a finite $\text{Sys_B}^{\text{elab}}$ boundary chain. Together with clause (2), this makes the drain set finite and non-replenishing.

(7) Sync-boundary capacity withdrawal — elaborated-boundary interpretation.

While a process is pending at an explicit synchronization call s , producer-facing availability for any back-annotated capacity belonging only to $\text{suff_S}(s)$ is withheld until s commits; work already accepted before s may still drain. For the Sys_ref proof model, this withdrawal is not an additional hidden enablement predicate on an otherwise ordinary E4 channel. It is represented only by choosing $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$ with explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that the refusal appears as ordinary `ChannelDisabled` behavior at an explicit abstract boundary.

For a pipelined or otherwise overlapped implementation, Appendix B’s automatic-flush rules are the implementation-side construction that supplies Definition Core.16. For ordinary non-overlapped control, the same definition may be discharged by a structural proof that no new source-later dynamic operation can be launched after the blocked point is reached. Thus the K.2b dominance/extraction premise ranges over every liberal interval that can contain eager issue, not only loop pipelines. In particular, eager issue permitted by Policy L must occur in Core.Phase’s L1 pre-settle window; the no-new-launch obligation begins at the L2 settled snapshot, not at the earlier issue of the frontier request.

Definition Core.Drain (activated drain-discipline qualification).

For a fixed liveness instance I and a Policy-L path or prefix p , $\text{Core.Drain_I}(p)$ holds when every Core.Phase L2 SettlePoint at which Definition Core.16 activates obeys all seven Core.16 clauses for as long as the same fully disabled frontier persists. The qualification is bound to the fixed instance, witness, environment, arbitration, reset, elaboration, and boundary attribution; its finite-drain clause is interpreted over the selected B-faithful explicit capacities or credits. For Appendix K applications, the same prefix/continuation must additionally satisfy $\text{Core.SettleKBridge_I}$ so that an L2 activation is never silently treated as the later global K-facing comparison state. Core.Drain is the Core-level continuation qualification consumed by Core.LAdm . Later theorem layers may require design- or cutpoint-specific evidence that discharges Core.Drain and $\text{Core.SettleKBridge}$, but that evidence does not redefine either qualification.

Remark Core.16a (ALL disabled $\Rightarrow$ stall).

The document’s intended policy is exactly the “ALL disabled $\Rightarrow$ stall” interpretation: the process stalls only when none of the frontier items in $\text{Front_P}(\sigma)$ is channel-enabled. Downstream drain is compatible with that stall provided no new later work is issued past the blocked frontier.

Definition Core.LAdm (admissible Policy-L prefixes and complete executions).

For Appendix K, a finite Policy-L prefix is admissible under Core.LAdm iff it is Policy-L-legal, follows L0–L4, respects nonwithdrawal and $K\epsilon$, satisfies Core.Drain from every activation point reached in the prefix, and satisfies $\text{Core.SettleKBridge}$ for every K-facing cutpoint selected to evaluate a persisting blocked frontier. Write $\text{Pref_L}^{\text{adm}}(I)$ for these finite prefixes. A complete Policy-L-admissible execution follows $\text{Core.CompleteCycle}$ for every real cycle and is either infinite or ends at an explicitly DeclaredTerminal state; it additionally satisfies the selected B2/B3/B5 progress/quiescence obligations and Core.IssueFair for Policy L. A state is not finite-maximal merely because no Σ -action or ϵ -step is enabled at the present cycle: if it is nonterminal, $\text{Core.CycleClosure/CompleteCycle}$ continues it, possibly through a SilentCycle. If a complete execution is extended indefinitely by the B6 convention, the

terminal state must also satisfy `Core.TerminalIdle` (including its `Core.EnvTerminal` component). Write `Exec_L^adm(I)` for this set. `A5-L`'s reachability clause quantifies over the source `K`-facing cutpoints `KCutReach_ref(I)` reached by members of `Pref_L^adm(I)`, not over `L2 SettlePoints` merely because they are combinationally settled. `K.2b.Q`'s continuation, stabilization, and limit arguments additionally require the existence of a member of `Exec_L^adm(I)` extending the finite counterexample prefix. Lemma `K.2b.E` constructs such an extension from its stated premises; the extension is not a theorem-application certificate field or an additional witness-package assumption. Fairness is therefore not decided by a finite prefix, but a finite prefix that has already violated the phase, freeze, or settle-to-`K`-cutpoint bridge rules is not admissible.

Normative-use convention. Elsewhere in this document, “Policy-L execution” means a Policy-L-legal path unless the context says otherwise. “Admissible Policy-L execution/prefix” in `A5-L`, `K.1A`, `K.2b`, `K.Ser`, the `K.CE` examples, and Appendix `K-P` means `Core.LAdm`-admissible. `Core.Drain` is therefore not silently part of the basic Policy-L issue rule; it is an explicit qualification of liveness admissibility. Policy `S` uses the same `L0–L4` cycle phases, with its stricter serial-blocking issue condition substituted for the Policy-L issue permission in `L1`.

Observable compatibility and cutpoint correspondence

Definition Core.17 (System-level observable compatibility).

For a witness prefix τ_{ref} of `Sys_ref` and a post-HLS prefix τ_{post} of `RTL_B`, define $\Sigma\text{Match}(\tau_{\text{ref}}, \tau_{\text{post}})$ to mean that the selected Σ -visible events/observable units match under the canonical occurrence convention of this theorem stack. For each message-passing channel `c`, the projected sequences of committed `Push_c` events and of committed `Pop_c` events have the same length; the k -th `Push_c` on one side matches the k -th `Push_c` on the other, and likewise for `Pop_c`, with the same endpoint and payload/value label. For each observable signal `sig` of a sequential process `P`, the canonicalized sequences of `Read(sig, ·)` and `Write(sig, ·)` occurrences have the same length and the k -th occurrences match with the same process/signal/direction and value label; within each source anchor interval, canonicalization records at most one Σ -visible `Read` and at most one Σ -visible `Write` for each (P, sig) at the `E2` anchor, while redundant physical sampling or re-driving is ϵ -internal. For each `R2C` combinational process or explicitly modeled combinational network component `C_comb`, the emitted fixed-point observation-unit sequences have the same length and the k -th units match by component identity, included process/signal/direction labels, and fixed-point values, including any cycle-locked participation requirement. For proof-internal `RESET(S_p, S_c)` units included in the selected theorem comparison, project each trace to the `RESET` occurrences of each reset scope (S_p, S_c) ; the k -th `RESET` occurrence for a scope on one side matches the k -th `RESET` occurrence for the same scope on the other, so matched `RESET` units have identical `S_p` and `S_c`. Independent disjoint `RESET` units may commute except where the reset well-formedness rules or another selected observable-order constraint imposes an order. Synchronization events match by their per-process occurrence index in source order. Independent matched Σ events may commute or occupy different global cycle buckets except where `E1–E5` or the selected observable-unit semantics impose an ordering or same-cycle constraint. An unmatched selected Σ event/unit on either side makes ΣMatch false. Define

$$\tau_{\text{ref}} \approx \tau_{\text{post}} :\Leftrightarrow \Sigma\text{Match}(\tau_{\text{ref}}, \tau_{\text{post}}) \wedge E1 \wedge E2 \wedge E3 \wedge E4 \wedge E5$$

This is an ordered compatibility predicate, not a mathematical equivalence relation. The left argument supplies the `Sys_ref` witness prefix, and the right argument supplies the `RTL_B` post-HLS prefix. ΣMatch and `E1–E5` are normative conjuncts of the `Core` definition. Appendix `G` expands the canonical Σ -event matching convention and the `E1–E5` rules but does not redefine this relation or add a separate matching conjunct. In particular, the `R2C` matching statement in `E2` below is the Appendix-`G` expansion of the `R2C` clause already contained in ΣMatch , not a second `R2C` matching requirement.

Definition Core.18 (Cross-model KObs correspondence).

For an exact selected source/RTL package with common typed record $\langle E, w, H, O \rangle$, cross-model KObs correspondence is equality of the source K-observation record with a post-side K-observation record of that same Core.18c type. On the source side, $\text{KObs_E}, w(\sigma_{\text{ref}}) = \langle E, w, H, O \rangle$. The Core fixes only the type and equality notion of this correspondence; it does not independently define how an RTL_B cutpoint yields such a record. Appendix J supplies that post-side representation as $\text{KObsPost_J}(\rho_{\text{post}}; E, w, H, O)$ and, for the selected package of Corollary J.1, proves by J.KObsConf that $\text{KObsPost_J}(\rho_{\text{post}}; E, w, H, O) = \text{KObs_E}, w(\sigma_{\text{ref}})$. This definition introduces no additional state clauses. Process, channel, frontier, request, enablement, WFG, and snapshot-drain facts are obtained by applying Definitions Core.18d–Core.18e to the common record. Appendix K separately defines and factors its deadlock, waiver, environment-terminal, and diagnostic predicates from those reconstructed inputs.

K-observation quotient and residual-offer layer

Status of this subsection. Definitions Core.18a–Core.18e provide the source-side KObs record and its pure Core reconstruction functions. Appendix J proves the source-side reconstruction lemmas J.KObs-Proc , J.KObs-Chan , J.KObs-Offers , and J.KObs-WFG , constructs the corresponding J-facing RTL record, and uses J.OfferReach / J.OfferConf / J.KObsConf to prove equal KObs at matched cutpoints. Appendix K then defines and proves the deadlock-, waiver-, and diagnostic-predicate factoring results over that record. This subsection does not characterize the image of arbitrary well-typed records beyond the explicit J.OfferReach predicate, and it does not alter the operational Issue, Policy-L/Policy-S, or DrainDiscipline semantics.

Definition Core.18a (Enriched replay history H).

Fix a theorem instance I consisting of the design, fixed stimulus, declared environment model, reset policy, chosen observation boundaries, and KObs-facing waiver set W . For the Policy-S A5-L discharge, I may additionally reference a response-monitor schema, finite-bound function, reset/end-of-test disposition, and response-waiver policy; those are operational certificate data and are not fields of KObs. I also records an environment-reconstruction mode for optional downstream diagnostics: either StandardEnv , meaning the normative B1-avail pull-based, consumption-ordered input model with default always-ready outputs, or EnvReplayAdequate , meaning a certificate that supplies a typed reconstructed environment state $\text{EnvStateRec_I}(H, w)$ and proves that permanent next-interaction availability is determined by H , w , and that reconstructed state. The exact environment-terminal and diagnostic predicates are defined in Appendix K. Fix the full elaboration package E and witness w . For a reachable source K-facing cutpoint $\sigma \in \text{KCutReach_ref}(I)$, let $H_{\text{E}, w}(\sigma)$ be the finite canonical annotated replay history through σ . It contains the observable units and annotations needed by Theorem J.1 and $I.\text{DR}/I.\text{DR}'$: committed $\text{Push_c}(v)/\text{Pop_c}(v)$ events with dynamic identities and per-channel ordinals; synchronization and wait units; anchored Read/Write units; START_P , FINISH_P , and $\text{RESET}(S_p, S_c)$ units where applicable; and the WC/witness-selected non-blocking, arbitration, signal, and ϵ -choice outcomes required to replay the selected execution. H retains explicitly atomic unit grouping, per-process replay order, per-channel FIFO order, reset-epoch structure, and the required-prefix constraints used by $\leq_J$. It quotients away incidental model-specific clock-bucket placement and ordering among independent units not required by those constraints. Raw source or RTL cycle buckets may be retained as certificate metadata, but they are not fields compared by KObs equality. $H_{\text{E}, w}(\sigma)$ contains no historical $\text{Issue}(q)$ timestamps merely because requests were asserted before commit.

Definition Core.18b (Attributed residual offer and residual-offer set).

At a reachable source K-facing cutpoint $\sigma \in \text{KCutReach_ref}(I)$, an attributed residual offer is a tuple $o = (P, x, q, c, \text{dir}, e)$ satisfying all of the following: P owns x ; $x \in \Omega_{\text{msg}, P}$; $q = \text{MsgOf_P}(x)$; $c =$

BoundaryOf_P(x); q is a blocking Push or Pop instance that has issued but not committed in reset epoch e; dir identifies the endpoint-owned producer-valid or consumer-ready direction; and BoundaryReq(x,σ) is asserted and attributed to that same x, q, c, direction, and epoch. Define the derived operation-kind projection $\text{kind}(o) := \text{kind}(q)$, with the typing invariant $\text{dir} = \text{producer-valid}$ iff $\text{kind}(q) = \text{Push}$ and $\text{dir} = \text{consumer-ready}$ iff $\text{kind}(q) = \text{Pop}$. Let $\text{ResOffers}_{E,w}(\sigma)$ be exactly the set of such tuples. $\text{ResOffers}_{E,w}(\sigma)$ is well typed and issue-prefix consistent under the existing E3/Axiom Core.IC semantics. On reachable source cutpoints, Core.SyncFence additionally ensures that no residual blocking offer owned by P belongs to a source interval earlier than P's current interval: such an offer would have had to process-facing commit before the intervening synchronization could commit. In particular: (a) its item and boundary fields are supplied by E; (b) if a source-later residual offer exists while an earlier blocking item in the applicable source prefix is also issued, uncommitted, and still requesting at an evaluated boundary, that earlier offer is also represented; (c) reset clears all offers in the ending epoch under RW2; and (d) for each explicit abstract boundary and endpoint direction there is at most one attributed outstanding source-level offer. A concrete token or transfer already accounted for as explicit channel occupancy is not additionally represented as an independent source-level residual offer unless E declares a distinct source-attributed frontier item at that boundary.

Definition Core.18c (K-observation KObs).

For the fixed instance I, elaboration E, witness w, and reachable source K-facing cutpoint $\sigma \in \text{KCutReach_ref}(I)$, define the K-observation record by $\text{KObs}_{E,w}(\sigma) := \langle E, w, H_{E,w}(\sigma), \text{ResOffers}_{E,w}(\sigma) \rangle$.

The fixed stimulus, environment model and reconstruction mode, reset policy, observation-boundary selection, and KObs-facing waiver set W are ambient parameters of the theorem instance I rather than duplicated in every record. K.Resp configuration and monitor ages are likewise ambient or CF-S operational data, not KObs fields. Equality of KObs records means typed equality of E and w and equality of the canonical replay-history quotient and residual-offer components. It does not require equality of the source and RTL clock-bucket decompositions permitted to differ by Theorem J.1. It is not equality of arbitrary process, scheduler, channel-realization, environment, response-monitor, or RTL microstate.

Definition Core.18c-1 (Observational waiver closure).

For fixed I,E,w, define $\text{KObsClosed}_{I,E,w}(W)$ to mean: for all $\sigma_1, \sigma_2 \in \text{KCutReach_ref}(I)$ of the same fixed instance, $\text{KObs}_{E,w}(\sigma_1) = \text{KObs}_{E,w}(\sigma_2)$ implies $(\sigma_1 \in W \text{ iff } \sigma_2 \in W)$. This is an observational-closure premise for any waiver-parameterized cutpoint predicate later claimed to be determined by KObs; it is not a claim that every well-typed KObs record is reachable. Environment-terminal adequacy and the optional diagnostic reconstructions are defined in Appendix K, where the relevant terminal predicates are introduced.

Definition Core.18d (Process and channel reconstruction from KObs).

For $K = \langle E, w, H, O \rangle$, define $\text{ProcRec}_P(K)$ to be the replay-derived process slice reconstructed by deterministic replay of P's projection of H under w: the source control location and reset epoch, source-visible local values needed by future predicates and payloads, completed/realized-item status, candidate NextFront, branch/guard outcomes, operation identities and per-channel ordinals, returned values, anchored signal facts, and source-level completion status. ProcRec does not contain the current asserted blocking-request inventory; PendingRec, BoundaryReqRec, and FrontRec are reconstructed separately from O. Define $\text{ChanRec}_c(K)$ for every explicit abstract channel c declared by E to be the E4 abstract state reconstructed from its reset/initial state and the committed Push/Pop history in H: for $B(c) > 0$, occupancy, empty/full status, and logical ordered contents/head information; for $B(c) = 0$, capacity-zero occupancy together with endpoint-request information supplied separately by O. For an

outer channel represented by $\text{Sys_B}^{\text{elab}}$, projected outer occupancy is obtained through Π_{elab} , while every proof-internal boundary used by Appendix K retains its own explicit ChanRec state.

Definition Core.18e (K-relevant derived functions).

For $K = \langle E, w, H, O \rangle$ define the following typed functions. $\text{DoneRec_P}(K)$, $\text{RemainRec_P}(K)$, and $\text{NextFrontRec_P}(K)$ are computed from $\Omega_{\text{P}}/\leq_{\text{P}}$ supplied by E and the realized-item history reconstructed from H . $\text{PendingRec_P}(K)$ is the set of frontier items x appearing in O for P . For records extracted from reachable source cutpoints, Core.SyncFence guarantees that every such outstanding blocking offer belongs to P 's current synchronization interval, so this unqualified offer projection is extensionally the interval-scoped Pending_P set. $\text{BoundaryReqRec}(x, K)$ holds exactly when O contains the attributed offer for x . $\text{FrontRec_P}(K) := \min_{\leq P}(\text{PendingRec_P}(K))$. $\text{ChannelEnabledRec}(x, K)$ and $\text{ChannelDisabledRec}(x, K)$ are computed at $\text{BoundaryOf_P}(x)$ from ChanRec and the complementary endpoint offers in O using Definitions Core.11–Core.13. $\text{WFGRec}(K)$ is obtained by applying Definition Core.WFG to FrontRec and $\text{ChannelDisabledRec}$. $\text{DrainClosedNowRec}(D, K)$ holds exactly when no already-offered non-frontier blocking item owned by a process $P \in D$ is ChannelEnabledRec at the cutpoint. Appendix K defines DeadKRec , waiver-parameterized deadlock reconstruction, environment-terminal reconstruction, WFG+ , Dead_KS , and response-monitor predicates from these Core functions and its own K -specific definitions; none of those later predicates is defined by the Core record alone. At this stage the Core functions are interpreted on records extracted from reachable source cutpoints; no image characterization is asserted.

Historical issue-time quotient. The record deliberately retains whether an operation survives as a current attributed offer, but not the earlier cycle in which that request first became asserted. Therefore two reachable source cutpoints with the same E , w , H , and O have the same KObs even if their operational executions used different legal historical issue times. This is only an observation equivalence at the selected cutpoint; it does not assert that the two executions have the same admissible continuations or satisfy the same transition-level DrainDiscipline for reasons independent of the common cutpoint.

Shared-memory and reset side conditions

Shared-memory lowering condition (J.Mem). The proof-internal observable alphabet Σ of Appendices I–K contains committed channel transfers, synchronization events, and signal Read/Write actions; it does not contain mapped memory read/write events, and the Theorem J.1 invariant carries per-process replay state and per-channel commit histories but no global shared-memory state. Accordingly, the formal results of Appendices I–K apply to cross-process shared-memory communication only under the following lowering condition, named J.Mem: every shared-memory dependency through which one process's memory write can affect another process's control flow, payload/value expressions, or the identity or order of its later operations shall be lowered by the array-access mapping layer onto operations already covered by the proof alphabet — explicit point-to-point message-passing channels governed by E4 (memory command/response channels), and/or Σ -visible anchored signal IO governed by E2 with the producing Writes observable — so that every memory-carried value is carried by a matched, value-labeled Σ event. Synchronization alone (E1) fixes access order but does not, within the present invariant, establish that the two models contain the same memory value; J.Mem closes that gap by making the value flow Σ -visible. Designs that instead rely on the non-lowered “declared memory semantics” branch of the shared-memory transformation entry in Appendix G are outside Theorem J.1, Corollary J.1, and Appendix K as proved here; for them, agreement under the declared memory semantics is a separate verification obligation (see the informative note below). J.Mem is a named entry in the side-condition discharge record. $K.CV$'s “declared memory serialization” clause remains a liveness-side premise and does not substitute for J.Mem in the safety proof.

Informative note (direct memory-state extension, not developed here). The alternative repair — adding mapped memory $\text{Read}(\text{addr}, \text{val})/\text{Write}(\text{addr}, \text{val})$ events to the proof-internal Σ , extending the required-order relation $<_J$ with the mapping layer’s declared serialization constraints, and adding a new global memory-state agreement clause to the Theorem J.1 invariant together with a memory-state component in the enriched replay/KObs reconstruction and the corresponding J.OfferReach/J.OfferConf interface — is compatible with the present proof architecture but is not developed in this document. A flow that requires direct coverage of non-lowered shared memory shall supply that extension and re-verify Lemmas I.DR/I.DR’, J.HCanon, and J.KObs-Proc–J.KObs-WFG and the corresponding Appendix K deadlock-factorization result, together with Corollary J.1, with the added clause.

Reset-boundary observable unit (RESET) and reset well-formedness (RW1–RW5). When dynamic reset is used, each dynamic reset occurrence is represented in the proof-internal event model as an explicitly atomic observable unit $\text{RESET}(S_p, S_c)$: a proof-internal boundary marker whose scope names the set S_p of processes and the set S_c of explicit abstract channels affected by that reset occurrence. RESET units may be projected away from the outer Σ_{DUT} alphabet, like other proof-internal units. The unit is ordered strictly after every observable unit of the ending epoch owned by the affected scope and strictly before every observable unit of the new epoch of that scope, and the first observable action of each $P \in S_p$ in the new epoch is its fresh dynamic START_P . The state-changing reset transition itself — which occurs before the new epoch’s START_P — is exactly what the RESET unit represents. In Core.CycleResult that dynamic occurrence is carried in resetUnits of the enclosing real cycle rather than replacing the cycle with a reset outcome; it therefore retains its own correspondence obligation (Lemma J.RS in Appendix J) rather than being folded into START_P matching. The following reset well-formedness conditions are implementation-side obligations for designs using dynamic reset:

RW1 (channel-consistent scope). For every explicit abstract channel c , either both endpoint processes of c are in S_p and $c \in S_c$ — in which case c re-initializes to reset-empty occupancy (A10) at the boundary and any in-flight (accepted but not yet delivered) contents are discarded consistently on both sides in both models — or neither endpoint is in S_p and $c \notin S_c$, in which case c ’s state and history are unaffected. A reset that would affect exactly one endpoint of a live channel is permitted only if the design models the cross-epoch disposition explicitly (for example, an explicit flush/abort protocol on that channel); that protocol’s events are ordinary Σ events, and the channel is treated per this clause after the protocol completes.

RW2 (request disposition). Every outstanding endpoint-owned boundary request of a process in S_p is cleared at the RESET unit; its owning dynamic instance does not commit, is not re-attributed across the boundary, and any post-reset re-execution is a fresh dynamic instance in the new epoch’s universe. The Appendix C non-withdrawal discipline is scoped per reset epoch.

RW3 (corresponding reset transitions). At the RESET unit, RTL_B ’s post-reset state at the chosen abstract boundary corresponds to Sys_ref ’s prescribed epoch-initial state: control at epoch-initial locations for every $P \in S_p$, source-visible values at their reset values, reset-empty occupancy on every $c \in S_c$, no stale synchronization state, no unaccounted pending endpoint request, no hidden accepted message absent from the source reference model, and direct-input signals obeying their per-epoch stability or sync-controlled contracts (main text, Direct Inputs).

RW4 (atomic boundary). The reset state transition is not interleaved with same-cycle observable actions of the affected scope. If the implementation asserts reset in a cycle in which affected-scope observable actions also commit, those actions belong to the ending epoch and precede the matching RESET occurrence in $\text{Core.CycleResult.resetUnits}$ in the required order; the two reset epochs of the affected scope share no observable cycle bucket ambiguity at the boundary. The enclosing CycleResult may still carry unaffected-scope progress because RW5 frames state outside the reset scope.

RW5 (locality). The RESET unit changes no state of processes outside S_p and no history or occupancy of channels outside S_c . Cross-scope effects, if any, must be expressed by ordinary modeled events.

Additional Semantic Assumptions (B-rules)

The following B rules are process assumptions used within the formal proofs. These B rules are in addition to the R rules presented in Appendix G.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
B1 Deterministic Trace Property	For any fixed test stimulus and initial state, the sequence of observable actions emitted by the post-HLS design is unique, including the channel identity and payload/value of each committed $\text{Push}_c(v)/\text{Pop}_c(v)$ and the value of each $\text{Write}(\text{sig}, \text{val})/\text{Read}(\text{sig}, \text{val})$. Internal ϵ-steps may differ between runs, but the externally visible Σ-trace (with labels) cannot. Clarification: B1 is assumed of RTL_B (post-HLS) only. The source-level models Sys/Sys_B may admit multiple ϵ-/latency-different executions with the same Σ-projection. When a single concrete witness execution of Sys_B is required for simulation/debug, Appendix L's snooping wrapper may be used to select one admissible witness whose latency-sensitive events align with the unique RTL Σ-trace. Derived per-process consequence. For every process P, the projection of the unique RTL_B Σ-trace onto the observable units owned by or attributed to P is also unique. This follows from the system-level B1 property; it does not assert that P is deterministic when considered in isolation under arbitrary peer, channel, arbitration, or environment behavior.	Ensures the per-process and system-level trace-compatibility proofs (App. I & J) can match one post-HLS trace to one pre-HLS trace without branching on scheduler nondeterminism.
B2 Weak Fairness of the Scheduler (WF)	If an action's enabling predicate remains continuously true from cycle t onward, the scheduler/arbiter must eventually select that action, and the selected action then commits in the selected cycle, within a	Required for all progress arguments, especially the liveness lemmas in Appendix K and the channel-progress corollaries.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	finite but unspecified number of cycles. Applies to Push, Pop, and Sync operations. For message-passing operations, “enabled” means ChannelEnabled under E4 at the chosen abstract boundary. Enablement is eligibility for selection; it is not itself a same-cycle commit. Thus B2 is not vacuous: an enabled Push/Pop may be delayed by fair arbitration, but it may not be delayed forever while it remains continuously enabled.	
B3 Channel Progress Invariants	B3 is exactly the following family of per-channel progress obligations. For every explicit abstract channel c with capacity $B(c)$, B3 consists of both $P_push(c)$ and $P_pop(c)$, as defined below. Thus references such as “B3,” “Appendix N’s B3,” and “the P_push/P_pop obligations” name the same channel-progress assumption; $P_push(c)$ and $P_pop(c)$ are not additional assumptions separate from B3. B3 is stated separately from B2 because the proofs often reason from the perspective of a currently stalled request. A blocked $Push_c$ at $occ_c = B(c)$, a blocked Pop_c at $occ_c = 0$, or a rendezvous endpoint waiting for its peer is not itself able to complete until the relevant complementary discharge occurs. B3 packages that discharge fact as a named per-channel progress obligation: under the persistent-request premises below, the full/empty/rendezvous-blocked condition is eventually discharged, after which ordinary E4 availability and B2 can be used for any still-pending enabled transfer.	Required by complete/fair progress arguments, principally Appendix K, to rule out permanent back-pressure loops in which both endpoints continuously request a transfer until the matching discharge event but the matching discharge never occurs; such progress is logically independent of the cycle-by-cycle R-rules. It is not a premise of finite-prefix Theorem J.1.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	For a single explicit abstract boundary with ordinary E4 buffered or rendezvous semantics, the corresponding B3 discharge is implied by B2 plus E4 once the complementary endpoint request in the B3 premise remains continuously asserted. In the buffered full case, $occ_c = B(c) > 0$ makes the complementary Pop_c continuously ChannelEnabled; B2 then forces eventual selection and commit of that Pop_c, freeing space. In the buffered empty case, $occ_c = 0 < B(c)$ makes the complementary Push_c continuously ChannelEnabled; B2 then forces eventual selection and commit of that Push_c, providing data. In the rendezvous case, the two persistent endpoint requests make the rendezvous pair continuously ChannelEnabled, so B2 similarly supplies eventual rendezvous selection and commit. B3 is nevertheless retained as an explicit named obligation because Lemma I.2, Appendix K, and the channel-validation methodology consume that modular form directly. It is also the form checked per explicit abstract channel in elaborated Sys_B^{elab} boundary chains. Thus B3 should be read as the modular channel-progress assumption used by the proof and checked against channel realizations, not as a claim that the ordinary single-boundary buffered clauses are logically independent of B2+E4. In the definitions below, “persistent,”	

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	“continuously requests,” and “remains true” are interval-scoped. The relevant endpoint request must remain asserted from the starting cycle t_0 through the first matching discharge event, if such a discharge occurs. If no matching discharge occurs, the request must remain asserted for all later cycles. A Push request also keeps its payload stable throughout the outstanding interval. B3 does not require an endpoint request or Push payload to remain asserted forever after the matching transfer has already committed. • $P_push(c)$: If a process P is stalled at a blocking $Push(c)$ with its boundary request true and a persistent request asserted with stable payload while the channel is full ($occ(c) = B(c)$), and the complementary endpoint process continuously requests the matching $Pop(c)$ until the matching discharge event ($Pop(c)$ endpoint request asserted; $Pop(c)$ boundary request remains true), then the matching full/rendezvous discharge must eventually occur. In particular, for $B(c) > 0$, some $Pop(c)$ eventually commits, freeing space; for $B(c) = 0$, the rendezvous completion eventually occurs. In the buffered case this conclusion is the complementary $Pop(c)$ commit, not a direct assertion that the original stalled $Push(c)$ commits; E4 and B2 govern that still-pending $Push(c)$ after space is freed. • $P_pop(c)$: If a process P is stalled at a blocking $Pop(c)$ with its boundary request true and a persistent request asserted while the channel is empty ($occ(c) = 0$), and the complementary endpoint process continuously requests the matching $Push(c)$ until the matching	

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	discharge event ($\text{Push}_{(c)}$ endpoint request asserted with stable payload; $\text{Push}_{(c)}$ boundary request remains true), then the matching empty/rendezvous discharge must eventually occur. In particular, for $B(c) > 0$, some $\text{Push}_{(c)}$ eventually commits, providing data; for $B(c) = 0$, the rendezvous completion eventually occurs. In the buffered case this conclusion is the complementary $\text{Push}_{(c)}$ commit, not a direct assertion that the original stalled $\text{Pop}_{(c)}$ commits; E4 and B2 govern that still-pending $\text{Pop}_{(c)}$ after data is provided.	
B4 Finite Stutter Bound	Every process P has a finite constant depth D_P such that whenever P is not externally stalled (i.e., P is advancing internal micro-state toward its next observable Σ-action, or its next Σ-action is locally enabled), P can execute at most D_P consecutive ϵ-steps before either (i) executing a Σ-action, or (ii) reaching a stable waiting state in which P has no further ϵ-step available until some external/peer/environment condition changes the enabling predicates.	This guarantees the ϵ-stutter closure needed to construct witness mappings in Appendix I and to bound internal micro-latency (pipeline/FSM bookkeeping) by $\leq D_P$. Unbounded waiting due to back-pressure or missing peer stimulus is governed by WF/B2 and the progress obligations B3, not by B4.
B5 System Quiescence Closure	B5 carries a finite system-level cycle bound C_{sys} for the selected bucket semantics. A standard discharge defines per-process cycle bounds C_P and takes $C_{\text{sys}} = \max_P C_P$. Each C_P may be supplied independently or derived from the B4 epsilon-step bound D_P by a proved charging lemma showing that every relevant silent cycle in which P remains internally advancing consumes at least one P-owned epsilon-step; under that lemma $C_P \leq D_P$. If, at some cycle t_0, no observable actions are enabled in any process, and no external or environmental change from t_0 through the closure interval changes the relevant enabling predicates, then within at most C_{sys} further cycles the	Needed to finish the starvation-vs-deadlock analysis in Appendix K: ensures an all-disabled state cannot hide behind an infinite tail of unobservable activity.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	system reaches an epsilon-fixed point. Thereafter, while no external or environmental change re-enables observable or internal work, the system executes no additional epsilon-steps.	
B6 Time-Divergence / Idle-Stutter Convention	After Core.TerminalIdle is established — a genuine B5/global fixed point together with proof that neither future registered/internal machine evolution nor any permitted future environment evolution can re-enable work — the global clock continues to advance. Core.EnvTerminal supplies the environment component of this obligation. We model each subsequent B6 idle clock tick as a stutter step that produces no Σ-event, performs no ϵ-prefix or ϵ-suffix work, and leaves all non-clock state unchanged. A present-instant observation that no action is enabled is not sufficient to establish this antecedent.	When a theorem explicitly represents a genuinely terminal-idle execution by an infinite B6 suffix, those idle ticks are treated as ϵ-stutter for the applicable E1–E5 prefix/trace matching. Thus B6 gives a time-divergent infinite extension only for a reachable state already proved to satisfy Core.TerminalIdle. Core.CycleClosure's environment-definedness clause or Core.EnvTerminal alone is insufficient: a total timed/reactive environment may legally deliver new stimulus at a later cycle. B6 is not an arbitrary deadlock-cutpoint or temporary-idle extension principle. Ordinary nonterminal cycles in which no Σ event commits are represented by Core.SilentCycle/CompleteCycle and may advance registered state, environment state, and cycle-valued monitors. A partial internal message-passing deadlock cutpoint in Appendix K need not be a global fixed point because outside-D activity, environment activity, or already-issued non-frontier drain work may still be possible. Such cutpoints are handled by the separate Appendix K complete-cycle/fair drain continuation; B6 is used only for a genuine terminal-idle suffix.

Appendix G – Trace Compatibility Rules

This document informally states that a primary goal is to present “no surprises” to a DV engineer using the same testbench to verify the pre-HLS and post-HLS models.

Somewhat more formally, we can show how the scheduling rules within this document establish a set of trace-compatibility rules between the pre-HLS and post-HLS models that provide strong guarantees about their behaviors.

The *basic conceptual model* establishes the base behavior for a single process:

- 1) Synchronization interface calls always remain in source code order.
- 2) Signal reads occur at closest preceding synchronization interface call.
- 3) Signal writes occur at closest succeeding synchronization interface call.
- 4) All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (For blocking message-passing operations (Push/Pop), this implies the corresponding commits occur no later than the synchronization call's commit cycle.)
- 5) All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
- 6) Two message-passing operations on separate interfaces which appear in sequence in the model are source-ordered. They may issue with timing overlap or in the same clock cycle where the selected issue policy allows, but they shall not be issued in the reverse sequence. Operations on the same message-passing interface are issued in source order (they are never reversed).
- 7) Synchronization calls can affect the scheduling of signal IO operations, and synchronization calls can affect the scheduling of message-passing calls, but message-passing calls cannot affect the scheduling of signal IO operations and vice-versa.
- 8) The STRICT_IO_SCHEDULING=relaxed option is not used.
- 9) We distinguish the rendezvous source model Sys (B(c)=0 for all channels) from the buffered source interpretation Sys_B, which uses bounded-FIFO channel capacities B(c) matching the post-HLS RTL (Appendices I–K).

Conceptually, systems are composed of many such processes which follow the basic conceptual model, and which communicate using only synchronization interface calls, latency-insensitive signal-level protocols, and committed message-passing transfers. If a design uses non-blocking message-passing calls (PushNB/PopNB), then unsuccessful polls are treated as internal ϵ -steps (no Σ -event), and each successful completion is represented in Σ as the corresponding committed Push_c/Pop_c transfer (Σ records transfers, not failed polls). If the resulting latency-sensitive behavior is externally visible at the DUT boundary, the snooping wrapper of Appendix L may be used to align the selected admissible pre-HLS execution with the RTL. Here we call this system of processes the *conceptual system*.

Practically, the modeling approach described above is too restrictive for real-world systems with optimized HW, so the scheduling rules in this document allow for key HW optimizations. But this is done in a carefully controlled manner, such that the HW optimizations do not break equivalent behavior with the *conceptual system*.

Here is a summary of the HW optimizations that the scheduling rules allow, and a brief description of how we maintain equivalent behavior with the *conceptual system*:

1. Pipelined loops: In the post-HLS RTL, the pipeline may overlap iterations to improve throughput. See Appendix B for a formal presentation of pipelined loops.
2. Pipelined loops: HLS by default will not break any latency-insensitive signal IO protocols when pipelining loops.
3. Direct inputs: Signals that do not change after a process comes out of reset can be read as late as possible using *hls_direct_input*, saving register area.

4. Direct inputs: When *hls_direct_input_sync* is used, signals read by a process are updated at the precise point at which the HW pipeline is guaranteed to be ramped down, so the signal values do not need to be saved within pipeline registers, saving register area.
5. Memory accesses: HLS can reorder memory accesses within a process only if it can prove that the reordering is conflict-free.
6. Shared memories: Memories shared between processes are modeled as simple C arrays but always include explicit synchronization between processes in both the pre-HLS and post-HLS models. For the formal results of Appendices I–K, the value-carrying accesses must additionally satisfy the J.Mem lowering condition (Normative Formal Core).
7. Non-blocking message-passing interfaces: They are modeled at the level of committed transfers (successful completion), with unsuccessful polls treated as ϵ -steps and thus not part of Σ . If latency differences in these committed-transfer events are externally visible at the DUT boundary and must be aligned for verification, equivalent behavior between pre-HLS and post-HLS systems can be maintained by snooping the post-HLS system and using this to delay pre-HLS message arrival (Appendix L).
8. Latency-sensitive global signal IO: If latency differences are externally visible to the DUT, equivalent behavior between pre-HLS and post-HLS systems can be maintained by snooping the post-HLS system and using this to delay pre-HLS signals.
9. Latency-sensitive local signal IO: Isolate local latency-sensitive protocols to small, dedicated transactors, and use latency-insensitive signal IO or message-passing to communicate with the rest of the system.
10. One-way signal handshake protocols: Only use in cases where backpressure is not possible and embed assertions into the model to check that this is always true.
11. Combinational processes: If the pre-HLS model contains combinational processes, their observable signal Read/Write behavior is interpreted at each cycle's ϵ -quiescent combinational fixed point. Under the well-formedness requirement that the combinational network is deterministic and reaches a unique fixed point, the RTL/tool-flow conformance obligation is to preserve the fixed-point signal relation, so combinational signal IO can be included in Σ without using synchronization anchors.

Taken as a whole, these rules support a verification methodology in which full-system verification is performed primarily on the pre-HLS system model, and post-HLS RTL verification can be reduced to targeted checks that the formal side conditions are met. The precise guarantee is the execution-level guarantee of Appendix J: for every reachable RTL_B prefix whose local certificate package satisfies Definition J.LocalGWR and whose selected source model satisfies the source-side Core.QSync premise required by J.GWR-Comp, Theorem J.GWR-Comp constructs a globally witness-realizable package and a matching Sys_ref behavior (Post $\rightarrow$ Ref). A directly validated J.GWR package carrying equivalent per-Horizon settling evidence is an alternate route. The reverse direction applies only to annotated RTL-consistent source-side prefixes or to source-side executions selected by the Appendix L snooping / witness-selection mechanism. Thus a source-side pass justifies the corresponding RTL_B behavior only within that annotated RTL-consistent / witness-selected subset. For liveness/deadlock guarantees, the

results additionally assume the Appendix K/N progress, fairness, drain, and A5 premises; the cycle-by-cycle R/E rules and the safety construction alone do not imply those liveness properties.

Formalization of the Trace Compatibility Rules in Appendix G

The following notation is used in this section.

Symbol	Meaning
P	A process (thread or method) in the design
Σ	The alphabet of observable IO actions
τ_P	A (finite or infinite) per-cycle trace of observable actions executed by process P: for each global clock cycle t , $\tau_P[t]$ is the (possibly empty) set of Σ -actions committed by P at t . Actions committed in the same cycle are treated as simultaneous (unordered); only cross-cycle order (and the explicit E1–E5 constraints) is semantically significant.
S	The subset of Σ that are synchronization calls (explicit wait, SyncChannel, START_P/FINISH_P indicating process start/finish)
R	The subset of Σ that are signal reads
W	The subset of Σ that are signal writes
M	The subset of Σ that are committed message-passing transfers (i.e., committed Push_c / Pop_c events on message-passing channels, including successful non-blocking transfers). Unsuccessful non-blocking polls contribute no Σ -event and are modeled as ε -steps, and therefore are not members of M .
Q_{exec}	The execution-level set of dynamic source-level message-call instances executed by a process at the Σ -visible endpoints (blocking Push/Pop, and non-blocking PushNB/PopNB calls). Elements of Q_{exec} may or may not commit. In particular, failed non-blocking polls are elements of Q_{exec} , have no committed Σ -event, and are modeled as ε -steps. The frontier-item universe Ω_P , defined below, is separate from Q_{exec} .
Ω_P	The local dynamic frontier-item universe for process P in the selected execution / witness. It contains synchronization-call items, signal-action items, and message-operation frontier items; it is distinct from $Q_{\text{exec},P}$, the universe of dynamic source-level message-call instances.
$\Omega_{\text{msg},P}$	The message-operation slice of Ω_P : frontier items x for which $\text{MsgOf}_P(x)$ is defined.
$\text{MsgOf}_P(x)$	Partial projection from a message-operation frontier item $x \in \Omega_{\text{msg},P}$ to its owning dynamic source-level message-call instance $q \in Q_{\text{exec},P}$. In $\text{Sys_B}^{\text{elab}}$, MsgOf_P need not be injective: one source-level q may own several elaborated frontier items.
$\text{BoundaryOf}_P(x)$	The explicit abstract channel boundary at which frontier item x is evaluated for BoundaryReq, ChannelEnabled, ChannelDisabled, Front_P, and Appendix K WFG reasoning. In ordinary unelaborated cases this boundary is unique for the

Symbol	Meaning
	owning source-level operation; in Sys_B ^{elab} it is declared by the elaboration package.

A *system trace* is the tuple
 $\tau = (\tau_P)_P$, one component per process.

For any action $a \in \Sigma$, let $\text{clk_pre}(a) / \text{clk_post}(a)$ be the clock cycle at which a is committed.

For any issued message-call instance $q \in Q_{\text{exec}}$, $\text{issue_pre}(q) / \text{issue_post}(q)$ denotes the auxiliary Issue timestamp for q in the pre-HLS / post-HLS trace, respectively, as defined in Appendix C and required to satisfy the associated Issue/Commit linkage (well-formedness) conditions with respect to the Σ -visible boundary request signal on q 's endpoint direction.

If q commits, it contributes a unique committed transfer $m(q) \in M$ with commit cycle $\text{clk_pre}(m(q)) / \text{clk_post}(m(q))$. If q does not commit (e.g., an unsuccessful non-blocking call), then $m(q)$ is undefined / absent and no Σ -event is added to M .

(Q_exec-matching / witness convention.) Because unsuccessful non-blocking polls are ϵ -steps and produce no Σ -event, Q_{exec} is not in general reconstructible from the Σ -projection alone. Whenever E3, E5, or any later proof compares executed message-call instances across Sys_ref and RTL_B, the comparison is made relative to the chosen execution witness. That witness supplies a partial matching μ_Q between dynamic source-level message-call instances in the compared traces, preserving process identity, endpoint/interface identity, dynamic source-order position, and source-level call-instance identity where applicable. If q commits, then $\mu_Q(q)$ commits the corresponding matched transfer $m(\mu_Q(q))$ with the same Σ -label/value under the event-matching convention. If q is an unsuccessful non-blocking poll, then q has no $m(q)$ and no Σ -label; its presence and its match are supplied only by the witness-compatibility condition WC, by the sufficient NB-CF condition when applicable, or by the Appendix L snooping / witness-selection construction. Thus E3/E5 quantify over Q_{exec} only relative to this chosen witness; they do not require failed non-blocking polls to be inferable from Σ alone.

(No deassert-before-commit assumption.) For blocking message requests $q \in Q_{\text{exec}}$, once q is issued, its Σ -visible boundary request is not withdrawable within that reset epoch: it remains asserted in every cycle from $\text{issue_pre}(q)$ (resp. $\text{issue_post}(q)$) through q 's commit cycle if it commits, and for Push the payload remains stable while the request is outstanding. If an affecting RESET occurs first, RW2 ends the dynamic instance at the epoch boundary; no other nonterminal transition may withdraw or reattribute it.

Notation convention (Issue/Commit functions). The function-like forms $\text{clk_pre}/\text{clk_post}$, $\text{issue_pre}/\text{issue_post}$, and $m(q)$ abbreviate the unique values supplied when the corresponding Appendix C existence/uniqueness and Issue/Commit-linkage conditions hold. In particular, $m(q)$ remains partial: a rule that requires a committed transfer must also require or entail that $m(q)$ is defined. This notation does not replace the underlying relational Issue/Commit semantics.

Dynamic source-order convention

The order relation used below is over dynamic source-level operation/frontier items in the compared execution / witness, not over all syntactic call sites in the source text.

For a process P , let $Q_{\text{exec},P}$ denote the execution-level set of dynamic source-level message-call instances of P at the chosen abstract boundary. $Q_{\text{exec},P}$ includes blocking Push/Pop calls, successful

non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Separately, let Ω_P denote the witness-specific dynamic universe of source-level frontier items that are actually reached under the fixed stimulus / selected witness and that are relevant to observable/frontier/deadlock reasoning. Items in Ω_P include synchronization-call instances, anchored signal Read/Write action instances, and message-operation frontier items at the chosen abstract boundaries. A message-operation frontier item is either (i) a blocking Push/Pop item that has been reached, whether pending or committed, or (ii) a successful non-blocking PushNB/PopNB item that contributes a committed Push_c(v) / Pop_c(v) event. A failed non-blocking poll q is an element of Q_exec, P , has no committed Σ -event $m(q)$, and is not an element of Ω_P merely because it executed.

Because Q_exec, P is a universe of dynamic message-call instances while Ω_P is a universe of source-level frontier items, their message-operation overlap is not a literal set intersection. Let $MsgOf_P(x)$ be the partial projection from a frontier item $x \in \Omega_P$ to its owning dynamic message-call instance, defined exactly for message-operation frontier items and undefined for pure synchronization or signal-anchor items. Define the message-frontier slice

$$\Omega_msg, P := \{ x \in \Omega_P \mid \exists q \in Q_exec, P : MsgOf_P(x) = q \}.$$

Define the corresponding message-instance slice induced by Ω_P as

$$Q_Q, P := \{ q \in Q_exec, P \mid \exists x \in \Omega_msg, P : MsgOf_P(x) = q \}.$$

Let Σ_P denote the corresponding observable labels/events contributed by items of Ω_P when they are Σ -visible. For a message-call instance $q \in Q_exec, P$, the committed Σ -event $m(q) \in M$ exists only if q commits; if q is a failed non-blocking poll, $m(q)$ is absent and q contributes no Σ -event. For a message-operation frontier item $x \in \Omega_msg, P$ with $MsgOf_P(x)=q$, item x contributes the committed Σ -event $m(q)$ only if q commits. A pending blocking message-operation frontier item x may therefore be in Ω_P and Ω_msg, P , and its owning message instance q may be in Q_Q, P , even though $m(q)$ is not yet present.

Operations on untaken branches and unexecuted loop iterations are not members of Ω_P , Ω_msg, P , Q_Q, P , or Q_exec, P . Distinct loop iterations, unrolled instances, and repeated calls are represented by distinct dynamic instances.

Default source order for ordinary message calls. For dynamic items x and y of process P , write $x <_{src} y$, or equivalently $x <_P y$ when the process must be explicit, to mean that P 's source control flow and the R-rules force x to precede y in that dynamic execution / witness. For ordinary user-written source-level message-call instances in a sequential process, this is the dynamic program order along the executed control path. Thus, if two executed message-call instances $q_1, q_2 \in Q_exec, P$ appear in sequence in the same process, exactly one of $q_1 <_{src} q_2$ or $q_2 <_{src} q_1$ holds according to that dynamic program order, including when q_1 and q_2 use distinct message-passing interfaces, occur in the same synchronization interval, arise from repeated loop iterations, or arise from unrolled instances whose source-level expansion is ordered by the selected source elaboration. Same-cycle issue, same-cycle commit, or issue of a later operation before an earlier operation commits is a timing/issue-policy freedom; it does not make ordinary source message-call instances incomparable. The no-reverse rule R4/E3 is therefore stated over this ordered dynamic source relation.

Limited sources of incomparability. The relation $\leq_P$ used for frontier reasoning is the reflexive closure

of the restriction of this source-induced order to Ω_P , extended only by explicitly modeled partial-order structure. Ordinary distinct-interface source calls are not incomparable merely because they are on different interfaces. A message-operation multi-frontier may arise only from explicit $\text{Sys_B}^{\text{elab}}$ proof-internal structure such as staged/bundle/join/epoch-gate abstract channels whose elaboration package declares the relevant frontier items, evaluated boundaries, partial order, and projection/attribution facts. Such elaborated frontier items are part of the selected Sys_ref proof model; they are not an implicit weakening of source order among the ordinary user-written message calls.

For sequential-process signal IO, $<_{\text{src}} / <_P$ is interpreted through the R2 anchor semantics rather than through the raw lexical statement location of the signal read or write. Thus a signal Read is ordered, for formal frontier and trace-compatibility purposes, at its pred_S anchor, and a signal Write is ordered, for formal frontier and trace-compatibility purposes, at its succ_S anchor. A lexically earlier signal Write whose succ_S anchor is a later synchronization call does not, merely by lexical placement, precede or block intervening message-operation instances in the same source interval; conversely, a lexically later signal Read whose pred_S anchor is an earlier synchronization call does not, merely by lexical placement, wait behind intervening message-operation instances. Signal IO may constrain message IO only through the explicit synchronization anchors and the stated R/E rules, not by raw lexical adjacency inside an interval.

Conceptual Model for a Single Process

For every process P the pre-HLS trace τ_P must satisfy the following *partial-order constraints* over these dynamic operation instances:

R1 (Program order of S).

$$\forall s1, s2 \in S : s1 <_{\text{src}} s2 \Rightarrow \text{clk_pre}(s1) < \text{clk_pre}(s2)$$

R2 (Location of R/W for sequential processes).

For a sequential process P, ordinary signal Read/Write events are anchored to synchronization events as follows:

$$\begin{aligned} \forall r \in R_P : \text{clk_pre}(r) &= \text{clk_pre}(\text{pred_S}(r)) \\ \forall w \in W_P : \text{clk_pre}(w) &= \text{clk_pre}(\text{succ_S}(w)) \end{aligned}$$

where $\text{pred_S}(r)$ (resp. $\text{succ_S}(w)$) is the closest dynamic synchronization-call instance that precedes (resp. follows) the signal Read/Write on the dynamic source path of the current execution / witness, after applying the anchoring convention above. This anchoring determines the signal event's formal position for $\leq_P / <_P$, Done_P , Remain_P , and NextFront_P reasoning. In particular, for a sequential-process signal Write w, the fact that the write statement appears lexically before a message operation q in the same synchronization interval does not imply $w <_P q$ when w is anchored to the succeeding synchronization call; the write's formal observable event is placed at $\text{succ_S}(w)$. Likewise, a signal Read anchored to $\text{pred_S}(r)$ is not delayed by intervening message operations merely because the read statement appears lexically after them.

Every observable signal Read/Write of a sequential process must have a real bounding synchronization anchor on the dynamic path on which it occurs. A process-start synchronization event START_P may serve as the initial predecessor anchor when the modeling convention explicitly includes START_P in S. A

terminating process event FINISH_P may serve as the terminal successor anchor when the process actually terminates and FINISH_P occurs. If no real predecessor anchor exists for a Read, or no real successor anchor exists for a Write on the dynamic path being modeled, the sequential-process signal IO is ill-formed for this formal model or lies outside the finite observable prefix being compared. No observable Read/Write event is assigned clock time ∞ .

R2C (Combinational signal IO at the ϵ -fixed point).

A combinational process P has no process-owned synchronization events and no message-passing events in this formal clause; it may, however, contribute observable signal $\text{Read}(\text{sig}, \text{val})$ and $\text{Write}(\text{sig}, \text{val})$ events to Σ through explicitly selected R2C observation components. These events are not anchored by $\text{pred_S}/\text{succ_S}$. In every clock cycle t , the global state first reaches the cycle's ϵ -quiescent combinational fixed point after all sequential clock-boundary updates and all internal combinational propagation for that cycle have settled. Let μ_t denote this unique fixed-point valuation of the signal network.

R2C cycle-entry valuation. The fixed point μ_t is reached from the cycle- t entry valuation established after the applicable sequential clock-boundary updates, with the registered/process-owned request and signal drives and the external/direct-input values selected for cycle t fixed. Internal combinational propagation then settles from that valuation without advancing the architectural cycle.

R2C unit-generation convention

Fix an explicitly modeled combinational process or combinational network component C_{comb} and its selected observable slice $\text{Obs}(C_{\text{comb}})$, consisting of the process/signal/direction labels whose fixed-point values are included in that component's R2C observation. Define $\text{ComponentParticipatesInCycle}(C_{\text{comb}}, t)$ to mean that C_{comb} is active in the selected reset epoch and $\text{Obs}(C_{\text{comb}})$ is defined at the cycle- t ϵ -quiescent fixed point μ_t ; when cycle-locked observation is used, the selected observation/elaboration package shall define this participation predicate explicitly. The fixed-point valuation μ_t exists every cycle, but for an ordinary non-cycle-locked component a Σ -visible R2C fixed-point signal-observation unit is emitted only at canonical observation occurrences: (i) the first cycle t for which $\text{ComponentParticipatesInCycle}(C_{\text{comb}}, t)$; (ii) any later participating cycle in which the vector of fixed-point values on $\text{Obs}(C_{\text{comb}})$ differs from the vector in the most recent emitted R2C unit; or (iii) any participating cycle in which an explicitly modeled observer/sampler requires a fresh observation unit. Participating cycles in which the observable fixed-point slice is identical to the most recent emitted unit and no explicit observer/sampler requires a fresh unit are R2C stutter cycles for C_{comb} and emit no new Σ -visible unit. Cycles in which the component does not participate likewise emit no C_{comb} unit. Delta-cycle re-evaluation, intermediate combinational values, redundant re-drive before the fixed point, and ordinary R2C stutter cycles are $\epsilon/\text{stutter}$ for this component and are not separately Σ -visible.

R2C cycle-locked completeness.

If C_{comb} is declared cycle-locked, then for every cycle t satisfying $\text{ComponentParticipatesInCycle}(C_{\text{comb}}, t)$, exactly one R2C fixed-point observation unit for C_{comb} shall be emitted from μ_t , even when $\text{Obs}(C_{\text{comb}})$ is unchanged and no separate observer/sampler trigger occurs. Conversely, no cycle-locked unit is emitted for a cycle in which $\text{ComponentParticipatesInCycle}(C_{\text{comb}}, t)$ is false. This is a positive completeness obligation as well as an emission guard: proving only that every emitted unit occurs in a permitted cycle is insufficient if a required participating-cycle unit is omitted.

If a combinational process P reads sig in an emitted R2C unit of C_{comb} , the corresponding label is $\text{Read}(\text{sig}, \mu_t(\text{sig}))$ for the cycle t at which that unit is emitted. If P drives sig in an emitted R2C unit of C_{comb} , the corresponding label is $\text{Write}(\text{sig}, \mu_t(\text{sig}))$, where $\mu_t(\text{sig})$ is the final resolved fixed-point

value of sig at the chosen observation boundary. All Read/Write labels inside one emitted R2C unit are interpreted at the same ϵ -quiescent fixed point and may not be split.

Combinational-process signal IO is well formed only when the relevant combinational network is deterministic and reaches a unique ϵ -fixed point in each cycle being compared, and when each observable R2C component has a well-defined selected observable slice $\text{Obs}(C_comb)$. For a multi-input combinational component whose inputs may be updated by independent latency-insensitive sources, the component is Σ -observable under the stutter-canonicalized R2C rule only if the chosen observation boundary provides a single explicit anchoring domain for the slice, or if an additional component-specific well-formedness proof shows that all admissible interleavings induce the same emitted R2C unit sequence. Otherwise the component's internal combinational propagation must remain ϵ -hidden, or the design must expose the relevant behavior through explicit synchronization, message passing, a cycle-accurate transactor, or a cycle-locked R2C observation boundary. Pure combinational oscillations, ambiguous multiple-driver resolution, hidden stateful behavior, or intentional combinational-loop behavior outside a unique fixed-point interpretation are outside this R2C model unless represented by explicit state or synchronization elsewhere in the formal model.

R2C determinism clarification. Here “deterministic” means that fixed cycle-entry registered state/drives and fixed cycle-t inputs determine one ϵ -quiescent fixed-point valuation for the selected observation network. It does not require the composed internal combinational microstep relation or evaluation order itself to be functional. Value-preserving re-evaluation, intermediate values, and redundant re-drive may be ϵ /stutter provided they do not change the unique settled result.

R3 (Isolation around S).

Let s be a dynamic synchronization-call instance of process P . Let $\text{pref_S}(s)$ be the set of executed dynamic message-call instances $q \in Q_exec, P$ in the immediately preceding source interval of P whose source-induced order places q before or at the boundary controlled by s . Let $\text{suff_S}(s)$ be the set of executed dynamic message-call instances $q \in Q_exec, P$ in the immediately following source interval of P whose source-induced order places q after s . Then

$\forall q \in \text{pref_S}(s) : \text{issue_pre}(q) \leq \text{clk_pre}(s) \quad \text{and} \quad \forall q \in \text{suff_S}(s) : \text{issue_pre}(q) > \text{clk_pre}(s).$

$\forall q \in \text{pref_S}(s)$ with q a blocking Push/Pop : $m(q)$ is defined and $\text{clk_pre}(m(q)) \leq \text{clk_pre}(s).$

The third clause is the synchronization-completion fence of Core.SyncFence ; it permits a blocking transfer and s to commit in the same cycle. In $\text{Sys_B}^{\text{elab}}$, $m(q)$ here is the designated process-facing commit that completes/releases q , not an arbitrary internal elaboration transfer. Failed non-blocking polls in Q_exec, P satisfy the issue-side synchronization-boundary rule, but they contribute no Σ -event and do not become Ω_P frontier items merely because they executed.

R4 (Safe message issue ordering).

For every pair $q1, q2 \in Q_exec, P$ within the same process and within the compared dynamic execution / witness,

$q1 <_{\text{src}} q2 \Rightarrow \text{issue_pre}(q1) \leq \text{issue_pre}(q2).$

(For same-channel operations, this states that dynamic calls on a single interface are issued in source-induced program order. For distinct channels, it forbids issuing dynamic message-call instances in the reverse of their source-induced order. This rule does not impose ordering between syntactic operations on untaken branches or between dynamic instances that are not ordered by the source-induced partial order of the selected witness. Failed non-blocking polls are included in this issue-order universe when they are executed, but remain ϵ -steps with no Σ -event.)

Prefix-closed reading of issue order. R4 and E3 are to be read not only as inequalities between already-defined issue timestamps, but also as closure conditions on issue itself. Within a compared dynamic execution / witness, if $q1 <_{src} q2$ and $q2$ has issued by cutpoint σ , then $q1$ has also issued by σ , and $Issue(q1) \leq Issue(q2)$ on the same side of the comparison ($Issue_pre$ for R4/source-side prefixes, $Issue_post$ for E3/implementation-side prefixes). Equivalently, in each source interval, the set of issued message-operation instances is downward closed under the source-induced order used by R4/E3. This rule is what excludes the case where a source-later message operation has issued while a required source-earlier message operation in the same interval remains merely reached but unissued.

R5 (Channel capacity semantics; Sys vs. Sys_B).

Appendix G defines the raw rendezvous source model Sys, in which every channel has effective capacity 0. Concretely: for every channel c and any clock cycle t , the number of unmatched committed Push_c actions is zero and the number of unmatched committed Pop_c actions is zero; equivalently, no Push_c shall commit unless a matching Pop_c also commits at t , and no Pop_c shall commit unless a matching Push_c also commits at t .

In Appendices I–K we additionally use the back-annotated source interpretation Sys_B: the same source processes as Sys, but interpreted under the case-split channel semantics of E4 with capacities $B(c)$ matching RTL_B. Thus:

- $B(c)=0$ is interpreted as the rendezvous case of E4: matching Push_c(v) and Pop_c(v) endpoint events commit in the same cycle, and no unmatched committed endpoint event may remain after any cycle boundary.
- $B(c)>0$ is interpreted as the cycle-boundary, non-fall-through bounded-FIFO case of E4, using the occupancy predicate $occ_c(t)$ and the FIFO availability predicates $occ_c(t) < B(c)$ for Push_c and $occ_c(t) > 0$ for Pop_c.

The source conceptual semantics for a process is thus a labeled partial order $(\Sigma, \leq_P)$ generated by R1–R4 plus the applicable E4 channel semantics: raw rendezvous for Sys, or the case-split back-annotated channel semantics of Sys_B / Sys_ref.

Operational semantics location

The selected Sys_ref transition system, registered sequential-interface semantics, Policy L and Policy S, and the admissible Policy-L execution domain are defined normatively in the preceding “Normative Formal Core for Appendices G–K.” Appendix G uses that core while stating the R1–R5 and E1–E5 trace-compatibility rules; it introduces no second operational semantics.

Trace Compatibility Relation

Let τ_pre and τ_post be the Σ -traces (finite or infinite) of the same process before and after HLS, where each Σ -event is labeled exactly as in “Normative Formal Core for Appendices G–K” (e.g., Push_c(v), Pop_c(v), Read(sig, val), Write(sig, val), and synchronization events).

Notation note. We write $\tau_pre \approx \tau_post$ as a named, ordered trace-compatibility predicate. The symbol $\approx$ is local to this document and is not used here to assert a mathematical equivalence relation; in particular, reflexivity, symmetry, and transitivity are not being claimed. Its arguments are ordered: the left trace supplies the pre-HLS/source reference witness, and the right trace supplies the post-HLS implementation trace constrained by E1–E5. In later appendices the same convention applies to $\tau_ref \approx$

τ_{post} , with τ_{ref} as the prescribed Sys_ref witness.

Convention (S): all references to S , $\text{pred_}S/\text{succ_}S$, $\text{pref_}S/\text{suff_}S$, and $<_src$ in E1–E5 use the dynamic source-induced order defined in R1–R4 for the compared execution / witness. Thus τ_{pre} denotes the Σ -trace of the pre-HLS source over the dynamic operation instances reached under the fixed stimulus / selected witness.

Σ -event matching convention (expansion of Definition Core.17 ΣMatch). We use a canonical per-endpoint occurrence matching to make “the same Σ -events” precise. For each channel c , match committed $\text{Push_}c$ and $\text{Pop_}c$ events by their ordinal occurrence on that channel (e.g., the k -th committed Push on c in τ_{pre} matches the k -th committed Push on c in τ_{post} ; likewise for $\text{Pop_}c$). For each such matched pair, the endpoint (c) and the payload/value label must agree. For the ordinal index k to be well-defined under the per-cycle “set/bucket” trace representation used in these appendices, we assume that for any fixed channel endpoint c , at most one committed $\text{Push_}c$ and at most one committed $\text{Pop_}c$ may occur in any single clock cycle.

For each observable signal sig of a sequential process, match $\text{Read}(\text{sig}, \cdot)$ and $\text{Write}(\text{sig}, \cdot)$ by their ordinal occurrence on that signal within the process trace, again requiring the value labels to agree.

For sequential processes, we apply the following canonicalization for observable signals: within each anchor interval between consecutive synchronization events in the order S , we record at most one Σ -visible $\text{Read}(\text{sig}, \text{val})$ and at most one Σ -visible $\text{Write}(\text{sig}, \text{val})$ for each (process, sig), with the anchor cycle given by E2. Any additional physical sampling of sig by the implementation, or redundant re-driving of sig within the same anchor interval, is treated as an internal ϵ -step and must not change the recorded value label. If a design intentionally relies on intra-interval changes of sig , or on distinguishing multiple reads/writes of sig within the same anchor interval, then sig must be modeled using an explicit synchronized interface rather than as an ordinary sequential-process observable signal.

For combinational processes, the canonical observable unit is the emitted R2C fixed-point signal-observation unit for the relevant combinational process/network component C_{comb} : the set of $\text{Read}(\text{sig}, \text{val})$ and $\text{Write}(\text{sig}, \text{val})$ labels for the selected observable slice $\text{Obs}(C_{\text{comb}})$, recorded at one ϵ -quiescent combinational fixed point under the R2C unit-generation convention. Intermediate delta-cycle reads/writes, repeated re-evaluations before the fixed point, and stutter cycles in which $\text{Obs}(C_{\text{comb}})$ is unchanged and no explicit observer/sampler requires a fresh unit are $\epsilon/\text{stutter}$ and are not separately Σ -visible.

R2C unit matching/index convention

For each combinational process or explicitly modeled combinational network component C_{comb} , project each trace to the subsequence of emitted R2C fixed-point signal-observation units for C_{comb} . The k -th emitted such unit in Sys_ref is matched with the k -th emitted such unit in RTL_B . This local emitted-occurrence ordinal, together with the component identity C_{comb} , is the correspondence rule for R2C fixed-point units; it is not the global observable-cycle bucket number of the whole-system trace. Thus repeated identical fixed-point observations across stutter cycles do not create additional units, while repeated equal-valued units created by explicit observer/sampler occurrences are distinguished by their local emitted-occurrence ordinal for C_{comb} .

Matched pre-HLS and post-HLS fixed-point units must agree on the included process/signal/direction labels and fixed-point values. The same-cycle agreement required here is local to the matched emitted

R2C fixed-point unit: all Read/Write labels inside that unit are interpreted at one ϵ -quiescent fixed point and may not be split. This local atomicity requirement is one of the explicit same-cycle semantic constraints and does not require unrelated Sys_ref and RTL_B events to share the same global bucket decomposition. If a component is cycle-locked, its local emitted-occurrence sequence is the ordered sequence of cycles t satisfying $\text{ComponentParticipatesInCycle}(\text{C_comb}, t)$, with one required unit per such cycle. The pre-HLS and post-HLS traces must therefore agree both on the participation-indexed unit values and on the absence of omitted required units. Cycle-locked components fall outside the bucket-stutter freedom claimed for ordinary latency-insensitive R2C components.

R2C cross-model/cycle-lock clarification. Sys_ref and RTL_B are not assumed to share internal combinational state, a common combinational microstep model, or the same global cycle-bucket placement; each side settles its own cycle-entry valuation. For a cycle-locked component, both sides must satisfy the per-side participation/completeness rule above, and the k -th matched units are compared within those respective component-local participation-indexed emitted sequences; equality of absolute Sys_ref and RTL_B cycle numbers is not required. If RTL_B does not expose an explicit combinational-network model from which participation can be derived, the implementation/certificate evidence for the selected annotated prefix shall justify that its emitted sequence obeys the declared cycle-lock/participation rule. The R2C matching statement in E2 is the Appendix-G expansion of this same R2C clause of Core.17 ΣMatch , not an additional independent matching conjunct.

For proof-internal $\text{RESET}(S_p, S_c)$ units included in the selected theorem comparison, match by reset scope and local occurrence within that scope's reset-epoch history: the k -th RESET for a given (S_p, S_c) scope in τ_{pre} matches the k -th RESET for the same scope in τ_{post} . Matched RESET units therefore have the same scope. Independent disjoint RESET occurrences may commute unless an explicit reset-boundary or other observable-order constraint orders them.

For synchronization events, match by their per-process occurrence index in the source order (R1; preserved on the post-HLS side by E1 below). Independent Σ -events on different endpoints (including distinct message-passing interfaces) may commute and/or be grouped differently within a clock cycle as permitted by the R-rules and E3. Here “independent” is meant in the Σ -equivalence sense: the pair is not additionally ordered by any explicit Σ -visible ordering/timestamp constraint in E1–E5 (notably E3/E5); it is not a statement about (in)comparability under the Appendix G causal-dependency relation ($\rightarrow$). Accordingly, τ_{pre} and τ_{post} are not required to be identical as a single total order, nor to agree on same-cycle “grouping” of distinct-interface Push/Pop operations, beyond the explicit ordering/timestamp constraints in E1–E5 below.

Consistently with Definition Core.17, we say $\tau_{\text{pre}} \approx \tau_{\text{post}}$ iff $\Sigma\text{Match}(\tau_{\text{pre}}, \tau_{\text{post}})$ holds under the convention above and all the following hold:

E1 (Per-process synchronization-order preservation).

For every process P ,

$$\forall s_1, s_2 \in S_P : s_1 <_P s_2 \Rightarrow \text{clk_post}(s_1) < \text{clk_post}(s_2).$$

Equivalently, synchronization events are matched and ordered by their per-process source occurrence index. E1 does not preserve incidental global clock ordering between synchronization events of different processes unless that ordering is induced by an explicit inter-process synchronization relation already represented in Σ .

E2 (Signal-visibility preservation).

Partition signal Read/Write events by process kind:

- R_seq and W_seq are the ordinary signal Read/Write events of sequential processes.
- R_comb and W_comb are the signal Read/Write events of combinational processes, interpreted by R2C.

For sequential-process signal IO:

$$\forall r \in R_seq : clk_post(r) = clk_post(pred_S(r))$$

$$\forall w \in W_seq : clk_post(w) = clk_post(succ_S(w))$$

Equivalently, ordinary signal Reads/Writes of sequential processes are logically timestamped at their synchronization anchors, as in R2. The same anchoring convention is used on the pre-HLS side when forming the matched trace.

For combinational-process signal IO, $pred_S/succ_S$ anchoring does not apply. Instead, each R2C fixed-point signal-observation unit is matched by the component-local occurrence ordinal defined above: for each combinational process or explicitly modeled combinational network component C_comb , the k -th R2C fixed-point unit of C_comb in Sys_ref matches the k -th R2C fixed-point unit of C_comb in RTL_B . The matched units must contain the same corresponding combinational Read/Write events and fixed-point values at their respective ϵ -quiescent combinational fixed points, as defined by R2C. Any delta-cycle re-evaluation, intermediate combinational value, or redundant re-drive before the fixed point is an ϵ -step and is not separately Σ -visible. This condition does not require the matched R2C units to occur in the same global observable-cycle bucket number, nor to be grouped with the same unrelated events, in the two executions.

E3 (Safe message issue ordering). (Post-HLS preservation of R4).

For every pair of executed message-call instances $q1, q2 \in Q_exec, P$ of the same process P :

(i) Same-interface source order: if $q1$ and $q2$ access the same message-passing interface and $q1 <_{src} q2$, then $issue_post(q1) \leq issue_post(q2)$. This preserves source order at cycle granularity; any stricter serialization required by the concrete interface is supplied by the channel/interface semantics, not by E3 itself.

(ii) Distinct-interface no-reverse: if $q1$ and $q2$ access distinct message-passing interfaces and appear in sequence in the source ($q1 <_{src} q2$), then $issue_post(q1) \leq issue_post(q2)$ (i.e., they shall not be issued in the reverse sequence).

(iii) Prefix-closed issue obligation: for every cycle-aligned cutpoint σ of the post-HLS execution / witness, if $q1 <_{src} q2$ and $q2$ has issued by σ , then $q1$ has also issued by σ and $Issue(q1) \leq Issue(q2)$. This applies across same-interface and distinct-interface source order. It is an implementation-side obligation on issued sets, not merely an inequality over pairs whose timestamps are already defined.

(Note: HLS internal pipeline staging, dequeue, and hand-off inside the back-annotated process/channel realization are not, by themselves, Σ -visible boundary issue events in Q_exec, P nor committed-transfer events in M ; therefore such internal movement does not constitute a counterexample to (ii) or (iii).)

E4 (Per-channel channel semantics: rendezvous and bounded FIFO).

For every observable channel c with capacity $B(c)$, the projection of τ_post to events on c is legal for that channel's capacity model. The channel model splits into two cases.

- Rendezvous case, $B(c)=0$. Channel c is a capacity-zero rendezvous channel. A committed transfer on c consists of a matching same-cycle pair $Push_c(v)$ and $Pop_c(v)$ with the same payload value v . No

unmatched committed Push_c or Pop_c may exist after any cycle boundary. Equivalently, the k-th committed Push_c(v) and the k-th committed Pop_c(v) occur in the same clock cycle and carry the same value. A Push_c endpoint can commit in cycle t only when the matching Pop_c endpoint boundary request is also asserted in cycle t, and symmetrically a Pop_c endpoint can commit in cycle t only when the matching Push_c endpoint boundary request is also asserted in cycle t.

- Buffered case, $B(c) > 0$. Channel c is a bounded FIFO with cycle-boundary, non-fall-through semantics. The committed Pop_c(v) events return exactly the FIFO-ordered sequence of values v previously committed by Push_c(v), with no drops, duplications, or reordering. For every cycle-aligned prefix π_t of the c-projection—i.e., π_t contains exactly the events on c committed in cycles $< t$ —letting push(π_t) and pop(π_t) be the number of Push_c and Pop_c events in π_t , we have

$$0 \leq \text{push}(\pi_t) - \text{pop}(\pi_t) \leq B(c).$$

Equivalently, the abstract occupancy after cycle $t-1$ is $\text{occ}_c(t) = \text{push}(\pi_t) - \text{pop}(\pi_t)$. Availability for commits in cycle t is evaluated against $\text{occ}_c(t)$ at the start of cycle t: Pop_c may commit in cycle t only if $\text{occ}_c(t) > 0$, Push_c may commit in cycle t only if $\text{occ}_c(t) < B(c)$, and simultaneous Push_c and Pop_c commits in the same cycle are permitted iff $0 < \text{occ}_c(t) < B(c)$.

Thus the inequality-based FIFO availability predicates are used only for $B(c) > 0$. The $B(c) = 0$ case is not interpreted by substituting $B(c) = 0$ into the bounded-FIFO inequalities; it is interpreted by the rendezvous rule above.

Note: $\text{occ}_c(t)$ is the logical occupancy of channel c at the chosen Σ boundary, induced solely by the Σ -visible boundary-commit history (Push_c/Pop_c) and the annotated capacity $B(c)$ in the buffered case $B(c) > 0$. For $B(c) = 0$, the corresponding channel fact is the same-cycle rendezvous request/commit predicate rather than a nonzero FIFO occupancy predicate.

E5 (Messages cannot cross their immediate synchronization boundary).

For every synchronization call s (in the source order used by R3 / pref_S / suff_S over Q_{exec}, P):

$\forall q \in \text{pref}_S(s) : \text{issue_post}(q) \leq \text{clk_post}(s)$

$\forall q \in \text{suff}_S(s) : \text{issue_post}(q) > \text{clk_post}(s)$

$\forall q \in \text{pref}_S(s)$ with q a blocking Push/Pop : m(q) is defined and $\text{clk_post}(m(q)) \leq \text{clk_post}(s)$.

The third clause is the implementation-side synchronization-completion fence: no earlier blocking request may survive s into the succeeding source interval, although q and s may commit in the same cycle. In $\text{Sys}_B^{\text{elab}}$, m(q) denotes the designated process-facing commit that completes/releases q. For a same-cycle q/s completion, the Core.SyncFence statement that q is “accounted for before” the interval advance is an abstract trace/certificate accounting convention used to preserve operation and interval attribution; E5 does not require a separately observable physical sub-cycle ordering between q and s on RTL_B. Under the Issue/Commit linkage above, any boundary request assertion that persists while the process is still pending at s reflects only already-issued outstanding operation(s) in pref_S(s); no operation $q \in \text{suff}_S(s)$ may have $\text{Issue}(q) \leq \text{clk_post}(s)$, and any back-to-back issuance on that endpoint direction may occur only in the first cycle strictly after $\text{clk_post}(s)$. For non-blocking calls, a later executed source-level PushNB/PopNB in suff_S(s) is a distinct q and cannot be identified with an earlier pref_S(s) call solely because the endpoint request signal remained asserted.

See Appendix I – Per-Process Replay and Preparation Proof for the local result used by the system theorem:

For any process P participating in a selected finite RTL_B prefix, the fixed stimulus, WC witness, dynamic-operation matching, and P -local observable-unit history determine the corresponding immediate source replay state. Appendix I proves that, from that state, a finite P -owned ϵ preparation path reaches the exact dynamic source operation for the next selected unit—either as next-frontier or as an already issued persistent request—while preserving E3/E5 and committing no unmatched observable unit. This is a local readiness theorem, not an independent channel-commit or global-trace existence theorem. Appendix J combines these local paths with channel and atomic-interaction certificates to construct the complete Sys_ref witness. The reverse direction remains restricted to $RTLConsistentRefPrefix$ / Appendix L witness-selected executions.

Allowed Transformations and Why They Preserve $\approx$

Transformation permitted by the rules	Formal justification
Loop pipelining (overlaps iterations)	Introduces internal pipeline stages (modeled as ϵ -steps) and overlaps iterations. R1–R5 (and hence E1–E5) are applied to the explicit S set. Loop pipelining may commute independent Σ -actions across iterations (e.g., across distinct channels)—where “independent” here means “not additionally ordered by any explicit Σ -visible constraint in E1–E5 between the pair (notably E3/E5),” rather than “incomparable under Appendix G’s causal-dependency relation ($\rightarrow$)”—but it must still preserve: (i) per-channel FIFO legality (E4), (ii) the required issue discipline (E3) for source-ordered pairs, and (iii) the sync-boundary constraints (E1/E5) with respect to the (explicit + implicit) S -actions.
Overlap of internal dequeue/staging vs. later pipeline work (ϵ)	Permitted because the internal staging/acceptance activity is ϵ -internal within the back-annotated channel realization (as defined in “Appendix B - Pipelined Loops”) and does not constitute a Σ -visible boundary message-issue reorder. Σ -visible boundary message issues must still satisfy E3, and per-channel committed-transfer legality is ensured by E4; automatic flush / Appendix K addresses deadlock preservation for overlap timing.
#pragma hls_direct_input (late sampling of static signals)	This pragma imposes a designer/environment stability contract: the signal’s value must remain stable after reset (or after its last permitted update) while the receiving process may consume it. The logical signal Read actions r remain anchored exactly as in E2 ($clk_post(r) = clk_post(pred_S(r))$). HLS may implement this by sampling the signal later (instead of inserting internal storage), but because the signal is stable the sampled value equals the value at $pred_S(r)$; therefore E2 and $\approx$ are preserved.
#pragma hls_direct_input_sync (controlled updates)	This pragma imposes an explicit timing contract on the environment: the controlled direct-input signals may change only on the cycle of the designated synchronization call s^*

Transformation permitted by the rules	Formal justification
	(i.e., during the sync handshake). With that contract, the logical anchoring required by E2 is preserved: for the receiving process, the relevant Reads are anchored at the closest preceding synchronization call $\text{pred_S}(r) = s^*$, and any environment update Write w_update is aligned with synchronization ($\text{succ_S}(w_update) = s^*$). HLS may still implement late sampling internally, but the value observed is the same as the value at the E2 anchor point, so $\approx$ is preserved without modifying E2. Instrumentation note (logical signal timestamping). In the Σ-traces used for $\approx$ checking, sequential-process signal Read/Write events are timestamped at their E2 anchor points (e.g., $\text{clk_post}(\text{pred_S}(r))$ for Reads), not at any later physical RTL sample cycle that HLS may introduce. Any internal late-sampling micro-steps are treated as ϵ-steps. Combinational-process signal Read/Write labels are handled by R2C rather than by $\text{pred_S}/\text{succ_S}$ anchoring or by global observable-cycle bucket number. A monitor records a combinational fixed-point value only after the simulator/RTL has reached an ϵ-quiescent combinational fixed point, and that value contributes to Σ only when the relevant R2C observation component emits an R2C fixed-point signal-observation unit under the R2C unit-generation convention. Intermediate delta-cycle reads/writes, redundant re-evaluations, transient combinational values before the fixed point, and R2C stutter cycles whose selected observable slice is unchanged are $\epsilon/\text{stutter}$ and are not separately Σ-visible. Therefore, equivalence checking must instrument/record the appropriate logical Read/Write event: the anchored event for sequential-process signal IO, or the emitted R2C fixed-point signal-observation unit for combinational-process signal IO. A wrapper/transactor may be used when needed to expose only those logical events rather than naive physical sample, delta-cycle, or per-cycle stutter events.
Memory-access reordering proven conflict-free	Let $\text{addr}(a)$ be the symbolic address of access a . If the tool proves $\text{addr}(a_1) \neq \text{addr}(a_2)$ for the overlapped window, then the commutation of a_1 and a_2 is observationally silent; no external signal/message depends on the internal order.
Implicit FSM states / added latency	Extra states introduce <i>silent</i> ϵ -steps between observable actions; the partial order on Σ is unchanged, so E1–E5 are unaffected.
Shared-memory arrays with explicit synchronization	When a memory is accessed by multiple processes, the designer explicitly coordinates access with synchronization operations modeled as S-actions, and the externally relevant

Transformation permitted by the rules	Formal justification
	memory behavior is defined by the array access mapping layer. E1 preserves the order of the synchronization actions that bracket or authorize the shared-memory accesses. The array access mapping layer then defines the mapped memory operations, their commit cycles, and the required serialization/coherence rule for the shared storage, and the pre-HLS and post-HLS traces must agree under that declared memory semantics. E2 is not used here except for ordinary signal IO. E4 is relevant only if the mapping layer represents memory commands/responses as message-passing channels; otherwise memory legality is discharged by the mapping layer's memory semantics, not by FIFO channel semantics. Under these conditions, the shared-memory transformation preserves the observable behavior required by $\approx$. Scope (J.Mem): the formal theorems of Appendices I–K discharge this entry only when the value-carrying memory traffic is lowered per the J.Mem shared-memory lowering condition (Normative Formal Core); the non-lowered “declared memory semantics” branch states an agreement obligation that Theorem J.1 as proved does not discharge.
Non-blocking message-passing (PushNB/PopNB) verified by post-HLS snooping	If the latency differences are externally visible to the DUT, a verification wrapper delays the pre-HLS side so that the committed transfer events (successful completions) occur in the same order/timing as observed on the post-HLS channel. This wrapper is itself purely synchronising (S), so it cannot violate R1–R5. Once the wrapper is assumed, τ_{pre} and τ_{post} coincide on the committed message-transfer history recorded in Σ (i.e., the Push_c/Pop_c commit events), so E3–E5 are preserved. Unsuccessful non-blocking polls remain ε-steps and are not required to match. This is not a transformation that inherently preserves $\approx$, but rather a verification technique to enforce equivalence for inherently latency-sensitive operations.
Latency-sensitive <i>global</i> signal IO matched by snooping	If the latency differences are externally visible to the DUT, the same “mirror-and-delay” wrapper used for NB channels is applied to any globally-visible signal whose timing matters. Because the wrapper inserts only S-actions, the partial-order constraints are unchanged. This is not a transformation that inherently preserves $\approx$, but rather a verification technique to enforce equivalence for inherently latency-sensitive operations.
Latency-sensitive <i>local</i> signal IO isolated in cycle-accurate transactors	Local timing-exact protocols are confined to dedicated transactor processes that expose only latency-insensitive M or S operations to the rest of the design. Since the transactor boundary is now the observable interface, R1–R5 apply

Transformation permitted by the rules	Formal justification
	<i>outside</i> the timing-sensitive region, so system-level $\approx$ still holds.
One-way signal-handshake protocols with run-time assertions	For single-direction vld or rdy signals, an assertion proves that back-pressure can <i>never</i> occur. That assertion establishes a refinement in which the missing handshake is equivalent to a permanent logical '1', so the protocol is observationally identical to the corresponding two-way handshake with the missing side tied true. E2 then applies only to the logical timestamping of the signal Read/Write events, not to the protocol-refinement step itself.
Combinational processes	Combinational processes have no process-owned S actions and no message-passing M actions in this formal clause, but they may have observable signal Read/Write events in Σ through explicitly selected R2C observation components. Their signal IO is interpreted at ϵ-quiescent combinational fixed points as specified by R2C: intermediate delta-cycle propagation is ϵ, and only emitted fixed-point Read/Write labels in the selected observable slice are Σ-visible. Repeated cycles in which the selected fixed-point slice is unchanged are R2C stutter for that component unless an explicitly modeled observer/sampler requires a fresh emitted unit. Under the well-formedness requirement that the combinational network is deterministic, reaches a unique fixed point in each compared cycle, and has a well-defined R2C unit-generation rule for the selected component, the RTL/tool-flow conformance obligation is to preserve the emitted fixed-point signal-observation unit sequence at the chosen observation boundary. Thus the required correspondence is not equality of simulator delta-cycle behavior, nor equality of global cycle-bucket numbers, nor an automatic consequence of ordinary combinational synthesis; it is equality of matched emitted R2C fixed-point units by component identity and local emitted-occurrence ordinal. Designs with oscillation, ambiguous multiple-driver resolution, hidden stateful behavior, intentional combinational-loop behavior outside a unique fixed-point interpretation, or multi-input observation slices whose interleavings can change the emitted unit sequence are outside R2C unless represented by explicit state, synchronization, a cycle-accurate transactor, a cycle-locked R2C observation boundary, or another formally modeled boundary.

For the purposes of the formal analysis, non-blocking message-passing is modeled at the level of committed transfers: a successful PushNB/PopNB contributes the same committed Push_c/Pop_c event to Σ as a blocking Push/Pop, while unsuccessful polls are ϵ -steps.

If a design contains latency-sensitive global signals or non-blocking transfers whose timing or ordering is externally visible at the DUT boundary and must be reproduced in the selected comparison, an Appendix L snooping/witness-selection wrapper may be used. When a witness satisfying WC and L.Dir exists, the wrapper selects an admissible pre-HLS execution whose relevant Σ -events align with the observed RTL execution. The wrapper does not enlarge the set of admissible pre-HLS behaviors; it selects one admissible execution. If the wrapper cannot realize such a witness, the compared run is outside the theorem scope and must be handled according to the Appendix L triage rules.

Appendix L defines the witness-existence conditions, monitoring, and triage required to establish this alignment for a concrete compared run.

Proof Structure

The proofs proceed in three stages:

First, Appendix I establishes process-local deterministic replay and finite preparation. For each process participation in a selected RTL_B observable unit, it identifies the exact dynamic source operation, proves the relevant issue/order and witness facts, and constructs a finite process-owned ϵ path that reaches the operation or its persistent issued request without committing an unmatched unit. These facts are packaged by J.PCert. Appendix I deliberately does not assert that the complementary endpoint is present, that a channel transfer is globally enabled, or that the local paths already form one Sys_ref execution.

Second, Appendix J combines J.PCert with J.CCcert, J.ACcert, environment/reset certificates, the normative field-sensitive footprint vocabulary, deferred-hole records, Dep_J, and LocalAgree. J.CanonicalRank is derived from the graph. J.Frame/J.LocalLift/J.PrepareCommute and J.RegPhaseNF prove that outcome-independent actions can be moved into L1, requests freeze before the L2 snapshot, and A3/A4 decisions and units execute in L3 modulo J.HCanon. Horizon-batched J.UnitExt and J.GWR-Comp construct J.GWR and J.RegPhaseReach; Proposition J.0 proves observable compatibility. This safety construction does not by itself prove K.DrainReach, A5-L, or RTL deadlock freedom.

Finally, Appendix K addresses a separate liveness-preservation obligation for internal message-passing deadlock. Appendix K does not discharge the Appendix I/J weak-fairness, finite-depth, channel-progress, realization-correctness, K.Drain, $K\epsilon$, J.KObsConf, K.RLAdmReach, QualifiedSynchronousModel/Core.KCutClosure_post, K.CertCov, or A6–A11 side conditions. Nor does it derive SDet, A5-L, A5-S, K.RespCov, K.RespClean, K.EnvMF, K.InterruptionGate/K.ResetEpoch/K.TerminalDisposition applicability, or any alternate A5-L discharge from the local R-rules. The core Theorem K.1A consumes A5-L and one K.CertifiedCutpoint package directly; Corollary K.1A-Total additionally consumes Core.KCutClosure_post and K.CertCov; in the standard scope Core.QSync-Post derives the closure premise from the qualified synchronous structural evidence for the universal conclusion. The primary user-facing route fixes the actual Policy-S pre-HLS simulator and its selected execution through SDet, satisfies K.EnvMF and the shared K.InterruptionGate applicability, instantiated first as K.ResetEpoch and then, only on K.ResetEpoch.Continue, as K.TerminalDisposition, supplies maximality/fairness/completeness and complete bounded-response evidence through A5-S/CF-S, uses Lemma K.2b and Corollary K.2c to discharge A5-L, and uses Core.KCutClosure_post plus K.CertCov to justify the universal reachable K-facing RTL conclusion.

Structural, invariant, exploration, direct Policy-L, and tool-certificate routes remain alternatives only for instances in which A5-L is actually true; they cannot rescue a design containing an admissible serial deadlock prefix.

Thus the architecture is non-circular. Appendix I supplies only local replay/preparation facts; J.GWR-Comp proves, rather than assumes, their global coexistence and complete-unit extension; Proposition J.0 consumes an already constructed trace pair; and Appendix K separately preserves internal message-passing deadlock freedom under A5-L and its recorded discharge route. The liveness theorem remains conditional on the named fairness, progress, drain, and serial-dominance premises and does not retroactively justify the safety construction.

Causal Dependency Between Processes

To prove system-level trace compatibility, we must distinguish between incidental timing and functionally significant ordering.

Formal Definition of Causal Dependency

To formalize the notion of functionally significant dependency and resolve ambiguity, we define a causal-dependency relation, denoted by $\rightarrow$, on the set of observable IO actions (Σ) across all processes in the system. The relation is causal/information-flow rather than purely temporal: events related by $\rightarrow$ may commit in the same clock cycle, and the direction of $\rightarrow$ indicates which event may be depended on by the other.

Important: $\rightarrow$ is a causal-dependency graph, not by itself the source order used for the Appendix J induction. In particular, because the methodology permits distinct-interface message operations to be issued with timing overlap or in the same cycle while still preserving dynamic source order, and because rendezvous transfers are same-cycle atomic Push/Pop transactions, the unquotiented dependency graph $\rightarrow$ need not be acyclic in every legal execution. A cycle in $\rightarrow$ can represent a legal same-cycle mutually enabling dependency among atomic transactions; it is not, by itself, an observable mismatch. When Appendix J needs a well-founded induction order, it uses the separate required-order relation $\prec_J$ over observable units. That relation is generated only by the explicit ordering, timestamp, FIFO, atomicity, and synchronization constraints that must be preserved by the trace-compatibility predicate, not by the full causal-dependency graph $\rightarrow$ and not by raw per-process source order on distinct-interface message commits.

Well-formedness use of causal-dependency. The well-formedness rule below uses $\rightarrow$ as a design-intent and source-admissibility relation: if correctness depends on the relative timing, value, availability, or cross-process observation of two events, that dependence must be represented by an explicit modeled causal mechanism such as message passing, synchronization, signal handshakes/anchors, declared memory serialization, or an Appendix L snooped/witness-selected boundary. This use of $\rightarrow$ does not add proof-preserved observable ordering beyond $\prec_J$, FIFO legality, rendezvous/synchronization atomicity, signal-anchor constraints, synchronization-boundary constraints, and declared memory-serialization constraints.

The causal-dependency relation $\rightarrow$ is the smallest directed relation satisfying the conditions below:

1. **Intra-Process Order:** If actions a and b occur within the same process P and a precedes b in the program's execution trace, then $a \rightarrow b$. This directly reflects the sequential nature of the code within a single thread. **Clarification (non-temporal).** Here “precedes” refers to the order of IO action *instances* in the process's trace/program order (i.e., the order induced by the process's

execution), **not** the relative order of their **commit cycles**; distinct-interface message actions may be **issued** in order while **committing** in either order depending on backpressure.

2. **Inter-Process Communication:** The relation is established by the direct exchange of information or synchronization between processes.

- **Message-passing (committed transfer):** If a is the commit of a send on channel c in process $P1$ (a committed Push_c , including a successful PushNB), and b is the commit of the corresponding receive on c in process $P2$ (a committed Pop_c , including a successful PopNB), for the same logical message instance, then $a \rightarrow b$. Here “corresponding receive” is defined by the channel’s FIFO semantics: b is the Pop_c commit that consumes the element produced by a (matching is by FIFO position / transfer instance, not by payload-value equality; duplicate payload values do not introduce ambiguity). Equivalently, if we enumerate committed Pushes on c as $\text{Push}_c^1, \text{Push}_c^2, \dots$ in commit order and committed Pops as $\text{Pop}_c^1, \text{Pop}_c^2, \dots$ in commit order, then $\text{Push}_c^k \rightarrow \text{Pop}_c^k$. For buffered channels with $B(c) > 0$, E4 also imposes the required FIFO legality used by Appendix J’s required-order relation $\leq_J$. For rendezvous channels with $B(c) = 0$, the matching Push_c and Pop_c commits occur in the same clock cycle and are treated in Appendix J as one explicitly atomic observable unit. The directed dependency $a \rightarrow b$ records unidirectional information flow — the Pop observes the value supplied by the Push — but it does not make the two rendezvous endpoints separately orderable for the Appendix J induction, and it does not imply an earlier commit cycle. No reverse edge $b \rightarrow a$ is implied unless there is an explicit reverse-direction communication, such as another channel or a signal handshake. Unsuccessful non-blocking polls are ϵ -steps and do not participate in $\rightarrow$.
- **Signal Handshake:** If a is a signal write action $w(\text{sig})$ in $P1$ and b is a signal read action $r(\text{sig})$ in $P2$ that reads the value written by a as part of an explicit handshake protocol, then $a \rightarrow b$. A complete two-way handshake (e.g., vld/rdy) creates a causal chain, such as $w(\text{vld})@P1 \rightarrow r(\text{vld})@P2 \rightarrow w(\text{rdy})@P2 \rightarrow r(\text{rdy})@P1$.
- **Explicit Synchronization (two-party SyncChannel / ac_sync):** Let sout be the commit of a $\text{SyncChannel } \text{sync_out}$ (or equivalent ac_sync call) in process $P1$, and let sin be the commit of the matching sync_in in process $P2$ for the same synchronization instance. Then this synchronization acts as a two-process barrier:
 - Every observable action in $P1$ that occurs before sout (in $P1$ ’s trace/program order) satisfies $a \rightarrow b$ for every observable action b in $P2$ that occurs after sin .
 - Symmetrically, every observable action in $P2$ that occurs before sin satisfies $a \rightarrow b$ for every observable action b in $P1$ that occurs after sout .
 - (The two sync commits sout and sin are treated as simultaneous; we do not impose $\text{sout} \rightarrow \text{sin}$ or $\text{sin} \rightarrow \text{sout}$.)

3. **Transitivity:** The relation is transitive. If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.

Using this relation, we now provide a formal definition for causal dependency between synchronization events, which are the fundamental ordering anchors in the scheduling model.

Definition: Causal Dependency

A synchronization event s_2 in process P_2 is **causally dependent** on a synchronization event s_1 in process P_1 if and only if $s_1 \rightarrow s_2$.

If neither $s_1 \rightarrow s_2$ nor $s_2 \rightarrow s_1$ holds true, the events s_1 and s_2 are considered **causally independent** (or concurrent). Any change in the observed temporal ordering of causally independent events between the pre-HLS and post-HLS simulations is considered an incidental, functionally insignificant variation in latency. A **well-formed design**, under this methodology, **shall not rely** on a specific ordering of causally independent events for functional correctness.

To be clear, the shall-not-rely requirement is a design rule. To adhere to this design rule, designs must express functionally significant cross-process dependencies through explicit modeled mechanisms such as message passing, SyncChannel/ac_sync operations, signal handshakes/anchors, declared memory-serialization constraints in the array-access mapping layer, or Appendix L snooping/witness-selection boundaries. Designs that violate this design rule are outside the formal guarantees. This rule concerns design well-formedness; it does not convert every causal-dependency edge into a strict proof-order edge.

System-Level Theorem (B1-Specialized Appendix G Statement)

Appendix G gives the high-level system theorem in the same standing-assumption regime used by Appendices J–K: B1 is assumed for RTL_B under the fixed stimulus, so the practical “unique RTL_B trace” reading is available throughout the main theorem stack. The authoritative finite-prefix trace-set theorem, including the exact reachable-prefix quantification and the conditional use of B4/B5 when ϵ -normalization or uniqueness up to finite ϵ -stutter is invoked, is Theorem J.1. Therefore, if this Appendix G statement and Theorem J.1 are read together, Theorem J.1 controls; Appendix G is the high-level summary of the same B1-standing finite-prefix theorem regime.

Fix an initial state and a fixed external stimulus/environment behavior for both the prescribed source reference model Sys_ref and RTL_B, with Sys_ref as defined in Definition Core.1 and Appendix J, Remark 2. (The raw rendezvous model Sys is the special case obtained when Sys_ref = Sys_B and all effective capacities are zero.)

Interpret “fixed stimulus” per Appendix I.2, “Fixed-stimulus interpretation (reactive environments).” Let Reach_post / Reach_ref denote the corresponding finite reachability sets with no fair-extension qualifier (Definition J.Reach, Appendix J).

Assume the Appendix I / Appendix L witness machinery and WC are available, together with the local certificate package of Definition J.LocalGWR: J.PCert for every process, J.CCert for every explicit abstract channel, J.ACert for every explicitly atomic unit, the applicable environment/reset certificates, and J.LocalAgree. Also assume the capacities/occupancies/enableness semantics of Sys_ref, finite capacities and B-faithfulness at every explicit abstract boundary, the standing RTL_B determinism assumption B1 under the fixed stimulus, and—on the normal compositional route—

QualifiedSynchronousModel_Sys_ref(I), so Core.QSync-Ref supplies the finite L2 settled snapshot used by J.RegPhaseNF/J.UnitExt. A directly validated J.GWR package may instead carry equivalent per-Horizon

settling evidence. No B2/B3 or generic fair-extension premise is used for either finite-prefix direction; use B4/B5 only for replay ϵ -normalization or uniqueness up to finite ϵ -stutter not discharged by the QSync same-cycle settle. Complete/fair execution obligations belong to later theorems that explicitly require them.

Then the B1-specialized system-level result is:

(Post $\rightarrow$ Ref existence) For every $\tau_{\text{post}} \in \text{Reach_post}$ equipped with a local certificate package satisfying Definition J.LocalGWR and the standard source-side QualifiedSynchronousModel_Sys_ref(I) premise of J.GWR-Comp, Theorem J.GWR-Comp constructs W with $\text{GlobalWitnessRealizable}(\tau_{\text{post}}, W)$, and there exists $\tau_{\text{ref}} \in \text{Reach_ref}$ such that $\tau_{\text{ref}, \text{system}} \approx_{\text{sys}} \tau_{\text{post}, \text{system}}$ under E1–E5. A directly validated J.GWR package may instead supply the equivalent checked per-Horizon settling evidence.

(Ref $\rightarrow$ Post existence, annotated RTL-consistent prefixes only) For every $\tau_{\text{ref}} \in \text{Reach_ref}$ equipped with a selected RTL_B prefix $\tau_{\text{post}}^* \in \text{Reach_post}$ and a compatible witness/annotation package satisfying the annotated RTL-consistency requirement of Theorem J.1 (RTLConsistentRefPrefix), there exists $\tau_{\text{post}} \in \text{Reach_post}$ such that $\tau_{\text{ref}, \text{system}} \approx \tau_{\text{post}, \text{system}}$.

Moreover, because B1 holds in this Appendix G specialization, the matching Σ -label sequence is unique (up to insertion/removal of finite ϵ -steps permitted by B4/B5). That uniqueness is only projection uniqueness; it does not by itself determine μ_Q , failed-poll instances, issue annotations, request attribution, E4 boundary histories, or the corresponding source-side witness prefix/state in Sys_ref.

Proof sketch. Appendix I proves the process-local replay and finite-preparation facts packaged by J.PCert. Theorem J.GWR-Comp combines those facts with J.CCert, J.ACet, environment/reset evidence, and the operational frame/commutation lemmas to construct one reachable Sys_ref witness and J.GWR package. Proposition J.0 then lifts the chosen compatible trace pair to system-level $\approx_{\text{sys}}$, and Theorem J.1 supplies the finite reachable-prefix trace-set statement. Thus the main result is explicitly about sets of finite reachable execution prefixes under fixed stimulus, not about fair-extension-qualified prefixes. The Ref $\rightarrow$ Post direction is not a symmetric arbitrary-source construction; it applies only to annotated RTL-consistent source prefixes, and under this Appendix G specialization B1 supplies only the unique RTL_B Σ -trace against which the projection component of that consistency is judged. The compatible annotation package remains a separate witness obligation and is not inferred from bare Σ -label matching.

Practical Implication (“No Surprises” Guarantee)

If a verification environment exercises only the alphabet Σ (signals, message ports, synchronization calls) and does not test internal latency, then—under the assumptions of Theorem J.1 in Appendix J—the prescribed source reference model Sys_ref and the post-HLS RTL agree at the chosen observable boundary in the following qualified finite-prefix sense:

- for every $\tau_{\text{post}} \in \text{Reach_post}$ whose local certificate package satisfies Definition J.LocalGWR and whose selected Sys_ref satisfies the source-side Core.QSync premise of J.GWR-Comp, Theorem J.GWR-Comp constructs a J.GWR package and a matching $\tau_{\text{ref}} \in \text{Reach_ref}$; a directly validated J.GWR package carrying equivalent per-Horizon settling evidence may be used instead; and
- in the reverse direction, only those source-side $\tau_{\text{ref}} \in \text{Reach_ref}$ that are annotated RTL-consistent in the sense of Theorem J.1—i.e., equipped with a selected RTL_B prefix $\tau_{\text{post}}^* \in \text{Reach_post}$ and a

compatible witness/annotation package satisfying $\text{RTLConsistentRefPrefix}$, not merely a bare Σ -projection match—are guaranteed to have a matching reachable τ_{post} .

Direction-of-proof clarification for source-side safety signoff. Although users often describe this use case as transferring a pre-HLS result “from Ref to Post,” the deductive direction ordinarily used for safety is the $\text{Post} \rightarrow \text{Ref}$ clause. If every reachable RTL_B prefix has a matching Sys_ref prefix with the same canonical Σ_{DUT} observation, then RTL_B introduces no Σ_{DUT} -observable safety behavior absent from Sys_ref . A safety property proved for every covered Sys_ref prefix therefore transfers without the restricted $\text{Ref} \rightarrow \text{Post}$ clause. When Definition J.RefProjCover holds, one complete pre-HLS execution covers those source-reference observations and can support the same transfer. The restricted $\text{Ref} \rightarrow \text{Post}$ clause instead concerns realization of a selected source behavior by RTL_B and remains annotation-dependent.

Latency-sensitive testbench caveat

A testbench that branches on raw cycle counts, uses timeout watchdogs, checks performance/latency bounds, or otherwise changes its stimulus or pass/fail behavior based on timing not represented in Σ is not covered by the same-stimulus trace-compatibility guarantee unless that timing dependence is itself modeled through explicit Σ -visible causal mechanisms such as synchronization events, handshakes, or message-passing transactions. Such checks may still be useful, but they should be treated as latency-sensitive testbench logic, isolated as described in Appendix D, or performed as post-HLS / RTL-level performance checks rather than as properties transferred automatically from an arbitrary pre-HLS simulation run.

Accordingly, the precise “no surprises” reading distinguishes two source-side uses. For safety, no Σ -observable behavior can appear in RTL_B under the fixed stimulus without a matching behavior in Sys_ref ; therefore universal Sys_ref safety, or one complete source run satisfying Definition J.RefProjCover, transfers through $\text{Post} \rightarrow \text{Ref}$ as stated by Corollary J.1-Safe. Separately, a claim that a particular source-side behavior is realized by RTL_B uses the restricted $\text{Ref} \rightarrow \text{Post}$ clause and requires $\text{RTLConsistentRefPrefix}$ or the applicable Appendix L snooping / witness-selection construction.

Thus, for a Σ -only testbench under the same fixed stimulus, any mismatch found on RTL_B corresponds to a matching mismatch on Sys_ref . A source-side safety pass justifies RTL_B through Corollary J.1-Safe under universal source verification or J.RefProjCover, provided J.TraceCertCov supplies the applicable $\text{Post} \rightarrow \text{Ref}$ construction evidence, including source-side Core.QSync on the normal compositional route or equivalent checked per-Horizon settling evidence on the direct J.GWR route; realization of a selected source trace still uses the restricted reverse direction. Here Sys_ref means Sys_B in the ordinary bounded-FIFO case and $\text{Sys_B}^{\text{elab}}$ in the elaborated back-annotated cases of Appendix B / Appendix Q. See Appendix J / Theorem J.1 and Corollary J.1-Safe for the full execution-level statements.

Appendix G states the trace-compatibility rules and a B1-specialized high-level implication. Appendix I supplies process-local deterministic replay and finite preparation certificates; Appendix J proves that these local process facts, together with local channel and atomic-interaction certificates, construct a single globally realizable source witness, and then proves the system theorem for the prescribed source reference model. Taken together, these appendices establish the stated execution-level correspondence at the chosen observable interfaces under fixed stimulus and the named local/tool obligations. The unrestricted guarantee is $\text{Post} \rightarrow \text{Ref}$; the reverse direction ranges only over annotated RTL-consistent source-side behaviors covered by Theorem J.1.

Appendix H – Possible Criticisms

This section highlights several potential challenges in the verification approach of Appendix G.

1. Designer-Managed Shared-Memory Synchronization

Appendix G requires that any memory shared between processes use explicit synchronization primitives inserted by the designer. The HLS tool does not perform static verification of those primitives. In practice, we mitigate this risk by encapsulating shared memories within parameterized library classes (for example, see examples 12_ping_pong_mem and 15_native_ram_fifo) that are pre-verified for correct memory access coordination in both the pre-HLS and post-HLS models. Typically such classes will be parameterized for aspects such as element type and memory size. Formally, the Appendix I–K theorems cover shared memories only under the J.Mem lowering condition; such library classes should present their value-carrying memory traffic at boundaries satisfying J.Mem, or carry their own separate correctness argument outside the Appendix I–K theorems.

2. Latency-Sensitive Signal Alignment (“Snooping”)

See Appendix L for detailed discussion on snooping.

3. By Default, Matchlib Connections Model a Skidbuffer inside In<> Ports

(Note that this overall document and specifically Appendix G are not intended to be strictly tied to Matchlib. Instead, this document is intended to be applicable to any pre-HLS model written in SystemC using signals, message-passing channels, and synchronization calls.)

Matchlib Connections supports throughput accurate modeling in the pre-HLS simulation. Currently the throughput accurate modeling mechanism relies on the Pre() and the Post() methods using a 1-place buffer to store transactions that are in flight on Connections::In<> ports. This means that by default In<> ports function like a skidbuffer. Connections::Out<> ports do not introduce any additional storage capacity into the pre-HLS simulation.

By default, Catapult adds skidbuffers on input ports into the post-HLS design, so the formal trace-compatibility relationship described in this document still holds. In other words, the default pre-HLS and post-HLS designs both still model rendezvous semantics, since both explicitly include a structural instantiation of a skidbuffer on input ports. In effect, we define the rendezvous boundary *before* the skidbuffer.

It is possible to remove some or all the skidbuffers during Catapult HLS. Because the abstract rendezvous boundary is defined before the skidbuffer, this choice is orthogonal to the trace-compatibility rules (R1–R5/E1–E5) as long as skidbuffer-internal behavior is not included in Σ . However, to keep the pre-HLS simulation’s port micro-architecture aligned with the RTL configuration, the following compile flag must then be used in this case:

`-DFORCE_AUTO_PORT=Connections::SYN_PORT`

This will remove all the skidbuffers from the pre-HLS model. In this case, if some of the skidbuffers are still inserted during HLS, the capacity effects of those specific skidbuffers should be added into Sys_B so that the formal trace-compatibility relationship still holds.

In this case the recommended methodology is to use both approaches:

- Use the default throughput accurate mode in Matchlib for most analysis and debug, and for pre-HLS performance verification.
- To keep the pre-HLS simulation's *port micro-architecture* strictly aligned with the RTL configuration, then use `-DFORCE_AUTO_PORT=Connections::SYN_PORT`, and rerun final verification tests.

Appendix I – Per-Process Replay and Preparation Proof

I.1 Overview

This appendix establishes the process-local replay and preparation results used by Appendix J. Under a fixed initial state, fixed external stimulus/environment behavior, and WC-compatible witness, a selected RTL_B observable-unit prefix determines the corresponding source-local replay state for each process and a finite process-owned ϵ preparation path to the exact dynamic operation participating in the next selected unit. Appendix I does not by itself commit a channel transfer, pair rendezvous or synchronization endpoints, or construct a complete global Sys_ref execution; those composition steps are proved in Appendix J by Lemmas J.Frame–J.UnitExt and Theorem J.GWR-Comp.

Downstream citation discipline. Later appendices may cite Appendix I for Implementation assumption I.A0; Definitions I.ReplayObs and I.CutpointOfferObs; Lemmas I.1, I.ReplayObs-Congruence, I.DR, I.DR', and I.3; and the process-local certificate obligations later packaged as J.PCert. Appendix I does not justify an unrestricted per-process or system-level existence theorem in isolation. A complete matching Sys_ref prefix is obtained only after Appendix J combines the process-local paths with the common explicit-channel state, endpoint co-presence, atomic grouping, environment/reset evidence, current request-issue evidence, and interface agreement. The converse direction likewise remains restricted to RTLConsistentRefPrefix / Appendix L witness-selected executions.

I.2 Formal Preconditions

- **Observable alphabet** Σ is as defined in *Normative Formal Core for Appendices G–K*: it consists of the event schemas {Push_c, Pop_c, Sync, wait, START_P, FINISH_P, Write, Read} instantiated only for channels/signals designated observable at the chosen proof boundary. For Appendices I–K, Σ is the proof-internal observable alphabet of the prescribed Sys_ref / RTL_B comparison. Thus all message-passing channels that can participate in Appendix K deadlock reasoning are designated observable at that proof boundary and hence included in Σ . In elaborated cases Sys_ref = Sys_B^elab, this proof-internal Σ may include introduced staged/bundle/join channels even when the projection Π_{elab} hides those internal events from the outer DUT/testbench-visible alphabet Σ_{DUT} . Such events are therefore Σ -visible for the proof-internal comparison, but may be projected away from the outer observation boundary.
- **Non-blocking message-passing interpretation.** In both RTL_B and Sys_ref, a non-blocking call (PushNB / PopNB) is treated as a poll. A successful poll contributes the same observable event

as the corresponding committed $\text{Push}_c / \text{Pop}_c$ on that channel. An unsuccessful non-blocking call (returns “false” / “not completed”) contributes no Σ -event and is modeled as an ϵ -step. A non-blocking poll may still assert a Σ -visible endpoint request in the cycle of that call. However, each executed non-blocking call is a distinct dynamic source-level instance $q \in Q_{\text{exec},P}$ for its owning process P . Accordingly, continuous assertion of an endpoint request with no intervening commit does not by itself collapse later executed non-blocking call instances into an earlier q . Only a poll that succeeds contributes the corresponding committed transfer $m(q) \in M$ for that call instance. If a request assertion persists without a new executed source-level call instance being entered, its attribution remains with that same currently outstanding q ; but repeated non-blocking polls in later loop iterations or later source-level call instances create additional $q \in Q_{\text{exec},P}$ even if req does not deassert.

- **Silent step** Any internal RTL microstate transition is written ϵ . In particular, unsuccessful non-blocking polls, arbitration bookkeeping, and pipeline advances are ϵ -steps.
- **Finite-pipeline-depth premise (FPD)** There exists a finite constant $\text{pipe_depth}(P) = D_P < \infty$ such that once P begins internal micro-progress toward its next observable Σ -action (or that action is locally enabled), P executes at most D_P consecutive ϵ -steps before committing a Σ -action or reaching a stable wait with no further ϵ -step available until external/peer conditions change. In particular, a process cannot perform an unbounded number of ϵ -only failed non-blocking polls while making no progress toward any Σ -action; designs that can spin indefinitely without reaching either a Σ -action or a stable wait state violate FPD/B4 and are outside the model.
- **Weak-fairness premise (WF)** If a micro-operation remains continuously enabled, the scheduler eventually selects it for progress. For an already-issued blocking message request whose BoundaryReq is asserted and whose ChannelEnabled predicate remains true, this means the enabled transfer is eventually allowed to commit; it does not mean that the same source-level operation is “issued” again. This is an assumption on the execution/scheduling environment (see Appendix N), not an automatic guarantee of “clock-synchronous RTL.” In practice it requires that any arbiters/back-pressure logic that can affect whether an enabled operation is selected or allowed to commit must be fair in the sense of B2/B3.
- **Finite-progress invariants (B3)** For every channel c , assume producer and consumer obligations $P_{\text{push}}(c)$ and $P_{\text{pop}}(c)$ hold. In a buffered full or empty condition, these obligations guarantee the matching complementary discharge under their persistent two-endpoint-request premises: $P_{\text{push}}(c)$ yields a Pop_c commit that frees space, and $P_{\text{pop}}(c)$ yields a Push_c commit that provides data. They do not directly assert that the original stalled Push_c or Pop_c commits. After the complementary discharge, E4 determines whether the still-pending original operation is ChannelEnabled , and B2 supplies eventual selection/commit while that enablement remains continuous. For $B(c)=0$, the discharge is the rendezvous completion itself. B3 is an assumption on the execution/scheduling environment (e.g., fairness of arbiters/back-pressure; see Appendix N), not a consequence of the cycle-by-cycle R-rules.
- **Channel enablement semantics.** Let $q \in Q_{\text{exec},P}$ be a dynamic source-level message-operation instance of its owning process P on message-passing channel c , and let $\text{kind}(q) \in \{\text{Push}_c, \text{Pop}_c\}$ denote its endpoint direction. If q has issued a persistent blocking request to transfer on c , then $\text{BoundaryReq}(q,\sigma)$ holds at a settled boundary-evaluation point σ (BoundaryEvalPoint in the Core sense): the endpoint-owned boundary request remains asserted, and the payload is stable if q is a Push. The channel-side commit-enabling predicate for q is determined as follows.

Rendezvous case, $B(c)=0$. A Push_c operation instance q is channel-enabled exactly when

BoundaryReq(q, σ) holds and the matching Pop_c endpoint boundary request is asserted in the same settled boundary-evaluation cycle. A Pop_c operation instance q is channel-enabled exactly when

BoundaryReq(q, σ) holds and the matching Push_c endpoint boundary request is asserted in the same settled boundary-evaluation cycle. Thus, when both endpoint requests are asserted in that cycle, the matched Push_c(v) and Pop_c(v) operation instances are eligible for selection as a rendezvous pair. If the scheduler selects that enabled rendezvous pair in that cycle, the matched Push_c(v) and Pop_c(v) commit as a same-cycle rendezvous pair and contribute the corresponding Σ -events. If the pair remains continuously enabled, B2 supplies eventual selection.

Buffered case, $B(c) > 0$. The channel-side eligibility predicate for q is exactly the FIFO availability predicate conjoined with BoundaryReq(q, σ):

for $\text{kind}(q) = \text{Push}_c$: $\text{ChannelEnabled}(q, \sigma) \Leftrightarrow \text{BoundaryReq}(q, \sigma) \wedge \text{occ}_c(\sigma) < B(c)$ (“not full”), and

for $\text{kind}(q) = \text{Pop}_c$: $\text{ChannelEnabled}(q, \sigma) \Leftrightarrow \text{BoundaryReq}(q, \sigma) \wedge \text{occ}_c(\sigma) > 0$ (“not empty”).

Here ChannelEnabled means that the abstract channel boundary permits the transfer to be selected; it does not by itself assert that the transfer has already committed in the current cycle.

Modeling note (fall-through / simultaneous empty/full cases). The buffered abstraction for $B(c) > 0$ is cycle-boundary, non-fall-through. A Push and a Pop may both be selected and commit in the same cycle when the FIFO begins the cycle neither empty nor full ($0 < \text{occ}_c(t) < B(c)$); in that case both commits are legal and the occupancy is unchanged. The abstraction does not permit a Pop to be selected in a cycle that begins empty solely because a Push is also selected in that same cycle, nor does it permit a Push to be selected in a cycle that begins full solely because a Pop is also selected in that same cycle. If an implementation uses a fall-through FIFO (allowing same-cycle pass-through when empty), model that behavior explicitly (e.g., as an explicit bypass/rendezvous path composed with the bounded FIFO), so that the Σ -level committed-transfer trace matches the intended cycle semantics.

Operational interpretation (availability, selection, and ready/valid): $\text{occ}_c(t)$ is an abstract quantity derived from Σ -visible boundary commits for $B(c) > 0$ buffered channels; processes do not read $\text{occ}_c(t)$ directly. For $B(c) = 0$ rendezvous channels, the corresponding channel-side availability fact is the matching endpoint request predicate in the same settled boundary-evaluation cycle. In both Sys_ref and RTL_B, ChannelEnabled/ChannelDisabled records the settled abstract boundary availability/eligibility predicate. A commit occurs only when an enabled operation, or enabled rendezvous pair, is selected by the scheduler/arbiter for that cycle.

Here Sys_ref denotes the prescribed source reference model at the chosen abstract boundary: Sys_B in the ordinary case, or Sys_B^elab in the elaborated back-annotated cases. A concrete realization is correct only if its committed ready/valid transfers are legal with respect to the corresponding rendezvous or buffered availability predicate at that chosen boundary, and if every transfer that remains continuously eligible is eventually selected as required by B2. Thus E4 is a safety/legality condition on which transfers may commit, while B2 is the progress/fairness condition that prevents continuously eligible transfers from starving.

No additional hidden availability predicate is part of the channel semantics. Any cross-channel

coordination, arbitration, grant, or back-pressure policy in the realization must be represented either

(1) as scheduler/arbiter selection among already ChannelEnabled actions, governed by B2, or
 (2) as explicit abstract boundary structure in Sys_B^{elab} with its own B($\cdot$), occ_ $\cdot$ (t), BoundaryReq, ChannelEnabled/ChannelDisabled, and B3 obligations. Such coordination must not be encoded as an unmentioned extra conjunct that silently changes the E4 availability predicate at an otherwise ordinary channel boundary.

Therefore, when the boundary request is true and the relevant rendezvous or FIFO availability predicate holds continuously, the transfer is continuously enabled; by weak fairness (B2), the corresponding commit occurs after a finite, though not necessarily bounded, delay.

- **No ill-formed signal IO.** Every observable signal Read/Write has a unique real bounding synchronization operation on each dynamic path on which it occurs; otherwise, the design is ill-formed. START_P may be used as an initial anchor only when it is included as a real synchronization event in S, and FINISH_P may be used as a terminal anchor only when the process actually terminates. The formal model does not use a virtual synchronization event at ∞ . (From RULE 1 and RULE 2.)
- **Fixed-stimulus interpretation (reactive environments).** If the “same stimulus” includes a reactive testbench and/or peer processes, it is interpreted as the same global Σ -causal behavior (same externally supplied inputs, and the same peer/environment responses to the same prior Σ -visible history), not as a function of P-local history alone. In Appendix I this interpretation is used only as a witness-replay condition: when a next RTL_B event e is already fixed in the chosen post trace prefix, any peer/environment enabling needed for e is part of that same fixed global Σ -causal behavior and is therefore replayed consistently for the matching Sys_ref construction. This is not a standalone assumption that arbitrary Sys_ref continuations are globally live.
- **WC (Witness-compatible non-blocking outcomes).** Because failed polls are modeled as ε (unobservable), all correspondence results in Appendices I–K are stated for witness-compatible comparisons: any ε -modeled non-blocking poll outcome (and any other ε -modeled internal choice that can affect the next Σ -visible control flow / next Σ -visible blocking frontier) is fixed/selected to be consistent with the compared RTL_B execution prefix τ_{post} (e.g., via the Appendix L witness-selection/snooping wrapper). WC also supplies the μ_Q matching for ε -modeled failed non-blocking call instances when E3/E5 or the proof construction refers to $q \in Q_{\text{exec},P}$; those failed q ’s are not inferred from Σ alone.
- **Remark (NB-CF as a sufficient condition).** If a design satisfies the following non-blocking control-flow property—failed PopNB/PushNB polls do not influence the next Σ -visible control flow/frontier—then the witness is trivial and WC holds automatically. Equivalently: between two consecutive Σ -events of a process, inserting/removing any finite sequence of failed PopNB/PushNB polls may change only internal ε -behavior/timing, not which Σ -visible action (Push/Pop/synchronization anchor) is next in program order.

I.3 Auxiliary Lemmas

Implementation assumption I.A0 (RTL message-issue discipline).

The post-HLS RTL_B execution satisfies the full E3 obligation as an implementation-side scheduling obligation: (i) per-interface issue order is preserved for all message operations; (ii) for distinct interfaces, the no-reverse rule holds; and (iii) issue is prefix-closed under source order, so at every compared

cutpoint a source-later issued message operation implies that every required source-earlier message operation has also issued, with Issue timestamps ordered accordingly. This condition is not derived inside Appendix I; it is the RTL/tool-compliance obligation corresponding to the scheduling rules stated in the main text and Appendix G. For hand-written RTL or other non-HLS implementations, I.A0 is the corresponding proof/qualification obligation on that implementation.

Lemma I.1 (Bounded ϵ -chain to Σ -action or stable wait).

Assume the finite-pipeline-depth / finite-internal-stutter premise FPD/B4. From any reachable well-formed cycle-boundary state of P in the Appendix I construction, if P is not already stably waiting on an external/peer enabling condition, then within at most D_P consecutive P-owned epsilon-steps of internal progress, P either:

- (i) commits its next observable Sigma-action, or
- (ii) reaches a stable waiting state in which no further P-local epsilon-step is available until some external, peer, channel, or stimulus condition changes the relevant enabling predicates.

Equivalently, P cannot produce an infinite P-local epsilon-only execution segment from a reachable well-formed state while remaining internally enabled. Any unbounded delay that remains after the stable-wait point is therefore attributable to external/channel/stimulus enabling and is governed by WF/B2 and B3, not by local epsilon-stutter. When a clock-cycle bound is required, the proof uses the separate B5 bound C_{sys} , or a proved bucket-semantic charging lemma deriving a per-process cycle bound C_P from D_P ; Lemma I.1 does not identify one epsilon-step with one clock cycle.

Proof. Direct from FPD/B4 for the epsilon-step conclusion, together with the separately stated B5 or bucket-semantic bridge only when a cycle-count conclusion is used. ■

Lemma I.2 (Eventual space / data). Assume P is blocked on either:

- Push_c with $occ_c = B(c)$, P's Push_c request remains asserted with stable payload for the outstanding transfer, and the complementary endpoint continuously requests the matching Pop_c until the matching discharge event (Pop_c endpoint request asserted; Pop_c boundary request remains true); or
 - Pop_c with $occ_c = 0$, P's Pop_c request remains asserted for the outstanding transfer, and the complementary endpoint continuously requests the matching Push_c until the matching discharge event (Push_c endpoint request asserted with stable payload; Push_c boundary request remains true).
- Then a complementary Pop_c (respectively Push_c) commits within finite time.

Proof. (By B3 / Appendix N.) In either case, P is stalled at a blocking channel call while its persistent request for that outstanding transfer remains asserted until commit, with stable payload if it is a Push. By the additional premise, the complementary endpoint also continuously requests the matching operation until the corresponding discharge event, with its boundary request remaining true. Therefore the interval-scoped premises of P_push/P_pop (Appendix N) hold, and the matching complementary discharge must eventually occur: for Push_c at full, some complementary Pop_c eventually commits, freeing space; for Pop_c at empty, some complementary Push_c eventually commits, providing data. The premise does not require either endpoint request or Push payload to remain asserted after that matching transfer has committed.

■

Definition I.ReplayObs (replay-derived Σ -relevant source-state slice).

Fix a process P, a fixed external stimulus, and a fixed witness w satisfying WC for the chosen observable boundary. Define $ReplayObs_{\{\Sigma, P, w\}}(\sigma)$ to be the logical P-local replay checkpoint represented at operational state σ : the tuple of source-level facts determined by the complete P-local observable-unit replay history under that stimulus and witness and needed to determine later P-local Σ -visible behavior or replay-derived Appendix I–K predicates. ReplayObs is a projection of σ , not the complete raw operational source state at σ . Its control-position field is the logical replay control position immediately

after the last matched observable unit (or at the selected initial/reset checkpoint), and its value fields contain only replay-relevant logical source variables and latched values reconstructed from committed history, the fixed stimulus, and w . Optional outcome-independent L1 preparation may advance the raw operational cursor, calculate prepared temporaries, or assert source-later requests without changing this logical checkpoint, but only when the field-sensitive footprint, alias-closure, and NoDependentUse obligations certify that the segment is replay-inert.

The tuple includes: the logical replay control position and reset epoch; replay-relevant source variables and latched values used to evaluate future control predicates Pred_x , payload/value expressions, anchored Write values, and anchored Read labels; witness-selected epsilon outcomes that can affect later Σ -visible control flow; $Q_{\text{exec},P} / Q_{\Omega,P}$ dynamic-operation attribution facts determined by the replay prefix; completed/realized-item status; candidate NextFront_P ; anchored signal values and returned Pop/NB results reconstructed from the committed history and witness; and P 's normal-completion status. Raw preparation progress—including the operational cursor, prepared local temporaries, current issue attribution, and current request assertions—is not part of ReplayObs . In particular, Pending_P , Front_P , and BoundaryReq are not ReplayObs fields. Define $\sigma \sim^{\text{replay}}_{\{\Sigma,P,w\}} \sigma'$ iff $\text{ReplayObs}_{\{\Sigma,P,w\}}(\sigma) = \text{ReplayObs}_{\{\Sigma,P,w\}}(\sigma')$. The replay-derived source-state slice of σ is its equivalence class $[\sigma]^{\text{replay}}_{\{\Sigma,P,w\}}$.

Replay-abstraction and selected-normalization convention. Not every epsilon-step is replay-inert. A witness-fixed epsilon transition, such as a failed non-blocking poll that writes a returned status or another replay-relevant field, may change ReplayObs . WC fixes the selected outcome; it does not make that transition neutral. Deterministic normalized replay therefore uses paired congruence: from equal logical replay checkpoints, corresponding transitions under the same witness produce equal successor checkpoints. A selected finite epsilon-normalization may contain both (a) certified replay-inert preparation segments, which leave each logical checkpoint unchanged, and (b) witness-fixed replay-changing transitions, which must map equal checkpoints to equal successors. The selected normalization convention is proof-relevant and must satisfy this congruence requirement; the document does not assert that every legal normalization has the same raw operational endpoint or that each normalization leaves one execution's ReplayObs unchanged.

Definition I.CutpointOfferObs (current blocking-offer slice).

For a reachable source state σ at the selected abstract boundaries, define $\text{CutpointOfferObs}_{\{P,E\}}(\sigma)$ to be the P -owned current blocking-offer facts: the uncommitted blocking message-operation frontier items whose endpoint requests are asserted, their Pending_P and Front_P membership, BoundaryReq values, owning dynamic message-call instances, evaluated explicit boundaries, endpoint directions, reset epochs, and the stable Push payload obligation where applicable. CutpointOfferObs is indexed by the actual operational issue schedule and raw L1 preparation progress and is not determined by the committed observable-unit history alone: two legal source executions can share the same logical ReplayObs checkpoint while differing in how far outcome-independent preparation has advanced and whether a source-later blocking operation has already issued. Appendix J constructs these request assertions through its phase-normal preparation/issue actions, represents the normalized cutpoint slice by the residual-offer set O , proves source reachability through $J.\text{OfferReach}$, and audits the corresponding RTL request inventory through $J.\text{OfferConf}$. The complete K -facing cutpoint observation therefore combines replay history H with current residual offers O ; deterministic replay never manufactures O from H .

This quotient intentionally ignores purely internal diagnostic state and epsilon-only micro-timing state that cannot affect any future Σ -visible label, replay-derived frontier predicate, payload/value expression, or Appendix I–K predicate. $\text{ChannelEnabled} / \text{ChannelDisabled}$ facts are not independently added to ReplayObs . They are obtained by combining the logical ReplayObs checkpoint with the operational I.CutpointOfferObs slice—including BoundaryReq , Pending_P , and Front_P —and with the corresponding

channel-state facts in the chosen Sys_ref boundary model, such as $B(c)$, $occ_c(\sigma)$, rendezvous complementary endpoint requests, and any explicit Sys_B^elab staged/bundle/join boundary state. For cadence-sensitive non-blocking polling sites, such as retry/timeout counters whose externally visible behavior depends on the number or cadence of failed polls, this definition is applied only after the site has been handled as an isolated or snoop-aligned latency-sensitive local protocol block under Appendix D / Appendix L. The ordinary Appendix I source-core proof does not derive retry/timeout behavior from an unconstrained hidden failed-poll cadence.

Lemma I.ReplayObs-Congruence (one-step closure of the replay-derived slice).

Fix P , the external stimulus, the chosen observable boundary, and a witness w satisfying WC. Let σ_1 and σ_2 be source states of P such that $ReplayObs_{\{\Sigma, P, w\}}(\sigma_1) = ReplayObs_{\{\Sigma, P, w\}}(\sigma_2)$. Suppose that, from σ_1 and σ_2 , the same next P -local observable unit u is produced under the same witness w and the same fixed-stimulus interpretation, where u may be a committed Push/Pop value, successful non-blocking transfer, synchronization/wait unit, anchored Read/Write unit, START_P/FINISH_P unit, R2C fixed-point observation unit, or another explicitly atomic Σ -visible boundary unit. Let $\hat{\sigma}_1$ and $\hat{\sigma}_2$ be the corresponding canonical immediate post-unit states. Then their ReplayObs checkpoints are equal. If σ'_1 and σ'_2 are instead the epsilon-quiescent endpoints selected from $\hat{\sigma}_1$ and $\hat{\sigma}_2$ by the fixed proof-relevant normalization convention, require that convention to preserve replay equivalence as specified above: replay-inert preparation segments leave the logical checkpoint unchanged, while corresponding witness-fixed replay-changing epsilon transitions map equal checkpoints to equal successors. Under that selected convention:

$$ReplayObs_{\{\Sigma, P, w\}}(\sigma'_1) = ReplayObs_{\{\Sigma, P, w\}}(\sigma'_2).$$

Proof. By case analysis on the next source construct and on the kind of observable unit u .

For deterministic local computation, equality of the logical replay control position and of all replay-relevant variables/latched values feeding future predicates, payloads, anchored writes/reads, guards, and replay-derived proof predicates implies equality of the updated ReplayObs fields. Raw L1 cursor movement, prepared temporaries, and request assertions are outside ReplayObs and may differ only under the certified replay/preparation noninterference obligations.

For anchored sequential signal reads/writes and emitted R2C fixed-point units, the relevant value and component-local observation identity are fixed by the common stimulus, witness, anchor/fixed-point semantics, and common unit u . Therefore all future-relevant latched values, anchored-label facts, and R2C replay facts agree after the unit.

For committed blocking Push/Pop and synchronization units, the common unit u , together with equality of the logical replay control position, relevant payload variables, replay-prefix dynamic-operation attribution, and completed-item history, determines the same replay-derived control/data update and candidate NextFront_P. Whether a not-yet-committed blocking operation was already issued before the unit is not a ReplayObs fact and is not concluded equal here; that current assertion state belongs to CutpointOfferObs and is established by explicit operational issue evidence in Appendix J.

For successful non-blocking PushNB/PopNB, the successful transfer is included in u as the corresponding committed Push_c(v) / Pop_c(v) unit. The common unit and equal current ReplayObs determine the same control, returned-data, completed-item, dynamic-attribution, and candidate-next-front updates.

For failed non-blocking PushNB/PopNB, the failed poll is an epsilon-step. If NB-CF holds, failed polls cannot affect the next Σ -visible control flow, dynamic-operation selection, guard values, payload values, or candidate NextFront, so any difference is outside ReplayObs. If NB-CF does not hold but WC is supplied by Appendix L, the selected failed-poll outcome may update returned-status, witness, or other replay fields. WC fixes that outcome; it does not make the step replay-neutral. The normalized proof therefore pairs the corresponding failed-poll transitions and uses equal prior checkpoints plus the same

witness-selected outcome to obtain equal successor ReplayObs. Cadence-sensitive sites remain subject to the Appendix D / Appendix L isolation or snoop-alignment requirement.

For any other epsilon-only implementation choice, WC requires that every outcome capable of affecting later Σ -visible behavior or replay-derived Appendix I–K predicates is selected by w and is causally consistent with the matched prefix. A certified replay-inert preparation step preserves each logical checkpoint by framing. A witness-fixed replay-changing step need not be neutral, but equal prior ReplayObs checkpoints and the same selected outcome must produce equal successor checkpoints. Current request assertions remain outside this conclusion.

Therefore equality of ReplayObs is preserved by the canonical immediate same-unit replay step and, for normalized cutpoints, by the two-sided congruence of the exact selected normalization paths. This is not a claim that each selected normalization preserves one side’s ReplayObs individually, nor a claim of global normalization confluence. No equality of CutpointOfferObs is asserted. ■

Lemma I.DR (Deterministic replay / replay-derived source-state determinacy under fixed stimulus + witness). Fix a process P , a fixed external stimulus, a fixed witness w satisfying WC, and the proof-relevant epsilon-normalization convention selected for the theorem instance. Consider two executions of the same P source code that start from the same initial source-level state and are run with that same witness. If their chosen P -local observable-unit replay projections are the same, then for every k , $\text{ReplayObs}_{\{\Sigma, P, w\}}$ at the selected epsilon-quiescent cutpoint after the first k P -local observable units is identical in the two executions. The selected normalization convention must satisfy the paired replay-congruence requirement of Definition I.ReplayObs and Lemma I.ReplayObs-Congruence; arbitrary maximal epsilon-extensions are not quantified here. “ P -local observable-unit replay projection” means the sequence obtained from the document’s event-matching / Appendix J observable-unit convention: explicitly atomic same-cycle transactions are grouped as single units; independent same-cycle bucket members are not given an intrinsic order by the bucket itself; and any ordering used for deterministic replay is the selected witness/replay order, together with w and μ_Q for epsilon-modeled failed non-blocking $q \in Q_{\text{exec}, P}$ when those instances are needed by E3/E5. Thus Lemma I.DR is neither a determinacy theorem from a bare unordered Σ -bucket projection nor a theorem that reconstructs current CutpointOfferObs/Pending_P/Front_P/BoundaryReq from committed history.

Proof. At $k = 0$ the executions start from the same initial source-level state, so their logical $\text{ReplayObs}_{\{\Sigma, P, w\}}$ checkpoints agree. For the induction step, assume the checkpoints agree after k P -local observable units. The two executions have the same $(k+1)$ -st P -local observable unit and are run under the same fixed stimulus and witness. Lemma I.ReplayObs-Congruence first gives equal canonical immediate post-unit checkpoints and then applies the paired congruence of the two exact selected normalization paths. Therefore the selected normalized ReplayObs checkpoint agrees after every finite P -local observable-unit prefix. ■

Lemma I.DR’ (Immediate post-unit deterministic replay under fixed stimulus + witness). Fix a process P , a fixed external stimulus, and a fixed witness w satisfying WC for the chosen observable boundary.

Consider two executions of the same P source code that start from the same initial source-level state and are run with that same witness. Suppose that their chosen P -local observable-unit replay projections through the first k units are the same. Stop each execution at the canonical immediate post-unit semantic state: after the transition, or explicitly atomic transition group, that realizes the k th P -local observable unit, and before any optional following L1 preparation or epsilon-normalization. Then the two executions agree on the P -owned logical ReplayObs checkpoint at that point, including:

- the logical replay control position, not the raw post-preparation operational cursor;
- all replay-relevant source variables and latched values that can influence future Pred_x predicates, payload/value expressions, or anchored Read/Write values, excluding prepared temporaries that are certified outside the replay projection;

- the dynamic-operation attribution and completed/realized-item facts determined by the replay prefix;
- the candidate NextFront_P, returned-result, anchored-signal, reset-epoch, and normal-completion facts determined by that prefix.

This lemma does not assert that the immediate post-unit state is epsilon-quiescent or drain-closed, and it does not identify that state with a later epsilon-normalized cutpoint. Lemma I.DR remains the corresponding theorem for endpoints selected by the fixed paired-congruent normalization convention. Neither lemma determines raw L1 preparation progress or the current asserted blocking-request inventory: Pending_P, Front_P, and BoundaryReq belong to I.CutpointOfferObs and, at the K-facing normalized cutpoint, are represented by O and justified by J.OfferReach/J.OfferConf.

Proof. By induction on k. The base case follows from equality of the initial source-level state. For the induction step, equality of the prior immediate ReplayObs state, together with the same fixed stimulus, witness-selected ϵ outcomes, dynamic-instance attribution, and next observable unit, determines the same finite source transition segment through the transition or explicitly atomic transition group that realizes the next unit. Therefore the two canonical immediate post-unit states agree on every listed replay-derived fact. The proof makes no claim about issue-schedule-dependent CutpointOfferObs, and no maximal ϵ -closure is invoked. ■

I.4 Local Preparation Construction for Finite Observable-Unit Prefixes (Post $\rightarrow$ Ref support)

Fix a process P, a finite selected RTL_B prefix τ_{post} , and a finite rank-independent required-prefix candidate H as defined by J.RequiredPrefixCandidate below. Let $\text{Replay}_P(H, w)$ be the deterministic immediate post-unit source replay state obtained from the P-local projection of H under the fixed stimulus, WC witness w, and μ_Q dynamic-instance matching. Let γ be a not-yet-matched observable unit in which P participates, and let $q_P(\gamma)$ denote P's matched dynamic source operation, if γ has a process-owned operation. This local construction is indexed without reference to Dep_J, J.DepAcyclic, or J.CanonicalRank; Appendix J later instantiates it at the prefixes selected by the validated construction order.

The process-local input to the construction consists only of P's source code and state, P-owned endpoint request state, the fixed witness and dynamic identities, and read-only summaries of the incident explicit boundaries named by γ . It does not include a premise that γ is globally enabled or that a complete Sys_ref extension exists.

Definition I.DeferredNBHole (carried non-blocking decision obligation).

A deferred hole for dynamic call q at boundary c is the proof-construction record $\text{NBHole}(q, c, \kappa, t, \text{dst}, e, \chi)$, where κ is the snapshot key, t is the selected Horizon, dst identifies the returned status/data destinations, e is the reset epoch, and χ records the WC-selected expected outcome together with $\text{NoDependentUse}(q)$. $\text{NoDependentUse}(q)$ is a locally checkable assertion that no action executed while the hole remains unresolved reads $\text{NBResult}(q)$, uses q's outcome to select reachability/control/endpoint/address/payload, or otherwise depends on the returned value/status.

The dependence test is semantic and transitively closed: it includes aliases and pointer-derived accesses, loads from result-containing locations, derived dataflow expressions, and control, address, endpoint-selection, and payload dependence. The local certificate or checker shall justify this closure using the normative field-sensitive location/footprint model rather than a purely syntactic scan for $\text{NBResult}(q)$. The record also retains the attributed one-cycle attempt and every location needed to reconstruct $\text{ReqSnapshot}_c(q)$. It is ghost construction state, not KObs state. All deferred holes for Horizon t must be resolved or retired before the exported post-boundary cutpoint of t.

Lemma I.3 (Finite process-local preparation script with carried shared-decision holes). Assume I.A0, WC, Core.RegSeq for sequential P, the source well-formedness conditions, and equality between P's current source state and $\text{Replay}_P(H, w)$. Assume the P-local certificate identifies $q_P(\gamma)$, the finite dynamic

source-control segment leading toward it, and every locally generated Dep_J predecessor required by that segment. Then there exists a finite preparation script $\text{Prep}_P(H, \gamma)$ consisting of P -owned ε /request actions plus zero or more carried $I.\text{DeferredNBHole}$ records. Each action has the field-sensitive footprint and Horizon metadata required by $J.\text{ConstructionAction}$, respects $E3/E5$, synchronization boundaries, same-interface order, issue prefix closure, persistence, stable payload, and commits no unmatched Σ unit. When a witness-significant $\text{PushNB}/\text{PopNB}$ decision is reached, the script asserts/records the one-cycle attempt, creates $\text{NBHole}(q, c, k, t, \text{dst}, e, \chi)$, and does not locally choose success or failure. The script may nevertheless continue within Horizon t through source-later actions certified by $\text{NoDependentUse}(q)$ and by disjoint snapshot/result footprints; this includes a later pipeline iteration when it is outcome-independent. It stops at the first action that depends on q 's result, would change $\text{ReqSnapshot}_c(q)$ without an ordered dependency, crosses a synchronization boundary, or leaves the $L1$ pre-settle phase. Every outcome-dependent continuation is assigned a Horizon strictly greater than t . A successful deferred decision remains a complete observable unit; a failed deferred decision is an ε boundary result. Filling all holes and completing the selected Horizon yields the applicable next-frontier/issued-pending state for γ , with no unresolved hole exported at the post-boundary cutpoint. Proof. Induct over the finite source-control segment certified for $q_P(\gamma)$, using Core.RegSeq and the typed location footprints. Deterministic local statements preserve the replay slice by Lemma $I.\text{ReplayObs-Congruence}$; anchored and direct-input values are fixed by their $E2$ /stability contracts. Blocking requests and NB attempts may be asserted in the certified $L1$ /source-order prefix and remain attributed with stable payload. At a witness-significant NB call, emit the enriched deferred-hole record instead of forcing a global stop. Continue only through actions whose reachability and operands are independent of the unresolved result and whose writes are disjoint from the hole's snapshot/result locations; record $\text{NoDependentUse}(q)$ for each such step. The first dependent use receives a strictly later Horizon by $\text{ReturnedAtCycle}/\text{Core.RegSeq}$. Appendix J later reconstructs the common settled snapshot and applies $J.\text{NBDecision}$. Success is a complete unit; failure writes the returned status and retires the attempt without channel commit. $\text{FPD}/B4$ makes each $L1$ segment and the complete per-Horizon script finite. Field-sensitive framing ensures that carried holes and independent continuation coexist without changing one another's semantics. ■

Corollary I.4 ($J.\text{PCert}$ construction). Applying Lemma I.3 to every process participation and every applicable finite $J.\text{RequiredPrefixCandidate}$ yields the process-local records required by Definition $J.\text{PCert}$: deterministic replay state, exact dynamic-operation identity, finite per-Horizon preparation scripts, issued/pending attribution, stable-payload facts, enriched deferred NB holes with result destinations and NoDependentUse evidence, field-sensitive action footprints, $\text{ReturnedAtCycle}/\text{Horizon}$ facts, and locally generated dependency facts. This corollary remains local; $J.\text{CCert}$ and $J.\text{NBDecision}$ must evaluate each shared-state poll from the common settled boundary snapshot, $J.\text{RegPhaseNF}$ must normalize the complete Horizon, and $J.\text{ACert}/J.\text{UnitEnable}$ must establish the complete observable units.

I.5 Countable Local Replay Chains and Scope

For a process whose selected RTL_B projection contains countably many observable units, repeated application of Lemma I.3 and Lemma $I.DR'$ constructs a countable family of process-local replay/preparation scripts. Each finite record is indexed by a finite rank-independent $J.\text{RequiredPrefixCandidate}$. $J.\text{GWR-Comp}$ later instantiates that family only at the candidates that occur as prefixes of the theorem-derived validated construction order. The construction does not independently evaluate shared-boundary non-blocking decisions, select channel commits, or choose an infinite global scheduler.

If P eventually emits no further observable unit, its local replay record simply stabilizes after its last participation; this does not justify appending a global $B6$ idle suffix. Appendix I constructs only the countable family of finite local replay/preparation records and makes no existence claim for an infinite global Sys_ref execution. If a later theorem requires a complete/fair source execution, that theorem

must supply or construct it under its own continuation/progress premises; Appendix K, for example, uses Definition Core.LAdm, Exec_L^adm, and Lemma K.2b.E. B6 may supply global idle stutter only when such a theorem explicitly uses an infinite terminal-idle suffix and the complete system satisfies Core.TerminalIdle, including both the genuine B5/global-fixed-point condition and its machine/environment no-future-enabling proof.

Thus Appendix I proves local replay determinacy and finite local preparation. Appendix J is solely responsible for gluing the local paths into one reachable global execution, validating shared channel and atomic interaction state, and establishing the Post $\rightarrow$ Ref system witness. ■

Appendix J – System-Level Trace Compatibility Proof

Synchronization-pair well-formedness for $\approx$.

For SyncChannel/ac_sync operations with paired endpoints, the compared Sys_ref and RTL_B executions use the same dynamic synchronization-transaction pairing. That is, the two endpoint events of a matched sync_out/sync_in transaction are matched by dynamic sync-instance identity, commit as one two-party barrier transaction, and are treated as simultaneous events in the same observable cycle bucket. This synchronization transaction induces the same cross-process before/after ordering as described in the causal-dependency definition.

Equivalently, E1 preserves the per-process source order of synchronization endpoint events, but E1 does not weaken or replace the paired-barrier semantics of SyncChannel/ac_sync. The system-level trace-compatibility predicate $\tau_{\text{ref},\text{system}} \approx \tau_{\text{post},\text{system}}$ is defined only over synchronization-well-formed traces in this sense: if one endpoint of a dynamic SyncChannel/ac_sync transaction appears in the trace, then the matching endpoint appears in the same observable synchronization transaction, with the same dynamic pairing, in both Sys_ref and RTL_B.

Theorem J.1 (Execution-Level / Trace-Set Compositional Compatibility)

Fix an initial state and a fixed external stimulus/environment behavior (e.g., the same testbench-driven inputs) for both the prescribed source reference model Sys_ref and RTL_B, with Sys_ref as defined in Remark 2. The raw rendezvous model Sys is the special case obtained when Sys_ref = Sys_B and all effective capacities are zero.

Clarification (reactive environments). Interpret “fixed stimulus” per Appendix I.2, “Fixed-stimulus interpretation (reactive environments).”

Definition J.Reach (finite reachability sets Reach_post / Reach_ref). Let Reach_post denote the set of all finite execution prefixes of RTL_B reachable under the fixed stimulus, with no fair-extension qualifier; define Reach_ref analogously for Sys_ref. Both finite-prefix directions of Theorem J.1 and the cutpoint correspondence of Corollary J.1 are stated over Reach_post / Reach_ref. Their finite-ideal/selected-pair arguments never use the existence of a fair infinite extension, so a genuinely bad reachable prefix (for example, one ending at a deadlock cutpoint) remains in scope regardless of whether it can be embedded in a fair infinite execution. Appendix J therefore introduces no generic fair-extension-qualified execution domain and no generic reachable-prefix-to-fair-execution theorem. Any later theorem that requires a complete/fair execution states its own execution domain and continuation premises; Appendix K uses Exec_L^adm and Lemma K.2b.E for the liberal source continuation and K.CompleteS for the selected Policy-S run.

Cycle-bucket and required-prefix convention for this theorem. A system execution may be represented as a sequence of observable cycle buckets $\beta_1 \beta_2 \dots$, where β_i is the finite set of Σ -events committed in one observable clock cycle of that particular execution. Same-cycle events inside a bucket are unordered except for the explicit constraints imposed by E1–E5, the channel semantics, signal-anchor semantics, and explicit synchronization constructs.

The bucket decomposition records the clock timing of one execution; it is not, by itself, part of the cross-model compatibility obligation. The relation $\tau_{\text{ref},\text{system}} \approx \tau_{\text{post},\text{system}}$ does not require Sys_ref and RTL_B to use the same observable cycle-bucket decomposition. Events that are not ordered by the explicit required constraints may be placed in different cycles or grouped into different buckets in the two executions, provided the matched executions preserve all ordering, timestamp, FIFO-legality, and atomicity constraints required by E1–E5 and by the explicitly modeled channel, signal, and synchronization semantics.

The prefix order used by Theorem J.1 is required-prefix order over matched observable units, not RTL_B bucket-prefix order and not the full Appendix G causal-dependency graph $\rightarrow$. An observable unit is either:

- a singleton Σ -event; or
- an explicitly atomic same-cycle transaction that the semantics requires not to be split, including a rendezvous $\text{Push_c}(v)/\text{Pop_c}(v)$ transfer, the two endpoints of a paired $\text{SyncChannel}/\text{ac_sync}$ transaction, an R2C combinational fixed-point signal-observation unit, a proof-internal RESET boundary unit when dynamic reset is used (Normative Formal Core, reset well-formedness RW1–RW5), and any other same-cycle transaction explicitly identified by the signal/channel semantics.

E2 signal-anchor grouping convention: a synchronization commit together with the sequential-process signal Read/Write events that E2/R2 anchor to it (and therefore require to share its commit cycle) may — and, for mechanization, should — be grouped as one explicitly atomic same-cycle unit. This grouping replaces the same-cycle equality constraint between separately enumerated units by unit-internal atomicity and is consistent with the $\text{J.RegPhaseNF}/\text{J.UnitExt}$ treatment of a synchronization commit and its anchored signal actions as one same-cycle observable unit. When anchored signal events are instead enumerated as singleton units, their E2 timestamp equality with the anchor is enforced by the Sys_ref realization semantics (R2) and the within-cycle listing convention, not by the strict order $<_J$, which never orders same-cycle-aligned events.

Let $<_J$ be the strict required-order relation over observable units generated only by the explicit constraints that must be preserved by $\tau_{\text{ref},\text{system}} \approx \tau_{\text{post},\text{system}}$:

- E1 synchronization source-order constraints within each process;
- E2 signal-anchor timestamp constraints, interpreted as required alignment with the relevant synchronization or R2C fixed-point unit rather than as raw ordering among all source actions;
- E4 channel/FIFO legality for explicit abstract channels, including same-channel FIFO order for $B(c)>0$ and atomic rendezvous grouping for $B(c)=0$;
- E5 immediate synchronization-boundary constraints;
- paired $\text{SyncChannel}/\text{ac_sync}$ barrier semantics; and
- any other explicitly stated signal/channel/synchronization atomicity constraint.

Per-process source order contributes to $<_J$ only where it is reflected in one of these explicit required constraints, such as synchronization order, same-endpoint/same-channel order, signal anchoring, or immediate synchronization-boundary legality. E3 contributes issue-order and request-attribution obligations, but it does not by itself create a strict predecessor edge between distinct-interface

committed observable units merely because their source calls are lexically ordered. The Appendix G causal-dependency relation $\rightarrow$ is not a generator of $\prec_J$.

A theorem prefix is a finite ideal of observable units under $\prec_J$, i.e. a finite down-closed set H such that $\gamma \in H$ and $\gamma' \prec_J \gamma$ imply $\gamma' \in H$. The proof below proceeds by induction along the finite ideal chain induced by the selected J .ValidatedConstructionOrder. On the normal compositional route, that order is derived from Definition J.DepJ, Lemma J.DepRank, Lemma J.DepAcyclic, and J.CanonicalRank. On the optional direct route, it is supplied with an independently checked well-founded construction order satisfying the same required-predecessor and ideal-prefix obligations. Acyclicity of $\prec_J$ on the fixed execution prefix. For the chosen legal RTL_B execution prefix, assign each observable unit $\gamma \in U(\tau_{\text{post}})$ its RTL_B commit cycle $\text{clk}(\gamma)$. If γ is an explicitly atomic same-cycle unit, such as a rendezvous Push_c(v)/Pop_c(v) transfer or a paired SyncChannel/ac_sync transaction, $\text{clk}(\gamma)$ is the common commit cycle of the endpoints in that atomic unit.

Every strict generator edge $\gamma_1 \prec_J \gamma_2$ implies a strict cycle inequality $\text{clk}(\gamma_1) < \text{clk}(\gamma_2)$ in that same legal execution prefix:

- E1 synchronization source-order constraints impose strict ordering between the relevant synchronization commits.
- E4 buffered-channel FIFO/order/capacity constraints impose strict commit-cycle ordering through reset-empty FIFO order, bounded occupancy, and non-fall-through cycle-boundary availability. In particular, for $B(c) > 0$, a Pop_c can consume only a value already present at the relevant cycle boundary, and a Push_c that depends on space becoming available after a full condition is enabled only in a later cycle after the required Pop_c effect has occurred.
- E5 immediate synchronization-boundary constraints impose strict separation between a synchronization commit and later suff_S work whose issue/commit is after that synchronization boundary.
- Paired synchronization/barrier before/after constraints impose the corresponding strict before/after cycle relation between distinct observable units when such a relation is explicitly part of the synchronization semantics.
- E2 signal-anchor constraints contribute timestamp equality/alignment to the relevant synchronization or R2C fixed-point unit, or same-cycle atomicity when explicitly specified; they do not by themselves introduce an independent strict edge between distinct observable units.
- Rendezvous Push/Pop endpoints, paired same-cycle synchronization endpoints, and other explicitly atomic same-cycle transactions are represented as single observable units, not as strict internal edges.
- Non-strict issue simultaneity permitted by E3 does not create a strict $\prec_J$ edge.

Therefore any cycle $\gamma_0 \prec_J \gamma_1 \prec_J \dots \prec_J \gamma_0$ over $U(\tau_{\text{post}})$ would imply $\text{clk}(\gamma_0) < \text{clk}(\gamma_1) < \dots < \text{clk}(\gamma_0)$, an impossibility. Hence $\prec_J$ is acyclic on $U(\tau_{\text{post}})$, which is the only acyclicity needed for the finite-ideal induction. For the proof of Theorem J.1, Definition J.DepJ and Lemmas J.DepSound–J.DepAcyclic derive J.CanonicalRank from the chosen RTL_B witness's concrete issue/decision/commit annotations and the local operational footprints. The induced observable-unit enumeration refines commit-cycle order and $\prec_J$. Its prefixes are ideals under $\prec_J$, but stable tie ordering between commuting actions has no semantic significance and must not be read as adding an observable order among independent events. There is no semantic cutpoint after only one endpoint of an explicitly atomic multi-endpoint unit has been taken; such units are split neither in Sys_ref nor in RTL_B.

Scope note (intentional). The finite-prefix Post $\rightarrow$ Ref clause of Theorem J.1 and the cutpoint correspondence of Corollary J.1 are stated over Reach_post / Reach_ref and do not require an infinite

fair extension. Their finite-ideal construction uses the process-local preparation records and the local channel/atomic certificates of Definition J.LocalGWR, combined by Lemma J.UnitExt and Theorem J.GWR-Comp with the Sys_ref operational transition semantics. On the normal compositional route, J.RegPhaseNF/J.UnitExt also require a finite source L2 settle for every constructed Horizon; in the standard scope that same-cycle existence obligation is supplied by QualifiedSynchronousModel_Sys_ref(I) and Theorem Core.QSync-Ref. A directly supplied J.GWR package may instead carry independently checked finite L2-settled segments for its Horizons and need not separately assert global source-side Core.QSync solely for Theorem J.1. Weak fairness B2, channel progress B3, and the time-divergence / idle-stutter convention B6 are not premises of either finite-prefix direction of Theorem J.1. They remain available only to later theorems that explicitly construct or validate complete/fair runs, such as Core.LAdm/K.2b.E and K.CompleteS as applicable. B4/B5 remain separate normalization premises for replay/quiescence uses not discharged by the same-cycle Core.QSync theorem.

Assume the Appendix I / Appendix L local replay machinery is available—namely I.A0, WC for ε -modeled choices, deterministic replay, and the finite process-local preparation theorem—with NB-CF as a sufficient WC route where applicable. Also assume the capacities/occupancies/enableness semantics of Sys_ref, finite capacities and B-faithful E4 behavior on every explicit abstract channel, the complete local certificate package of Definition J.LocalGWR, and the standing RTL_B determinism assumption B1 under the fixed stimulus. For the normal compositional Post $\rightarrow$ Ref route, additionally assume QualifiedSynchronousModel_Sys_ref(I); Theorem Core.QSync-Ref then supplies the finite L2 SettlePoint required by J.RegPhaseNF and J.UnitExt for every reachable constructed Horizon. The direct semantic route may instead supply a J.GWR package whose checked reachable phase-normal chain already contains the required finite L2-settled segment for each Horizon; such a package need not separately discharge global source-side Core.QSync merely to apply finite-prefix Theorem J.1. Core.QSync-Post is not a premise of plain finite-prefix J.1 unless a post-side KCut/NormPost interface is additionally invoked. No B2/B3 premise is used by the finite-prefix clause. Use B4/B5 additionally when replay ε -normalization or uniqueness up to finite ε -stutter outside the QSync same-cycle settle is invoked. When a later theorem explicitly asserts a complete/fair execution, separately require that theorem's applicable B2/B3 and continuation/closure premises. Additionally assume the shared-memory lowering condition J.Mem for every cross-process shared-memory dependency in the compared design and, when dynamic reset is used, the reset well-formedness conditions RW1–RW5 (both in Normative Formal Core). Then:

Compatibility-relation typing convention. The symbol $\approx$ is an ordered cross-model compatibility predicate, not a mathematical equivalence relation. It is intentionally directed from a Sys_ref execution/projection to an RTL_B execution/projection, and no symmetry or transitivity property is implied. Its carrier is not a bare Σ -label trace alone.

For each process P, the per-process carrier is an annotated observable projection record. Such a record contains the P-local bucketed Σ projection, the relevant observable-unit projection, the dynamic source/witness universe Ω_P and $Q_{exec,P}$ used for that comparison, the Issue/Commit annotation for dynamic message-call instances, the selected witness w, and the μ_Q matching supplied by WC / Appendix L when ε -modeled failed non-blocking calls or other witness-selected choices are relevant. Forgetting these annotations yields the ordinary P-local observable Σ projection, but E3, E5, BoundaryReq attribution, and the witness construction may depend on the annotations and are not reconstructed from the Σ projection alone.

Accordingly, write

$$\approx_P \subseteq \text{AnnTraces}_{\{\Sigma, P, w\}}(\text{Sys_ref}) \times \text{AnnTraces}_{\{\Sigma, P, w\}}(\text{RTL_B})$$

for the per-process annotated observable-compatibility predicate induced by the P-local R/E obligations: synchronization ordering, signal anchoring or R2C fixed-point agreement, message issue-order constraints, synchronization-boundary issue constraints, and the P-local witness/request-attribution facts needed for those obligations. Here $\text{AnnTraces}_{\{\Sigma, P, w\}}(\cdot)$ denotes the annotated projection records just described, not merely $\text{Traces}_{\{\Sigma, P\}}(\cdot)$.

Similarly, write

$$\approx_{\text{sys}} \subseteq \text{AnnTraces}_{\{\Sigma, w\}}(\text{Sys_ref}) \times \text{AnnTraces}_{\{\Sigma, w\}}(\text{RTL_B})$$

for system-level annotated observable compatibility. A system-level annotated record includes the per-process annotated projections plus the explicit abstract-channel histories and boundary-state facts needed to check E4 at the chosen Sys_ref / RTL_B boundaries.

For system executions $\tau_{\text{ref}, \text{system}}$ and $\tau_{\text{post}, \text{system}}$, the assertion $\tau_{\text{ref}, \text{system}} \approx_{\text{sys}} \tau_{\text{post}, \text{system}}$ is therefore an abbreviation: there exist compatible annotation/witness records for the two executions such that every process projection satisfies the corresponding $\tau_{\text{ref}, P} \approx_P \tau_{\text{post}, P}$ obligation and every explicit abstract channel in the chosen Sys_ref / RTL_B comparison satisfies the applicable E4 channel semantics at the agreed abstract boundary. When the annotations are clear from context, the same $\tau_{\text{ref}} \approx \tau_{\text{post}}$ notation may be used for readability; it should not be read as typing $\approx$ over bare Σ -traces alone.

Definition J.RTLConsistentRefPrefix (annotated RTL-consistent source prefix). A source-side prefix $\tau_{\text{ref}} \in \text{Reach_ref}$ is RTL-consistent for the $\text{Ref} \rightarrow \text{Post}$ clause only when it is equipped with a selected RTL_B prefix $\tau_{\text{post}}^* \in \text{Reach_post}$ and a witness/annotation package W such that: (1) τ_{post}^* is admitted by RTL_B under the same fixed stimulus; (2) the complete selected observable-unit projection of τ_{ref} matches the complete selected observable-unit projection of τ_{post}^* under the canonical occurrence matching of Definition Core.17; in particular, neither selected projection may contain an additional unmatched observable unit beyond the other; (3) W provides compatible annotated projection records for τ_{ref} and τ_{post}^* , including the dynamic operation universes Ω_P and $Q_{\text{exec}, P}$, μ_Q matching, the operational Issue/Commit and E3/E5 annotations needed by the execution-level construction, request-attribution facts (including failed non-blocking poll witnesses where relevant), and the applicable per-channel E4 histories and abstract boundary facts; and (4) whenever Corollary J.1, Appendix K, or any cutpoint transfer uses the selected pair, the same selected witness and normalized source prefix satisfy J.OfferReach, the selected RTL cutpoint satisfies J.OfferConf, and the composite J.KObsConf relation holds for the same H and O . Label-projection matching alone is therefore not sufficient to establish $\text{Ref} \rightarrow \text{Post}$; the compatible annotations and, when needed, the KObs cutpoint package are part of the RTLConsistentRefPrefix witness.

Then:

(Post $\rightarrow$ Ref existence, compositional route) For every $\tau_{\text{post}} \in \text{Reach_post}$ together with a local certificate package L satisfying $\text{LocalGWRPackage}(\tau_{\text{post}}, L)$ (Definition J.LocalGWR), there exist a package W and a prefix $\tau_{\text{ref}} \in \text{Reach_ref}$ such that $\text{GlobalWitnessRealizable}(\tau_{\text{post}}, W)$ holds and the annotated system projections satisfy $\tau_{\text{ref}, \text{system}} \approx_{\text{sys}} \tau_{\text{post}, \text{system}}$ under E1–E5.

(Post $\rightarrow$ Ref existence, direct semantic route) The same core conclusion holds when the application directly supplies a validated package W satisfying $\text{GlobalWitnessRealizable}(\tau_{\text{post}}, W)$. The compositional route is the normal theorem path; the direct route permits translation validation, symbolic replay, or another independently checked global certificate to target the stable J.GWR interface.

(Ref $\rightarrow$ Post existence, annotated RTL-consistent prefixes only) For every $\tau_{\text{ref}} \in \text{Reach_ref}$ for which there exist $\tau_{\text{post}}^* \in \text{Reach_post}$ and a witness/annotation package W satisfying

$\text{RTLConsistentRefPrefix}(\tau_{\text{ref}}, \tau_{\text{post}}^*, W)$ under the same fixed stimulus, Definition J. $\text{RTLConsistentRefPrefix}$ supplies the already-selected annotated source/RTL pair and its compatible per-process projection records. The theorem-wide finite-capacity and B-faithful abstract-boundary assumptions, together with the per-channel E4 histories and abstract-boundary facts carried by W , supply the corresponding per-channel well-formedness premises of Proposition J.0. Proposition J.0 therefore applies to this selected pair. Hence there exists $\tau_{\text{post}} \in \text{Reach_post}$ with the required compatibility, witnessed by $\tau_{\text{post}} := \tau_{\text{post}}^*$, and the annotated system projections satisfy $\tau_{\text{ref}, \text{system}} \approx_{\text{sys}} \tau_{\text{post}, \text{system}}$. In Appendix L snooping/witness-selection applications, the selected τ_{post}^* and W are the RTL_B reachable prefix and witness package selected by the snooping wrapper/witness.

Moreover, because B1 is a standing assumption in this theorem instance (deterministic Σ -projected RTL_B trace under fixed stimulus), the matching observable Σ projection of a selected RTL_B prefix is unique (up to insertion/removal of finite ϵ -steps permitted by B4/B5). This uniqueness is only a uniqueness statement about the observable projection; it does not by itself determine μ_Q , failed-poll instances in Q_{exec} , issue annotations, request-attribution facts, the selected package W , or the full stateful Sys_ref witness. The corresponding execution prefix witness τ_{ref} need not be unique as a stateful prefix, since Sys_ref may admit multiple ϵ -different realizations with the same Σ projection. Sys_ref may also admit executions whose Σ projection does not match RTL_B's Σ trace when latency-sensitive / non-blocking effects are Σ -visible; such executions are outside the scope of $(\text{Ref} \rightarrow \text{Post})$ unless constrained by $\text{RTLConsistentRefPrefix}$ / the snooping/witness-selection technique (Appendix L).

Definition J.RefProjCover (selected-reference-run observable coverage)

Fix a theorem instance I , a DUT-boundary observable projection Σ_{DUT} , and a selected complete source-reference execution ρ_{ref}^0 . Define $\text{RefProjCover}_I, \Sigma_{\text{DUT}}(\rho_{\text{ref}}^0)$ to hold iff, for every $\tau_{\text{ref}} \in \text{Reach_ref}$ of I , the finite canonical Σ_{DUT} history $\text{CanonHist}_{\Sigma_{\text{DUT}}}(\tau_{\text{ref}})$ is a finite ideal (a down-closed canonical prefix) of $\text{CanonHist}_{\Sigma_{\text{DUT}}}(\rho_{\text{ref}}^0)$ under the required observable-unit order, modulo J.HCanon and the permitted finite ϵ -stutter. The definition covers observable ideals/down-closed prefixes; it does not require identical cycle buckets or a fixed linear order among independent observable units.

Definition J.TraceCertCov ($\text{Post} \rightarrow \text{Ref}$ certificate coverage)

Fix a theorem instance I and a coverage domain $D \subseteq \text{Reach_post}$. $\text{TraceCertCov}_I(D)$ holds iff every $\tau_{\text{post}} \in D$ is equipped with one of the following route-complete $\text{Post} \rightarrow \text{Ref}$ packages: (a) a J.LocalGWR package accepted by J.GWR-Comp, together with $\text{QualifiedSynchronousModel}_{\text{Sys_ref}}(I)$ so that Core.QSync-Ref supplies the finite L2 settle required by J.RegPhaseNF/J.UnitExt, and the remaining applicable $\text{Post} \rightarrow \text{Ref}$ premises of Theorem J.1; or (b) a directly validated J.GWR package whose checked reachable phase-normal chain supplies the required finite L2-settled segment for every represented Horizon, together with the remaining applicable $\text{Post} \rightarrow \text{Ref}$ premises of Theorem J.1. All evidence shall be bound to the same theorem instance, stimulus, observation boundary, and selected RTL_B prefix. The definition names only safety-side certificate/package coverage; it does not strengthen Theorem J.1 and does not imply K.CertCov.

Corollary J.1-Safe (source-side safety transfer through $\text{Post} \rightarrow \text{Ref}$)

Assume $\text{J.TraceCertCov}_I(D)$ for a selected coverage domain $D \subseteq \text{Reach_post}$ containing every reachable RTL_B prefix relevant to the claimed property. J.TraceCertCov includes the source-settling evidence appropriate to the selected $\text{Post} \rightarrow \text{Ref}$ construction route: source-side Core.QSync for the normal

compositional route, or equivalent checked per-Horizon settling evidence for a direct J.GWR route. Let Φ be a prefix-closed predicate on finite canonical Σ_{DUT} histories and invariant under J.HCanon. Then:

- (1) If every $\tau_{\text{ref}} \in \text{Reach}_{\text{ref}}$ satisfies Φ , then every $\tau_{\text{post}} \in D$ satisfies Φ .
- (2) If one selected complete pre-HLS execution ρ_{ref}^0 satisfies Φ at every finite canonical ideal (down-closed prefix) and $\text{RefProjCover}_I, \Sigma_{\text{DUT}}(\rho_{\text{ref}}^0)$ holds, then every $\tau_{\text{post}} \in D$ satisfies Φ .

Proof. For any $\tau_{\text{post}} \in D$, J.TraceCertCov supplies the applicable $\text{Post} \rightarrow \text{Ref}$ package, and Theorem J.1 constructs a reachable matching Sys_{ref} prefix with the same canonical Σ_{DUT} observation. Clause (1) follows from the universal Sys_{ref} premise. For clause (2), J.RefProjCover places that matching observation among the finite canonical ideals/down-closed prefixes covered by ρ_{ref}^0 . Prefix closure and J.HCanon-invariance of Φ yield the RTL_B conclusion. $\square$

Common-case discharge note. For blocking deterministic dataflow, or for designs in which NB-CF makes failed non-blocking polls irrelevant to later Σ_{DUT} behavior, J.RefProjCover may be discharged from fixed stimulus, deterministic/value-consistent process behavior, deterministic or witness-fixed arbitration and environment choices, and completeness of the selected source execution. Lemma K.2b.P1 supplies the payload and dynamic-operation-identity determinacy ingredient in its blocking-only scope, but it is not by itself the full observable-prefix coverage theorem. Static extraction of B(c) and any elaboration package define Sys_{ref} ; B-faithfulness and J.TraceCertCov remain separate obligations.

Scope. Corollary J.1-Safe transfers safety predicates only. It does not transfer raw cycle latency or performance properties, prove eventual RTL realization of every source event, or establish unrestricted equality of the two behavior sets.

Proof strategy. Theorem J.1 is not obtained from pairwise composition alone. Appendix I proves the local replay/preparation scripts with carried deferred NB holes, packaged as J.PCert. Definition J.LocalGWR adds per-channel, atomic-unit, environment/reset, field-sensitive footprint, request-snapshot, generated-dependency, and interface-agreement certificates. Lemmas J.DepSound/J.HorizonOrderSafe/J.DepComplete/J.DepRank/J.DepAcyclic derive a canonical action order rather than accepting one as a premise. Lemma J.RegPhaseNF then normalizes each Horizon into the single Core.Phase L1/L2/L3 phase vocabulary, preserving the J.HCanon history quotient. Horizon-batched J.UnitExt and Theorem J.GWR-Comp construct one coherent source execution and the theorem-derived J.ValidatedConstructionOrder/J.GWR package. J.GWR-Direct instead supplies an equivalent directly validated order, phase-normal chain, and cutpoint invariants. Proposition J.0 remains only the conditional lifting step for an already selected compatible trace pair.

Definition J.ConstructionAction and operational footprints

Fix a finite RTL_B prefix τ_{post} and a candidate local package L. $\text{Act}_L(\tau_{\text{post}})$ is the finite set of primitive source-construction actions named by the process scripts, channel records, deferred-hole records, and complete observable units. An action is one of: (A1) a process-owned ϵ evaluation/control transition, including outcome-independent continuation past a carried NB hole; (A2) assertion or retention of an endpoint-owned blocking request, or assertion of a one-cycle attributed NB attempt; (A3) a shared-boundary NB decision/return that reads the settled ReqSnapshot, writes NBResult(q), retires NBPending(q)/the attempt, and on success is contracted with the corresponding transfer commit; (A4) any other complete explicit-channel or observable atomic-unit commit; or (A5) an environment, reset, or explicit $\text{Sys}_B^{\text{elab}}$ transition. A3 does not write the ordinary control/operand locations used by certified same-Horizon outcome-independent actions; dependent continuation is an A1 action at a strictly later Horizon. Every action has a typed identity, owner set, reset epoch, Horizon, associated unit, semantic stage, local ordinal, and finite ReadSet/WriteSet over a normative location vocabulary. That vocabulary distinguishes at least Ctrl(P), Local(P,x)/Operand(P,e), NBResult(q), NBPending(q), endpoint request and Push-payload locations, each explicit channel/elaboration location, environment locations,

and reset locations. Certificates may realize these locations as components, fields, or lenses, but shall map them to this common vocabulary and prove the corresponding disjointness/frame laws. The vocabulary is intentionally finer than a monolithic “process-local state” component. Custom $\text{Sys_B}^{\text{elab}}$ components require local footprint/commutation and well-founded ordinal certificates.

Definition J.ReqSnapshot and J.NBDecision

For every witness-significant NB dynamic call q at explicit boundary c , let $\kappa(q)$ be its unique snapshot key and let $\text{NBPending}(q)$ be the carried I.DeferredNBHole record. $\text{ReqSnapshot_c}(q)$ is the common source-boundary state at the L2 settled point of q ’s Horizon, after every L1 request/attempt write and every other Dep_J predecessor of the decision has executed, but before q ’s A3 success/failure return and before any channel-state update caused by that decision. The snapshot contains the reset epoch; E4 channel state; every attributed persistent request and one-cycle attempt present at c , including q ’s own attempt with stable Push payload; and every explicit gate/join/arbitration/environment field affecting c . $\text{J.NBDecision}(q, \text{ReqSnapshot_c}(q))$ is the unique result prescribed by the source call semantics and E4. For $B(c) > 0$, PopNB succeeds exactly when nonempty and PushNB exactly when nonfull; for $B(c) = 0$, success requires the matching complementary attributed request/attempt with value agreement. Failure is an ϵ A3 return with $\text{ReturnedAtCycle}(q, t)$ and no channel commit; success is the contracted A3/committed-transfer unit. WC selects the implementation outcome and J.CC^{ert} proves equality. Process-local preparation may create $\text{NBPending}(q)$ and may continue through certified outcome-independent actions, but it may not execute A3 or read the settled snapshot locally.

Lemma J.BufSameCycle (same-cycle buffered Push/Pop sequentialization)

Let c have $B(c) > 0$, let the pre-cycle logical FIFO contents be Q with $n = |Q|$, and suppose one $\text{Push_c}(v)$ and one $\text{Pop_c}(w)$ both commit in the same RTL cycle under the non-fall-through E4 semantics. Then $0 < n < B(c)$, $w = \text{head}(Q)$, and both source serializations $\text{Push_c}(v); \text{Pop_c}(w)$ and $\text{Pop_c}(w); \text{Push_c}(v)$ are enabled from Q . Both terminate with logical contents $\text{tail}(Q)$ followed by v , occupancy n , the same producer and consumer ordinals, and the same unordered same-cycle observable bucket. The proof may therefore execute the two source transitions in either order while retaining their common RTL-cycle annotation. If $n = 0$, the Pop cannot be enabled by the same-cycle Push; if $n = B(c)$, the Push cannot be enabled by the same-cycle Pop. For $B(c) = 0$ the endpoints remain one indivisible rendezvous action and this lemma does not apply. Proof: direct case analysis on the E4 enablement predicates and FIFO update equations. ■

Definition J.DepJ (generated construction-dependency relation)

After contracting every explicitly atomic action, let $a \prec_{\text{D}^{\text{sem}}} b$ denote a direct semantic-prerequisite edge. The semantic relation is generated only by (D1) same-process dynamic control, data, source-issue, synchronization-boundary, and ReturnedAtCycle dependencies; (D2) request/attempt-before-settle/snapshot-before-A3-decision edges for every location read by $\text{ReqSnapshot_c}(q)$, plus A3-decision-before-continuation edges only for actions whose reachability or operands depend on $\text{NBResult}(q)$; (D3) per-channel FIFO/history, occupancy/head, and non-commuting same-boundary update predecessors, with J.BufSameCycle as the named buffered exception; and (D4) reset, environment, and explicit elaboration-component predecessors. Let $a \prec_{\text{D}} b$ denote the full construction-dependency relation obtained by adding (D5) concrete earlier-Horizon proof-stratification edges from the selected RTL annotation. A D5 edge that is not already implied by $(\prec_{\text{D}^{\text{sem}}})^+$ is admissible only when the local footprint/frame certificate proves that the two actions commute modulo J.HCanon , so choosing the earlier-Horizon action first preserves transition legality, complete units, and the post-settle semantic state. Thus D5 selects a canonical construction representative; it is not by itself a universal Sys_ref happens-before fact. An outcome-independent same-Horizon action reached past an

unresolved q acquires no A3-to-action edge: its I .DeferredNBHole NoDependentUse fact and field-sensitive footprints must instead show that it is independent of $NBResult(q)$ and of q 's settled snapshot, so ordinary framing/commutation applies. An edge may not be inserted merely for replay convenience. Mere source order between distinct-interface operations creates the applicable issue-order obligations, but not a decision/commit-to-later-request edge when $Core.RegSeq$ and the local certificate establish independent co-issue.

Lemma J.DepSound (semantic generated edges are operational prerequisites)

If $a <_D^{sem} b$, then no legal Sys_ref construction segment realizing both actions can execute b before a unless the actions are contracted into one explicit atomic unit. Proof: D1 follows from source control/data/issue rules and ReturnedAtCycle/Core.RegSeq; D2 from the settled-snapshot read and true NB-result dependence; D3 from E4/FIFO state dependence except J.BufSameCycle; and D4 from the named reset/environment/elaboration rule. Soundness of every declared footprint against the normative location semantics, including frame and disjointness laws, is part of the local-certificate obligation. $\square$

Lemma J.HorizonOrderSafe (D5 proof stratification is construction-safe).

Let $a <_D b$ be a D5 edge with no path $a <_D^{sem} b$. The LocalGWR package must then supply the D5 commutation certificate required by Definition J.DepJ: a and b may be exchanged without changing transition legality, the complete observable units, the terminal post-settle semantic state, or the J.HCanon quotient. Hence the canonical construction may choose a before b even though a is not an operational prerequisite of b in every legal Sys_ref execution. If the pair does not commute, the actual overlap must instead be represented by D1–D4 and is covered by J.DepSound. $\square$

Lemma J.DepComplete (all non-commuting construction actions are ordered)

Let a and b be distinct construction actions that are not endpoints of one contracted atomic action. If neither $a <_D^{sem} b$ nor $b <_D^{sem} a$, then either (i) their field-sensitive ReadSet/WriteSet interference tests are disjoint and the transitions commute by J.Frame, or (ii) they are the legal same-cycle buffered Push/Pop dual update of J.BufSameCycle. Every pair that cannot be swapped while preserving transition legality, the same complete observable units, the same post-settle semantic state, and the same J.HCanon history quotient is therefore ordered in at least one direction. In particular, narrowed A3(q) commutes with a same-Horizon owner action carried past q only when NoDependentUse(q) and the location footprints show disjointness from $NBResult(q)$, $NBPending(q)$, q 's attempt, and every location in $ReqSnapshot_c(q)$. The finite A1–A5 case analysis invokes D1–D4 for every actual overlap; D5 may additionally order already-commuting cross-Horizon pairs under Lemma J.HorizonOrderSafe, and custom components use their local certificate. $\blacksquare$

Definition J.FoundationRank (rank independent of graph acyclicity)

For each construction action a , define $FR(a)$ lexicographically as $\langle EpochIndex(a), Horizon(a), SemanticStage(a), LocalOrdinal(a), ComponentTie(a) \rangle$. Horizon is the selected RTL cycle/decision horizon or enclosing unit cycle. If an action consumes a value or status with ReturnedAtCycle(q, t), $Core.RegSeq$ requires $Horizon(a) > t$. Same-Horizon actions carried past q must instead satisfy NoDependentUse(q). SemanticStage is the fixed order: reset/environment prerequisites; local evaluation plus request/attempt writes ($Core.Phase$ L1); completed settled-snapshot formation (L2); shared A3 decision or contracted/other boundary commit (L3); and a reserved continuation-bookkeeping stage usable only for within-Horizon sinks whose outgoing same-Horizon edges never return to an earlier stage. Outcome-dependent process continuation after an NB result is not assigned to that reserved stage; it is an ordinary evaluation/request action at a later Horizon. LocalOrdinal is the action-kind-specific script, request/snapshot, transfer, environment, reset, or component ordinal. ComponentTie is used only when

all prior coordinates agree and a local certificate proves commutation or a component-local order. FR is computed before Dep_J and does not use Height or J.CanonicalRank .

Lemma J.DepRank (every generated dependency advances the foundation rank)

For every well-formed LocalGWRPackage and generated direct edge $a \prec_D b$, $\text{FR}(a) < \text{FR}(b)$. D5 strictly increases Horizon; reset edges increase EpochIndex. D1 either advances the within-L1 local ordinal/stage, or—when b depends on a boundary result $\text{ReturnedAtCycle}(q, t)$ —strictly increases Horizon under Core.RegSeq . Independently reached same-Horizon actions carried past an unresolved q have no A3-to-action edge; their true control/data/issue predecessors advance the L1 ordinal or stage. D2 advances request/attempt $\rightarrow$ settled snapshot $\rightarrow$ A3 decision/contracted commit. A decision-to-dependent-continuation edge strictly increases Horizon; any retained within-Horizon continuation-bookkeeping target is a certified sink and therefore cannot generate a later stage regression. D3 advances transfer/history ordinal or a non-commuting substage, with J.BufSameCycle removing the legal dual-update ordering; D4 advances the certified component ordinal/substage. Thus the prior failed-NB decision(t) $\rightarrow$ dependent request(t) case is impossible by construction, and the decision(t) / independent request(t) pair is unordered and commuting. ■

Lemma J.DepAcyclic (well-foundedness of the generated dependency graph)

For every well-formed LocalGWRPackage, the graph Dep_J is acyclic after atomic contraction. Proof. Suppose a directed cycle $a_0 \prec_D a_1 \prec_D \dots \prec_D a_0$ existed. Repeated application of Lemma J.DepRank would give $\text{FR}(a_0) < \text{FR}(a_1) < \dots < \text{FR}(a_0)$, contradicting irreflexivity of the well-founded lexicographic order on the finite rank components. This argument uses one global semantic rank that every D1–D5 edge advances; it does not infer acyclicity from a union of separately acyclic process, channel, request, environment, reset, or component orders. Height and J.CanonicalRank are defined only after this lemma and play no role in proving it. ■

Definition J.CanonicalRank (derived construction order)

Because $\text{Act_L}(\tau_{\text{post}})$ is finite and Dep_J is acyclic by the independent FR argument, define $\text{Height}(a)=0$ for actions with no incoming $\prec_D$ edge and $\text{Height}(a)=1+\max\{\text{Height}(b) \mid b \prec_D a\}$ otherwise. Define $\text{CanonicalRank}(a):=\langle \text{FR}(a), \text{Height}(a), \text{KindKey}(a), \text{StableId}(a) \rangle$ lexicographically. FR preserves the concrete epoch/horizon and semantic prerequisite structure; Height is a theorem-derived topological depth used only after acyclicity has been established; and KindKey/StableId only totalize otherwise commuting actions and create no observable $\prec_J$ edge. The rank of an observable unit is the rank of its contracted commit action. J.CanonicalRank is reconstructed from checked action identities, local operational coordinates, and Dep_J ; no phase or replay rank is supplied by the implementation flow.

Definition J.RequiredPrefixCandidate (rank-independent local-certificate index)

For a finite RTL_B prefix τ_{post} , $\text{RequiredPrefixCandidate}(\tau_{\text{post}}, H)$ means that H is a finite set of complete observable units of τ_{post} satisfying only order-neutral prefix conditions: (R1) H is down-closed under the execution-specific required-order relation $\prec_J$; (R2) for every process P , the P -local projection of H is a prefix of the selected P -local observable-unit projection, with no dynamic operation identity used twice and with reset epochs respected; (R3) every explicitly atomic unit is either wholly present or wholly absent; and (R4) each explicit-channel projection is a reset-epoch-consistent prefix of the selected rendezvous or FIFO history. The definition does not mention Dep_J , J.DepAcyclic , J.CanonicalRank , a proposed global replay order, or reachability of a common Sys_ref state. It is only the finite index domain on which Appendix I can compute process-local replay states and preparation scripts. Theorem J.GWR-Comp later selects from this domain the candidates that arise along its derived construction chain.

Definition J.PCert (process-local replay and preparation certificate)

For a process P and finite RTL_B prefix τ_{post} , PCert_ $P(\tau_{\text{post}}, L_P)$ holds when L_P supplies, for every relevant RequiredPrefixCandidate and process participation: (P1) WC/ μ_Q identity, reset epoch, and matched dynamic operation; (P2) Replay_ $P(H, w)$ with future- Σ -relevant values, control, completion, and operation ordinals; (P3) the finite per-Horizon Lemma I.3 script, every enriched I.DeferredNBHole, its NBResult/NBPending/result-destination locations, and NoDependentUse evidence for all same-Horizon actions carried past it; (P4) Issue/E3/E5, prefix closure, persistence, stable payload, Core.RegSeq, direct-input stability, and synchronization facts; (P5) exact dynamic NB attempt identity, desired WC outcome, and snapshot key, with no local A3 decision; (P6) complete A1/A2/A3 inventory, normative field-sensitive footprints, Horizon/stage/local ordinal, and every D1/D2 generator; and (P7) proof that local actions modify only their declared locations and export no unmatched commit or unresolved same-Horizon hole at a cutpoint. PCert contains no J.CanonicalRank and does not assume the global order or phase-normal chain.

Definition J.CCert (explicit-channel history, request, and enablement certificate)

For every explicit abstract channel c of E , CCert_ $c(\tau_{\text{post}}, L_c)$ supplies: (C1) boundary, $B(c)$, ownership, epochs, and B -faithfulness/ ϵ -opacity; (C2) complete prefix-indexed E4 history, ordinals, payloads, occupancy/head, and reset-empty restarts; (C3) producer/consumer agreement on transfer identity, value, ordinal, direction, and epoch; (C4) for every Horizon and witness-significant q , the complete L2 settled ReqSnapshot_ $c(q)$, including all absent/present requests/attempts, stable payloads, and explicit gate/join/environment fields; (C5) proof that $WC(q) = J.NBDecision(q, \text{ReqSnapshot}_c(q))$; (C6) exact local channel/A3 state updates with no hidden availability; (C7) J.BufSameCycle evidence for every legal same-cycle buffered pair; (C8) every D2/D3 edge and field-sensitive channel/snapshot/result footprint; and (C9) proof that every one-cycle NB attempt is retired and every deferred-hole record for the Horizon is resolved before the exported post-boundary cutpoint.

Definition J.ACert, J.EnvCert, and J.ResetCert

ACert_ y identifies the complete participant set, dynamic identities, values, reset epoch, and atomic transition rule for every rendezvous transfer, paired SyncChannel/ac_sync transaction, E2 anchor/signal group, R2C fixed-point unit, RESET unit, and other explicitly atomic unit. It proves that every participant record names the same contracted action and supplies the endpoint requests, values, guards, and component state from which the generic Sys_ref rule can derive enablement; it does not take complete-unit enablement itself as an unexplained certificate field. An endpoint cannot be certified independently of its counterpart. EnvCert proves, for every selected external interaction, that the source and RTL records use the same response of the fixed reactive environment to the same preceding Σ -causal history and supplies the corresponding footprints, horizon/stage/ordinal fields, and dependencies. ResetCert supplies RW1–RW5 for each reset occurrence and scope, including request clearing, reset-empty channel state, source/RTL reset-state correspondence, atomic reset splicing, framing outside the reset scope, and the D4 dependency and J.DepRank coordinate facts.

Definition J.LocalAgree and J.LocalGWR

LocalAgree(L) is the finite conjunction of equality, uniqueness, ownership, and coverage checks on duplicated local-record fields: shared action/unit identity; endpoint pairing; boundary identity; payload/value; transfer ordinal; request/attempt attribution; deferred-hole identity and result destination; snapshot key/contents; NoDependentUse facts; reset epoch; witness/elaboration identity; Horizon and Core.LAdm phase; semantic stage/local ordinal; the normative field-sensitive ReadSet/WriteSet locations; and every generated dependency edge. Every process, explicit boundary, construction action, atomic unit, deferred hole, and witness-significant ϵ choice in τ_{post} must be

covered, and each endpoint request/attempt has at most one dynamic owner. LocalAgree does not assert a global execution, acyclicity, phase-normality, or extension.

Definition J.LocalGWR. LocalGWRPackage(τ_{post}, L) consists of PCert_P for every process, CCert_c for every explicit boundary, ACert for each atomic unit, applicable EnvCert/ResetCert, the complete A1–A5 action/deferred-hole inventory, the normative field-sensitive footprints and footprint-soundness proofs, Core.RegSeq/direct-input conformance facts, locally checkable epoch/Horizon/stage/ordinal metadata, every local fact from which Dep_J is generated, custom-component certificates, and LocalAgree, all bound to one instance/stimulus/E/w/observation boundary/RTL prefix. J.DepRank, J.DepAcyclic, J.CanonicalRank, J.RegPhaseNF, J.RegPhaseReach, and the global source chain are theorem-derived, not supplied as unexplained local premises.

Definition J.ConstructionOrderProvenance and J.ValidatedConstructionOrder

For fixed τ_{post} and W , a ValidatedConstructionOrder contains a complete unit enumeration $\gamma_1 \dots \gamma_m$, the finite predecessor relation $<_W$, and the partition of actions/units into nondecreasing Horizon buckets. It proves: (V1) well-foundedness; (V2) refinement of $<_J$, reset epochs, atomic grouping, and every strict Horizon constraint; (V3) exact action/unit/deferred-hole coverage; (V4) coverage of every preparation, request, settled snapshot, A3 decision, channel, environment, reset, and atomic prerequisite; (V5) every flattened enumeration prefix is a finite required-prefix ideal; and (V6) within each Horizon, the order admits the Core.LAdm phase normal form of J.RegPhaseNF, with all L1 requests fixed before L2 settle, all boundary decisions/commits in L3, and no deferred hole surviving the bucket. The theorem-derived form uses LocalGWR, Dep_J, J.CanonicalRank, and J.RegPhaseNF. The direct form independently validates V1–V6. Stable keys order only actions/units that commute modulo J.HCanon.

Definition J.GWR (GlobalWitnessRealizable(τ_{post}, W))

A selected package W is globally witness-realizable when it contains one coherent annotation package—WC/ μ_Q , Issue/ReturnedAtCycle/Commit attribution, E4 histories, atomic groups, environment/reset identities, dynamic operations, deferred-hole/snapshot identities, and construction actions—and there exist: (G1) one J.ValidatedConstructionOrder with finite Horizon buckets and a flattened complete-unit enumeration; (G2) required-prefix ideals for the flattened prefixes and matched cycle-entry/post-boundary bucket prefixes; and (G3) reachable Sys_ref states such that each Horizon bucket extends by one finite legal phase-normal segment: L1 process-owned A1/A2 evaluation and request/attempt assertion, then L2 freeze/ ϵ -settle to the common snapshots, then L3 A3/A4 boundary decisions and complete units in commit-action rank order, with J.BufSameCycle used for legal buffered dual updates. No new L1 request occurs after the bucket's L2 settle; every action depending on a ReturnedAtCycle result is placed in a later Horizon; and every deferred hole is resolved or retired before the exported post-boundary state. The segment has the same complete units and terminal post-settle state and the same CanonHist/J.HCanon quotient as the selected construction order, even when same-Horizon independent actions are permuted. The chain preserves replay, channel, witness, attribution, reset, phase, and snapshot invariants and is compatible with τ_{post} under E1–E5. The direct route supplies the same bucketed interface. Historical operational data remains evidence rather than KObs fields.

J.GWR is a stable semantic interface. The normal route derives its ValidatedConstructionOrder and the complete witness chain by Theorem J.GWR-Comp from Definition J.LocalGWR. A flow may instead establish the same interface directly through the direct-validated construction-order provenance and a checked reachable chain. No theorem below infers J.GWR from bare Σ -label equality, and the direct route may not refer to J.CanonicalRank unless it also supplies the LocalGWR package and theorem-derived provenance that define that rank.

Lemma J.Frame (local transition framing)

Let Sys_ref state be viewed through the normative typed location vocabulary of $\text{J.ConstructionAction}$. A transition segment whose certified WriteSet is F changes only locations in F . Every location disjoint from F is equal before and after, and every guard/expression whose ReadSet is disjoint from F has the same value. This includes separate framing of $\text{Ctrl/Local/Operand}$, NBResult , NBPending , endpoint request/payload, channel/elaboration, environment, and reset locations. Each certificate must prove soundness of its declared locations against the actual Sys_ref transition semantics; the representation may use components, fields, or lenses. ■

Lemma J.LocalLift (lifting a certified process-local preparation path)

If the P -owned locations of a reachable global state equal the start state of a PCert_P segment, every shared location read by that segment equals its certified summary, and the field-sensitive footprint conditions hold, then the segment lifts to a finite global $\varepsilon/\text{request}$ path. The lifted path has the same P -local result and endpoint writes, commits no unmatched Σ unit, and frames every unrelated location. At an NB call it creates/carries I.DeferredNBHole and performs no A3 decision; unlike the previous stop-at-hole formulation, it may continue through certified NoDependentUse same-Horizon actions. Proof: induction over the segment using J.Frame . ■

Lemma J.PrepareCommute (commutation of local preparation paths)

Consider the executable L1 preparation actions and other construction actions in one Horizon. Field-sensitive disjoint ReadSet/WriteSet actions commute by J.Frame . Separately owned request writes may accumulate before the common settle when neither reads a snapshot whose fields remain mutable. A narrowed $\text{A3}(q)$ never moves before an action whose write appears in $\text{ReqSnapshot_c}(q)$, but it commutes with a carried same-Horizon owner action when $\text{NoDependentUse}(q)$ and the footprints establish disjointness from $\text{NBResult}(q)$, $\text{NBPending}(q)$, q 's attempt, and all snapshot locations. J.DepComplete orders every other non-commuting pair; J.BufSameCycle handles the legal buffered L3 dual update. Thus topological executions can be permuted into the normal form stated next, preserving terminal state and the J.HCanon quotient. No fairness premise is used. ■

Lemma J.RegPhaseNF (registered sequential-interface phase normal form)

Assume Core.RegSeq , a well-formed LocalGWRPackage , and that the frozen L2 state of the selected Horizon admits a finite $\rightarrow \varepsilon, \text{L2}$ path to a SettlePoint . In the standard compositional scope, $\text{QualifiedSynchronousModel_Sys_ref}(I)$ and $\text{Theorem Core.QSync-Ref}$ discharge this existence premise uniformly for every reachable Horizon; a directly validated J.GWR route may instead carry the equivalent checked per-Horizon settling evidence. For every finite construction fragment whose actions have one selected Horizon t , there is a permutation respecting Dep_J and atomic contraction with the following order: (1) all required A1 local evaluation/control and A2 request/attempt assertions permitted by Core.Phase L1 , including outcome-independent actions carried past deferred NB holes; (2) one L2 request freeze and ε -settle producing every ReqSnapshot for t ; and (3) the A3/A4 boundary decisions and complete units of L3 , ordered by commit-action rank except for explicitly atomic units and J.BufSameCycle sequentializations. Actions depending on a value/status ReturnedAtCycle in t occur only at a later Horizon. The normalized fragment contains the same complete observable units, reaches the same post-boundary semantic state, and yields the same canonical enriched history under J.HCanon ; literal ordering of independent same-Horizon labels need not be identical. Every deferred hole for t is resolved or retired at the end. Proof: repeatedly commute adjacent unordered actions using $\text{J.Frame/J.PrepareCommute}$, use NoDependentUse and field-sensitive footprints for decision/independent-continuation swaps, use J.BufSameCycle within L3 , and use J.DepRank to exclude any required edge from a later phase back to an earlier phase. Termination follows from finiteness. ■

Lemma J.NBDecision (globally valid non-blocking decision step)

Suppose the matched Horizon has completed its J.RegPhaseNF L1 actions and L2 settle, every Dep_J predecessor of NBPending(q) has executed, and J.CCert reconstructs ReqSnapshot_c(q)=S. Then q has exactly one legal A3 result J.NBDecision(q,S). On failure, A3 writes the status to NBResult(q), retires NBPending(q)/the one-cycle attempt, produces ReturnedAtCycle(q,t), and changes no channel history. On success, A3 is contracted with the corresponding committed transfer and writes the returned result. A3 does not perform ordinary outcome-independent control advance: such same-Horizon actions have already occurred in L1 under NoDependentUse, while every outcome-dependent continuation is an A1 action at a later Horizon. The result agrees with WC by C5 and no process-local script may choose or overwrite it. ■

Lemma J.UnitEnable (local certificates imply complete-unit enablement)

After J.RegPhaseNF has executed all required L1 actions and formed the common L2 snapshots for a Horizon, the endpoint requests/attempts, stable payloads, channel state, participant state, and deferred-hole records satisfy the premises of each selected L3 complete unit. J.NBDecision supplies failed or successful A3 results from those snapshots; failure is an ϵ result and success is the contracted unit. Buffered enablement follows from the common pre-update occupancy/head and non-fall-through E4, with J.BufSameCycle for legal dual updates; rendezvous requires simultaneous attributed endpoint requests/attempts and value agreement. Synchronization, E2 anchor, R2C, environment, and RESET units use their named certificates. ACert/CCert supply constituent facts while this lemma derives complete-unit enablement. ■

Lemma J.UnitExt (one Horizon-batched locally certified extension)

Assume the matched cycle-entry invariant for the next nonempty Horizon bucket B_t of the J.ValidatedConstructionOrder and LocalGWRPackage. Execute the finite Dep_J-minimal L1 A1/A2 predecessor set by J.LocalLift/J.PrepareCommute, carrying deferred holes through NoDependentUse work. Apply J.RegPhaseNF to obtain the canonical L1/L2/L3 arrangement, freeze requests, settle the explicit network, reconstruct all snapshots, and execute the bucket's failed A3 ϵ returns and complete A3/A4 units in commit-action rank order, using J.BufSameCycle where applicable. J.UnitEnable supplies each complete transition. No L1 action is executed after the L2 settle; dependent continuation is deferred to a later Horizon; all t-holes are empty at the exported post-boundary state. The resulting finite extension preserves replay, channel/atomic, attribution, reset, and canonical-history invariants and advances from the prior bucket ideal to the ideal containing all units in B_t. It uses no B2/B3 fairness. ■

Theorem J.GWR-Comp (local certificates construct global witness realizability)

Fix a finite reachable RTL_B prefix τ_{post} , the theorem instance, stimulus, elaboration E, observation boundaries, witness w, and reset interpretation. Assume Core.RegSeq, LocalGWRPackage(τ_{post} ,L), the Sys_ref decomposition used by J.Frame-J.UnitExt, I.A0/WC, the source/RW premises, B-faithful E4 boundaries, and QualifiedSynchronousModel_Sys_ref(I). By Theorem Core.QSync-Ref, every reachable constructed Horizon therefore has the finite L2 SettlePoint required by J.RegPhaseNF/J.UnitExt. Then Dep_J satisfies J.DepSound/J.HorizonOrderSafe/J.DepComplete; FR/J.DepRank prove J.DepAcyclic; J.CanonicalRank and the Horizon partition define the theorem-derived J.ValidatedConstructionOrder; J.RegPhaseNF proves its V6 phase property; and Horizon-batched J.UnitExt constructs W and a reachable τ_{ref} satisfying GlobalWitnessRealizable(τ_{post} ,W).

Proof. Generate the field-sensitive actions, deferred-hole records, Dep_J, and FR metadata.

J.DepSound/J.HorizonOrderSafe/J.DepComplete establish prerequisite coverage and commutation; J.DepRank/J.DepAcyclic derive the order. Partition the canonical order by Horizon. For each bucket,

J.RegPhaseNF permutes unordered actions into the Core.Phase L1/L2/L3 order modulo J.HCanon, and J.UnitExt performs the finite phase-normal extension. J.CCert/ACert reconstruct common snapshots and updates; J.NBDecision derives shared outcomes; J.BufSameCycle handles legal buffered dual updates; J.Frame and LocalAgree preserve unrelated locations and coherence; ResetCert/J.RS handle resets. Outer induction over Horizons, with inner commit-action order within L3, yields a reachable chain covering every unit and no exported deferred holes. Assemble W from the records, order, phase-normal chain, snapshots, and invariants; G1–G3 and E1–E5 follow. ■

Non-circularity. LocalGWR supplies field-sensitive actions, deferred-hole/NoDependentUse records, snapshots, local dependencies, and Core.RegSeq evidence, but no global order, phase-normal execution, or extension. J.DepComplete/J.DepRank/J.DepAcyclic derive the order; J.RegPhaseNF derives the per-Horizon phase arrangement; J.NBDecision derives shared outcomes; and J.UnitExt/J.GWR-Comp prove the reachable bucketed chain. The normal form is computed from graph-independent Horizon/stage/footprint facts and therefore does not assume the order it proves replayable.

Proposition J.0 (Pairwise Composition Lemma)

For each process P, let $A_{\text{ref},P}$ and $A_{\text{post},P}$ be the annotated P-local projection records induced by $\tau_{\text{ref},\text{system}}$ and $\tau_{\text{post},\text{system}}$ under the chosen witness/annotation package. Their forgotten observable components are the ordinary projections $\tau_{\text{ref},P}$ and $\tau_{\text{post},P}$, but the records also carry the issue annotations, dynamic message-call universe, μ_Q matching, and request-attribution facts needed by E3/E5 and WC. Assume that for every process P, these annotated projections satisfy the per-process compatibility predicate $A_{\text{ref},P} \approx_P A_{\text{post},P}$. In readable prose below, this may be abbreviated as $\tau_{\text{ref},P} \approx_P \tau_{\text{post},P}$ when the annotation package is fixed by context.

Thus Proposition J.0 is only a conditional composition result for a chosen compatible trace pair. It neither constructs a source execution nor proves J.GWR. Theorem J.GWR-Comp performs the local-to-global execution construction; Theorem J.1 then instantiates Proposition J.0 with that constructed pair. The restricted reverse clause continues to use the selected pair carried by RTLConsistentRefPrefix.

Assume every explicit abstract channel c in the chosen $\text{Sys}_{\text{ref}} / \text{RTL}_B$ comparison has finite capacity $B(c) \geq 0$. Also assume, as a per-channel $\text{Sys}_{\text{ref}} / \text{RTL}_B$ well-formedness condition, that both chosen traces satisfy the applicable E4 channel semantics at the chosen abstract boundaries. Thus, for $B(c)=0$ channels, committed Push_c / Pop_c events obey the same-cycle rendezvous semantics; for $B(c)>0$ channels, the committed Push_c / Pop_c history obeys the reset-empty, cycle-boundary, non-fall-through bounded-FIFO semantics with capacity B(c), including FIFO order, no underflow, no overflow, no duplication, and no loss. Proposition J.0 does not derive these per-channel E4 facts; it assumes them as part of the definition of the compared well-formed $\text{Sys}_{\text{ref}} / \text{RTL}_B$ traces.

No weak-fairness or channel-progress premise is needed for this conditional pairwise lifting lemma: Proposition J.0 does not construct a matching trace, but only lifts already-compatible per-process trace projections and already well-formed per-channel communication histories to the aggregate system trace. The finite selected-unit witness-construction premises are used later in Theorem J.1 when proving the finite execution-prefix existence result; weak fairness and channel progress are used only when a corresponding infinite fair execution is asserted. Then the aggregate traces are compatible: $\tau_{\text{ref},\text{system}} \approx_{\text{sys}} \tau_{\text{post},\text{system}}$ under rules E1–E5 (given the R rules and B rules).

Notation: The order relation $<_{\text{src}}$ used below is the dynamic source-induced order from Appendix G / R1–R4: it ranges over dynamic operation instances in the compared execution / witness, not over all syntactic call sites in the source text. Equivalently, when the process must be explicit, write $<_P$. Untaken branches and unexecuted loop iterations are not members of the witness-specific dynamic universe.

Proof

We prove each compatibility property. The per-channel E4 facts are not derived inside this proposition; they are part of the per-channel Sys_ref / RTL_B well-formedness hypothesis for the chosen traces.

E1 (Synchronization-order preservation):

Fix any process P. By the proposition hypothesis, $\tau_{\text{ref},P} \approx_P \tau_{\text{post},P}$, so the synchronization events of P occur in the same source order in both traces. Since this holds for every P, E1 holds for the system traces $\tau_{\text{ref},\text{system}}$ and $\tau_{\text{post},\text{system}}$.

E2 (Signal-visibility preservation):

Fix any process P. If P is a sequential process, then by the proposition hypothesis (using the E2 anchoring relation embodied in $\tau_{\text{ref},P} \approx_P \tau_{\text{post},P}$), every signal Read in P is anchored to P's closest preceding synchronization event, and every signal Write in P is anchored to P's closest succeeding synchronization event, in both $\tau_{\text{ref},P}$ and $\tau_{\text{post},P}$.

If P is a combinational process, then R2C rather than pred_S/succ_S anchoring applies: P's observable signal Read/Write events are represented by the R2C fixed-point signal-observation unit for the relevant combinational process/network component, matched by that component's local R2C occurrence ordinal as specified in the R2C unit matching/index convention. By the combinational well-formedness condition and the compatibility hypothesis for the fixed stimulus / selected witness, the pre-HLS and post-HLS combinational networks reach corresponding unique ϵ -fixed-point valuations for the matched R2C occurrence unit, so the matched Read/Write labels agree within that fixed-point unit. This local R2C-unit agreement is an explicit atomicity constraint; it does not require Sys_ref and RTL_B to use the same global observable cycle-bucket decomposition for unrelated independent events.

Since the appropriate sequential or combinational signal-visibility condition holds for every process P, E2 holds for $\tau_{\text{ref},\text{system}}$ and $\tau_{\text{post},\text{system}}$.

E3 (Safe message issue ordering):

Fix any process P. E3 is the post-side issue-order conjunct of the per-process compatibility relation $\approx_P$, not an additional ordering property derived from the Sys_ref trace. Thus what must be checked here is that P's post-HLS execution satisfies E3 as stated in Appendix G: for any $q_1, q_2 \in Q_{\text{exec},P}$ with $q_1 <_{\text{src}} q_2$, including ordinary distinct-interface message calls that appear in dynamic source sequence, $\text{issue_post}(q_1) \leq \text{issue_post}(q_2)$, and the issued set is prefix-closed under that source order. Same-cycle issue is permitted where allowed, and a later operation may be issued before an earlier operation commits under Policy L, but a source-later operation may not be issued before the source-earlier operation has issued. This post-side property is part of the local per-process hypothesis $\tau_{\text{ref},P} \approx_P \tau_{\text{post},P}$; in the execution-level use of Proposition J.0 / Theorem J.1, the needed post-side witness/order fact is supplied by Appendix I / Implementation assumption I.A0. Since the E3 post-side conjunct holds for every P, the aggregate system pair satisfies E3.

E4 (Per-channel channel semantics):

E4 holds by the per-channel Sys_ref / RTL_B well-formedness hypothesis of Proposition J.0. That hypothesis states that, for every explicit abstract channel c in the chosen comparison, the Sys_ref and RTL_B traces already satisfy the applicable E4 channel semantics at the chosen abstract boundary: same-cycle rendezvous semantics when $B(c)=0$, and reset-empty, cycle-boundary, non-fall-through bounded-FIFO semantics when $B(c)>0$. Thus Proposition J.0 uses E4 as an assumed channel-realization condition for the chosen traces; it does not attempt to derive E4 from the per-process compatibility relation. Accordingly, Proposition J.0 should be read as a composition theorem under already-established process and channel conformance hypotheses. It does not prove that the implementation realizes the chosen abstract channel boundaries, nor that the back-annotated capacities $B(c)$ satisfy B-faithfulness as defined in the Normative Formal Core. Those facts are separate RTL/tool-flow compliance obligations required before the theorem is applied to a concrete generated design.

E5 (Messages cannot cross syncs):

For every synchronization call s (in the source order used by $R3$ / pref_S / suff_S over Q_{exec}, P):

$\forall q \in \text{pref}_S(s) : \text{issue_post}(q) \leq \text{clk_post}(s)$

$\forall q \in \text{suff}_S(s) : \text{issue_post}(q) > \text{clk_post}(s)$

$\forall q \in \text{pref}_S(s)$ with q a blocking Push/Pop : $m(q)$ is defined and $\text{clk_post}(m(q)) \leq \text{clk_post}(s)$.

The third clause is the E5/Core.SyncFence completion fence. Same-cycle completion is allowed; on the post side, the “accounted for before interval advance” convention is proof/certificate attribution for the same-cycle batch, not a requirement for observable physical sub-cycle ordering in RTL. Failed non-blocking polls in Q_{exec}, P are constrained at the issue level by E5 but contribute no committed Σ -event when they fail.

This property is guaranteed per-process by the proposition hypothesis on the chosen compatible pair (with Appendix I supplying the witnesses in the execution-level $\text{Post} \rightarrow \text{Ref}$ use case) and requires no inter-process reasoning, so it lifts directly to the system level.

Conclusion:

All five compatibility properties hold at the system level for this chosen compatible trace pair. The composition is valid because:

- intra-process properties are preserved by the per-process compatibility hypothesis;
- inter-process communication respects the applicable E4 channel semantics by the per-channel Sys_ref / RTL_B well-formedness hypothesis; and
- no progress or fairness argument is needed inside Proposition J.0, because the lemma composes already-given traces.

The existence of the source trace used in the normal $\text{Post} \rightarrow \text{Ref}$ application is supplied by Theorem J.GWR-Comp; a J.GWR-Direct package supplies the alternate entry route. Therefore $\tau_{\text{ref}, \text{system}} \approx_{\text{sys}} \tau_{\text{post}, \text{system}}$. $\square$

Remark 1. When $\text{Sys_ref} = \text{Sys_B}$ and all $B(c)=0$, Sys_ref coincides with the rendezvous interpretation of Sys , so Proposition J.0 and Theorem J.1 both specialize to the rendezvous case. In the elaborated cases, the source-side proof target is $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$.

Remark 2 (Reference-model ladder: raw Sys , abstract Sys_B , and elaborated $\text{Sys_B}^{\text{elab}}$).

Definition Core.1 is the normative definition of Sys_ref . This remark explains the model ladder, back-annotation intent, and circumstances selecting Sys_B or $\text{Sys_B}^{\text{elab}}$; it introduces no alternative reference-model definition.

The Matchlib library provides a capacity back-annotation feature that extracts finite capacities $B(c)$ from the RTL realization of each message-passing channel and makes them available to the source-level model. To avoid ambiguity, this document distinguishes three source-side models:

- Sys : the raw source-level rendezvous interpretation of the user-written processes, i.e., the model obtained when every effective channel capacity is zero and no additional abstract realization structure is introduced.
- Sys_B : the abstract bounded-FIFO source-level interpretation of the same user-written processes under E4, with finite capacities $B(c)$ at the chosen abstract channel boundaries. When all $B(c)=0$ and no additional abstract realization structure is needed, Sys_B coincides with Sys .
- $\text{Sys_B}^{\text{elab}}$: the elaborated back-annotated source-level model used in the more complex cases discussed in Appendix B / Appendix Q, in which additional explicit abstract staged/bundle/join channels are introduced so that conjunctive/join dependencies, synchronization-boundary ramp-down / epoch-gating, occupancies, and enablement are represented at explicit abstract boundaries rather than as

hidden predicates at the outer Σ -visible endpoints.

Formal proof target. Unless a statement explicitly says otherwise, the formal results of Appendices J–K target the prescribed source reference model Sys_ref , defined as follows:

- (i) $\text{Sys_ref} = \text{Sys_B}$ in the ordinary case where finite capacities $B(c)$ at the chosen abstract boundaries suffice to capture the RTL realization; and
- (ii) $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$ in the elaborated cases where additional explicit abstract staged/bundle/join channels or explicit sync-boundary / epoch-gate channels are required.

In particular, if an RTL pipeline with an explicit synchronization boundary ramps down by draining already accepted work and withholding producer-facing availability for capacity usable only after the synchronization call commits, then the prescribed source reference model is $\text{Sys_B}^{\text{elab}}$. That withholding shall be represented as ordinary `ChannelDisabled` behavior at an explicit abstract boundary of $\text{Sys_B}^{\text{elab}}$. It shall not be modeled as an extra hidden disablement predicate on an outer bounded channel whose E4 occupancy test would otherwise make the transfer enabled.

For a simple pipelined loop with a single message-passing input and output, Sys_ref is ordinarily just Sys_B : the RTL pipeline’s internal staging capacity is accounted for entirely by the back-annotated effective capacity $B(c)$ at the chosen Sys_B channel boundary. Concretely, Sys_B still has exactly one Σ -visible `Push_c(v)` at the producer endpoint and one Σ -visible `Pop_c(v)` at the consumer endpoint. With overlapped iterations, the post-HLS RTL may accept/read-ahead up to $B(c)$ values into internal pipeline staging before older iterations produce their corresponding outputs; these accept/read-ahead transfers are modeled as internal microstate (ϵ) within the concrete realization contributing to $B(c)$. The RTL may realize this staging on either side of a particular module-local “Pop interface” handshake point; that handshake point need not coincide with the chosen Σ -visible Pop endpoint. By construction, the chosen Sys_B boundary encloses all staging counted in $B(c)$ between the Σ -visible endpoints, so movements within that enclosed realization (including stage-to-stage handoff) do not create additional Σ -visible committed Push/Pop events beyond the single boundary Push/Pop events.

In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the source-side simulation into $\text{Sys_B}^{\text{elab}}$ by introducing additional explicit abstract realization structure. In those cases the formal target is $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$, and $B(\cdot)$, $\text{occ}(\cdot)$, E4, `BoundaryReq`, `ChannelEnabled/ChannelDisabled`, and the Appendix K blocking/WFG machinery are interpreted over the explicit abstract channels of that elaborated model. See Appendix B and Appendix Q.

Operational note. In all three cases, Sys_ref is a transition system with explicit process states, explicit abstract channel states, operation-attributive pending requests, and ϵ -microstate as defined in Appendix G’s operational interpretation of Sys_ref . Sys_B and $\text{Sys_B}^{\text{elab}}$ differ in the channel/elaboration component of that transition system, not in the ordinary dynamic source order of the user-written process code. Source-level message-call instances that appear in sequence in a process remain ordered by $<_{\text{src}}$; elaboration may introduce additional explicit proof-internal frontier items and abstract channels, but it does not make ordinary source-sequenced calls incomparable.

Note that in all cases the elaborated structure (staged/bundle/join channels, combinational couplers, and the projection map Π_{elab}) introduced for a particular pipelined realization is local to that process’s side of the abstract channels at its boundaries. The complementary endpoint processes see only the abstract Σ -visible channels at the boundary and are unaffected by the elaboration choices.

For Lean-style formalization, the phrase “chosen Sys_ref” should be read as an explicit theorem parameter: either Sys_B itself, or Sys_B^elab(E) for a well-formed elaboration package E as defined in Appendix Q. Likewise, references to the Appendix I/L witness machinery assume that the actual WC premise has been discharged. The corresponding audit entry records whether WC was proved through NB-CF, an Appendix L snooping / witness-selector construction satisfying Lemma L.2, another direct witness construction, or complete mixed per-site coverage. NB-CF and Appendix L snooping are provenance routes for proving WC, not independent theorem premises alongside WC.

Side-condition audit before theorem application. Applying Theorem J.1, Corollary J.1, or Appendix K to a concrete design requires the side-condition discharge record in Appendix M.7a. The normal Post $\rightarrow$ Ref route shall identify Sys_ref, discharge WC, and provide Definition J.LocalGWR: complete J.PCert, J.CCcert, J.ACcert, environment/reset, construction-action inventory, typed footprint, request-snapshot, locally generated Dep_J, LocalAgree, and coverage evidence, together with per-channel B-faithfulness. J.CanonicalRank is derived by the theorem. A flow may instead record J.GWR-Direct together with its direct-provenance J.ValidatedConstructionOrder and independently checked V1–V5 obligations. When Corollary J.1 or Appendix K is used, the application must additionally discharge J.OfferReach, J.OfferConf, and J.KObsConf for the same theorem-produced or directly supplied W, source prefix, RTL prefix, H, O, and cutpoints. Liveness uses the additional K.Drain, K ϵ , K.CV, A5-L, selected A5-L discharge route, progress, NB-Align, and L.Dir obligations as applicable. Ordinary synthesis does not automatically discharge any of these named conditions.

Remark 3 (Practical rendezvous liveness screening under determinism).

When B1 holds and the design’s control flow does not branch on empty/full observations of non-blocking interfaces (i.e., it depends only on the ordered history of observable Push/Pop/synchronization actions, not on “probe” outcomes (NB-CF, Appendix I.2)), the rendezvous specialization B(c)=0 is often an effective practical screen for design-level deadlocks: it exercises the most constrained communication discipline and can expose true cyclic data-dependency deadlocks early. Because rendezvous is strictly more constrained than buffered/elaborated operation, it can also yield false positives: a deadlock observed under B(c)=0 may disappear once finite buffering or explicit elaborated staged/bundle/join structure is present. However, bounded buffering can still introduce capacity-induced deadlocks (FIFO-depth artifacts) when FIFO depths are underestimated; these deadlocks are real for that modeled capacity configuration but may disappear once capacities are increased to the intended design point. Increasing buffer capacity (or applying dynamic buffer-growth strategies) is a standard way to distinguish depth-limited deadlocks from true cyclic dependency deadlocks. Accordingly, the formal results of Appendices J–K deliberately target the prescribed source reference model Sys_ref of Remark 2, with the actual back-annotated capacities and explicit abstract boundaries required by the chosen realization, rather than relying on rendezvous alone.

Remark 4 (When raw Sys may be used instead of the prescribed source reference model for safety-only checks).

If verification is concerned only with safety properties over the chosen DUT-boundary projection Σ (denote it Σ_{DUT}) (and not with deadlock/liveness), the prescribed source reference model Sys_ref of Remark 2 remains the default reference model. This is because Σ_{DUT} includes committed channel-transfer events, and bounded buffering / explicit staged-bundle-join structure generally changes the set/timing of those committed Push_c/Pop_c events relative to the raw rendezvous case.

Practical note (early bring-up): Before accurate capacities B(c) are available (or before back-annotation/elaboration can be used), running the rendezvous specialization Sys against the intended testbench is still useful to validate functional intent and catch gross protocol/ordering bugs early;

however, it remains a screening check, not a proof about the buffered/elaborated RTL-correspondent model.

Important (soundness): When any channel or explicit abstract staged/bundle/join boundary that can influence Σ_{DUT} -observable behavior has positive effective capacity or nontrivial enablement structure, the raw rendezvous model Sys is often a restriction of Sys_ref / RTL_B at Σ_{DUT} (it can admit fewer Σ_{DUT} behaviors). Therefore, using Sys in place of Sys_ref is generally suitable only as a debug/screening check: a failure can be useful diagnostically, but a pass does not by itself prove Σ_{DUT} -safety of the buffered/elaborated implementation.

Sys may be used as a proof-equivalent substitute for Sys_ref only when Sys and Sys_ref are Σ_{DUT} -equivalent for the chosen Σ_{DUT} (i.e., their Σ_{DUT} -projected trace sets coincide under the intended stimulus). A simple sufficient condition is that every Σ_{DUT} -visible explicit abstract channel/boundary in Sys_ref has effective capacity zero and that buffering/elaboration elsewhere is Σ_{DUT} -opaque (it cannot enable/disable/reorder Σ_{DUT} -events or change when Σ_{DUT} -events occur). In all other cases—i.e., if any Σ_{DUT} -visible explicit abstract boundary has positive effective capacity, or if buffered/elaborated internal channels can affect when Σ_{DUT} -events occur—use the prescribed Sys_ref of Remark 2.

Formal soundness condition. Let $\text{Traces}_{\Sigma_{\text{DUT}}}(\text{M})$ denote the set of Σ_{DUT} -projected traces of model M under the intended fixed stimulus. Sys may replace Sys_ref for proving a universal Σ_{DUT} -safety property ϕ only if $\text{Traces}_{\Sigma_{\text{DUT}}}(\text{Sys}) = \text{Traces}_{\Sigma_{\text{DUT}}}(\text{Sys_ref})$; under that condition, $\text{Sys} \models \phi$ iff $\text{Sys_ref} \models \phi$ (and otherwise Sys is only a diagnostic/screening check as stated above).

Lemma J.RS (MatchedResetEpochSplice).

Assume RW1–RW5 and the fixed-stimulus interpretation. Let H be a matched Theorem J.1 prefix satisfying $\text{Inv}(\text{H})$ whose next unit in the selected validated construction-order enumeration is a RESET unit $\gamma_{\text{R}} = \text{RESET}(\text{S}_{\text{p}}, \text{S}_{\text{c}})$, matched between RTL_B and Sys_ref with the same scope. Then Sys_ref extends from $\sigma_{\text{ref}}[\text{H}]$ by its corresponding reset transition, and $\text{Inv}(\text{H} \cup \{\gamma_{\text{R}}\})$ holds: (I1) holds because γ_{R} is matched with the same scope and the only required orders at the boundary are the epoch-boundary orders above; (I2) holds because, for every $P \in \text{S}_{\text{p}}$, the P-local replay state after γ_{R} is the deterministic epoch-initial state prescribed by RW3 — identical in both models by the reset-correspondence obligation — while for $P \notin \text{S}_{\text{p}}$ the replay state is unchanged by RW5; RW2 guarantees that no request-attribution fact survives the boundary for S_{p} , so Lemma I.DR' applies epoch-locally with γ_{R} delimiting the P-local projection; and (I3) holds because channels in S_{c} restart with reset-empty histories per RW1 and A10 — the $\text{occ}_{\text{c}}(t)$ definition is already measured from the most recent reset boundary — while channels outside S_{c} retain their histories by RW5. ■ Subsequent induction steps within the new epoch proceed exactly as in the initial epoch; START_{P} for $P \in \text{S}_{\text{p}}$ is matched as the new epoch's first ordinary unit. Open dispositions that are not covered by RW1–RW5 — for example, an un-modeled partial reset of one endpoint of a live channel, retention of in-flight messages across the boundary, or cross-epoch request re-attribution — place the design outside Theorem J.1, Corollary J.1, and Appendix K until modeled explicitly. Under the Core.CycleResult interface, γ_{R} is the emitted resetUnits occurrence corresponding to that scoped reset transition; the induction index remains γ_{R} itself, not the enclosing cycle result. Thus the CycleResult refactor does not change the reset splice case or its epoch-local replay/channel-history obligations.

Expanded proof of Theorem J.GWR-Comp and Theorem J.1

Post $\rightarrow$ Ref, compositional route. Fix $\tau_{\text{post}} \in \text{Reach}_{\text{post}}$ and $\text{LocalGWRPackage}(\tau_{\text{post}}, \text{L})$. Generate the complete field-sensitive action and deferred-hole inventory, Dep_J, FR, and J.CanonicalRank. Partition the complete units by their selected Horizon, retaining explicit atomic groups. Within each

Horizon, stable proof keys order only otherwise commuting actions; J.RegPhaseNF later removes dependence on those incidental choices modulo J.HCanon.

Let $B_1 \dots B_n$ be the nonempty Horizon buckets and H_i the finite required-prefix ideal containing all units through B_i ; within B_i retain the commit-action suborder only for the inner L3 pass. Construct by outer induction reachable cycle-entry and post-boundary states. The invariant is: (I1) exactly H_i has committed with matching units/values/identities and required order; (I2) process replay and every pending/deferred-hole record agree with the local certificates at cycle entry, while no deferred hole survives a post-boundary state; (I3) explicit boundary/channel/atomic state equals the CCert/ACert reconstruction; (I4) requests, witness choices, reset epochs, direct-input anchors, and environment history agree; and (I5) the constructed prefix through H_i satisfies J.RegPhaseReach and has the same CanonHist quotient as the selected RTL prefix through those units.

Base case. Use the matched initial state, reset-empty channel state, empty request inventory, fixed environment history, and LocalAgree. For later dynamic resets, ResetCert and Lemma J.RS re-establish the same invariant per reset epoch while framing components outside the reset scope.

Induction step. Let B_t be the next Horizon bucket. Execute all finite L1 A1/A2 predecessors, including actions carried past unresolved NB calls under NoDependentUse. Field-sensitive framing and J.DepComplete permit the required commutations; true dependencies remain ordered. J.RegPhaseNF then freezes the complete request set, performs the L2 settle, and orders all A3/A4 decisions/commits in L3. A failed NB writes its returned status and retires its hole; a successful NB is one of the bucket's complete units. No decision-dependent continuation remains in t , and no new request is issued after settle.

Every witness-significant failed or successful NB call is therefore evaluated from one common settled incident-boundary snapshot. The construction cannot choose failure with a matching rendezvous request present, success in an empty/full state that forbids it, or an outcome for the wrong dynamic call. WC identifies the selected result; J.CCert/J.NBDecision prove it unique. Outcome-independent same-Horizon work is already in L1; outcome-dependent work begins only in a later bucket.

J.UnitEnable proves every complete unit in the bucket enabled at the appropriate L3 point. The buffered case uses common pre-update occupancy/head and J.BufSameCycle; rendezvous uses simultaneous attributed requests/attempts and payload identity; synchronization/E2/R2C/environment/reset use their named certificates. Executing the inner commit-action order reaches the post-boundary state for H_{iUB_t} . The phase-normal segment has the same complete units, same post-boundary semantic state, and same J.HCanon quotient as any permitted same-Horizon topological ordering.

Lemma I.DR' updates participant replay state; J.CCert/ACert update exact channel/atomic state; J.Frame preserves nonparticipants; LocalAgree preserves coherence; and the all-holes-resolved invariant establishes the clean post-boundary cutpoint. Thus (I1)–(I5) hold for the next bucket. Finiteness follows from the local certificates/FPD, not fairness.

After the final Horizon bucket, the chain covers every unit of τ_{post} . Assemble W from the local certificates, generated graph, derived bucketed order, phase-normal segments, snapshots, and invariants. Definition J.GWR and J.RegPhaseReach hold. The terminal source prefix and τ_{post} have compatible annotated projections and well-formed explicit-channel histories, so Proposition J.0 proves Theorem J.1's compositional $\text{Post} \rightarrow \text{Ref}$ clause.

$\text{Post} \rightarrow \text{Ref}$, direct route. If a validated W satisfying Definition J.GWR is supplied directly, take the reachable terminal source prefix from its chain and apply Proposition J.0 to the annotated pair. This route has the same semantic conclusion but bypasses the local-certificate derivation.

Normalization and cutpoints. Neither route appends a maximal ε -normalization suffix after each matched unit. Corollary J.1 separately applies B4/B5 to the selected terminal prefixes and then requires J.OfferReach, J.OfferConf, and J.KObsConf for the normalized cutpoints. The theorem-produced W is the authoritative source of the operational identities and history used by those obligations.

Ref $\rightarrow$ Post clause. Let $\tau_{\text{ref}} \in \text{Reach_ref}$ and choose $\tau_{\text{post}}^* \in \text{Reach_post}$ and W with $\text{RTLConsistentRefPrefix}(\tau_{\text{ref}}, \tau_{\text{post}}^*, W)$. By definition, the selected pair already carries compatible annotated projections, Issue/Commit and request-attribution evidence, dynamic identities, WC/μ_Q data, and E4 histories. Proposition J.0 therefore gives $\tau_{\text{ref}, \text{system}} \approx_{\text{sys}} \tau_{\text{post}}^*, \text{system}$; take $\tau_{\text{post}} = \tau_{\text{post}}^*$. B1 supplies uniqueness only of the selected RTL Σ projection, not of the annotation package. This is not a construction from an arbitrary source run. $\square$

Lemma J.HCanon (Canonical replay-history quotient agreement)

Fix the same theorem instance I , elaboration E , witness w , reset interpretation, dynamic operation identities, and theorem-produced or directly validated J.GWR annotation package. Let τ_{post} be a finite RTL_B prefix and τ_{ref} the Theorem J.1 source witness prefix selected for τ_{post} . If τ_{ref} and τ_{post} satisfy the annotated observable-unit compatibility relation of Theorem J.1 under that fixed package, then their canonical enriched replay-history quotients are equal:

$$\text{CanonHist}_{E,w}(\tau_{\text{ref}}) = \text{CanonHist}_{E,w}(\tau_{\text{post}}).$$

Proof. The compatibility relation matches the same atomic observable units with the same labels, values, dynamic identities, and reset scopes. The fixed witness w and μ_Q/WC package identify the same witness-selected failed non-blocking, arbitration, signal, and ε -choice outcomes used by replay. Theorem J.1 preserves each process-local required order, each per-channel committed-transfer ordinal/FIFO order, every explicit atomic rendezvous or paired-synchronization grouping, every RESET marker, and every predecessor required by $\prec_J$. CanonHist retains exactly those fields. It discards only incidental source-versus-RTL clock-bucket placement and ordering among units that are independent under the retained constraints. Therefore both annotated prefixes map to the same typed quotient H . This lemma asserts equality of the canonical replay quotient, not equality of concrete cycle buckets or historical Issue timestamps. $\blacksquare$

Definition J.RegPhaseReach (phase-correct reachability of the selected source witness)

For fixed I, E, w and a selected J.GWR source prefix τ_{ref} ending at σ_{ref} , $\text{RegPhaseReach}_{I,E,w}(\tau_{\text{ref}}, \sigma_{\text{ref}})$ means that τ_{ref} is Policy-L-legal under Core.RegSeq and admits the Horizon-bucket decomposition of J.RegPhaseNF: every cycle's process-owned A1/A2 actions occur in L1, one L2 settle fixes the requests/snapshots, all A3/A4 decisions/commits occur in L3, ReturnedAtCycle-dependent work occurs only in later Horizons, and no deferred NB hole remains at σ_{ref} . A J.GWR-Direct package must supply this checked property for its selected prefix. RegPhaseReach establishes the L0–L4 ordering/legality component used by Definition Core.LAdm through Core.Phase; it does not establish nonwithdrawal, $K\varepsilon$, or the continuation-level Core.Drain/K.Drain discipline.

Corollary J.RegPhaseReach-Comp (compositional derivation of phase-correct reachability)

Under the hypotheses of Theorem J.GWR-Comp, every constructed source prefix τ_{ref} ending at σ_{ref} satisfies $\text{RegPhaseReach}_{I,E,w}(\tau_{\text{ref}}, \sigma_{\text{ref}})$. Proof. Lemma I.3 carries each enriched deferred NB hole through exactly the same-Horizon work certified by NoDependentUse; J.RegPhaseNF normalizes each Horizon modulo J.HCanon into L1 process actions, one L2 settled snapshot, and L3 decisions/commits, using field-sensitive framing and J.BufSameCycle; and Horizon-batched J.UnitExt preserves that normal form while constructing the reachable prefix. ReturnedAtCycle-dependent work is assigned only to later Horizons, and the exported-cutpoint invariant removes every unresolved deferred hole at σ_{ref} . $\blacksquare$

Definition J.OfferReach (reachable source realization of a KObs record)

Fix I, E, w, H, O , and the particular Sys_ref witness prefix selected for the current Theorem J.1 / Corollary J.1 application. $\text{OfferReach}_{I,E,w}(H, O; \tau_{\text{ref}}, \sigma_{\text{ref}})$ holds when τ_{ref} is a finite reachable Sys_ref prefix under the same stimulus; its selected proof-relevant source normalization satisfies

NormRef_I($\tau_{\text{ref}}, v_{\text{ref}}, \sigma_{\text{ref}}$) for some recorded v_{ref} and ends at $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$; $H=H_E, w(\sigma_{\text{ref}})$; $O=\text{ResOffers_E}, w(\sigma_{\text{ref}})$; the Issue/ReturnedAtCycle/Commit, E3/E5, Axiom Core.IC, reset, elaboration, and explicit-boundary rules hold; and no proof-internal deferred NB hole is exported at σ_{ref} . OfferReach remains an image/realizability predicate, not a Core.LAdm qualification. OfferReach is an image/realizability predicate. It is not implied by type-correctness, source-prefix closure, or endpoint cardinality of an arbitrary tuple $\langle E, w, H, O \rangle$. In Corollary J.1 it is discharged for the same normalized source witness selected from Theorem J.1, not for an unrelated existential source state. A source execution log, mechanized reachability proof, or tool/witness certificate may discharge it.

Historical Issue and ReturnedAtCycle data may be used internally to prove reachability, phase legality, and the current residual-offer set, but they are not KObs fields and are not compared as cross-model cycle timelines. OfferReach records the realized H/O cutpoint; J.RegPhaseReach records the theorem-derived/direct L0–L4 phase property; and K.DrainReach records the remaining Appendix K continuation discipline. Their composed K.RLAdmReach is the stricter Pref_L^adm(I) qualification.

Definition K.DrainReach and K.RLAdmReach (composed Core.LAdm qualification)

Fix $I, E, w, H, O, \tau_{\text{ref}}, \sigma_{\text{ref}}$ satisfying OfferReach. DrainReach_I, E, w($H, O; \tau_{\text{ref}}, \sigma_{\text{ref}}$) holds when the exact prefix and instance binding satisfy request nonwithdrawal and $K\epsilon$; discharge Core.Drain through K.Drain from every L2 fully-disabled-frontier SettlePoint reached by τ_{ref} ; and discharge Core.SettleKBridge for the exact normalization/cutpoint path that ends at σ_{ref} , including the no-new-later-work, finite non-replenishing drain, sync-boundary, attribution, and ϵ -inertness obligations.

RLAdmReach_I, E, w($H, O; \tau_{\text{ref}}, \sigma_{\text{ref}}$) is the conjunction of J.RegPhaseReach_I, E, w($\tau_{\text{ref}}, \sigma_{\text{ref}}$) and DrainReach_I, E, w($H, O; \tau_{\text{ref}}, \sigma_{\text{ref}}$), with the same fixed I/E/w/stimulus/environment/arbitration/reset choices. Hence RLAdmReach proves $\tau_{\text{ref}} \in \text{Pref_L}^{\text{adm}}(I)$. On the compositional route the phase component is theorem-derived, while DrainReach remains an explicit checked bridge from implementation/source operational evidence; on the direct route both components are directly validated. Neither equal KObs nor OfferReach implies DrainReach. A K-facing CF-0 package shall carry/refer to the RegPhaseReach provenance and the DrainReach/Core.SettleKBridge proof for the exact cutpoint tuple.

Definition J.RelevantReq (KObs-relevant RTL endpoint request)

At an RTL K-facing cutpoint $p_{\text{post}} \in \text{KCutReach_post}(I)$, an asserted request level r is KObs-relevant for elaboration E when: (i) r is endpoint-owned and lies at an explicit abstract boundary selected by E ; and (ii) the presence or absence of r can affect the reconstruction of BoundaryReq, a rendezvous endpoint-request/co-presence fact, ChannelEnabled/ChannelDisabled, Front, WFG, WFG+, or a derived Dead_K/Dead_KS diagnostic predicate. Relevance is determined from the selected physical/abstract boundary and the reconstructed cutpoint interface, not from whether the flow happened to supply an attribution. Thus an asserted boundary request with missing attribution remains relevant and causes J.OfferConf to fail; it cannot be dropped from the abstraction by calling it unrecognized. K.Resp timers are not KObs fields and are validated separately by CF-S.

The dynamic operation/frontier/epoch attribution used below is supplied by the operational request-attribution component constructed by J.GWR-Comp or supplied by J.GWR-Direct, constrained by Appendix C's Issue/Commit linkage and Implementation Assumption I.A0. It is not inferred from the ready/valid wire level alone. A post-cycle request level proved by those annotations to belong only to an operation that committed in the selected cycle, with no next dynamic operation owning the continuing level, is not a residual offer. For a back-to-back continuing assertion, the annotation shall identify the next owning dynamic operation.

Obligation J.OfferConf (bidirectional RTL residual-offer conformance)

For an RTL K-facing cutpoint $p_post \in KCutReach_post(I)$, fixed E and w , and candidate residual-offer set O , $OfferConf_I,E,w(p_post,O)$ holds only if the following boundary conditions are satisfied at every explicit abstract boundary used by the theorem instance. All references to attribution use the theorem-produced or directly supplied operational package, Appendix C, and I.A0 as specified by Definition J.RelevantReq; no attribution is inferred solely from an asserted wire level.

- (1) Offer-to-request completeness. For every $o = (P,x,q,c,dir,e) \in O$, the corresponding endpoint-owned RTL request at c and direction dir is asserted at p_post , is KObs-relevant, and is attributed by the selected J.GWR-Comp-produced operational package (or J.GWR-Direct package) to that same frontier item x and source operation q in reset epoch e , with the operation kind derived from q .
- (2) Request-to-offer completeness and uniqueness. Every KObs-relevant endpoint-owned RTL request asserted at the selected RTL K-facing cutpoint is attributed by the selected J.GWR-Comp-produced operational package (or J.GWR-Direct package), Appendix C, and I.A0 to exactly one member of O , unless that package proves that the level belongs only to an operation that committed in the selected cycle and no next operation owns the continuing assertion. If the level remains high for a back-to-back operation, the attribution shall identify that next dynamic operation and the corresponding member of O . No relevant asserted request may be hidden, left unattributed, or omitted from O . At most one attributed outstanding source-level offer exists per explicit boundary and endpoint direction.
- (3) Snapshot issue-order consistency. The issued-summary represented by the committed/replay history H , the WC witness information, and the current residual offers O is prefix-closed under the applicable dynamic source order and is consistent with $E3$, $E5$, reset epochs, and the selected elaboration. This is a cutpoint set invariant; OfferConf does not compare historical first-issue timestamps across models.
- (4) Boundary and reset typing. Every x , q , c , direction, kind, owner, and epoch in O is supplied by E and agrees with the live RTL boundary attribution. Reset clears ending-epoch offers as required by RW2. Internal movement already represented as $E4$ occupancy is not also exposed as a second source-level offer.
- (5) Payload scope. O need not contain pending Push payload values. Stable request payload is still required by Appendix C/Axiom Core.IC, and committed payload equality is checked by the value-labeled replay history. A flow that needs mid-flight data-wire correspondence may add it as a separate implementation obligation.

Audit-critical status of J.OfferConf(2). J.OfferConf is the authoritative RTL request-abstraction boundary for the KObs cutpoint result. In particular, request-to-offer completeness, no-hidden-request, explicit-boundary completeness, and attribution uniqueness are load-bearing obligations: omitting any asserted request that is KObs-relevant can remove a reconstructed wait-for edge or rendezvous-enablement fact and invalidates J.OfferConf, J.KObsConf, and the equal-KObs conclusion. B-faithfulness/ $E4$ and canonical-history agreement remain separate independent obligations; satisfying them does not compensate for an incomplete request abstraction.

Obligation J.KObsConf (composite KObs cutpoint conformance)

For every RTL K-facing cutpoint $p_post \in KCutReach_post(I)$ used by Corollary J.1 or Appendix K, the flow shall select one exact package $(\tau_ref, \sigma_ref, p_post, I, E, w, H, O)$ such that: (a) the selected source and RTL prefixes use the same fixed instance, elaboration, witness, stimulus, reset-epoch interpretation, and observation boundaries; (b) Lemma J.HCanon yields H as their common canonical replay-history quotient, without requiring equal incidental cycle bucketings; (c) $J.OfferReach_I,E,w(H,O;\tau_ref,\sigma_ref)$ holds and therefore realizes O at the reachable source cutpoint; (d) $J.OfferConf_I,E,w(p_post,O)$ holds and therefore identifies O bidirectionally and uniquely with the complete KObs-relevant RTL request inventory; and (e) the selected RTL abstract boundaries satisfy $E4$, B-faithfulness, reset correspondence,

and ϵ -opacity of the finite normalization. $J.KObsConf$ is tied to the particular source witness chosen by Theorem J.1; an unrelated existential source cutpoint or an unrealizable residual-offer record does not discharge it.

Define the J-facing RTL observation certified by that package as $KObsPost_J(\rho_post; E, w, H, O) := \langle E, w, H, O \rangle$. $J.OfferReach$ and Definition Core.18c give $KObs_E, w(\sigma_ref) = \langle E, w, H, O \rangle$. Hence $J.KObsConf$ entails $KObsPost_J(\rho_post; E, w, H, O) = KObs_E, w(\sigma_ref)$. This typed equality is the sole cutpoint-level correspondence used by Appendix K. It is not equality of arbitrary RTL and source microstates, and it creates no independent clause-based state relation.

J.2a KObs Reconstruction Theory

The following lemmas are source-side reconstruction results over reachable Core.18c KObs records. They are placed here because Corollary J.1 consumes them after $J.OfferReach$ and $J.OfferConf$ establish one common typed record across the selected source and RTL cutpoints.

Lemma J.KObs-Proc (Process replay reconstruction).

Let $\sigma \in KCutReach_ref(I)$ under I, E, w and let $K = KObs_E, w(\sigma)$. For every process P , $ProcRec_P(K)$ equals the $I.ReplayObs$ source slice of σ used by Lemmas I.DR/I.DR' and the Appendix J finite-ideal construction. Consequently $DoneRec_P(K)$, $RemainRec_P(K)$, $NextFrontRec_P(K)$, all replay-derived next-item selection predicates, all values needed to determine future committed Σ labels, and P 's normal-completion status agree with their operational definitions at σ . $ProcRec$ does not reconstruct $Pending_P$, $Front_P$, or $BoundaryReq$; those current offer facts are supplied by O and proved separately by Lemma J.KObs-Offers. Proof. Project H to P , include the WC/witness choices and RESET epoch markers, and apply deterministic replay I.DR/I.DR'; the replayed process state together with Definition Core.6 determines completed/remaining items and candidate $NextFront$. $\square$

Lemma J.KObs-Chan (Abstract channel reconstruction).

Let σ and K be as in Lemma J.KObs-Proc. For every explicit abstract channel c declared by E , $ChanRec_c(K)$ equals the $E4$ abstract channel state of σ . For $B(c) > 0$, occupancy and logical contents are uniquely obtained from reset/initial contents and the committed Push/Pop sequence and per-channel ordinals in H . For $B(c) = 0$, occupancy is identically zero and current rendezvous enablement additionally uses the endpoint offers in O . In Sys_B^{elab} , the same statement holds at each explicit proof boundary, and Π_elab supplies the declared outer projection. Proof. Induction over the committed channel-history projection of H , using $E4$, B-faithfulness, RESET/RW1–RW3, and Π_elab conservation. $\square$

Lemma J.KObs-Offers (Residual-offer, BoundaryReq, and frontier reconstruction).

Let σ and $K = \langle E, w, H, O \rangle$ be as above. For every process P and blocking message-operation frontier item x , $x \in Pending_P(\sigma)$ iff $x \in PendingRec_P(K)$, and $BoundaryReq(x, \sigma)$ iff $BoundaryReqRec(x, K)$. Hence $Front_P(\sigma) = FrontRec_P(K)$. For every point-to-point rendezvous channel, Req_prod and Req_cons at σ are exactly the existence of the corresponding producer-direction and consumer-direction offers in O . Proof. O was defined as the exact set of current operation- and boundary-attributed blocking endpoint requests. Core.SyncFence excludes a cross-interval survivor: after any synchronization commit, no blocking offer from the preceding interval remains outstanding. Therefore the P -owned blocking offers in O are exactly the current-interval items admitted by $Pending_P$. Apply Definitions Core.14–Core.15 and take the $\leq_P$ -minimal subset. The one-offer-per-endpoint-direction invariant makes the rendezvous projection functional. $\square$

Lemma J.KObs-WFG (Enablement and wait-for-graph reconstruction).

For every residual offer x at σ , $\text{ChannelEnabledRec}(x, K) = \text{ChannelEnabled}(x, \sigma)$, and likewise for ChannelDisabled . Consequently $\text{WFGRec}(K) = \text{WFG}(\sigma)$. Proof. Use Lemma J.KObs-Chan for buffered occupancy/content state, Lemma J.KObs-Offers for rendezvous endpoint requests and attributed frontier membership, and Definitions Core.11–Core.13 together with Definition Core.WFG at the explicit boundary supplied by E . The no-hidden-cross-channel-disablement convention ensures that no additional enabling fact is required. $\square$

Corollary J.1 (K-observation Correspondence at the End of a Matched Observable Prefix)

Purpose.

This corollary is the bridge between execution-level trace compatibility and the K-facing cutpoint observation used directly by Appendix K. Its primary and authoritative cutpoint conclusion is equality of the compact KObs record, not equality of arbitrary source and RTL state. K-relevant frontier, request, channel, enablement, WFG, drain, and deadlock facts are derived from that equality by the named reconstruction functions; no clause expansion is required by the Appendix K transfer layer. Corollary J.1 establishes a reachable source realization of the common record but does not by itself establish membership of that witness in $\text{Pref_L}^{\text{adm}}(I)$. An Appendix K application supplies the separate K.RLAdmReach proof for the exact selected source prefix.

Statement. Assume the hypotheses of Theorem J.1 for Sys_ref versus RTL_B under the fixed stimulus, including B1 for the $\text{RTL_B} \Sigma$ projection. Let $\tau_{\text{post}} \in \text{Reach_post}$ carry the J.GWR package constructed by J.GWR-Comp or supplied by J.GWR-Direct. Select and record a finite RTL normalization v_{post} with $\text{NormPost_I}(\tau_{\text{post}}, v_{\text{post}}, \rho_{\text{post}})$, so $\rho_{\text{post}} \in \text{KCutReach_post}(I)$. Select the matching Theorem J.1 source witness prefix and the proof-relevant source normalization recorded by J.OfferReach, ending at $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$. B4/B5 may be used to discharge existence/termination of these selected normalizations only under their stated premises; Corollary J.1 does not identify an L2 SettlePoint with either K-facing endpoint. Fix the same I , E , w , H , and O throughout, and assume J.KObsConf for this exact package, namely: canonical-history agreement H from Lemma J.HCanon, $\text{J.OfferReach_I}, E, w(H, O; \tau_{\text{ref}}, \sigma_{\text{ref}})$, $\text{J.OfferConf_I}, E, w(\rho_{\text{post}}, O)$, B-faithful/E4 interpretation at E 's explicit boundaries, reset correspondence, and ϵ -opacity. Then:

- (1) $\tau_{\text{ref}, \text{system}} \approx \tau_{\text{post}, \text{system}}$ under $E1$ – $E5$; and
- (2) $\text{KObsPost_J}(\rho_{\text{post}}; E, w, H, O) = \text{KObs_E}, w(\sigma_{\text{ref}})$.

Derived K-facing consequence. Let K denote the common record in conclusion (2). Every typed reconstruction function in Definitions Core.18d–Core.18e has the same value on the selected post and source views. In particular, ProcRec , ChanRec , NextFrontRec , PendingRec , BoundaryReqRec , FrontRec , $\text{ChannelEnabledRec}/\text{ChannelDisabledRec}$, WFGRec , and DrainClosedNowRec agree. These are functional consequences of conclusion (2), not additional implementation-side state-equality assumptions. Corollary J.1 makes no Dead_K , Dead_K^W , Dead_KS^W , or K.RespFail conclusion; Appendix K applies its own factoring results to the common record when those predicates are needed.

Proof of Corollary J.1

Step 0 (Choose the K-facing normalized cutpoints). Use the recorded NormPost_I witness to obtain $\rho_{\text{post}} \in \text{KCutReach_post}(I)$, and use the source normalization recorded by J.OfferReach to obtain $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$. Each endpoint is globally ϵ -quiescent for its own model under Core.4 and has settled explicit-boundary enablement under Core.KCut. Neither endpoint is the L2 SettlePoint merely by definition. B4/B5 or another stated bounded-normalization argument discharges finiteness where applicable; uniqueness of the normalization is not required because all conclusions are relative to the selected package.

Step 1 (Obtain the matched replay history). Apply Theorem J.1 to the selected RTL prefix and its theorem-produced or directly supplied J.GWR package to obtain the matching Sys_ref witness prefix under the same stimulus. Lemma J.HCanon then gives

$$\text{CanonHist_E,w}(\tau_{\text{ref}}) = \text{CanonHist_E,w}(\tau_{\text{post}}) = H.$$

Thus conclusion (1) holds, and the replay-history field of the two KObs records is equal even when their concrete cycle-bucket decompositions differ. The retained fields and the erased incidental ordering are exactly those stated in Lemma J.HCanon.

Step 2 (Establish the common residual offers). J.OfferReach identifies O exactly with ResOffers_E,w(σ_{ref}) at the selected reachable source cutpoint. J.OfferConf identifies that same O bidirectionally and uniquely with the relevant asserted endpoint-owned RTL requests at p_{post} . The reset-epoch, source-prefix-closure, boundary-cardinality, and no-hidden-request conditions ensure that O is a well-typed residual-offer component for both sides.

Step 3 (Conclude equal KObs). Both records have the same elaboration E, witness w, replay history H, and residual-offer set O. Therefore $\text{KObsPost_J}(p_{\text{post}}; E, w, H, O) = \langle E, w, H, O \rangle = \text{KObs_E,w}(\sigma_{\text{ref}})$, proving conclusion (2). B-faithfulness and E4 ensure that the post-side interpretation of the committed history at each explicit boundary is the same abstract channel interpretation used by ChanRec; finite normalization is ϵ -opaque by J.KObsConf.

Step 4 (Derive the K-facing facts). Lemmas J.KObs-Proc–J.KObs-WFG reconstruct the process slice, explicit abstract channel state, NextFront, residual endpoint requests, Pending/BoundaryReq attribution, enablement, Front, WFG, and DrainClosedNow from the common record K. These conclusions follow by functional extensionality of the named reconstruction functions; no broad state correspondence is assumed. Deadlock-, waiver-, environment-terminal-, and diagnostic-predicate factoring is deliberately deferred to Appendix K, which consumes this equal-KObs conclusion. The Policy-S K.Resp discharge is separately checked over $p_{\text{S}^{\text{det}}}$ and is not a KObs cross-model predicate. $\square$

J.3 Causal Dependency and What System-Level Compatibility Guarantees (Informative)

The system-level result of Appendix J establishes compatibility of the aggregate observable behavior, i.e. $\tau_{\text{ref}, \text{system}} \approx \tau_{\text{post}, \text{system}}$ over the alphabet Σ of channel operations, synchronization events, and signal Read/Write actions.

This compatibility is intentionally *observational*: it constrains what an external environment can distinguish at Σ , not internal RTL micro-timing.

As a consequence, Appendix J should be read as preserving explicitly required observable ordering, atomicity, and FIFO/synchronization semantics, not incidental latency. The causal-dependency relation ($\rightarrow$) defined in the “Causal Dependency Between Processes” section remains useful for explaining information flow and for documenting which interactions a designer intends to be functionally significant. However, $\rightarrow$ is not itself the strict induction order for Theorem J.1, and the theorem does not require the unquoted dependency graph $\rightarrow$ to be acyclic. The proof order is the required-order relation $\prec_J$ over observable units.

Accordingly, the system-level theorem implies the following required-order preservation principle:

- If two observable units γ_1 and γ_2 are ordered by the required-order relation $\gamma_1 \prec_J \gamma_2$, then the matched Sys_ref and RTL_B executions preserve that required order. This follows because each elementary $\prec_J$ generator is preserved by the corresponding rule: synchronization source order by E1; signal-anchor timestamping by E2; issue-side and request-attribution obligations by E3; channel FIFO legality and rendezvous atomicity by E4; immediate synchronization-boundary constraints by E5; and paired-barrier semantics by the explicit synchronization well-formedness conditions. In particular, for

each explicit abstract channel c , the matched executions preserve the committed $\text{Push}_c/\text{Pop}_c$ history: for $B(c)=0$, a committed transfer is one explicitly atomic rendezvous $\text{Push}_c(v)/\text{Pop}_c(v)$ unit; for $B(c)>0$, the k -th committed Pop_c observes the k -th earlier unmatched Push_c according to reset-empty, FIFO-order, no-underflow/no-overflow, and non-fall-through E4 semantics.

- If two observable units are not ordered by $\leq_J$ and are not members of the same explicitly atomic unit, then their relative cycle placement or interleaving is not constrained by the methodology, and differences are treated as incidental latency variation. A designer who needs a particular cross-process ordering to be preserved must express it through an explicit Σ -visible mechanism such as message-passing FIFO order, a rendezvous/handshake atomic unit, explicit synchronization, or signal-anchor semantics.

This perspective is what makes the “single testbench / no surprises” implication precise: for the fixed-stimulus execution sets covered by Theorem J.1, any testbench that observes only Σ and encodes its expectations through causal dependencies (channels, barriers, or explicit handshakes) cannot distinguish the matched Sys_ref / RTL_B execution pairs provided by the theorem.

Equivalently, RTL_B introduces no new Σ -observable behavior relative to Sys_ref under that fixed stimulus, and source-to-RTL transfer is asserted only for the annotated RTL-consistent source prefixes covered by Theorem J.1. (Here Sys_ref is as fixed in Remark 2. The rendezvous specialization Sys with $B(c)=0$ is covered only under the explicit conditions of Remark 4.)

Appendix K – Preservation of Internal Wait-Cycle Freedom (Reachable-Cutpoint Deadlock-Freedom) for the Prescribed Source Reference Model

K.1 Scope, Notation, and Definition of Internal Message-Passing Deadlock

Terminology alignment

Throughout Appendix K, when evaluating K-facing predicates at a cutpoint σ in $\text{KCutReach_ref}(I)$ or the corresponding certified RTL K-facing view, let t_σ denote the cycle of that cutpoint and use the cutpoint shorthand $\text{occ}_c(\sigma) \triangleq \text{occ}_c(t_\sigma)$. For any frontier item x on an explicit abstract channel $c = \text{BoundaryOf_P}(x)$ of Sys_ref , with owning dynamic source-level message instance $q = \text{MsgOf_P}(x)$, use $\text{BoundaryReq}(x, \sigma)$, $\text{ChannelEnabled}(x, \sigma)$, and $\text{ChannelDisabled}(x, \sigma)$ for the endpoint-owned request, enablement, and disabledness facts at that specific Core.KCut evaluated boundary. In the ordinary unelaborated case, where q has a unique evaluated boundary and a unique frontier item, $\text{BoundaryReq}(q, \sigma)$, $\text{ChannelEnabled}(q, \sigma)$, and $\text{ChannelDisabled}(q, \sigma)$ may be used as shorthands. In $\text{Sys_B}^{\text{elab}}$, however, the frontier-item form is authoritative because one source-level operation may own multiple proof-internal frontier items at different explicit boundaries. Thus “boundary request is asserted” means $\text{BoundaryReq}(x, \sigma)$, “channel-enabled” means $\text{ChannelEnabled}(x, \sigma)$, and “channel-disabled” means $\text{ChannelDisabled}(x, \sigma)$, all evaluated at the Core.KCut boundary fixed point of the selected cutpoint cycle and read through $q = \text{MsgOf_P}(x)$ and $\text{BoundaryOf_P}(x)$. These predicates are not applied to already-committed Σ -event labels except through the operation/frontier item that contributes $m(q)$ when it commits.

We work with a fixed design:

System Sys_ref : The collection of pre-HLS processes obeying the R-rules and B-rules, interpreted under the prescribed source reference model of Definition Core.1 and Appendix J, Remark 2. In this appendix, capacities/occupancies/enableness predicates are interpreted at the chosen abstract boundaries of Sys_ref ; in elaborated cases, this means the explicit abstract staged/bundle/join channels of that model. Each explicit abstract channel c of Sys_ref has finite capacity $B(c) \geq 0$, chosen to match the corresponding RTL realization at that abstract boundary.

Post-HLS implementation RTL_B : The hardware implementation of the same processes in which each chosen abstract channel c is realized with finite capacity $B(c) \geq 0$. In this appendix, we compare Sys_ref (the prescribed source-side proof target) to RTL_B (the micro-architectural realization). The proof does not require mapping buffered RTL states to a distinct raw rendezvous ($B(c)=0$) state.

Witness-selection and post-view convention. Corollary J.1 supplies a selected reachable Sys_ref cutpoint σ_{ref} and a common record K with $\text{KObs_E},w(\sigma_{\text{ref}}) = \text{KObsPost_J}(p_{\text{post}};E,w,H,O)$ for the selected RTL K -facing cutpoint p_{post} . Appendix K does not compare arbitrary RTL microstate with source state. Define the K -facing post abstract view by applying the reconstruction functions of Definitions Core.18d–Core.18e to KObsPost_J . Unless a raw RTL signal is expressly named as part of $J.\text{OfferConf}$, every post-side occurrence of NextFront , Pending , BoundaryReq , channel state, $\text{ChannelEnabled/ChannelDisabled}$, Front , WFG , DrainClosedNow , or Dead_K in Appendix K denotes that reconstructed view. If a reproducible source witness is desired for debugging, the Appendix L snooping wrapper may select one aligned to the RTL latency-sensitive events; uniqueness is not required by the theorem.

A global state σ of either system consists of:

1. For each process P : a control location (program point) and local state.
2. For each explicit abstract channel c of Sys_ref : its abstract FIFO state at the chosen Sys_ref boundary—its current occupancy $\text{occ_c}(t)$ and (for buffered channels) its ordered contents. Here $\text{occ_c}(t)$ counts all items that have been committed by Push_c but have not yet been committed by Pop_c at that chosen abstract boundary. In RTL_B , this count includes items residing anywhere in the concrete sub-realization that contributes to that particular abstract channel’s back-annotated capacity $B(c)$. Thus internal movement within the concrete realization of one abstract channel c is ε and does not create additional committed Push_c/Pop_c Σ -events beyond the boundary events counted by occ_c ; however, transfers between distinct explicit abstract channels introduced in $\text{Sys_B}^{\text{elab}}$ are represented in Sys_ref by those channels themselves and are not collapsed back into one outer Sys_B boundary.
3. Any additional architectural state not representing buffered messages on any channel (e.g., computation pipeline registers, arbitration state, bookkeeping, etc.).

Appendix K uses the normative wait-for graph $\text{WFG}(\sigma)$ of Definition Core.WFG, restricted there to internal message-passing channels. We write WFG_ref for the graph reconstructed at a Sys_ref cutpoint and WFG_post for the corresponding K -facing graph at the selected RTL_B cutpoint. All vertices, edge conditions, the edge-versus-blocked and multi-frontier clarifications, frontier-item attribution, complementary-endpoint ownership, buffered/rendezvous treatment, and exclusion of environment-facing channels are exactly those of Definition Core.WFG; this appendix adds the deadlock, closure, waiver, and preservation predicates but does not redefine WFG.

(No deassert-before-commit assumption.) Blocking Push_c/Pop_c requests are non-withdrawable within one reset epoch: once the request corresponding to a frontier item $x \in \Omega_{\text{msg},P}$ is asserted through its owning message instance $q = \text{MsgOf_P}(x)$, it is not deasserted before q commits, and Push_c payload remains stable while asserted. If an affecting RESET occurs first, RW2 ends that dynamic instance at the epoch boundary. No other nonterminal

transition may withdraw or reattribute it. This rules out “try-and-withdraw” behavior for blocking transfers within the WFG abstraction.

K-facing cutpoint convention. All WFG and Dead_K reasoning is over Core.KCut states: the source state lies in $\text{KCutReach_ref}(l)$, and the selected/certified RTL state lies in $\text{KCutReach_post}(l)$. Core.4 quiescence means no internal ϵ -step is enabled in the corresponding model at its current cycle/phase; it does not mean that no Σ action or environment action is enabled. Purely combinational ready/valid propagation is additionally required to be at its fixed point by Core.KCut. Every source-side L2 SettlePoint is a KCut by Lemma Core.SettleIsKCut, although a Dead_K evaluation may occur later after Core.16-permitted drain has reached a drain-closed KCut. Core.SettleKBridge/K.DrainReach supplies that continuation relation. On RTL_B, Core.KCutClosure_post is the separate non-vacuity premise ensuring that each reachable K-facing cycle-evaluation origin reaches a finite KCut; with $K\epsilon$, Lemma Core.KCutObserve prevents a persistent relevant blocker from being hidden solely in pre-normalization RTL microstate.

(WFG-inert ϵ premise.) Appendix K uses ϵ -steps only after the WFG-relevant boundary-request / attribution cutpoint has been fixed. More precisely, consider an ϵ -path $\sigma \xrightarrow{\epsilon^*} \sigma'$ that contains no Σ -visible committed Push_c/Pop_c event, no synchronization event, no signal Read/Write event, and no source-level issue/attribution transition that creates, removes, or reassigns a pending blocking boundary request. Such ϵ -steps are observationally silent with respect to the WFG abstraction: they do not change (i) the potentially-blocking frontier $\text{Front_P}(\sigma)$ for any process P (i.e., which guarded blocking Push/Pop frontier items are minimal and pending), nor (ii) the truth or attribution of any boundary requests relevant to those frontier items, nor (iii) the channel-side enabling status (as defined in K.1) of any $x \in \text{Front_P}(\sigma)$, read through $q = \text{MsgOf_P}(x)$. Equivalently, these WFG-inert ϵ -steps do not change the channel state relevant to ChannelEnabled/ChannelDisabled (e.g., $\text{occ_c}(t)$ for $B(c)>0$ buffered channels), and they do not create/remove rendezvous enablement for a frontier transfer except via Σ -visible committed Push_c/Pop_c events.

This premise does not say that every non- Σ microstep is WFG-inert. In particular, an ϵ -normalization segment may include source-level issue/attribution steps that establish a persistent blocking request, such as moving from a cutpoint before a Pop has issued to a cutpoint where the Pop request is pending and BoundaryReq is true. Those steps may change Front_P , BoundaryReq, or request attribution, and are therefore not the ϵ -steps quantified over by the WFG-inert premise. Appendix K’s WFG construction is applied only at the resulting Core.KCut cutpoints, after such request/attribution state has been fixed and the boundary network has settled.

In particular (pipelined loops / overlapped iterations): when HLS introduces overlap that may initiate later-iteration message reads “early,” we assume the design obeys the Automatic flush (drain-only blocking discipline) defined in Appendix B. Under that discipline, once the process has reached a stable blocked frontier, overlapped execution does not introduce any later-iteration blocking Push/Pop frontier item into $\text{Front_P}(\sigma)$, so transient internal pipeline activity that merely moves already-accepted work inside the concrete realization remains WFG-inert ϵ and does not contribute wait-for edges. Overlapped execution in pipelined loops is modeled only as a potential retiming of ordinary Σ -visible boundary commits (Push_c/Pop_c) relative to the unpipelined source, and is accounted for by the back-annotated semantics at the chosen abstract boundary of Sys_ref (E4). In particular, the channel facts relevant to ChannelEnabled/ChannelDisabled and WFG edges—e.g., $\text{occ_c}(t)$ for $B(c)>0$, and rendezvous enablement for $B(c)=0$ —change only on Σ -visible committed Push_c/Pop_c events at that boundary, or on explicit source-level issue/attribution steps before the WFG cutpoint is formed;

internal movement within the concrete realization of a single abstract channel after that cutpoint is WFG-inert ε and does not affect ChannelEnabled/ChannelDisabled or introduce/remove WFG edges.

Precondition ($K\varepsilon$: WFG-inert ε / no hidden micro-timing control decisions). All results in Appendix K are stated under the precondition $K\varepsilon$ that the narrowed WFG-inert ε premise above holds for the design on the ε -normalization segments leading to and between the Core.KCut points used for WFG reasoning. If an implementation intentionally uses internal micro-timing on such a segment to change Front_P membership, boundary-request attribution, or ChannelEnabled/ChannelDisabled facts without a corresponding Σ -visible event or explicit source-level issue/attribution transition, then $K\varepsilon$ is violated unless that behavior is made Σ -observable (e.g., via explicit modeling/snooping) or is witness-selected under the Appendix I.2 framework.

A common instance is branching on the success/failure of a non-blocking poll in a way that changes the next Σ -visible candidate frontier **NextFront_P(σ)** (Definition Core.6; see Lemma J.KObs-Proc and Corollary J.1); in that case, either NB-CF must hold (Appendix I.2), or the poll outcomes must be witness-selected as stated there.

Non-blocking polling attempts and other purely internal actions do not contribute edges to the WFG; they are modeled as internal ε -steps. When a non-blocking call succeeds, that success is already represented at the Σ level as the corresponding committed Push_c/Pop_c event, but it does not itself add a wait-for edge because it is not a blocking operation.

SyncChannels and barrier well-formedness.

SyncChannel (barrier) operations are treated as pure synchronization events and are omitted from the Wait-For Graph, which tracks only blocking Push/Pop dependencies. We therefore interpret “deadlock” in this appendix as **internal message-passing deadlock** (i.e., a deadlocked set in which all processes are blocked on some internal Push/Pop dependency captured by the WFG). Any global stall at a barrier caused by mismatched or conditional participation (e.g., a process can permanently bypass the barrier or reach it a different number of times) is a specification-level error already present in the pre-HLS model; such stalls are not created by the HLS transformation and are excluded by an explicit barrier well-formedness assumption (A8 below), not by the internal message-passing deadlock premise. Under this assumption, omitting SyncChannels from the WFG is sound: a post-HLS deadlock implies the existence of a Push/Pop wait-cycle as characterized below.

Definition (Blocked on message passing). Using Front_P(σ) as defined above, we say that P is blocked on message passing in state σ iff Front_P(σ) is non-empty and, for every frontier item $x \in \text{Front_P}(\sigma)$, BoundaryReq(x, σ) holds and ChannelDisabled(x, σ) holds (equivalently, BoundaryReq(x, σ) $\wedge$ $\neg$ ChannelEnabled(x, σ)), read operation-attributively through the owning message instance $q = \text{MsgOf_P}(x)$. Note (ALL disabled $\Rightarrow$ stall). If any frontier item $x \in \text{Front_P}(\sigma)$ is channel-enabled at the chosen Σ -visible boundary, then P is not “blocked on message passing” for WFG purposes. Consequently, an implementation may not be physically stalled at the chosen Σ boundary by a cross-channel “join” predicate while some frontier item $x \in \text{Front_P}(\sigma)$ at that boundary remains channel-enabled; any such conjunctive/join dependency must be represented as an explicit staged/bundle/join boundary channel so that, when the process is physically stalled, the relevant frontier items on that explicit boundary are all channel-disabled.

Definition (Drain-closed cutpoint for deadlock extraction). Let D be a non-empty set of processes. A K-facing state $\sigma \in \text{KCUTReach_M}(I)$ is drain-closed for D iff no already-issued non-frontier blocking Push/Pop request of any process $P \in D$ is channel-enabled at σ . Equivalently, after the frontier

members $\text{Front_P}(\sigma)$ are all disabled, there is no remaining already-issued downstream transfer of any process in D that can still commit as drain-only progress.

Here “already-issued downstream transfer” means an already-issued non-frontier blocking message-operation frontier item $x \in \text{Pending_P}(\sigma) \setminus \text{Front_P}(\sigma)$, with owning message instance $q = \text{MsgOf_P}(x)$, interpreted at the chosen abstract Sys_ref / RTL_B boundary and within the current source interval, i.e., the interval between the adjacent synchronization calls that bracket σ in P ’s current execution; Core.SyncFence makes this restriction lossless for blocking requests: no issued-but-uncommitted blocking item from an earlier interval can survive the synchronization commit that established the current interval; hidden internal pipeline staging or later-iteration prefetch absorbed into $B(c)$ is not a separate source-level operation for this definition unless it has been made an explicit abstract channel in $\text{Sys_B}^{\text{elab}}$.

If such a non-frontier drain transfer is enabled, σ is not yet a stable deadlock cutpoint for D ; either that drain transfer must be allowed to commit finitely often until a drain-closed cutpoint is reached, or the resulting commit changes the WFG-relevant channel facts so that some frontier operation becomes enabled and the candidate deadlock disappears.

Finite drain note. Finiteness here is not derived from finite channel capacity alone. Definition Core.16(6) , as discharged by $K.\text{Drain}$, explicitly requires the downstream drain set to be finite and non-replenishing. Finite B -faithful storage/credit on every selected explicit boundary, finite ε -depth D_P for internally advancing work, and Core.16(2) ’s no-new-later-work rule are the standard evidence used to discharge that obligation. Once $\text{Core.16(6)}/K.\text{Drain}$ is established, only work already accepted before the blocked condition can drain, and the number of D -owned downstream drain commits is finite. $\text{Core.SettleKBridge}$ requires those explicit drain commits and intervening WFG-inert ε -normalizations to be accounted for before a persisting blocked frontier is evaluated at a later drain-closed K -facing cutpoint. A certificate may use a coarse bound derived from the relevant in-flight occupancies, explicit finite credits, finite elaboration chains, and finite per-process ε -depths; Theorem K.1 consumes only the proved finiteness/non-replenishment fact, not a particular numeric bound.

Definition K.Dead (ordinary internal message-passing deadlock predicate). For a K -facing cutpoint $\sigma \in \text{KCutReach_M}(I)$ of a source-side or lowered proof model M used by Appendix K, $\text{Dead_K}(\sigma)$ holds iff there exists a non-empty set of processes D such that σ is drain-closed for D and:

- Every process P in D is blocked on message passing, and in particular there exists at least one frontier item $x \in \text{Front_P}(\sigma)$ on an internal message-passing channel c (i.e., a channel included in the WFG), with owning message instance $q = \text{MsgOf_P}(x)$, such that $\text{BoundaryReq}(x, \sigma)$ holds and $\text{ChannelDisabled}(x, \sigma)$ holds.
- The vertices in D , together with the internal channels they access, form a strongly connected component in the WFG that has no outgoing edges (a closed cycle): processes in D can only wait on each other through internal WFG channels. (Members of D may additionally hold environment-facing frontier items, which are excluded from the WFG; see the Terminology note below.)

Intuitively, D is a set of processes that are waiting only on each other (via internal message-passing channels tracked by the WFG), with no remaining enabled drain-only transfer of any process in D that could change the WFG-relevant channel facts. Thus D cannot be unblocked by internal modeled message-passing progress represented in the closed WFG component, nor by already-issued downstream drain from within D . This is a reachable K -facing-state predicate, not a permanence claim: a member of D may also hold environment-facing frontier items (excluded from the WFG), and later environment/testbench activity under the fixed stimulus may dissolve such a state; the Appendix K results and the A5 premise quantify over reachability of these states, not over their permanence. On the RTL side Appendix K evaluates the corresponding reconstructed predicate $\text{DeadKRec}/\text{DeadKRec}^W$ on

the certified KObs view; it does not apply this operational source definition to an arbitrary RTL microstate.

- Every process P in D is blocked on message passing, and in particular there exists at least one frontier item $x \in \text{Front}_P(\sigma)$ on an internal message-passing channel c (i.e., a channel included in the WFG), with owning message instance $q = \text{MsgOf}_P(x)$, such that $\text{BoundaryReq}(x, \sigma)$ holds and $\text{ChannelDisabled}(x, \sigma)$ holds.
- The vertices in D , together with the internal channels they access, form a strongly connected component in the WFG that has no outgoing edges (a closed cycle): processes in D can only wait on each other through internal WFG channels. (Members of D may additionally hold environment-facing frontier items, which are excluded from the WFG; see the Terminology note below.)

Intuitively, D is a set of processes that are waiting only on each other (via internal message-passing channels tracked by the WFG), with no remaining enabled drain-only transfer of any process in D that could change the WFG-relevant channel facts. Thus D cannot be unblocked by internal modeled message-passing progress represented in the closed WFG component, nor by already-issued downstream drain from within D . This is a reachable-state predicate, not a permanence claim: a member of D may also hold environment-facing frontier items (excluded from the WFG), and later environment/testbench activity under the fixed stimulus may dissolve such a state; the Appendix K results and the A5 premise quantify over reachability of these states, not over their permanence. A process that is blocked solely on channels whose complementary endpoint is the external environment/testbench (excluded from the WFG) does not witness an internal message-passing deadlock in this appendix.

Terminated-process convention. For purposes of Appendix K, termination is interpreted per reset epoch. After a process P emits FINISH_P for a given reset epoch, the infinite execution is padded only by process-local idle stutter for P for the remainder of that epoch. This padding creates no new P -owned Σ -events, no new BoundaryReq , no new Front_P or NextFront_P item, and no new outgoing WFG edge from P . This epoch-local padding does not preclude a later reset boundary affecting P . If such a reset occurs and P later exits reset, the subsequent START_P is a new dynamic occurrence that begins a fresh reset epoch, and the earlier FINISH_P has no ordering, anchoring, or WFG force across that reset boundary. Any later termination of P is represented by a distinct dynamic FINISH_P occurrence. Thus a process blocked waiting for a complementary Push/Pop from a peer that has terminated in the current reset epoch is not, by itself, an internal message-passing deadlock under the Dead_K predicate. It is instead outside the Dead_K predicate unless the design includes an explicit modeled termination protocol that makes such waiting part of the internal channel protocol being analyzed.

Terminology (adopted reading: reachable closed internal wait-cycle cutpoint). The predicate just defined is, precisely, a reachable, drain-closed, closed internal wait-cycle cutpoint. It is deliberately weaker than a conventional (permanent) deadlock: a member of D may hold environment-facing frontier items excluded from the WFG, and — in a declared timed or reactive environment outside the B1-avail convention — a fair continuation in which the environment later enables such an item can leave the state. (Under the default B1-avail pull-based stimulus convention, an environment-facing input item that is disabled at a cutpoint is stimulus-exhausted and remains disabled forever, per Lemma K.EnvX-Auto in K.4b; the later-enablement possibility, and with it the input-side $W.\text{Env}$ rescue class, arises only for declared timed/reactive environments.) Three consequences are adopted normatively throughout this appendix. (1) The property preserved by Theorem K.1 — absence of reachable states satisfying this predicate — is an invariance (safety-style) reachability property over admissible executions, not a temporal liveness property in the eventually/always sense; the historical word “liveness” is retained in titles for continuity but shall be read with this meaning. (2) The phrase “processes in D can only wait on

each other” means, wherever it appears, “only wait on each other through internal WFG channels”; it does not exclude coexisting environment-facing waits. (3) The A5 premise is correspondingly conservative: it can be violated by a transient internal wait-cycle cutpoint even in a design whose fixed environment is guaranteed to release an environment-facing frontier item so that the system always progresses; such a report is a true positive for the defined predicate and a deliberate conservatism with respect to permanent deadlock. Designs for which this conservatism is unacceptable may instead be assessed against the stronger persistent-deadlock predicate — every blocking frontier item of every member of D internal, with its complementary owner in D , and the state a trap under all admissible continuations — but the theorems of this appendix neither require nor establish results for that stronger predicate; adopting it would require re-proving Lemmas K.1a–K.2b and Theorem K.1 with an added permanence argument.

Scope warning. This exclusion is only a scope convention for Theorem K.1. It does not imply that waiting forever for a terminated peer is benign or acceptable design behavior. If termination, EOF, done, cancellation, or end-of-stream behavior is part of the intended protocol, that behavior must be modeled explicitly in `Sys_ref` and `RTL_B`, or verified separately outside the Appendix K internal message-passing-deadlock theorem.

Our notion of the preserved property (historically labeled “liveness”) in this appendix is: no reachable internal message-passing-deadlock state (reachable closed internal wait-cycle cutpoint) as defined above, under the fixed stimulus and fair scheduling. Formally this is an invariance (safety-style) reachability property over admissible executions — see the Terminology note above — and it does not assert that a reached deadlock state is permanent when environment-facing frontier items are present. (Starvation of a single process in an otherwise live system is ruled out separately by weak fairness.)

Implication note (relation to persistent deadlock). Within the closed internal wait-cycle class captured by `Dead_K`, the reachable-cutpoint property is conservative with respect to permanence: it excludes even a transient reachable `Dead_K` cutpoint, and therefore also excludes a persistent internal hang whose blocked condition contains such a drain-closed closed WFG component. This implication is intentionally limited to the `Dead_K` structural class. The theorem does not establish the broader operational guarantee that every form of permanent internal-communication non-progress is impossible. In particular, starvation or perpetual under-supply by a complementary owner that remains live on unrelated work can leave downstream operations permanently unanswered without producing a closed `Dead_K` component; Example K.CE-5/H2 is the canonical case. Waiting for a terminated peer and the other stalls excluded by the scope warnings above likewise require explicit protocol modeling or a separate argument. The stronger reachability form is retained as the normative `Dead_K` predicate because, within its stated class, it is safety-shaped—an invariance over reachable states—which makes it checkable by inductive invariants and preservable through finite-prefix correspondence (Definition J.Reach, Corollary J.1). A theorem covering permanent non-progress outside that closed-wait-cycle class would instead require additional continuation/fairness and permanence reasoning and is not claimed here. Designs with intentional environment-rescued `Dead_K` cutpoints are handled by the Waiver `W.Env` mechanism (K.2).

K.2 Assumptions and Imported Results

K.2.1 Standing Assumptions We assume throughout Appendix K:

A1. R-rules and B-rules. The prescribed source reference model `Sys_ref` and the post-HLS implementation `RTL_B` satisfy the process- and channel-level semantic rules defined earlier (R-rules and B-rules).

A2. Finite Pipeline Depth (FPD) / B4 finite ϵ -stutter.

For each process P there is a finite bound D_P such that, along any consecutive epsilon-only evolution in which P is internally advancing toward its next observable Σ -action or its next Σ -action is locally enabled, and P is not externally stalled by a peer/environment/back-pressure condition, P executes at most D_P consecutive epsilon-steps before either (i) executing a Σ -action, or (ii) reaching a stable waiting state in which no further epsilon-step is available until some external/peer/environment condition changes the relevant enabling predicates. This is not a bound on clock cycles spent waiting for external stimulus, peer action, or back-pressure relief; such waiting is governed by WF/B2 and B3. The system-level all-disabled quiescence bound is the separate B5 cycle-valued bound C_{sys} . A theorem instance may discharge B5 by supplying C_{sys} directly or by proving per-process cycle bounds C_P from D_P through the bucket-semantic charging lemma and taking $C_{sys} = \max_P C_P$. The identification $C_{sys} = \max_P D_P$ is permitted only when that bridge is proved.

A3. Weak Fairness (WF) (B2). If an action remains continuously enabled (its guard remains true), the scheduler eventually selects it.

A4. Channel Progress (B3 (Additional Semantic Assumptions (B-rules)); Appendix N). For each channel c with capacity $B(c)$, assume the progress invariants of Appendix N hold. This is an obligation on the scheduler/environment (e.g., fairness of arbiters/back-pressure) and is not implied by the local R-rules alone.

A5. Primary selected-instance Policy-S bounded-response premise (A5-S). For the selected Sys_K instance, let p_S^{det} be the deterministic Policy-S execution selected by $SDet$ after fixing capacities, elaboration, initial/reset state, stimulus, witness choices, environment behavior, arbitration, concrete pre-HLS scheduling semantics, Policy-S issue semantics, ϵ -normalization, and $K.Low$ lowering. A5-S requires: (i) $K.CompleteS$ evidence that p_S^{det} is maximal, fair, and complete; (ii) $K.RespCov$ evidence defining a checkable proof disposition and finite bound for every static internal process-facing blocking site and relevant dynamic class and proving that every dynamically reachable instance belongs to a monitored or statically bounded class, with coverage-preserving waivers and the separate environment-site rule; and (iii) $K.RespClean$ evidence that no finite prefix of the complete run witnesses a non-waived $K.RespFail$. $K.RespFail$ is a finite-prefix predicate covering certified-bound expiration without process-facing commit/return under the deadline convention of Definition $K.RespFail$, a declared maximal finite terminal run with the monitored frontier still pending, or an affecting RW2 RESET that clears the monitored dynamic instance before response without an explicit waiver. A5-S is used by Corollary $K.2c$ to derive $A5L_Sys_K$ and then, through $K.Low.A5$, A5-L only when $K.EnvMF$ and the shared $K.InterruptionGate$ applicability hold for the candidate components, with $K.ResetEpoch$ evaluated first and $K.TerminalDisposition$ required only on $K.ResetEpoch.Continue$.

For A5, Policy S is the conservative serial-blocking interpretation of Appendix G. Within each source interval, a process does not issue a source-later blocking operation until every source-earlier blocking operation required by the serial discipline has committed in a strictly earlier cycle. The condition is process-facing: proof-internal staged/bundle/join/epoch-gate transfers do not satisfy it unless the elaboration identifies them as the commit that releases the source-level call. Channel-side E4 movement, FIFO drainage, independent-process execution, and explicit elaborated transfers remain concurrent.

Maximality, completeness, and fairness of p_S^{det} are part of A5-S/ $K.CompleteS$, not $SDet$. $K.RespCov$ and $K.RespClean$ are additional obligations checked over that complete artifact, not definitions of completeness itself. A finite simulation discharges A5-S only when it reaches a declared maximal terminal state and has no pending covered frontier, or when a complete proof establishes the required monitor result. A finite prefix of a still-running system is insufficient. For a nonterminating design, the selected run requires an invariant, sound model checking, recurrence/lasso evidence, or another

complete argument that every covered response occurs within its finite bound and no reset-pending failure occurs.

Instance dependence. A5-S and SDet are not properties of the raw SystemC text and are not discharged by an arbitrary simulation. They are re-established whenever capacities, boundaries, elaboration, lowering, stimulus, witness assignment, environment/reset contract, arbitration rule, concrete simulator scheduling semantics, Policy-S implementation, or normalization changes. Such a change defines a different selected theorem instance.

Practical discharge route (Corollary K.2c). Under SDet there is one selected Policy-S execution p_S^{det} for the selected instance, with a committed history unique modulo finite WFG-inert ε -stutter. $K.EnvMF$ restricts the supported environment/frontier cases. $K.ResetEpoch$ and $K.TerminalDisposition$ instantiate the shared $K.InterruptionGate$ schema in the normative nested order: reset first, then terminal only on $K.ResetEpoch.Continue$. A $DirectFail$ branch yields the corresponding finite $K.RespFail$ without invoking dominance; the two nested $Continue$ branches select $K.2b.E/R/P/Q$. A5-S/CF-S separately supply $K.CompleteS$, $K.RespCov$, and $K.RespClean$ evidence. Lemma K.2b shows that any internal deadlock reached by a finite Policy-L prefix admissible under Definition Core.LAdm forces that same selected execution to produce a finite non-waived $K.RespFail$ witness. Therefore a complete clean bounded-response certificate for p_S^{det} is sufficient on that supported fragment. No alternate Policy-S schedulers or interleavings exist within the selected instance, and no confluence theorem is relevant. A source region whose liveness depends on favorable same-cycle or overlapped issue of multiple blocking operations is outside the scheduler-robust A5-L framework whenever an admissible serial-shaped Policy-L prefix reaches $Dead_K^W$. In that case A5-L is false, not merely difficult to discharge: direct Policy-L verification, structural conditions, invariants, exploration, and tool certificates cannot prove it for the unchanged instance. The design must be rewritten or buffered, placed under explicit protocol/synchronization structure that removes the serial deadlock prefix, or verified under a different scheduler-specific theorem not supplied by this document.

A5 and Appendix K do not assert freedom from specification-level barrier errors, reset stalls, or external-environment stalls that do not contain the internal wait structure represented by the operative predicate.

A6. Point-to-point channels (single-producer/single-consumer). For each message-passing channel c (including buffered channels with $B(c)>0$ and rendezvous channels with $B(c)=0$), exactly one process performs all $Push_c$ operations on c (the producer) and exactly one process performs all Pop_c operations on c (the consumer). Hence the complementary endpoint for any blocked $Push_c/Pop_c$ is unique, and WFG edges are well-defined even for mixed WFGs that include both buffered and rendezvous channels.

If a design has a shared resource that would violate this uniqueness, it must be modeled via an explicit arbiter process plus point-to-point channels (as noted in “Normative Formal Core” $B(c)$).

A7 (RTL determinism / witness selection / equal-KObs correspondence). Assume B1 for RTL_B . J.GWR-Comp/Theorem J.1 and Corollary J.1 supply one selected source witness and equal KObs. For Appendix K, $J.RegPhaseReach$ is derived from the revised registered, deferred-hole, phase-normal construction (or directly supplied), while $K.DrainReach$ validates nonwithdrawal, $K\varepsilon$, and every activated Core.Drain/ $K.Drain$ obligation for that exact prefix; together they are $K.RLAdmReach$. $J.OfferReach$, $J.OfferConf$, $J.KObsConf$, $RegPhaseReach$, $DrainReach$, and $K.RLAdmReach$ must bind the same RTL/source cutpoints, H,O,E,w , instance, and stimulus. Core.KCutClosure_{post} and $K.CertCov$ are additionally required only for the universal reachable-cutpoint conclusion. In the standard scope Core.QSync-Post derives the closure premise from $QualifiedSynchronousModel_RTL_B(I)$; closure then establishes a finite $KCut$ observation for each K-facing RTL cycle origin, and certificate coverage supplies the correspondence package at every such cutpoint.

When a deterministic, reproducible Sys_ref witness is desired for simulation/debug, apply the Appendix L snooping wrapper to select a particular witness aligned with the latency-sensitive ϵ -choices. B4 and B5 justify the selected ϵ -normalized endpoints required by Corollary J.1. Every Appendix K transfer of a reconstructed frontier, request, channel, enablement, WFG, drain, or deadlock predicate uses the same equal-KObs package. A discharge for an unrelated source cutpoint, different witness, different O, or noncanonical unmatched history is insufficient.

Premise SDet (deterministic selected Policy-S simulator). Fix the selected Sys_K, initial/reset state, capacities/elaboration, stimulus and reactive environment, witness-selected ϵ outcomes, deterministic arbitration, concrete SystemC/pre-HLS scheduler, Policy-S issue rule, and ϵ -normalization. These define a functional Policy-S next-state semantics and select one run artifact p_S^{det} . Every finite or infinite artifact generated from the same initial state by those exact semantics has a committed history prefix-comparable with that of p_S^{det} modulo finite WFG-inert ϵ -stutter; hence no two incompatible committed histories arise from the selected instance. WC and I.DR/I.DR' determine replay state/actions relative to the selected history; SDet excludes multiplicity caused by an unfixed scheduler, arbiter, environment, witness, issue, or normalization choice. SDet does not by itself assert maximality, fairness, completeness, response coverage, or response cleanliness, and a shorter finite artifact may be a proper prefix of the selected history. K.CompleteS and A5-S/CF-S separately establish maximality/fairness/completeness and K.RespCov/K.RespClean for p_S^{det} . Changing any listed choice defines a different selected instance.

A8. Barrier well-formedness (SyncChannels). For each SyncChannel instance, the set of designated participant processes reaches the barrier the same number of times under the fixed stimulus/initial state; equivalently, no participant can permanently bypass a barrier call while another participant is waiting at that barrier. (Barrier stalls are therefore outside the behaviors considered in Appendix K's internal message-passing deadlock analysis.)

A9. Single-transfer-per-cycle endpoint constraint (scalar channel interface). For every message-passing channel c, in any clock cycle t, at most one Push_c commit and at most one Pop_c commit may occur on c. Multi-lane/burst resources must be modeled explicitly.

A10. Reset-empty FIFO assumption. After each reset boundary, every buffered channel has logical occupancy 0 at the chosen Σ boundary.

A11. WFG-inert ϵ (K ϵ). The design satisfies K ϵ .

No further numbered A-assumption is introduced beyond A1–A11; SDet is a separately named selected-simulator premise. A5-S is part of the selected-instance liveness obligation, not a conclusion derived from the local R-rules, WC alone, or the no-reverse rule.

Drain-only blocked-frontier discharge for Appendix K (K.Drain). K.Drain is the design/application evidence obligation that the selected theorem instance satisfies Definition Core.Drain. Its continuation content is the complete seven-clause discipline of Definition Core.16. The K.2b dominance/extraction bridge and Appendix K drain-normalization arguments require that discipline for every interval in which Policy L can leave source-later same-interval work outstanding behind a disabled earlier frontier. Pipelined or otherwise overlapped control discharges this through Appendix B automatic flush. Ordinary non-overlapped control may discharge it through a structural proof of the same discipline. In every case, the discharge must establish the blocked point, no new source-later dynamic-operation launch, request non-withdrawal, exclusion of non-frontier requests as WFG sources, drain-only downstream progress, finite non-replenishing B-faithful boundary storage or credit, and explicit sync-boundary / epoch-gate treatment for any withheld producer-facing capacity. This is the K.Drain application-specific discharge of the existing scheduling and channel assumptions, not an additional numbered A-assumption.

Definition K.A5L (model-indexed scheduler-robust bad-cutpoint exclusion). For a source proof model M used by Appendix K, write $A5L_M(I, W)$ for the A5-L formula interpreted in M : every finite Policy-L prefix admissible under M 's $Core.LAdm/Pref_L^{adm}$ domain, with the fixed theorem-instance stimulus, capacities, elaboration/lowering, environment, arbitration, reset interpretation, and witness-selected outcomes, excludes every reached $\sigma \in KCutReach_M(I)$ satisfying the corresponding $Dead_K^W$ predicate. The model index fixes the operation/frontier universe and explicit boundaries on which the predicate is evaluated; it is not an additional execution choice.

Premise A5-L (scheduler-robust liberal source liveness). A5-L is the abbreviation $A5L_Sys_ref(I, W)$. Thus, for every $p_L \in Pref_L^{adm}(I)$ of the selected Sys_ref instance, under the fixed stimulus, capacities $B(c)$, elaboration, and fixed witness-selected outcomes, no source K -facing cutpoint $\sigma \in KCutReach_ref(I)$ reached by p_L satisfies $Dead_K^W$. Theorem K.1A consumes this Sys_ref form for one source prefix whose $Pref_L^{adm}(I)$ membership is supplied by $K.RLAdmReach$. Where synchronization waits or witness-selected non-NB-CF sites can participate in a potential wait cycle, the serial route first proves the corresponding $A5L_Sys_K(I, W_K)$ result on the normalized blocking instance Sys_K and then transports it to A5-L by Lemma K.Low.A5.

Policy-S-primary architecture. $SDet$ fixes one concrete deterministic Policy-S execution p_S^{det} . $K.EnvMF$ limits the serial reduction to $StandardEnv/B1$ -avail when relevant $Dead_K$ components are multi-frontier, and to certified single-internal-frontier components for timed/reactive or otherwise nondefault environments. $K.InterruptionGate$ supplies the common applicability branch discipline; $K.ResetEpoch$ is evaluated first and $K.TerminalDisposition$ only on $K.ResetEpoch.Continue$. Lemma K.2b compares a liberal counterexample with the selected execution and proves that a Policy-L internal deadlock forces a finite $K.RespFail$ witness on p_S^{det} . Corollary K.2c therefore reduces A5-L to the single complete A5-S check consisting of $K.CompleteS$, total $K.RespCov$, and $K.RespClean$. There is no family of Policy-S scheduler interleavings inside one selected instance and no confluence question. $Dead_KS$, $EnvStop$, $DoneStop$, $K.EnvOrd$, and $K.Done$ are optional diagnostic classifications only. Normative organization of Appendix K. The spine is: the $Dead_K$ predicate; K.0 frontier reconstruction; K.2 WFG extensionality; K.2a $DrainClosedNow$ extensionality; the $K.RLAdmReach$ qualification and $K.CertifiedCutpoint$ interface; Theorem K.1A from A5-L for one certified cutpoint; $Core.KCutClosure_post$, $K.CertCov$, and Corollary K.1A-Total for the universal reachable-cutpoint conclusion; Sys_K and the $SDet$ -selected p_S^{det} ; $K.EnvMF$; the shared $K.InterruptionGate$ schema with reset-first $K.ResetEpoch$ and nested $K.TerminalDisposition$ instantiations; deterministic serial-dominance K.2b; Corollary K.2c; A5-S on p_S^{det} ; and Theorem K.1. Appendix K-P contains the operational companion proof. Structural and direct certificates remain alternate A5-L routes only when A5-L is true for the selected instance.

Waiver $W.Env$ (declared environment-rescued cutpoints). The Appendix M.7a side-condition record may exclude a precisely identified cutpoint class when a declared environment-facing co-frontier is intended and proved to rescue it. The record shall identify the component class, rescuing item, environment assumption, and design justification. A waiver does not alter the default predicates and does not excuse an Example K.CE-style bypass without an accompanying no-bypass or equivalent argument.

Definition K.DeadW and response waivers. Let W be the union of declared $KObs$ -facing waived cutpoint classes. $Dead_K^W(\sigma)$ means $Dead_K(\sigma)$ and $\sigma \notin W$; optional diagnostic $Dead_KS^W$ is defined analogously. $KObs$ -facing waiver closure requires $KObsClosed_I, E, w(W)$ whenever W is non-empty. Separately, let W_resp be the declared dynamic $K.Resp$ monitor-waiver classes. $RespFail^W$ means a $K.RespFail$ whose monitor instance is not in W_resp . $K.RespCov$ requires coverage-preserving waiver closure: for every non-waived liberal $Dead_K$ component admitted to the K.2b premise, at least one permanently pending serial monitor extracted by K.2b.Q is outside W_resp . A response waiver depending on hidden issue timing, reset disposition, or future environment behavior must carry the

corresponding operational proof. Neither W nor W_resp may be used as a broad label to hide an unclassified deadlock or end-of-test hang.

Scope of the preservation theorem. Theorem K.1A transfers a deadlock at one $K.CertifiedCutpoint$ RTL cutpoint to a $Core.LAdm$ -admissible $Policy-L$ Sys_ref deadlock through equal $KObs$ and contradicts $A5-L$; it consumes neither $SDet$ nor the serial-route premises and makes no package-totality claim. Corollary K.1A-Total adds $Core.KCutClosure_post$ and $K.CertCov$ and obtains the universal reachable-cutpoint conclusion. Theorem K.1 adds $SDet$, $K.CV$, $NB-Align/K.Low$ where applicable, $K.Resp$, and the remaining serial conditions, obtains $A5-L$ from $A5-S$ for the $SDet$ -selected execution p_S^{det} via Corollary K.2c, and uses $Core.KCutClosure_post$ together with $K.CertCov$ for the user-facing universal RTL conclusion. Within Lemma K.2b, $K.InterruptionGate$ is instantiated first by $K.ResetEpoch$ and, only on $K.ResetEpoch.Continue$, by $K.TerminalDisposition$. Either $DirectFail$ branch bypasses the dominance construction; the two nested $Continue$ branches select $K.2b.E/R/P/Q$. $K.CompleteS$ supplies maximality/fairness/completeness, $K.RespCov$ supplies total finite-bound coverage, and $K.RespClean$ is the contradiction used by K.2c.

The no-reverse rule is not by itself a liveness theorem. Its practical force is captured by the deterministic $Policy-S$ reduction: the selected simulator withholds source-later requests and serves as the conservative deadlock-stress execution. K.2b does not claim that p_S^{det} reproduces the $Policy-L$ trace or the same component; it states that any $Policy-L$ internal deadlock forces a permanent covered process-facing serial frontier and therefore a finite $K.RespFail$ witness.

Example: blocking rendezvous operations whose liveness depends on same-cycle/eager issue. Consider two rendezvous channels a and b with $B(a)=B(b)=0$ and two processes; assume v and w are independent of any data/status returned by the preceding Pops:

```
P1: b.Pop(); a.Push(v)
P2: a.Pop(); b.Push(w)
```

Under $Policy-S$'s conservative serial-blocking issue discipline, each process issues only its first source-order blocking operation: $P1$ waits on Pop_b and $P2$ waits on Pop_a ; neither process reaches the complementary $Push$. That two-Pop execution is not merely a $Policy-S$ artifact—it is itself a $Core.LAdm$ -admissible $Policy-L$ prefix. After the L2 settled snapshot exposes both Pops disabled, $Core.Drain/K.Drain$ forbids newly issuing the source-later Pushes, and the resulting drain-closed source K -facing cutpoint contains the closed internal rendezvous deadlock. Thus $A5-L$ is false for the unchanged design, even though, when the selected instance admits the straight-line $Core.ReachPrep$ case, a different eager $Policy-L$ schedule may assert both distinct-interface requests and complete both rendezvous transfers in the same cycle; the RTL may likewise exhibit that behavior when its validated scheduling permits it. No direct $Policy-L$ proof, structural proof, invariant, exploration, or alternate $A5-L$ certificate can establish a false premise. The Appendix K scheduler-robust framework therefore cannot certify a design whose deadlock freedom depends on this favorable eager/overlapped issue; the design must be rewritten, buffered, explicitly synchronized, or verified under a different scheduler-specific theorem. Completeness boundary (normative): if some $p \in Pref_L^{adm}(I)$ reaches $Dead_K^W$, then $\neg A5-L(I)$, regardless of whether another admissible schedule is live.

The $Front/NextFront$ bridge used later in Appendix K is the source-local reconstruction result of Lemma K.0. Cross-model frontier, raw rendezvous-request, enablement, and WFG agreement are obtained by applying the same reconstruction functions to the equal $KObs$ record of Corollary J.1. No asserted rendezvous request is identified with frontier membership merely because it is present in O .

Remark K.0-pre (Derived bridge, not an extra premise).

The asserted blocking-message slice of NextFront is not an additional theorem premise. Core.7a defines the candidate blocking slice, Core.10 and Axiom Core.IC supply operation-attributive request persistence, and Lemma K.0 derives that the asserted slice is exactly the pending blocking frontier. BoundaryReq remains undefined for non-message frontier items, and successful non-blocking frontier items do not create pending blocking WFG members.

Definition K.StimX. Under convention B1-avail, the next fixed input item is available as soon as earlier items on that environment channel have been consumed. An environment-facing Pop is stimulus-exhausted exactly when no item remains; an environment-facing Push is exhausted only under a declared permanent-stop contract, and never under the default always-ready output environment. Diagnostic Definition K.WFG+ and K.DeadKS. For debugging and failure triage only, extend the ordinary wait-for graph with two terminal-node classes. For each environment-facing boundary c whose next required interaction is permanently unavailable under Definition K.StimX, create EnvStop_ c and add $P \rightarrow$ EnvStop_ c when blocked P has the corresponding disabled frontier. For each internal point-to-point boundary c whose unique complementary owner has normally completed before the demanded ordinal, create DoneStop_ c and add $P \rightarrow$ DoneStop_ c . Ordinary internal dependencies remain process-to-process edges $P \rightarrow Q$. At a reachable drain-closed Policy-S K-facing cutpoint, Dead_KS holds when the extended graph contains a nonempty closed blocked component containing at least one process-to-process edge; Dead_K is the terminal-free special case. These predicates remain KObs-reconstructible under their stated environment and waiver premises, but they are not complete for live endpoint under-supply and are not used to prove Corollary K.2c. They may classify why a K.Resp monitor fired and support familiar diagnostic reports for Examples K.CE through K.CE-4.

K.2.1 KObs Deadlock Factoring and Extensionality

These results factor the Appendix K snapshot predicates through the common Core.18c KObs record. They transfer only cutpoint facts; Core.Drain/K.Drain, future issue legality, fairness, and bounded-response monitor obligations remain operational premises.

Definition K.EnvTerminalAdequate (environment-terminal reconstruction premise).

EnvTerminalAdequate(I) holds when the selected instance records either StandardEnv or an EnvReplayAdequate certificate. Under StandardEnv, Definition K.StimX and the default always-ready output convention determine permanent next-interaction availability from the committed consumption history; Lemma K.EnvX-Auto supplies the corresponding environment-co-frontier fact used by the serial route. Under EnvReplayAdequate, the certificate supplies a typed reconstructed environment state EnvStateRec_ I (H, w) and proves that the permanent-availability predicates used by K.WFG+ are determined by H , w , and that state. This is an extensionality premise for optional diagnostics, not a claim that every well-typed KObs record is reachable.

Definition K.KObsDerived (K-specific reconstruction functions).

For a Core.18c record $K = \langle E, w, H, O \rangle$, define DeadKRec(K, I) by evaluating Definition K.Dead on the Core-reconstructed FrontRec, WFGRec, process-completion, and DrainClosedNowRec facts. Define DeadKRec[^] $W(K, I, W)$ by applying Definition K.DeadW to DeadKRec and the KObs-facing waiver set W . When EnvTerminalAdequate(I) holds, define EnvStopRec, DoneStopRec, WFGPlusRec, DeadKSRec, and their KObs-closed waived variants by applying Diagnostic Definition K.WFG+ and K.DeadKS to the same Core-reconstructed process, frontier, request, channel, and committed-history facts together with the selected environment-reconstruction mode. These are Appendix K functions over a source-realizable

Core record. No $K.\text{RespFail}$ reconstruction is defined from K alone because monitor activation, age, bound class, and reset/terminal disposition state are operational data outside KObs .

Lemma K.KObs-Dead (DrainClosedNow and K-predicate factoring).

For every non-empty process set D , $\text{DrainClosedNowRec}(D, K)$ holds iff the operational cutpoint σ is drain-closed for D in the snapshot sense used by Appendix K. The unwaived ordinary predicate $\text{Dead}_K(\sigma)$ is therefore determined by K . If $\text{KObsClosed}_{I,E,w}(W)$ holds, $\text{Dead}_{K^W}(\sigma)$ is determined by $\text{DeadKRec}^W(K, I, W)$. If $\text{EnvTerminalAdequate}(I)$ holds, the optional diagnostic $\text{WFG}^+(\sigma)$, EnvStop , DoneStop , and unwaived $\text{Dead}_{KS}(\sigma)$ are determined by $\text{WFGPlusRec}(K, I)$ and $\text{DeadKSRec}(K, I)$; with KObsClosed , the diagnostic waived predicate is also determined. No $K.\text{RespClean}$ or $K.\text{RespFail}$ conclusion follows from KObs .

Proof. Lemmas J.KObs-Proc–J.KObs-WFG reconstruct process completion, consumed ordinals, frontier membership, endpoint requests, channel enablement, and ordinary WFG edges. The optional terminal reconstructions use the stated environment adequacy premises. KObsClosed supplies waiver extensionality for cutpoint predicates. The snapshot drain clause is exactly the absence of an enabled non-frontier residual offer owned by D . No continuation-level claim about Definition Core.16/DrainDiscipline or response-monitor age follows from this factoring result. $\square$

Corollary K.KObs-Ext (KObs extensionality).

Let $\sigma_1, \sigma_2 \in \text{KCutReach_ref}(I)$ be source K -facing cutpoints of the same fixed instance with $\text{KObs}_{E,w}(\sigma_1) = \text{KObs}_{E,w}(\sigma_2)$. Then their reconstructed process slices, explicit abstract channel states, NextFront and Front sets, operation-attributive BoundaryReq values, channel-enable predicates, ordinary WFGs, DrainClosedNow predicates, and unwaived Dead_K truth values are equal. If $\text{KObsClosed}_{I,E,w}(W)$ holds, their Dead_{K^W} truth values are equal. If $\text{EnvTerminalAdequate}(I)$ holds, their optional diagnostic $\text{EnvStop}/\text{DoneStop}$ classifications, WFGPlusRec graphs, and unwaived Dead_{KS} truth values are equal; with KObsClosed , their diagnostic Dead_{KS}^W values are equal. The conclusion is independent of legal historical Issue timestamps not retained in O , and it makes no claim about $K.\text{Resp}$ timer activation or age. Without waiver closure or environment-terminal adequacy, the extensionality conclusion is correspondingly limited. Proof. Functional extensionality of the named reconstruction definitions and Lemmas J.KObs-Proc–J.KObs-WFG and K.KObs-Dead. $\square$

Cross-model-use note. Corollary K.KObs-Ext is the source-side extensionality theorem for reachable records. Appendix J supplies J.OfferReach and J.OfferConf and proves through J.KObsConf that the selected post certificate and source cutpoint share one record. Lemma K.0, Corollary K.0a, Lemma K.2, Lemma K.2a, and Theorem K.1A state the required cross-model consequences directly from that equality.

K.2.2 Lemma K.0 — Frontier Reconstruction from KObs

Purpose. Appendix K forms WFG edges from the minimal pending blocking frontier. The KObs layer already separates the candidate source frontier reconstructed from H from the currently asserted attributed offers reconstructed from O . This lemma identifies the operational source frontier with the corresponding pure reconstruction and states the cross-model consequence by equal-KObs extensionality.

Statement. Fix I, E, w and $\sigma \in \text{KCutReach_ref}(I)$, and let $K = \text{KObs}_{E,w}(\sigma)$. For process P define $\text{BlockingMsgNextFrontRec}_P(K) := \{ x \in \text{NextFrontRec}_P(K) \mid x \in \Omega_{\text{msg},P} \text{ and } \text{MsgOf}_P(x) \text{ is a blocking Push or Pop instance} \}$.

Then

$\text{Front}_P(\sigma) = \text{FrontRec}_P(K) = \{ x \in \text{BlockingMsgNextFrontRec}_P(K) \mid \text{BoundaryReqRec}(x, K) \}$.

Here NextFrontRec is witness-relative over the fixed Ω_P and $\leq_P$ supplied by E ; it is a candidate frontier, while BoundaryReqRec selects the currently outstanding blocking offers.

Cross-model consequence. Let $K_{\text{post}} = \text{KObsPost}_J(\rho_{\text{post}}; E, w, H, O)$ and $K_{\text{ref}} = \text{KObs}_E, w(\sigma_{\text{ref}})$ be the exact records selected by Corollary J.1. If $K_{\text{post}} = K_{\text{ref}}$, then for every process P , $\text{FrontRec}_P(K_{\text{post}}) = \text{FrontRec}_P(K_{\text{ref}})$, and $\text{BoundaryReqRec}(x, K_{\text{post}}) = \text{BoundaryReqRec}(x, K_{\text{ref}})$ for every typed blocking message-operation frontier item x . Thus the reconstructed post frontier is exactly the operational source frontier at σ_{ref} . This consequence uses only the common record; it does not compare historical Issue times or raw RTL control state.

Mixed blocking/non-blocking clarification. Successful non-blocking transfers may contribute committed Push/Pop events to H , and failed polls may contribute WC-selected ε outcomes, but neither creates a pending blocking frontier member. A blocking endpoint may nevertheless wait on a complementary process whose source endpoint uses a non-blocking call; the non-blocking side can change occupancy or complete the complementary transfer, but does not itself become a WFG source unless it also owns a separate pending blocking frontier.

Proof. For a reachable source cutpoint, Lemma J.KObs-Offers identifies Pending and BoundaryReq with PendingRec and BoundaryReqRec and therefore identifies Front_P with FrontRec_P . The displayed BlockingMsgNextFrontRec characterization is the Core.7a and Definitions Core.14–Core.15 classification of the $\leq_P$ -minimal pending blocking items; source-prefix issue order, synchronization anchoring, and the exclusion of failed polls ensure that no earlier unrealized nonblocking, signal, or synchronization item invalidates the candidate-frontier characterization. The cross-model conclusion follows by substituting $K_{\text{post}} = K_{\text{ref}}$ into the pure reconstruction functions. $\square$

Corollary K.0a (Rendezvous endpoint-request extensionality). For a point-to-point rendezvous channel c with $B(c)=0$, define $\text{ReqProdRec}_c(K)$ and $\text{ReqConsRec}_c(K)$ as the producer-direction and consumer-direction projections of O . If $K_{\text{post}} = K_{\text{ref}}$ as above, then $\text{ReqProdRec}_c(K_{\text{post}}) = \text{ReqProdRec}_c(K_{\text{ref}})$ and $\text{ReqConsRec}_c(K_{\text{post}}) = \text{ReqConsRec}_c(K_{\text{ref}})$. This raw endpoint-request equality is independent of frontier membership: an already-asserted non-frontier blocking offer remains represented in O and can participate in rendezvous enablement.

K.2.3 Example K.0b — Front_P vs. BlockingMsgNextFront $_P$ Example

The following example illustrates the distinction between Front_P and $\text{BlockingMsgNextFront}_P$. In this example all message-operation items under discussion are blocking Push/Pop operations, so $\text{BlockingMsgNextFront}_P$ is the relevant WFG-facing slice of the source-level NextFront_P frontier over Ω_P . The broader notation MsgNextFront_P , when used elsewhere, denotes the general message-operation-item slice and may also include successful non-blocking transfer items; it is not the object used by the $\text{Front}/\text{NextFront}$ bridge for WFG construction.

```
void P() {
  while (1) {
    wait();           // sync point
    int x = in.Pop(); // blocking Pop
    out.Push(x + 1);  // blocking Push
  }
}
```

Consider two source K -facing cutpoints for this same process:

Cutpoint σ_0 : just after $\text{wait}()$ has committed, before $\text{in.Pop}()$ has issued

- $\text{BlockingMsgNextFront}_P(\sigma_0) = \{ \text{Pop}_{\text{in}} \}$
- $\text{BoundaryReq}(\text{Pop}_{\text{in}}, \sigma_0) = \text{false}$
- $\text{Front}_P(\sigma_0) = \emptyset$

Cutpoint σ_1 : in.Pop() has issued, rdy is asserted, but no data has arrived yet

- BlockingMsgNextFront_P(σ_1) = { Pop_in }
- BoundaryReq(Pop_in, σ_1) = true
- Front_P(σ_1) = { Pop_in }

So the difference is:

- BlockingMsgNextFront_P says which blocking message-operation items are next candidates in the source-level frontier over Ω_P for WFG/frontier classification.
- Front_P says which of those next blocking message-operation items are actually outstanding pending blockers right now.

K.3 Lemma K.1a — Characterization of Internal Message-Passing Deadlock (Buffered and Rendezvous Cases)

Elaboration prerequisite. The channel boundary c used in this lemma is the chosen abstract boundary of the prescribed Sys_ref / RTL_B comparison after applying any required elaboration from Definition Core.1 and Appendix J, Remark 2. In particular, synchronization-boundary ramp-down / epoch-gating is not treated as a hidden exception to E4 on an otherwise ordinary buffered channel. If such gating can withhold producer-facing availability while outer projected capacity would otherwise appear available, the relevant blocking point is represented by an explicit abstract channel of Sys_B^{elab}.

Statement: Let $\sigma_{\text{post}} \in \text{KCutReach_post}(I)$ be a reachable RTL K-facing cutpoint, with its selected normalization witness recorded when the theorem application begins from an earlier prefix. Suppose there is a non-empty set of processes D that is internally message-passing deadlocked (as defined in K.1). Then:

Every process P in D is blocked on a blocking channel operation (Push or Pop), not on an internal step. Buffered-channel case ($B(c) > 0$):

- If P is blocked on a frontier item $x \in \text{Front_P}(\sigma_{\text{post}})$, with owning message instance $q = \text{MsgOf_P}(x)$, $\text{kind}(q) = \text{Push_c}$, and $B(c) > 0$, then $\text{occ_c}(\sigma_{\text{post}}) = B(c)$ (channel is full at that cutpoint).
- If P is blocked on a frontier item $x \in \text{Front_P}(\sigma_{\text{post}})$, with owning message instance $q = \text{MsgOf_P}(x)$, $\text{kind}(q) = \text{Pop_c}$, and $B(c) > 0$, then $\text{occ_c}(\sigma_{\text{post}}) = 0$ (channel is empty at that cutpoint).

Rendezvous case ($B(c) = 0$):

- If P is blocked on a frontier item $x \in \text{Front_P}(\sigma_{\text{post}})$, with owning message instance $q = \text{MsgOf_P}(x)$, $\text{kind}(q) = \text{Push_c}$, and $B(c) = 0$, then the complementary endpoint is not simultaneously requesting the matching Pop_c in σ_{post} (i.e., the complementary Pop_c endpoint request is not true in that cycle, so rendezvous enabling for x is false).
- If P is blocked on a frontier item $x \in \text{Front_P}(\sigma_{\text{post}})$, with owning message instance $q = \text{MsgOf_P}(x)$, $\text{kind}(q) = \text{Pop_c}$, and $B(c) = 0$, then the complementary endpoint is not simultaneously requesting the matching Push_c in σ_{post} .

The processes in D form a closed strongly connected component in the WFG.

Proof:

Internal Steps: Because $\sigma_{\text{post}} \in \text{KCutReach_post}(I)$, Core.4 gives Quiescent_RTL_B(σ_{post}), so no RTL_B internal ϵ -step is enabled in σ_{post} . In particular, if some process had an internal step whose guard is true in σ_{post} , then Quiescent_RTL_B(σ_{post}) would be false: the selected ϵ -normalization could be extended by at least one additional ϵ -step. Therefore, no process in D is blocked on an internal step with a true guard; since each $P \in D$ is blocked on message passing (K.1), the blocking operation must be a blocking channel operation (Push or Pop).

Blocked Push: By the definition of “blocked on message passing” in K.1, if P is blocked due to a frontier

item $x \in \text{Front_P}(\sigma_{\text{post}})$, with owning message instance $q = \text{MsgOf_P}(x)$ and $\text{kind}(q) = \text{Push_c}$, then $\text{BoundaryReq}(x, \sigma_{\text{post}})$ is true and $\text{ChannelEnabled}(x, \sigma_{\text{post}})$ is false.

- If $B(c) > 0$, the channel-side enabling condition for x is space availability, i.e., $\text{occ_c}(\sigma_{\text{post}}) < B(c)$. Hence, if P is blocked on such a Push_c frontier item x in σ_{post} , necessarily $\text{occ_c}(\sigma_{\text{post}}) = B(c)$ (channel is full).

- If $B(c) = 0$ (rendezvous), the channel-side enabling condition for x is that the complementary Pop_c endpoint request is true in the same cycle. Since P is blocked, this rendezvous enabling condition is false; equivalently, the complementary endpoint is not simultaneously requesting Pop_c in σ_{post} .

Blocked Pop: Symmetric. By the definition of “blocked on message passing,” if P is blocked due to a frontier item $x \in \text{Front_P}(\sigma_{\text{post}})$, with owning message instance $q = \text{MsgOf_P}(x)$ and $\text{kind}(q) = \text{Pop_c}$, then $\text{BoundaryReq}(x, \sigma_{\text{post}})$ is true and $\text{ChannelEnabled}(x, \sigma_{\text{post}})$ is false.

- If $B(c) > 0$, the channel-side enabling condition for x is data availability, i.e., $\text{occ_c}(\sigma_{\text{post}}) > 0$. Hence $\text{occ_c}(\sigma_{\text{post}}) = 0$ (channel is empty).

- If $B(c) = 0$ (rendezvous), the channel-side enabling condition for x is that the complementary Push_c endpoint request is true in the same cycle. Since P is blocked, this rendezvous enabling condition is false; equivalently, the complementary endpoint is not simultaneously requesting Push_c in σ_{post} .

Closed WFG cycle: By the definition of internal message-passing deadlock, every WFG edge from a process in D points to a process in D . Hence no process in D waits, through the literal WFG edge relation, on a complementary endpoint owner outside D .

K.4 Lemma K.2 — WFG Extensionality under Equal KObs

Statement. Let $\rho_{\text{post}} \in \text{KCutReach_post}(I)$ and $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$ be the selected K -facing cutpoints of one Corollary J.1 application, and write

$K_{\text{post}} := \text{KObsPost_J}(\rho_{\text{post}}; E, w, H, O),$

$K_{\text{ref}} := \text{KObs_E}, w(\sigma_{\text{ref}}).$

If $K_{\text{post}} = K_{\text{ref}}$, then

$\text{WFGRec}(K_{\text{post}}) = \text{WFGRec}(K_{\text{ref}}).$

Equivalently, the K -facing reconstructed post wait-for graph and the operational source wait-for graph have exactly the same vertices and directed edges. The statement applies uniformly to buffered and rendezvous boundaries and to $\text{Sys_B}^{\text{elab}}$ explicit staged, bundle, join, and epoch-gate boundaries supplied by E .

Proof. WFGRec is a typed pure function of K . FrontRec and BoundaryReqRec are reconstructed from O by Lemma K.0/J.KObs-Offers; buffered channel state is reconstructed from H and E by Lemma J.KObs-Chan; rendezvous complementary requests are direction projections of O by Corollary K.0a; $\text{ChannelEnabledRec}/\text{ChannelDisabledRec}$ and the edge relation are then reconstructed by Lemma J.KObs-WFG. Substitution of $K_{\text{post}} = K_{\text{ref}}$ therefore gives graph equality by functional congruence. No separate case split over full, empty, or rendezvous channels and no clause-by-clause state correspondence is required. Lemma K.1a remains the local semantic characterization of what a disabled frontier means at a concrete boundary, but is not needed to transfer the graph. $\square$

K.4a Lemma K.2a — DrainClosedNow Extensionality under Equal KObs

Statement. Let K_{post} and K_{ref} be the equal records of Lemma K.2, and let D be a non-empty process set. Then

$\text{DrainClosedNowRec}(D, K_{\text{post}}) = \text{DrainClosedNowRec}(D, K_{\text{ref}}).$

Consequently, if the reconstructed post cutpoint is drain-closed for D in the DrainClosedNow snapshot sense, the selected operational source cutpoint σ_{ref} is drain-closed for D ; the converse snapshot implication also holds.

Proof. $\text{DrainClosedNowRec}(D, K)$ is the pure predicate that no residual offer owned by D and lying outside FrontRec is ChannelEnabledRec . All of its arguments are fields or reconstruction functions of K , so equality follows by functional congruence from $K_{\text{post}} = K_{\text{ref}}$ and Lemmas $J.\text{KObs-Offers}/J.\text{KObs-WFG}$ and $K.\text{KObs-Dead}$. This lemma transfers only the cutpoint predicate. It does not transfer, erase, or reconstruct the continuation-level $\text{Core.Drain}/K.\text{Drain}$ discipline, which remains an operational premise for the executions used by Appendix K and Appendix K-P. $\square$

K.4b Deterministic Serial-Source Dominance and the Primary Policy-S Route

Organization. The serial route normalizes the selected instance, applies SDet to select the Policy-S run artifact p_S^{det} and to exclude any incompatible committed history under the same selected functional semantics, and uses separately supplied $K.\text{CompleteS}$ evidence to establish that this artifact is maximal, fair, and complete. $K.\text{RespCov}$ assigns every static internal blocking site and relevant dynamic class a proof disposition and proves that every actual persistent process-facing serial blocker is covered by a monitor or static bounded-response proof; $K.\text{RespClean}$ proves that the complete selected run violates none of those obligations. Lemma K.2b proves that a Policy-L internal deadlock would force a finite $K.\text{RespFail}$ witness on p_S^{det} . The $\text{Dead_KS}/K.\text{EnvOrd}/K.\text{Done}$ owner-walk remains diagnostic machinery and is not needed for Corollary K.2c.

Definition K.BF (blocking-only fragment). Every wait relevant to the WFG is a persistent blocking Push/Pop at an explicit point-to-point boundary with finite B-faithful capacity; control, payloads, and later operation identities are deterministic functions of certified value/identity provenance, fixed stimulus, and fixed witness choices under $K.\text{CV}$, including Definition K.CVPred and $K.\text{CV-WF}$ below; shared arbitration is remodeled as explicit point-to-point subchannels through an arbiter whose choices are deterministic or witness-fixed. Bounded local progress: computation between message operations terminates, so at every point each process either normally completes, reaches its next message operation, or is blocked at one; a process that diverges in pure computation is outside the fragment. Persistence/interruption scope: because every WFG-relevant wait in K.BF is an ordinary blocking Push/Pop, the exhaustive Core.IC lifecycle applies once its request has issued: the request remains the same asserted, attributed, pending dynamic instance until process-facing commit or an affecting RW2 RESET; a DeclaredTerminal disposition may end the execution while it remains pending. Independent nonterminal cancellation, withdrawal, retry, or reattribution of an outstanding blocking request is not part of the G–K theorem model and requires a separately defined protocol extension outside this theorem stack. Lemma K.2b.Q uses K.BF to identify a permanently non-advancing process with a persistent process-facing frontier that $K.\text{RespCov}$ must cover. No missing-supply owner trichotomy is assumed.

Lemma K.BF-Persist (frozen-frontier interruption completeness). Fix a K.BF instance, a candidate component D , and a selected WFG-relevant blocking frontier f_P whose attributed endpoint request is pending at a Core.LAdm -admissible prefix. Along any finite continuation prefix before a DeclaredTerminal state, if f_P has not process-facing committed and no RESET affecting P or its explicit boundary has occurred, then the request for f_P remains asserted, pending, and attributed to the same dynamic operation. In particular, no independent nonterminal cancellation, withdrawal, retry, reattribution, retirement, or replacement can dissolve a K.2b frozen component. Proof. Definition K.BF establishes that f_P is an ordinary blocking Push/Pop request. $\text{Core.IC}(1)–(2)$ exhausts the ordinary blocking-instance lifecycle and gives persistence plus stable attribution until process-facing commit or an affecting RW2 RESET. A DeclaredTerminal disposition is handled separately by $K.\text{TerminalDisposition}$ and may end the run with the request still pending. Therefore, absent process-facing commit, affecting RESET, or DeclaredTerminal , the selected request remains the same pending dynamic instance. $\square$

Definition K.Low and normalized instance Sys_K . Let $\Lambda_W(\text{Sys_ref})$ be the witness-indexed lowering that represents synchronization by explicit epoch-gate channels and every aligned non-NB-CF non-blocking

dependency by a fresh blocking guard channel. The default guard has $B(g)=1$ with ordinary non-fall-through E4 timing; the finite skew after supplier commit is WFG-inert and cannot remain at a drain-closed cutpoint. Define $\text{Sys_K} \triangleq \Lambda_W(\text{Sys_ref})$ where lowering is needed, and $\text{Sys_K} \triangleq \text{Sys_ref}$ otherwise. All serial-route predicates below are evaluated on Sys_K .

Lemma K.Low.C (normalization correspondence) — statement. Projection erasing guard/epoch transitions maps executions of Sys_K to executions of the original selected instance with the same external Σ history; witness-consistent executions lift to Sys_K ; and K-facing internal wait cutpoints correspond to literal blocking wait cutpoints of Sys_K . More precisely, the load-bearing cutpoint clause relates $\text{KCutReach_Sys_K}(I)$ to $\text{KCutReach_ref}(I)$ under the lowering projection. The local proof obligations are per lowering rule and are collected in Appendix K-P.

Definition K.OpKey (common cross-policy dynamic-operation key and source order). For each selected K.BF theorem instance I , let OpKey_I be a theorem-instance universe of typed process-local dynamic-operation occurrence keys under the fixed source code, stimulus, witness, elaboration/lowering, and reset-epoch schema. A key κ records at least its owner P , dynamic source-position/iteration occurrence identifier, operation kind/site, typed boundary/direction schema, and logical reset-epoch slot; the key exists independently of whether a particular admissible execution reaches that occurrence. Each admissible execution p has a partial realization map $\text{Realize_p} : \text{OpKey_I} \rightarrow Q_exec, P(p)$, and every realized execution-local operation q has a unique $\text{Key_p}(q)=\kappa$. Realization preserves kind/boundary/direction and the process-local source order. Define $\kappa_1 <^{\text{K_src}} \kappa_2$ on common keys by that source-induced order. The execution-internal relation $<_{\text{src}}$ of Core.5 remains unchanged; whenever Appendix K compares an operation from Policy S with one from Policy L , notation such as $g_P \leq_{\text{src}} f_P$ is a readability shorthand for $\text{Key}(g_P) \leq^{\text{K_src}} \text{Key}(f_P)$, never a direct comparison of elements of two execution-local Q_exec universes. Likewise, cross-policy “same dynamic operation identity” means the same OpKey together with the typed fields/value facts determined by K.CV. Lemma K.Low.A5 (lowered-to-source A5-L transport). Let π_K be the K.Low projection from Sys_K cutpoints to Sys_ref cutpoints, and let W_K be the pullback of the selected source waiver class W along π_K ; any additional lowering-only waiver must be separately proved not to waive the lift of a non-waived Sys_ref component. Under Lemma K.Low.C, including preservation of Policy-L admissibility on projection/lift and the cutpoint correspondence of part (iii), $\text{A5L_Sys_K}(I, W_K)$ implies $\text{A5L_Sys_ref}(I, W)$, i.e. A5-L. Proof by contrapositive. A finite admissible Sys_ref prefix reaching a non-waived Dead_K cutpoint is a witness-consistent execution and therefore lifts by K.Low.C(ii) to an admissible Sys_K prefix. An ordinary Sys_ref internal message wait cycle is an instance of the extended wait-cycle class in K.Low.C(iii), so the lifted drain-closed K-facing state contains the corresponding literal blocking Dead_K component in Sys_K ; waiver pullback preserves non-waiver. This contradicts $\text{A5L_Sys_K}(I, W_K)$. If no lowering is required, π_K and the lemma are the identity case. $\square$

Example K.CE (why ordinary Policy-S Dead_K avoidance is not a sufficient serial check). Let P execute Pop_e ; $\text{Push_c}(v0)$, where e is an exhausted environment input and Push_c is independent of Pop_e . Let downstream processes use c and then form an internal wait cycle. Under Core.ReachPrep , the ordinary straight-line Push_c may become reached and issue in the L1 window after Pop_e has issued but before Pop_e process-facing commits, because its reachability/payload is independent of the unresolved Pop_e result; source order and issue-prefix closure are still preserved. Policy L can therefore reach the downstream internal cycle. Policy S withholds Push_c and instead leaves covered process-facing frontiers pending along a chain ending at exhausted e . Thus absence of a Policy-S internal cycle alone does not imply A5-L; K.Resp detects the unanswered serial frontier without requiring a closed Policy-S WFG component.

Definition K.RespMon (process-facing bounded-response monitor). For a dynamic blocking source-operation instance q of process P in reset epoch e , let x be its E-typed process-facing frontier item or certified joint-frontier representative. The monitor triggers at the first selected Policy-S normalized cycle

t in which q is the active source-minimal pending blocking frontier, its persistent request attribution is established, and q has not process-facing committed. The response is the commit/return that releases q from P 's source execution; proof-internal channel movement does not count unless E identifies it as that process-facing commit. The monitor carries a finite bound $B_resp(P, q, e, state_class)$ measured in selected Policy-S clock cycles. Its age advances with the selected clock even while unrelated processes commit. For a certified multi-frontier source operation, the monitor is attached to operation return and the K-O2 composition lemma proves that permanent joint blockage leaves the monitor pending.

Definition K.RespFail, K.RespCov, and K.RespClean. K.RespFail is a predicate on finite Policy-S prefixes. A non-waived witness exists for a covered monitor when: (i) timeout—the monitor triggers with age zero in normalized cycle t ; a response through the boundary-action phase of cycle $t + B_resp$ is timely, and otherwise timeout is witnessed at the first normalized cutpoint after that phase; (ii) maximal-pending—the selected execution reaches a declared maximal finite terminal state while the monitor remains pending; or (iii) cancelled-pending—an affecting RESET governed by RW2 clears q before response. A temporarily silent/noncommitting cycle is not maximal: Core.CompleteCycle continues the clock and therefore continues K.Resp aging. An affecting reset is a failure unless the exact dynamic class is explicitly waived with the K.Resp coverage-preservation proof. Under the present G–K blocking-request model, case (iii) is the reset-epoch cancellation case; there is no independent nonterminal cancellation/withdrawal disposition for an outstanding blocking request. A modeled abort or end-of-test that ends the selected execution is represented by DeclaredTerminal and is therefore covered by maximal-pending when the monitor remains pending. K.RespCov is default-total over a declared coverage universe of static internal process-facing blocking sites and relevant dynamic classes. Each class has one disposition: a literal monitor, a validated static bounded-response proof, or an unreachability proof. Completeness requires every dynamically reachable instance to belong to a declared class; once reached, an unreachability disposition cannot apply. The user or checker need not first decide whether a site can become the persistent serial blocker; that fact motivates coverage but is not its quantification domain. A production-count argument alone is not an exemption. K.RespClean states that no finite prefix of the complete p_S^{det} witnesses a non-waived K.RespFail.

Environment and waiver rule for K.RespCov. Every internal process-facing blocking site is covered by default. For StandardEnv/B1-avail, a source-earlier environment operation that committed in the frozen liberal prefix has the same consumption ordinal available to Policy S and, under persistent request, B2/B3, and Core.IssueFair, cannot be the permanent serial blocker; that theorem may discharge coverage for the corresponding dynamic instance. For a timed/reactive or otherwise nondefault environment, K.EnvMF first requires every relevant liberal Dead_K component to be certified single-internal-frontier. Environment-facing operations that may block the selected Policy-S execution must then be monitored, proved available, or placed outside the serial route. A per-site monitor, availability proof, no-bypass argument, or waiver does not admit a timed/reactive multi-frontier liberal component; that combination is outside K.2b/K.2c. A response waiver is valid only if every non-waived liberal Dead_K component extracted by K.2b maps to at least one non-waived covered serial monitor.

Complete-run and bound rule. K.CompleteS remains the semantic proof that p_S^{det} is the complete selected artifact. K.RespClean is a separate property checked over it. For a terminating artifact, the checker validates every monitor through the declared maximal terminal state; absence of current commits in an otherwise nonterminal cycle does not make the artifact complete, because Core.CompleteCycle continues the selected clock and monitor ages. For an infinite artifact, a finite trace is insufficient: an invariant, exhaustive model check, recurrence/lasso proof, or equivalent certificate must establish that monitor ages never exceed their bounds. For theorem soundness, the bound need only be finite and compliance must be proved over the complete run; a separately justified worst-case-latency proof is required only to classify a timeout as a design-specification violation rather than as a failed certificate attempt. No K.Resp bound is transferred to RTL by Theorem K.1.

Diagnostic Definition K.Cone, Condition K.EnvOrd, and Condition K.Done. K.Cone and its may-follow closure may be used to explain environment-rooted or completion-rooted response failures and to prove that particular eager bypasses are impossible. K.EnvOrd retains the transitive, loop-aware dynamic no-bypass meaning described below. K.Done remains only a normal-completion audit: it can prove DoneStop unreachable by showing that every normally terminating endpoint produces or consumes enough operations before FINISH, or by proving that no process normally completes. The latter does not establish continuing service on every endpoint and does not address the H2 perpetual-under-supply case. Consequently K.EnvOrd/K.Done may simplify diagnostic Dead_KS reports but never permit omission of K.Resp coverage and never discharge A5-L by themselves.

Example K.CE-2 (transitive starvation—diagnostic value and uniform K.Resp detection). Let e be an environment input channel with empty stimulus, and let c_E , c_V , a , b be internal point-to-point channels. E : Pop $_e$; Push $_cE$. W : Pop $_cE$; Push $_cV(v0)$, with $v0$ independent of the Pop $_cE$ payload. V : Pop $_cV$; Push $_a$; Pop $_b$. $P2$: Push $_b$; Pop $_a$. The straight-line dependency-independent successors Push $_cE$ and Push $_cV$ use the same Core.ReachPrep permission as Example K.CE: they may become reached/issued in L1 before the earlier blockers commit while source order remains intact. An admissible Policy-L schedule can therefore feed the downstream internal cycle $\{V, P2\}$, so A5-L fails. Policy S withholds the feeding operations and leaves the relevant downstream/internal serial frontiers pending. Total K.RespCov therefore produces a finite timeout or maximal-pending failure without needing to walk the chain to EnvStop. K.Cone/K.EnvOrd may still diagnose the transitive environment root and explain which bypass admitted the liberal cycle, but that diagnosis is not part of the soundness proof.

Example K.CE-3 (loop-carried bypass—diagnostic value and uniform K.Resp detection). Let e and c_E be as in Example K.CE-2, with E : Pop $_e$; Push $_cE(f(e))$ fully dependent, and let W be the pipelined loop W : loop { Push $_cV(v0)$; Pop $_cE$ }, with $v0$ independent of the Pop $_cE$ payload. Policy L can issue iteration $i+1$'s Push $_cV$ while iteration i 's Pop $_cE$ remains pending and feed the downstream cycle; Policy S withholds the crossing and leaves a covered process-facing serial frontier unanswered. K.Resp therefore rejects the selected run by a finite response failure. The may-follow form of K.EnvOrd remains useful diagnostically because it identifies the loop-carried crossing that a merely textual source-later analysis misses.

Example K.CE-4 (completed-producer starvation—diagnostic value and uniform K.Resp detection). Let Z : Push $_c(w)$; end produce exactly one item on internal channel c and normally complete, and let Y : Pop $_c$; Pop $_c$; Push $_d(v0)$ demand two, with $v0$ independent; V : Pop $_d$; Push $_a$; Pop $_b$ and $P2$: Push $_b$; Pop $_a$. Policy L may issue Push $_d$ before Y 's second Pop $_c$ settles and thereby reach the downstream internal cycle. Under Policy S, Push $_d$ is withheld and Y 's second Pop $_c$ remains unanswered after Z completes. K.RespCov monitors that process-facing Pop and therefore yields a finite timeout or maximal-pending failure. The diagnostic WFG+ may classify the cause as DoneStop $_c$, but K.2b no longer depends on completion being an exhaustive owner-walk case.

Example K.CE-5 / H2 (perpetual under-supplying owner). Let Z : Push $_c(1)$; while (1) { Push $_d1(0)$; Pop $_d2$; }, W : while (1) { Pop $_d1$; Push $_d2(0)$; }, X : Pop $_c$; Pop $_c$; Push $_g(v0)$ with $v0$ independent of the Pops, V : Pop $_g$; Push $_a$; Pop $_b$, and $P2$: Push $_b$; Pop $_a$. Policy L may issue Push $_g$ before Pop $_c\#2$ settles and then reach the internal cycle $\{V, P2\}$. Under Policy S, Push $_g$ is withheld; Z and W may continue their $d1/d2$ ping-pong forever while X and the downstream chain remain permanently unanswered. No closed Dead_KS component need exist because the owner Z stays live. K.RespCov nevertheless monitors the process-facing serial blockers, and their finite bounds are violated on the complete run. This example is the reason the owner trichotomy and terminal taxonomy are not soundness-critical in the revised route.

Definition K.EnvMF (Policy-S environment/multi-frontier applicability). For every application of Lemma K.2b.E to a component D at cutpoint σ_L , one of the following shall hold: (1) the selected instance uses StandardEnv/B1-avail and the default always-ready output environment; or (2) the selected instance uses a timed/reactive or otherwise nondefault environment, and every $P \in D$ has a certified single frontier at σ_L consisting solely of the selected internal blocking item f_P . Under clause (2), no environment-facing co-frontier, second internal frontier item, elaborated join/bundle alternative, or other multi-frontier item may coexist in $\text{Front_P}(\sigma_L)$. Timed/reactive/nondefault multi-frontier components are outside Lemma K.2b.E, Lemma K.2b, Corollary K.2c, K.PSF, and the Policy-S form of Theorem K.1.

Lemma K.EnvX-Auto (environment co-frontiers). Under B1-avail and the default always-ready output environment, every environment-facing frontier item held by a blocked member of a Dead_K component is stimulus-exhausted. Proof. Blockedness requires every frontier item to be channel-disabled. A remaining input item or ready output environment would enable the environment-facing item. Hence no separate environment-freeze premise is needed under StandardEnv. Scope emphasis (normative): B1-avail is a genuine environment-model restriction, not a bookkeeping convention—it asserts that stimulus availability is consumption-ordered and untimed. A testbench that delivers stimulus at declared clock times, or a nondefault output environment that may release back-pressure later, is outside B1-avail. For such an environment the Policy-S route is supported only through K.EnvMF's certified single-internal-frontier clause. Timed/reactive/nondefault multi-frontier cases require a non-serial A5-L proof or another theorem and are not admitted by a substitute exhaustion/no-rescue certificate in this revision.

Definition K.InterruptionGate (shared nested comparison-interruption schema). Fix a theorem instance I , a candidate liberal component D , its Core.LAdm-admissible counterexample prefix p_L ending at σ_L , and the selected frozen frontier operation f_P for each $P \in D$. A gate specification G supplies two gate-specific predicates. First, $\text{Interrupt_G}(I, D, \{f_P\}, \eta)$ is a prefix/state-local interruption predicate evaluated only on finite Core.LAdm-admissible extension prefixes η of p_L that are consistent with the fixed instance choices and throughout which every selected f_P remains pending; it may inspect the fixed contract, η 's final state, and the legality of the next proof-relevant transition/unit (with any executed real-cycle occurrence recorded in the enclosing Core.CycleResult), but it shall not depend on a continuation already constructed by Lemma K.2b.E. Second, DirectFail_G supplies an independently certified mapping $P \in D$, f_P , a process-facing Policy-S serial frontier x_P^S , and $g_P := \text{Op}(x_P^S)$ with $g_P \leq_{\text{src}} f_P$, together with a finite selected Policy-S prefix on which a non-waived K.Resp obligation for x_P^S has already triggered and remains pending, plus gate-specific evidence that the corresponding interruption/disposition occurs and therefore witnesses K.RespFail. The direct mapping and disposition shall be established without K.2b.R, K.2b.P, or K.2b.Q Step 1. A K.InterruptionGate_G discharge uses one of two branch forms: Continue, when every such frozen prefix η satisfies $\neg \text{Interrupt_G}(I, D, \{f_P\}, \eta)$; and DirectFail, when DirectFail_G is established. If neither branch is established, the candidate is outside the Policy-S reduction and requires another A5-L discharge. Multiple gate instances are not flattened into one unordered enumeration: the theorem states their evaluation order explicitly. For Lemma K.2b the required order is K.ResetEpoch first and, only on K.ResetEpoch.Continue, K.TerminalDisposition second. Exhaustiveness for K.BF. For the K.BF serial-reduction fragment, Axiom Core.IC and Lemma K.BF-Persist establish that, apart from process-facing commit, the only modeled interruptions that can terminate or invalidate persistence of a selected frozen blocking instance are an affecting RW2 RESET or a DeclaredTerminal disposition ending the execution while the request remains pending. Accordingly, Reset and Terminal are the complete interruption-gate instantiations consumed by Lemma K.2b. Adding another gate is a theorem-model extension and requires an explicit operational definition plus re-proof of K.BF-Persist and the affected K.2b obligations; it is not a routine additional instantiation of this schema.

Definition K.DomResetScope (dominance reset scope). For a candidate D , p_L , $\{f_P\}$, fixed Sys_K elaboration, fixed source/dynamic operation universe, and selected Policy-S comparison artifact, a $K.\text{DomResetScope}$ certificate supplies finite sets S_p^{dom} of processes and S_c^{dom} of explicit abstract boundaries. The certificate is valid iff: (D1) $D \subseteq S_p^{\text{dom}}$; (D2) for every $c \in S_c^{\text{dom}}$, both endpoint processes of c belong to S_p^{dom} ; (D3) every process-owned endpoint operation and every explicit-boundary transfer identity consumed by the $K.2b.R/K.2b.P$ comparison for the fixed selected Policy-S artifact is owned by a process in S_p^{dom} and, when boundary-specific, by a boundary in S_c^{dom} ; and (D4) S_p^{dom} and S_c^{dom} are determined from the fixed elaboration, source/dynamic operation universe, and selected Policy-S comparison artifact and do not depend on the later liberal continuation constructed by Lemma K.2b.E. Define $K.\text{DomResetScope}_I(D, \{f_P\}) := (S_p^{\text{dom}}, S_c^{\text{dom}})$ for any validated certificate. Minimality is not required; a larger conservative scope is permitted. For any comparison prefix η , $\text{DomEpoch}_I(\eta)$ is the vector of reset-epoch identifiers of the processes and explicit boundaries in this validated scope. A RESET is dominance-affecting exactly when its RW scope intersects this dominance scope. RW5 permits RESET occurrences disjoint from $K.\text{DomResetScope}$; such resets may change epochs outside the dominance scope but cannot change any dynamic identity, boundary state, or request fact consumed by $K.2b.R/P$. $K.\text{ResetEpoch.Continue}$ therefore keeps DomEpoch fixed without requiring a global no-reset execution.

Definition K.ResetEpoch (reset instantiation of $K.\text{InterruptionGate}$). For the candidate D , p_L , and $\{f_P\}$, let $\text{Interrupt_Reset}(I, D, \{f_P\}, \eta)$ hold exactly when the fixed reset contract permits, as the next proof-relevant transition/unit from η 's final state, a RESET whose scope intersects $K.\text{DomResetScope}_I(D, \{f_P\})$. When that RESET executes within a real cycle, the occurrence is recorded in the enclosing legal $\text{Core.CycleResult.resetUnits}$. Let DirectFail_Reset be the independently mapped reset witness: $P \in D$, selected liberal f_P , process-facing Policy-S serial frontier x_P^{AS} in the same pre-reset endpoint epoch, $g_P := \text{Op}(x_P^{\text{AS}})$ with $g_P \leq_{\text{src}} f_P$, an already-triggered non-waived $K.\text{Resp}$ obligation that remains pending, and the first matched RESET affecting that monitored frontier whose emitted RESET unit causes RW2 to clear that exact dynamic instance, yielding cancelled-pending $K.\text{RespFail}$. Define $K.\text{ResetEpoch} := K.\text{InterruptionGate_Reset}$. Thus $K.\text{ResetEpoch.Continue}$ is the dominance-scope-reset-free comparison branch and $K.\text{ResetEpoch.DirectFail}$ is the matched-reset direct-failure branch. The Continue predicate is tested before RW2 clearing can occur and is contract-wise rather than inferred from absence of an observed reset. A reset intersecting $K.\text{DomResetScope}$ but not covered by DirectFail_Reset makes the candidate ineligible for the Policy-S reduction and requires another A5-L discharge; a reset disjoint from $K.\text{DomResetScope}$ is permitted and framed by RW5. All request-identity, endpoint-ordinal, nonwithdrawal, and persistence arguments on the Continue branch are scoped by the fixed DomEpoch rather than by a nonexistent single global reset epoch.

Definition K.TerminalDisposition (terminal instantiation of $K.\text{InterruptionGate}$). This gate is instantiated only after $K.\text{ResetEpoch.Continue}$ has been established for the candidate D . For the same p_L and $\{f_P\}$, let $\text{Interrupt_Terminal}(I, D, \{f_P\}, \eta)$ hold exactly when η 's final state satisfies $\text{DeclaredTerminal_Sys}_K, I$. Ordinary Core.SilentCycle progression does not satisfy this predicate. Let $\text{DirectFail_Terminal}$ be the independently mapped terminal witness: $P \in D$, selected liberal f_P , process-facing Policy-S serial frontier x_P^{AS} , $g_P := \text{Op}(x_P^{\text{AS}})$ with $g_P \leq_{\text{src}} f_P$, a finite selected Policy-S prefix on which a non-waived $K.\text{Resp}$ obligation for x_P^{AS} has already triggered and remains pending, and independently checked evidence that p_S^{det} reaches the specified declared maximal terminal state while that same monitor remains pending, yielding maximal-pending $K.\text{RespFail}$. Define $K.\text{TerminalDisposition} := K.\text{InterruptionGate_Terminal}$. Thus $K.\text{TerminalDisposition.Continue}$ is the terminal-free comparison branch and $K.\text{TerminalDisposition.DirectFail}$ is the declared-terminal direct-failure branch. Modeled abort, watchdog, cycle-count, or end-of-test dispositions are handled here only when they satisfy DeclaredTerminal ; independent nonterminal cancellation/withdrawal is outside the present G–K model. The nesting is normative: $K.\text{TerminalDisposition}$ is not evaluated on $K.\text{ResetEpoch.DirectFail}$.

Lemma K.2b.E (fair complete frozen extension). Let $\rho_L \in \text{Sys_K.Pref_L}^{\text{adm}}(I)$ end at a source K-facing cutpoint $\sigma_L \in \text{KCutReach_Sys_K}(I)$ satisfying Dead_K^W , let D be a selected Dead_K component witnessing Dead_K^W at σ_L , and let f_P be the component frontier operation selected for each $P \in D$. By Definition K.Dead, D is drain-closed at σ_L ; this fact is extracted from the deadlock hypothesis and is not an additional cutpoint premise. Assume $\text{K.EnvMF}(I, D, \sigma_L)$, $\text{K.ResetEpoch.Continue}$, and—on that reset-free branch— $\text{K.TerminalDisposition.Continue}$ for this application. Under Definition Core.LAdm , Core.CycleClosure , $\text{Core.Drain/K.Drain}$, $B2/B3/B5$, Core.IssueFair , nonwithdrawal, $\text{K}\epsilon$, K.BF , point-to-point ownership, and the same fixed stimulus, witness, environment, reset, arbitration, termination contract, and normalization choices, there exists an infinite fair complete continuation $\bar{\rho}_L \in \text{Exec_L}^{\text{adm}}(I)$ extending ρ_L in which every f_P remains pending while the K.DomResetScope DomEpoch remains fixed. Under StandardEnv , Lemma K.EnvX-Auto prevents later environment rescue of any co-frontier. Under a timed/reactive or otherwise nondefault environment, K.EnvMF excludes all co-frontiers by requiring each $P \in D$ to have only the selected internal frontier. The continuation may schedule all enabled work outside the frozen component and may drain already-issued downstream work, but closure and DrainDiscipline prevent any newly issued source-later request from releasing the frozen frontiers. A disabled frozen request is not ChannelEnabled and therefore creates no $B2$ obligation to commit; Lemma K.BF-Persist applies the exhaustive Core.IC blocking lifecycle to that WFG-relevant blocker; and a complementary source-later operation prohibited by Core.16 is not IssueEnabled and therefore creates no Core.IssueFair obligation to issue. This lemma supplies only the reset-free, terminal-free continuation used by the dominance comparison. $\text{K.ResetEpoch.DirectFail}$ and $\text{K.TerminalDisposition.DirectFail}$ are direct K.RespFail branches of Lemma K.2b and do not invoke this extension.

Lemma K.2b (deterministic serial-source dominance with bounded-response extraction). Fix Sys_K , stimulus, capacities, elaboration, environment, witness, concrete Policy-S simulator/scheduler and arbitration semantics, Policy-S issue semantics, ϵ -normalization, the reset and termination/end-of-test contracts, the $\text{K.InterruptionGate}$ schema, and K.Resp . Assume SDet , $A1-A4$ and Core.IssueFair , $A6-A11$, K.CV , K.BF , point-to-point ownership, nonwithdrawal, $\text{K}\epsilon$, K.Drain , Core.CycleClosure , K.EnvMF , K.ResetEpoch for every candidate component, $\text{K.TerminalDisposition}$ on every $\text{K.ResetEpoch.Continue}$ candidate, K.RespCov , the same fixed instance choices for the liberal prefix and its applicable continuation, and the applicable K.Low correspondence. Let $\rho_{S^{\text{det}}}$ be the Policy-S execution selected by SDet , with separate K.CompleteS evidence establishing maximality, fairness, and completeness. If some Policy-L prefix ρ_L in $\text{Sys_K's Pref_L}^{\text{adm}}(I)$ domain reaches a source K-facing cutpoint whose K_L satisfies DeadKRec^W , then $\rho_{S^{\text{det}}}$ has a finite prefix containing a non-waived K.RespFail witness. The proof follows the gate order, not a flat case split. First evaluate K.ResetEpoch . Its DirectFail branch supplies the independently mapped pending Policy-S monitor and matched affecting RESET , so RW2 yields cancelled-pending directly. Only on $\text{K.ResetEpoch.Continue}$ evaluate $\text{K.TerminalDisposition}$. Its DirectFail branch independently supplies the mapped pending monitor and DeclaredTerminal occurrence, yielding maximal-pending directly. Only when both nested gates take Continue does Lemma K.2b.E construct the infinite fair frozen continuation; K.2b.R and K.2b.P establish deterministic serial dominance on it, and K.2b.Q extracts the finite timeout/maximal-pending witness from the complete selected Policy-S artifact. If either required gate instance satisfies neither shared-schema branch, the candidate is outside the Policy-S reduction. The direct-failure evidence is independent of K.2b.R/P/Q Step 1, so no gate branch is proved by the dominance continuation it is used to select.

Proof status. Appendix K-P gives explicit paper proofs of K.Low.C , K.Low.A5 , K.2b.P0 , K.2b.P1 , K.2b.R , K.2b.P , K.2b.E , and K.2b.Q and assembles them into Lemma K.2b. The remaining work is mechanization and tool/flow qualification rather than an omitted paper argument. The prerequisite mechanization now includes the per-lowering-rule cases of K.Low.C/K.Low.A5 ; the common K.OpKey realization/source-order layer; the K.2b.P0 occurrence/reachability lemma; and the K.2b.P1 determinacy proof over

Definition K.CVPred/K.CV-WF. K-O0 covers K.OpKey plus the K.CV fact-key/provenance certificate, well-foundedness/rank proof, and K.2b.P1; K-O1 covers Core.IssueFair; K-O2 covers multi-frontier joint freeze and monitor-trigger composition; K-O3 covers deterministic serial progress dominance on the reset-free, terminal-free infinite-fair branch; K-O4 covers K.2b.P0/K.2b.R and the Continue predicates of the two K.InterruptionGate instantiations; K-O5 covers default-total K.Resp coverage and waiver preservation; K-O6 validates the gate-independent DirectFail payload shape and its reset-specific cancelled-pending and terminal-specific maximal-pending instantiations; and K-O7 mechanizes the shared K.InterruptionGate anti-circularity/prefix-local rule, the nested ResetEpoch.Continue $\rightarrow$ TerminalDisposition.Continue ordering, Core.CycleChoiceExt use, and the resulting infinite fair frozen extension. K-O8 remains optional diagnostic machinery. The former missing-supply-owner trichotomy, owner-walk closure, K.EnvOrd/K.Done collapse, and KObs drain-bag descent are not part of the primary soundness stack.

K.4c Corollary K.2c — The Deterministic Policy-S Execution Suffices

Definition K.RespSafe. For the selected $p_S^{\wedge det}$ and certified K.Resp configuration, $RespSafe_S^{\wedge W}$ means $K.RespCov \wedge K.RespClean$. Here K.RespClean means that no finite prefix of the complete run witnesses a non-waived K.RespFail. $RespSafe_S^{\wedge W}$ is an operational complete-run property, not a KObs cutpoint predicate and not an RTL latency statement.

Premise A5-S (primary Policy-S check). Let $p_S^{\wedge det}$ be the Policy-S execution selected by SDet. A5-S requires K.CompleteS evidence establishing maximality, fairness, and completeness of that run, K.RespCov establishing total finite-bound monitor coverage, and K.RespClean establishing $RespSafe_S^{\wedge W}$. CF-S records these separate layers together with the shared K.InterruptionGate schema, K.ResetEpoch evidence for every liberal candidate component quantified by K.2b, and K.TerminalDisposition evidence only for candidates on K.ResetEpoch.Continue. A5-S alone is not an A5-L discharge outside those applicability conditions.

Corollary K.2c (deterministic Policy S is a sufficient bounded-response check). Under SDet, K.EnvMF, the shared K.InterruptionGate schema with a certificate-bound K.DomResetScope and K.ResetEpoch for every candidate component and K.TerminalDisposition on every K.ResetEpoch.Continue candidate, and Lemma K.2b's remaining premises, A5-S implies A5-L. More precisely, Lemma K.2b first proves $A5L_Sys_K(I, W_K)$ for the selected normalized blocking instance. If that lowered property were false, its finite Core.LAdm-admissible counterexample prefix would satisfy K.EnvMF and one shared-schema branch of K.ResetEpoch. K.ResetEpoch.DirectFail independently supplies a finite non-waived cancelled-pending K.RespFail at the matched affecting RESET. On K.ResetEpoch.Continue, evaluate K.TerminalDisposition: its DirectFail branch independently supplies the mapped pending Policy-S response obligation and actual DeclaredTerminal disposition, yielding maximal-pending directly. Only under K.ResetEpoch.Continue followed by K.TerminalDisposition.Continue does Lemma K.2b.E supply the infinite fair frozen continuation required by K.2b, after which K.2b.P0/R/P/Q force a finite non-waived K.RespFail on the complete selected $p_S^{\wedge det}$. Every branch contradicts K.RespClean, so $A5L_Sys_K(I, W_K)$ holds. Lemma K.Low.A5 then transports that result to $A5L_Sys_ref(I, W)$, which is A5-L. The corollary makes no claim for candidates that fail K.EnvMF or for which a required gate instantiation satisfies neither Continue nor DirectFail. $\square$

Definition K.CompleteS and practical completeness rule. The SDet-selected $p_S^{\wedge det}$ is complete when it is maximal and fair for the selected deterministic simulator semantics under the Core.CompleteCycle projection of the selected Core.CycleResult semantics. A terminating run must reach a declared maximal terminal state; a cycle with no selected Σ commit is not terminal merely for that reason. A nonterminating run requires invariant, model-checking, recurrence/lasso, or equivalent completeness evidence. K.CompleteS identifies the entire artifact over which K.RespClean is checked; it does not itself imply any response bound. Changing capacities, elaboration, witness, stimulus, environment/reset

contract, arbitration, simulator scheduler, Policy-S implementation, response-monitor configuration, waiver set, or normalization creates a different selected instance.

Determinism and bound note. Corollary K.2c concerns the selected artifact p_S^{det} generated by the concrete Policy-S simulator semantics and certified complete by K.CompleteS. It neither assumes nor proves confluence among multiple Policy-S histories. A successful proof may use any finite response bounds if it proves complete-run compliance; increasing an unproved bound after a timeout changes the certificate attempt but not the functional design instance. A separately justified bound is needed to call the timeout a functional performance violation. WC and deterministic replay support reproducibility relative to the selected history; SDet fixes the remaining global choices that could change that history or the monitor outcomes.

K.4d Structural Sufficient Conditions for A5-L

The theorems of this subsection are alternate direct discharges of A5-L. They do not use Policy S, Lemma K.2b, or Corollary K.2c. They remain useful for nonterminating systems, designs outside the serial fragment, timed/reactive multi-frontier instances, or flows that prefer a compact structural certificate—but only when A5-L is actually true. They cannot certify the eager-dependent crossing example above, because that design has an admissible serial-shaped prefix that already falsifies A5-L.

Definition K.Dep (full static dependency graph). G_dep is the directed graph over the processes of the selected instance with an edge $P \rightarrow Q$ for every way P can, at some reachable cutpoint, hold a disabled blocking frontier item whose actual unique complement owner is Q : a data edge from the consumer to the producer of every internal channel; a capacity (back-pressure) edge from the producer to the consumer of every finite-capacity internal channel; both edges for every rendezvous channel; both directions between synchronization partners through their epoch-gate channels; and, for every arbiter process, edges in both directions between the arbiter and each client along the remodeled point-to-point sub-channels. Caution (normative): G_dep shall be computed on this full relation. The data-flow topology alone is insufficient: a feed-forward data graph with finite capacities, synchronization, or arbitration can still contain dependency cycles through capacity, epoch-gate, or arbiter edges.

Theorem K.Str-1 (acyclic dependency). If G_dep is acyclic, then A5-L holds for the selected instance.

Proof. At every reachable source K-facing cutpoint under every admissible issue policy, every literal WFG edge is an instance of a G_dep edge, by the E4 evaluation of ChannelDisabled at point-to-point boundaries and the WFG edge definition; a WFG cycle would therefore yield a G_dep cycle. ■

Theorem K.Str-2 (ranked dependency). If there is a function r from processes (or from process-boundary pairs) to a well-ordered set such that every G_dep edge realizable as a WFG edge at a reachable cutpoint strictly decreases r , then A5-L holds, since a WFG cycle would yield an infinite descent in r . Edges provably unrealizable at reachable cutpoints may be excluded from the obligation, with each exclusion proved as an invariant of the selected instance and recorded in the side-condition discharge record. ■

Theorem K.Str-3 (credit / initial-token invariant). If every cycle of G_dep contains at least one edge that is unrealizable as a WFG edge at any reachable cutpoint — for instance, an edge of a channel with a proved occupancy invariant $0 < occ_c < B(c)$ whenever both endpoints are pending, as arranged by an initial token or a credit protocol — then A5-L holds. ■ Each such invariant is itself a proof obligation over the selected instance, including its capacities and elaboration, and shall be recorded in the side-condition discharge record.

K.4e Certificate and Discharge Formats (Normative)

A concrete theorem application uses one versioned certificate package. It may contain CF-0 cutpoint certificates and one A5-L discharge payload. A K-facing CF-0 carries or references the theorem-derived/direct J.RegPhaseReach provenance and the explicit K.DrainReach proof whose conjunction is

K.RLAdmReach for the exact selected source witness. A universal K.1A-Total or K.1 claim records the Core.QSync structural evidence and theorem-derived Core.KCutClosure_post reference, together with K.CertCov; one CF-0 never implies package totality. Policy-S certificates remain separate complete-run artifacts.

Definition K.CertHdr (common certificate header). The header binds every artifact to one theorem instance and records schema/model versions; source, Sys_ref/Sys_K, and RTL identities; elaboration, capacities, projections, ownership, and lowering; fixed stimulus, environment, reset, witness/WC provenance; the Core.CycleResult/Core.CycleClosure semantics identity where a complete source or Policy-S run is consumed, including full nextState environment/auxiliary typing and reset-unit emission; and, for the Policy-S route, the concrete deterministic simulator/scheduler, deterministic arbitration, Policy-S issue semantics, ϵ -normalization identity, SDet evidence, K.Resp monitor schema, finite-bound function, reset/end-of-test disposition, the K.InterruptionGate schema/version, each candidate K.DomResetScope/DomEpoch definition and provenance, K.ResetEpoch evidence for candidate components, K.TerminalDisposition evidence on K.ResetEpoch.Continue candidates, and response-waiver policy. It also records cutpoint waivers, route, side-condition references, Core.QSync structural-evidence provenance and theorem references, K.CertCov coverage method when applicable, generator/checker versions, and the trusted computing base. Any QS/settling premise accepted solely from an unverified HLS/synthesis-tool report shall be named in the declared trusted base; independently checked concrete evidence records the checker/certificate identity instead. Different header identities denote different selected instances.

Definition K.CertifiedCutpoint (K-ready package for one reachable RTL cutpoint). Fix an RTL K-facing cutpoint $p_post \in \text{KCutReach_post}(I)$, a finite reachable RTL_B prefix τ_post , and an explicitly recorded finite normalization witness v_post satisfying $\text{NormPost}_I(\tau_post, v_post, p_post)$.

CertifiedCutpoint_I($\tau_post, v_post, p_post$) holds when one K.CertHdr-bound package records that triple and establishes: (1) J.LocalGWR/J.GWR-Comp or J.GWR-Direct for the selected prefix; (2) the exact Corollary J.1 tuple $(\tau_ref, \sigma_ref, E, w, H, O)$, with $\sigma_ref \in \text{KCutReach_ref}(I)$; (3) J.HCanon, J.OfferReach, J.OfferConf, and J.KObsConf; (4) J.RegPhaseReach for that exact τ_ref/σ_ref , theorem-derived on the normal route or directly validated; (5) K.DrainReach, including Core.SettleKBridge, for the same tuple; and hence (6) K.RLAdmReach and membership of τ_ref in $\text{Pref}_L^{\text{adm}}(I)$. The package proves one prefix/normalization/K-cutpoint triple only and does not imply K.CertCov.

Definition K.CertCov (K-ready package coverage). K.CertCov(I) holds when every reachable RTL K-facing cutpoint $p_post \in \text{KCutReach_post}(I)$ has at least one recorded reachable-prefix/normalization pair (τ_post, v_post) satisfying $\text{NormPost}_I(\tau_post, v_post, p_post)$ and a corresponding CertifiedCutpoint_I($\tau_post, v_post, p_post$) package. Under the standard Core.QSync route, the canonical same-cycle normalization/endpoint supplied by Theorem Core.QSync-Post may be used for this K-facing NormPost selection. This canonicalization does not replace Appendix I/J ReplayNormalizationConvention or its selected replay-normalization/congruence witnesses. K.CertCov does not require every arbitrary reachable RTL prefix to admit a normalization before the next Σ action; coverage is over the complete reachable K-facing domain on which DeadKRec^W is defined. Equivalently, the package-existence relation is total on $\text{KCutReach_post}(I)$. A cutpoint itself may serve as the selected prefix endpoint with an empty normalization when appropriate. This is a per-instance semantic coverage premise, not a conclusion of one successful CF-0 validation. For Corollary K.1A-Total and Theorem K.1, K.CertCov is paired with Core.KCutClosure_post(I): in the standard scope the latter is theorem-derived from the independently audited Core.QSync structural facts; closure makes the KCut domain observationally complete for persistent K-relevant blocking configurations, while K.CertCov supplies the complete correspondence package at every cutpoint in that domain. A later theorem such as J.CertExist may discharge K.CertCov constructively from static semantic-compliance relations; until such a theorem is proved, the selected flow shall provide an explicit independently checked K.CertCov discharge.

CF-0 — KObs cutpoint correspondence certificate. This certificate supports Corollary J.1 and, when augmented with item (8), Theorem K.1A for one cutpoint; it neither discharges A5-L nor proves K.CertCov. It records or commits to: (1) K.CertHdr; (2) selected source/RTL prefixes and finite normalizations; (3) canonical H; (4) typed residual offers O; (5) J.HCanon/J.OfferReach; (6) complete RelevantReqSet/ReqProjection equality, attribution, cardinality, prefix-closure, and no-hidden-request evidence for J.OfferConf; (7) B-faithfulness/E4, reset, and ϵ -opacity for J.KObsConf; and (8), for Appendix K, the J.RegPhaseReach provenance plus K.DrainReach evidence whose checked conjunction is K.RLAdmReach for the exact tuple. The phase provenance includes Core.RegSeq, enriched deferred-hole/NoDependentUse records, field-sensitive footprints, J.RegPhaseNF, Horizon-batched J.UnitExt, and the no-unresolved-hole cutpoint invariant, or a direct equivalent. Derived Front/WFG/deadlock records are checker-reconstructed. Historical Issue/ReturnedAtCycle logs and K.Resp timers need not be duplicated, but referenced artifacts must validate their consumed obligations.

CF-S — Primary deterministic Policy-S bounded-response certificate. The payload records the header; the functional Policy-S simulator semantics, initial state, fixed witness/stimulus/environment, deterministic scheduler and arbitration, capacities, elaboration, lowering, normalization, response-waiver policy, K.CVPred/K.CV-WF provenance, and the shared K.InterruptionGate schema with each certificate-bound K.DomResetScope/DomEpoch and its ordered K.ResetEpoch and K.TerminalDisposition instantiations; the selected run artifact p_S^{det} as a sequence of Core.CycleResult records or an equivalent symbolic representation whose CompleteCycle projection is checkable; SDet evidence; fairness/maximality and B2/B3/B5/K.Drain/K ϵ /nonwithdrawal evidence, including the exhaustive Core.IC lifecycle and Lemma K.BF-Persist; and K.CompleteS. For a successful clean discharge, K.ResetEpoch.Continue is the normal prospective reset evidence, and only on that branch K.TerminalDisposition.Continue is the normal prospective terminal evidence; both are proved contract-wise over the applicable frozen-prefix classes rather than inferred from absence of observed failures. Either DirectFail branch may be proved conditionally for a hypothetical candidate or instantiated to classify a direct failure; a concrete DirectFail instantiation is itself a K.RespFail witness and cannot coexist with K.RespClean on a successful clean run. The generic direct-failure payload shape binds P, f_P, x_P^A, g_P with $g_P \leq_{src} f_P$ and an already-triggered pending non-waived monitor independently of K.2b.R/P/Q Step 1. The ResetEpoch.DirectFail specialization additionally binds the common pre-reset epoch, matched affecting RESET in CycleResult.resetUnits, and RW2 clearing of that exact monitor, yielding cancelled-pending. The TerminalDisposition.DirectFail specialization instead binds the independently validated declared maximal terminal state while that monitor remains pending, yielding maximal-pending. CF-S also records timer progression through CycleResult.nextState, complete-run results, K.EnvMF applicability, and waiver preservation. Checker obligations include validating the K.CVPred provenance equations and K.CV-WF/rank certificate used by K.2b.P1; validating the common gate schema and its K.BF exhaustiveness once, validating each certificate-supplied K.DomResetScope against D1–D4 and its associated DomEpoch, each instantiation's Interrupt predicate and DirectFail specialization, the required reset-first nesting, and rejection of any clean candidate for which a required gate instantiation satisfies neither branch.

Definition K.A5Cert (A5-L discharge package). The package consists of K.CertHdr plus exactly one declared route payload: CF-S; a proof reference for K.Str-1; CF-1; CF-2; CF-3; CF-4; CF-5; or a direct Policy-L invariant/exploration/proof artifact. The package records the route and target property and shall expose enough data for an independent checker to validate the route without trusting a tool-emitted conclusion string. Policy-S route packages target K.RespClean and include dominance, coverage, finite-bound, waiver-preservation, and complete-run evidence; direct Policy-L packages target Dead_K^W. Optional diagnostic environment-terminal records do not substitute for the response payload.

CF-1 — Rank certificate. A function r from processes (or process–boundary pairs) of the selected instance to a declared well-ordered set, together with the Definition K.Dep edge enumeration, such that every realizable edge strictly decreases r (Theorem K.Str-2). Checker obligations: the edge enumeration covers data, capacity back-pressure, rendezvous, epoch-gate, and arbiter edges of the selected instance including its elaboration and lowering; every non-excluded edge strictly decreases r ; every excluded edge carries a recorded invariant proof of unrealizability.

CF-2 — Credit / cycle-breaking certificate. For each cycle of the full dependency graph G_{dep} , provide an invariant—such as token presence, occupancy bounds, or a circulating-credit invariant—establishing that at least one edge of the cycle is unrealizable as a WFG edge at any reachable cutpoint (Theorem K.Str-3). Checker obligations: the dependency-cycle enumeration is complete; each invariant is inductive over the selected instance; and the proved edge exclusion applies at the explicit capacities and elaborated boundaries. Petri-net or siphon analyses may be used by a tool as one way to generate this certificate, but no Petri-net equivalence theorem is part of normative Appendix K.

CF-3 — Inductive-invariant certificate. Supply an abstraction map α from the selected operational transition system into a finite abstraction A . For direct Policy-L verification, the observation component shall be KObs, or an explicitly proved quotient sufficient to reconstruct DeadKRec^W . For the Policy-S response route, A shall additionally carry the monitor activation, dynamic attribution, age, bound-class, reset/terminal-disposition, and maximality state needed to reconstruct $K.\text{RespFail}$; KObs alone is intentionally insufficient. Supply an invariant Inv such that $\alpha(\text{Init})$ is contained in Inv , $\text{Post}_A(\text{Inv})$ is contained in Inv , and Inv excludes the route’s bad event. Checker obligations: α and Post_A soundly overapproximate the selected operational relation; every cutpoint or monitor fact is reconstructed from the appropriate observation/monitor components and ambient instance data; reset/environment/waiver distinctions are represented; witness-selected non-blocking outcomes are source-describable through NB-CF or K.Low; and all invariant inclusions are mechanically validated. Equality of KObs records alone shall not be treated as transition equivalence or as proof of future Issue legality, fairness, DrainDiscipline, or response timing.

CF-4 — Exploration certificate. Record an exhaustive or soundly bounded model-checking result over the selected Policy-L relation establishing unreachability of DeadKRec^W , or over the selected Policy-S relation establishing absence of $K.\text{RespFail}$, with $K.\text{CertHdr}$, the explored transition-system identity, abstraction map, cutpoint normalization rule, monitor semantics, tool/version, bound or completeness justification, fairness interpretation, and coverage argument. The checker shall recompute Dead_K from KObs for the direct route and shall recompute monitor outcomes from the augmented Policy-S state for the response route. A finite exploration depth or simulation prefix without a proof of completeness is supporting evidence only.

CF-5 — Tool-emitted certificate. Any CF-S or CF-1 through CF-4 emitted by the HLS tool or a companion tool over the selected instance may exploit the tool’s knowledge of the scheduled control graph, stage allocation, actual FIFO and skid-buffer capacities, request ownership, join and arbitration structure, automatic-flush implementation, and hidden storage or credits. It shall use $K.\text{CertHdr}$ and the applicable CF-0/A5 payload schema and be validated by an independent checker whose trusted base is recorded. The checker shall validate structure, evidence references, and reconstructed K -facing predicates, not merely the certificate generator’s final claim. Labeling rule (normative): a certificate stated over the generated RTL itself, rather than over the selected Sys_ref instance, is admissible evidence of deadlock-freedom for that design but bypasses the Appendix K preservation architecture — it is output verification, not a discharge of A5-L — and shall be labeled as such in the record.

Soundness restriction (normative). An arbitrary eventuality claim—such as ‘the arbiter eventually grants’—does not by itself discharge A5-L, because A5-L prohibits reaching the closed wait-cycle cutpoint. The $K.\text{Resp}$ route is different: Lemma K.2b proves that any such liberal cutpoint forces a permanent covered serial frontier and therefore a finite timeout/maximal-pending or matched-reset

cancelled-pending event under a certified finite bound. Only response properties satisfying $K.\text{RespCov}$, the $K.2b$ trigger mapping, complete-run checking, and waiver preservation may be used in Corollary $K.2c$. An unproved timeout threshold or a finite clean simulation is not a certificate.

K.4f The Policy-S Standard Fragment (K.PSF)

Definition K.PSF (Policy-S standard applicability fragment). A selected instance is in K.PSF when it is in Sys_K form; channels are point-to-point with finite B-faithful capacities; SDet holds for the concrete Policy-S simulator; $K.CV$ holds, including $K.CVPred/K.CV\text{-}WF$, and arbiters are deterministic and part of that simulator semantics; $K.\text{EnvMF}$ holds; the shared $K.\text{InterruptionGate}$ schema is fixed with a certificate-bound $K.\text{DomResetScope}/\text{DomEpoch}$ for every candidate, $K.\text{ResetEpoch}$ holds for every candidate component, and $K.\text{TerminalDisposition}$ holds only for candidates on $K.\text{ResetEpoch}.\text{Continue}$; requests are nonwithdrawable under the exhaustive Core.IC lifecycle, with Lemma $K.BF\text{-}Persist$ supplying the frozen-frontier specialization for every WFG-relevant $K.BF$ blocking request; $K\epsilon/K.\text{Drain}$ hold; and a finite-bound $K.\text{Resp}$ configuration with total $K.\text{RespCov}$ is supplied. Completeness, fairness, and $K.\text{RespClean}$ of p_S^{det} belong to $A5\text{-}S/\text{CF}\text{-}S$.

Theorem K.PSF-1. For a K.PSF instance, the SDet -selected Policy-S execution p_S^{det} establishes $A5L_Sys_K(I, W_K)$, and hence $A5\text{-}L$ by Lemma $K.Low.A5$, when $\text{CF}\text{-}S$ supplies $K.\text{CompleteS}$ and $K.\text{RespClean}$ for the totally covered finite-bound monitor set. Proof: Lemma $K.2b$, Corollary $K.2c$, and $K.Low.A5$. $\square$

Intent. K.PSF is the recommended standardized applicability target for new latency-insensitive designs. The general user-facing evidence is the conservative Policy-S execution checked by complete process-facing bounded-response obligations. For finite transactions and bounded hardware protocols, these are ordinary timeout/service assertions and are expected to be highly automatable. A correct design with fair but genuinely unbounded response gaps lies outside this serial fragment and must use a structural, direct Policy-L, invariant, exploration, or tool-certificate route. $\text{Dead_KS}/K.\text{EnvOrd}/K.\text{Done}$ remain optional diagnostic aids rather than primary proof premises.

K.5 Theorems K.1A, K.1A-Total, and K.1 — Reachable-Cutpoint Deadlock-Freedom Preservation (“Liveness Preservation”)

Theorem K.1A (certified-cutpoint preservation from the scheduler-robust premise). Statement: Assume $A1\text{--}A4$ and $A6\text{--}A11$, $\text{WC}/\text{Core.RegSeq}$, the Appendix I/J correspondence prerequisites, and the point-to-point evaluated-boundary condition (the $A6$ ownership premise applied at every explicit boundary on which $KObs/WFG$ facts are evaluated). Fix one $p_post \in K\text{CutReach_post}(I)$ and one recorded reachable-prefix/normalization pair (τ_post, v_post) with $\text{NormPost}_I(\tau_post, v_post, p_post)$ whose package satisfies $\text{CertifiedCutpoint}_I(\tau_post, v_post, p_post)$. Thus Corollary J.1 yields equal $KObs$ for p_post and σ_ref ; Corollary J.RegPhaseReach-Comp supplies $J.\text{RegPhaseReach}$ for the exact source prefix on the normal compositional route (or the direct package supplies the equivalent fact); and $K.\text{DrainReach}$ proves nonwithdrawal, $K\epsilon$, and $\text{Core.Drain}/K.\text{Drain}$. Their conjunction $K.\text{RLAdmReach}$ places that same source prefix in $\text{Pref}_L^{\text{adm}}(I)$. If W is non-empty, also assume $KObsClosed_I, E, w(W)$. If $A5\text{-}L$ holds for the selected Sys_ref instance, then p_post does not satisfy DeadKRec^W , equivalently the Dead_K^W predicate of the selected theorem instance. With $W = \emptyset$, the fixed certified cutpoint is not an internal message-passing-deadlock cutpoint. The theorem consumes neither $K.CV$, NB-Align , nor Lemma $K.2b$, and it makes no $K.\text{CertCov}$ or package-totality claim.

Proof (by contradiction).

1. Assume, for contradiction, that the fixed certified RTL K -facing cutpoint $p_post \in K\text{CutReach_post}(I)$ has K -facing reconstructed observation K_post satisfying $\text{DeadKRec}^W(K_post, I, W)$. By Definitions $K.\text{Dead}$ and $K.\text{DeadW}$ and the reconstruction interface,

- this includes a non-empty blocked closed WFG component and DrainClosedNow for that component. The asserted RTL requests and channel facts used in this observation are covered by J.OfferConf and B-faithfulness/E4; no unrecorded raw RTL request may be omitted.
2. By CertifiedCutpoint_I($\tau_{\text{post}}, v_{\text{post}}, p_{\text{post}}$), apply the recorded Corollary J.1 instance to this same finite prefix and package. It supplies the reachable source K-facing cutpoint $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$ and $K_{\text{ref}} = \text{KObs_E}, w(\sigma_{\text{ref}})$ such that $K_{\text{post}} = K_{\text{ref}}$. J.OfferReach ensures that σ_{ref} is the endpoint of the selected reachable source witness rather than an arbitrary or unrealizable record.
 3. By Lemma K.0, Lemma K.2, and Lemma K.2a — equivalently, by functional extensionality of the reconstruction functions — $\text{DeadKRec}^W(K_{\text{ref}}, I, W)$ has the same truth value as $\text{DeadKRec}^W(K_{\text{post}}, I, W)$. When W is non-empty, $\text{KObsClosed_I}, E, w(W)$ makes the waiver predicate well-defined on the common observation, and Lemma K.KObs-Dead identifies the reconstructed source predicate with $\text{Dead_K}^W(\sigma_{\text{ref}})$. Hence $\text{Dead_K}^W(\sigma_{\text{ref}})$ holds.
 4. The same K.CertifiedCutpoint package supplies theorem-derived/direct J.RegPhaseReach and explicit K.DrainReach for the exact Corollary J.1 witness. The former proves Policy-L legality and L0–L4 phase ordering by construction; the latter proves nonwithdrawal, $K\epsilon$, and every activated Core.Drain/K.Drain clause. Therefore K.RLAdmReach establishes $\tau_{\text{ref}} \in \text{Pref_L}^{\text{adm}}(I)$. No phase inference is made from equal KObs, and no drain inference is made from J.RegPhaseNF alone. Thus σ_{ref} is the reachable Core.LAdm-admissible cutpoint prohibited by A5-L.
 5. This contradicts A5-L. Therefore the fixed certified cutpoint p_{post} cannot satisfy Dead_K^W .
-

Mechanization note. The proof core is Corollary J.1, equality of typed KObs, pure reconstruction of Dead_K^W , theorem-derived/direct J.RegPhaseReach, and explicit K.DrainReach for the exact witness. Corollary J.RegPhaseReach-Comp proves J.RegPhaseReach upstream from Core.RegSeq, enriched holes, field-sensitive footprints, J.RegPhaseNF, and bucketed J.UnitExt; Theorem K.1A consumes only the composed K.RLAdmReach fact. K.Drain continuation evidence is not derived from KObs or phase normality. Package totality remains outside this theorem.

Corollary K.1A-Total (universal reachable-cutpoint preservation under K.CertCov and KCut closure). Assume the premises of Theorem K.1A hold uniformly for the selected instance, assume $\text{Core.KCutClosure_post}(I)$, and assume $K.\text{CertCov}(I)$. Then no $p_{\text{post}} \in \text{KCutReach_post}(I)$ satisfies Dead_K^W . With $W = \emptyset$, RTL_B is internally message-passing-deadlock-free over the complete reachable K-facing cutpoint domain, and by Lemma Core.KCutObserve a persistent K-relevant blocked configuration cannot evade that domain solely through endless pre-normalization ϵ activity. Standard QSync instantiation. In the standard theorem scope, Theorem Core.QSync-Post supplies $\text{Core.KCutClosure_post}$ from the accepted structural QSync evidence; the corollary retains the closure premise as the downstream semantic interface. Proof. $\text{Core.KCutClosure_post}$ supplies a finite KCut for every reachable RTL K-facing cycle-evaluation origin; $K\epsilon$ preserves any blocker that persists through that normalization. Any alleged prohibited KCut has a K.CertifiedCutpoint package by K.CertCov and is therefore excluded by Theorem K.1A. □

Theorem K.1 (user-facing deterministic Policy-S preservation under cutpoint closure and certificate coverage). Assume Theorem K.1A's per-cutpoint premises uniformly, $\text{Core.KCutClosure_post}(I)$, $K.\text{CertCov}$, SDet , $K.\text{CV}$, NB-Align/K.Low where applicable, point-to-point boundaries, nonwithdrawal, $K\epsilon$, K.Drain, K.EnvMF, the shared K.InterruptionGate schema with K.ResetEpoch and nested K.TerminalDisposition applicability, and K.RespCov. If A5-S holds for the SDet -selected execution $p_{\text{S}^{\text{det}}}$ —supplying K.CompleteS and K.RespClean—then RTL_B cannot reach a persistent K-relevant internal message-passing blocking configuration whose normalized K-facing observation satisfies Dead_K^W . Equivalently, no reachable RTL KCut satisfies Dead_K^W , and KCut closure prevents persistent blockers from remaining forever outside that observation domain. Proof: K.2c gives A5-L; in

the standard scope `Core.QSync-Post` derives `Core.KCutClosure_post`, and `Core.KCutClosure_post`, `K.CertCov`, and Corollary K.1A-Total give the universal RTL conclusion. Without `K.CertCov`, the same A5-L discharge supports only the certified-cutpoint conclusion of Theorem K.1A; without `KCutClosure_post`, no universal claim is made that persistent pre-normalization RTL blocking states become observable as KCuts. The Policy-S form makes no claim for timed/reactive/nondefault multi-frontier components or for candidates for which a required nested gate instantiation satisfies neither `Continue` nor `DirectFail`. □

Implementation status. The operational Policy-S check is specified in terms of conventional pre-HLS simulation and bounded-response assertions and is expected to be highly automatable for finite transactions and bounded protocols. Treating a clean run as a verified discharge of A5-L retains proposed-reduction status until the K-P proof obligations and required certificate interfaces are mechanized and qualified. A universal RTL conclusion additionally requires independently checked `Core.QSync` structural evidence (from which `Core.QSync-Post` derives `Core.KCutClosure_post`) and an independently checked `K.CertCov` discharge. The former establishes K-facing cutpoint existence/non-vacuity through a generic theorem; the latter supplies total certificate coverage over that domain.

Discharge routes for A5-L (normative summary). Primary: an `SDet`-qualified CF-S certificate and K.2c from A5-S for the `SDet`-selected p_S^{det} , including `K.CompleteS`, `K.RespCov`, and `K.RespClean`. Alternatives are structural, invariant, exploration, CF-5, or direct Policy-L artifacts. The header records the selected normalized instance and, when Policy S is used, the deterministic simulator/scheduler, arbitration, Policy-S issue semantics, normalization identity, `SDet`, response-monitor schema and finite bounds, completeness/fairness, waivers, and trust boundary.

Appendix K-P – Proofs Companion for the Serial-Reduction Route

Status. This companion collects the deterministic serial-reduction proof. SDet fixes the selected functional Policy-S semantics and one run artifact $\rho_{\text{S}}^{\text{det}}$ at the outset; K.CompleteS separately establishes that this artifact is maximal, fair, and complete. Below write $\rho_{\text{S}} := \rho_{\text{S}}^{\text{det}}$. The soundness-critical proof retains K.Low.C , identity/dataflow lemmas, serial-prefix reachability, progress dominance, joint frontier freeze, and finite bounded-response extraction. The former missing-supply owner trichotomy, stuck-owner walk, EnvStop/DoneStop closure, and K.EnvOrd/K.Done collapse are retired from the trusted route and remain at most diagnostic. Optional serial replay is unchanged.

K-P KObs interface convention. For every source K-facing cutpoint $\sigma \in \text{KCutReach_Sys_K(I)}$ used below, write $\text{K}_{\sigma} := \text{KObs_E,w}(\sigma) = \langle \text{E,w,H}_{\sigma}, \text{O}_{\sigma} \rangle$, where $\text{O}_{\sigma} = \text{ResOffers_E,w}(\sigma)$. For $\text{K} = \langle \text{E,w,H,O} \rangle$, use owner(o) , item(o) , op(o) , boundary(o) , dir(o) , and epoch(o) for the projections of the attributed residual-offer tuple $\text{o} = \langle \text{P,x,q,c,dir,e} \rangle$ of Definition Core.18b. Write Proc(I) for the finite process set of the selected instance. For a process set D define $\text{Offers_D(K)} := \{\text{o} \in \text{O} \mid \text{owner(o)} \in \text{D}\}$; $\text{FrontOffers_D(K)} := \{\text{o} \in \text{Offers_D(K)} \mid \text{item(o)} \in \text{FrontRec_owner(o)(K)}\}$; and DrainBag_D(K) as the finite multiset of offers $\text{o} \in \text{Offers_D(K)}$ whose item lies outside $\text{FrontRec_owner(o)(K)}$. $\text{DrainCandidates_D(K)}$ is the submultiset of DrainBag_D(K) whose item is ChannelEnabledRec at K . CmtRec_P(K) , PushOrdRec_c(K) , and PopOrdRec_c(K) denote the committed-identity and per-endpoint ordinal projections of H . These names are pure snapshot selectors; at an operational cutpoint K_{σ} they agree with the corresponding Sys_K source facts by Lemmas $\text{J.KObs-Proc-J.KObs-WFG}$ and K.KObs-Dead .

Operational and elaboration boundary. The graph and residual-offer arguments in this appendix range over the full E-typed frontier items and tuples, including explicit $\text{Sys_B}^{\text{elab}}$ staged, bundle, join, guard, and epoch-gate boundaries. When the proof uses an operation symbol f_P , g_P , or q , it abbreviates MsgOf_P(x) only after the corresponding typed item x and boundary have been fixed; the item identity, boundary, direction, epoch, and owner remain part of the formal object. KObs deliberately omits historical Issue times and does not determine future issue legality. $\text{Core.Drain/K.Drain}$ remains the continuation-level DrainDiscipline used to forbid replenishment past a blocked frontier.

DrainClosedNowRec and DrainCandidates are only cutpoint predicates. Channel-side E4 movement inside the selected elaboration is handled by B4/B5 ϵ -normalization and is not duplicated as a source residual offer.

Lemma K.Low.C (normalization correspondence) — restated with proof. For the same fixed stimulus and witness choices: (i) every admissible execution of $\Lambda_{\text{W}}(\text{Sys_ref})$ projects, by erasing guard-channel and epoch-gate transitions, to an admissible execution of Sys_ref with the same issue-policy class (S or L), the same committed message/synchronization/observation history, and the same Σ projection; (ii) every admissible execution of Sys_ref in which each witness-selected site takes its W -selected outcome lifts to an admissible execution of $\Lambda_{\text{W}}(\text{Sys_ref})$ in which each guard Pop commits within a finite WFG-inert ϵ -delay of the commit of its modeled obstruction — in the default $\text{B(g_s)} = 1$ form, in the cycle after that commit once the guard request is asserted; exactly at that commit only in a declared atomic rendezvous variant; (iii) a reachable source K-facing cutpoint $\sigma_{\text{K}} \in \text{KCutReach_Sys_K(I)}$ of $\Lambda_{\text{W}}(\text{Sys_ref}) = \text{Sys_K}$ satisfies Dead_K if and only if its lowering projection σ_{ref} lies in KCutReach_ref(I) and contains a closed internal wait cycle in the extended sense — a non-empty process set closed under waiting through internal message frontiers, infeasible NB-aligned selected outcomes, and pending explicit synchronizations. Proof. Parts (i) and (ii) are per-rule stuttering simulations: the added transitions touch only the fresh channels; by NB-Align and K.CV , W 's selected outcome at each lowered site is a deterministic function of the commit of its modeled obstruction, so the guarded process's post-guard behavior is identical in both models; clause (7) already requires synchronization refusal to be represented as ChannelDisabled at an explicit boundary, so the synchronization/epoch-gate lowering rule is representational rather than semantic; point-to-point ownership and E3 prefix closure identify

the unique g_s producer; $K\epsilon$ keeps the added microsteps WFG-inert. The guard skew of the default $B(g_s) = 1$ form is absorbed by the same stuttering simulations: during the skew the guard Pop is channel-enabled with its unique producer already committed, so it is WFG-inert and is a drain candidate, and therefore no drain-closed KCut analyzed by part (iii) contains a half-completed guard; an exactly-at reading would be inconsistent with the non-fall-through E4 semantics for positive-capacity channels. For (iii), left to right: a member blocked at a guard or epoch-gate frontier projects to a process whose W -selected outcome or synchronization is infeasible, and the unique complement owner of that frontier projects to the modeled supplier, so a lowered literal WFG cycle projects to a closed wait cycle in the extended sense; right to left: each frozen or waiting node of an extended cycle is, in the lowered instance, message-blocked at its guard or epoch-gate frontier with its complement owner the image of its modeled supplier, so the extended cycle is a literal all-message-blocked $Dead_K$ component in $\Lambda_W(Sys_ref)$. ■ Mechanization note. Lemma K.Low.C is the single successor to the former obstruction-pointer conversion obligation. It shall be discharged per lowering rule — one case lemma for the epoch-gate rule and one for the guard rule — as a local correspondence at each lowered site, not as a global argument over terminal cutpoints. Part (iii) is the load-bearing claim; a development that mechanizes only the blocking-only Lemma K.2b without K.Low.C(iii) covers only designs natively in the blocking-only fragment.

Definition K.Id (committed-identity state and serial frontier). For an execution prefix p and process P , $Cmt_P(p)$ is the set of dynamic source-operation identities of P that have process-facing committed in p , and $Push_c(p)$, $Pop_c(p)$ are the per-endpoint commit ordinals at channel c . At a terminal cutpoint σ with $K_\sigma = \langle E, w, H_\sigma, O_\sigma \rangle$, write $CmtRec_P(K_\sigma)$, $PushOrdRec_c(K_\sigma)$, and $PopOrdRec_c(K_\sigma)$ for the same facts reconstructed from H_σ . Because operations on a single message-passing interface issue and commit in source order (R4/E3 with FIFO transfer order and point-to-point ownership), the per-channel per-endpoint commit ordinal of any fixed dynamic operation is the same in every admissible execution in which it commits; per-channel counts therefore identify committed transfers by identity through Lemma K.2b.P1. Across distinct interfaces, however, Policy L may commit a source-later operation while a source-earlier operation remains uncommitted, so $Cmt_P(p_L)$ need not be a source-order prefix, and a scalar commit count identifies neither the committed set nor the frontier under Policy L. Under Policy S, $Cmt_P(p_S)$ is always a source-order prefix of P 's dynamic operation sequence; in the blocking-only fragment every commit advances the frontier, and $SerFront_P(p_S)$ — P 's serial frontier — is its source-minimal uncommitted blocking operation, with every source-earlier operation committed. All cross-policy progress comparisons identify dynamic operations through Definition K.OpKey and remain statements over operational prefixes, keyed committed-identity sets, and per-channel ordinals; the reconstructed KObs projections are used only when a terminal cutpoint is named. Scalar counts are consumed only as inequalities (Lemma K.2b.P). The earlier identification of a process's liberal frontier as operation $prog_P + 1$ was unsound and remains retired.

Definition K.CVPred and K.CV-WF (well-founded value/identity provenance). Fix a K.BF theorem instance I , its common OpKey_ I universe of Definition K.OpKey, and its K.CV discharge. Let CVFact_ I be the common semantic fact-key universe used by the K.CV proof. It contains at least OpFact(κ) for $\kappa \in OpKey_I$, whose deterministic value is an optional typed operation descriptor: none when the fixed predecessor/control facts exclude that keyed occurrence, and some(desc) when κ is selected, with desc recording operation kind/site identity, typed boundary/direction, and any Push payload; ObsFact(u), recording the value or status of each explicitly modeled observation that K.CV permits to influence a later operation; and XferFact(c, n), the distinguished ObsFact for the payload of the n -th committed transfer at explicit boundary c . Fixed constants, fixed stimulus values, and witness-fixed choices are roots rather than recursive facts. For an admissible execution p , Occurs_ $p(\kappa)$ means Realize_ $p(\kappa)$ is defined; whenever Occurs_ $p(\kappa)$, OpFact(κ)=some(desc) for that realization. The K.CV certificate supplies, for every non-root fact b , a finite predecessor set Pred_ $CV(b)$ and a deterministic evaluator Eval_ b .

Occurrence closure is explicit: whenever a fact b occurs in an admissible execution, every $a \in \text{Pred_CV}(b)$ also occurs in that execution; and whenever the predecessor facts are available, Eval_b determines the same fact value from the fixed roots and those keyed predecessor values. For $\text{OpFact}(\kappa)$, this evaluator includes the control/branch/iteration selection needed to determine none versus some(desc), not merely the fields of an already-assumed occurrence. The certificate shall validate this provenance against $\text{Core.RegSeq/Core.ReachPrep}$ and the applicable explicit causal mechanisms; for $\text{XferFact}(c,n)$, it also validates the producer OpKey attribution and ordinal relation using point-to-point ownership, R4/E3, E4/FIFO or rendezvous semantics, and the fixed elaboration. Define $a \prec_{\text{CV}} b$ iff $a \in \text{Pred_CV}(b)$. The same OpKey/CVFact keys, predecessor sets, and evaluators are bound to the fixed source, elaboration, stimulus, witness, and reset schema across the admissible executions compared by Lemmas K.2b.P0/P1; they are not reconstructed from incidental commit timing. The K.CV discharge shall additionally supply or derive K.CV-WF: WellFounded($\prec_{\text{CV}}$). A checked CVRank into the natural numbers or another well-founded lexicographic order, with $a \prec_{\text{CV}} b$ implying $\text{CVRank}(a) < \text{CVRank}(b)$, is a sufficient certificate form. No $\prec_{\text{CV}}$ edge is introduced merely because two requests mutually enable a rendezvous, share a cycle, or are related by the broad Appendix-G causal-dependency graph $\rightarrow$. Thus a legal cycle in $\rightarrow$ does not imply a cycle in $\prec_{\text{CV}}$. Conversely, an actual circular value/control provenance for which no K.CV-WF certificate exists is outside K.CV and therefore outside the K.2b/K.2c serial-reduction route.

Lemma K.2b.P1 (keyed dataflow determinacy) — restated with proof. In the blocking-only fragment, for the fixed instance, stimulus, witness choices, and OpKey_I universe: for every $\kappa \in \text{OpKey_I}$, any two admissible executions—of either policy—in which $\text{Occurs}_p(\kappa)$ holds realize κ with the same operation kind/site identity, typed boundary/direction, and any K.CV-determined Push payload; and for every channel c and ordinal n , the payload of the n -th committed transfer at c is the same in every admissible execution in which that transfer fact occurs. Proof. Strengthen the induction claim to all facts in CVFact_I and perform well-founded induction on $\prec_{\text{CV}}$ using K.CV-WF; this induction does not use the broad causal-dependency graph $\rightarrow$. Fix a fact b and assume the execution-independent equality claim for every $a \prec_{\text{CV}} b$. By provenance adequacy in Definition K.CVPred, every admissible execution in which b occurs evaluates b with the same deterministic Eval_b from the same fixed roots and the same keyed predecessor facts $\text{Pred_CV}(b)$. Occurrence closure supplies an occurrence of every $a \in \text{Pred_CV}(b)$ in each such execution; each predecessor has smaller $\prec_{\text{CV}}$ rank and therefore has the same value/identity by the induction hypothesis. The fixed roots are identical by the theorem-instance stimulus/witness binding. Hence b has the same value/identity in every execution in which it occurs. For $\text{OpFact}(\kappa)$, this yields the same typed realization fields and any Push payload for the common operation key κ ; for $\text{XferFact}(c,n)$, the certified producer-key attribution plus point-to-point ownership, R4/E3, and E4/FIFO or rendezvous ordinal semantics make the n -th transfer payload the deterministic value of its lower-ranked producer/provenance facts. Other K.CV observation kinds are handled by their explicitly modeled deterministic Eval_b equations. Same-cycle rendezvous request coincidence is an enablement condition only and is not a $\prec_{\text{CV}}$ value predecessor, so it creates no induction cycle. $\square$ This lemma is deliberately conditional on occurrence; existence/reachability of the corresponding operation in the other execution is the separate content of Lemma K.2b.P0. At any named terminal cutpoint, the determined committed keys, payload ordinals, and process-local replay facts are exactly the corresponding projections of H in its KObs record; this observation does not replace the operational induction.

Lemma K.2b.P0 (keyed next-operation occurrence and reachability) — Statement. Fix a common key $\kappa \in \text{OpKey_I}$ owned by process P that is reached in the selected Policy-S execution, and fix Policy-S/Policy-L comparison prefixes π_S and π_L within the same K.DomResetScope DomEpoch. Assume the K.2b comparison invariant supplies in π_L every K.CV predecessor fact required by $\text{OpFact}(\kappa)$, with the same keyed values as in π_S ; every source-earlier key that Policy S requires to complete before κ may issue

has committed as required; and no still-frozen Core.16 frontier of P is key-earlier than κ . Then either κ is already realized/reached in π_L or π_L has a finite phase-legal Policy-L extension, within the same DomEpoch, that reaches $q_L := \text{Realize}(\kappa)$. The realized q_L has the operation kind/site, typed boundary/direction, and payload determined by $\text{OpFact}(\kappa)$. This lemma establishes occurrence/reachability only; request assertion and persistence are supplied by K.2b.R.

Proof. Lemma K.2b.P1 alone cannot prove this conclusion because P1 is conditional on occurrence. Here the predecessor-agreement hypothesis makes every input of $\text{Eval_OpFact}(\kappa)$ available in the liberal execution. Because the selected Policy-S execution reaches κ , its evaluator value is $\text{some}(\text{desc})$; determinism of the common K.CV evaluator therefore gives the same $\text{some}(\text{desc})$ value from the equal keyed predecessors on the liberal side, establishing that the same keyed occurrence is control-selected rather than an untaken-branch artifact. Definition K.OpKey supplies that potential process-local occurrence independently of execution, and realization preserves source order. Core.ReachPrep together with K.BF bounded local progress advances P through the finite dependency-independent local computation needed to reach the selected occurrence; Core.CycleChoiceExt supplies a typed completion whenever the finite reach segment crosses a source-cycle boundary. The only relevant blocked-frontier obstruction would be a Core.16 frontier key-earlier than κ , which is excluded by hypothesis. Therefore κ has the claimed finite liberal realization/reach extension, with the typed fields/value fixed by $\text{OpFact}(\kappa)$ and K.CV/P1. $\square$

Lemma K.2b.R (serial-prefix request reachability) — Statement. Under K.ResetEpoch.Continue and K.TerminalDisposition.Continue, let D be the frozen component associated with the infinite fair continuation $\bar{p}_L$ of Lemma K.2b.E, and let q_S be a blocking endpoint operation of process P reached by the selected Policy-S execution, with common key $\kappa := \text{Key}(q_S)$. By the definition of K.DomResetScope, P and q_S 's explicit boundary belong to the dominance scope whenever κ is consumed by this comparison; let $e_{\{P,\kappa\}}$ denote the corresponding endpoint/boundary reset epoch within the fixed DomEpoch. Fix Policy-S and Policy-L comparison prefixes π_S and π_L immediately before a first alleged Policy-S overtake, with π_L a prefix of $\bar{p}_L$. Assume every source-earlier key that Policy S requires before κ , every endpoint transfer earlier than κ in the fixed serial commit order and K.2b.P dominance domain, and every K.CV predecessor fact required by $\text{OpFact}(\kappa)$ are present in π_L with the keyed identities/values supplied by the comparison invariant. If κ 's liberal realization has not committed in π_L , then $\bar{p}_L$ has a finite extension of π_L , with the same DomEpoch and endpoint epoch, in which the attributed endpoint request for $q_L := \text{Realize}(\kappa)$ is asserted. On this K.BF reset-free, terminal-free comparison branch, once asserted, Lemma K.BF-Persist retains that request as the same pending keyed dynamic request until q_L commits. No reachability or nonwithdrawal claim crosses a dominance-affecting RESET or a declared-terminal branch; K.ResetEpoch.DirectFail and K.TerminalDisposition.DirectFail are handled directly through K.RespFail.

Proof. Under Policy S, every source-earlier blocking operation of P required by the serial discipline has committed before q_S 's request may issue. By the comparison hypothesis, those committed operation keys, returned values, earlier endpoint-transfer facts, and the remaining K.CV predecessors of $\text{OpFact}(\kappa)$ also occur with the same keyed values in the liberal prefix. Lemma K.2b.P0 therefore supplies the corresponding liberal realization $q_L := \text{Realize}(\kappa)$, either already reached or after a finite phase-legal local extension; Lemma K.2b.P1/K.CV supplies its common typed identity fields and payload. Because P and q_L 's explicit boundary belong to K.DomResetScope and this comparison remains within the fixed DomEpoch, q_L is in the corresponding endpoint/boundary reset epoch $e_{\{P,\kappa\}}$. If π_L has already issued q_L , nonwithdrawal gives the conclusion. Otherwise Core.16 does not prohibit its advance or assertion. If $P \notin D$, no component-freeze restriction applies to P. If $P \in D$, issue legality requires that no $x \in \text{Front}_P(\sigma_L)$ have $\text{Key}(\text{MsgOf}_P(x)) <^{\wedge K_src} \kappa$. In the single-frontier branch, E3 prefix closure and the comparison invariant force κ to be the key of the selected frontier operation f_P , whose attributed request is already present at the freeze cutpoint. In the certified StandardEnv/B1-avail multi-frontier

branch, the joint-frontier comparison tracked by K-O2 establishes that κ is not source-later than any member of $\text{Front_P}(\sigma_L)$, so the Core.16 prohibition likewise does not reach q_L . Thus q_L is either already asserted or remains legally IssueEnabled under Core.16. Policy L admits the issue because it admits every Policy-S issue and assertion of a persistent blocking request does not require the channel transfer itself already to be enabled. At every finite comparison prefix on which the relevant frozen requests remain pending, P and q_L 's explicit boundary are members of $K.\text{DomResetScope}$ by the comparison-domain definition, so $K.\text{ResetEpoch.Continue}$ forbids a RESET affecting either as the next transition and preserves the same DomEpoch and $e_{\{P,\kappa\}}$; $K.\text{TerminalDisposition.Continue}$ makes the corresponding frozen-prefix final state nonterminal. Because q_L remains legally IssueEnabled at every applicable L1 opportunity, Core.IssueFair therefore schedules the assertion after finitely many such opportunities; the infinite fair continuation cannot postpone it forever. Lemma K.BF-Persist together with nonwithdrawal then retains the attributed request as the same pending keyed dynamic instance until q_L commits: the exhaustive Core.IC lifecycle admits no independent nonterminal withdrawal/cancellation transition, $K.\text{ResetEpoch.Continue}$ excludes an affecting RESET on the comparison prefix, and $K.\text{TerminalDisposition.Continue}$ excludes a declared-terminal escape. RW2 would end the realized dynamic instance at an affecting RESET, which is why the conclusion is expressly DomEpoch- and endpoint-epoch-local and $K.\text{ResetEpoch.DirectFail}$ is handled separately. $\square$

Lemma K.2b.P (deterministic serial progress dominance) — Statement. Under $K.\text{ResetEpoch.Continue}$ and $K.\text{TerminalDisposition.Continue}$, fix a Policy-L prefix and the infinite fair reset-free, terminal-free continuation $\bar{p}_L$ supplied by Lemma K.2b.E, and let p_S^{det} be the Policy-S execution selected by SDet, with separate $K.\text{CompleteS}$ evidence establishing maximality, fairness, and completeness. Every endpoint-transfer identity and payload committed in p_S^{det} whose owning endpoints/boundary belong to $K.\text{DomResetScope}$ for this candidate, while the comparison remains in the fixed DomEpoch, also occurs in $\bar{p}_L$. The conclusion is transfer inclusion over the dominance comparison footprint; it does not assert identical cycle timing, the same global interleaving, or cross-epoch inclusion for reset activity framed outside that scope.

Proof. Order the endpoint-transfer commits of p_S^{det} in the $K.\text{DomResetScope}$ dominance domain and fixed DomEpoch by operational transition index, using the fixed normalization and boundary-action order to break any same-cycle ties. The tie-break orders identities only for the least-missing-transfer induction: every occupancy and enablement fact below is evaluated from the common begin-of-cycle state, so a same-cycle Push cannot supply a Pop and a same-cycle Pop cannot create capacity for a Push under non-fall-through E4. Suppose the conclusion is false and choose the least serial transfer T absent from the complete liberal continuation. Let c be the explicit E-typed abstract boundary named by T in Sys_K —including any staged, bundle, join, guard, or epoch-gate boundary introduced by elaboration or lowering—and let n be T's ordinal in the applicable endpoint direction and reset epoch. Every earlier serial transfer occurs in $\bar{p}_L$ by minimality. Lemma K.2b.P1/K.CV therefore gives the same dynamic endpoint identity and payload in both executions. Lemma K.2b.R supplies the persistent liberal request for every endpoint needed by T unless T has already committed, which would contradict the choice of T.

Buffered Pop case. Assume $B(c) > 0$ and T is the n -th Pop_c transfer. Under the non-fall-through E4 semantics, the FIFO is nonempty before that Pop commits, so at least n Push_c transfers have already committed before T. Those Push transfers precede T and therefore occur in $\bar{p}_L$. If the n -th Pop_c has already occurred there, T is not missing. Otherwise same-interface order gives $\text{PopOrd}_c < n$, while $\text{PushOrd}_c \geq n$; hence $\text{occ}_c = \text{PushOrd}_c - \text{PopOrd}_c > 0$. The persistent Pop request supplied by K.2b.R is therefore ChannelEnabled. Because $\bar{p}_L$ is an infinite fair continuation and the persistent request is continuously ChannelEnabled, B2/B3 force the transfer. Thus T occurs, a contradiction.

Buffered Push case. Assume $B(c) > 0$ and T is the n -th Push_c transfer. Immediately before its serial commit there are $n-1$ prior Push_c commits, and $E4$ enablement gives $(n-1) - \text{PopOrd}_c < B(c)$. Therefore at least $m = \max(0, n - B(c))$ Pop_c transfers precede T , and minimality places all of them in $\bar{p}_L$. If the n -th Push_c is absent from the liberal continuation, same-interface order gives $\text{PushOrd}_c \leq n - 1$. Consequently $\text{occ}_c = \text{PushOrd}_c - \text{PopOrd}_c \leq (n-1) - m \leq B(c) - 1$, so the channel is nonfull. The persistent Push request is ChannelEnabled throughout the relevant suffix of the infinite fair continuation, so $B2/B3$ force its commit. Again T occurs, a contradiction.

Rendezvous case. Assume $B(c) = 0$. Then T is the atomic n -th $\text{Push}_c/\text{Pop}_c$ rendezvous transfer. If T is absent, neither endpoint can have committed it separately. Apply Lemma K.2b.R to both endpoint operations. After the later of the two finite assertion points, both attributed requests are present and remain present by nonwithdrawal. $E4$ therefore makes the rendezvous transfer ChannelEnabled with the payload fixed by $K.2b.P1/K.CV$. Because $\bar{p}_L$ is infinite fair and both persistent endpoint requests keep the rendezvous ChannelEnabled , $B2/B3$ force the atomic commit. Hence T occurs, a contradiction. Conclusion. Every possible type of least missing transfer yields a contradiction. Therefore every endpoint transfer committed by $\rho_{S^{\det}}$ in the $K.\text{DomResetScope}$ dominance domain while DomEpoch is fixed occurs, with the same dynamic identity, channel ordinal, and payload, in $\bar{p}_L$. The proof establishes only this one-way dominance inclusion; $\bar{p}_L$ may contain additional liberal transfers and may order independent transfers differently. $\square$

Composition with Lemma K.2b.E. Lemma K.2b.P is a conditional contradiction argument on the reset-free $K.\text{ResetEpoch.Continue}$, terminal-free $K.\text{TerminalDisposition.Continue}$ branch: if a transfer committed by the selected Policy-S execution were absent from the frozen liberal continuation, the preceding cases would force that transfer to occur there. Lemma K.2b.E keeps each selected frozen transfer pending and therefore absent. Consequently, K.2b.P implies that the selected Policy-S execution cannot commit that frozen transfer, or overtake it with a source-later transfer. This is the implication used in Lemma K.2b.Q, Step 1; the two lemmas are complementary rather than contradictory. Lemma K.2b.E (fair complete frozen extension)—Proof. At every finite prefix constructed below while the selected f_P remain pending, $K.\text{ResetEpoch.Continue}$ forbids any RESET whose scope intersects $K.\text{DomResetScope}$ as the next transition; RW2 therefore never clears the selected requests on this branch. $K.\text{TerminalDisposition.Continue}$ makes every such frozen-prefix final state nonterminal, so the construction has no finite declared-terminal escape. Lemma K.BF-Persist supplies the remaining interruption-completeness fact by applying the exhaustive Core.IC blocking lifecycle to the selected WFG-relevant K.BF blockers. Thus every persistence and dynamic-identity claim below remains within the fixed DomEpoch for $K.\text{DomResetScope}$ and the same selected frozen request instances; RW5-framed resets disjoint from that scope remain permitted. Start from the finite Core.LAdm-admissible prefix ρ_L and its drain-closed component D . Retain the persistent requests f_P already present at σ_L . First discharge $K.\text{EnvMF}$. Under $\text{StandardEnv}/B1\text{-avail}$, Lemma K.EnvX-Auto makes every disabled environment-facing co-frontier permanently unavailable. Under a timed/reactive or otherwise nondefault environment, $K.\text{EnvMF}$ certifies that each $P \in D$ has exactly one frontier item, namely the selected internal f_P , so no environment-facing or other co-frontier can later become enabled, deactivate Core.16, and release the component. By component closure and Core.Drain/ $K.\text{Drain}$, a complementary request whose new issue could release a frozen f_P is source-later than some frozen frontier and is therefore not legal to introduce after settling. Consequently that complementary operation is not IssueEnabled under Definition Core.IssueFair and creates no issue-fairness obligation. Already-issued source-later work is finite at the cutpoint, may drain under $B2/B3/B5$ and $K\epsilon$, and cannot replenish the frozen set. Construct the continuation cycle by cycle using Lemma Core.CycleChoiceExt, derived from Core.CycleClosure C2, together with the Core.CycleResult/CompleteCycle relation. In each L1 window, fairly dovetail every continuously IssueEnabled nonfrozen issue obligation together with the permitted process-owned preparation work, while Core.16 excludes the source-later operations that

could release D. Complete L2 settling, schedule every continuously enabled nonfrozen boundary action fairly under B2/B3, perform L4, and advance the selected environment through the EnvStep component of each CycleResult nextState. Retain every disabled frozen request by Lemma K.BF-Persist and nonwithdrawal. A frozen request is not ChannelEnabled and therefore creates no B2 obligation to commit. If no Σ event commits in a cycle, advance by Core.SilentCycle; current inactivity does not make the execution terminal. At every newly constructed finite frozen prefix, Lemma K.BF-Persist continues to retain the same pending dynamic requests, while the two nested Continue branches exclude a dominance-affecting RESET as the next transition and a DeclaredTerminal current state. After each selected finite fair-dovetail segment has been extended to a typed CycleResult by Core.CycleChoiceExt, dependent choice over those CycleResult successors yields an infinite fair continuation through their CompleteCycle projection. The resulting $\bar{\rho}_L$ belongs to $\text{Exec}_L^{\text{adm}}(I)$, extends ρ_L , satisfies Core.IssueFair, and preserves every f_P pending with the same dominance-scope DomEpoch. K.ResetEpoch.DirectFail and K.TerminalDisposition.DirectFail do not invoke this construction; each is discharged by its independently certified direct K.RespFail branch. $\square$

Lemma K.2b.Q (deterministic bounded-response extraction)—Statement. Assume SDet, separate K.CompleteS evidence for ρ_S^{det} , K.RespCov, K.ResetEpoch.Continue, K.TerminalDisposition.Continue, Lemma K.2b.E, and K.2b's remaining premises. Let a Policy-L prefix reach σ_L with $\text{DeadKRec}^W(K_L)$. Then the SDet-selected execution ρ_S^{det} has a finite prefix containing a non-waived K.RespFail witness. No conclusion about Dead_KS, an owner terminal, or a closed serial WFG component is required.

Proof, Step 1 (joint frontier freeze and permanent serial blockage). Let D be the non-waived Dead_K^W component reconstructed at K_L . In the single-frontier case, for each $P \in D$ let x_{P^L} be its unique internal item in $\text{FrontRec}_P(K_L)$ and $f_P := \text{MsgOf}_P(x_{P^L})$. Exact residual offers and source-prefix closure show that every process-facing blocking Push/Pop required by the Sys_K/K .BF serial prefix and source-earlier than f_P has committed by σ_L ; failed non-blocking polls are handled through their WC-selected return outcomes and are not claimed to have committed. K.EnvMF either supplies StandardEnv/B1-avail for any certified multi-frontier case or, for a timed/reactive/nondefault environment, reduces this step to the single-internal-frontier case. Invoke Lemma K.2b.E to obtain the infinite fair complete Core.LAdm continuation in which all f_P remain pending. K.ResetEpoch.Continue applies at each finite frozen prefix and therefore prevents any relevant RW2 reset from occurring as the next transition, so those requests are never cleared or re-identified during the comparison. The closure and drain-closed hypotheses, point-to-point ownership, and Core.Drain/K.Drain make this a joint freeze: complementary requests needed to release one f_P are source-later than another frozen frontier and cannot be newly asserted, while already-issued downstream offers may only drain finitely and cannot replenish the frozen set. Lemmas K.2b.R and K.2b.P then prevent ρ_S from overtaking this continuation. Hence each $P \in D$ eventually becomes permanently non-advancing at a process-facing Policy-S serial frontier x_{P^S} with operation g_P satisfying $\text{Key}(g_P) \leq^{\text{K_src}} \text{Key}(f_P)$ (the Appendix-K shorthand $g_P \leq_{\text{src}} f_P$), and that frontier never process-facing commits. K.BF excludes an intervening pure-computation divergence. A certified multi-frontier freeze uses K-O2 only under StandardEnv/B1-avail; timed/reactive/nondefault multi-frontier components are outside the lemma.

Proof, Step 2 (coverage and non-waiver). By the default-total definition of K.RespCov, every dynamically reachable internal x_{P^S} produced by Step 1 already has a checkable proof disposition; no modal selection argument about whether its static site can become a serial blocker is permitted. A validated unreachability disposition cannot apply to the reached dynamic instance, and a validated static bounded-response disposition supplies the same finite-response fact as a literal monitor. If x_{P^S} is environment-facing and the instance uses StandardEnv/B1-avail, then $g_P <_{\text{src}} f_P$ committed in the liberal prefix; its fixed consumption ordinal is therefore available in Policy S, and persistent request plus Core.IssueFair and B2/B3 would force its commit, contradicting permanent blockage. Thus such an

environment frontier cannot be the Step 1 blocker. For a timed/reactive or otherwise nondefault environment, $K.EnvMF$ has already excluded every multi-frontier liberal component; any environment-facing serial blocker arising on $p_S^{\wedge}det$ requires an explicit $K.Resp$ monitor or availability proof. Coverage-preserving waiver closure ensures that the original non-waived component D cannot map solely to waived dispositions. Therefore at least one non-waived covered response obligation triggers at a finite Policy-S cycle t and remains unsatisfied thereafter unless the process-facing response occurs; any reset or run-ending disposition is classified separately by Definition $K.RespFail$ and the applicable reset/terminal branch.

Proof, Step 3 (finite response-failure witness). Let B be the finite bound of the monitor selected in Step 2. If p_S continues through the boundary-action phase of cycle $t+B$ without response, the first normalized cutpoint after that phase witnesses timeout. If p_S reaches a declared maximal finite terminal state before that post-phase cutpoint while the monitor remains pending, the terminal prefix witnesses maximal-pending. On this reset-free branch these cases exhaust the complete artifact supplied by $K.CompleteS$: a nonterminating complete run eventually reaches the finite timeout bound, while a terminating complete run ends only at DeclaredTerminal. The matched-reset cancelled-pending case is the separate $K.ResetEpoch.DirectFail$ branch handled directly in Lemma K.2b. Therefore p_S contains a finite non-waived $K.RespFail$. $\square$

Diagnostic note. The retired owner walk may still be run after $K.RespFail$ to classify a likely root as an internal cycle, $EnvStop$, $DoneStop$, or another owner-chain condition. Example $K.CE-5/H2$ shows why that taxonomy cannot be used as an exhaustive proof premise: a complementary owner can remain live forever on unrelated channels while permanently under-supplying the demanded endpoint. Failure to classify a response failure affects diagnostics only, not Corollary K.2c.

Proof of Lemma K.2b. Apply $K.Low.C$ if lowering is needed, then instantiate the shared $K.InterruptionGate$ in the normative order. First evaluate $K.ResetEpoch$. On $K.ResetEpoch.DirectFail$, use the independently certified $P \in D$, selected liberal frontier f_P , Policy-S serial frontier $x_P^{\wedge}S$, and $g_P := Op(x_P^{\wedge}S)$ with $g_P \leq_{src} f_P$ in the same pre-reset epoch. The associated non-waived $K.Resp$ monitor has already triggered and remains pending before the first matched affecting RESET; RW2 clears that exact monitored dynamic instance at the RESET unit, so $K.RespFail$ supplies cancelled-pending immediately. This branch does not invoke K.2b.Q Step 1. Only on $K.ResetEpoch.Continue$ evaluate $K.TerminalDisposition$. On $K.TerminalDisposition.DirectFail$, the independently certified mapping supplies the same D-to-serial-frontier data, a finite prefix of $p_S^{\wedge}det$ with the mapped non-waived monitor already triggered and pending, and independently checked evidence that $p_S^{\wedge}det$ reaches the specified declared maximal terminal state while that monitor remains pending; $K.RespFail$ therefore supplies maximal-pending directly. Only on $K.TerminalDisposition.Continue$ invoke Lemma K.2b.E to obtain the infinite fair frozen continuation and then K.2b.R/P/Q to derive the response failure. Thus each DirectFail branch is discharged independently, and the dominance continuation is constructed only after both nested Continue checks succeed. $\square$

Lemma K.Ser (conditional serial trace replay). Statement. Fix the selected instance Sys_K , the fixed stimulus, and the fixed witness choices, and let p_L be a finite Policy-L prefix admissible under Definition Core.LAdm ending at a source K-facing cutpoint $\sigma_L \in KCutReach_Sys_K(I)$ with $K_L = \langle E, w, H_L, O_L \rangle$. Suppose: (a) commit-prefix condition — for every process P , $CmtRec_P(K_L)$ is a source-order prefix of P 's dynamic operation sequence; (b) serial-precedence acyclicity — the union of the causal dependencies of p_L (per-channel transfer order and capacity dependencies, rendezvous atomic pairing, epoch-gate and guard dependencies, and process-local source order) with the Policy-S precedence constraints, requiring each source-earlier blocking operation of a process to commit strictly before a source-later one issues, is acyclic; and (c) terminal offer admissibility — every $o \in O_L$ is attributed to its owner's source-minimal uncommitted blocking item, equivalently $DrainBag_Proc(I)(K_L)$ is empty, so σ_L contains no source-later asserted request that Policy S would have withheld. Then there exists a

finite admissible Policy-S execution of the same instance, stimulus, and witness with the same per-channel committed transfer sequences and payloads ordinal for ordinal, the same committed-identity sets and canonical process-local replay history H_L , and therefore the same ProcRec, ChanRec, NextFrontRec, and source-relevant terminal values as K_L . If a separate terminal-offer replay check also establishes the same residual-offer set O_L , the two terminal cutpoints have equal KObS; equal O is not inferred from the committed history alone. Proof sketch. Linearize the finite acyclic combined precedence relation and replay commits in that order, inserting idle cycles to satisfy the strictly-earlier-cycle serial condition. The induction invariant — equal per-channel counts and payloads, equal process-local committed histories, equal channel state — is preserved at each step: source-earlier blockers have committed by the added precedence constraints, so each issue is Policy-S-legal; channel enabling conditions transfer by the equal-channel-state invariant; Lemma K.2b.P1 supplies operation-identity and payload agreement; and I.DR supplies per-process state agreement from equal replay histories. The optional equal-O conclusion requires one additional finite check that the terminal serial requests are asserted at the same typed boundaries and epochs. Mechanization note: a finite structural induction with no fairness or limit reasoning; mechanization is required before the lemma is cited as certified.

Engineering note (serial replay of recorded traces; scope of Lemma K.Ser). Lemma K.Ser characterizes when a recorded liberal or RTL-matched execution can be replayed under the conservative serial scheduler with identical committed histories — and, with the additional terminal-offer replay check, identical KObS. KObS alone is not a sufficient replay certificate. The recorded artifact must additionally contain the historical dynamic operation identities, Issue events, persistent-request intervals and attribution, failed non-blocking outcomes as witness selections, elaborated-boundary transfers, and arbiter selections needed to justify the acyclic operational replay; a committed-event history determines neither those issuance obligations nor the intermediate enablement facts. O_L records only requests still outstanding at the terminal cutpoint. Conditions (a) and (c) fail exactly where the recorded run exploited eager issue; Examples K.CE and K.CE-2 are minimal failures, and their liberal histories are reproducible by no Policy-S execution. Lemma K.Ser is therefore deliberately not a substitute for Lemma K.2b: the non-serializable case is where the dominance content of the serial-reduction route resides. Mechanization obligations. K-O0: mechanize Definition K.OpKey, its partial per-execution realization maps and key-level $<^K_src$ order; mechanize Definition K.CVPred on the common CVFact_I key universe, validate predecessor occurrence closure and the per-fact deterministic provenance equations, establish K.CV-WF (or an equivalent checked CVRank whose strict decrease proves it), and mechanize Lemma K.2b.P1 by well-founded induction on $<_{CV}$; do not use the possibly cyclic Appendix-G causal-dependency graph $\rightarrow$ as that induction order. K-O1: mechanize Definition Core.IssueFair and the Core.16 compatibility fact. K-O2: prove multi-frontier joint-freeze and process-facing monitor-trigger composition. K-O3: prove Lemma K.2b.P over the K.DomResetScope dominance domain with fixed DomEpoch on the reset-free, terminal-free infinite-fair branch. K-O4: validate the certificate-supplied finite S_p^{dom}/S_c^{dom} against K.DomResetScope conditions D1–D4, derive DomEpoch, mechanize Lemma K.2b.P0's keyed occurrence/reachability result, and then prove Lemma K.2b.R using the shared K.InterruptionGate.Continue rule: instantiate Interrupt_Reset as prefix-local legality of a RESET intersecting K.DomResetScope as the next proof-relevant transition/unit, with any executed occurrence recorded in the enclosing CycleResult.resetUnits and Interrupt_Terminal as DeclaredTerminal of the frozen-prefix final state; prove both predicates prefix-locally, show every K.2b.P0/R/P-consumed endpoint lies in K.DomResetScope, and preserve DomEpoch-local request attribution through Lemma K.BF-Persist. K-O5: mechanize default-total K.RespCov and waiver closure. K-O6: mechanize the common K.InterruptionGate.DirectFail payload shape independently of K.2b.P0/R/P/Q Step 1, then prove the reset specialization yields cancelled-pending through matched RESET/RW2 and the terminal specialization yields maximal-pending through DeclaredTerminal. K-O7: mechanize the schema's K.BF exhaustiveness fact, anti-circularity rule, and the normative nesting K.ResetEpoch.Continue $\rightarrow$

$K.TerminalDisposition$; prove the $Core.CycleClosure$ C2 partial-cycle extension interface and Lemma $Core.CycleChoiceExt$ for the selected source semantics; then on the two Continue branches construct the infinite fair sequence of $Core.CycleResult$ witnesses used by Lemma K.2b.E, while either DirectFail branch exits without invoking that construction. The supported environment branches remain those of $K.EnvMF$. K-O8 is needed only for the optional diagnostic drain-normalization/Dead_KS construction. These operational results shall be proved from $Core.ReachPrep$, $Core.CycleResult/Core.CompleteCycle/Core.CycleClosure/Core.CycleChoiceExt$, Core.16, Appendix C Issue/Commit linkage, RW2, nonwithdrawal, $K\epsilon$, $K.OpKey$, $K.CV$ including $K.CVPred/K.CV-WF$, B1-B5, $Core.IssueFair$, $K.BF$, point-to-point E4 ownership, $K.EnvMF$, $K.InterruptionGate$ with $K.DomResetScope$ and its two exhaustive $K.BF$ instantiations, $K.Low.C/K.Low.A5$, and $K.Resp$ —not from finite state-space reasoning. KObs selector facts do not replace operational monitor premises.

Appendix L – Snooping Introduction, Implementation and Concerns

Snooping Introduction

If designs are completely latency insensitive, verification of the pre-HLS versus post-HLS models is straightforward. However, if the design contains some components such as arbiters which have latency-sensitive behavior, and if that latency-sensitive behavior is in some cases externally visible to the DUT, then verification becomes somewhat more complex. Techniques can be applied to simplify verification.

Consider a design which has an arbiter like Matchlib toolkit example 09*. The arbiter uses non-blocking PopNB() on its inputs, and blocking Push() on its output to emit the winner of the arbitration. Since the latency between the pre-HLS and post-HLS designs will differ, the order of transactions presented to the arbiter in the two scenarios will differ, and thus the order of the winners will differ. This may result in verification mismatches between the two models if the order of the winners is externally visible to the DUT.

To force the pre-HLS and post-HLS simulations to match, we can run the two designs side-by-side. We can snoop the inputs to the arbiter in the post-HLS model, and only allow the inputs to the pre-HLS arbiter to proceed when the corresponding inputs are seen in the post-HLS model. This will force the order of the inputs to be equivalent between the pre-HLS and post-HLS models, and thus both arbiters will pick the same winners. When this technique is used, the pre-HLS simulation will be throttled by the post-HLS simulation, but the overall verification will still work properly.

Another related example involves interrupt request signals feeding into an interrupt controller within a CPU model. Typically, each interrupt request signal is a single bit `sc_signal<>`, indicating that an interrupt request is pending. If the requests originate from accelerator blocks that are being synthesized through HLS, then because of the latency differences between the pre-HLS and post-HLS models, the order of interrupt requests arriving at the interrupt controller will differ between the two models, and this will likely result in verification mismatches if the differences are externally visible to the DUT. To force the order of the requests to match, we snoop the requests arriving at the controller in the post-HLS model, and only then allow the requests to be seen in the pre-HLS model.

Simplified Snooping Approaches

In the snooping example directly above in which the arbiter is snooped, the entire post-HLS RTL DUT is simulated alongside the pre-HLS model to force the arbiter requests to be aligned in time. This is the most general case and assumes the input delays to the arbiter are difficult to determine via analysis.

Sometimes there are simpler cases where the input delays to the arbiter in the RTL DUT are easier to determine. For example, each input to the arbiter might arrive a fixed number of clock cycles after one of the primary inputs to the RTL DUT is pushed by the testbench. In this case, a much simpler model can be used to align the pre-HLS model with the post-HLS model. We can simply monitor the primary inputs to the pre-HLS model and then apply the fixed cycle delays to the pre-HLS arbiter inputs.

The two cases above illustrate two ends of a spectrum of possible approaches for extracting the delays from the RTL DUT. Between these two points there exist other possible approaches.

Snooping Implementation

To enable perfect matching between the pre-HLS and post-HLS system simulations, latency-sensitive global signals and latency-sensitive non-blocking message-passing operations need to be synchronized between the two simulations if their latency differences are externally visible to the DUT.

As explained earlier in this appendix, in the general case the entire post-HLS DUT RTL must be run alongside the pre-HLS system to achieve alignment. Examples of signals that need to be synchronized include global interrupt request signals (as discussed earlier in this appendix) and rdy/vld signal pairs for non-blocking operations on message-passing channels. All such synchronization signals and their corresponding handshake signals must be level-stable:

- A latency-sensitive signal s is **level-stable** if, once s becomes 1 in cycle t , it must remain 1 until the corresponding handshake signal h is 1 in some cycle $t' \geq t$.

Once the set of signals that need to be aligned between the two simulations is identified, the general rule to implement snooping is simple:

- For each polarity-normalized Boolean commit-enabling snoop signal (for example, the vld/rdy handshake signals of snooped non-blocking operations, or level-stable request lines), make all readers in the pre-HLS simulation see the logical AND of the value driven in the pre-HLS simulation and the recorded post-HLS observation of the same dynamic instance ordinal — captured at the post-HLS-side event and retired at the pre-HLS-side handshake of that ordinal, per Wrapper Realization Requirements W1–W4 below. Payload/data values are never ANDed or substituted; they are checked for equality at the matched ordinal (Requirement W4). The post-HLS simulation runs free: its readers see only its own driven values, and the wrapper observes but never gates, delays, or modifies any post-HLS-side signal.

Often the post-HLS side will not run ahead of the pre-HLS side at the snoop points, because HLS typically only adds (rather than subtracts) latency within each process. This tendency is a heuristic, not an invariant: HLS can also subtract latency, for example by removing conservative waits present in the source, by improving loop initiation intervals, or by realizing channels with fall-through timing. The one-sided rule above is therefore normative, and a symmetric variant — driving the logical AND of the two simulations' nets into the readers on both sides, so that the pre-HLS side can gate the post-HLS side — must not be used, even as a fallback when the post-HLS side runs ahead. Under B1 the post-HLS Σ -trace under fixed stimulus is unique, so there is no post-HLS nondeterminism for a wrapper to select among: gating a post-HLS-side reader (in particular an internal net such as an arbiter input or an interrupt request line) does not select an admissible RTL_B behavior; it produces a different circuit RTL_B' with different internal latencies, and any result obtained about RTL_B' does not transfer to the RTL_B that ships. Gating the pre-HLS side, by contrast, is witness selection over an intentionally under-specified specification (see Snooping Concerns below) and is always permitted.

Two situations must be distinguished when the post-HLS side does run ahead at a snoop point. (a) Benign cycle skew: the RTL asserts a snooped level-stable signal some cycles before the pre-HLS side does, but the order of Σ -visible events at the snoop points is unchanged. Because the compatibility

results of Appendices G–K are stated over Σ -event order and values, with finite ϵ -stutter tolerated (B4/B5), a matching pre-HLS witness still exists. The wrapper handles this case by recording the RTL assertion (level-stability makes the capture clean; the recorded assertion is held until the pre-HLS-side handshake of the same ordinal, per Requirement W1 below) and letting the pre-HLS side catch up, arbitrarily behind in host time. Cycle skew is not an error, and a cycle-locked per-cycle “perfect match” comparison must not be used as a validity gate, since it would misclassify a mere HLS speedup as a failure. (b) Divergence: no admissible pre-HLS execution can consume or produce the RTL’s event sequence — its ordinal order, kinds, and payloads — at the chosen boundary; the witness required by WC does not exist. This is the genuine failure mode; it violates Condition L.Dir below, must cause the wrapper to abort loudly, and is triaged as described under Snooping Concerns. Comparison of the pre-HLS-side AND against the value driven on the post-HLS side remains a mandatory monitor (Requirement W4 below), but its role is diagnostic: a per-cycle mismatch reports skew, while the L.Dir violation itself is defined at the event level — ordinal order, kind, and payload — and is detected by an order/kind/payload check together with a watchdog/timeout that converts witness non-existence, or an inconclusive failure to realize the witness within an unproved bound (Requirement W4), into a reported failure rather than a silent gate or an indefinite hang.

Wrapper Realization Requirements (Recorded-Observation AND)

The AND rule above is stated over a recorded observation, not over the live post-HLS net. A naive realization in which the pre-HLS reader sees the AND of the two currently driven values is a cycle-window mechanism: it delivers a snooped event to the pre-HLS side only if the post-HLS assertion is still asserted when the pre-HLS side is ready to commit. Because the post-HLS side runs free, its assertion window ends at its own handshake, which is unrelated to whether the pre-HLS side has caught up; level-stability guarantees that the signal remains asserted until the post-HLS-side handshake, not until the pre-HLS-side handshake. The live-AND realization is therefore faithful exactly when, for every dynamic instance ordinal k at every snoop point, the pre-HLS-side commit of instance k falls inside the post-HLS-side assertion window of instance k . This per-ordinal window-overlap condition is strictly stronger than Condition L.Dir, and it is an empirical per-run property: it shall be checked, not assumed.

Where the window-overlap condition fails — including on runs that are L.Dir-clean, i.e., runs exhibiting only benign cycle skew — the live AND fails in three ways. First, it drops events: the post-HLS assertion retires before the pre-HLS side arrives, the AND never rises for the pre-HLS side, and the event is silently lost (for example, a lost interrupt downstream, or a pre-HLS process waiting forever on a request that never becomes visible), converting an L.Dir-clean run into an apparent failure or value divergence — precisely the misclassification this appendix forbids. Second, it aliases ordinals: if the post-HLS side runs two or more transactions ahead on the same signal, the pre-HLS side’s k -th commit can be enabled by the post-HLS side’s $(k+1)$ -th assertion, and pulse-type signals held across two post-HLS instances can merge into one pre-HLS observation, or drop. Dropped and aliased events violate the no-add/remove/reorder premise of Lemma L.2 and break the per-instance μ_Q matching supplied by WC: the harness is then sampling a wire, not replaying the witness. Third, it detects nothing: genuine order divergence (a real L.Dir violation) manifests as a hang indistinguishable from a dropped-event hang, so the triage of Snooping Concerns cannot begin; moreover, the harness-induced infinite wait appears as a wait-for dependency in any Appendix K analysis — a phantom deadlock manufactured by the harness rather than by the design, and outside the finite-delay tolerance that B2 fairness extends to the wrapper on L.Dir-clean runs.

W1 (Recorded per-ordinal observation). For each snoop point, the wrapper shall capture each post-HLS snooped event — the committed transfer for message-passing snoop points, the assertion edge for level-stable global signals such as interrupt request lines — indexed by its dynamic instance ordinal, and shall hold the captured observation for the pre-HLS side until the pre-HLS-side handshake of the same ordinal, at which point the observation is retired. For a single-outstanding interface this reduces to one sticky flop, set by the post-HLS event and cleared by the pre-HLS-side handshake; in the general case it is a small per-snoop-point event FIFO. The AND shall consume the captured observation, never the raw net; level-stability is what makes the capture clean, and pulse-like signals shall be queued as distinct edge events so that dynamic instances are neither merged nor dropped.

W2 (Gate only the commit-enabling mirror signal). Per operation kind, the wrapper shall gate only the mirror signal that enables the pre-HLS-side commit: the observed vld for a pre-HLS-side Pop, and the observed rdy for a pre-HLS-side Push. The endpoint-owned request driven by the pre-HLS side itself is never modified, consistent with the Appendix C non-withdrawal discipline.

W3 (Registered sampling point). In a shared-clock co-simulation, the post-HLS observation shall be registered by at least one cycle before it enters the AND, so that delta-cycle ordering races between the two simulations cannot affect the observed value. This is safe by construction: it adds only pre-HLS-side latency, which is witness selection.

W4 (Monitor and watchdog; Lemma L.5(iv)). At each pre-HLS-side commit at a snoop point, the wrapper shall compare the committed event against the recorded post-HLS event of the same ordinal, in order, kind, and payload. Payloads must match (K.CV); the wrapper shall never substitute a post-HLS payload into the pre-HLS side, since substitution relabels Σ -events and masks value divergence. A pre-HLS-side snoop commit with no recorded post-HLS counterpart, or an order/kind/payload mismatch at a matched ordinal, shall abort loudly as an L.Dir violation. A recorded event outstanding beyond a configured watchdog bound, or a pre-HLS-side request that remains unmatched beyond that bound, shall likewise abort loudly, but as an L.Dir/WC discharge failure: if the bound has been proved sufficient for the selected instance, the abort may be classified as an L.Dir violation; otherwise it shall be triaged as an inconclusive witness-realization failure — the compared run remains outside the discharged theorem scope, but the abort shall not be converted into a claimed design deadlock or a claimed proven violation. In all cases the abort converts witness non-existence, or failure to realize the witness within the bound, into a reported failure rather than a silent gate or an indefinite hang. Per-cycle skew shall be logged separately as diagnostics and shall not be treated as a failure.

W5 (Coverage remains a separate obligation). Requirements W1–W4 concern the fidelity of the replay mechanism at each snoop point; they do not discharge the first premise of Lemma L.2. If the snooped set misses an ϵ -modeled choice that can affect Σ -visible control flow, frontier membership, request attribution, guard values, or payload values, the replayed witness is underdetermined regardless of the recording discipline, and the resulting failure shall be triaged as harness incompleteness under Snooping Concerns.

Remark (relation to the naive live AND). The live-AND and recorded-observation realizations coincide on single-outstanding snoop points at which the pre-HLS consumer commits within every post-HLS assertion window — for example, a non-blocking polling consumer that polls every cycle while the pre-HLS side remains at-or-ahead in cycle time at that snoop point throughout the run. Because W4 requires the W1 recording in any case as its comparison reference, gating with the live net offers no economy;

the recorded observation is therefore the normative gating term, and the live net may be used only as an additional diagnostic tap.

Shared reactive testbenches. Once the pre-HLS side is permitted to lag by more than a cycle, a single shared reactive testbench instance is problematic: its reactions are driven by whichever model it is wired to, so the two models no longer receive the same stimulus, contrary to the fixed-stimulus setting of the formal results. The flow shall therefore either replicate the stimulus/testbench per model, or structure the shared testbench as a function of the DUT-visible trace so that the wrapper’s recorder can replay its reactions to the lagging side under the same recorded-observation discipline of W1–W4.

Definition L.1 (Witness selector / Snooping).

Fix an initial state, fixed external stimulus, and an RTL_B execution prefix τ_{post} . A witness selector W for a source process P is a partial strategy that, at each ϵ -quiescent source cutpoint σ_{ref} and each source-level choice point whose outcome is ϵ -modeled but may affect later Σ -visible behavior, returns the outcome selected by the matched RTL_B execution.

Such choice points include failed/successful outcomes of non-blocking PushNB/PopNB calls when the outcome affects later control flow, frontier membership, request attribution, or payload/guard values; and any other ϵ -only implementation choice that can affect the next Σ -visible frontier.

Cadence-sensitive non-blocking polling. If a source process computes externally visible behavior from the number or cadence of failed PushNB/PopNB polls — for example, retry counters, timeout counters, or arbitration policies whose winner depends on how long an input remained empty/full — then that polling logic is latency-sensitive in the sense of Appendix D. Such a process is not covered by the ordinary Appendix I/J latency-insensitive source-core argument merely by treating the failed polls as hidden ϵ -steps. To bring such a design into scope, the cadence-sensitive polling logic shall be isolated as a latency-sensitive local protocol block, or shall be aligned by an Appendix L snooping/witness construction that makes the selected source execution an admissible execution of the same isolated protocol behavior observed in RTL_B. The observable interface to the rest of the system shall then be the chosen boundary of the isolated block, not the internal failed-poll cadence itself.

The selected outcome must be causally available from the matched RTL_B prefix and the fixed global Σ -causal history. W is not allowed to depend on future RTL_B events beyond the matched prefix needed to identify the current source choice. For cadence-sensitive polling logic, the witness must select an admissible execution of the isolated latency-sensitive block; it may not manufacture a retry/timeout outcome that is inconsistent with the RTL_B behavior being used as the witness, the fixed stimulus, or the chosen observable boundary of that block.

Lemma L.2 (Snooping establishes WC).

Let τ_{post} be an RTL_B execution under fixed stimulus, and let W be the witness selector induced by the Appendix L snooping wrapper for the corresponding source execution. Assume:

1. the snooped signals / outcomes include every ϵ -modeled source choice whose result can affect the next Σ -visible control flow, frontier, request-attribution, guard value, or payload value;
2. the snooping wrapper does not add, remove, reorder, or relabel any Σ -visible event except by selecting ϵ -only source outcomes at the chosen observable boundary;
3. each selected ϵ -only outcome is consistent with the matched RTL_B prefix and the fixed global Σ -causal history;
4. failed non-blocking polls remain ϵ -events and successful non-blocking transfers produce exactly the corresponding committed Push_c / Pop_c Σ -event; and

5. if the source contains cadence-sensitive non-blocking polling whose externally visible behavior depends on the number or cadence of failed polls, that polling logic has been isolated or snoop-aligned as a latency-sensitive local protocol block as described in Appendix D and Definition L.1. The WC witness is then applied to the chosen observable boundary of that block, not to an unconstrained hidden failed-poll cadence inside the ordinary latency-insensitive source core.

Then the source execution selected by W satisfies the witness-compatibility premise WC used in Appendix I.2 and Appendix J.

Corollary L.3 (NB-CF identity witness).

If NB-CF holds for process P , then failed non-blocking poll outcomes do not affect the next Σ -visible control flow, frontier membership, request attribution, guard values, or payload values. Therefore the witness selector W may be taken to be the identity / arbitrary selector on failed non-blocking outcomes, and WC holds without additional snooping for those polls.

Corollary L.4 (Cadence-sensitive polling is handled by isolation).

If NB-CF does not hold because the source observes the number or cadence of failed PushNB/PopNB polls — for example, retry counters, timeout counters, or arbitration policies whose later Σ -visible behavior depends on how long a channel remained empty/full — then the design is latency-sensitive at that polling site. Such a site must be isolated or snoop-aligned under Appendix D / Appendix L before applying the ordinary Appendix I/J source-core proof. If no such isolation, snooping, or equivalent explicit timing model is supplied, the process is outside the scope of the Appendix I/J trace-compatibility theorem for the chosen observable boundary.

Condition L.Dir (One-directional alignment / RTL-side non-interference).

Fix the free-running RTL_B execution τ_{post} under the fixed stimulus (B1) and the Appendix L wrapper for a chosen set of snoop points. L.Dir holds for a compared run iff the source-side snoop sequence selected by the witness selector W of Definition L.1 is exactly an ordinal-preserving consumption of the free-running RTL_B snoop sequence at the chosen boundary: (i) each pre-HLS-side snoop commit consumes the next recorded RTL_B event of the same snoop point, with matching kind and payload; (ii) no pre-HLS-side snoop commit occurs without such a next recorded RTL_B event; and (iii) every recorded RTL_B snoop event within the compared run is eventually either consumed by the source witness or reported outstanding according to the proved or declared comparison bound of Requirement W4 above. Throughout, the alignment never gates, delays, or modifies any RTL_B-side signal or event (RTL-side non-interference): the wrapper never drives a post-HLS-side reader with a value different from the value the free-running RTL_B drives. Cycle-level lag of the pre-HLS side (benign skew) is permitted under L.Dir provided the ordinal-preserving consumption above is maintained via recorded/buffered replay of the level-stable assertions; only order, kind, or payload divergence — non-existence of the WC witness at the chosen boundary — violates L.Dir. L.Dir is a named side condition with its own entry in the side-condition discharge record (Normative Formal Core). A run that violates L.Dir provides no theorem-backed guarantee. The harness shall abort loudly on any detected divergence and on any exceeded Requirement W4 watchdog bound, with the classification of bound-exceeded aborts governed by Requirement W4, and every abort shall be triaged as described under Snooping Concerns below.

Lemma L.5 (Online realization of W / harness soundness).

The equivalence and liveness results of Appendices I–K are stated over the free-running RTL_B execution τ_{post} under fixed stimulus (B1), together with the source-side witness selector W of Definition L.1. This statement is one-directional by construction: no premise of those results requires RTL_B to be stallable by the source, and τ_{post} is a fixed object rather than a participant that can be gated. Separately, suppose the co-simulation wrapper (i) observes but never gates, delays, or modifies any post-HLS-side

signal or event; (ii) records snooped level-stable assertions and replays them to the pre-HLS side in RTL order, permitting arbitrary finite host-time skew (Wrapper Realization Requirements W1–W3); (iii) satisfies the premises of Lemma L.2 (see Requirement W5); and (iv) monitors Condition L.Dir and aborts loudly upon violation (Requirement W4). Then on every run on which the wrapper does not abort, the wrapper computes W online, the selected source execution satisfies the WC premise used by Appendices I–J (by Lemma L.2), and the results proved over free-running $RTL_B + W$ apply to that run. Bidirectional stalling therefore never enters any theorem whose conclusion is transferred to the shipping RTL: the post-stalls-pre direction is witness selection over the intentionally under-specified source, and the pre-stalls-post direction is excluded by the monitored Condition L.Dir rather than assumed, endorsed, or repaired by gating the RTL.

Snooping Concerns

The snooping technique is used to align pre-HLS and post-HLS latency-sensitive global signals and latency-sensitive non-blocking message-passing operations in cases where their latency differences are externally visible at the DUT boundary. This includes cadence-sensitive polling blocks, such as retry/timeout or polling-arbiter logic, when their externally visible behavior depends on the number or cadence of failed non-blocking polls. When such a wrapper is applied, the pre-HLS model is throttled to follow the latency choices made by the post-HLS RTL, and the resulting traces at the observable interfaces are forced to agree. Snooping does not make failed polls into committed message-transfer events; failed polls remain ϵ -steps. Rather, snooping selects an admissible execution of the latency-sensitive local protocol block, while the rest of the system is compared at the block’s chosen observable boundary.

Readers with a formal background may be concerned by this technique, because it appears to use the post-HLS RTL implementation to modify the behavior of the pre-HLS specification. From a formal point of view, the important observation is that the pre-HLS model is intentionally under-specified with respect to micro-timing/latency: subject to the R-rules, B-rules, weak fairness, and the explicit Σ -visible ordering, FIFO, signal-anchor, synchronization, and atomicity constraints used by $\approx$, it can admit multiple ϵ -/latency-different executions that respect the same required-order constraints over the chosen observable units. This is compatible with B1, which asserts that the post-HLS Σ -projected trace is unique under a fixed stimulus; the pre-HLS side may still have multiple admissible realizations that (when projected to Σ) match that unique RTL Σ -trace.

Snooping does not change what executions are allowed by the pre-HLS specification. Instead, it selects—purely for verification purposes—one admissible pre-HLS execution whose latency-sensitive events align with those observed in the RTL. In other words, the implementation is used to pick a witness execution of the specification, not to redefine the specification.

Triage of L.Dir violations. A violation of Condition L.Dir (order, kind, or payload divergence at a snoop point, including a pre-HLS-side snoop commit with no recorded RTL_B counterpart) is a relational failure: the compared run is outside the discharged theorem scope, and no theorem-backed claim attaches to it. It shall not be defined a priori as a bug in the pre-HLS model, because witness non-existence has three possible root causes, and defining the event as a model bug by fiat forecloses detection of the third: (1) an ill-formed pre-HLS model — for example, reliance on the ordering of causally independent events, cadence-sensitive polling that was not isolated per Corollary L.4, or a snooped signal that is not level-stable — in which case the failure is of the same character as a standalone pre-HLS test failure and the

model is outside the well-formedness envelope of the stress-testing section below; (2) an incomplete snooping harness — the snooped set misses an ε -modeled choice covered by the first premise of Lemma L.2, so the replayed witness is underdetermined; or (3) a non-compliant tool/RTL — the RTL exhibits behavior outside the admissible envelope (for example, an R/B-rule or B-faithfulness violation), in which case the Post $\rightarrow$ Ref result itself fails and the divergence is evidence of a synthesis defect. In a flow in which the harness has been certified against Lemma L.2 and the tool is trusted, the pre-HLS model is the default suspect and the failure should be filed against it first; the formal status of the run, however, is “L.Dir/WC undischarged — outside theorem scope,” not “pre-HLS model incorrect,” until triage assigns the root cause. A watchdog abort under a bound that has not been proved sufficient for the selected instance is triaged in the same way but is recorded as an inconclusive witness-realization failure per Requirement W4, not as a proven L.Dir violation. Benign cycle skew (unchanged Σ -event order with the RTL merely earlier in cycle time) is not an L.Dir violation and shall not be classified as a model defect. Experienced SoC architects tend to view this slightly differently, in terms of design tradeoffs and verification cost.

First, it is usually possible at the architectural level to avoid latency-sensitive behaviors entirely, for example by insisting that all communication be latency-insensitive and fully synchronized. However, the quality-of-results (QOR) costs of such designs may be unacceptable. Introducing latency-sensitive behavior—non-blocking arbiters, latency-sensitive global interrupt signals, and so on—is a deliberate choice made by the designer to meet performance, power, or area goals.

Second, in many practical systems, latency-sensitive behavior is not functionally observable at the DUT boundary under the trace-compatibility predicate $\approx$ with Σ instantiated to include only the DUT-boundary observables (Appendix G; Normative Formal Core above). As an example, consider a DUT that contains a single-port RAM used to exchange data between two internal processes. An arbiter is required to control access to that RAM port, and the arbiter uses latency-sensitive non-blocking Pop operations on its request channels. During HLS, internal latencies will change, so the order in which the arbiter sees pending requests and chooses winners may differ between the pre-HLS and post-HLS models.

In this example:

- The individual RAM read/write operations and the internal arbitration decisions are not directly visible at the DUT interface.
- Only the higher-level IO of the two processes (for example, their message-passing interfaces to the rest of the system) is externally visible.

One might initially conclude that it is necessary to snoop the arbiter’s inputs to make the pre-HLS and post-HLS traces match. However, the modeling rules impose two additional constraints:

1. **Weak fairness for the arbiter.** The arbiter must satisfy the weak fairness assumptions, so that any request that remains pending is eventually granted in both the pre-HLS and post-HLS models.
2. **Causal-dependency discipline on shared memory.** The system must make any functionally significant shared-memory dependence explicit, either through synchronization that orders the relevant accesses or through declared array-access mapping / memory-serialization semantics. This discipline ensures that sufficient synchronization or declared serialization exists between the processes so that there are no read/write races through the RAM in either model.

Under these conditions, the different arbitration orders merely change the relative timing of internal operations and of externally visible actions that are not ordered by the explicit required Σ -constraints. They do not change the FIFO, synchronization, signal-anchor, atomicity, or other required ordering constraints at the DUT boundary. In the terminology of Appendices G and J, the differences are incidental variations in latency rather than changes to the required-order relation $\leq_J$ over the chosen observable units, and therefore they are not considered observable differences in behavior at the DUT boundary. **In such cases, snooping is not required.** (See also Note 1 below).

(More precisely: the trace-compatibility predicate $\approx$ is parameterized by the chosen observable alphabet Σ . If internal functionality (such as the above RAM arbiter) uses channels/signals that are not designated observable in the chosen instantiation of Σ (see Normative Formal Core), then mismatches on those internal actions are outside $\approx$ by construction: they are not in Σ . If those internal channels/signals are designated observable (e.g., by snooping for debug, or by expanding Σ for Appendix K deadlock analysis), then they are Σ -events and must match under $\approx$. In addition, even when B1 holds (unique post-HLS Σ -trace under fixed stimulus), the pre-HLS model may still admit multiple ε -/latency-different executions consistent with the same required Σ -ordering, FIFO, signal-anchor, synchronization, and atomicity constraints; Theorem J.1 captures the intended quantification (“existence of a matching witness execution prefix”) rather than requiring a single pre-chosen trace.)

Experienced SoC architects therefore try to avoid designs in which latency-sensitive internal behaviors leak directly into the observable behavior of the system under test, since this both complicates verification and makes RTL behavior less predictable. When they do introduce such behaviors—examples include non-blocking arbiters whose winners affect externally visible ordering, global interrupt request signals at the DUT boundary—they typically know exactly where those latency-sensitive interfaces are and can isolate them.

A useful analogy is a subsystem whose clock frequency is adjusted dynamically based on on-die temperature. As temperature changes, the externally visible behavior of the SoC can change (for example, in terms of throughput or timing of events), and designers may choose such a temperature-dependent scheme to meet overall system requirements. To verify such a system, we may wish to compare a concrete RTL trace captured at a specific temperature profile with a higher-level reference model. To do so, the reference model must be driven with the **same** temperature evolution as the RTL saw. If that temperature profile is not otherwise under direct control, the simplest way to obtain it is to “snoop” the temperature as observed in the RTL trace and replay it into the reference model. In this analogy, temperature is an external parameter of the environment rather than a fundamental part of the functional specification, but it still influences observable behavior. Snooping simply provides a mechanism to recover that parameter from the implementation when it cannot be easily prescribed a priori. Similarly, snooping of latency-sensitive IO provides a mechanism to supply implementation-specific latency choices — subject to the fairness assumptions and the explicit required Σ -ordering, FIFO, signal-anchor, synchronization, and atomicity constraints — back to the pre-HLS specification so that the two models can be compared under a common environment.

Note 1 (Unobservable non-blocking micro-timing).

Safety / functional equivalence. The trace-compatibility predicate $\approx$ is parameterized by the chosen observable alphabet Σ (Appendix G). If, for safety-only checking, we instantiate Σ to include only DUT-boundary observables (Σ_{DUT}) and we assume that any latency-sensitive internal behavior is not visible at that boundary under $\approx$ —i.e., it may change only internal micro-timing, but it does not change the values or presence of Σ_{DUT} events, and does not change the ordering of causally *dependent* Σ_{DUT}

events—then snooping of those internal latency-sensitive interfaces is not required for proving Σ_{DUT} -safety. Any mismatches on actions outside Σ_{DUT} are outside $\approx$ by construction. When source-reference safety is proved universally, or when one complete pre-HLS run satisfies Definition J.RefProjCover, Corollary J.1-Safe combines that result with Theorem J.1 Post $\rightarrow$ Ref to obtain RTL_B safety at Σ_{DUT} . This common-case transfer does not use the restricted Ref $\rightarrow$ Post clause and does not require snooping.

Liveness / deadlock analysis. For message-passing deadlock analysis (Appendix K WFG), any latency-sensitive interface whose ε -level choices can affect whether a blocking Push/Pop is enabled, or can create/remove a wait-for dependency without an intervening committed Σ -event, must be accounted for. Practically, the default is to snoop and replay every such latency-sensitive (typically non-blocking) arbitration/probe interface in the DUT. Snooping can be avoided only if the design is certified “WFG-inert” with respect to that interface: it cannot change WFG edges except via committed Push_c/Pop_c (or explicit synchronization) events.

Stress Testing Pre-HLS Models for Latency Robustness

The formal trace-compatibility results in Appendices G–K establish that, under the scheduling rules and fairness assumptions, the post-HLS RTL is trace-compatible with the pre-HLS/source-reference model at all observable interfaces.

These results implicitly assume that the pre-HLS specification is *well-formed*: it must not rely on incidental, implementation-specific timing alignments for functional correctness. Designs that do rely on such incidental alignments fall outside the formal guarantees.

Designs that use non-blocking message-passing operations (e.g., PushNB/PopNB, ac_channel nb_read/nb_write) or latency-sensitive global signals are particularly vulnerable to this kind of fragility. For such designs, it is strongly recommended to subject the pre-HLS model to aggressive *latency stress tests* in which message and handshake latencies are varied across executions. The intent is to validate that the design behaves correctly across the range of weakly fair schedules permitted by the modeling rules, not just under one convenient execution order.

More concretely, verification should check that:

- **Causal dependencies are explicit.** All functionally significant causal-dependency relationships are enforced via message-passing, SyncChannel/ac_sync operations, explicit signal handshakes/anchors, declared memory-serialization constraints, or Appendix L snooping/witness-selection boundaries, rather than by relying on the incidental execution order of concurrent processes. In the terminology of Appendix G, the design shall not depend on the ordering of causally independent events for correctness.
- **Weak fairness does not hide latent deadlocks or starvation.** The design continues to make progress under adversarial but *weakly fair* scheduling—i.e., enabled actions may be delayed arbitrarily long but not forever—consistent with the B2/B3 obligations on the scheduler and environment summarized in Appendix N.

If the pre-HLS model fails under such randomized-latency stress scenarios, then the specification itself is outside the formal model of this document: it violates the design rule that well-formed systems must not rely on the relative ordering of causally independent events. In that situation, the trace-compatibility theorems still apply to well-formed traces, but they no longer guarantee that the “golden” specification is robust under the full range of latency variations allowed by the environment.

The Matchlib library provides practical mechanisms to automate these stress tests in pre-HLS simulation, including:

- **Random stall injection on message-passing channels**, to simulate variable communication delays while preserving rendezvous semantics.
- **Latency and capacity back-annotation**, to model specific per-channel buffer depths and delays, including the capacities $B(c)$ chosen during HLS.

Worked examples of this methodology are provided in Catapult Matchlib examples 60* and 72*, which illustrate how to configure randomized stalls and back-annotated latencies when validating that a design remains well-formed and latency-robust.

Appendix M – Formal Results Overview

M.1 Purpose and status

This appendix gives a readable overview of the formal results and the relationship among their principal components. It is informative: the definitions, premises, theorem statements, proof obligations, applicability restrictions, and coverage requirements in Appendices G-L, Appendix N, and the Normative Formal Core remain authoritative. The overview intentionally uses formal names where they help a reviewer locate the controlling result, but it does not reproduce the complete proof API.

The document proves two related but distinct kinds of result. The safety layer establishes ordered observable trace compatibility between the post-HLS implementation model and a prescribed source reference model. The liveness layer excludes a specified prohibited internal wait structure under stronger and more global conditions. Neither layer independently certifies that a particular source design, HLS invocation, generated RTL instance, environment, or back-annotation satisfies the premises required to instantiate the result.

M.2 Models and observation boundary

The implementation-side model is `RTL_B`. The source-side theorem target is the selected `Sys_ref` under the fixed stimulus and the recorded environment, reset, arbitration, witness, and abstraction choices. In the ordinary bounded-channel case, `Sys_ref` is `Sys_B` with capacities `B(c)` faithful to the selected explicit abstract channel boundaries. The rendezvous case is the special case `B(c)=0` at the relevant ordinary boundaries. When the implementation contains staged, bundled, joined, coupled, or synchronization-gated realization structure that cannot be represented faithfully by one ordinary bounded FIFO, `Sys_ref` is `Sys_B^elab` for a well-formed elaboration package as described in Appendix Q.

The safety relation is defined over a selected observable alphabet and an ordered trace-compatibility relation rather than over equality of internal states or equality of cycle-by-cycle execution. Observable units include the committed channel transfers, synchronization events, and anchored signal reads and writes selected by the formal model. Independent units may commute or be grouped into cycles differently when the required ordering, value, atomicity, and observation-boundary conditions are preserved. Proof-internal elaboration events and epsilon steps are hidden or projected as specified by the applicable abstraction map.

Shared-memory communication between processes is covered by the proved safety result only under the J.Mem mapping-layer lowering condition stated in the Normative Formal Core. That condition makes memory-carried dependencies visible through operations already represented in the proof alphabet. A direct global memory-state extension is discussed as a possible future extension but is not part of the present theorem.

M.3 Principal safety result: Post -> Ref

The ordinary safety direction is Post -> Ref. For a covered reachable `RTL_B` prefix, the flow supplies Definition J.LocalGWR or, alternatively, a directly validated J.GWR package with its required provenance and checks. J.LocalGWR organizes design-specific evidence into process-local, explicit-channel, named atomic-interaction, environment/reset, construction-action, footprint, request-snapshot, generated-dependency, local-agreement, and coverage components, together with the applicable channel-capacity faithfulness obligations.

Theorem J.GWR-Comp is the generic local-to-global construction. It generates the dependency structure, proves the required acyclicity and canonical construction order, combines the local preparation scripts, settles shared non-blocking decisions under the registered phase discipline, and

produces one reachable globally witness-realizable Sys_ref package. Proposition J.0 then composes the selected source/RTL pair to establish the ordered observable compatibility required by Theorem J.1. The design-specific premises are primarily local even though the conclusion is system-wide; the global gluing argument is proved once rather than supplied as an unexplained replay premise for each design.

The practical safety interpretation is that a covered RTL_B behavior cannot introduce a new canonical observable safety behavior that is absent from the selected source reference model. For source-side safety signoff, Corollary J.1-Safe transfers a J.HCanon-invariant safety predicate from the covered Sys_ref prefixes to the covered RTL_B prefixes. When one complete pre-HLS execution is used as the source-side verification artifact, Definition J.RefProjCover is the additional coverage fact showing that its observable prefixes represent all covered source-reference observable prefixes relevant to the claimed safety property.

M.4 Restricted Ref -> Post use and witness selection

The converse direction is intentionally not an unrestricted theorem saying that every raw source simulation execution is reproduced by RTL_B. HLS may change latency, cycle grouping, arbitration timing, non-blocking outcomes, and other witness-sensitive choices while remaining within the scheduling contract. Accordingly, direct Ref -> Post transfer is limited to annotated RTL-consistent source prefixes or to source executions selected by the Appendix L snooping and witness-selection mechanism.

This asymmetry is important for interpreting the methodology. The primary safety argument does not require a claim that one arbitrary pre-HLS run is cycle-for-cycle or choice-for-choice identical to the implementation. Instead, it shows that each covered implementation behavior has a compatible source-reference explanation. A particular source run can be used directly in the reverse direction only when the additional annotation or witness-selection conditions establish that it is the run corresponding to the implementation behavior under review.

M.5 Cutpoint correspondence used by liveness

The liveness proof begins from a stronger cutpoint interface than ordinary trace compatibility. Corollary J.1 and CF-0/J.KObsConf connect one selected reachable RTL_B cutpoint to one reachable Sys_ref cutpoint through an equal KObs record. KObs contains the canonical replay history and the complete attributed residual-offer information needed to reconstruct the relevant frontier and wait-for structure. It is an observation record for the bad-state predicate, not by itself a complete operational state for future Policy-L or Policy-S execution.

For the exact source witness selected by that correspondence, J.RegPhaseReach supplies theorem-derived L0-L4 phase-correct reachability. K.DrainReach separately supplies the nonwithdrawal, epsilon-normalization, and activated Core.Drain/K.Drain evidence. Their checked conjunction is K.RLAdmReach, which places the witness in the admissibility domain consumed by the Appendix K theorem. Keeping these components separate prevents the trace correspondence alone from silently assuming the drain and progress properties needed by the liveness argument.

M.6 Deadlock-preservation result

Primary user workflow. The primary user-facing discharge route is deterministic Policy-S serial reduction. For one fixed selected Sys_K instance, including capacities, elaboration, initial/reset state, stimulus, witness choices, environment behavior, arbitration, concrete pre-HLS scheduler, Policy-S issue semantics, lowering, and normalization, SDet selects one Policy-S execution artifact p_S^{det} . Policy S is conservative: within each source interval, a process does not issue a source-later blocking operation until every required source-earlier blocking operation has committed in a strictly earlier cycle. The flow

supplies total finite-bound $K.\text{RespCov}$ for every relevant internal process-facing blocking site or dynamic class. For a terminating selected instance, the design user runs this instrumented model to a declared maximal terminal state; the complete artifact must contain no covered timeout, no covered frontier pending at termination, and no non-waived reset-pending failure.

Why the run suffices. $A5\text{-}S/CF\text{-}S$ combines three separately checked layers for the selected execution: $K.\text{CompleteS}$ establishes maximality, fairness, and completeness; $K.\text{RespCov}$ establishes total finite-bound response coverage; and $K.\text{RespClean}$ establishes that the complete run contains no non-waived $K.\text{RespFail}$ witness. Lemma K.2b shows that a prohibited deadlock reached by an admissible Policy-L prefix would force that same selected Policy-S execution to produce a finite response failure. Corollary K.2c therefore derives $A5\text{-}L$ when $SDet$, $K.\text{EnvMF}$, $K.\text{ResetEpoch}$, $K.\text{TerminalDisposition}$, and the remaining reduction premises hold. This replaces exploration of alternative Policy-S scheduler interleavings with one selected complete artifact; no confluence theorem is required. For a terminating selected instance, the complete-run response check itself does not require model checking or a hand-written liveness invariant, although the flow must still qualify the selected semantics, finite bounds, applicability, coverage, fairness, and other stated premises.

What transfers to RTL. Theorem K.1A is first a certified-cutpoint theorem. At a reachable K -relevant RTL_B cutpoint carrying a complete $K.\text{CertifiedCutpoint}$ package, the equal-KObs source witness, $K.\text{RLAdmReach}$, $A5\text{-}L$, and the applicable point-to-point, channel, structural, waiver, environment, progress, and fairness premises exclude the prohibited closed internal wait structure. A universal statement additionally requires $\text{Core}.K\text{CutClosure_post}$, which establishes a finite $K\text{Cut}$ observation for every reachable K -facing RTL cycle origin, and $K.\text{CertCov}$, which establishes a complete certificate package for every reachable cutpoint in that domain. Corollary K.1A-Total combines both with the per-cutpoint theorem. Thus one clean Policy-S execution discharges the source-side premise through K.2c; it does not by itself certify one RTL cutpoint or prove universal RTL cutpoint coverage, and its finite response bounds are not transferred as RTL latency bounds. K.4e and M.7a specify valid $K\text{CutClosure_post}$ and $K.\text{CertCov}$ discharge routes, but this document does not currently supply a completed generic package-existence theorem or verified generator; a concrete flow must instantiate and independently check one of those routes.

Scope, alternatives, and status. Timed, reactive, or otherwise nondefault environment behavior in a relevant multi-frontier component is outside the serial route unless the certified single-internal-frontier condition and the other $K.\text{EnvMF}$ conditions hold. Dynamic-reset instances additionally require $K.\text{ResetEpoch}$; every candidate on $K.\text{ResetEpoch}.\text{Continue}$ additionally requires $K.\text{TerminalDisposition}$. For every candidate liberal component, the flow first binds a validated $K.\text{DomResetScope}/\text{DomEpoch}$ and establishes $K.\text{ResetEpoch}$. On $K.\text{ResetEpoch}.\text{Continue}$, the comparison continuation must remain free of any RESET whose RW scope intersects the candidate's $K.\text{DomResetScope}$; RESETs disjoint from that scope remain permitted under RW5. Alternatively, $K.\text{ResetEpoch}.\text{DirectFail}$ must independently map a selected liberal frontier to an already-triggered, non-waived process-facing Policy-S response monitor that the matched RESET clears under RW2. A RESET that intersects $K.\text{DomResetScope}$ but does not satisfy the independently mapped $K.\text{ResetEpoch}.\text{DirectFail}$ conditions satisfies neither serial-route branch; that candidate is outside the serial-reduction route and requires another $A5\text{-}L$ discharge. On the $K.\text{ResetEpoch}.\text{Continue}$ branch, $K.\text{TerminalDisposition}$ likewise requires either a terminal-free frozen comparison or an independently mapped Policy-S declared-terminal maximal-pending $K.\text{RespFail}$; otherwise the cutpoint is outside the serial route. A source region whose deadlock freedom genuinely depends on favorable same-cycle or overlapped issue is outside the scheduler-robust $A5\text{-}L$ framework when an admissible serial-shaped prefix reaches Dead_K^W . A correct system with fair but genuinely unbounded response gaps also lies outside the finite-bound Policy-S fragment. Structural, direct Policy-L, invariant, exploration, and tool-certificate methods remain alternative $A5\text{-}L$ routes when $A5\text{-}L$ is true.

Dead_KS and the related owner/terminal classifications are optional diagnostic aids, not primary proof premises. Operationally, the route is expressed using conventional source simulation and timeout/service assertions and is expected to be highly automatable for finite transactions and bounded protocols. Accordingly, one complete clean run is the principal dynamic design-user artifact rather than the entire proof package. Its use as a verified theorem discharge retains proposed-reduction status until the K-P obligations and certificate interfaces are mechanized and qualified; a universal conclusion separately requires checked Core.KCutClosure_post and K.CertCov discharges.

M.7 Implementation qualification and evidence

The formal proofs specify a contract between source modeling, implementation behavior, and validation evidence. They do not prove that ordinary synthesis automatically discharges the named conditions. A concrete theorem application must identify the selected Sys_ref and abstraction boundaries; establish source well-formedness and witness conditions; provide the local process, channel, atomic-interaction, environment, reset, footprint, request-snapshot, agreement, and coverage evidence required by the safety construction; and establish faithful realization of every selected channel capacity or elaborated boundary.

For liveness, the application must additionally establish the exact cutpoint correspondence, phase and drain bridge, A5-L and its chosen discharge route, fairness and progress assumptions, nonwithdrawal, applicable waiver rules, environment/frontier scope, point-to-point ownership or explicit arbiter modeling, and universal certificate coverage when the total conclusion is claimed. Tool documentation, generated certificates, static checks, formal proofs, simulation monitors, randomized-latency stress tests, and targeted RTL/interface checks may all contribute evidence, but the evidence must be sufficient for the named premise it is used to discharge.

At the architecture level, the proof separates operational semantics, safety/replay construction, KObs cutpoint correspondence, and complete-run Policy-S evidence. This separation prevents trace compatibility, cutpoint reconstruction, and source-side liveness evidence from being treated as interchangeable assumptions.

Division of verification responsibilities. The following table separates the principal dynamic design-user activity from per-instance flow evidence, reusable tool qualification, and framework work that remains to be completed. Ownership may be shared in a particular implementation; the table identifies the expected primary source of evidence.

Primary source of evidence	Principal items	Current interpretation
Design user	Run the selected instrumented Policy-S model to its declared maximal terminal state for a terminating instance; review response failures, declared waivers, and design/environment contracts.	This complete execution is the principal dynamic user artifact. It supplies K.RespClean only when bound to the qualified selected instance and accompanied by the remaining evidence.
Per-instance tool / verification flow	Select and bind Sys_ref/Sys_K, capacities, elaboration, stimulus, witness, reset and environment contracts, and discharge the applicable per-instance side conditions recorded in M.7a, including	These obligations are not discharged merely by a clean observed run. They may be generated or checked automatically, but remain theorem premises for the selected design instance.

Primary source of evidence	Principal items	Current interpretation
	K.PSF/K.EnvMF/K.ResetEpoch applicability, K.CompleteS, total K.RespCov, B-faithfulness, K.Drain/K ϵ , and J.TraceCertCov/K.CertCov when the corresponding total claim is made.	
Reusable tool qualification	Qualify the concrete Policy-S simulator semantics and deterministic scheduling/arbitration used by SDet; validate standard lowering/elaboration schemes, certificate schemas, generators, and independent checkers.	Much of this qualification can be reusable across designs. SDet and the selected certificate binding must nevertheless be re-established whenever capacities, elaboration, lowering, arbitration, normalization, environment/reset behavior, or another history-affecting choice changes.
Framework / mechanization work not yet completed	Mechanize the Appendix K-P obligations, including K-O3/K-O4 for progress dominance and request reachability; qualify the certificate interfaces; and provide a generic J.CertExist theorem or an equivalent verified package-existence route if a generic universal-coverage construction is desired.	This document now supplies explicit paper proofs for K.2b.R and K.2b.P, but the reduction and certificate stack retain proposed, not fully mechanized and flow-qualified, status.

M.7a Side-condition discharge record

Before applying Theorem J.1, Corollary J.1 cutpoint transfer, or the Appendix K liveness-preservation theorem to a concrete design, the verification flow shall maintain an explicit side-condition discharge record. The record need not use one physical file, but every referenced artifact shall be versioned and bound to one selected theorem instance through the common certificate header of Definition K.CertHdr. For each theorem premise identified below, the record shall state whether the premise is inapplicable to the selected theorem scope, satisfied by a simple syntactic/design restriction, discharged by tool/RTL evidence, or discharged by a source/RTL witness construction. A condition required by the selected theorem scope may not be marked inapplicable. Remark Core.Det uses this same evidence axis; derived results such as I.DR or theorem-derived Core.KCutClosure_post are not converted into independent assumptions merely because their proof artifacts are recorded. The named audit entries are Core.RegSeq registered sequential semantics; Core.ReachPrep overlap-aware source reach/preparation; ClockScope single-global-clock theorem scope; Core.QSync qualified synchronous same-cycle settling; Core.CycleClosure full-state source cycle qualification/closure; Core.IssueFair source-issue weak fairness; B1 observable RTL projection determinism; B2/B3 scheduler/channel progress; B4/FPD finite internal stutter; B5/B6-Terminal terminal-idle adequacy; A6 point-to-point ownership; A8/A9/A10 barrier/cardinality/reset-empty well-formedness; Core.SyncFence synchronization-completion conformance; K.CertPkg certificate-package integrity; WC; J.LocalGWR local witness-construction certificates (or optional J.GWR-Direct); Sys_B^elab; B-faithfulness; J.OfferReach

residual-offer realizability; J.OfferConf residual-offer boundary conformance; the composite J.KObsConf KObs cutpoint-conformance wrapper; K.RLAdmReach admissibility of the exact selected source witness; Core.KCutClosure_post theorem-derived K-facing cutpoint existence/non-vacuity for universal RTL conclusions; K.CertCov total K-ready package coverage for universal reachable-cutpoint conclusions; J.Mem shared-memory lowering; RW reset well-formedness (RW1–RW5, when dynamic reset is used); K.Drain; Kε; K.BF blocking-only serial fragment; K.InterruptionGate shared nested interruption schema with K.ResetEpoch/K.TerminalDisposition instantiations; SDet deterministic selected Policy-S simulator; A5-S selected-instance serial liveness; A5-L scheduler-robust liberal liveness; K.CV value determinism; NB-Align witness-selected non-blocking alignment; and L.Dir one-directional alignment / RTL-side non-interference (required whenever Appendix L snooping / co-simulation is used), as applicable to the selected theorem scope.

Core.RegSeq — registered sequential-interface scope and conformance.

Trigger: Required for every Appendix I/J/K theorem instance containing a sequential process whose message requests, Push payloads, or continuation can depend on a message result.

Discharge: Classify each process as R2C combinational or Core.RegSeq sequential. For each sequential process, prove or validate that process-driven request and Push-data outputs for Horizon t are determined from registered state and operands stable before the L2 settle point; that direct inputs satisfy their E2/direct-input stability contract; and that every use of a cycle-t returned data/status result has strictly later Horizon. Tool-side Cat# or equivalent schedule/RTL evidence and source-side simulation, lint, wrapper, or Appendix L evidence shall identify and reject, isolate, or explicitly remodel zero-time NB-result cascades and retry loops. Evidence examples include schedule reports, control-step allocation, generated RTL cone/register checks, source lint, and cycle-accurate transactor or snooping artifacts.

Core.ReachPrep — overlap-aware source reach/preparation.

Trigger: Required whenever the selected Sys_ref/Policy-L theorem instance permits a source-later dynamic operation to become reached before an earlier same-interval blocking operation process-facing commits, including the straight-line eager cases used by Examples K.CE/K.CE-2, the Appendix K A5-L blocking-rendezvous example, and pipelined/overlapped cases.

Discharge: Identify the source/tool rule or certificate that validates each permitted finite dependency-independent L1 preparation segment under Definition Core.ReachPrep, including dynamic source order, no synchronization crossing, Core.RegSeq/NoDependentUse dependencies, and the Core.16 post-settle prohibition. For every RTL_B prefix included in a Post → Ref coverage claim, the record shall verify that the selected Sys_ref ReachPrep semantics admit every dependency-independent preparation/reach segment required by the accepted J.LocalGWR or J.GWR construction for that prefix; the source semantics may be more permissive than the observed RTL schedule but shall not exclude preparation needed to realize a covered RTL witness. The record shall state explicitly whether ordinary straight-line q0; q1 eager reach is admitted. If it is not admitted for an instance, examples or proof arguments that rely on that straight-line liberal behavior may not be used unchanged.

Evidence examples: a schedule/preparation certificate, source dependency/footprint check, a qualified eager-source simulator semantics, or a tool-generated mapping from RTL issue stages to legal Core.ReachPrep segments.

ClockScope — single-global-clock theorem scope.

Trigger: Required for every theorem instance using the formal results of Appendices G–K.

Discharge: Confirm that the selected theorem instance is interpreted under one global clock/cycle relation for every process and explicit abstract channel to which the G–K rules are applied. A larger multi-clock design may be partitioned into synchronous islands, but any clock-domain crossing used by the selected theorem instance must be excluded from the single-clock

comparison or represented by a separately specified and verified CDC/refinement boundary under an explicit multi-clock semantic layer. In the absence of that layer, the G–K theorems shall not be applied across clock domains. Evidence examples include a clock-domain/elaboration manifest, structural CDC partition report, or an equivalent checked instance-scope declaration. Core.QSync — qualified synchronous same-cycle settling scope and conformance.

Trigger: Source-side Core.QSync is required on the normal J.LocalGWR → J.GWR-Comp → finite-prefix Theorem J.1 route, because J.RegPhaseNF/J.UnitExt form a finite L2-settled source snapshot for every constructed Horizon; it is also required whenever the application otherwise invokes source Core.Settle/NormRef/KCut settling. A directly validated J.GWR package may instead carry independently checked finite L2-settled segments for its Horizons and need not separately discharge global source-side Core.QSync solely for Theorem J.1. Post-side Core.QSync is required whenever the application invokes post KCut/NormPost settling, including Corollary J.1 cutpoint transfer and every Appendix K application. Any R2C combinational-process fixed-point obligation remains independently applicable.

Discharge: Establish the hypotheses of Definition Core.QSync for the concrete instance. Use the ClockScope entry as the concrete audit evidence for QS1. For the source side, show the Core.Settle L2 frame, stable sequential drives under Core.RegSeq/direct-input contracts, inclusion of R2C combinational components in the settle network, QS3 relation-to-network realization/adequacy for every phase-legal same-cycle settling step, finite acyclic deterministic cycle-local settling, and explicit-boundary completeness. For Sys_B^elab(E), cite the Appendix Q qualified-settling/template evidence required by Q.1(i) for the elaboration-owned subnetwork and additionally establish template/design settling composition for the complete source network. Every dependency edge crossing the design/template partition shall be included in the whole-network analysis, the combined transition relation shall satisfy the QS3 realization/adequacy condition, and the combined network shall satisfy QS2–QS5; in particular, no combinational settling cycle may be introduced across the partition. A sufficient mechanically checkable QS4 discharge is a combined dependency rank or topological order in which every design-local, template-local, and cross-partition settling edge advances the order. For RTL_B, establish the analogous K-origin frame, stable registered drives/cycle-t inputs, QS3 relation-to-network realization/adequacy for the complete K-facing same-cycle settling relation, finite acyclic deterministic combinational/handshake settling, and complete selected-boundary mapping/B-faithfulness. The checker shall validate enough of these structural facts to invoke Theorems Core.QSync-Ref and Core.QSync-Post.

Evidence examples: a syntactic/design restriction plus source/elaboration template proof; an independently checked combinational-dependency/clock/register-frame certificate; a combined dependency-rank or template/design interface-compatibility certificate covering cross-partition edges; structural RTL/netlist analysis; a qualified template-library theorem; or equivalent tool/RTL evidence. Trust accounting follows K.CertHdr: an unverified tool report used as an axiom enlarges the declared trusted base, while an independent check of the concrete artifact does not require trusting the producing HLS tool for that fact.

Scope boundary: Core.QSync is same-cycle settling evidence only. It does not discharge Core.CycleClosure full-state/source-cycle/environment definedness, B2/B3 cross-cycle progress, or the terminal Core.EnvTerminal premise. No generic RTL cross-cycle totality premise is required by the finite-prefix J.1 or K-facing cutpoint-closure results.

Core.CycleClosure — full-state source cycle qualification and closure.

Trigger: Required when a theorem constructs a complete/fair source continuation through Core.LAdm, notably Lemma K.2b.E/K-O7. It is not required by finite-prefix Theorem J.1 or by RTL K-facing cutpoint closure.

Discharge: Bind Definition `Core.CycleResult` and its `Core.CompleteCycle` projection to the selected source/environment semantics; prove the `Core.CycleClosure` record: full-state `CycleResult` typing, `nextState` clock/environment/auxiliary advancement, exact scoped `RESET` emission under `RW1-RW5`, nonterminal source-cycle existence, finite L1 preparation, finite L2 settling (`Core.QSync-Ref` or an allowed equivalent route), compatible `EnvStep` definedness, and the C2 partial-cycle extension property needed by Lemma `Core.CycleChoiceExt`: every finite phase-legal selected issue/boundary segment used by a continuation proof must admit completion to a typed `CycleResult`. Verify that source-side `SilentCycle` is a `CycleResult` with no selected Σ commit rather than an ϵ /identity step, and that any proof-internal `RESET` remains present in `resetUnits`.

Evidence examples: mechanized source cycle semantics with a cycle-completion lemma; source FPD/preparation plus `QSync` certificate and checked partial-cycle extension; environment state-machine totality proof; or a qualified source-model theorem whose concrete artifact is independently checked.

Core.IssueFair — weak fairness of source issue.

Trigger: Required for complete `Core.LAdm` executions consumed by Lemma `K.2b.E/K.2b/K.2b.R` and wherever the `Policy-S` route invokes issue fairness.

Discharge: For each relevant reached operation, prove that if its issue transition remains legal at every applicable L1 opportunity within the reset epoch, it is eventually issued. The legality predicate must include `Core.16`; an operation blocked as source-later is not `IssueEnabled` and therefore creates no `IssueFair` obligation. Intermittently legal operations do not trigger this weak-fairness antecedent.

Evidence examples: simulator scheduler proof, source operational-semantics theorem, fairness assertion over issue-enable state, or mechanized dovetailing argument.

B2/B3 — boundary-action and channel progress.

Trigger: Required whenever a complete/fair continuation is used by Lemma `K.2b.E`, the `K.2b` dominance/response arguments, `K.CompleteS`, or another theorem that explicitly requires such a run. Appendix I.5 and the finite-prefix J.1 correspondence do not require B2/B3 merely to construct finite replay prefixes.

Discharge: Record the concrete scheduler/arbiter and channel evidence establishing B2 weak fairness for continuously `ChannelEnabled` `Push/Pop/Sync` actions and the B3 `P_push/P_pop` channel-progress obligations on every relevant explicit abstract channel. This entry is distinct from `Core.IssueFair`, which concerns request assertion before a boundary action is enabled.

Evidence examples: fair-arbiter RTL proof, ready/valid liveness assertions, channel-library qualification, or a mechanized scheduler invariant.

B5/B6-Terminal — quiescence closure and terminal idle adequacy.

Trigger: B5 is required where its bounded quiescence/normalization role is invoked. The B6 terminal-idle branch is required only when an infinite execution is extended by idle ticks after a permanently idle state.

Discharge: Prove the B5 closure premise for the selected bucket semantics and, for any B6 suffix, prove `Core.TerminalIdle`: no future registered/internal machine transition and no permitted future environment evolution can change non-clock state or re-enable work.

`Core.EnvTerminal` supplies only the environment component; `Core.CycleClosure`'s environment-definedness clause or `Core.EnvTerminal` alone is not sufficient. `StandardEnv` stimulus exhaustion plus a machine-side stable-terminal/permanent-idle proof is one sufficient route. A currently idle but nonterminal state must instead continue through `Core.SilentCycle`.

Evidence examples: bounded quiescence proof plus stimulus-exhaustion and stable-machine

certificate, explicit terminal state with no registered successor effects, or a model-checked permanent-idle invariant.

B1 — observable RTL projection determinism. Trigger: Required wherever B1 is a theorem premise, including the finite-prefix J.1 correspondence and Appendix K routes that cite RTL observable determinism. Discharge: Under the fixed initial state and stimulus, prove uniqueness of the RTL_B observable Σ projection required by Definition B1. Internal ϵ paths, issue annotations, request attribution, and next-cycle totality need not be unique; those stronger properties are discharged separately. Evidence examples: deterministic RTL semantics/refinement proof, qualified simulation theorem, or formal trace-projection uniqueness check.

B4/FPD — finite internal stutter / bounded local source progress. Trigger: Required wherever Lemma I.1, finite replay/preparation normalization, B5 charging, or the generic source-cycle finiteness argument relies on FPD/B4. Discharge: For each relevant process P, provide the finite D_P bound (or an equivalent well-founded measure) showing that consecutive internally advancing ϵ -work reaches the next Σ action or a stable external/peer wait in finitely many steps. A certified Lemma I.3 preparation segment may cite its local certificate; generic Core.CycleClosure use requires the corresponding uncapped per-cycle source-control finiteness evidence for its source-cycle-existence clause. Evidence examples: finite pipeline/FSM-depth proof, source/control graph ranking, simulator semantics theorem, or tool-generated progress certificate.

A6 — point-to-point ownership / explicit arbitration modeling. Trigger: Required wherever Appendix K constructs WFG edges, complementary endpoint ownership, or K.2b point-to-point dominance obligations. Discharge: Show that every explicit message boundary used by the theorem has one producer and one consumer, or remodel shared resources through an explicit arbiter plus point-to-point subchannels whose choices are included in the selected instance. Evidence examples: elaboration/ownership manifest, structural channel check, arbiter-lowering certificate, or source/RTL connectivity proof.

A8/A9/A10 — barrier, endpoint-cardinality, and reset-empty well-formedness. Trigger: A8 is required whenever SyncChannels/barriers are omitted from the internal Push/Pop WFG; A9 is required for scalar per-cycle endpoint semantics; A10 is required for buffered channels at initial/reset boundaries. Discharge: For A8 prove matched barrier participation/no permanent bypass under the fixed stimulus; for A9 prove at most one Push and at most one Pop commit per channel per cycle unless an explicit multi-lane model is used; for A10 prove logical buffered occupancy is zero at each selected reset boundary or explicitly model any preload. Evidence examples: source/elaboration checks, barrier-count/participation proof, interface cardinality assertions, and reset-state/channel initialization checks.

Core.SyncFence — synchronization-completion conformance.

Trigger: Required whenever a synchronization call s selected into S can have a blocking Push/Pop q in $\text{pref}_S(s)$ on a side of the comparison. The obligation is vacuous for a synchronization boundary with no such blocking predecessor.

Discharge: Establish the applicable R3/E5/Core.SyncFence condition that every such q process-facing commits no later than s and that no issued-but-uncommitted q survives the synchronization into the succeeding source interval. For source operational semantics, a same-cycle q/s completion is represented by the selected L3 boundary-action batch with q accounted for before the interval update. For RTL_B, no physical sub-cycle ordering is required: the implementation evidence must establish the same commit cycle and the certificate/annotation convention must attribute the predecessor completion before advancing the formal source-interval bookkeeping. In $\text{Sys}_B^{\text{elab}}$, bind q to the designated process-facing completion event

rather than an arbitrary internal staging transfer. Evidence examples include schedule/control-state evidence, an RTL/interface monitor or assertion, a structural/formal refinement check, or a checked process-facing completion map in the elaboration certificate.

K.CertPkg — certificate identity, schema, and trust boundary.

Trigger: Required whenever a generated or manually assembled certificate package is used to discharge Corollary J.1, Theorem K.1A, Corollary K.1A-Total, Theorem K.1, A5-S/A5-L, or CF-1 through CF-5.

Discharge: Provide the common K.CertHdr fields: certificate-schema and document/theorem version; a digest or unambiguous identifier for the selected source, Sys_ref/Sys_K, and RTL artifact; E, B_E, explicit boundaries, projections, ownership, and K.Low lowering where applicable; fixed stimulus, environment, reset policy, witness w, and WC provenance; KObs-facing waiver set W and closure evidence; the selected A5-L discharge route; the Core.QSync structural-evidence provenance and Core.QSync-Ref/Core.QSync-Post theorem references used to derive standard K-facing settling/closure, together with the K.CertCov coverage route when a universal reachable-cutpoint conclusion is claimed; and, for the Policy-S route, the concrete deterministic simulator/scheduler, deterministic arbitration, Policy-S issue semantics, ϵ -normalization identity, SDet evidence, K.Resp monitor schema, finite bounds, reset/end-of-test disposition, response waivers, and coverage-preservation evidence; plus the theorem scope, clock-scope and Core.SyncFence side-condition references where applicable, other side-condition references, generator/checker versions, and declared trusted base. Every subordinate KObs or A5 certificate shall cite the same header identity. Derived K-facing predicates and K.Resp outcomes shall be recomputed by the checker from their respective artifacts rather than accepted as authoritative fields. A QSync property accepted only from an unverified tool report is part of the declared trusted base; a property independently checked from the bound artifact is recorded as checked evidence instead.

Evidence examples: A signed or hashed manifest binding all artifacts, an independently checked structural QSync certificate, qualified checker/tool versions, and reproducible source/RTL identities.

- WC — witness-compatible ϵ choices / non-blocking outcomes.

Trigger: Required when failed non-blocking polls or other ϵ -modeled choices can affect later Σ -visible control flow, frontier membership, request attribution, guard values, payload values, or other later Σ -visible choices.

Discharge: Provide evidence that proves the actual WC premise used by Appendices I–J. The audit entry shall identify the provenance route by which WC was established:

- NB-CF route: show that failed PushNB/PopNB outcomes do not affect later Σ -visible control flow, frontier membership, request attribution, guard values, payload values, or other later Σ -visible choices. Under this route, the failed-poll witness may be the identity or arbitrary selector, subject to the applicable latency-sensitive local-protocol conditions.
- Appendix L snooping route: provide a witness selector W satisfying Lemma L.2. Show that the snooped outcomes cover every ϵ -modeled source choice that can affect later Σ -visible behavior, that the wrapper changes only ϵ -choice selection, and that it does not add, remove, reorder, or relabel Σ -visible events.
- Direct witness-construction route: otherwise construct and prove the μ_Q / WC witness used by Appendix I/J.
- Mixed per-site route: partition the relevant non-blocking or ϵ -choice sites among the applicable NB-CF, snooping, or direct-construction routes, and show that the partition covers every relevant site.

NB-CF and Appendix L snooping are therefore provenance routes for proving WC, not separate theorem premises that replace WC. When useful for review or automation, the WC audit entry may include optional per-site metadata identifying each relevant call site or ϵ -choice site, its selected provenance route, and the corresponding source restriction, tool/RTL evidence, wrapper evidence, or witness construction.

Evidence examples: Source inspection or a static control/dataflow check establishing NB-CF; a generated WC witness or certificate; a wrapper proof or monitor specification satisfying Lemma L.2; a co-simulation/snooping harness, subject to the L.Dir entry below; or a complete per-site coverage report for a mixed discharge.

- **Sys_B^{elab}** — elaborated source reference model.

Trigger: Required when ordinary Sys_B capacities at the outer abstract boundaries are not sufficient to represent the RTL realization as ordinary E4 bounded-FIFO/rendezvous behavior; examples include multi-input/join couplers, staged/bundle/join channels, and sync-boundary / epoch-gate capacity withdrawal.

Discharge: Provide a well-formed elaboration package E defining the explicit abstract channels, capacities, projection Π_{elab} , and E4 ChannelEnabled/ChannelDisabled behavior used by Sys_B^{elab}(E). The package shall additionally satisfy Definition Q.1(i): its cycle-local elaboration/ready-valid/guard/join/coupler/epoch-gate subnetwork must satisfy the applicable Core.Settle frame and template-side finite-acyclic deterministic-settling/boundary-completeness conditions. That elaboration-side result may be proved once for each qualified elaboration template and then bound to the concrete E instance. It does not by itself establish whole-source QualifiedSynchronousModel_Sys_ref(I); the Core.QSync discharge above additionally combines the template evidence with the design-owned Core.RegSeq/direct-input/R2C/non-template-boundary facts and the checked template/design settling-composition condition, including complete-network QS4.

Evidence examples: Tool-generated elaboration plus an independently checkable template certificate, structural back-annotation report, Appendix Q-style construction, qualified elaboration-library theorem, or manual elaboration justified against the RTL channel structure; for the complete source-QSync discharge, also provide the applicable combined dependency/interface certificate required by the Core.QSync entry.

- **B-faithfulness** — faithfulness of each back-annotated capacity B(c) and chosen explicit abstract channel boundary.

Trigger: Required for every back-annotated explicit abstract channel c whose B(c), occupancy occ_c(t), E4 ChannelEnabled/ChannelDisabled behavior, BoundaryReq facts, or Appendix K blocking/WFG behavior is used by the selected theorem instance. For such a channel, B-faithfulness is a required per-channel obligation and shall not be marked inapplicable.

Discharge: Show all of the following for the chosen abstract boundary of c:

Boundary completeness — every Σ -visible transfer of c in RTL_B crosses the chosen abstract boundary as the corresponding committed Push_c or Pop_c, with no hidden side entrance or exit.

Exact boundary behavior — for every relevant execution state and boundary action, the concrete realization permits the action exactly when the E4 abstract channel permits it. In particular, realized unpoped occupancy is bounded by B(c), and the realization contains no hidden storage or credit beyond the capacity represented by B(c).

No hidden enablement or disablement — no unmodeled grant, arbitration, throttling, or backpressure condition changes whether a boundary action is permitted. Any such condition

that affects boundary behavior shall instead be represented by explicit $\text{Sys_B}^{\text{elab}}$ boundary structure.

ϵ -opacity of internal movement — movement of data within the realized channel between chosen boundary commits is internal ϵ -behavior and creates no additional Σ -visible Push/Pop event or Appendix K WFG blocking cause.

Evidence examples: A trusted back-annotation algorithm; a per-channel capacity and boundary certificate; a structural RTL check; an RTL/interface monitor; a formal refinement check comparing the realized boundary behavior with E4; or equivalent tool/RTL-flow evidence establishing the complete B-faithfulness obligation.

- J.TraceCertCov — Post $\rightarrow$ Ref certificate coverage for safety claims.
Trigger: Required whenever Corollary J.1-Safe is applied over a coverage domain D.
Discharge: Identify D and prove that every $\tau_{\text{post}} \in D$ carries one route-complete Post $\rightarrow$ Ref package. On the normal compositional route, provide the applicable J.LocalGWR evidence, source-side QualifiedSynchronousModel_Sys_ref(I)/Core.QSync-Ref settling evidence required by J.GWR-Comp, and all remaining applicable Post $\rightarrow$ Ref premises of Theorem J.1. On the direct route, provide a directly validated J.GWR package whose checked reachable phase-normal chain contains a finite L2-settled segment for every represented Horizon, together with all remaining applicable Post $\rightarrow$ Ref premises. Bind all evidence to the same theorem instance, stimulus, observation boundary, and selected RTL_B prefix. For a universal RTL safety claim, D shall contain every reachable RTL_B prefix relevant to Φ . This safety-side coverage fact does not imply K.CertCov.
- J.OfferReach / J.OfferConf / J.KObsConf — KObs cutpoint correspondence. Trigger: Required whenever Corollary J.1 cutpoint correspondence is used, including every Appendix K application that transfers Pending_P, BoundaryReq, rendezvous endpoint requests, ChannelEnabled/ChannelDisabled, Front_P, WFG, or derived deadlock facts between RTL_B and Sys_ref. Appendix R only summarizes this transfer and adds no separate obligation. The evidence shall be packaged as, or mapped unambiguously onto, the CF-0 KObs cutpoint certificate of K.4e. J.OfferReach discharge: For the same selected Theorem J.1 source witness, witness w, and ϵ -normalization used by the application, exhibit H and O and show that the selected source K-facing cutpoint $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$ satisfies $H = H_{E,w}(\sigma_{\text{ref}})$ and $O = \text{ResOffers}_{E,w}(\sigma_{\text{ref}})$, with all operational Issue/Commit, E3/E5, reset-epoch, elaboration, and source-transition rules respected. This is an image/realizability obligation; well-typedness of $\langle E, w, H, O \rangle$ alone is insufficient. The complete Issue history may remain in the referenced source witness rather than being duplicated in the CF-0 record. J.OfferReach does not by itself establish Definition Core.LAdm admissibility; that K-specific fact is K.RLAdmReach below. J.HCanon discharge: Establish the hypotheses of Lemma J.HCanon for the selected source/RTL annotated prefixes and fixed theorem-produced or directly validated J.GWR package; record that both prefixes produce the same canonical enriched replay-history quotient H. J.OfferConf discharge: At the selected RTL K-facing cutpoint $p_{\text{post}} \in \text{KCutReach_post}(I)$, enumerate every KObs-relevant asserted endpoint-owned request at the explicit boundaries of E and prove bidirectional, unique attribution between that physical request set and O using the operational-attribution component produced by J.GWR-Comp (or supplied by J.GWR-Direct), Appendix C, and I.A0. Record the physical RelevantReqSet_E(p_{post}), ReqProjection_E(O), their equality, request-to-offer completeness, no-hidden-request, boundary completeness, issued-summary prefix closure, reset-epoch typing, and endpoint cardinality. A monitor or certificate that reports only requests it can already attribute is insufficient unless it separately proves that no other KObs-relevant asserted request exists. J.KObsConf discharge: For that same selected tuple $(p_{\text{post}}, \tau_{\text{ref}}, \sigma_{\text{ref}}, E, w, H, O)$, additionally establish B-faithful/E4 interpretation at every

explicit boundary, reset alignment, and ε -opacity of normalization. J.KObsConf is the composite application wrapper; it no longer requires independent proof of a broad source-value/frontier/channel/enablement slice, because those facts are reconstructed from equal KObs by Lemmas J.KObs-Proc–J.KObs-WFG. Audit priority: J.OfferConf is the authoritative RTL request-abstraction boundary. Its completeness and no-hidden-request directions shall receive the same explicit review and certificate coverage previously assigned to the request/pending clauses of the broad J.KObsConf relation. Evidence examples: A CF-0 tool-generated paired KObs/cutpoint certificate; a source execution log proving OfferReach; an RTL/interface request-attribution monitor plus boundary-completeness proof proving OfferConf; structural back-annotation and B-faithfulness evidence; a co-simulation witness log; a bounded proof over generated RTL; or an equivalent combined certificate.

K.RLAdmReach — composed Core.LAdm qualification of the exact selected source witness.

Trigger: Required whenever Theorem K.1A, Corollary K.1A-Total, or Theorem K.1 uses the source witness selected by Corollary J.1. It is not required for an ordinary safety-only Corollary J.1 application.

Discharge: Bind the exact tuple $(\rho_{\text{post}}, \tau_{\text{ref}}, \sigma_{\text{ref}}, E, w, H, O)$ used by J.OfferReach/J.OfferConf/J.KObsConf. On the normal compositional route, reference Corollary J.RegPhaseReach-Comp and its theorem-derived J.RegPhaseReach fact obtained from Core.RegSeq, the enriched Lemma I.3 records, J.RegPhaseNF, and Horizon-batched J.UnitExt; on the direct route, supply an equivalent independently checked phase-normal witness. Separately supply K.DrainReach for the same prefix, proving nonwithdrawal, $K\varepsilon$, Core.Drain/K.Drain from every L2 fully-disabled-frontier activation point, and Core.SettleKBridge from each persisting activation through any intervening drain to the exact selected K-facing cutpoint, together with the common instance/stimulus/witness binding. The checker combines J.RegPhaseReach and K.DrainReach to establish $\tau_{\text{ref}} \in \text{Pref_L}^{\text{adm}}(I)$. KObs equality, OfferReach alone, a bare “admissible=true” flag, or ambient premise names do not discharge this obligation.

Core.KCutClosure_post — K-facing cutpoint existence / non-vacuity (derived in the standard QSync scope).

Trigger: Required by Corollary K.1A-Total and the universal user-facing Theorem K.1 conclusion. It is not required to apply Theorem K.1A to one already selected certified KCut.

Standard discharge: Cite the accepted Core.QSync audit entry for RTL_B and Theorem Core.QSync-Post. No independent per-instance semantic normalization argument is required beyond the structural QSync evidence: the theorem proves that every reachable RTL K-facing cycle-evaluation origin admits one finite same-cycle normalization to a unique $\rho_{\text{post}} \in \text{KCutReach_post}(I)$ before a later cycle-advancing registered update. With $K\varepsilon$, Lemma Core.KCutObserve supplies the persistent-blocker observation consequence. Record the theorem/certificate identity in K.CertHdr.

Scope note (intentional standard-scope narrowing): Core.KCutClosure remains the downstream semantic interface consumed by Appendix K. The standard G–K scope now requires the finite-acyclic QSync settling discipline and therefore withdraws the former independent standard-scope Core.KCutClosure_post discharge alternatives based solely on an arbitrary inductive RTL invariant or checked normalization certificate. A non-QSync design is not asserted incorrect, but it is outside the standard user-facing theorem scope unless an explicitly defined and separately qualified extension proves the same Core.KCutClosure interface under its alternative settling semantics.

K.CertCov — total K-ready package coverage for universal reachable-cutpoint conclusions.

Trigger: Required whenever Corollary K.1A-Total or the universal user-facing Theorem K.1 conclusion is claimed, together with Core.KCutClosure_post, which is theorem-derived from the

Core.QSync structural evidence in the standard scope. It is not required to apply Theorem K.1A to one already certified cutpoint.

Statement and discharge: Under the fixed theorem instance and stimulus, every $p_post \in KCutReach_post(l)$ shall admit at least one recorded reachable-prefix/normalization pair (τ_post, v_post) satisfying $NormPost_l(\tau_post, v_post, p_post)$ and a $K.CertifiedCutpoint$ package for that triple. The package includes the applicable $J.LocalGWR/J.GWR-Direct$ and Corollary J.1 evidence, $J.HCanon/J.OfferReach/J.OfferConf/J.KObsConf$, $B-faithfulness/E4$ and the selected canonical QSync NormPost facts (or the explicitly qualified extension normalization facts), and $K.RLAdmReach$ including $Core.SettleKBridge$, all bound to one $K.CertHdr$ identity. $K.CertCov$ may be discharged by a proved total certificate-extraction function on $KCutReach_post(l)$, an inductive simulation/refinement relation whose invariant contains the package data, exhaustive finite-state exploration, a verified certificate generator, a tool-specific package-existence theorem, or the target $J.CertExist$ metatheorem. Merely supplying a package for the observed run, a finite collection of sampled K-facing cutpoints, or a clean simulation does not establish $K.CertCov$.

Scope: Coverage is required exactly on the reachable $Core.KCut$ RTL domain on which the K-facing deadlock predicate is defined; arbitrary transient RTL microstates and arbitrary pre-normalization prefixes need not carry CF-0 packages. $Core.KCutClosure_post$ separately proves that a persistent K-relevant blocking configuration cannot be hidden indefinitely in those excluded transient states.

- **K.Drain** — drain-only blocked-frontier discipline and settle-to-KCut bridge. Trigger: Required for Appendix K whenever the selected liberal Sys_ref or RTL_B execution may have source-later same-interval blocking work already issued while an earlier minimal blocking frontier remains uncommitted. This includes pipelined-loop overlap, other explicitly modeled eager/overlapped issue under Policy L, and ordinary non-overlapped control when Lemma K.2b or drain normalization relies on the absence of newly launched work past the blocked point. Discharge: Show the full Definition Core.16 discipline—blocked-frontier identification, no new later work, no request withdrawal, drain-only downstream progress, finite explicit boundary storage/credit, and any required sync-boundary capacity withdrawal in Sys_B^{elab} —and discharge $Core.SettleKBridge$ from every persisting L2 activation to the selected later drain-closed $KCut$, with $K\epsilon/WFG$ -inert normalization and every intervening drain commit represented explicitly. Ordinary source operations on distinct interfaces remain source-ordered; any multi-frontier behavior used by Appendix K must be represented by explicit source/elaboration structure rather than by implicit incomparability. Under Definition Core.LAdm this discipline and bridge are part of Appendix K admissibility once the settled-frontier trigger occurs; they are not part of the basic Policy-L issue rule before that trigger.

K.BF — blocking-only serial fragment. Trigger: Required by Lemma K.2b, Corollary K.2c, $K.PSF$, and the Policy-S route whenever the bounded-response reduction is used. Discharge: Show that every WFG-relevant wait is a persistent blocking Push/Pop on a finite B-faithful point-to-point boundary; control, payload, and later operation identity are deterministic under $K.CV$ /fixed witness choices; shared arbitration is explicit and deterministic or witness-fixed; and local computation between message operations is bounded so pure-computation divergence cannot replace the process-facing serial frontier used by K.2b.Q. Because those waits are ordinary blocking requests, Axiom Core.IC supplies the exhaustive lifecycle: once issued, the request remains the same pending dynamic instance until process-facing commit or an affecting RW2 RESET, while DeclaredTerminal may end the execution with it still pending. Lemma K.BF-Persist packages that fact for the frozen-frontier proof. A protocol with independent nonterminal cancellation, withdrawal, retry, or reattribution of an outstanding blocking request is outside the

G–K theorem model and requires a separately defined operational extension before any A5-L route in this theorem stack can be used. Evidence examples: source/control-flow qualification, explicit arbiter lowering, finite local-progress proof, per-boundary B-faithfulness certificate, and conformance of every admitted blocking interface to Axiom Core.IC/RW2.

- $K\epsilon$ — WFG-inert ϵ / no hidden micro-timing control decisions.
Trigger: Required for Appendix K liveness/deadlock preservation when ϵ -steps occur between Σ -visible cutpoints.
Discharge: Show that ϵ -only steps do not create, remove, or redirect the relevant Appendix K WFG blocking causes except through the explicit Σ -visible or E4-modeled channel state already represented in Sys_ref .
Evidence examples: RTL structural argument, scheduler proof, bounded pipeline/drain argument, formal check over the WFG abstraction, or tool certificate.
- SDet — deterministic selected Policy-S simulator. Trigger: Required for K.2b, K.2c, K.1, K.PSF-1, or CF-S. After fixing Sys_K , capacities/elaboration, initial/reset state, stimulus, environment function, witness-selected ϵ outcomes, deterministic arbitration, concrete SystemC/pre-HLS scheduling, Policy-S issue semantics, and ϵ -normalization, SDet defines a functional Policy-S next-state semantics and selects one run artifact p_S^{det} . For every finite or infinite run artifact p generated from the same initial state under exactly those selected semantics, the committed histories of p and p_S^{det} are prefix-comparable modulo finite WFG-inert ϵ -stutter; equivalently, the selected semantics generate no second incompatible committed history. A finite artifact may therefore be a proper prefix of p_S^{det} without contradicting SDet. WC and deterministic replay give reproducibility relative to the selected history; SDet is the separate global premise fixing every remaining history-affecting choice. Changing any such choice creates a different theorem instance. Discharge by a simulator-semantics proof, deterministic qualification, schedule/arbitration certificate, or mechanized functional-determinism and history-comparability theorem. SDet does not assert that p_S^{det} is maximal, fair, complete, or bounded-response clean. Those facts are separate K.CompleteS and A5-S/CF-S obligations.
- A5 — primary selected-instance Policy-S bounded-response discharge (A5-S). Trigger: Required whenever Theorem K.1 is invoked; Theorem K.1A and Corollary K.1A-Total instead consume A5-L. A5-S is a property of the execution p_S^{det} selected by SDet for the selected normalized instance Sys_K , including capacities, elaboration, fixed stimulus, witness choices, deterministic simulator/scheduler and arbitration semantics, environment/reset contracts, K.Low lowering, and the K.Resp configuration. It combines three obligations for that exact run: (1) K.CompleteS evidence establishing maximality, fairness, and completeness over the CompleteCycle projection of the selected CycleResult semantics; (2) K.RespCov evidence defining a checkable disposition for every static internal process-facing blocking site and relevant dynamic class and proving that every dynamically reachable instance belongs to a monitored or statically bounded class, together with finite bounds and waiver-preservation, while environment-facing sites are handled by the separate StandardEnv/monitor/exclusion rule; and (3) K.RespClean evidence proving that the complete run contains no non-waived K.RespFail witness. Discharge: Supply CF-S identifying p_S^{det} and carrying all three obligations. A finite prefix of a nonterminating execution is supporting evidence only unless accompanied by a complete invariant, model-checking, recurrence/lasso, or equivalent argument that all monitor ages remain bounded. K.Resp evidence is source-side A5-L-discharge evidence; it does not establish an RTL response-time bound.
- A5-L — scheduler-robust liberal source liveness. Statement: A5-L abbreviates Definition K.A5L on Sys_ref : no finite Policy-L prefix admissible under Definition Core.LAdm for the selected Sys_ref instance reaches a source K-facing cutpoint $\sigma \in \text{KCutReach_ref}(I)$ satisfying $\text{Dead_K}^{\text{AW}}$.

A5-L is consumed by Theorem K.1A for one certified cutpoint, by Corollary K.1A-Total together with Core.KCutClosure_post and K.CertCov for the universal RTL conclusion, and by Theorem K.1 after the selected A5-S route discharges it. Discharge: Primarily by an SDet-qualified CF-S certificate proving the model-indexed lowered property $A5L_Sys_K(I, W_K)$, Corollary K.2c, and Lemma K.Low.A5 transporting that result to Sys_ref; alternatively by K.Str-1/K.Str-2/K.Str-3, direct Policy-L verification, an inductive invariant, exploration, or CF-5 directly on Sys_ref. These alternatives do not weaken A5-L: if any admissible Sys_ref Policy-L prefix—including a Policy-S-shaped prefix—reaches Dead_K^W, A5-L is false and no alternate discharge can certify the unchanged instance. The Policy-S package records the selected normalized instance, capacities, elaboration, lowering/projection, witness, stimulus, environment/reset model, concrete simulator/scheduler, arbitration, Policy-S issue semantics, normalization identity, SDet evidence, K.EnvMF, K.ResetEpoch, K.TerminalDisposition, K.CompleteS, K.RespCov, K.RespClean, response bounds, monitor waivers, W_K pullback, and the trust boundary.

- K.CV — design-level value determinism.

Serial-route provenance trigger (K.OpKey/K.CVPred/K.CV-WF): Required whenever Lemma K.2b.P0 or K.2b.P1 is consumed, including every use of Lemma K.2b, Corollary K.2c, K.PSF, CF-S, or the Policy-S form of Theorem K.1, regardless of whether the design has cross-process observations beyond blocking Pop values.

Additional K.CV observation-check trigger: Required when a process can branch on, compute payloads from, or select later operations based on cross-process observations beyond ordinary blocking Pop transfer values — anchored signal reads, direct-input(-sync) reads, shared/external-memory reads through the array-access mapping layer, emitted R2C fixed-point observation units, or synchronization outcomes.

Discharge: For the serial-route provenance trigger, supply or derive Definition K.OpKey plus the Definition K.CVPred certificate: one theorem-instance OpKey_I universe with partial per-execution realization maps and a checked key-level source order, one common CVFact_I key universe, finite Pred_CV(b) sets, explicit predecessor occurrence closure, deterministic Eval_b equations adequate for every admissible execution in the comparison scope, and for each OpFact(κ) a checked none/some(desc) evaluator that includes control/branch occurrence selection rather than assuming occurrence, plus K.CV-WF: WellFounded($\leq_CV$), or an equivalent checked CVRank whose strict decrease proves it. The $\leq_CV$ relation shall contain actual value/control provenance only; rendezvous mutual enablement, same-cycle coincidence, and the broad Appendix-G causal-dependency graph $\rightarrow$ do not become predecessor edges merely by being causal/temporal neighbors. A design with circular value/control provenance that cannot satisfy K.CV-WF is outside the K.2b/K.2c serial route. For the additional observation-check trigger, also show that every such dependency is carried by an explicitly modeled causal mechanism and that the observed value is a deterministic function of the certified value/identity provenance feeding that mechanism, not of incidental schedule timing. These additional design checks include RULE 1 / RULE 2 anchored signal IO, proper handshakes or direct-input contracts on cross-process signals, mapping-layer discipline on shared memories, and R2C well-formedness; latency-sensitive sites are isolated or witness-aligned per Appendices D and L. Evidence examples: a source/tool-generated value-provenance graph with a checked ranking function; a mechanized well-foundedness proof over typed dynamic operation/observation keys; source inspection or lint establishing RULE 1 / RULE 2 and handshake conformance where the additional observation-check trigger applies; a coverage-justified Appendix L latency-robustness stress-testing result with provenance/rank evidence; a refinement argument; or an Appendix D isolation argument. Randomized stress testing by itself is supporting evidence, not a complete K.CV discharge, unless accompanied by a coverage/refinement argument and the

required K.OpKey/K.CVPred/K.CV-WF evidence. This K.CV evidence is necessary but not sufficient for the separate NB-Align condition (next entry), which is discharged independently.

- **NB-Align** — witness-selected non-blocking alignment.
Statement: NB-Align is the scope restriction stated normatively in the Lemma K.2b Statement (Appendix K), which is its authoritative wording: every witness-selected non-NB-CF non-blocking site that can participate in a potential wait cycle must satisfy the Appendix L alignment conditions that convert its selected outcome dependency into an explicitly modeled message/synchronization obstruction. NB-Align is consumed as the applicability condition of the Definition K.Low witness-indexed blocking lowering: an aligned site is lowered to a blocking guard on an explicit point-to-point supplier channel, and a witness schedule containing a non-aligned such site admits no lowering and is outside the Lemma K.2b / Corollary K.2c / Theorem K.1 scope. Theorem K.1A does not consume NB-Align. Trigger: Required whenever Lemma K.2b or Corollary K.2c is invoked (including through Theorem K.1) and the design contains witness-selected non-blocking outcomes that are not NB-CF. Discharge: Show that each witness-selected non-blocking site is NB-CF, is Appendix L-aligned (so that Definition K.Low applies to it), or is structurally excluded from every potential wait cycle of the selected instance. Evidence examples: An NB-CF classification per Appendix L; a per-site Appendix L alignment argument identifying the modeled supplier used by the Definition K.Low guard channel; or a structural argument that no witness-selected site can participate in a wait cycle of the selected instance. NB-Align is distinct from, and not implied by, K.CV.
- **K.Low** — witness-indexed blocking lowering, cutpoint correspondence, and A5 transport.
Trigger: Required whenever the serial-reduction route (Lemma K.2b / Corollary K.2c / Theorem K.1) is applied to a design containing explicit synchronization waits or NB-Aligned witness-selected non-blocking sites in potential wait cycles.
Discharge: Exhibit the lowered instance $\Lambda_W(\text{Sys_ref})$ of Definition K.Low — epoch-gate representation of synchronization per Definition Core.16 clause (7), and a blocking guard channel per aligned site with its modeled supplier as unique producer — and discharge the Lemma K.Low.C correspondence obligations (i)–(iii) for the lowering rules used. Define the waiver pullback W_K and discharge Lemma K.Low.A5, including Policy-L admissibility preservation and the no-waive-on-lift condition, so that $A5L_Sys_K(I, W_K)$ proved by the Policy-S route transports to the Sys_ref A5-L consumed by Theorem K.1A. Evidence examples: The elaboration record for $\Lambda_W(\text{Sys_ref})$; per-site guard-channel construction tied to the Appendix L alignment; a mechanized or reviewed K.Low.C case proof per lowering rule; and a checked K.Low.A5 projection/waiver-transport proof. Guard channels carry the corrected default $B(g_s) = 1$ skewed timing, or a declared atomic rendezvous variant, under Definition K.Low.
- **K.EnvMF** — Policy-S environment/multi-frontier applicability. Trigger: Required for Lemma K.2b.E, Lemma K.2b, Corollary K.2c, Theorem K.PSF-1, Theorem K.1, and CF-S. Discharge: Establish either (a) StandardEnv/B1-avail with the default always-ready output environment, in which case certified multi-frontier components remain permitted subject to K-O2; or (b) a timed/reactive or otherwise nondefault environment together with a certified single frontier for every process in every candidate Dead_K component at the K.2b.E activation cutpoint, that frontier consisting solely of the selected internal blocking item. A timed/reactive/nondefault instance with any relevant multi-frontier—including an environment-facing co-frontier—is outside the Policy-S serial-reduction route. It may use Theorem K.1A only with a non-serial proof of A5-L, or a different theorem outside this document. This revision deliberately does not admit a substitute future-environment no-rescue certificate for the excluded combination.
- **K.InterruptionGate / K.ResetEpoch / K.TerminalDisposition** — nested comparison-interruption applicability. Trigger: Required whenever Lemma K.2b, Corollary K.2c, K.PSF, CF-S, or the Policy-S

form of Theorem K.1 uses the serial reduction. Discharge the shared schema once, including its K.BF exhaustiveness rule: every gate instance has a prefix-local Continue predicate and an independently mapped DirectFail payload; Continue may depend only on the fixed contract and frozen prefix, DirectFail must bind P , f_P , x_P^S , g_P with $g_P \leq_{\text{src}} f_P$ and an already-triggered pending covered monitor independently of K.2b.R/P/Q Step 1, and a candidate satisfying neither branch is outside the route. For each candidate, bind finite $S_p^{\text{dom}}/S_c^{\text{dom}}$, validate K.DomResetScope conditions D1–D4—including D membership, both-endpoint boundary closure, coverage of every K.2b.R/P-consumed endpoint/boundary identity, and independence from the later K.2b.E continuation—and derive DomEpoch. Instantiate reset first. K.ResetEpoch.Continue proves that no RESET intersecting K.DomResetScope is legal as the next proof-relevant transition/unit while the selected frozen requests remain pending and DomEpoch is fixed, with any executed reset occurrence required to appear in the enclosing CycleResult.resetUnits; K.ResetEpoch.DirectFail binds the matched affecting RESET and exact RW2 clearing, yielding cancelled-pending. Only on K.ResetEpoch.Continue instantiate terminal. K.TerminalDisposition.Continue proves every applicable frozen-prefix final state is non-DeclaredTerminal; K.TerminalDisposition.DirectFail independently binds the declared maximal terminal state while the same mapped monitor remains pending, yielding maximal-pending. For a successful clean CF-S discharge the two nested Continue branches are the normal prospective evidence; concrete DirectFail witnesses are themselves K.RespFail classifications and are not required on every clean run. The audit record shall name the K.InterruptionGate schema version, the K.BF exhaustiveness citation, each K.DomResetScope certificate, D1–D4 validation/provenance, and DomEpoch artifact, the reset-first nesting, each Interrupt predicate, and each invoked DirectFail specialization.

- K.Resp — validated process-facing bounded-response coverage. Trigger: Required for Lemma K.2b, Corollary K.2c, Theorem K.1, K.PSF-1, and CF-S. K.RespCov is default-total over a declared coverage universe of static internal process-facing blocking sites and relevant dynamic classes. Each class has one checkable disposition: (a) a monitor with an unambiguous trigger at the first normalized cycle in which the source operation is the active minimal pending frontier and its persistent boundary request is established, the process-facing commit/return as response, a finite Policy-S-cycle bound, and reset/end-of-test handling; (b) a validated static bounded-response proof; or (c) a proof that the class is unreachable in the selected instance. Completeness requires every dynamically reachable instance to belong to a declared class; once an instance is reached, disposition (c) cannot apply. A production-count argument alone is not an exemption, because the complementary owner may itself be blocked. Under StandardEnv, a source-earlier environment interaction that committed in the liberal witness may be proved incapable of remaining the serial blocker by replay availability and fairness. Under a timed/reactive or otherwise nondefault environment, K.EnvMF first requires the relevant liberal Dead_K components to be single-internal-frontier; environment-facing sites that may block the selected Policy-S execution then require monitoring, a specific availability proof, or exclusion from the serial route. No monitor or no-bypass proof admits a timed/reactive multi-frontier liberal component. K.RespFail is a finite-prefix predicate/witness: certified-bound expiration without response under Definition K.RespFail’s deadline convention, a declared-terminal maximal finite selected run with the frontier still pending, or an affecting RW2 RESET that clears the monitored dynamic instance before response unless explicitly waived. A terminating abort/end-of-test disposition is represented by DeclaredTerminal and is covered by the maximal-pending case. K.RespClean proves absence of non-waived K.RespFail witnesses over the complete p_S^{det} . A successful complete-run proof may use any finite bound; a bound

violation under an unproved design-latency bound makes that certificate attempt inconclusive rather than proving a design bug.

- K.EnvOrd / K.Done — optional diagnostic starvation classification. These conditions are no longer part of the soundness-critical Policy-S discharge and cannot substitute for K.RespCov/K.RespClean. They may be used to classify a reported K.RespFail through the historical WFG+/EnvStop/DoneStop owner-walk machinery or to simplify diagnostic reporting from Dead_KS to Dead_K. K.EnvOrd builds the finite may-follow starvation cone and proves the corresponding dynamic no-bypass property. K.Done audits normal completion sufficiently to eliminate DoneStop terminals; the alternative that every process is perpetual eliminates normal completion only and says nothing about live endpoint under-supply. In particular, neither K.Done nor process perpetuity permits omission of a K.Resp monitor. The K.CE-2/K.CE-3/K.CE-4 and H2 examples remain useful diagnostics, but no Corollary K.2c conclusion depends on completeness of this terminal taxonomy.
- W.Env — environment-rescue and response-monitor waiver. Trigger: Optional; used only for a specifically declared cutpoint or dynamic monitor class whose environment/reset/end-of-test behavior is intentional and proved compatible with the theorem scope. For KObs predicates, record the component class, rescuing item, environment contract, and KObs-closure justification. For K.Resp, record the exact dynamic frontier class, the reason timeout/reset interruption is permitted, and a coverage-preservation proof that a non-waived Policy-L deadlock component cannot map only to waived serial monitors. A broad end-of-test or reset waiver is insufficient. Waivers are removed explicitly from Dead_K^W and optional diagnostic Dead_KS^W, and from K.RespFail^W, and propagate to Theorems K.1A/K.1. A waiver depending on hidden issue timing or unconstrained future behavior must carry the corresponding operational proof; a label alone does not discharge the obligation.
- L.Dir — one-directional alignment / RTL-side non-interference.
Statement: L.Dir is the scope restriction stated normatively in Condition L.Dir (Appendix L), which is its authoritative wording: the source-side snoop sequence selected by W is exactly an ordinal-preserving consumption of the free-running RTL_B snoop sequence at the chosen boundary — each pre-HLS-side snoop commit consumes the next recorded RTL_B event of the same snoop point with matching kind and payload, no pre-HLS-side snoop commit occurs without such a next recorded RTL_B event, every recorded RTL_B snoop event within the compared run is eventually consumed or reported outstanding per the Requirement W4 bound, and no RTL_B-side signal or event is gated, delayed, or modified. Benign cycle skew is permitted; only order, kind, or payload divergence — non-existence of the WC witness at the chosen boundary — violates the condition. A run violating L.Dir is outside the Appendix I–K theorem scope for that comparison and carries no theorem-backed guarantee.
Trigger: Required whenever WC is discharged through an Appendix L snooping / witness-selector co-simulation harness (Lemma L.2 / Lemma L.5).
Discharge: Provide a per-run monitor per Lemma L.5(iv) — realized by a wrapper conforming to the Appendix L Wrapper Realization Requirements W1–W5 — that aborts loudly on violation, together with the run’s clean-monitor result and a statement of whether the Requirement W4 watchdog bound is proved sufficient for the selected instance (bound-exceeded aborts under an unproved bound are inconclusive witness-realization failures, not proven violations); or a static/structural argument — for example, latency back-annotation or delay padding of the pre-HLS model, or latency-interval analysis at each snoop point in the spirit of the simplified snooping approaches — showing that the post-HLS side cannot run ahead in event order; or a model-checking proof that order divergence is unreachable for the selected instance.
Evidence examples: A harness monitor log showing no L.Dir abort, with cycle-skew statistics

reported separately as diagnostics; a static latency analysis at the simplified-snooping end of the Appendix L spectrum; or a wrapper certification against Lemma L.2 together with a watchdog configuration that converts witness non-existence into a reported failure rather than a hang. L.Dir violations are triaged per the Snooping Concerns section of Appendix L: the pre-HLS model is the default suspect in a certified-harness, trusted-tool flow, but harness incompleteness and tool non-compliance must not be excluded a priori.

- J.LocalGWR — local certificates for the compositional J.GWR construction.

Trigger: Required whenever Theorem J.1's normal Post $\rightarrow$ Ref route, Corollary J.1, or any result consuming their witnesses is applied, unless the optional J.GWR-Direct route below is used.

Discharge: For each compared RTL_B prefix, provide the package of Definition J.LocalGWR. The package shall cover every process with J.PCert; every explicit abstract channel with J.CCert and B-faithfulness; every rendezvous, paired synchronization, R2C, anchored synchronization/signal, RESET, and other explicitly atomic unit with J.ACert; and the applicable J.EnvCert and J.ResetCert obligations. It shall also provide the finite typed construction-action inventory, per-action ReadSet/WriteSet, ownership facts, and locally derived epoch/horizon/stage/ordinal metadata for J.FoundationRank, request-snapshot keys for every witness-significant non-blocking call, the locally generated semantic D1–D4 edges and D5 construction-stratification edges required by Definition J.DepJ, including J.HorizonOrderSafe commutation evidence for every D5-only pair, and J.LocalAgree for every duplicated endpoint, identity, payload, reset-epoch, request-attribution, boundary, footprint, and snapshot field. The package shall prove complete coverage and the premises needed for the case-by-case J.DepRank proof: every relevant process, explicit boundary, construction action, non-blocking decision, and atomic unit in the selected prefix is represented exactly once or by a uniquely identified shared record. J.CanonicalRank is reconstructed from the checked dependency graph and is not a separately supplied certificate field.

The discharge may not contain a renamed global realizability premise. In particular, it may not assume that the local certificates admit a realizable merge, that an arbitrary phase/rank already gives a legal replay order, that the next complete unit has a global extension, or that a matching Sys_ref execution already exists. Dependency edges must arise only from the enumerated local rules of Definition J.DepJ, and non-commuting action pairs must be covered by Lemma J.DepComplete or by a component-local footprint/commutation certificate. Global well-foundedness, coexistence, and extension are conclusions of Lemma J.DepRank, Lemma J.DepAcyclic, J.PrepareCommute, J.UnitExt, and Theorem J.GWR-Comp.

Evidence examples: per-process schedule/replay scripts with typed action footprints; per-channel boundary, occupancy, same-cycle transfer, and request-snapshot certificates; static or generated atomic-unit identity records; locally justified Dep_J edge records; source/RTL structural checks; component-local footprint/commutation certificates for custom Sys_B^elab elements; a translation-validation package whose individual records map to J.PCert/J.CCert/J.ACert; or equivalent independently checkable evidence.

J.GWR-Direct — optional direct global witness-realizability route.

Trigger: Optional alternate route when a flow does not use J.LocalGWR/J.GWR-Comp for a selected prefix.

Discharge: Supply a directly validated package satisfying Definition J.GWR, including the complete reachable Sys_ref witness chain, all operational annotations, and a direct-provenance J.ValidatedConstructionOrder with an independently checked well-founded predecessor relation, complete unit enumeration, required-order refinement, operational-prerequisite coverage, and ideal-prefix proofs. This route may be implemented by translation validation, bounded symbolic replay, a validated global trace certificate, or a previously proved tool-specific

theorem. It is not required when J.LocalGWR and Theorem J.GWR-Comp are used, and it may not cite the theorem-derived J.CanonicalRank without also supplying the corresponding LocalGWR package.

Historical Issue timestamps may remain inside either route’s operational witness evidence, but they are not fields of KObs and are not independently compared across cutpoints.

- J.Mem — shared-memory lowering.

Trigger: Required whenever the compared design contains a cross-process shared-memory dependency that can affect control flow, payloads, or operation identity/order.

Discharge: Show every such dependency is lowered by the array-access mapping layer onto covered IO operations (explicit point-to-point E4 channels and/or Σ -visible anchored signal IO), or supply the memory-state invariant extension described in the informative note in Normative Formal Core.

Evidence examples: use of pre-verified shared-memory library classes presenting channel/signal boundaries; mapping-layer configuration review; structural check that no un-lowered cross-process array access exists.

- RW — reset well-formedness (RW1–RW5).

Trigger: Required whenever dynamic reset can occur in the compared executions.

Discharge: Show every dynamic reset occurrence satisfies RW1–RW5 so that Lemma J.RS (MatchedResetEpochSplice) applies at each RESET boundary unit.

Evidence examples: reset-tree/scope review; RTL check that both endpoints of each live channel reset together or that an explicit modeled flush/abort protocol is used; confirmation that outstanding requests clear and affected channels re-initialize reset-empty.

This record is not an additional semantic assumption beyond the named premises already used in Appendices I–K. It makes the required single-global-clock scope check and the discharge of Core.SyncFence, Core.ReachPrep, Core.CycleClosure, Core.IssueFair, B1, B2/B3, B4/FPD, B5/B6-Terminal, A6, A8/A9/A10, WC, J.LocalGWR (or J.GWR-Direct), elaboration, B-faithfulness, J.OfferReach, J.OfferConf, J.KObsConf, J.Mem, RW, K.Drain, K ϵ , K.BF, SDet, A5-L, and the selected A5-L discharge route—including K.CompleteS, K.RespCov, K.RespClean, and the shared K.InterruptionGate schema with K.ResetEpoch/nested K.TerminalDisposition when the deterministic Policy-S route is used—and K.CV explicit for the selected theorem scope.

M.8 Practical interpretation

The architecture provides two different forms of scalability. Safety scales by keeping most design-specific evidence local to processes, explicit channel boundaries, and named atomic interactions, while using one generic theorem to construct the global replay witness. Liveness scales by separating the system-wide source premise A5-L from the available methods for proving it and by making the remaining progress, environment, drain, and cutpoint-coverage obligations explicit.

For a qualified HLS flow, the intended engineering result is that the pre-HLS SystemC model remains the primary place for system-level functional reasoning, coverage, and debug, while RTL work is concentrated on showing that the implementation conforms to the scheduling contract and its selected abstraction. The approach does not eliminate implementation verification; it changes its purpose from repeating the full functional test campaign at RTL to establishing the evidence needed to justify transfer from the source-reference verification result.

The same proof architecture is not inherently limited to HLS-generated RTL. Hand-written RTL, modified generated RTL, or RTL produced by another generation flow can use the same results when the same process, channel, observation-boundary, reset, environment, progress, and implementation-side

obligations are established. The user-visible and tool-neutral form of the contract is also what makes the document suitable as a starting point for standardization discussions.

Appendix N – Scheduler and Environment Fairness Assumptions

The formal results in Appendices I–K rely on several semantic assumptions about schedulers and environments. The B-rule table in “Normative Formal Core for Appendices G–K” gives the proof-level names B1–B6. This appendix restates the boundary-action fairness/progress rules in operational ready/valid form and separately restates Core.IssueFair for source request assertion. In particular, Appendix N’s “B3 Channel Progress Invariants” are exactly the $P_push(c)$ and $P_pop(c)$ obligations for each explicit abstract channel c ; they are not additional assumptions beyond B3.

- **B2 (Weak fairness of the scheduler).** If the enabling predicate for an action (Push, Pop, Sync) remains continuously true from some cycle onward, the scheduler must eventually select that action within a finite, but unspecified, number of cycles.

Core.IssueFair (source issue; separate from B2). If a reached source operation remains legally IssueEnabled at every applicable L1 opportunity from some cycle onward within the same reset epoch, it is eventually issued. This is weak fairness over request assertion, not over commit. Core.16 is part of issue legality, so a source-later operation prohibited by an active blocked-frontier discipline is not IssueEnabled and imposes no IssueFair obligation. B2 begins after an applicable Push/Pop/Sync boundary action is ChannelEnabled. Core.LAdm/K.2b consume Core.IssueFair as a premise of complete admissible continuations; finite-prefix Appendix J does not require a generic issue-fair extension theorem.

- **B3 Channel Progress Invariants (ready/valid form).**
For every channel c with capacity $B(c)$, interpret “continuously enabled” in terms of continuous endpoint request, not as an immediate commit.
(**No deassert-before-commit assumption.**) For blocking $Push_c/Pop_c$ transfers, these requests are non-withdrawable during the outstanding transfer attempt within one reset epoch: once asserted for a transfer attempt, they are not deasserted before the corresponding commit, and $Push_c$ payload remains stable while vld_c is asserted for that outstanding transfer. If an affecting RESET occurs first, RW2 ends that dynamic instance at the epoch boundary. No other nonterminal transition may withdraw or reattribute it. After the corresponding commit, the endpoint may deassert or may present a different request/payload for a later transfer.

Let vld_c denote the producer’s persistent request to transfer (e.g., $valid=1$ with a stable payload for a $Push_c$), and let rdy_c denote the consumer’s persistent request to transfer (e.g., $ready=1$ for a Pop_c). In the definitions below, “remains asserted continuously” is scoped to the outstanding interval from t_0 until the first matching commit, or until an affecting RESET ends the dynamic instance under RW2. If neither occurs and the execution remains nonterminal in that reset epoch, then the request must remain asserted for all later cycles.

- **$P_push(c)$:** If vld_c is asserted at cycle t_0 while the channel is full ($occ_c = B(c)$), and both endpoint requests continue until the matching discharge event — i.e., vld_c remains asserted with stable payload and the complementary Pop_c endpoint request remains asserted with $BoundaryReq(Pop_c, t)$ true until the first Pop_c commit on c — then some Pop_c commit must eventually occur, discharging the full condition. Equivalently: when both endpoints continuously request the matching transfer until discharge, the system cannot remain stuck forever in the full state without a Pop_c commit.
- **$P_pop(c)$:** If rdy_c is asserted at cycle t_0 while the channel is empty ($occ_c = 0$), and both

endpoint requests continue until the matching discharge event — i.e., rdy_c remains asserted and the complementary $Push_c$ endpoint request remains asserted with stable payload and $BoundaryReq(Push_c, t)$ true until the first $Push_c$ commit on c — then some $Push_c$ commit must eventually occur, discharging the empty condition. Equivalently: when both endpoints continuously request the matching transfer until discharge, the system cannot remain stuck forever in the empty state without a $Push_c$ commit.

For $B(c)=0$ rendezvous channels: if both endpoint requests are asserted at cycle t_0 and both remain asserted until the matching rendezvous discharge event, then the rendezvous completion eventually occurs. If no rendezvous completion occurs, both requests must remain asserted for all later cycles.

These obligations rule out “permanent back-pressure loops” that are logically independent of the local R-rules. They do not require an endpoint request or $Push$ payload to remain asserted forever after the matching transfer has already committed.

Clarification (scope of the B3 conclusion).

For $B(c)>0$, $P_push(c)$ concludes that a complementary Pop_c eventually commits and $P_pop(c)$ concludes that a complementary $Push_c$ eventually commits. Neither buffered clause directly states that the original blocked endpoint operation commits. Once the complementary discharge changes occupancy, E4 determines whether the original persistent request is $ChannelEnabled$, and B2 supplies eventual selection/commit if that enablement remains continuous. For $B(c)=0$, the matching discharge is the original same-cycle rendezvous completion.

- **B5 (System quiescence closure).** If at some cycle no observable actions are enabled in any process, and no external or environmental change during the closure interval changes the relevant enabling predicates, then within a bounded number of cycles the system reaches an ϵ -fixed point. Thereafter, while no external or environmental change re-enables observable or internal work, the system executes no additional ϵ -steps. This prevents a dead, all-disabled state from being hidden behind an infinite tail of internal activity, without requiring the system to remain quiescent after a later reset, input change, externally supplied message, or other environmental event re-enables work.

B6 (terminal idle only). B6 may provide an indefinite idle-clock suffix only when $Core.TerminalIdle$ holds: the system is at a genuine B5/global fixed point, future registered/internal machine evolution cannot re-enable work, and $Core.EnvTerminal$ establishes the same no-future-enabling fact for the environment. A time-aware environment can be perfectly total while delivering an input at a later cycle, so $Core.CycleClosure$'s environment-definedness clause does not imply $Core.EnvTerminal$; likewise environment terminality alone does not prove that an internal timer/FSM cannot advance. Temporary cycles with no committed Σ action are ordinary $Core.SilentCycle$ transitions, not B6 terminal stutter. These B-rules are **assumptions on the execution environment**, not guarantees automatically provided by the HLS tool. In practice they place concrete requirements on arbiters, back-pressure, and external masters that interact with the post-HLS RTL.

Priority arbiter example: fair vs unfair priority

Consider a simple two-input priority arbiter inside the post-HLS RTL. Each input is driven by a message-passing channel; the arbiter issues Pop operations to select which request to serve in each cycle.

- Input 0: high-priority channel c_high
- Input 1: low-priority channel c_low

Both channels may have pending requests at the same time.

1. Fair priority arbiter (satisfies B2 / B3).

A fair implementation might still give *c_high* strict priority in *individual* decisions, but it ensures that a continuously pending *c_low* request cannot starve. In RTL terms, this can be realized by, for example:

- A **round-robin or rotating-priority** arbiter that periodically moves the “highest” priority to the next requester, or
- A **bounded-burst priority** scheme in which *c_high* may win at most *N* consecutive cycles while *c_low* is requesting; after that, the next grant is forced to *c_low*.

Under these implementations:

- If *Pop_low*’s *ChannelEnabled* predicate remains true indefinitely (the low-priority channel has a pending request and the relevant abstract boundary availability predicate holds), the scheduler/arbiter must eventually select *Pop_low*. The selected *Pop_low* then commits in the selected cycle. This satisfies B2 for that action.
- If the low-priority FIFO is full and keeps requesting service, the system ensures that some complementary *Pop_low* eventually commits, discharging data and freeing space. This is consistent with *P_push/P_pop* in B3; it is a channel-progress obligation that may be needed before a blocked producer’s *Push* becomes *ChannelEnabled*.

Intuitively, the arbiter is still “priority-based,” but its micro-architecture ensures that every continuously enabled requester is eventually selected, and that full/empty channel states covered by B3 cannot persist forever under the stated persistent-request premises.

2. Unfair priority arbiter (violates B2 / B3).

A more naive design might implement:

```
if (req_high) grant_high();  
else if (req_low) grant_low();
```

combined with an environment in which *req_high* can remain asserted indefinitely. In this case, even if *req_low* is also continuously asserted:

- *Pop_low*’s *ChannelEnabled* predicate may be true forever, but *Pop_low* is never selected by the scheduler/arbiter.
- Low-priority requests can starve indefinitely, even though they are continuously eligible for selection.

This behavior violates B2, because B2 requires eventual selection of any continuously enabled action. If *c_low*’s FIFO is full and the only way to make space is to service it, permanent starvation can also violate B3’s progress expectations for that channel, because the full condition may never be discharged under the stated persistent-request premises.

In such a design, the safety properties E1–E5 may still hold (trace-compatibility on actions that do occur), but the liveness/results that depend on B2/B3—especially the starvation-vs-deadlock arguments in Appendix K—no longer apply.

Patterns that typically satisfy the fairness assumptions

The following design patterns normally satisfy B2 and are compatible with B3/B5, provided the rest of the system is well-behaved:

- **Round-robin or rotating-priority arbiters.** Any requester that remains enabled will eventually be at the front of the rotation and will be granted.
- **Age-based or credit-based arbiters.** Requesters accumulate age or credits while waiting; the arbiter prefers older or more-starved requests, guaranteeing that long-pending actions eventually win.

- **Bounded-burst fixed priority.** Fixed priority is combined with a counter that limits how long a higher-priority requester can dominate while lower-priority requesters are pending. After the burst limit is reached, lower-priority requests are forced to win until the system is “caught up.”
- **Back-pressure with bounded stall.** When ready/valid or similar handshake signals are used, the design (or its environment) must enforce that ready cannot remain low forever while valid remains high, and vice versa. For example:
 - Downstream consumers are required (by specification) to service queues at least once every N cycles when data is present.
 - External bus masters are configured such that they cannot indefinitely defer reading from full DUT FIFOs that are logically part of the interface contract.
- **Clock-gating and power-management schemes that respect B5.** Once no observable actions are enabled, the design either:
 - Quietly stabilizes (no further internal toggling), or
 - Explicitly enters a low-power state from which it only wakes when some observable action again becomes enabled.

In each case, the intent is the same: continuous ChannelEnabled status implies eventual scheduler/arbitrator selection and commit under B2; persistent full/empty/rendezvous-blocked request premises imply eventual discharge under B3; and once nothing is enabled, the system converges rather than oscillating internally.

Patterns that tend to violate the fairness assumptions

Conversely, the following patterns are likely to **break B2/B3/B5** and therefore fall outside the scope of the liveness arguments in this document:

- **Pure fixed-priority arbitration with unbounded high-priority traffic.** A static priority tree with no rotation or aging, combined with workloads in which a high-priority master can keep its request asserted indefinitely, can starve lower-priority channels forever.
- **“Best-effort” external masters with no progress guarantee.** For example:
 - A software driver that reads from a DUT output FIFO only when it happens to poll, and that may be pre-empted indefinitely by higher-priority threads.
 - An external bus or DMA engine that is architecturally allowed to ignore some requesters forever if higher-priority traffic remains heavy.
Such environments can violate **P_push/P_pop** by allowing FIFOs to remain full or empty indefinitely while the DUT is continuously requesting progress.
- **Circular back-pressure loops with no escape.** Two or more processes form a cycle in which each is waiting for the other’s channel to change state, and no other process can break the cycle. Without an explicit design-level guarantee that some process in the loop will eventually act (for example, by dropping priority or issuing a compensating Pop/Push), **B3** is not satisfied.
- **Internal oscillation without quiescence.** A scheduler or control FSM that can toggle internal ϵ -level state forever, even after all observable actions are disabled, violates **B5**. While such designs are uncommon in practice, patterns involving mis-configured clock gating, asynchronous feedback, or “keep-alive” timers that never expire can have this effect.

In all these cases, the trace-compatibility theorems **still describe what happens when actions do occur**, but the liveness claims that depend on B2/B3/B5 (for example, “a continuously enabled channel will not starve”) are no longer guaranteed. Designers and verification engineers should therefore either:

- Architect their arbiters and environments to satisfy these B-rules, or
- Treat the liveness guarantees in Appendix K as *out of scope* for those particular interfaces, and rely only on the safety-oriented parts of the model.

Appendix O – Support for Multiple Clock Domains (Informative Sketch)

This appendix is an informative sketch of how the single-clock formalism of Appendices G–K could be organized in a multi-clock design. The formal results of Appendices G–K are stated for a single global clock unless an explicit multi-clock semantic layer is added.

Within a single clock domain d , one may treat that domain as a synchronous island with a local cycle counter clk_d and apply the existing R- and E-rules to processes wholly inside that domain, interpreting $\text{clk}(\cdot)$ as $\text{clk}_d(\cdot)$ for local synchronization, signal I/O, and message-passing events. Inter-domain communication should be represented by explicit clock-domain-crossing components, such as CDC FIFOs, whose source-side and destination-side commit semantics are specified at their respective clock-domain boundaries.

A full formal lift requires additional definitions not supplied by Appendices G–K themselves: a global event order or partial order relating local-domain cycles, precise CDC commit semantics, occupancy sampling rules across source and destination clocks, synchronizer fairness/metastability abstraction assumptions, and progress obligations for CDC components. Once those additional definitions are supplied, the intended proof strategy is to treat each synchronous island using the existing single-domain rules and to treat each CDC component as an explicit channel/refinement boundary with its own stated E4-style legality and progress obligations. Without those additional definitions, Appendix O should not be read as claiming that the single-clock proofs apply unchanged to arbitrary multi-clock systems.

Appendix P – AI Discussion of Alternative Formal Frameworks

The following links provide an AI discussion of alternative formal frameworks compared to the one presented in this document:

https://drive.google.com/file/d/1ccU-eRW7H2ggh-AZyKnCOzO-KLPejob_/view?usp=sharing

<https://chatgpt.com/share/69457119-ec30-8006-8684-9a2a8b19f96f>

<https://drive.google.com/file/d/1R3-46DtZLsBn1hdTbLEXyC0FVB50Gymz/view?usp=sharing>

Appendix Q – Multiple Input Pipelines Back-Annotation

Appendix J Remark 2 discusses back-annotation of pipelined loops with a single message passing input. In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the Sys_B (source-level) simulation by introducing additional internal realization structure. In this document, we adopt the following simplified modeling discipline:

1. Terminology (elaboration and channel accounting). In an elaborated realization, the quantities $B(\cdot)$, $occ_{\cdot}(t)$, and the E4 availability predicate are interpreted over the explicit abstract channels present in the elaborated model (including any introduced staged/bundle channels). When we continue to write a source-level channel name c at the outer DUT/testbench observation boundary, we mean the chosen outer abstract boundary channel associated with c in that elaboration; downstream staging that is reachable only through a join (and therefore may be withheld when “join not met”) is represented by distinct internal channels with their own $B(\cdot)$ and $occ_{\cdot}(t)$, rather than as independent “slack” for c in isolation.
2. All buffering / capacity effects are represented explicitly as finite-capacity message channels (annotated with bounded capacity) that may appear upstream and/or downstream of any internal glue logic; and
3. Any internal “coupler” used to enforce mutual-stall between multiple inputs is purely combinational (stateless) handshake logic.

Observation-boundary convention for Appendix Q. In elaborated cases, distinguish the outer projected observation boundary from the proof-internal abstract boundaries. The outer Σ_{DUT} / testbench projection records only the chosen outer $Push_c$ / Pop_c history after applying Π_{elab} . By contrast, Appendix K WFG reasoning is carried out over the explicit abstract channels of $Sys_ref = Sys_B^{elab}$, including introduced staged/bundle/join channels. Thus an introduced internal channel may be hidden by Π_{elab} from the outer DUT-visible trace while still being an explicit proof-internal boundary with its own $B(\cdot)$, $occ_{\cdot}(t)$, $BoundaryReq$, $ChannelEnabled/ChannelDisabled$, $Front_P$, and WFG facts.

Concretely, a combinational coupler is a stateless component that (a) reads the upstream channels’ valid and data signals and computes/drives the downstream channels’ valid and data signals, and (b) reads downstream ready signals and upstream valid signals and then computes/drives upstream ready signals. The coupler contains no internal state and has no separate credit/capacity state; any shared pipeline capacity, per-input staging, skid/elastic buffering, or credit-like effects are modeled only by the explicit finite-capacity channels inserted by the back-annotation realization.

Example (two inputs, staged Pops, $ll=1$, latency=4):

Consider a 4-stage RTL pipeline that (i) performs Pop_{c1} in stage 1, (ii) performs Pop_{c2} in stage 2, and (iii) performs $Push_{cout}$ in stage 4, with $ll=1$. To model the pipeline’s internal staging and the stage-2 input coupling in the source-level Sys_B^{elab} simulation while keeping the same outer projected Σ boundary, the back-annotation elaboration may introduce the following internal realization structure:

- An explicit bounded **channel F1 of capacity 1** on the $c1$ path to represent stage-1 in-flight storage of values that have been accepted from $c1$ but have not yet reached the stage-2 join point.
- An explicit internal **rendezvous channel F2** on the $c2$ path at the stage-2 join boundary, with $B(F2)=0$. $F2$ represents the $c2$ -side handoff into the stage-2 join. It is not additional $c2$ -side buffering or slack, and it does not permit a $c2$ value to be accepted independently before the join is ready. Its purpose is to make the $c2$ -side join blocking point an explicit proof-internal abstract boundary of Sys_B^{elab} , so that any mutual-stall condition involving $c1/F1$ and $c2/F2$ is represented as ordinary $BoundaryReq$ / $ChannelEnabled$ / $ChannelDisabled$ facts on explicit internal channels rather than as a hidden cross-channel disablement predicate on the outer projected $c2$ boundary.
- A **purely combinational coupler/join** at the stage-2 boundary that enforces mutual stall: it allows the joined transfer to proceed only when (a) a token is available from $F1$, (b) the $c2$ -side rendezvous over $F2$ is requested/enabled, and (c) downstream staging has space; it then

presents the paired value(s) to the downstream staging. The join's possible refusal to proceed is therefore visible to Appendix K through the explicit internal boundaries F1, F2, and F34, even though those internal events are projected away from the outer DUT/testbench-visible Σ_{DUT} trace by Π_{elab} .

- An explicit bounded **channel F34 of capacity 2** after the coupler to represent the in-flight capacity of stages 3–4 for the joined/paired work.

This example illustrates why multi-input pipelines cannot always be captured by a single per-channel capacity number in isolation: c1 and c2 do not have independent “slack” because acceptance across the stage-2 join boundary is coupled. In this particular example, the c1 side has one stored stage-1 token available through F1, the c2 side has no independent pre-join storage and is therefore modeled by the rendezvous channel F2 with $B(F2)=0$, and the joined/paired work has two downstream storage slots through F34.

All storage-bearing buffering/capacity resides in the explicit bounded channels F1 and F34, and in any additional positive-capacity stages the realization uses. The explicit rendezvous channel F2 has $B(F2)=0$ and contributes no storage; it is an explicit proof-internal join boundary used to type the mutual-stall condition. The coupler itself remains stateless combinational handshake logic.

This elaboration is an implementation/refinement of the abstract bounded-FIFO channels at the chosen outer DUT/testbench observation boundary; it does not change the outer interface Σ_{DUT} or E4's per-channel FIFO meaning at that outer boundary. It may, however, introduce additional proof-internal Σ boundaries in $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$ for Appendix K blocking/WFG reasoning. Concretely, any back-annotation elaboration (including extra FIFO stages and/or combinational couplers) must satisfy:

- No new outer DUT-boundary observables: It introduces no new Σ_{DUT} -observable actions at the chosen DUT/testbench boundary, and it does not change the labeling of existing outer Σ_{DUT} events ($\text{Push_c}(v)$, $\text{Pop_c}(v)$, etc.).
- Boundary/E4 preservation at the outer projection: At the chosen outer DUT/testbench boundary, each outer Σ_{DUT} -visible committed $\text{Push_c}/\text{Pop_c}$ event still denotes a committed transfer on the same abstract channel c , with per-channel FIFO order and the same E4 capacity/enablement interpretation under the annotated $B(c)$ and $\text{occ_c}(t)$. Any additional internal channels introduced by the elaboration (e.g., staged/bundle/join channels) are separate proof-internal abstract channels in $\text{Sys_B}^{\text{elab}}$ with their own $B(\cdot)$, $\text{occ_c}(t)$, and E4 availability predicates; if they participate in Appendix K, their committed transfers are proof-internal Σ -events even when Π_{elab} hides them from Σ_{DUT} .
- No double-counting of internal staging: internal transfers within the concrete realization of a single explicit abstract channel that contributes to that channel's effective capacity (extra FIFO stages, skid/elastic buffering, FIFO→pipeline-stage handoffs, and value propagation through purely combinational couplers that does not cross an explicit abstract channel boundary) are ϵ -steps and do not create additional committed $\text{Push_c}/\text{Pop_c}$ events at that abstract boundary. By contrast, a transfer across an explicit staged/bundle/join channel introduced in $\text{Sys_B}^{\text{elab}}$ is a committed Push/Pop event at that proof-internal abstract boundary. Such an event may be projected away from the outer DUT/testbench-visible Σ_{DUT} trace by Π_{elab} , but it is not ϵ for the Appendix K proof-internal WFG boundary.
- Conservation / projection of internal staging (normative): For each outer Σ_{DUT} -visible abstract channel c at the chosen DUT/testbench boundary, the elaborated realization shall admit a fixed projection/accounting map from the occupancies of the explicit internal channels introduced by the elaboration (including staged/bundle/join channels, as applicable) to the outer boundary occupancy $\text{occ_c}(t)$. This projection/accounting map is part of the realization and witnesses that the proof-internal occupancy bookkeeping is only a refinement of the outer boundary model: (i) every ϵ -step internal

transfer and every committed transfer across an introduced proof-internal abstract channel preserves the projected outer boundary occupancy for each outer Σ _DUT-visible channel, except when it corresponds under Π_{elab} to an outer boundary commit, and (ii) each outer Σ _DUT-visible committed Push_c/Pop_c updates the projected outer boundary occupancy exactly as required by E4 (equivalently, exactly as the outer boundary occ_c(t) defined from Σ _DUT-visible boundary commits). For bundled/joined work items, the projection may assign one proof-internal token to multiple outer Σ _DUT-visible channels as specified by the realization. Therefore, the internal capacities/occupancies $B(\cdot)$, $\text{occ}_\cdot(t)$ do not introduce an additional independent outer Σ _DUT-visible state; they refine the same bounded-FIFO accounting already represented at the chosen outer boundary, while still serving as proof-internal Σ state for Appendix K when those internal channels are explicit abstract boundaries.

- **Multi-input pipelines / combinational couplers (mutual stall + explicit capacity):**

To preserve per-channel E4 at the Σ boundary, any downstream staging that is reachable only through a join (and therefore may be withheld when “join not met”) shall be modeled as distinct internal finite-capacity channels with their own $B(\cdot)$ and $\text{occ}_\cdot(t)$, rather than being treated as additional independent “slack” for an upstream Σ -visible channel in isolation; the back-annotation elaboration may use combinational couplers to enforce the intended mutual-stall behavior when some required inputs are unavailable. Any “shared capacity” or “distributed staging” effects needed for such a pipeline shall be represented only by explicit finite-capacity channels in the realization (e.g., bounded staging FIFOs, bounded slot/credit channels, bounded bundle channels), not by coupler state. The coupler itself remains stateless combinational handshake logic and does not redefine the per-channel E4 capacity/availability predicates at the chosen observable boundary.

Projection/accounting invariant for $\text{Sys_B}^{\text{elab}}$ (normative implementation of Definition Core.ElabProj). Any elaborated reference model $\text{Sys_B}^{\text{elab}}$ used for multi-input / coupler cases shall come with a projection map Π_{elab} from the explicit staged/bundle/join channels of the elaboration back to the chosen outer Σ -visible channel boundaries. This projection map has these roles:

- **Observable-boundary preservation.** When the internal staged/bundle/join events introduced by elaboration are hidden or projected away, the remaining outer-boundary Push_c(v) / Pop_c(v) history is the same boundary history that the non-elaborated model would expose at the chosen Σ -visible endpoints. Elaboration may add explicit internal abstract channels for proof and WFG purposes, but it may not create extra outer-boundary Push_c / Pop_c events, drop outer-boundary events, duplicate them, or change their payload labels.
- **Capacity/occupancy accounting.** For each outer channel c represented by an elaborated subnetwork, the outer-boundary availability facts used by E4 are preserved by the aggregate accounting of the explicit internal channels in the preimage/fiber $\Pi_{\text{elab}}^{-1}(c) = \{ d : \Pi_{\text{elab}}(d) = c \}$. Internal staged/bundle/join occupancies may refine where storage and blocking occur, but after projection they must implement the same externally visible capacity/acceptance behavior at the chosen boundary. Conversely, any cross-channel dependency or join condition that can block progress must appear as ordinary BoundaryReq / ChannelEnabled / ChannelDisabled facts on some explicit internal abstract channel of $\text{Sys_B}^{\text{elab}}$, not as a hidden extra predicate on an otherwise E4-enabled outer-boundary operation.

All examples and constraints in Appendix Q are to be read relative to this projection/accounting invariant.

Explicit join boundary requirement (normative): The mutual-stall point shall be an explicit

staged/bundle/join boundary inside the elaborated realization (i.e., the endpoint of an explicit finite-capacity message channel). “Join not met” shall therefore be expressed as ordinary ChannelDisabled at that explicit internal boundary (because that boundary’s own E4 availability predicate fails at the settled BoundaryEvalPoint). The realization must not force a Σ -visible operation on an abstract channel c at the chosen boundary to be ChannelDisabled solely due to the state of some other channel; any cross-channel dependency shall be expressed only via the explicit staged/bundle channels introduced by the elaboration. Operationally, the coupler is invisible at the outer DUT/testbench observation boundary: it only propagates ready/valid between the explicit staged/bundle channels inside the elaborated realization, and it does not change the projected outer $\text{occ}_c(t)$, which is defined solely by projected outer-boundary commits under Π_{elab} . This invisibility is only with respect to the outer projection; it does not hide any explicit staged/bundle/join channel from Appendix K’s proof-internal WFG boundary.

- **WFG compatibility (Appendix K):** The realization must not introduce a new blocking point that is invisible to the proof-internal abstract boundary of $\text{Sys_B}^{\text{elab}}$. In particular, if the consuming process cannot advance the pending blocking frontier $\text{Front_P}(\sigma)$ (Appendix K.1), then at least one blocking frontier operation (Push/Pop) at either the chosen outer boundary or an introduced explicit staged/bundle/join boundary is asserted with a true boundary request and is channel-disabled by the ChannelEnabled/ChannelDisabled test; the process is therefore “blocked on message passing” exactly as defined in K.1. Because couplers are purely combinational, they have no internal state transitions; any changes in readiness/validity induced by the coupler are purely a function of (i) downstream readiness and (ii) the same channel occupancies / Σ -visible committed transfers already accounted for by E4 and Appendix K, with “downstream readiness” interpreted in the ε -quiescent (ε -normalized) state used for WFG evaluation.

Pure internal data movements that do not change any proof-internal BoundaryReq, ChannelEnabled/ChannelDisabled, Pending_P, or Front_P fact are ε -steps and are WFG-inert under the same pending-blocking-frontier / $\text{Front_P}(\sigma)$ discipline used elsewhere in Appendix K. By contrast, a staged/bundle/join channel boundary that can block progress is not hidden from Appendix K; it is part of the explicit $\text{Sys_B}^{\text{elab}}$ proof boundary, even if it is projected away from the outer Σ_{DUT} trace by Π_{elab} .

- **Combinational well-formedness:** The network of combinational couplers and ready/valid connections shall be combinational acyclic (no pure combinational ready/valid loops). Any required feedback that would otherwise create a combinational loop must pass through an explicit finite-capacity channel stage (and is therefore represented in the back-annotated realization and its effective capacities).

Accordingly, the formal development need not depend on the particular back-annotation construction, provided it yields an explicit source-side reference model $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}$ (Definition Core.1 and Appendix J, Remark 2) equipped with a projection/accounting map Π_{elab} satisfying the invariant above. The elaborated model must respect E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG semantics at its explicit proof-internal abstract boundaries, while its projection back to the chosen outer Σ_{DUT} -visible boundaries must preserve the externally visible Push_c / Pop_c history and aggregate capacity/occupancy behavior. In these cases the proofs do not target the raw outer-boundary Sys or the non-elaborated outer-boundary Sys_B directly; they target the proof-internal explicit abstract channels of $\text{Sys_B}^{\text{elab}}$ together with their Π_{elab} projection back to the outer DUT/testbench-visible boundary.

Lean-oriented clarification / proof-parameter convention

The formal theorems do not require a unique algorithm that constructs $\text{Sys_B}^{\text{elab}}$ from RTL_B . Instead, the chosen comparison boundary Sys_ref is part of the theorem instance. For ordinary bounded-FIFO cases, Sys_ref may be Sys_B . For cases involving proof-relevant internal staging, multi-input joins, combinational couplers, sync-boundary epoch gates, or shared-capacity effects, Sys_ref shall be supplied as $\text{Sys_B}^{\text{elab}}(E)$ for some well-formed elaboration package E .

Thus elaboration is not an extra semantic freedom inside the proof. It is an explicit input to the proof instance, and all Appendix I–K definitions are interpreted over that chosen Sys_ref .

Definition Q.1 (Elaboration package for $\text{Sys_B}^{\text{elab}}$).

For a chosen outer source reference boundary Sys_B and outer observable alphabet Σ_{DUT} , an elaboration package E consists of:

1. an elaborated source reference model $\text{Sys_B}^{\text{elab}}(E)$;
2. a finite set of explicit abstract channels $\text{Chan_elab}(E)$, each with finite capacity $B_E(d) \geq 0$ and cycle-boundary occupancy $\text{occ}_d(t)$;
3. a distinguished subset of outer-boundary channels Chan_outer and a projection/accounting map

$$\Pi_{\text{elab}} : \text{Chan_elab}(E) \rightarrow \text{Chan_outer} \cup \{\perp\};$$

4. a trace projection map $\pi_{\Sigma, E}$ from proof-internal Σ_{elab} traces to outer Σ_{DUT} traces; and
5. a state-accounting map A_E that computes each outer $\text{occ}_c(t)$ from the occupancies and token/accounting state of the elaborated channels in $\Pi_{\text{elab}}^{-1}(c)$.

The package is well formed only if the following obligations hold:

(a) Outer-trace preservation. $\pi_{\Sigma, E}$ neither creates, drops, duplicates, nor relabels any outer-boundary $\text{Push}_c(v)$ / $\text{Pop}_c(v)$ event.

(b) Occupancy preservation. For every outer channel c and every ε -quiescent cycle-boundary state σ at time t ,

$$\text{occ}_c(\sigma) = A_E(c, \sigma),$$

and projected Push_c / Pop_c commits update A_E exactly as required by E4 for the outer channel c .

(c) Internal refinement. Any internal staged/bundle/join transfer either is projected away by $\pi_{\Sigma, E}$ or projects to a unique corresponding outer-boundary commit. Projected-away internal transfers preserve all outer occupancies $A_E(c, \sigma)$.

(d) No hidden cross-channel disablement. If progress can be blocked by a join, mutual-stall condition, sync-boundary gate, or shared-capacity condition, then that blocking condition appears as ordinary BoundaryReq / ChannelEnabled / ChannelDisabled status at some explicit abstract channel $d \in \text{Chan_elab}(E)$. It is not an additional hidden predicate on an otherwise E4-enabled outer-boundary operation.

(e) WFG visibility. Every proof-relevant blocking point introduced by the elaboration is represented at an explicit abstract boundary of $\text{Sys_B}^{\text{elab}}(E)$, and therefore may be used by Appendix K's Front_P , ChannelDisabled , and WFG definitions, even if its events are projected away from the outer Σ_{DUT} trace.

(f) Combinational well-formedness. Any combinational coupler logic in the elaboration is stateless and acyclic after ε -settling; any feedback or storage needed for correctness is represented by an explicit finite-capacity abstract channel.

(g) Point-to-point elaborated channels. Every explicit abstract channel in $\text{Chan_elab}(E)$ at which BoundaryReq , $\text{ChannelEnabled/ChannelDisabled}$, Front_P , or Appendix K WFG facts are evaluated has exactly one producer process and exactly one consumer process at that boundary of evaluation. A join, bundle, or shared-capacity structure that would multiplex several producers or several consumers at an evaluated boundary shall be remodeled with explicit per-endpoint point-to-point sub-channels (for example, via an explicit arbiter process), as required by the point-to-point premise of Lemma K.2b.

(h) Frontier-item declaration and boundary attribution. The elaboration package declares every proof-internal frontier item introduced by $\text{Sys_B}^{\text{elab}}(E)$, including its owning process, its membership in $\Omega_P / \Omega_{\text{msg},P}$ when applicable, its owning source-level message instance $\text{MsgOf_P}(x)$, its evaluated abstract boundary $\text{BoundaryOf_P}(x) \in \text{Chan_elab}(E)$, where P is the owning process of x , and the partial order $\leq_P$ among ordinary source items and introduced elaborated frontier items. For ordinary unelaborated message calls there is a unique frontier item and a unique evaluated boundary. For elaborated calls, one source-level message instance q may own multiple frontier items at different explicit abstract boundaries; in that case BoundaryReq , ChannelEnabled , ChannelDisabled , Pending_P , and Front_P are interpreted for the pair $(q, \text{BoundaryOf_P}(x))$ represented by the frontier item x , not for q alone. Thus $\text{BoundaryReq}(q, \sigma)$ remains only a shorthand for the ordinary unique-boundary case, while $\text{BoundaryReq}(x, \sigma)$ means the endpoint-owned request for $\text{MsgOf_P}(x)$ at $\text{BoundaryOf_P}(x)$. The package also states how these introduced items project through $\pi_{\Sigma, E} / \Pi_{\text{elab}}$ back to outer Σ_{DUT} events and occupancy accounting.

(i) Qualified same-cycle settling. The elaborated source model as a whole shall satisfy the Core.Settle L2 frame and the source-side hypotheses of Definition Core.QSync . The elaboration package is responsible for the settling facts contributed by the staged/bundle/join/guard/coupler/epoch-gate structure. For that elaboration-owned cycle-local subnetwork, every applicable $\rightarrow_{\varepsilon, L2}$ step shall respect the L2 frame—preserving the cycle index, registered process/FSM/pipeline state, registered explicit-channel storage/occupancy/credit state, and cycle- t exogenous inputs, with no explicit-boundary Σ commit or next-cycle registered update—and, after cutting at the state-bearing boundaries, the subnetwork shall be finite, deterministic, combinational acyclic, and complete for the boundary predicates it owns. The package shall provide or cite a compositional/template proof of these template-side facts; that proof may be qualified once for each legal elaboration template and then bound to the concrete E instance.

Whole-source composition requirement. The template-side facts alone do not imply $\text{QualifiedSynchronousModel_M}(I)$ for $M = \text{Sys_B}^{\text{elab}}(E)$. The selected theorem instance shall additionally establish the applicable design-owned source-settling facts, including stable sequential drives under $\text{Core.RegSeq/direct-input}$ contracts, inclusion and deterministic evaluation of R2C combinational components, the applicable non-template L2-frame facts, and completeness at non-template selected boundaries. It shall also establish a checked template/design settling-composition condition: every dependency edge crossing the design/template partition is included in the complete same-cycle dependency analysis, and connecting the two subnetworks introduces no violation of QS2–QS5. In particular, the dependency graph of the complete combined same-cycle settling network shall satisfy QS4. A sufficient discharge is a checked combined dependency rank or topological order in which

every design-local, template-local, and cross-partition settling edge advances the order. The template-side, design-side, and composition evidence together establish the corresponding $\text{QualifiedSynchronousModel_M(I)}$ fact for $M = \text{Sys_B}^{\text{elab}}(E)$, so Theorem Core.QSync-Ref supplies existence and uniqueness of the source SettlePoint/KCut . This obligation makes the former implicit source-settling termination premise explicit; Definition Core.Settle by itself does not assume a fixed point exists.

When $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}(E)$, all references to $B(\cdot)$, occ_ , BoundaryReq , $\text{ChannelEnabled/ChannelDisabled}$, Front_P , and the Appendix K WFG are interpreted over the explicit channels of $\text{Chan_elab}(E)$, with outer observations recovered only through $\pi_{\Sigma, E}$ and Π_{elab} .

Notation convention for A_E . This document writes the accounting function in channel-first form $A_E(c, \sigma)$. A mechanization may equivalently use the state-first form $A_E(\sigma, c)$ so that state-indexed maps compose naturally with elaborated-state projections. The two spellings denote the same accounting function with the argument order reversed; no semantic distinction is intended.

Locality of elaboration

The elaboration package E is constructed locally, per process and per channel boundary. Each elaboration pattern in this appendix (multi-input pipelined-loop staging, sync-boundary epoch gating, R2C combinational coupler, etc.) is determined by the local channel structure of the process at the boundary in question — its number of input channels, its sync-boundary discipline, its pipelining configuration — and does not depend on the structure or elaboration of any other process. Independently chosen per-process elaborations compose into a system-level $\text{Sys_B}^{\text{elab}}(E)$ without further global modeling. Π_{elab} then projects the introduced internal channels back to the outer Σ_{DUT} alphabet on a per-channel basis. In this sense, the elaboration step preserves the per-process compositionality stated in the introduction: it refines each process's abstract boundary, it does not introduce cross-process modeling obligations.

Appendix R – Compact Derived Formal Core and Mechanization View

R.1 Purpose and normative status

This appendix is a derived companion to the normative theorem stack. It introduces no definition, premise, execution domain, reconstruction function, or theorem interface consumed by an earlier section. The normative dependency order is: the Normative Formal Core; Appendix I process-local replay and preparation; Appendix J system construction, KObs reconstruction, and equal-KObs correspondence; and Appendix K wait-for/deadlock factoring and preservation. This appendix may be used as a compact review or mechanization map, but citations needed to instantiate a theorem shall point to those normative locations.

Deletion-safety rule. Removing this appendix shall not make any earlier formula, proof obligation, certificate field, or theorem statement undefined. A future edit that introduces an upstream citation to an Appendix R item is therefore an architectural regression.

R.2 One-way dependency structure

$\text{Core} \rightarrow \text{I} \rightarrow \text{J} \rightarrow \text{K} \rightarrow \text{R}$.

The Core fixes Sys_ref , explicit abstract channels and elaboration, Σ/ϵ , global Core.4 quiescence, L2 Core.Settle points and Core.QSync, the Core.KCut comparison domain, Core.ReachPrep, dynamic frontier items and source order, Commit/Issue/BoundaryReq, Core.IssueFair,

ChannelEnabled/ChannelDisabled, Pending and Front, Core.7a BlockingMsgNextFront, Core.ElabProj, Core.WFG, Core.RegSeq, Core.LLegal, Core.Phase, Core.PolicyL/Core.PolicyS, Core.16/Core.Drain discipline, Core.CycleState/Core.CycleResult/Core.CompleteCycle/Core.SilentCycle, DeclaredTerminal/Core.TerminalIdle, Core.EnvStep/Core.EnvTerminal, Core.CycleClosure, Core.LAdm, and the Core.18c KObs datatype and pure reconstruction functions. Appendix I proves process-local deterministic replay and finite preparation. Appendix J composes the local evidence, proves the J.KObs reconstruction lemmas, and establishes Corollary J.1 equal-KObs correspondence. Appendix K factors the deadlock predicates through that record and adds the operational liveness, drain, fairness, response, terminal/reset applicability, and certificate-coverage obligations. Appendix R only summarizes the completed chain.

R.2a K-observation architecture

The formal development has three explicit layers. First, the operational semantics define Core.RegSeq, Core.ReachPrep, ReturnedAtCycle, Policy L/S, channel commits, Core.IssueFair, Core.Settle/Core.QSync, Core.CycleState/Core.CycleResult/Core.CompleteCycle/Core.SilentCycle, DeclaredTerminal/Core.TerminalIdle, environment/cycle closure, and Core.Drain/K.Drain. Second, Appendix I records enriched deferred NB holes and field-sensitive local preparation; Appendix J derives the dependency order, proves J.RegPhaseNF, performs Horizon-batched J.UnitExt, constructs J.GWR/J.RegPhaseReach, and proves Theorem J.1/J.HCanon. Third, the K cutpoint layer combines H and O in KObs. J.KObsConf transfers one selected cutpoint; K.DrainReach supplies the remaining nonwithdrawal/K_e/Core.16 bridge, and K.RLAdmReach is its conjunction with J.RegPhaseReach. K.CertifiedCutpoint and Theorem K.1A operate per cutpoint; Core.QSync-Post derives Core.KCutClosure_post from the qualified synchronous structural evidence, and K.CertCov/K.1A-Total supply the separate certificate-totality/universal-preservation layer. The Policy-S reduction then adds K.EnvMF, K.DomResetScope, the shared K.InterruptionGate schema with K.ResetEpoch and nested K.TerminalDisposition instantiations, K.BF, K.Resp, and the K.2b/K.2c operational route.

(1) Operational Issue is retained. Issue transitions and the Issue/Commit linkage remain part of the operational semantics and continue to govern R4/E3/E5, Policy L, Policy S, Axiom Core.IC, Core.Drain/K.Drain, the applicable progress and fairness premises including K-O1, Lemmas K.2b.R/K.2b.P/K.2b.Q, and the K.Resp monitor trigger. KObs removes historical issue timestamps only from the cross-model cutpoint-observation and certificate interface; it does not redefine Issue(q), identify Issue(q) with Commit(q), or permit an operational proof to ignore whether work was already issued before a block.

(2) KObs is an observation-layer quotient, not a normalized execution. The factoring theorems prove that K-relevant cutpoint predicates depend only on replay history plus the current attributed residual offers. They do not construct a replacement execution in which committed operations issue in their commit cycles, and no issue-erasure normal form is assumed.

(3) Residual offers are typed over frontier items and explicit boundaries. A residual offer identifies the owning process P, frontier item $x \in \Omega_msg.P$, owning dynamic message-call instance $q = MsgOf_P(x)$, evaluated explicit abstract boundary $c = BoundaryOf_P(x)$, endpoint direction, and reset epoch. Its operation kind is the explicitly derived field $kind(o) := kind(q)$, with well-typedness requiring producer-valid exactly for a Push instance and consumer-ready exactly for a Pop instance. A bare set of source calls q is insufficient in Sys_B^elab, where one source-level call may own several proof-internal frontier items at different explicit boundaries.

(4) The full elaboration package E is a parameter of KObs. E includes the selected Sys_ref form, explicit abstract channels, capacities B(c), Π_elab , ownership, boundary and projection maps, frontier-item universes and partial orders, and any bundle/join/epoch-gate structure. KObs is not parameterized by B alone.

(5) Cross-model residual-offer conformance is the bidirectional boundary fact $J.OfferConf$ of Appendix J: every abstract residual offer has its attributed RTL boundary request asserted, and every relevant asserted RTL boundary request is attributed to exactly one stated residual offer. The record enforces at most one attributed outstanding source-level offer per endpoint direction at each explicit abstract boundary. This prevents a proof-internal realization transfer already counted in channel occupancy from also being represented as a second source-level offer.

(6) Residual-offer reachability remains separate from boundary conformance. $J.OfferReach$ establishes that a proposed $KObs$ record is in the image of an admissible reachable Sys_ref cutpoint with the same annotated replay history and witness w . $J.OfferConf$ establishes agreement between the record's residual offers and the RTL requests at the selected cutpoint. $J.KObsConf$ requires both; $OfferConf$ alone cannot manufacture an unreachable source offer set, and $I.ReplayObs/I.DR$ replay matching alone cannot determine $I.CutpointOfferObs$ or which uncommitted requests are presently asserted.

(7) Snapshot drain closure is distinct from the transition discipline. $DrainClosedNow(D, KObs)$ means that, at the selected $Core.KCut$ K -facing cutpoint, no already-offered non-frontier blocking request owned by a process in D is channel-enabled. It is a function of the cutpoint observation. $DrainDiscipline$ denotes the continuation-level no-new-later-work, non-withdrawal, finite non-replenishing drain, and sync-boundary conditions of $Core.Drain/K.Drain$; it remains an operational premise.

(8) The minimal K -specific residual-offer abstraction carries no pending Push payload. Payload stability while a blocking Push request is asserted remains required by Axiom $Core.IC$ and Appendix C, and the committed payload remains checked by the value-labeled replay/trace results. A proof or checker needing mid-flight data-wire correspondence may add that separate implementation obligation without enlarging the deadlock observation itself.

(9) Elaboration and issue policy remain separate. Pipeline storage, bundle/join structure, and internal stage movement are represented by E and the explicit $Sys_B^{\wedge}elab$ channels. Policy L/S governs issue of the actual source-level operation instances and their elaboration-provided frontier items. $KObs$ records current source-attributed offers but does not turn intra-realization movement into additional source calls or double-count capacity already represented by $E4$ occupancy.

(10) $KObs$ -facing waiver classes are observationally closed. For any theorem, certificate, or checker that claims a waiver-parameterized cutpoint predicate is determined by $KObs$, write $KObsClosed_I,E,w(W)$ for the requirement that, for all $\sigma_1, \sigma_2 \in KCutReach_ref(I)$ of the same fixed instance, $KObs_E,w(\sigma_1) = KObs_E,w(\sigma_2)$ implies $(\sigma_1 \in W \text{ iff } \sigma_2 \in W)$. $K.Resp$ monitor waivers are a separate complete-run certificate interface: they are keyed by the dynamic process-facing frontier instance and must satisfy $K.RespCov$'s coverage-preservation rule. A waiver class that depends on historical Issue timing, implementation microstate, monitor age, or any other fact not reconstructible from $KObs$ cannot be used in $KObs$ extensionality unless a separate proof establishes $KObsClosed_I,E,w(W)$.

(11) The primary serial route and diagnostic terminal reconstruction are separate instance data. The ambient theorem instance records the $K.Resp$ monitor schema, finite-bound function, reset/end-of-test disposition, and response-waiver policy used by $CF-S$. Those monitor fields are not part of $KObs$ and are not transferred to RTL as latency claims. Separately, an instance may request diagnostic reconstruction of $WFG+$, $EnvStop$, $DoneStop$, $Dead_KS$, $K.EnvOrd$, or $K.Done$. For $EnvStop$ diagnostics, the instance records either the standard $B1$ -avail, pull-based, consumption-ordered input model with default always-ready outputs, or an explicit $EnvReplayAdequate$ certificate showing that permanent next-interaction availability is a function of H and w . Without one of these environment conditions, $KObs$ still factors the ordinary internal $Dead_K$ predicate but makes no extensionality claim for $EnvStop$ -dependent diagnostic predicates.

(12) The certificate boundary is layered and explicit. A proof-backed application records: (a) $K.CertHdr$, including $Core.RegSeq/Cat\#$ and source-simulation conformance plus the $Core.QSync$ structural-evidence provenance; (b) $CF-O$ with $H,O,J.HCanon/J.OfferReach/J.OfferConf/J.KObsConf$ and, for

Appendix K, J.RegPhaseReach provenance plus K.DrainReach; (c) the selected A5 payload; and (d) the Core.QSync-Post theorem reference establishing Core.KCutClosure_post plus K.CertCov for a universal claim. J.RegPhaseReach is theorem-derived on the compositional route from enriched holes, field-sensitive footprints, J.RegPhaseNF, and Horizon-batched J.UnitExt; DrainReach remains independently checked. Deferred-hole records are proof-internal and empty at exported cutpoints, so they do not enlarge KObs. Derived Front/WFG/deadlock values remain checker-reconstructed.

R.3 Compact theorem-instance package

A theorem instance fixes the design, source and RTL identities, Sys_ref form, elaboration package E and Π_{elab} , explicit abstract boundaries and capacities, initial/reset state, stimulus and environment model, witness choices, arbitration and ε -normalization conventions, observation boundary, waiver classes, and—when the Policy-S route is used—the concrete deterministic simulator and K.Resp monitor configuration. These bindings are supplied by the Core and the certificate interfaces in J and K rather than by this appendix.

At a selected source K-facing cutpoint $\sigma_{\text{ref}} \in \text{KCutReach_ref}(I)$, Core.18c records $\text{KObs_E}, w(\sigma_{\text{ref}}) = \langle E, w, H, O \rangle$. J.OfferReach proves that the record is source-realizable, J.OfferConf proves the attributed RTL request abstraction, and J.KObsConf gives the same typed record to the selected RTL cutpoint. J.KObs-Proc, J.KObs-Chan, J.KObs-Offers, and J.KObs-WFG reconstruct the process, channel, frontier, request, enablement, and WFG facts. K.KObs-Dead and K.KObs-Ext factor the relevant deadlock and waiver-dependent snapshot predicates.

R.4 Compact safety and cutpoint bridge

Derived summary R-C1. Under the hypotheses of Theorem J.1 and Corollary J.1, every covered reachable RTL_B prefix has a compatible reachable Sys_ref witness under E1–E5, and the selected ε -normalized RTL/source cutpoints share one Core.18c KObs record. Equal reconstructed K-facing facts are functional consequences of that typed equality; they are not independent state-equality assumptions and do not transfer response-monitor age or continuation-level drain discipline.

R.5 Compact liveness bridge

Derived summary R-C2. For the exact source witness selected by Corollary J.1, J.RegPhaseReach supplies the Core.Phase L0–L4 phase-correct reachability component. K.DrainReach separately supplies nonwithdrawal, $K\varepsilon$, and every activated Core.Drain/K.Drain obligation. Their conjunction K.RLAdmReach places that exact prefix in $\text{Pref_L}^{\text{adm}}(I)$. If A5-L excludes Dead_K^W throughout that admissible domain, Theorem K.1A excludes the corresponding reconstructed deadlock at the certified RTL cutpoint; Core.KCutClosure_post and K.CertCov are additionally required for the universal reachable-cutpoint conclusion.

For the principal serial-reduction route, SDet selects one concrete Policy-S execution and CF-S supplies complete bounded-response coverage and cleanliness under K.EnvMF, a certificate-bound K.DomResetScope/DomEpoch, and the shared K.InterruptionGate schema. K.ResetEpoch is instantiated for every candidate component; only on K.ResetEpoch.Continue is K.TerminalDisposition instantiated. Their DirectFail branches are independently mapped finite K.RespFail witnesses, while the nested Continue branches select the K.2b.E/R/P/Q dominance route. K.OpKey and K.CV including K.CVPred/K.CV-WF and Lemmas K.2b.P0/P1, K.BF including Lemma K.BF-Persist and the exhaustive Core.IC lifecycle plus the gate-level exhaustiveness citation, lowering, fairness, progress, drain, environment conditions, and K.Resp coverage remain separate premises. Corollary K.2c first establishes the lowered A5L_Sys_K result and K.Low.A5 transports it to A5-L on Sys_ref. Appendix R does not weaken, combine, or replace any of those premises.

R.6 Mechanization decomposition

A faithful mechanization should preserve the same module boundaries: (1) Core types and operational semantics; (2) Appendix I local replay/preparation; (3) Appendix J finite-ideal composition and KObs reconstruction; (4) Appendix K snapshot factoring, admissible continuation, serial reduction, and certificate coverage; and (5) this appendix as generated documentation or a compact theorem-export layer. In particular, the mechanization should not define a second `Sys_ref`, a second admissibility relation, or a second KObs correspondence inside the companion layer.

R.7 Audit checklist

A structural audit passes when: every shared definition is located in the Core before first use; Appendix I depends only on the Core and its explicit premises; Appendix J imports I and the Core; Appendix K imports the Core, I/J results, and its named liveness premises; no earlier section cites Appendix R for a required definition or lemma; and Appendix R's compact statements are traceable to named normative results without adding stronger claims.

Appendix S – Lean Proof Strategy Document

A separate Lean proof strategy document is under development. It should import, rather than duplicate, the normative interfaces fixed here: `Core.ElabProj`; `Core.RegSeq` and `ReturnedAtCycle`; `Core.WFG`; `Core.16/Core.Drain/Core.CycleState/Core.CycleResult/Core.CompleteCycle/Core.SilentCycle/Core.Cycle Closure/Core.CycleChoiceExt/Core.EnvStep/Core.EnvTerminal/DeclaredTerminal/Core.TerminalIdle/Core.LAdm`; `I.DeferredNBHole/NoDependentUse`; `J.ConstructionAction` and the field-sensitive location vocabulary; `J.ReqSnapshot/J.NBDecision/J.BufSameCycle`; `Dep_J` semantic-versus-D5 stratification/`J.HorizonOrderSafe/FR/acyclicity/canonical order`; `J.RegPhaseNF/J.RegPhaseReach`; `PCert/CCert/LocalGWR`; `frame/local-lift/commutation/enablement/Horizon-batched UnitExt`; `J.GWR-Comp/J.GWR`; `J.TraceCertCov/J.RefProjCover/J.1-Safe`; `J.HCanon/OfferReach/OfferConf/KObsConf`; `J.KObs-Proc/J.KObs-Chan/J.KObs-Offers/J.KObs-WFG`; `K.KObs-Dead/K.KObs-Ext`; `K.DrainReach/K.RLAdmReach`; `K.OpKey/K.CVPred/K.CV-WF/K.2b.P0/K.2b.P1`; `K.Low.C/K.Low.A5`; `K.DomResetScope/DomEpoch`; `K.InterruptionGate` and its exhaustive `K.BF` `K.ResetEpoch/K.TerminalDisposition` instantiations with `Continue/DirectFail` branch typing and reset-first nesting; `Core.SettleIsKCut/Core.KCutClosure/Core.KCutObserve`; `K.CertifiedCutpoint/K.CertCov` and `K.1A/K.1A-Total`; and Appendix R's deletion-safe, nonnormative derived-companion status. The exported interface must preserve the distinction between supplied local facts, theorem-derived order and phase normality, the residual drain bridge, the KObs observation boundary, and universal package coverage. The separate Lean document should be updated in the same named revision before mechanization relies on the old stop-at-hole or unit-at-a-time interfaces.

Appendix T – Scheduling-Specific Obligations Beyond Standard Simulation Theory

T.1 Purpose and Nature of the Formal Result

Standard formal-methods libraries provide reusable foundations for labeled transition systems, weak or ϵ -abstracted transitions, forward simulation, finite and infinite executions, trace inclusion, well-founded induction, fixed-point reasoning, and related proof techniques. Those foundations can substantially simplify a mechanization of the results in this document. They do not, however, establish the HLS scheduling contract or its safety and liveness guarantees by themselves.

In particular, the top-level safety result in this document is not ordinary trace inclusion or trace equivalence. It is the document’s witness-relative ordered trace-compatibility relation, written $\approx$, over the selected proof-internal observable alphabet Σ . This relation is defined over complete observable units and bucketed executions, preserves the required $\leq_J$ ordering constraints, and permits independent observable units to commute or to appear in different cycle groupings in Sys_ref and RTL_B . It also treats the $\text{Post} \rightarrow \text{Ref}$ and restricted $\text{Ref} \rightarrow \text{Post}$ uses differently. Consequently, a generic result of the form “forward simulation implies trace inclusion” does not directly yield the compatibility theorem proved here.

The principal proof obligations arise from features specific to HLS scheduling and hardware communication: source-order constraints, same-cycle and atomic observable behavior, witness-selected replay, explicit channel-boundary semantics, back-annotated capacity, process composition, KObs cutpoint observation and certificate correspondence, fair channel progress, pipeline drain behavior, and the connection between overlapped execution and conservative serial-source deadlock. This appendix summarizes the obligations that must remain explicit in any faithful formalization, even when standard transition-system and simulation libraries are reused.

The obligations divide into two structurally different groups. Safety has local design evidence and a generic global conclusion: process-local scheduling/replay certificates, explicit-channel certificates, and named atomic-interaction certificates are combined by the proved J.GWR-Comp construction and then by pairwise trace composition. Liveness is a conditional global preservation result over complete admissible executions and wait-for components and cannot in general be obtained by composing independent per-process liveness theorems.

The purpose of this appendix is not to prescribe a particular theorem prover, transition-system library, or Lean architecture. Rather, it identifies the scheduling-specific semantic and proof obligations that a mechanization must discharge in addition to any generic infrastructure supplied by standard formal-methods libraries.

T.2 Safety-Specific Obligations

T.2.1 Initial-State and Reset-Epoch Correspondence

The induction used to construct a matching Sys_ref execution requires a valid base case before the first observable unit is matched. The post-reset RTL_B state must correspond to the prescribed source initial state, including process control locations, source-visible values, channel state, request state, and the selected abstraction relation.

For the document-aligned theorem instances, A10 requires reset-empty initial occupancy at the selected abstract channel boundaries. There must also be no unaccounted pending endpoint request, stale synchronization state, or hidden accepted message that is absent from the source reference model. Reset correspondence is not necessarily a one-time property limited to initial system startup. When dynamic reset is supported, each affected process enters a new reset epoch, and the formalization must re-establish the applicable state, channel, anchor, and direct-input correspondence for that epoch. The hold-stable and synchronization-controlled rules for direct inputs are part of ensuring that reset and restart do not introduce a pre-HLS/post-HLS mismatch. In this document, the Normative Formal Core supplies the RESET boundary observable unit and the reset well-formedness conditions RW1–RW5, while Lemma J.RS (MatchedResetEpochSplice) in Appendix J adds an explicit reset-splice case to the Theorem J.1 induction.

T.2.2 Source Enablement of the Next Selected Observable Unit

A compatibility proof cannot merely observe that RTL_B performed an action and then assert that Sys_ref can perform the corresponding action. It must prove that, in the currently constructed source-reference state, the specific dynamic operation instance represented by the next selected RTL observable unit is enabled.

The required source-side prerequisites can include the process control position, branch predicates, loop-iteration identity, prior source-ordered operations, synchronization boundaries, signal anchors, channel occupancy, channel availability, and witness-selected non-blocking outcomes. The proof must also establish that the selected unit is the correct dynamic occurrence rather than another static call at the same source location.

This obligation is central because RTL_B may contain added FSM states, overlapped loop iterations, internal pipeline stages, or eager/same-cycle issue of operations that were sequential in the source. The document no longer packages this bridge as an unexplained global premise. Appendix I proves finite process-local preparation (Lemma I.3/J.PCert); J.CCert checks the common boundary state and witness-significant non-blocking outcomes; J.ACcert checks simultaneous atomic participants; and Lemma J.UnitExt proves the complete next unit has a legal global source extension. Theorem J.GWR-Comp iterates that result to derive Definition J.GWR.

T.2.3 Same-Cycle Independence and Atomic Observable Units

A single hardware cycle may contain multiple observable events that are independent and therefore have no meaningful intrinsic order within that cycle. Imposing an arbitrary total order on such events could introduce false dependencies or make two compatible executions appear different merely because independent actions were listed differently.

Conversely, some groups of events must be treated as one indivisible observable unit. Examples include the paired endpoints of a rendezvous transfer, a synchronization handshake, and a stutter-canonical R2C fixed-point observation unit. The formalization must prevent the internal pieces of such a unit from being split across different matching steps.

The resulting observable model is partial-order and independence based, and is analogous in some respects to pomset or Mazurkiewicz-trace semantics. It is not, however, assumed to be an exact instance of a fixed trace-monoid model, because the relevant independence and ordering relationships may depend on dynamic operation instances, channel relationships, atomic grouping, and the execution-specific RequiredJOrder relation.

T.2.3a Same-Cycle Buffered-Channel Sequentialization

For a positive-capacity channel, a same-cycle Push and Pop are not modeled as fall-through operations. E4 evaluates both from the common pre-cycle occupancy. Therefore both can commit in the same cycle only when the channel is neither empty nor full. Lemma J.BufSameCycle proves that, in precisely that interior-occupancy case, the two corresponding source transitions may be serialized in either order while retaining their common RTL-cycle annotation and producing the same FIFO contents and occupancy. At empty or full occupancy the operation disabled at the cycle boundary remains disabled; the other same-cycle operation cannot enable it. Capacity-zero endpoints remain one atomic rendezvous unit.

T.2.4 Derived Validated Construction-Order Ideal Chain

The J.1 construction does not extend the source witness in an arbitrary ordering of observable actions. It follows the finite $\leq_J$ -ideal chain carried by one J.ValidatedConstructionOrder. On the compositional route, J.GWR-Comp derives that order from the concrete RTL_B operational annotations and locally certified prerequisites through J.FoundationRank, J.DepRank, J.DepAcyclic, and J.CanonicalRank. On the direct route, the package supplies an independently validated well-founded order and proof that every required and operational predecessor occurs earlier. In both cases the chain remains tied to the concrete RTL_B behavior; no separately supplied time-based selection rule is normative.

At each induction step, the next ideal adds one complete observable unit whose required predecessors have already been included. This ensures that the constructed Sys_ref prefix corresponds to the actual RTL_B behavior being analyzed rather than to an unrelated legal reordering of the same actions.

The validated construction-order chain provides the induction index needed to maintain process replay state, witness consistency, channel occupancy, synchronization state, signal anchoring, and matched-

prefix compatibility. Its order is not supplied as phase metadata on the compositional route. Definition J.DepJ generates local operational prerequisites from typed process, request, snapshot, channel, reset, environment, and elaboration facts; J.DepComplete proves that every unordered action commutes; J.FoundationRank and J.DepRank prove that every generated edge advances one graph-independent semantic rank; J.DepAcyclic follows; and J.CanonicalRank derives the theorem-produced total proof order. Stable keys only list otherwise commuting actions. The direct route instead supplies an independently validated order satisfying the same J.ValidatedConstructionOrder interface. Thus the enumeration is either a construction produced by J.GWR-Comp or an explicitly checked direct certificate, never an unexplained externally supplied time chain.

T.2.5 Witness Compatibility and Faithful Witness Selection (WC)

The source reference model may admit multiple internal ϵ -step choices, non-blocking outcomes, arbitration outcomes, or replay paths that produce superficially similar observable behavior. A selected witness therefore cannot be treated as an unconstrained proof artifact. WC requires the witness choices used by the proof to be compatible with the actual RTL_B-observed behavior and with the relevant channel and boundary state.

This is especially important for failed or successful non-blocking operations that may be represented partly or wholly through ϵ -state. WC prevents the proof from selecting a successful poll when the actual boundary state required failure, selecting a failure when the operation was available, or attributing an issue or outcome to the wrong dynamic operation instance.

Snooping, NB-CF, direct witness construction, and mixed per-site reasoning are methods for discharging WC. They are not alternative theorem premises that replace WC. The theorem application must ultimately carry evidence for the actual WC predicate used by Appendices I and J.

WC is load-bearing for both source enablement and deterministic replay. Those arguments determine a source state and next action relative to a chosen witness; WC is what ties that witness to the implementation behavior rather than to a hypothetical source schedule.

T.2.5a Common Request Snapshots and Non-Blocking Decisions

A witness-significant non-blocking call reads shared channel/request state and therefore cannot be decided independently inside a process-local replay path. Lemma I.3 stops at a marked decision hole. Appendix J reconstructs one ReqSnapshot for the exact dynamic call after all dependency-predecessor request/attempt writes are present and before the decision or channel update. J.NBDecision then derives success or failure from E4 and that common snapshot. Request issue and a poll that reads the request are read/write dependent and may not be commuted; the generated Dep_J relation orders them. A failed call changes only the caller's continuation and leaves committed channel history unchanged. A successful call is the corresponding observable decision/transfer unit; it cannot be consumed as an ϵ preparation step for a later unit. This is the formal reason that a failed rendezvous PopNB cannot coexist with a matching persistent Push in the same snapshot.

T.2.6 Deterministic Replay Under the Selected Witness

Equality of visible labels alone does not necessarily determine the source state reached after a prefix. The source model may contain control-flow choices, failed non-blocking polls, hidden ϵ -state, arbitration choices, or witness-selected outcomes that affect later behavior without appearing as independent Σ labels.

Deterministic replay states that, once the fixed stimulus, WC-compliant witness, dynamic-operation attribution, and complete process-local observable-unit history are fixed, the logical replay checkpoint is fixed. This is the I.ReplayObs slice: the replay control position immediately after the last matched unit, replay-relevant source values that influence later guards or payloads, completed-item status, candidate NextFront, anchored signal values, Pop/NB return values reconstructed from committed history and the witness, reset epoch, and normal completion. It is a quotient of the raw operational source state modulo certified replay-inert L1 preparation. Witness-fixed epsilon transitions may change replay fields,

so normalized replay uses two-sided congruence of the exact selected normalization paths rather than single-path neutrality or global confluence. ReplayObs does not determine the raw operational cursor, prepared temporaries, or which uncommitted blocking requests are currently asserted. Those operational facts form $I.CutpointOfferObs$, are represented at the normalized K -facing cutpoint by O , and are justified by $J.OfferReach$ and audited against RTL by $J.OfferConf$.

The formalization must distinguish the canonical immediate post-unit logical replay checkpoint from an explicitly epsilon-normalized cutpoint. Plain J.1 may use the immediate post-unit form without claiming epsilon-quiescence after every matched unit. Corollary J.1 and Appendix K use the normalized form only with the exact proof-relevant normalization witnesses selected for the theorem package and the paired-congruence obligation above.

T.2.7 E4 Occupancy and Explicit Boundary Behavior

E4 defines the legal message-transfer behavior at each explicit abstract channel boundary. For a bounded FIFO, this includes the occupancy update, full and empty conditions, and the circumstances in which Push and Pop are ChannelEnabled or ChannelDisabled. For a rendezvous boundary, it includes the coordinated producer and consumer behavior required for a same-cycle transfer.

The compatibility proof must show more than equality of transferred values. Every committed transfer must be legal under the selected boundary semantics, occupancy must evolve consistently, and the source and RTL_B interpretations of boundary availability must agree at the points used by the theorem. Generic simulation theory does not distinguish an ordinary bounded FIFO from a rendezvous boundary, synchronization gate, staged channel, join, bundle, or another elaborated boundary structure. These semantics are supplied by the scheduling model and must be proved for the selected comparison instance.

T.2.8 B-Faithfulness

The back-annotated capacity $B(c)$ is intended to summarize all storage or credit that can accept, retain, or backpressure messages between the selected explicit producer and consumer boundaries. That storage may be physically distributed across FIFOs, pipeline stages, skid buffers, registers, credit state, or other implementation structures.

B-faithfulness is stronger than a capacity upper-bound claim. The selected RTL boundary must exhibit exactly the Push and Pop behavior permitted by E4 for the declared $B(c)$, including the corresponding ChannelEnabled and ChannelDisabled behavior. Internal movements within the represented realization may be ϵ -steps, but they may not create, delete, duplicate, reorder, or relabel the Σ -visible boundary transfers.

If hidden implementation structure changes availability in a way that cannot be represented by an ordinary E4 channel with the declared capacity, that structure must instead be represented by explicit Sys_B^{elab} boundaries. B-faithfulness is therefore an implementation-side compliance obligation, not a consequence of the abstract scheduling theorem itself.

T.2.8a Qualified Synchronous Settling and the L2 Frame

The standard G–K model now factors same-cycle settling through Definition Core.QSync. The L2/K-origin frame freezes cycle index, registered process/channel state, and selected cycle-t exogenous inputs while allowing only cycle-local combinational/settling state to change; no Σ commit or next-cycle update occurs during settling. Core.RegSeq supplies the design-owned sequential-drive part of this contract, R2C supplies the design-owned combinational-process fixed-point semantics, Appendix Q supplies reusable elaboration-template settling facts plus the template/design composition requirement, and B-faithfulness/selected-boundary evidence supplies the RTL boundary mapping.

The formal development should prove two generic results: Core.QSync-Ref, establishing a unique source SettlePoint/KCut for each source K -origin including $Sys_B^{elab}(E)$, and Core.QSync-Post, establishing a unique post-HLS settled KCut and therefore Core.KCutClosure_post for every RTL K -origin. This moves

closure from a design-specific semantic-normalization proof to a theorem over independently audited structural hypotheses. It does not reduce the separate $K.CertCov$, $A5-L/K.2b/K.2c$, $K.Resp$, or drain/response-monitor obligations. The checker/trusted-base split is explicit: accepting a producer-tool settling claim without independent validation adds that claim to the trusted base; independently checking the concrete artifact keeps it as ordinary per-instance evidence.

T.2.9 Proved Local-to-Global Witness Construction

Process-local replay results do not by conjunction imply one common source execution or a cycle-correct phase order. LocalGWR supplies enriched deferred-hole records, field-sensitive footprints, snapshots, local dependencies, and $Core.RegSeq$ facts, but no global replay or phase certificate. $J.DepComplete/J.DepRank$ derive the order; $J.RegPhaseNF$ converts each finite Horizon fragment to the single $Core.Phase L1/L2/L3$ normal form modulo $J.HCanon$, using $J.Frame$, $NoDependentUse$, and $J.BufSameCycle$. Horizon-batched $J.UnitExt$ then performs outer induction over cycles and inner $L3$ commit order. Theorem $J.GWR-Comp$ constructs both $J.GWR$ and $J.RegPhaseReach$. This simultaneously closes $H3$'s rank/replay mismatch and supplies $H2$'s phase-order component by construction; nonwithdrawal, $K\epsilon$, and $Core.Drain/K.Drain$ remain the separate $K.DrainReach$ obligation. Lemmas $J.Frame$ and $J.LocalLift$ embed field-sensitive local segments and carry deferred NB holes through $NoDependentUse$ work. $J.PrepareCommute$ and $J.BufSameCycle$ justify the allowed swaps; $J.DepComplete/DepRank/DepAcyclic$ derive the order. $J.RegPhaseNF$ normalizes each Horizon to $Core.Phase L1/L2/L3$ modulo $J.HCanon$, $J.NBDecision$ evaluates the common settled snapshots, $J.UnitEnable$ derives the $L3$ units, and Horizon-batched $J.UnitExt$ constructs the reachable source chain. $J.GWR-Comp$ derives $J.GWR$ and $J.RegPhaseReach$. The direct route must independently supply the same order, phase-normal, and extension interface.

The non-blocking request-co-presence case is load-bearing. A process certificate records the implementation-selected outcome but stops before deciding it. The incident $J.CCcert$ reconstructs the complete request/availability snapshot, and $J.NBDecision$ proves the unique source outcome. Thus a persistent matching rendezvous request and a failed complementary poll cannot coexist in a valid package even when the committed channel history is empty; nor can a local path commute the poll backward across a request write that its snapshot reads.

Dependency completeness is also load-bearing. The flow does not supply a convenient phase/rank. Every edge must be generated by a specific local control, request-snapshot, channel-state, reset/environment, or explicit-component rule. $J.DepComplete$ proves that actions left unordered are genuinely commutative, including the explicitly proved interior-occupancy same-cycle buffered Push/Pop case. $J.DepRank$ proves every generated edge advances one global semantic foundation rank, $J.DepAcyclic$ follows, and stable proof keys totalize only commuting actions. A custom Sys_B^{elab} component must provide a local footprint/commutation certificate, not a global replay certificate. Proposition $J.0$ remains separate. It assumes an already selected compatible source/RTL trace pair and lifts the process/channel facts to $\approx_sys$; it does not construct the source trace. This separation keeps the global witness construction auditable and prevents circular use of a composition lemma whose trace pair has not yet been shown to exist.

T.2.10 KObs cutpoint correspondence and direct Appendix K consumption

Trace compatibility alone does not distinguish a process that has not reached a blocking call from one that currently has an attributed uncommitted request. The interface therefore augments the enriched committed/replay history H with the terminal residual-offer set O and uses the compact KObs record $\langle E, w, H, O \rangle$.

Lemma $J.HCanon$ is the named history-side bridge: compatible annotated source/RTL prefixes under the one $J.GWR$ package produced by $J.GWR-Comp$ or supplied by $J.GWR-Direct$ map to the same canonical replay-history quotient H . $J.OfferReach$ proves that $\langle E, w, H, O \rangle$ is realized by the selected reachable Sys_ref cutpoint. $J.OfferConf$ proves bidirectional unique correspondence between O and the complete

set of KObs-relevant asserted RTL boundary requests. B-faithfulness/E4 interprets the committed history at the explicit boundaries. J.KObsConf packages these facts and yields equality between KObsPost_J and the source KObs. For an Appendix K application, K.RLAdmReach is a separate operational proof that the exact source witness lies in $\text{Pref_L}^{\text{adm}}(\text{I})$; equal KObs and OfferReach do not imply that phase/drain property.

The process slice, channel contents, frontier, endpoint requests, enablement, Front, WFG, and snapshot drain facts are then reconstructed by typed functions. Mechanization should therefore target a small record plus pure reconstruction functions, rather than a relation containing seven independently supplied state clauses. The initial mechanization targets should include: (i) CanonHist and Lemma J.HCanon; (ii) the typed physical-boundary request set $\text{RelevantReqSet_E}(p_post)$; (iii) the projection $\text{ReqProjection_E}(O)$; and (iv) the equality $\text{RelevantReqSet_E}(p_post) = \text{ReqProjection_E}(O)$, together with unique attribution and endpoint-cardinality invariants.

Historical Issue remains in the transition semantics and in any operational witness needed by Theorem J.1, but it is not part of the cross-model KObs equality. For the RTL request-abstraction boundary, the mechanized checker or certificate must range over the complete selected explicit-boundary request interface, not merely over requests for which attribution records were successfully produced. A missing or unattributed KObs-relevant asserted request is a failed OfferConf proof, not an absent observation. Appendix K consumes the equal record directly for K.0, K.2, and K.2a; Theorem K.1A additionally consumes the separately checked K.RLAdmReach proof and one K.CertifiedCutpoint package. K.CertCov is a higher-level totality premise and should not be hidden inside reconstruction/extensionality lemmas.

T.2.10a Certificate and independent-checker boundary

The certificate interface separates recorded and reconstructed facts. K.CertHdr fixes the instance and trust boundary. CF-0 records H,O,the complete request inventory,and source/RTL cutpoint evidence. Its Appendix K extension references J.RegPhaseReach and K.DrainReach; the checker composes them into K.RLAdmReach. Field-sensitive deferred-hole records may be retained in the referenced construction artifact but are empty at the exported cutpoint and are not KObs fields. Front, BoundaryReq, enablement, WFG, DrainClosedNow, and the deadlock predicate are reconstructed. K.CertCov remains a separate instance-wide artifact.

This design concentrates implementation trust in auditable boundary obligations rather than in an opaque full-state relation. In particular, OfferConf completeness is checked against the entire RelevantReqSet at every selected explicit boundary, not only against successfully attributed requests. B-faithfulness and the instance header bind the same physical boundaries and capacities to the channel reconstruction. HCanon and OfferReach bind the replay history and residual offers to a real source witness; K.RLAdmReach binds that witness to the Core.LAdm execution domain used by A5-L. Large histories may be content-addressed, but the independent checker must be able to retrieve and replay them. A universal RTL deadlock-freedom claim additionally requires independently checked Core.QSync structural evidence from which Core.QSync-Post derives Core.KCutClosure_post/K-facing cutpoint existence, and a K.CertCov argument that no reachable K-facing cutpoint lacks this complete package. Historical Issue and ReturnedAtCycle timelines are intentionally not duplicated in CF-0. That omission is safe at the cutpoint observation interface because the referenced J.RegPhaseReach artifact validates Core.RegSeq and L0–L4, while K.DrainReach validates nonwithdrawal, Kε, and continuation discipline. KObs itself determines neither component.

T.2.11 Finite-Local-Trace Continuation

A process may cease producing local observable units while the overall system continues to execute indefinitely. In that case, the proof cannot simply append idle cycles to the process by invoking B6, because local inactivity does not imply that the complete system has reached a global fixed point.

Instead, the proof must construct or select an admissible infinite global Sys_ref continuation and project that execution onto the process whose local observable trace is finite. The process-local projection may remain constant while other processes, channels, or the environment continue to evolve.

A separate case applies when the entire system eventually satisfies Core.TerminalIdle: it has reached a genuine B5/global fixed point and neither future registered/internal machine evolution nor the fixed environment can re-enable work. Only in that case may B6 justify indefinite global idle stutter; temporary system-wide inactivity with a later environment or registered-state transition is represented by Core.SilentCycle instead.

This distinction is required to construct valid infinite reference witnesses without incorrectly treating a process-local terminal projection as a globally terminal execution.

T.2.12 Finite ϵ -Normalization and Existence of Canonical Cutpoints

A Core.KCut endpoint cannot simply be assumed to exist from an arbitrary operational microstate. The relevant requirement is Definition Core.KCutClosure at the per-cycle K-facing origin, not total normalization from every arbitrary prefix. In the standard qualified synchronous scope, source and RTL closure are theorem-derived rather than assumed from the word “settle.” Theorem Core.QSync-Ref proves that every selected source L1 endpoint has one finite L2 SettlePoint/KCut, including qualified Sys_B^elab(E) instances; Theorem Core.QSync-Post proves that every reachable RTL K-facing cycle-evaluation origin has one finite same-cycle NormPost endpoint in KCutReach_post(l), and therefore establishes Core.KCutClosure_post. B4 remains the finite ϵ -stutter bound where internally advancing proof-hidden work is used; B5 remains the stronger global fixed-point condition only where complete system quiescence is required. In Appendix K, K ϵ plus Lemma Core.KCutObserve ensures that a K-relevant blocker that persists through the selected normalization is represented at the resulting KCut. A future non-QSync extension would have to re-prove the same closure interface explicitly.

B4 provides the applicable finite ϵ -stutter or normalization bound for the safety and cutpoint layers. B5 identifies the stronger global fixed-point conditions used where complete system quiescence is required. In the Appendix K layer, K ϵ additionally requires the relevant ϵ behavior to be WFG-inert and appropriately bounded so that normalization does not alter the blocking structure being analyzed.

The proof must establish both the existence of the normalized cutpoint and preservation of the certificate facts needed afterward. These include the canonical history H, residual offers O, B-faithful boundary interpretation, and the J.OfferReach/J.OfferConf premises required to construct the common KObs record. Appendix K then reconstructs frontier, enablement, WFG, and snapshot drain facts from that record.

The immediate post-unit state used by Theorem J.1, the L2 SettlePoint used to trigger Core.16, and a later drain-closed KCut used by Corollary J.1 or Appendix K remain distinct proof roles even though Lemma Core.SettlesKCut shows that an existing source L2 SettlePoint itself satisfies Core.KCut. On the normal compositional Theorem J.1 route, Core.QSync-Ref supplies the finite source L2 settle required by J.RegPhaseNF/J.UnitExt; a direct J.GWR package may instead carry equivalent per-Horizon settling evidence. J.RegPhaseReach supplies the phase-correct L1/L2/L3 construction; Core.Settle names the activation boundary; Core.QSync-Ref supplies source settle existence/uniqueness in the standard scope; NormRef/NormPost and Core.KCut name the selected comparison endpoints; Core.QSync-Post supplies RTL KCut existence and derives Core.KCutClosure_post; and K.DrainReach/Core.SettleKBridge connects a persisting L2 blocked frontier to the later drain-closed source KCut used for Dead_K. The proof should not infer drain closure merely from SettlePoint/KCut membership.

T.3 Liveness-Specific Global Obligations

The safety obligations above are largely local or compositional. The remaining obligations concern global executions and cannot generally be discharged by proving that each process is locally live. The liveness theorem depends on the interaction of all processes, channels, fairness assumptions, environment behavior, drainable work, and the global wait-for structure.

T.3.1 B2, B3, B5, and Admissible Global Executions

Definitions Core.LLegal, Core.Phase, and Core.LAdm jointly define the normative source-side execution domain for Appendix K. Core.LLegal defines stepwise Policy-L legality and fixed-instance binding; Core.Phase defines the L0–L4 cycle structure and L1/L2 settled-frontier boundary; and Core.LAdm defines the finite admissible prefixes used for A5-L reachability and the complete admissible executions used for continuation, stabilization, fairness, and limit arguments. A syntactically legal transition sequence is insufficient if it violates the fixed witness/environment choices, the Core.Phase cycle-settle boundary, nonwithdrawal, $K\epsilon$, or Core.Drain/K.Drain after activation.

B2 supplies the applicable fair scheduler or arbiter progress condition: an action that remains continuously ChannelEnabled must eventually be selected and commit. B3 supplies the corresponding channel-progress property for persistent blocking requests at full, empty, or rendezvous boundaries. For a simple explicit channel boundary, B3 may be derived from B2 plus E4, but it remains a useful modular obligation for checking channel realizations and elaborated boundary chains. Under Core.LAdm these are complete-execution obligations; they do not retroactively reject an otherwise legal finite prefix merely because fairness cannot yet be decided.

B5 governs the stronger global fixed-point closure and environmental stability condition; Core.4 Quiescent_M is only the absence of enabled ϵ -steps and may coexist with enabled Σ -actions. B5 therefore prevents a proof from declaring complete-system termination while hidden internal or environmental progress remains possible under its stated premises, but it is not the definition of a K-facing cutpoint and does not turn an L2 SettlePoint into one. B5 remains part of complete Core.LAdm admissibility and may help discharge selected normalization/terminal-state arguments where its trigger holds.

Together with Definition Core.LAdm, these premises determine which complete Policy-L source executions belong to Exec_L^adm for the Appendix K liveness argument. The post-HLS preservation theorem remains a finite reachable-KCut result and does not impose a generic fair-extension-qualified RTL execution domain. The proof therefore separates phase-correct finite reachability/correspondence from the fair, progress-respecting complete source continuations that are explicitly required by K.2b.E and related liveness results.

T.3.2 Drain Normalization and Drain Closure

An ϵ -quiescent state need not be drain-closed. No internal ϵ -step may be available even though already-issued downstream message transfers can still commit and reduce the amount of in-flight work.

Drain normalization permits such previously issued work to complete fairly, subject to B2/B3, without accepting or issuing new work beyond the blocked source frontier. It therefore requires the applicable drain-only blocked-frontier discipline: no new source-later dynamic operation may be launched past the blocked point, already asserted requests are not withdrawn, and only already-issued downstream work may drain. The resulting cutpoint is drain-closed when no further permitted downstream drain transfer associated with the relevant blocked component remains enabled.

Snapshot/continuation separation. Lemma K.2a transfers only DrainClosedNow at a selected cutpoint by equality of KObs. Appendix K-P reuses the same cutpoint interface inside Appendix K-P: current outstanding requests are $O = \text{ResOffers}$, and the finite non-front subset DrainBag is the drain-normalization measure. Neither step establishes future issue legality. Core.Drain/K.Drain, B2/B3 progress, nonwithdrawal, and the no-replenishment proof remain operational obligations used to show that DrainBag cannot increase and that drain normalization terminates.

This construction separates persistent blocking causes from transient pipeline or channel contents. Without drain closure, a wait-for graph cycle might disappear merely because already-issued work had not yet been allowed to complete.

The drain-normalization proof must also show that the relevant closed WFG component and source frontier survive the normalization, that the witness correspondence remains valid, and that the

normalized cutpoint still satisfies the KObs reconstruction premises required by Lemmas J.KObs-Proc–J.KObs-WFG and K.KObs-Dead. $K\epsilon$ is used to ensure that the ϵ behavior involved in this process is WFG-inert.

The applicable discipline is not limited to pipelines. Automatic flush supplies it for pipelined or otherwise overlapped control. Ordinary non-overlapped control may supply it through a structural proof that no new source-later dynamic operation can be launched after the blocked point. In either case, finite B-faithful explicit boundary storage or credit makes the already-issued drain set finite, and the no-new-launch rule makes it non-replenishing.

T.3.3 K.2b Dominance and Bounded-Response Extraction

[Revision note: this subsection summarizes the deterministic Policy-S-primary proof architecture of Appendix K.4b and Appendix K-P.]

The Sys_ref witness selected from RTL may use liberal/overlapped issue. Policy S withholds source-later blocking issue. SDet fixes the actual pre-HLS Policy-S simulator as a functional selected transition system and selects one run artifact $p_S^{\wedge det}$; every other artifact generated under the same selected semantics has a prefix-comparable committed history modulo finite WFG-inert ϵ -stutter, and no incompatible history exists. A5-S/CF-S separately establish maximality, fairness, and completeness under K.CompleteS and validated bounded-response coverage/cleanliness under K.Resp.

Lemma K.2b proves deterministic dominance only after two nested K.InterruptionGate.Continue checks and uses independently mapped DirectFail certificates on the exceptional branches. It first instantiates K.ResetEpoch. K.ResetEpoch.DirectFail maps a selected liberal frontier to a pending covered Policy-S serial monitor and a matched affecting RESET, yielding cancelled-pending through RW2 without K.2b.Q. Only on K.ResetEpoch.Continue does it instantiate K.TerminalDisposition.

K.TerminalDisposition.DirectFail independently supplies the mapped pending monitor and declared maximal terminal state, yielding maximal-pending without K.2b.R/P/Q Step 1. Only K.ResetEpoch.Continue followed by K.TerminalDisposition.Continue permits Lemma K.2b.E to construct the infinite fair frozen liberal continuation; K.2b.R proves request reachability/persistence, K.2b.P proves serial dominance, and K.2b.Q extracts the response failure. Thus the shared schema removes duplicated branch logic without flattening or reordering the proof.

Any Policy-L internal deadlock therefore forces $p_S^{\wedge det}$ to produce a finite K.RespFail witness. A complete proof that the K.CompleteS-certified selected artifact is K.RespClean discharges A5-L. There is no family of incompatible Policy-S histories to reconcile under one selected SDet instance, so confluence and universal quantification over serial scheduler interleavings are absent.

The historical K.EnvOrd/K.Done and Dead_KS machinery is retained only for diagnostics. It can explain a fired monitor through an environment-starvation cone or normal-completion terminal, but it is intentionally not exhaustive: the H2 perpetual under-supplier stays live on unrelated channels and may produce no closed terminal component. Soundness rests on the pending process-facing monitor, not on classification of its remote owner.

The proof still requires the common K.OpKey cross-policy operation universe and K.CV, including the K.CVPred fact/provenance interface and K.CV-WF well-foundedness certificate used by Lemmas K.2b.P0/P1; point-to-point evaluated boundaries; nonwithdrawable requests; $K\epsilon$; Core.Drain/K.Drain; K.BF including Lemma K.BF-Persist and the exhaustive Core.IC blocking-instance lifecycle; K.EnvMF; the shared K.InterruptionGate schema with K.ResetEpoch and nested K.TerminalDisposition; and complete K.Resp coverage. K.2b.P1 performs well-founded induction only on $\leq_CV$, while K.2b.P0 separately proves occurrence/reachability from keyed predecessor agreement: the broad Appendix-G causal-dependency graph $\rightarrow$ may contain legal same-cycle cycles and is not an induction order. Multi-frontier elaborations require K-O2 and are supported by the serial route only under StandardEnv/B1-avail. A timed/reactive or otherwise nondefault environment is supported only for components certified to have one internal frontier item per blocked process; the corresponding multi-frontier case is out of scope.

T.3.3a A5 certificate interface

The primary CF-S certificate is a deterministic-execution, applicability, completeness, and bounded-response certificate. It identifies the concrete simulator/scheduler, arbitration, environment, witness and normalization semantics; supplies SDet ; records the K.OpKey realization/order certificate plus the K.CVPred provenance equations and K.CV-WF/rank certificate consumed by K.2b.P0/P1 ; records K.EnvMF ; records the $\text{K.InterruptionGate}$ schema/version and its K.BF exhaustiveness citation; records the certificate-bound $\text{K.DomResetScope/DomEpoch}$ and K.ResetEpoch evidence for each candidate component; records $\text{K.TerminalDisposition}$ evidence only for $\text{K.ResetEpoch.Continue}$ candidates; records or symbolically represents p_S^{det} ; proves progress/drain/fairness and K.CompleteS ; records the total process-facing monitor set and finite bounds; and proves K.RespClean . The common DirectFail payload binds $P, f_P, x_P^{\text{S}}, g_P$ with $g_P \leq_{\text{src}} f_P$ and the already-triggered pending monitor independently of K.2b.R/P/Q Step 1. $\text{ResetEpoch.DirectFail}$ adds the common pre-reset epoch, matched affecting RESET , and RW2 clearing; $\text{TerminalDisposition.DirectFail}$ adds the declared maximal terminal state while the monitor remains pending. A clean certificate normally proves $\text{ResetEpoch.Continue}$ and then $\text{TerminalDisposition.Continue}$ prospectively over the applicable candidate classes. The checker validates the certificate-supplied K.DomResetScope sets against D1–D4 , the fixed DomEpoch on the Continue branch, and this nesting rather than accepting a flat reset/terminal disposition list.

CF-3 and CF-4 must keep the snapshot/transition distinction explicit. KObs is the right observation component for the bad-state predicate, but it is not by itself a Markov state for Policy L or Policy S because it omits historical Issue timing and future issue eligibility. An invariant or exploration abstraction may add operational phase, fairness, or drain-control state, and must prove that its abstract Post relation soundly overapproximates the selected operational semantics. This is the mechanization-facing reason the certificate checker should expose separate modules for KObs reconstruction, operational transition validation, fairness/completeness, and route composition.

A5 certificates also bind response-waiver and environment/frontier applicability reasoning. CF-S records K.EnvMF and the exact dynamic classes that may be reset-cleared or left pending at DeclaredTerminal , and proves K.RespCov 's preservation condition: a non-waived liberal deadlock cannot map only to waived monitors. A timed/reactive/nondefault multi-frontier component cannot be admitted by a waiver, monitor, or substitute no-rescue assertion; it is outside the serial route. Optional diagnostic Dead_KS evidence may additionally record $\text{EnvTerminalAdequate}$ and KObs closure, but those records do not substitute for K.EnvMF or K.RespClean .

The H1/H2/H3 interface is intentionally factored. CF-0/J.KObsConf establishes one source/RTL KObs pair. J.RegPhaseReach , now proved by the registered deferred-hole phase-normal construction, establishes the L0–L4 component for that source witness. K.DrainReach separately establishes nonwithdrawal, $\text{K}\epsilon$, $\text{Core.Drain/K.Drain}$, and $\text{Core.SettleKBridge}$; together they are K.RLAdmReach . Theorem K.1A excludes one certified KCut . For the universal RTL result, the independently audited Core.QSync structural facts feed Theorem Core.QSync-Post , which derives $\text{Core.KCutClosure_post}$ and a canonical settled RTL KCut for each K-origin ; K.CertCov then supplies a complete certificate package for every cutpoint in that domain; Corollary K.1A-Total combines both with the per-cutpoint theorem. A future J.CertExist may derive K.CertCov ; no separate ad hoc RTL normalization theorem is needed in the standard QSync scope.

T.3.4 WFG and Conservative Serial-Source Deadlock Reasoning

The WFG identifies the liberal closed component transferred through the equal- KObs correspondence packaged by J.KObsConf . The serial route does not replay that state. It names the liberal source cutpoint K_L , checks K.EnvMF , uses operational dominance to establish a permanently pending process-facing serial frontier on p_S^{det} , and applies K.Resp 's finite-bound monitor. Thus A5-L reduces to A5-S for one SDet -selected Policy-S execution on the supported environment/frontier fragment: K.CompleteS supplies maximality/fairness/completeness, K.RespCov supplies total monitor coverage, and K.RespClean supplies the contradiction.

A6 remains essential for the dominance and request-reachability arguments: point-to-point ownership gives each frozen boundary one actual complementary endpoint and fixes transfer ordinals. Shared resources must be represented through explicit arbiter processes and point-to-point channels before K.2b applies. The revised proof does not need to continue an owner walk beyond the permanently pending process-facing frontier.

Once K.EnvMF and A5-S/CF-S establish applicability, K.CompleteS, K.RespCov, and K.RespClean for the SDet-selected Policy-S execution $p_S^{\wedge det}$, Corollary K.2c supplies A5-L and Theorem K.1A rules out the RTL-matched deadlock at any certified cutpoint. Core.KCutClosure_post, K.CertCov, and Corollary K.1A-Total are additionally required for the universal RTL conclusion; in the standard scope Core.QSync-Post derives the closure premise from the independently audited structural QSync facts: closure prevents a persistent relevant blocker from remaining outside KCut, and K.CertCov covers every cutpoint in that domain. No Dead_KS or K.EnvOrd/K.Done avoidance theorem is required. Timed/reactive or otherwise nondefault multi-frontier components require a non-serial A5-L proof and do not use this reduction. This argument is specific to the scheduling contract. It is stronger than ordinary trace inclusion or per-process local progress because it connects a global implementation-level blocking structure to a permanent process-facing obstruction in the selected serial source semantics. The bounded-response certificate is source-side proof machinery only; it does not state that RTL completes within the same bound.

T.4 Summary

Reusable transition-system and simulation libraries can reduce the amount of generic proof infrastructure needed to mechanize this document. They can provide definitions and standard theorems for finite and infinite executions, ϵ -closure, simulation, trace lifting, well-founded induction, and temporal reasoning.

The substantive guarantees depend on scheduling-specific obligations. In the safety layer, local process/channel/atomic/environment/reset certificates, Core.RegSeq evidence, enriched deferred holes, field-sensitive footprints, and request snapshots are converted by Dep_J, J.RegPhaseNF, and Horizon-batched J.GWR-Comp into one reachable phase-correct source witness; the dependency order, NB outcomes, and J.RegPhaseReach are theorem-derived. The remaining safety theorems establish ordered trace compatibility and KObs cutpoint agreement. In the liveness layer, K.DrainReach supplies the residual Core.16/nonwithdrawal/K ϵ bridge, while the A5 route establishes admissible progress, serial dominance, response coverage, and bounded-response exclusion of liberal deadlock.

Accordingly, generic simulation theory is reusable infrastructure, not a replacement for the scheduling contract. K.CertHdr binds Core.RegSeq/Cat# and all instance choices plus the Core.QSync structural-evidence provenance; CF-0 validates one source/RTL KObs pair and references J.RegPhaseReach plus K.DrainReach; their conjunction K.RLAdmReach qualifies the exact witness; K.CertifiedCutpoint packages the per-cutpoint evidence; Core.QSync-Post derives Core.KCutClosure_post and universal K-facing cutpoint observability from the qualified synchronous structural model; K.CertCov establishes universal package coverage over that domain; and K.A5Cert validates the chosen liveness discharge. The checker separately validates field-sensitive construction/phase evidence, qualified settling/frame evidence, full-state CycleResult/CycleClosure evidence where complete source runs are used, drain/fairness/completeness, and K.Resp coverage/bounds/waivers.

Appendix U – Revision Record

15 August 2026 — minimal mechanization-driven semantic clarification pass.

A narrow clarification pass incorporates only semantic distinctions exposed by pre-mechanization, without changing theorem architecture or introducing Lean-specific representation choices. Core.18 now defines cross-model KObs correspondence through the common typed record and leaves

J.KObsConf as the downstream theorem establishing it. The historical-name lookup marks Core.15a retired in favor of Core.WFG. Appendix G states explicitly that function-like Issue/Commit notation abbreviates the Appendix-C relational existence/uniqueness facts. R2C now identifies the cycle-entry valuation and the intended unique-settled-result meaning of determinism while allowing internal ϵ /stutter re-evaluation; it also states that Sys_ref and RTL_B need not share an internal combinational model or absolute cycle buckets. The cycle-locked R2C clause now makes per-side participation/completeness, implementation/certificate justification, and the Core.17/E2 single-clause relationship explicit. Core.16 clarifies that “launch” is intentionally broader than message Issue. Existing reactive-environment replay wording and the conditional certification status of K.Ser are unchanged.

14 August 2026 — deliberate proof-audit repair: cross-policy keys, lowering transport, dependency stratification, and cycle-extension interface.

A focused pre-mechanization audit produced eight repairs without changing the user-facing scheduling contract. (1) Definition K.OpKey now supplies a theorem-instance common dynamic-operation key universe, partial per-execution realization maps, and a key-level cross-policy source order; execution-local Core.5 universes remain unchanged. (2) New Lemma K.2b.P0 separates cross-policy occurrence/reachability from K.2b.P1’s conditional-on-occurrence value/identity determinacy, and K.2b.R now consumes P0 explicitly. (3) Definition K.A5L model-indexes the scheduler-robust premise; the Policy-S route proves the lowered Sys_K form and new Lemma K.Low.A5 transports it to the Sys_ref A5-L consumed by K.1A, with waiver pullback made explicit. (4) Definition J.DepJ now distinguishes semantic D1–D4 prerequisites from D5 Horizon proof stratification; J.DepSound is limited to semantic edges and new Lemma J.HorizonOrderSafe validates D5-only commutation. (5) Core.CycleClosure C2 now includes finite partial-cycle extension, exported by new Lemma Core.CycleChoiceExt and consumed by K.2b.E’s fair continuation construction. (6) K.2b.R’s Policy-S premise is corrected from commitment before q is “reached” to commitment before q’s request may issue. (7) J.RefProjCover/J.1-Safe consistently use finite canonical ideals/down-closed prefixes. (8) the Appendix K finite-drain note now cites Core.16(6)/K.Drain as the actual finiteness premise rather than appearing to derive finiteness from capacity alone. M.7a, K-P mechanization obligations, and Appendices S/T are aligned to these interfaces.

14 August 2026 — Core.17 Σ Match, persistent-deadlock scope, and QSync adequacy repairs.

Definition Core.17 now includes the canonical Σ -event/value occurrence matching as an explicit Σ Match conjunct of the normative observable-compatibility relation, and Appendix G is identified as an expansion rather than a redefinition of that convention. Appendix K’s persistent-deadlock implication note is narrowed to the closed internal wait-cycle class actually captured by Dead_K and explicitly excludes starvation/perpetual under-supply cases such as K.CE-5/H2 from the broader “never hangs permanently” reading. Definition Core.QSync QS3 now includes a transition-relation-to-settling-network realization/adequacy condition, and the QSync-Ref proof sketch and Appendix M.7a evidence checklist are aligned to that condition. No M.7a premise-entry expansion is made because the current revision already contains B1, B2/B3, B4/FPD, B5/B6-Terminal, A6, A8/A9/A10, K.BF, and Core.IssueFair audit entries.

14 August 2026 — K.CV audit-trigger and provenance-closure cleanup.

Appendix M.7a now separates the unconditional serial-route K.CVPred/K.CV-WF trigger from the additional design-level checks for cross-process observations beyond blocking Pop transfer values, so a pure blocking Push/Pop Policy-S route may not mark the P1 provenance obligation inapplicable. Definition K.CVPred now states predecessor occurrence closure explicitly and K-O0 checks it. Lemma K.2b.R now attributes dynamic identity, payload, and typed boundary/direction to K.2b.P1/K.CV while deriving reset-epoch agreement from K.DomResetScope and fixed DomEpoch. No Appendix J finite-prefix or serial-dominance architecture is otherwise changed.

14 August 2026 — K.2b.P1 well-founded value/identity provenance repair (H2 closed).

Lemma K.2b.P1 no longer invokes an undefined “causal order of commits.” New Definition K.CVPred/K.CV-WF gives the K.CV serial fragment a common semantic fact-key/provenance relation with deterministic per-fact evaluators and an explicit WellFounded certificate or checked rank. P1 now proves the strengthened all-facts determinacy claim by well-founded induction on $\prec_{CV}$. Same-cycle rendezvous enablement and the broader Appendix-G causal-dependency graph $\rightarrow$ are explicitly not predecessor edges merely by causal or temporal proximity, so legal cycles in $\rightarrow$ do not undermine the proof. K.BF, K-O0 and the K-P mechanization obligations, CF-S, K.PSF, M.7a K.CV, Appendix R, Appendix S, and Appendix T are aligned. No Appendix J construction order is imported into K.2b.P1, and finite-prefix Post $\rightarrow$ Ref is unchanged.

14 August 2026 — M.6 reset-scope disclosure and DomResetScope certificate typing cleanup.

M.6 now states the actual K.ResetEpoch applicability rule: Continue excludes every RESET whose RW scope intersects the candidate’s K.DomResetScope, RW5 permits disjoint resets, and an intersecting RESET that does not satisfy the independently mapped ResetEpoch.DirectFail conditions lies outside the serial route and requires another A5-L discharge. Definition K.DomResetScope is retyped as a certificate-supplied finite process/boundary scope with explicit D1–D4 coverage, endpoint-closure, and anti-circularity conditions; larger conservative scopes are permitted and minimality is not required. K-O4, CF-S, M.7a, and Appendix T are aligned. No finite-prefix Appendix J result is changed.

14 August 2026 — ReachPrep coverage-direction and rendezvous-example clarification.

Core.ReachPrep now states the Post $\rightarrow$ Ref coverage direction explicitly: every covered RTL_B prefix must have selected Sys_ref ReachPrep semantics permissive enough for the dependency-independent preparation/reach segments required by its accepted J.LocalGWR or J.GWR construction, while more-permissive source preparation remains conservative. The Appendix K blocking-rendezvous A5-L example is named as a straight-line ReachPrep-dependent example, states that its Push operands are independent of the preceding Pop results, and makes the eager Policy-L alternative conditional on the selected instance admitting that ReachPrep case. Appendix M.7a carries the same coverage and example-dependency obligations.

14 August 2026 — K.BF gate exhaustiveness and dominance-reset-scope cleanup.

K.InterruptionGate now states explicitly that, for the K.BF serial-reduction fragment, Core.IC and Lemma K.BF-Persist make Reset and Terminal the complete non-commit interruption-gate instantiations; any third gate is a theorem-model extension requiring re-proof. New Definition K.DomResetScope binds the processes and explicit boundaries whose dynamic transfer identities can participate in K.2b.R/P, with DomEpoch denoting their reset-epoch vector. K.ResetEpoch.Continue now excludes RESETs intersecting that dominance scope while RW5-framed disjoint resets remain permitted. K.2b.R and K.2b.P are correspondingly scoped to the fixed DomEpoch, eliminating the former use of K.ResetEpoch.Continue to forbid resets of $P \notin D$ that the old gate definition did not cover. K-O3/K-O4/K-O7, K.CertHdr, CF-S, K.PSF, M.7a, Appendices R/S/T, and Corollary K.2c are aligned. Core.CycleState is also added to the Appendix R and Appendix S Core-interface inventories. No finite-prefix Appendix J result or step-0–2 semantic interface is changed.

14 August 2026 — shared nested interruption-gate schema (step 3 complete).

Definition K.InterruptionGate now factors the common Policy-S comparison-interruption machinery into a prefix-local Continue branch and an independently mapped DirectFail branch with a single anti-circularity rule. K.ResetEpoch and K.TerminalDisposition are retained as gate-specific instantiations rather than parallel ad hoc definitions: reset is evaluated first; only K.ResetEpoch.Continue permits evaluation of the terminal gate; and only the two nested Continue branches enter K.2b.E/R/P/Q. ResetEpoch.DirectFail remains the matched-RESET/RW2 cancelled-pending route, while TerminalDisposition.DirectFail remains the DeclaredTerminal maximal-pending route. Lemma K.2b/K.2c, K-O4/K-O6/K-O7, K.CertHdr, CF-S, K.PSF, Theorem K.1, Appendix M.7a, Appendix R, Appendix S, and

Appendix T are aligned. Steps 0–2 are unchanged, and no flat interruption enum or new third cancellation gate is introduced.

14 August 2026 — full-state CycleResult and cycle-closure consolidation (step 2 complete).

Definition Core.CycleResult is introduced as the primary typed real-cycle object. Its nextState is the full theorem-instance state, including registered/internal machine state, time-aware environment state, global cycle index, and cycle-valued auxiliary/monitor state; committedUnits records selected ordinary complete observable units / Σ commits and resetUnits records each dynamic scoped RESET occurrence. Core.CompleteCycle is retained as the nextState projection of CycleResult for compatibility with existing theorem statements, and Core.SilentCycle is the no-selected- Σ -commit case; a reset-only cycle may therefore be silent at the selected outer alphabet while still carrying the proof-internal RESET occurrence. DeclaredTerminal remains a predicate on states rather than a CycleResult outcome. RESET is explicitly an emitted in-cycle unit, not a sum-type branch: RW4/RW5 and Lemma J.RS are aligned so partial resets preserve unaffected-scope continuation and the J induction remains indexed by the RESET unit itself. The former standalone environment-totality and source-cycle-closure premises are folded into the normative Core.CycleClosure record together with full-state step typing, source-cycle existence/finiteness, environment-step definedness, auxiliary/monitor advancement, and reset-emission fidelity. K.2b.E/K.2b, K-O7, M.7a, K.CertHdr/CF-S/K.CompleteS, Appendix R, Appendix S, and Appendix T are aligned. Step 3 interruption-gate unification is not included.

14 August 2026 — finite-prefix Reach interface and generic Exec/J.FE removal (step 1 complete).

Appendices I/J now use Reach_post/Reach_ref as the sole generic finite-prefix execution domain. The restricted Ref $\rightarrow$ Post clause and Definition J.RTLConsistentRefPrefix are retyped over annotated reachable prefixes; because the supplied τ_post^* is already a reachable RTL witness, the reverse proof still takes $\tau_post := \tau_post^*$ without any fair-extension argument. The generic Exec_post/Exec_ref definitions and Lemma J.FE are removed. Core.PostCycleTotal is deleted after the dependency audit found no remaining semantic consumer; Core.CycleClosure is narrowed to the source complete-cycle/environment closure used by Core.LAdm/K.2b.E. Appendix I.5 is explicitly local/finite and makes no global infinite-execution existence claim. Appendix G, M.7a, Appendix R, Appendix T, B6 explanatory text, Core.IssueFair/B2-B3 scope, and the J.1 proof prose are aligned. Complete/fair execution obligations remain only where later liveness results explicitly require them, including Exec_L^adm/K.2b.E and K.CompleteS. No CycleResult or interruption-gate unification refactor is included in this revision.

14 August 2026 — blocking-request lifecycle closure and frozen-frontier interruption-completeness repair (step 0 complete).

Axiom Core.IC now states an exhaustive ordinary blocking-request lifecycle for the G–K theorem model: after issue, an outstanding blocking dynamic instance remains asserted and stably attributed until its process-facing commit or an affecting RW2 RESET; no other nonterminal transition may deassert, retire, reattribute, cancel, or replace it, and DeclaredTerminal may instead end the execution with the request still pending. Core.SilentCycle, the Appendix C/G/K/N no-deassert-before-commit statements, and Core.IssueFair are aligned to that reset-epoch-scoped rule. Definition K.BF and Lemma K.BF-Persist now consume the Core lifecycle rather than introducing a separate cancellation exception; K.BF-Persist therefore closes the frozen-frontier interruption enumeration used by Lemma K.2b.E/K.2b.R. Definition K.RespFail retains cancelled-pending only for an affecting RW2 RESET; terminating abort/end-of-test behavior is represented by DeclaredTerminal and uses maximal-pending. K.TerminalDisposition clause (2), K.2b.Q Step 3, the K.2b proof, K-O4/K-O6/K-O7, CF-S, K.PSF, Appendix M.7a, Appendix R, and Appendix T are aligned. The K.2b.R heading-format regression is repaired. Protocols with independent nonterminal cancellation/withdrawal of outstanding blocking requests require a separately defined operational extension and are outside the present G–K theorem stack. No J.FE/CycleResult/gate-unification refactor is included in this revision.

14 August 2026 — K.ResetEpoch prefix-local applicability repair.

K.ResetEpoch clause (1) is revised from a universal property over already-frozen complete extensions to a non-vacuous prefix/state-local reset-contract condition: at every finite admissible comparison prefix while all selected frozen requests remain pending in the same epoch, no affecting RESET is permitted as the next transition, so RW2 cannot clear those requests before the gate applies. K.TerminalDisposition clause (1) is put in the parallel prefix/state-local form by requiring every such frozen-prefix final state to be non-DeclaredTerminal. Lemma K.2b.R, the proof of K.2b.E, K.2b.Q Step 1, K-O4/K-O7, and the Appendix M.7a audit entries are aligned. K.2b.E continues to construct the infinite fair continuation from machine-closure, drain, fairness, and issue-fairness premises; neither applicability gate assumes that continuation exists. All previously recorded H1, terminal-branch, watchdog-scope, and finite-prefix Post → Ref conclusions are unchanged.

14 August 2026 — K.2b declared-terminal applicability, termination-contract scope, and audit-consistency repair.

Definition K.TerminalDisposition now separates the reset-free serial route into a terminal-free dominance branch and an independently certified Policy-S terminal/cancellation failure branch, parallel to K.ResetEpoch. Lemma K.2b.E/K.2b.R/K.2b.P/K.2b.Q and their Appendix K-P proofs use only the infinite-fair continuation on the terminal-free branch; a declared terminal no longer substitutes for old finite-maximal reasoning. The direct terminal branch must independently bind the liberal frontier to a non-waived Policy-S response monitor and prove the actual Policy-S terminal/cancellation disposition while that monitor remains pending, avoiding circular dependence on K.2b.P/Q. The clause-(1) branches of both K.ResetEpoch and K.TerminalDisposition are stated as contract-level applicability properties rather than referring to a continuation constructed by K.2b.E; the later prefix-local repair below makes their mechanizable finite-prefix form precise. DeclaredTerminal scope is clarified: a modeled cycle-count/watchdog/end-of-test disposition inside the selected machine/test semantics counts, whereas an external harness watchdog that merely stops host simulation does not truncate Core.CompleteCycle or defeat the terminal-free gate. CF-S and M.7a identify clause (1) as the normal prospective clean-run applicability evidence and clause (2) as an independently certified direct-failure branch that may be conditional or instantiated but is not required as an observed witness on every clean run. The remaining Policy-S 'maximal finite' paraphrases are aligned to the normative declared-maximal-terminal wording. Corollary K.2c, K.PSF, Theorem K.1, CF-S, K-O3–K-O7, Appendix M.6/M.7a, Appendix R, and Appendix T remain aligned. The legacy Core.PolicyL returned-result sentence is synchronized with Core.ReachPrep, the stale J.FE environment-receptiveness name is removed, Core.CycleClosure receives an explicit Core definition, and M.7a includes B1, B4/FPD, A6, A8/A9/A10, K.BF, and K.TerminalDisposition audit entries. Finite-prefix Post → Ref remains unchanged.

14 August 2026 — fair-extension, complete-cycle, and source-issue formalization repair.

The machine-closure path has been revised to separate same-cycle settling, ordinary silent clock cycles, and terminal B6 stutter. Definition Core.ReachPrep now makes the intended Policy-L eager-reach semantics explicit, including dependency-independent ordinary straight-line successors used by K.CE/K.CE-2; Definition Core.IssueFair gives the previously unnamed source-issue weak-fairness predicate; and new Core.CompleteCycle/Core.SilentCycle, EnvCycleTotal/EnvTerminal, SourceCycleClosure, and PostCycleTotal definitions type cross-cycle progression. Lemma J.FE is rewritten as a phase-structured source construction plus a distinct RTL totality/progress discharge, so a presently idle state is no longer inferred to be a B5 fixed point and B6 is reserved for Core.TerminalIdle, which requires both machine-side and environment-side permanent no-future-enabling evidence. Core.LAdm, K.2b.E, K-O1/K-O3/K-O6/K-O7, K.Resp maximality/aging, Appendix N, and Appendix M.7a are aligned. The generic J.FE route is explicitly separated from Appendix K's stronger Exec_L^adm continuation route. These changes do not alter the finite-prefix Post → Ref safety construction.

13 August 2026 — Appendix Q compositional source-QSync repair.

Appendix Q.1(i) now distinguishes reusable elaboration-template settling facts from the whole-source Core.QSync discharge. Template-side finite/deterministic/acyclic settling of staged/bundle/join/guard/coupler/epoch-gate logic no longer implies whole-source QS2–QS5 by itself. For $\text{Sys_B}^{\text{elab}}(E)$, the concrete theorem instance must also supply the design-owned Core.RegSeq/direct-input/R2C/non-template-boundary settling facts and a checked template/design settling-composition condition covering every cross-partition dependency; QS4 is applied to the complete combined same-cycle settling graph. Appendix M.7a now names this composition obligation and lists a combined dependency-rank/topological-order certificate as a sufficient mechanically checkable discharge. The Core.QSync-Ref proof note and $\text{Sys_B}^{\text{elab}}$ audit entry are aligned accordingly. This is a qualification/discharge-interface correction only: Definition Core.QSync, QS1–QS5, Theorems Core.QSync-Ref/Core.QSync-Post, J.1 scope, and H3 are unchanged.

13 August 2026 — J.TraceCertCov source-settling propagation.

Definition J.TraceCertCov and its Appendix M.7a discharge entry now state the route-sensitive source-settling evidence explicitly rather than leaving it only inside “remaining Post $\rightarrow$ Ref premises”: the normal J.LocalGWR/J.GWR-Comp route carries QualifiedSynchronousModel_Sys_ref(I)/Core.QSync-Ref, while a direct J.GWR package may instead carry checked finite L2-settled segments for its represented Horizons. Corollary J.1-Safe and the abbreviated front-matter safety-signoff prose now point to that same route-sensitive requirement. This is a propagation/qualification clarification only; it adds no unconditional post-side Core.QSync premise to plain finite-prefix J.1 and does not change H3.

13 August 2026 — qualified synchronous hardware interpretation.

An informative hardware-interpretation paragraph now precedes Definition Core.QSync. It states the intended single-clock cycle-level model directly: registered state and cycle inputs are fixed, sequential process outputs are derived from registered/fixed operands without same-cycle interface-result through-paths, the complete same-cycle combinational/handshake network settles deterministically with pure combinational feedback outside the standard scope, and boundary actions precede the next registered update. It also clarifies that SettlePoint, BoundaryEvalPoint, and Core.KCut are proof views of the ordinary settled cycle snapshot rather than additional hardware states, while preserving explicit latency-sensitive/WC and Policy-L nondeterminism.

13 August 2026 — Stage A terminology, L2 frame repair, and qualified synchronous-settling Stage B.

Stage A adds an explicit determinism taxonomy and separates δ -settling, real silent τ -cycles, and B6 idle ticks without redefining the general ϵ relation. Definition Core.Settle now states the normative L2 frame: cycle index, registered process/FSM/pipeline and explicit-channel storage/occupancy/credit state, and selected cycle-t exogenous/environment values are frozen during $\rightarrow\epsilon, L2$; no Σ commit or next-cycle registered update occurs. SettlePoint existence is no longer implicit in the definition. Stage B adds Definition Core.QSync and Theorems Core.QSync-Ref/Core.QSync-Post. The standard K-facing/cutpoint scope now requires finite acyclic deterministic same-cycle settling over explicit state boundaries; QSync-Ref proves source/ $\text{Sys_B}^{\text{elab}}$ SettlePoint/KCut existence and uniqueness, and QSync-Post derives Core.KCutClosure_post plus a canonical K-facing NormPost endpoint. Appendix Q adds the matching elaboration obligation; Appendix M.7a adds a first-class Core.QSync audit entry and treats Core.KCutClosure_post as theorem-derived in the standard scope; K.CertHdr records the structural evidence, theorem provenance, and any trusted-base delta when a settling claim is accepted without independent artifact checking. The normal compositional finite-prefix J.1 route now explicitly consumes source-side QSync-Ref settling existence through J.RegPhaseNF/J.UnitExt, while a directly validated J.GWR package may carry equivalent per-Horizon settling evidence; plain J.1 does not thereby acquire post-side QSync unless a post KCut/NormPost interface is used. This revision intentionally narrows the standard Core.KCutClosure_post discharge scope: the former independent inductive-invariant and checked-normalization alternatives are withdrawn as standard-scope routes and require an explicitly qualified non-QSync theorem extension. QS1 now states the single-clock semantic condition directly,

with M.7a ClockScope serving as its evidence entry. Appendix I/J replay normalization remains separate. No change is made to Appendix H.3 in this revision.

13 August 2026 — Lean/scheduling alignment clarifications.

The post-side interpretation of Core.SyncFence/E5 now states explicitly that same-cycle predecessor/synchronization accounting is a trace/certificate attribution convention and does not impose an observable physical sub-cycle ordering within one RTL clock edge. Appendix M.7a adds named ClockScope and Core.SyncFence discharge entries, and K.CertPkg records their references where applicable. Theorem J.1's restricted Ref $\rightarrow$ Post clause now exhibits its existential witness explicitly as $\tau_{\text{post}} := \tau_{\text{post}}^*$. Lemma K.2b and K.2b.E now state that drain-closedness is obtained from the Dead_K / DeadKRec^W hypothesis itself rather than imposed as an additional narrowing of A5-L's source-KCut domain. Definition L.1 remains intentionally stated over generic ϵ -quiescent source witness-choice cutpoints rather than K-facing Core.KCut points.

12 August 2026 — synchronization-completion fence formalization (H4).

The previously prose-only rule that a synchronization call cannot complete while an earlier blocking Push/Pop in its source interval remains uncommitted is promoted into the formal contract. New Axiom Core.SyncFence gates source synchronization commits, including same-cycle message/synchronization completion and Sys_B^{elab} process-facing commit semantics; Core.Phase L3 and the operational transition relation consume the fence. R3 and E5 now contain explicit blocking-commit-before-sync clauses rather than relying on a parenthetical appeal to blocking-call control flow. Appendix C, Core.14, Core.18e, the drain-closed definition, the compact E1–E5 table, the system-level E5 proof, and Lemma J.KObs-Offers are aligned. This makes the interval scoping of Pending_P/Front_P/drain candidates lossless and closes the residual-offer versus PendingRec reconstruction seam without changing the intended deadlock predicate.

12 August 2026 — L2 settle / global ϵ -quiescence, KCut closure, and K-cutpoint-domain correction.

Core.4 defines model-specific global ϵ -quiescence; Core.Settle defines the complete L2 phase-restricted ϵ relation and BoundaryEvalPoint; Remark Core.1a classifies phase advancement as meta-level rather than ϵ/Σ activity; and Lemma Core.SettlesKCut proves that every phase-bearing source L2 SettlePoint is a source KCut. Core.Phase L2/Core.16 use SettlePoint for activation, while WFG, KObs, Dead_K, Corollary J.1, and K.1a use the Core.KCut comparison domain. Core.SettleKBridge remains the named activation-to-later-drain-closed-KCut obligation and is consumed by K.DrainReach/Core.LAdm. RTL_B retains its own Quiescent/NormPost/KCut_post semantics without importing L0–L4, while Core.KCutClosure_post plus Core.KCutObserve supply the non-vacuity/observability obligation ensuring that every RTL K-facing cycle-evaluation origin reaches a finite KCut and that persistent relevant blockers cannot remain forever outside that domain. K.CertCov remains certificate totality over KCutReach_post and is paired with KCutClosure_post for universal conclusions. Stale Core-table, Appendix I.2, K ϵ , and K.Low.C cutpoint terminology is aligned; Appendix T, M.7a, the front guarantee summary, and the certificate header are updated accordingly. B5 remains the stronger global fixed-point condition, not a synonym for Core.4 quiescence.

1 August 2026 — logical replay-checkpoint clarification and B4/B5 unit correction.

Definition I.ReplayObs now denotes the logical replay checkpoint represented at an operational state, not the raw post-preparation cursor/store. Replay-inert L1 preparation is separated from witness-fixed epsilon transitions that may update replay fields; normalized I.ReplayObs-Congruence and I.DR now use paired congruence of exact selected normalization paths rather than per-path neutrality or global confluence. I.CutpointOfferObs, I.DR', and Appendix T are aligned. B4/FPD use the per-process epsilon-step bound D_P; B5 uses a separate cycle bound C_sys, optionally derived through per-process C_P charging lemmas. Lemma I.1 and A2 likewise keep the units distinct. Appendix Q records the equivalence of channel-first A_E(c, σ) and state-first A_E(σ , c) notation.

31 July 2026 — replay/offer separation, R2C cycle-lock completeness, and B3 conclusion clarification.

Appendix I now separates the replay-derived process slice `I.ReplayObs` from the issue-schedule-dependent current blocking-offer slice `I.CutpointOfferObs`. Lemmas `I.ReplayObs-Congruence`, `I.DR`, and `I.DR'` no longer infer `Pending_P`, `Front_P`, or `BoundaryReq` from committed observable-unit history; Appendix J continues to construct and audit those facts through `O`, `J.OfferReach`, `J.OfferConf`, and `J.KObs-Offers`. `Core.18d`, Appendix R, and Appendix T are aligned to the split. `R2C` now defines `ComponentParticipatesInCycle` and requires one emitted fixed-point unit for every participating cycle of a cycle-locked component, while forbidding units outside the participation domain. `B3` now states explicitly that its buffered `P_push/P_pop` clauses conclude the complementary discharge; `E4` plus `B2` govern eventual commit of any original request that remains pending and continuously enabled afterward.

31 July 2026 — ordinary-source frontier-uniqueness lemma.

Lemma `Core.OrdFrontSingleton` now derives the at-most-one-member property of `Front_P` for ordinary non-elaborated source intervals from the dynamic-program-order convention of Clarification `Core.5a` and the one-call/one-frontier-item ordinary-source mapping. Appendix B, Definition `Core.15`, and Clarification `Core.WFGb` now cite the lemma explicitly.

26 July 2026 — reset-branch mapping and multi-frontier legality tightening.

`K.ResetEpoch` clause (2) now supplies an independent, certificate-bound mapping from a selected liberal component frontier to a process-facing `Policy-S` serial frontier and binds the exact pending monitor and `RW2` clearing event, rather than relying on the reset-free `K.2b.Q` mapping. `K.2b.R` now states `Core.16` issue legality against the full frozen frontier set and tracks the `StandardEnv` multi-frontier case through `K-O2/K-O4`. The Formal Guarantees at a Glance section, the dynamic-reset main-body guidance, Appendix M.6/M.7a, CF-S, and Appendix T now disclose and name the dynamic-reset applicability condition consistently.

26 July 2026 — serial-reduction seam and front-matter scope clarification.

`K.2b.R` now proves explicitly that `Core.16` does not block the reached serial-prefix request, removes the redundant independent-work disjunct, and scopes request persistence to one reset epoch. `K.2b.P` now states its elaborated-boundary coverage, explains why the same-cycle tie-break is harmless under begin-of-cycle non-fall-through `E4` semantics, and includes an explicit composition note with `K.2b.E`. Definition `K.ResetEpoch` separates the reset-free dominance branch from an already-triggered matched-reset cancelled-pending branch; pre-reset cutpoints that satisfy neither branch are explicitly outside the `Policy-S` reduction. `K.4b`, `K-P`, CF-S, `K.PSF`, Theorem `K.1`, Appendix M.7a, and Appendix T are aligned. The four-item Formal Guarantees at a Glance section now restores a compact scope exclusion, instance-qualification warning, and direct pointer to Appendix M.7 without reintroducing proof-interface labels or the detailed table.

26 July 2026 — K.2b proof completion and user-facing responsibility/status clarification.

Appendix K-P now contains explicit proofs of Lemma `K.2b.R` serial-prefix request reachability and Lemma `K.2b.P` deterministic serial progress dominance, including the least-missing-transfer argument and separate buffered-Pop, buffered-Push, and rendezvous cases. `K-O3` now tracks `K.2b.P` mechanization and `K-O4` tracks `K.2b.R`; `K.4b` and Appendix T are aligned. The Formal Guarantees at a Glance section has been reduced to four user-level statements and proof-interface labels have been removed from that section. The detailed responsibility/current-status table has been moved to Appendix M.7. Appendix M.6 continues to distinguish the design user's complete instrumented `Policy-S` execution from the full theorem discharge and to identify valid universal-coverage routes without claiming that a generic construction already exists.

25 July 2026 — WFG, elaboration-map, and drain-interface completion.

Definition `Core.WFG` now restores the edge-versus-blocked-process and explicit multi-frontier out-degree clarifications from the former Appendix K definition, while continuing to cover mixed buffered/rendezvous graphs. Definition `Core.ElabProj` promotes the four-role `$\Pi$ _elab` interface—trace

projection, occupancy projection, conservation/accounting, and proof-internal WFG visibility—into the Core; Appendix Q now normatively instantiates that interface. Definition Core.Drain removes the remaining Core-to-K definitional reference, and K.Drain is stated as the application evidence obligation that discharges it. Core.6, Core.7, Core.15, Clarification Core.5a, and Core.PolicyS now have navigable formal headings; Remark K.0-pre restores the derived-bridge framing. Definition Core.PolicyL now gives the liberal issue policy a citable heading parallel to Core.PolicyS. The K.Drain and Appendix Q normative run-in labels are restored to bold, and the A9–A11 assumption paragraph has normalized run boundaries so that only the assumption labels are emphasized.

25 July 2026 — formal-core split attribution and side-condition organization cleanup.

Remark Core.1a and Appendix T now attribute Policy-L step legality, cycle-phase semantics, and admissibility to Core.LLegal, Core.Phase, and Core.LAdm respectively. J.Mem and RESET/RW1–RW5 now appear under a dedicated Core subsection, and the RESET label is restored as a bold run-in. The normative wait-for-graph definition is renumbered Core.15a to preserve ascending definition order at its logical location after Core.15, and Definition Core.17 now refers to the Core rather than to an appendix. The active historical-name lookup maps old R.20 to Core.15a; older preserved revision text is otherwise unchanged.

25 July 2026 — formal-core dependency audit follow-up.

The follow-up audit removes the remaining $J \rightarrow K$ proof citation by limiting Corollary J.1 to equal-KObs and J-owned reconstruction consequences; Appendix K now performs Dead_K and waiver/diagnostic factoring. Appendix K imports Definition Core.15a for WFG instead of restating a second normative-looking graph definition. The Core KObs layer now exports only Core-owned reconstruction functions; Definition K.EnvTerminalAdequate and Definition K.KObsDerived, together with K.StimX and K.WFG+, now reside in Appendix K before the factoring lemmas. The KObs architecture explanation has moved to downstream Appendix R, the side-condition discharge record to Appendix M.7a, and Lemma J.RS to Appendix J. Core section headings are now descriptive rather than reusing definition numbers, the WFG definition fills the former Core.15a gap, the reading paths follow document/dependency order, and the Core scope text distinguishes Appendix G’s raw Sys presentation from the Sys_ref comparison used by Appendices I–K.

Historical-name lookup for preserved earlier revision entries: unlabelled Policy L $\rightarrow$ Core.PolicyL; R.RegSeq $\rightarrow$ Core.RegSeq; R.LAdm $\rightarrow$ Core.LLegal/Core.Phase/Core.LAdm; R.16 $\rightarrow$ Core.16/Core.Drain; R.18e $\rightarrow$ Core.18e; R.20 and the interim Core.15a label $\rightarrow$ Core.WFG; former Definition R.1a $\rightarrow$ Core.ElabProj; R.21b–R.21e $\rightarrow$ J.KObs-Proc/J.KObs-Chan/J.KObs-Offers/J.KObs-WFG; and R.21f–R.21g $\rightarrow$ K.KObs-Dead/K.KObs-Ext. Core.15a is a retired/interim label: occurrences in preserved revision-history text describe the name in force at that time and shall not be used as current normative cross-references; the current primary name is Core.WFG.

25 July 2026 — normative formal-core dependency reorder.

The former “Common Formal Definitions for Appendices I–K” has been promoted and moved immediately before Appendix G as the “Normative Formal Core for Appendices G–K.” The operational Sys_ref/Policy L/Policy S material, Core.RegSeq, Core.LAdm, the exact Core.16 drain discipline, the shared frontier/request definitions including Core.7a BlockingMsgNextFront, the normative Core.15a WFG definition, and the Core.18c KObs datatype/reconstruction functions now occur before first use. Appendix J now contains the source-side KObs reconstruction lemmas J.KObs-Proc/J.KObs-Chan/J.KObs-Offers/J.KObs-WFG; Appendix K contains K.KObs-Dead and K.KObs-Ext. Appendix R has been retained as a downstream compact/mechanization companion with an explicit deletion-safety rule and no normative upstream dependencies. Active cross-references have been redirected from the former late R-numbered definitions and lemmas to their new Core/J/K locations; historical revision entries are preserved unchanged. The dependency audit now has no Core/J forward reference to Appendix K for WFG, and Appendix R remains deletion-safe.

24 July 2026 — Policy-S user workflow and status clarification.

The Abstract, Introduction, Formal Guarantees at a Glance, Guide to the Formal Appendices, and Appendix M.6 now present the deterministic Policy-S route in user-workflow order: one selected, complete instrumented pre-HLS execution is the primary design-user activity, while the flow separately establishes SDet, applicability, completeness, total finite-bound response coverage, and certificate validity. The text now distinguishes the operational practicality of simulation monitors from the outstanding mechanization and qualification of the K-P reduction and certificate interfaces, and it states separately that K.CertCov is required for a universal reachable-RTL-cutpoint conclusion. The underlying theorem premises and conclusions are unchanged.

24 July 2026 — formal cross-reference, deadlock naming, snooping, and K.2b.Q cleanup.

The stale clause-based Corollary J.1 signal-grouping reference has been redirected to the J.RegPhaseNF/J.UnitExt unit treatment. Appendix K now names the ordinary deadlock predicate Definition K.Dead and uses Dead_K/Dead_K^W consistently instead of overloading “K.1” as a predicate or component name; the phantom Definition K.Dead citation is thereby made valid. The J.Mem extension note no longer reuses invariant clause number I4. Appendix G now states snooping alignment conditionally on existence of a WC/L.Dir witness and points failed realization to Appendix L triage. K.2b.Q Step 1 now limits its source-prefix commitment claim to the process-facing blocking Sys_K/K.BF serial prefix and explicitly excludes failed non-blocking polls. Appendix C now contains the normative ReturnedAtCycle definition directly rather than a dangling main-body cross-reference, and Definition R.18e cites both K.Dead and K.DeadW for the ordinary and waiver-parameterized reconstructions.

24 July 2026 — main-body formal-detail cleanup.

The pre-appendix scheduling discussion has been simplified without changing the formal contract. The Basic Conceptual Model now explains overlapping issue without proof-internal multi-frontier or elaboration terminology; the R1/E1 and R3/E5 rationale has been removed from the pipelined-loop section; and the synchronization-boundary subsection now gives only the operational drain/next-interval rule with an Appendix B reference. ReturnedAtCycle and the detailed registered sequential-interface semantics have been removed from the main body and replaced by one plain-language registered-cycle rule, with the normative ReturnedAtCycle definition relocated to Appendix C and the dependent-use rule retained in R.RegSeq. The two formal-architecture bullets at the end of the main Summary have been removed; their unique Policy-S coverage content and a concise statement of proof-layer separation are consolidated in Appendix M.

24 July 2026 — introductory restructuring and formal-results navigation revision.

The former theorem-by-theorem opening discussion has been replaced by a short front abstract, a practical Introduction, and a separate Formal Guarantees at a Glance section. Broad claims concerning RTL verification cost, customer experience, implementation coverage, and source-level signoff have been qualified to avoid implying unconditional tool or RTL certification. A new unlettered Guide to the Formal Appendices has been inserted immediately before Appendix G without changing any existing appendix identifier. Appendix M has been rewritten as Formal Results Overview, preserving the precise Post → Ref safety direction, restricted Ref → Post use, local-to-global J.GWR-Comp architecture, KObs/K.RLAdmReach cutpoint bridge, certified-cutpoint versus total-coverage distinction, Policy-S/A5-S serial-reduction route, applicability limits, and implementation-evidence obligations.

18 July 2026 — M1 safety-transfer direction and common-case pre-HLS signoff clarification.

The document now states explicitly that ordinary source-side safety transfer uses Theorem J.1 Post → Ref: RTL_B introduces no new canonical Σ_{DUT} safety behavior absent from Sys_ref. Definition J.RefProjCover names the optional observable-prefix coverage fact under which one complete pre-HLS execution represents all covered source-reference observations, and Corollary J.1-Safe packages both the universal-source and one-run transfer cases. The Appendix G “No Surprises” discussion and Appendix L Note 1 cross-reference this corollary. The restricted Ref → Post clause remains an

annotation-dependent selected-behavior realization clause and is not presented as the common safety-signoff mechanism. The former “source-side pass only through the restricted reverse direction” wording in the Practical Implication section is replaced by this safety-versus-realization distinction.

18 July 2026 — H4 eager-issue completeness boundary and H5 environment/frontier scope correction.

H4 is made explicit: because Policy-S-shaped prefixes are Policy-L-admissible, the two-channel crossing example contains an admissible two-Pop deadlock prefix and therefore falsifies A5-L for the unchanged design. The misleading suggestion that another A5-L proof route could rescue that instance is removed; alternate proof routes remain available only when A5-L is true. H5 is closed by the named applicability condition K.EnvMF. Certified multi-frontier components are supported by K.2b.E/K.2b/K.2c only under StandardEnv/B1-avail; timed/reactive or otherwise nondefault environments are restricted to a certified single frontier consisting solely of the selected internal blocker. K.EnvX-Auto, A5-S/CF-S, K.PSF, Theorem K.1, K-O2/K-O7, the Appendix K-P proof, Appendix T, the side-condition record, and the summaries are aligned. No general future-environment no-rescue certificate is introduced.

18 July 2026 — K.1A statement and interface-seam correction.

Theorem K.1A’s statement now explicitly restores A5-L, conditional KObsClosed waiver closure, the point-to-point evaluated-boundary premise, and the certified-cutpoint conclusion that its proof already consumed. K.CertCov now quantifies over an explicit reachable prefix/cutpoint pair. ReturnedAtCycle has one normative main-text definition and an Appendix C pointer. Definition J.RegPhaseReach and Corollary J.RegPhaseReach-Comp are separate citable anchors. R.RegSeq requires checked identification before rejection or theorem-scope exclusion of zero-time dependent NB cascades. The historical 17 July sequential-result-availability entry is restored verbatim; supersession remains recorded only in the later 18 July H3 entry.

18 July 2026 — consolidated H1/H2/H3 registered-phase revision.

H1 remains split into Definition K.CertifiedCutpoint/Theorem K.1A for one cutpoint and K.CertCov/Corollary K.1A-Total for universal coverage. H3 is reclassified as a formalization-fidelity defect: the intended registered sequential-interface semantics has no same-Horizon decision-to-dependent-request edge, but the former stop-at-NBHole replay forced decision-before-independent-continuation. Definition R.RegSeq and ReturnedAtCycle now cover successful and failed NB results, request and Push payload outputs, stable direct inputs, Cat#/source conformance, and later-Horizon dependent use. Lemma I.3 promotes NBHole to an enriched carried ghost obligation with result destinations and NoDependentUse. J.ConstructionAction adopts a normative field-sensitive location vocabulary; A3 is narrowed to the shared decision/result; Dep_J/FR are revised; J.RegPhaseNF proves the R.LAdm L1/L2/L3 normal form modulo J.HCanon using J.BufSameCycle; and J.UnitExt/J.GWR-Comp use outer Horizon induction with an inner L3 pass. No generic micro-position or new action taxonomy is introduced. For H2, Corollary J.RegPhaseReach-Comp now derives J.RegPhaseReach on the compositional route and supplies L0–L4; K.DrainReach remains the explicit nonwithdrawal/K ϵ /R.16/K.Drain bridge, and K.RLAdmReach is their conjunction. CF-0, K.CertifiedCutpoint, K.1A, the side-condition record, Appendix S/T, summaries, and the no-unresolved-hole cutpoint invariant are aligned. The separate Lean strategy must propagate the same interface change before mechanization.

18 July 2026 — H1 certificate coverage and H2 admissibility-interface revision.

The Appendix K preservation theorem was split into a per-cutpoint result and a coverage corollary. The consolidated H1/H2/H3 revision above retains that H1 architecture but refines H2: the L0–L4 portion is now J.RegPhaseReach, derived from the revised Appendix I/J construction, while K.DrainReach carries the remaining nonwithdrawal/K ϵ /R.16 evidence. Their conjunction is the K.RLAdmReach consumed by K.1A.

17 July 2026 — H2 bounded-response serial-route revision.

The primary K.2b/K.2c/CF-S route now uses validated process-facing bounded-response monitoring rather than the incomplete missing-supply owner trichotomy and Dead_KS terminal taxonomy. Definitions K.RespMon/K.RespCov/K.RespFail/K.RespClean require default-total internal-site proof coverage, process-facing trigger/response attribution, maximal-finite and cancellation handling, complete-run evidence, and waiver preservation. Lemma K.2b.E and mechanization obligation K-O7 explicitly construct the fair complete frozen Policy-L continuation from a bare A5-L counterexample prefix; K.2b.Q then proves that the liberal Dead_K cutpoint forces a finite Policy-S response failure, and K.2c converts K.RespClean into A5-L. The perpetual under-supplying-owner example is added explicitly. Dead_KS, EnvStop, DoneStop, K.EnvOrd, and K.Done remain optional diagnostics only. CF-S, K.PSF, Theorem K.1, Appendix R's KObs boundary, Appendix T, the side-condition record, and both the abstract and main-body Summary are aligned; no Policy-S response bound is transferred to RTL.

17 July 2026 — Policy-L admissibility and settle-window clarification.

Definition R.LAdm now supplies one normative execution domain for Appendix K. It distinguishes Policy-L step legality, finite-prefix admissibility for A5-L reachability, and complete admissible executions for fairness, stabilization, and limit arguments; binds the fixed stimulus/environment/witness/arbitration choices; and defines the L1 pre-settle issue window, the L2 ϵ -settle snapshot, the exact activation point of Definition R.16/K.Drain, and next-cycle availability of committed results. A5-L, K.1A, K.2b, K.Ser, Examples K.CE-2/K.CE-3/K.CE-4, Appendix R, and Appendix T now cite this single definition. The change resolves the ambiguity without changing which independent eager issues Policy L permits or making K.Drain part of the basic Policy-L issue rule.

17 July 2026 — sequential-result-availability clarification.

The Issue/Commit semantics now state explicitly that a result committed by a sequential process in cycle t may affect a dynamically succeeding dependent operation only in a strictly later cycle. The clarification is added to the main Issue definition, Appendix C linkage rules, the Sys_ref/Policy-L operational interpretation, Lemma I.3 and Corollary I.4, J.DepSound, J.FoundationRank, and the D1 case of Lemma J.DepRank. Same-cycle rendezvous, buffered-channel updates, eager issue, and pipeline overlap remain permitted for requests whose reachability, control, and operands were established independently before the cycle boundary.

15 July 2026 — J.GWR formal-ordering and SDet clarification revision.

The J.DepAcyclic proof now uses one graph-independent semantic foundation rank and the new Lemma J.DepRank; the prior union-of-local-orders argument is removed, and Height/J.CanonicalRank are defined only after acyclicity. J.RequiredPrefixCandidate removes the circular reference from J.PCert to the not-yet-derived canonical order. J.ConstructionOrderProvenance/J.ValidatedConstructionOrder provide one interface for theorem-derived and directly validated J.GWR packages. SDet now states functional history determinism as prefix-comparability/no-incompatible-history for run artifacts, while K.CompleteS separately establishes maximality, fairness, and completeness. Remaining legacy time-chain terminology is replaced by the derived or directly validated construction-order ideal chain.

15 July 2026 — J.GWR snapshot, same-cycle, and dependency-completeness refinement.

The local-to-global construction now derives its proof order from a generated typed dependency graph rather than accepting J.UnitRank/phase metadata. Definition J.ConstructionAction introduces explicit ReadSet/WriteSet footprints; J.DepSound and J.DepComplete prove prerequisite soundness and non-commutation coverage; J.FoundationRank/J.DepRank prove every edge advances one graph-independent semantic rank; J.DepAcyclic follows; and J.CanonicalRank is reconstructed only afterward. Lemma J.BufSameCycle proves legal sequentialization of simultaneous positive-capacity Push/Pop commits without permitting empty/full fall-through. Definition J.ReqSnapshot and Lemma J.NBDecision move witness-significant non-blocking outcomes out of process-local replay paths and evaluate them from one common attributed boundary snapshot. Lemma I.3, J.PCert/J.CCet/J.LocalGWR,

J.PrepareCommute/J.UnitEnable/J.UnitExt, the expanded J.GWR-Comp proof, the side-condition record, and Appendices S/T are updated accordingly.

15 July 2026 — J.GWR compositional-construction revision.

Definition J.GWR remains the stable semantic interface but is no longer a mandatory unexplained Post $\rightarrow$ Ref premise. J.LocalGWR supplies the local certificates, and Lemma I.3 supplies finite process-owned preparation scripts with shared-decision holes. J.Frame/J.LocalLift/J.PrepareCommute/J.UnitEnable/J.UnitExt provide generic gluing; J.GWR-Comp constructs a reachable Sys_ref witness and J.GWR package. J.O remains the separate pairwise composition lemma.